RODRIGO MAXIMIANO A. ALMEIDA
CARLOS HENRIQUE V. MORAES
THATYANA F. PIOLA SERAPHIM

PROGRAMAÇÃO DE Sistemas Embarcados

Desenvolvendo software para microcontroladores em linguagem C

ELSEVIER

Material na WEB
www.elsevier.com.br

PROGRAMAÇÃO DE SISTEMAS EMBARCADOS:

Desenvolvendo software para microcontroladores em linguagem C

Rodrigo Maximiano Antunes de Almeida

Carlos Henrique Valério de Moraes

Thatyana de Faria Piola Seraphim

ELSEVIER

© 2016, Elsevier Editora Ltda.
Todos os direitos reservados e protegidos pela Lei 9.610 de 19/02/1998.
Nenhuma parte deste livro, sem autorização prévia por escrito da editora, poderá ser reproduzida ou transmitida sejam quais forem os meios empregados: eletrônicos, mecânicos, fotográficos, gravação ou quaisquer outros.

ISBN: 978-85-352-8518-5
ISBN (versão digital): 978-85-352-8519-2

Copidesque: Clarissa Penna

Desenvolvimento de eBook: Loope – design e publicações digitais | www.loope.com.br

Elsevier Editora Ltda.
Conhecimento sem Fronteiras
Rua Sete de Setembro, 111 – 16º andar
20050-006 – Centro – Rio de Janeiro – RJ

Rua Quintana, 753 – 8º andar
04569-011-? Brooklin – São Paulo – SP

Serviço de Atendimento ao Cliente
0800 026 53 40
atendimento1@elsevier.com

Consulte nosso catálogo completo, os últimos lançamentos e os serviços exclusivos no site www.elsevier.com.br.

NOTA

Muito zelo e técnica foram empregados na edição desta obra. No entanto, podem ocorrer erros de digitação, impressão ou dúvida conceitual. Em qualquer das hipóteses, solicitamos a comunicação ao nosso serviço de Atendimento ao Cliente para que possamos esclarecer ou encaminhar a questão.
Para todos os efeitos legais, nem a editora, nem os autores, nem os editores, nem os tradutores, nem os revisores ou colaboradores, assumem qualquer responsabilidade por qualquer efeito danoso e/ou malefício a pessoas ou propriedades envolvendo responsabilidade, negligência etc. de produtos, ou advindos de qualquer uso ou emprego de quaisquer métodos, produtos, instruções ou ideias contidos no material aqui publicado.

A Editora

CIP-BRASIL. CATALOGAÇÃO NA PUBLICAÇÃO SINDICATO NACIONAL DOS EDITORES DE LIVROS, RJ

A45p

Almeida, Rodrigo Maximiano Antunes de
Programação de sistemas embarcados : desenvolvendo software para microcontroladores em linguagem C / Rodrigo Maximiano Antunes de Almeida, Carlos Henrique Valério Moraes, Thatyana de Faria Piola Seraphim. – 1. ed. – Rio de Janeiro : Elsevier, 2016.
il. ; 24 cm.

Inclui índice
ISBN 978-85-352-8518-5

1. C (Linguagem de programação de computador). 2. Sistemas embarcados (Computadores). 3. Software - Desenvolvimento. 4. Controladores programáveis. 5. Sistemas operacionais (Computadores). I. Moraes, Carlos Henrique Valério. II. Seraphim, Thatyana de Faria Piola. III. Título.

16-33452 CDD:005.13 CDU:004.43

"*Entre todas as verdadeiras buscas humanas, a busca pela sabedoria é a mais perfeita, a mais sublime, a mais útil e a mais agradável*"

São Tomás de Aquino

Aos meus pais, Paulo e Carminha, e à Ana Paula e nossa família.
Rodrigo Almeida

Aos meus pais, João e Valéria, minhas irmãs, Claudia e Cecília e a minha querida Kátia.
Carlos Henrique Moraes

A Enzo, Miguel, Raphael, e aos meus pais, meus irmãos e amigos.
Thatyana Seraphim

OS AUTORES

Rodrigo M. A. Almeida possui doutorado em Engenharia Elétrica pela Universidade Federal de Itajubá onde é professor na área de sistemas embarcados e coordenador do curso de Engenharia Eletrônica. Leciona nas áreas de eletrônica, interface e periféricos, sistemas embarcados e sistemas operacionais. É articulista do portal embarcados com vários artigos voltados ao desenvolvimento de software. Desenvolve atividade de pesquisa em sistemas operacionais de tempo real, eletrônica embarcada, automação e segurança de sistemas computacionais, tendo palestrado em diversos eventos nacionais e internacionais.

Carlos Henrique V. Moraes possui doutorado em Engenharia Elétrica pela Universidade Federal de Itajubá onde é professor na área de automação inteligente. Leciona nas áreas de programação, sistemas embarcados, matemática discreta, automação e inteligência artificial. Desenvolve atividade de pesquisa em controle inteligente, visão computacional, sistemas embarcados inteligentes, processamento de sinais, navegação autônoma e robótica.

Thatyana F. P. Seraphim possui doutorado em Física Aplicada, Opção Computacional, pela Universidade de São Paulo. É professora da Universidade Federal de Itajubá onde leciona nas áreas de programação, estrutura de dados, linguagens de programação, linguagens formais e compiladores. Desenvolve atividades de pesquisa nas áreas de processamento paralelo, estrutura de dados, compiladores e ferramentas para análise de desempenho de programas paralelos. Trabalhou também como revisora técnica do livro Sistemas de banco de dados, de Elmasri/Navathe.

AGRADECIMENTOS

Primeiramente a meus pais, pelo zeloso e silencioso exemplo de sabedora e retidão. Aos meus irmãos, Marcela e Daniel, pelo carinho e companheirismo. À minha amada esposa, Ana Paula, que me completa exatamente naquilo que me falta. À minha família que começa, que já me muda para melhor, para melhor eu ser para ela. A todos os amigos da caminhada, colegas da Unifei, e incentivadores, que acreditaram nesta ideia quando ainda era apenas uma ideia. A todos os meus alunos, em especial meus orientados, que me permitiram ser um melhor professor e a juntar todo o conhecimento necessário para este livro. A Deus, por ser aquele que é.
Rodrigo

Aos meus pais, João e Valéria por toda a compreensão dispendida. Às minhas radiantes irmãs, Claudia e Cecília. Aos meus grandes amigos, Rodrigo e Tathyana por todo o apoio dispendido ao longo deste trabalho. À minha querida Kátia por todo suporte e companhia. Por fim a todos aqueles que me ajudaram, toleraram e suportaram durante esta fase.
Carlos Henrique

A Deus por tudo! Ao Enzo pelo seu apoio, incentivo, paciência e estímulo para desenvolver este trabalho. Com todo amor, a Miguel e Raphael, por todas as horas roubadas de brincadeiras e atenção. À Helenice pela ajuda e paciência incondicional todos os dias. Aos meus pais, Ana Isabel e Oswaldo, por todo amor e apoio em todos os momentos. Agradeço em especial aos amigos e parceiros Rodrigo e Carlos pela coragem, incentivo e bom relacionamento o que nos proporcionou a criação deste livro. Aos amigos Helaine e João Francisco pelas longas horas de conversas.
Thatyana

Os autores gostariam ainda de agradecer à Unifei e, em especial, ao IESTI, pela oportunidade de ministrar as disciplinas relacionadas com os assuntos abordados no livro, e a todos os diretores, professores, secretárias e técnicos pelo suporte oferecido.

Sumário

Capa
Folha de Rosto
Copyright
Epígrafe
Dedicatória
Os autores
Agradecimentos
Parte I | Linguagem C
    Capítulo 1 | Introdução
        1.1 O que são sistemas embarcados
        1.2 O hardware
        1.3 Hardware utilizado
            Arduino UNO - Atmel ATmega328
            Chipkit UNO32 - Microchip PIC32MX220
            Freedom KL05z - NXP ARM
        1.4 Ambiente de programação
        1.5 Uso da linguagem C
    Capítulo 2 | Sistemas de numeração
        2.1 As bases
            Decimal
            Binária
            Hexadecimal
        2.2 Conversão entre bases
        2.3 BCD e BCD compactado
        2.4 Código Gray
        2.5 Codificação ASCII
        2.6 Exercícios
    Capítulo 3 | Linguagem C
        3.1 Processo de compilação
        3.2 Organização dos programas em C
        3.3 Padrão de escrita

3.4 Diretivas de pré-compilação
Inclusão de arquivos
Definição e expansão de macros
Compilação condicional
Diretiva pragma
3.5 Função main
Começando com Wiring: Arduino e Chipkit
3.6 Entrada e saída de dados
3.7 Exercícios
Capítulo 4 | Variáveis
4.1 Utilização de números e seus tipos
4.2 Declaração de variáveis
Inicialização das variáveis
Inicialização de conjunto de caracteres
Vírgula
4.3 Conversão de tipos
Promoção de tipos
Perda de informação na conversão de tipos
4.4 Modificadores
Modificadores de tamanho e sinal
Modificadores de sinal
Modificadores de acesso
Modificador de armazenamento
4.5 Ponteiros
4.6 Exercícios
Capítulo 5 | Estruturas compostas
5.1 Estruturas homogêneas
Vetor de char x Strings
Matrizes
5.2 Estruturas heterogênas
5.3 Bit fields
5.4 Enumeradores
5.5 Definições de tipo
5.6 Exercícios
Capítulo 6 | Operações binárias
6.1 Álgebra booleana

Operação NÃO
Operação E
Operação OU
Operação OU EXCLUSIVO
6.2 Operações binárias (bitwise)
Bitwise NÃO
Bitwise E
Bitwise OU
Bitwise OU EXCLUSIVO
6.3 Operação de deslocamento
Shift circular ou rotacionamento de bits
6.4 Manipulando apenas 1 bit de cada vez
Criando funções através de define's
6.5 Exercícios
Capítulo 7 | Estruturas condicionais
7.1 Comando condicional if
7.2 Comando condicional aninhado
7.3 Comando de seleção switch...case
7.4 Exercícios
Capítulo 8 | Estruturas de repetição
8.1 Repetição com teste no início
8.2 Repetição com teste no final
8.3 Repetição com variável de controle
8.4 Comandos de desvio
8.5 Rotinas de tempo
8.6 Exercícios
Capítulo 9 | Funções e bibliotecas em linguagem C
9.1 Criando funções
Chamada de função
Protótipo de funções
9.2 Bibliotecas
Referência circular
Padrão para um header
Projetando uma biblioteca
9.3 Driver ou biblioteca?
9.4 Composição de bibliotecas

9.5 Exercícios
Capítulo 10 | Planejando o software embarcado
10.1 Primeiro modelo: o loop infinito
10.2 A evolução do loop no tempo
Capítulo 11 | Debug de sistemas embarcados
11.1 Externalizar as informações
Usando os terminais de entrada e saída
11.2 Programação incremental
11.3 Cuidado com a otimização de código
11.4 Reproduzir e isolar o erro
11.5 Crie rotinas de teste
11.6 Criação de uma biblioteca para debug
Parte II | Controlando periféricos de sistemas embarcados
Capítulo 12 | Introdução a microcontroladores
12.1 A unidade de processamento
12.2 Memória
12.3 Mapeando periféricos na memória
12.4 Clock e tempo de instrução
12.5 Microcontroladores
Atmel ATMega328
NXP KL05z
Microchip PIC32MX320F128
12.6 Registros de configuração do microcontrolador
12.7 Requisitos elétricos do microcontrolador
12.8 Exercícios
Capítulo 13 | Programação dos periféricos
13.1 Controlando os terminais do microcontrolador
Mapeando os terminais nas placas de controle
13.2 Configuração dos periféricos
13.3 Exercícios
Capítulo 14 | Saídas digitais
14.1 Acionamentos
Leds
Transistor
Relé
Relé de estado sólido

Ponte H
14.2 Controle de Led RGB
Criação da biblioteca
14.3 Expansão de saídas
Conversor serial-paralelo
Criação da biblioteca
14.4 Exercícios
Capítulo 15 | Display de 7 segmentos
15.1 Multiplexação de displays
Criação da biblioteca
15.2 Projeto: Relógio
15.3 Exercícios
Capítulo 16 | Entradas digitais
16.1 Debounce por hardware
16.2 Debounce por software
Debounce para mais de uma entrada
16.3 Arranjo matricial
16.4 Criação da biblioteca
16.5 Detecção de eventos
16.6 Aplicações
Reed Switch
Encoder
16.7 Exercícios
Capítulo 17 | Display LCD
17.1 Circuito de conexão
17.2 Comunicação com o display
Comandos
Posicionando os caracteres no LCD
17.3 Criação da biblioteca
17.4 Desenhar símbolos personalizados
17.5 Criando um console com displays de LCD
17.6 Exercícios
Capítulo 18 | Comunicação serial
18.1 I2C
Soft I2C
Relógio de tempo real

18.2 SPI
18.3 CAN
18.4 RS232
RS232 ou UART?
18.5 USB
Serial sobre USB
18.6 Serial sem fios
18.7 Leitura e processamento de protocolos
O protocolo NMEA de GPS
18.8 Exercícios
Capítulo 19 | Conversor analógico digital
19.1 Elementos sensores
Divisor resistivo
Sensores ativos
Sensores da placa de desenvolvimento
19.2 O conversor eletrônico
Canais e multiplexação de entradas analógicas
19.3 Processo de conversão
Criação da biblioteca
19.4 Aplicação
19.5 Exercícios
Capítulo 20 | Saídas PWM
20.1 Conversor digital-analógico usando um PWM
20.2 Soft PWM
20.3 O periférico do PWM
20.4 Criação da biblioteca
20.5 Aplicações
Servomotores
Controle da frequência e emissão de sons
20.6 Exercícios
Capítulo 21 | Temporizadores
21.1 Criação da biblioteca
21.2 Aplicação
Geração de uma base de tempo
Contador de frequência de eventos
Relógio/Calendário

Reprodução de melodias
21.3 Exercícios
Capítulo 22 | Interrupção
22.1 Fonte de interrupção
22.2 Acessando a rotina de serviço da interrupção
22.3 Compartilhando informações
22.4 Exercícios
Capítulo 23 | Watchdog
23.1 Modo de uso
Parte III | Arquiteturas para desenvolvimento de software embarcado
Capítulo 24 | Arquiteturas de software embarcado
24.1 One-single-loop
24.2 Sistema controlado por interrupções
24.3 Multitask cooperativo
Fixação de tempo para execução dos slots
Utilização do tempo livre para interrupções
24.4 Kernel
24.5 Sistemas operacionais
Processo
Escalonadores
24.6 Exercícios
Capítulo 25 | A Desenvolvimento de um kernel cooperativo
25.1 Buffers circulares
25.2 Ponteiros para void
25.3 Ponteiros de função
25.4 Execução das tarefas
25.5 Adição e reexecução de processos
25.6 Exercícios
Capítulo 26 | Projeto de kernel com soft realtime
26.1 O tempo real: soft e hard realtime
26.2 Atendendo requisitos temporais
26.3 Kernel cooperativo com soft realtime
26.4 Exercícios
Capítulo 27 | Controladora de dispositivos
27.1 Padrão de um driver
Driver do timer

27.2 Mecanismo da controladora
Utilizando a controladora de drivers
Camada de abstração da interrupção
27.3 Exercícios
Parte IV | Anexos
Circuitos utilizados nas experiências
Projeto da placa de desenvolvimento
Exercícios resolvidos
Índice Remissivo

Parte I

Linguagem C

1 ---Introdução
2 ---Sistemas de numeração
3 ---Linguagem C
4 ---Variáveis
5 ---Estruturas compostas
6 ---Operações binárias
7 ---Estruturas condicionais
8 ---Estruturas de repetição
9 ---Funções e bibliotecas em linguagem C
10 ---Planejando o software embarcado
11 ---Debug de sistemas embarcados

CAPÍTULO

1

Introdução

1.1 O que são sistemas embarcados
1.2 O hardware
1.3 Hardware utilizado
Arduino UNO - Atmel ATmega328
Chipkit UNO32 - Microchip PIC32MX220
Freedom KL05z - NXP ARM
1.4 Ambiente de programação
1.5 Uso da linguagem C

"O verdadeiro perigo não é os computadores começarem a pensar como humanos, mas o humanos começarem a pensar como computadores."

Sydney J. Harris

Este livro foi escrito com o intuito de servir como guia para aqueles que querem aprender a programar sistemas embarcados. O tema de sistemas embarcados é bastante amplo, indo de sistemas simples com poucos bytes de memória a sistemas de entretenimento complexos com comunicações de alta velocidade, passando por sistemas de suporte à vida, controles de operações críticas e com altos requisitos de segurança, confiabilidade e estabilidade. Esta variedade traz consigo diferentes abordagens de programação, frameworks e camadas de abstração. Cada projeto apresenta estruturas próprias e diferenças substantivas na programação dos periféricos. Abordar todos estes aspectos é uma tarefa complexa, principalmente para aqueles que estão ingressando nesta área.

Os sistemas abordados são aqueles baseados em microcontroladores com pouca capacidade de processamento e armazenamento. Estes componentes oferecem um bom balanço entre simplicidade e quantidade de recursos, sendo ideal para aqueles que estão começando seus estudos sobre programação embarcada. Isto também oferece uma boa oportunidade para entendermos a relação entre eletrônica e programação, bem como conceitos relacionados à evolução temporal dos circuitos.

Mesmo que a programação de sistemas embarcados seja bastante dependente do microcontrolador e das estruturas de hardware utilizadas, os conceitos por traz dos códigos são os mesmos. No intuito de apresentar estes conceitos de maneira independente do hardware, foram selecionadas três arquiteturas distintas para ilustrar as diferenças e similaridades entre os códigos.

Visando ainda a possibilidade de que o leitor possa realizar as experiências sem se ater a apenas um hardware ou placa, este livro contempla exemplos de uso com diferentes placas de controle. As atividades que demandam acesso a periféricos externos podem ser realizadas conectando as placas de controle a uma placa base ou utilizando um simples protoboard. A placa base pode ser construída pelo leitor. Os esquemáticos e layout estão todos disponíveis ao final do livro.

Visando facilitar o processo de aprendizado, o livro foi dividido em três partes: a programação para sistemas embarcados, a programação dos periféricos e a organização dos códigos em arquiteturas mais eficientes.

Os capítulos 1 a 11 formam a primeira parte do livro. Eles apresentam a programação em linguagem C voltada para sistemas embarcados, funcionando como introdução para aqueles que não possuem conhecimento prévio em computação. No entanto, diferente de outros livros sobre este tema, os conceitos são explicados sob a ótica de sistemas embarcados, a ênfase recai sobre as necessidades destes sistemas. Os exemplos, demonstrações e códigos apresentados foram desenvolvidos para serem executados em dispositivos embarcados com poucos recursos, tanto de interface quanto de processamento.

Os capítulos 12 a 23 aprofundam o tema de desenvolvimento de software para sistemas embarcados, focando nas interações do programa com o meio externo através dos periféricos. Nesta parte são apresentados os circuitos eletrônicos e suas necessidades, bem como o impacto destas necessidades na concepção e no desenvolvimento de um programa. Além dos periféricos externos mais comuns, são apresentados um conjunto de periféricos internos que podem ser utilizados para simplificar a coordenação das atividades a serem executadas pelo programa.

Os capítulos 24 a 27 são voltados para arquiteturas de desenvolvimento de software em sistemas embarcados. Esta parte apresenta metodologias de organização do código que facilitam o processo de programação e auxiliam na garantia de estabilidade e funcionamento dos códigos.

1.1 O que são sistemas embarcados

Sistemas embarcados são sistemas eletrônicos microprocessados que, após serem programados, possuem uma função específica que geralmente não pode ser alterada. Uma impressora, por exemplo, mesmo possuindo um processador que poderia ser utilizado para qualquer tipo de atividade, tem sua funcionalidade restrita apenas à impressão de páginas. Um computador de propósito geral, no entanto, pode ser utilizado num instante como um ambiente de entretenimento, em outro como estação de trabalho, ou até mesmo um telefone.

Os sistemas embarcados estão presentes em praticamente todos os ambientes, cobrindo uma ampla gama de funcionalidades: antenas retransmissoras, televisões, fornos de micro-ondas, controles PID's industriais, sistemas de gerenciamento de aviação, esteiras transportadoras etc. Atualmente, quase todo dispositivo que funcione com eletricidade possui um sistema embarcado coordenando seu funcionamento.

Outra característica da maioria dos sistemas embarcados é a restrição de recursos, tanto computacionais: memória e processamento, quanto físicos: número de terminais e interfaces de exibição/entrada de dados. Grande parte desta restrição se deve a questões de custo, consumo de energia ou robustez. Isto leva ao desenvolvimento de alguns sistemas que não possuam nenhum botão ou luz indicativa.

Por se tratar de sistemas com funções específicas e com recursos de interface e computacionais bastante limitados, as rotinas e técnicas de programação são diferentes daquelas usadas em computadores convencionais. Os sinais e informações se modificam ao

longo do tempo, independentemente da entrada de dados pelo usuário. Alguns periféricos necessitam de constante atualização para que mantenham seu funcionamento. A necessidade de se verificar constantemente os dados e atualizar as saídas apresenta também restrições temporais, algumas vezes conflitantes entre si, e que devem ser atendidas para que o sistema funcione corretamente.

Outra questão específica destes sistemas é que cada microcontrolador apresenta uma arquitetura de hardware e um conjunto de periféricos diferentes. Mesmo dois projetos que utilizam o mesmo microcontrolador, mas possuem um arranjo eletrônico diferente dos componentes externos, apresentam diferenças suficientes para que os códigos não possam ser compartilhados. Como cada projeto visa atender requisitos muito específicos, esta variabilidade é bastante comum. Isto exige que os programadores entendam as relações básicas entre os componentes da placa utilizada para desenvolver códigos funcionais.

1.2 O hardware

A programação para sistemas embarcados é intimamente ligada à arquitetura do processador, aos tipos de periféricos disponíveis no microcontrolador e ao modo com que esses periféricos foram conectados com os demais circuitos na placa. Conhecer o sistema é, portanto, fundamental para se construir um programa que funcione da maneira desejada.

Mesmo com as diferenças entre os diversos chips, o modo de funcionamento básico dos periféricos é bastante similar. Para não prender o leitor a apenas uma arquitetura ou fabricante, este livro apresenta o funcionamento dos dispositivos de modo independente. Para evitar a perda de detalhes que poderia acontecer com esta abordagem, serão apresentados exemplos práticos em 3 plataformas distintas: NXP ARM Cortex M0+ (Freedom KL05z), Microchip MIPS PIC32 (Chipkit UNO32) e Atmel ATMega 328 (Arduino Uno R3). Todas essas plataformas apresentam um barramento de pinos compatíveis entre si.

Na primeira parte do livro, os códigos e exemplos fazem uso de um conjunto de bibliotecas pré-implementados e voltados para a placa de desenvolvimento. Optou-se por esta abordagem porque ela traz uma série de simplificações que permitem ao leitor aprender a programar em linguagem C em um ambiente embarcado, sem se ater a detalhes de hardware num primeiro instante.

Na segunda parte do livro, onde serão abordadas as questões relacionadas aos periféricos, as bibliotecas pré-implementadas serão explicadas em detalhes, permitindo ao leitor entender como acessar e controlar os periféricos utilizando linguagem C.

A placa de desenvolvimento foi projetada pelos autores para permitir a execução de todas as experiências do livro, podendo ser conectada a qualquer uma das três plataformas: Arduino, Chipkit ou Freedom. O projeto desta placa está disponibilizado em licença aberta, permitindo sua livre reprodução por qualquer pessoa.

No entanto, não é preciso adquirir ou fabricar a placa de desenvolvimento para acompanhar os exercícios do livro. O leitor que desejar poderá realizar a montagem dos circuitos para cada atividade. Todos os esquemáticos e conexões dos projetos utilizados serão apresentados de modo simples para montagem em protoboard. Por uma questão de espaço, todos os circuitos serão apresentados conectados ao Arduino. Como as placas possuem barramentos compatíveis, as ligações são idênticas em qualquer modelo de placa de controle.

No acesso aos periféricos, o livro utiliza uma abordagem mais próxima ao hardware, não fazendo uso de bibliotecas prontas. A maioria dos compiladores disponibilizam um conjunto de bibliotecas que permite o acesso simplificado ao hardware. No entanto essas bibliotecas abstraem os detalhes da programação do hardware, o que pode prejudicar o entendimento do funcionamento dos periféricos. Deste modo, os autores julgam importante que o programador

de sistemas embarcados entenda o funcionamento e consiga programar diretamente o hardware. Mesmo que posteriormente sejam utilizadas as bibliotecas desenvolvidas pelos fabricantes, o programador precisa deste conhecimento para solucionar os problemas e interferências que podem acontecer ao longo do desenvolvimento do projeto.

1.3 Hardware utilizado

Como o enfoque deste curso é a programação de sistemas embarcados e não a eletrônica, utilizaremos um kit de desenvolvimento que reúne um conjunto de periféricos que, apesar de relativamente simples, formam a base de qualquer tipo de interface eletrônica. São eles:

- 1 led RGB conectado a terminais de entrada e saída.
- 1 expansor de saídas digitais 74 hc595 .
- 4 displays de 7 segmentos com o barramento de dados compartilhados com o LCD.
- 10 chaves organizadas em formato matricial 5 x2 .
- 1 display LCD 2 linhas por 16 caracteres (compatível com HD77480).
- 1 sensor de luminosidade.
- 1 sensor de temperatura.
- 1 potenciômetro.
- 1 buzzer ligado a uma saída PWM.
- 1 canal de comunicação serial assíncrono com interface USB.
- 1 relógio de tempo real com memória interna conectado através de um canal de comunicação serial síncrono.

Arduino UNO - Atmel ATmega328

A placa de controle Arduino (figura 1.1) foi desenvolvida em 2005 por Massimo Banzi para baratear e simplificar o estudo de programação embarcada.

Ela foi inicialmente baseada num microcontrolador da Atmel: o Atmel ATmega8, possuindo 14 entradas/saídas digitais e 6 entradas analógicas. O bootloader instalado permite que a placa seja reprogramada sem a necessidade de um gravador dedicado, todo o processo é realizado através de uma porta serial.

Visando facilitar a compatibilidade destas placas, um conversor RS232-USB foi adicionado ao projeto.

Como linguagem de programação foi escolhida a Processing. Esta linguagem tem como objetivo facilitar o processo de aprendizagem da programação por pessoas que não tem muito contato com programação.

Visando simplificar o acesso ao hardware, foi utilizado o framework Wiring. Ele abstrai as questões mais complexas do hardware provendo um conjunto de bibliotecas e funções pré-definidas.

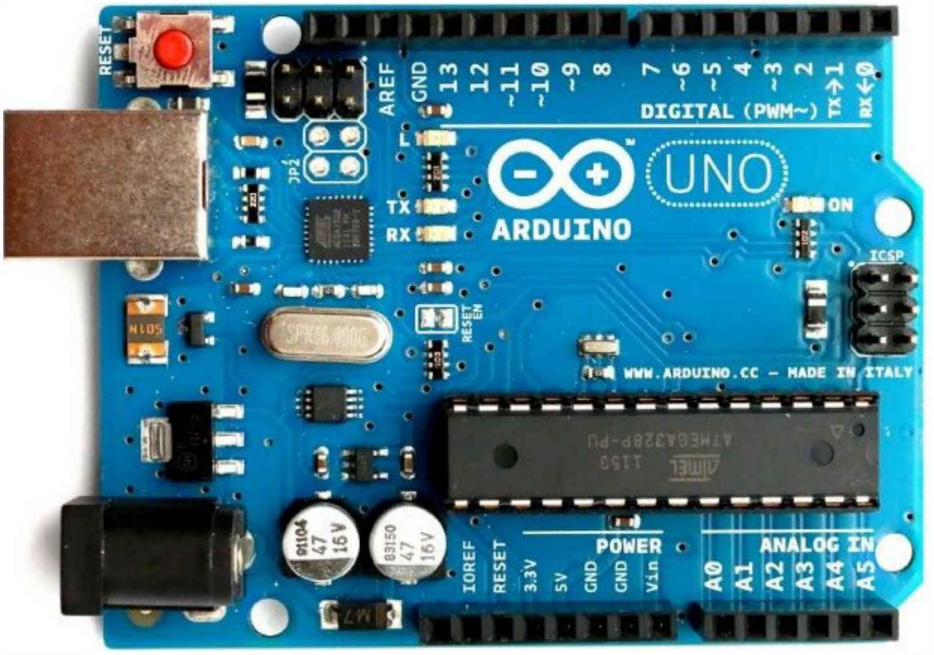


Figura 1.1: Arduino Uno

A escolha da linguagem Processing, em conjunto com o framework Wiring, bem como a utilização de um bootloader para fácil reprogramação da placa, permitiu que a plataforma Arduino se tornasse acessível a grande parte dos hobbystas e estudantes.

Chipkit UNO32 - Microchip PIC32MX220

A placa Chipkit UNO32 (figura 1.2) foi desenvolvida para ser compatível, tanto em hardware, quanto em código, com as placas Arduino. Elas compartilham os mesmos barramentos (footprint), bem como a mesma base de código.

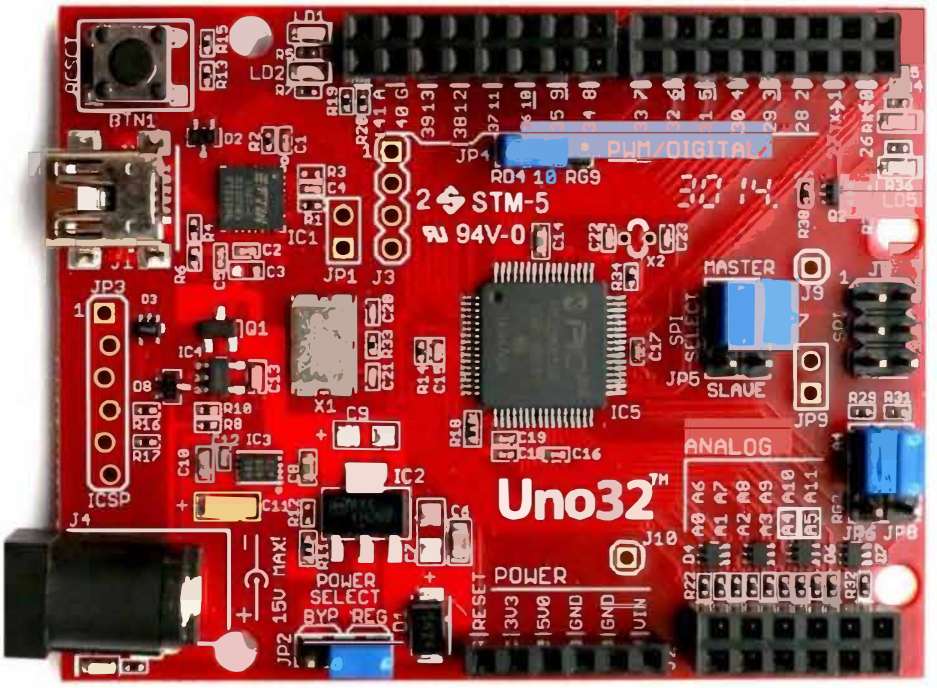


Figura 1.2: Chipkit UNO32

A Microchip, fabricante dos processadores PIC32, reescreveu parte do framework Wiring de modo que os códigos que rodam no processador da Atmel também funcionassem em seu processador.

O processador PIC32 é baseado na arquitetura de 32 bits da MIPS, com um core M4K, chegando a velocidades de 80MHz. Por esse motivo, é bem mais rápido que os Arduinos originais, rodando em 8 bits, com velocidades de 16MHz.

Freedom KL05z - NXP ARM

A família de placas de controle Freedom é baseada num microcontrolador Kinetis-L da NXP, com um core ARM Cortex M0+. A variante utilizada neste livro é a Freedom KL05z (figu ra 1.3) com o menor custo entre as alternativas.

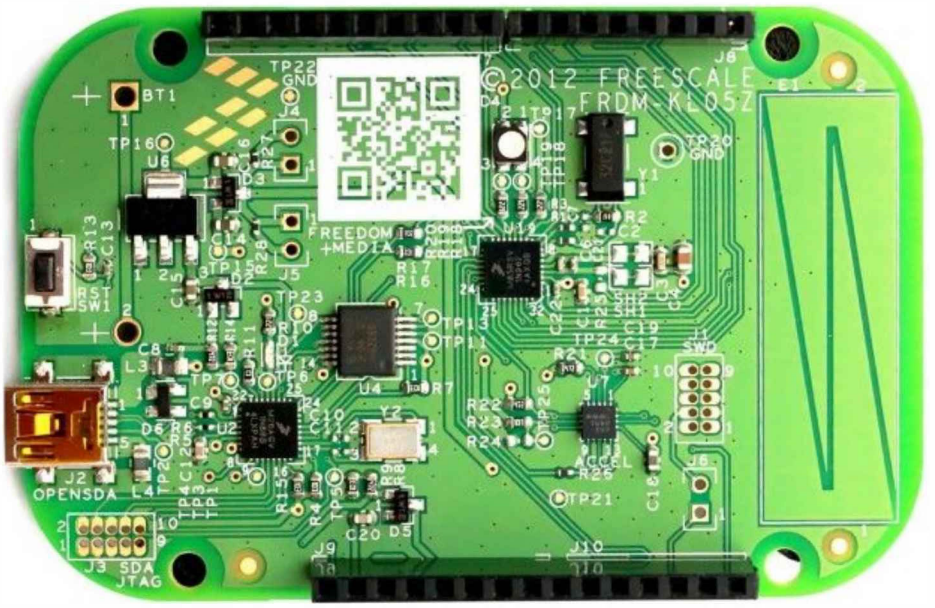


Figura 1.3: Freedom KL05z

Apesar de possuir praticamente o mesmo barramento que as placas do Arduino, existem pequenas alterações em alguns terminais, como a ausência de um barramento I²C junto aos terminais analógicos A4 e A5.

Além disso, o framework Wiring não está portado para essa plataforma, fazendo com que o programador precise tomar conta de todos os detalhes do acesso ao hardware. Esta será a placa utilizada como referência na segunda parte do livro, permitindo que o leitor entenda melhor os detalhes da programação de baixo nível.

1.4 Ambiente de programação

Para facilitar o processo de gravação e desenvolvimento dos programas, faremos uso de aplicativos de interface de edição que automatizam grande parte do processo de compilação e gravação. Estes aplicativos são conhecidos como IDE (Integrated Development Environment) ou ambiente integrado de desenvolvimento.

Todas as placas utilizadas no livro possuem um programa já instalado em sua memória chamado bootloader. Esse programa permite que a gravação de um novo programa seja feita sem o uso de um gravador/depurador dedicado, reduzindo o custo do kit de desenvolvimento.

O bootloader funciona como um gestor na inicialização do equipamento. Se não houver nenhum comando externo, ele inicializa o programa que estiver armazenado no microcontrolador. Quando um comando é percebido, quando a placa é ligada, o bootloader passa a entrar no modo de gravação. Após este momento, o computador pode enviar um novo programa para a placa. O bootloader irá armazenar esse novo programa numa região adequada na memória para que ele possa ser executado na próxima vez. Este processo pode ser visto na figura 1.4:

Figura 1.4: Fluxo de gravação com bootloader

O modo de indicar à placa que ela deve entrar no modo de bootloader depende do microcontrolador utilizado e das escolhas que o projetista fez.

O Arduino e o Chipkit possuem um terminal digital que faz essa indicação à placa. Tanto o Chipkit quanto o Arduino conectam esse terminal numa das conexões da comunicação serial. Deste modo, para regravar a placa, basta que a IDE envie o código que acionará o terminal remotamente.

A plataforma Freedom implementa um sistema diferente. Quando conectada ao computador, ela cria um pendrive virtual. Este pendrive permite que o usuário copie o novo programa como se fosse um arquivo normal. A IDE utilizada, no entanto, faz esse procedimento automaticamente.

A geração do código compilado tem que ser feita de acordo com a placa utilizada. Cada arquitetura de processador precisa de um compilador diferente.

Para facilitar esse processo, utilizaremos os ambientes desenvolvidos para cada plataforma. A relação é apresentada na tabela 1.1:

Tabela 1.1: Softwares de programação das placas

Placa base	IDE	Versão
Arduino	Arduino IDE	1 .06
Chipkit	MPIDE	0 .9
Freedom	Kinetis Design Studio	3 .2 .0

1.5 Uso da linguagem C

A linguagem C foi escolhida para este livro porque, além de ser a mais utilizada em sistemas embarcados, é relativamente simples para quem está começando. Além disso, permite que façamos acesso direto aos registros de hardware, item fundamental para garantir que os sistemas eletrônicos se comportem do modo desejado.

O processo de programação em C, para sistemas embarcados, apresenta algumas diferenças para a programação em C para computadores tradicionais. Existem preocupações maiores com questões de tamanho de código, consumo de memória e velocidade de execução. Isso leva a cuidados extras na declaração das variáveis e dos tipos utilizados.

Além disso, é muito comum a manipulação de bits na memória, principalmente nas regiões de memória onde os periféricos são mapeados.

Por fim, a possibilidade de utilização de interrupções em conjunto com variáveis que representam valores físicos leva o programador a utilizar arquiteturas diferentes de programação.

CAPÍTULO

2

Sistemas de numeração

2.1 As bases
- Decimal
- Binária
- Hexadecimal

2.2 Conversão entre bases

2.3 BCD e BCD compactado

2.4 Código Gray

2.5 Codificação ASCII

2.6 Exercícios

"Números perfeitos, assim como homens perfeitos, são muito raros."

René Descartes

Para poder trabalhar com a informação é necessário representá-la em um formato acessível. A música, por exemplo, possui diversas características que devem ser informadas para que ela seja tocada corretamente. Para executar uma nota devemos saber sua intensidade, frequência e duração. A tabela 2.1 apresenta uma listagem com essas informações para as quatro primeiras notas de uma música:

Tabela 2.1: Sequenciamento de notas musicais

Tempo	Frequência (nota)	Intensidade
10 ms	440	15
12 ms	720	30
5 ms	0	0 (sem som)
12 ms	720	30

Este modo de representação, apesar de correto, não é adequado para os músicos, principalmente por ocupar muito espaço e dificultar a leitura durante a execução da música.

Para isso foi desenvolvida a partitura. Ela apresenta as notas de um modo gráfico mais natural para os músicos, como na figura 2.1. Esta mudança de formato faz com que mais informações sejam disponibilizadas, como a relação entre duas notas e como é o andamento

da melodia. Apesar desta informação também estar disponível na tabela, o formato em que os dados são apresentados pode simplificar a leitura e utilização dos dados.

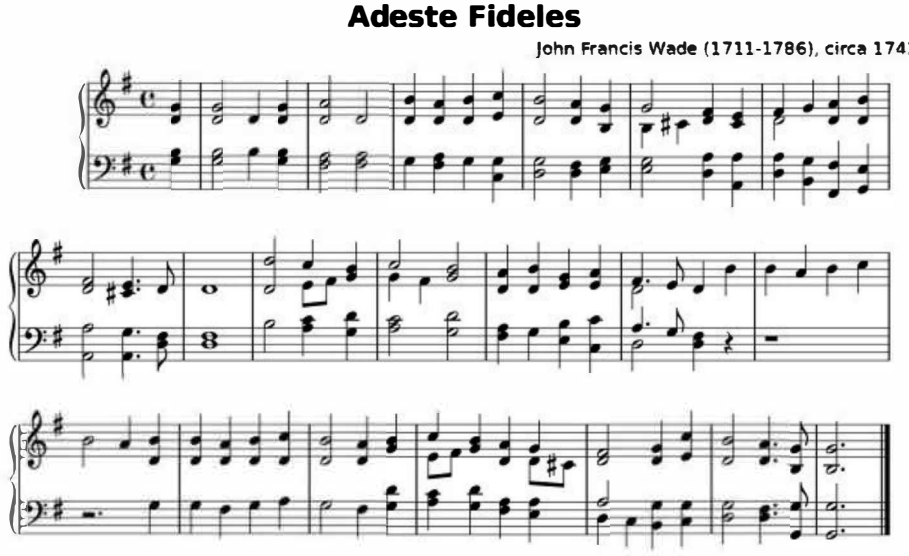


Figura 2.1: Partitura musical

Muitas das informações que desejamos armazenar são formadas por quantidades. A ideia de representar uma quantidade através de símbolos é bastante antiga. A representação atual é baseada nos algarismos arábicos, através de uma base decimal.

No entanto, para simplificar o desenvolvimento dos computadores, adotou-se a base binária. Isto permitiu a criação de sistemas mais estáveis e robustos. Na base binária, cada posição de um número pode conter apenas os valores **`0`** ou **`1`**.

Neste capítulo abordaremos as diferentes bases comumente utilizadas em computação e também as diferentes representações destes números.

2.1 As bases

As bases mais utilizadas durante a programação dos sistemas embarcados são a decimal, por ser a base que utilizamos no dia a dia, a binária, por ser o modo que os processadores trabalham, e a hexadecimal. Esta última serve para facilitar a leitura ou a operação com números binários, principalmente por reduzir a quantidade de algarismos exibidos.

No entanto devemos ressaltar que dentro do processador todos os números são armazenados em binário, até mesmo as letras de um texto.

Decimal

Um número é composto por diferentes algarismos. Para saber quanto um algarismo contribui para o número precisamos de duas informações: o valor do algarismo e a posição que se encontra. A figura 2.2, apresenta o número 1234, na base decimal. Este número é formado por 4 algarismos.

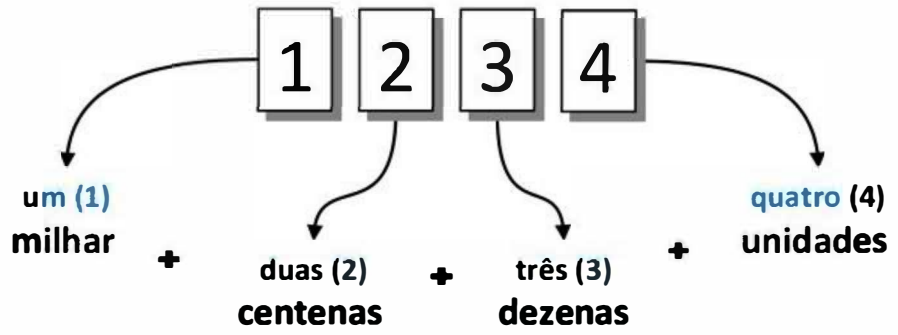


Figura 2.2: Valor dos algarismos

O valor do número pode ser definido de acordo com a posição de cada algarismo. Neste caso o número 1 vale 1000, o 2 vale 200, o 3 vale 30 e o 4 vale 4. O resultado final é a soma destes valores. Este processo está representado na figura 2.3:

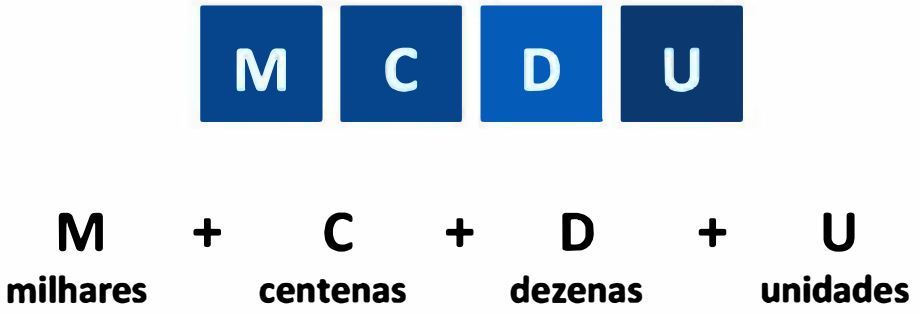
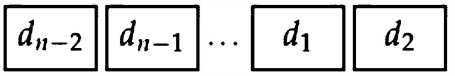


Figura 2.3: Algarismos de milhar, centena, dezena e unidade

A valor final pode ser definido como a soma de cada algarismo multiplicado por uma potência de 10. O expoente da potência depende da posição do algarismo no número:



$$d_{n-1} \times 10^{n-2} + d_{n-2} \times 10^{n-1} + \cdots + d_1 \times 10^1 + d_0 \times 10^0 \qquad (2.1)$$

Este é um conceito relativamente simples. O que estamos fazendo ao criar um número na base 10 é contar quantas vezes uma quantidade é múltipla de 1 (10^0) 10 (10^1), de 100 (10^2), de 1000 (10^3), etc. Para as demais bases a ideia é a mesma, mas ao invés de usarmos o valor 10 como base, utilizaremos os valores 2 para a binária e 16 para a hexadecimal. A representação matemática de um número N na base B é dada por N_b conforme a equação a seguir:

$$N_b = \sum_{i=0}^{m-1} d_i b^i \qquad (2.2)$$

Para que essa formulação seja válida, os valores devem obedecer a seguinte equação:

$$b > 1, 0 <= d^i <= (b-1), (b, d^i, N_b) \epsilon N^3 \quad (2.3)$$

Binária

Conforme apresentado, a base binária possui grande utilidade em sistemas computacionais por ser o modo como, em geral, os computadores armazenam e executam as operações com números.

Especificando, a notação matemática de uma base genérica para uma base 2, temos que um número na base 2 (N_2) será representado por:

$$N_2 = \sum_{i=0}^{m-1} d_i \times 2^i \quad (2.4)$$

$$d^i \epsilon (0, 1) \quad (2.5)$$

Esta expressão diz que um algarismo num número binário representa a existência de um valor numa potência de 2, como representado na figura 2.4. Na figura podemos ver que cada digito d_n representa um valor igual ao dobro do digito d_{n-1}.

Os algarismos do número apresentado na figura 2.4, podem ser apenas zero ou um. Deste modo, na soma, o algarismo indica a presença (**1**) ou ausência (**0**) daquela potência de dois no número final. A figura 2.5 apresenta um exemplo de um número binário com 5 algarismos.

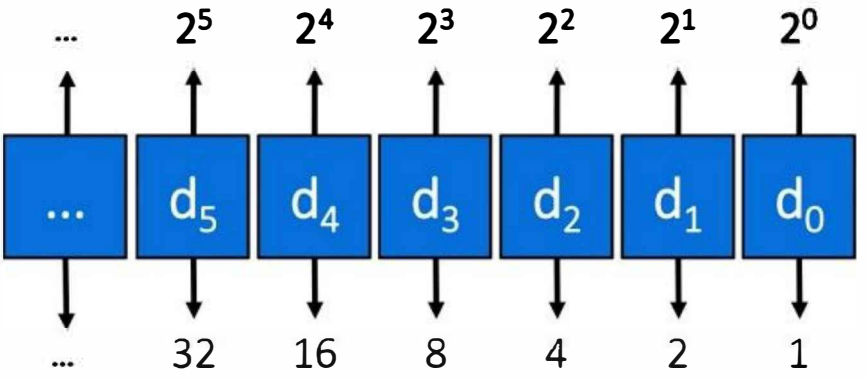


Figura 2.4: Valores dos algarismos num número binário

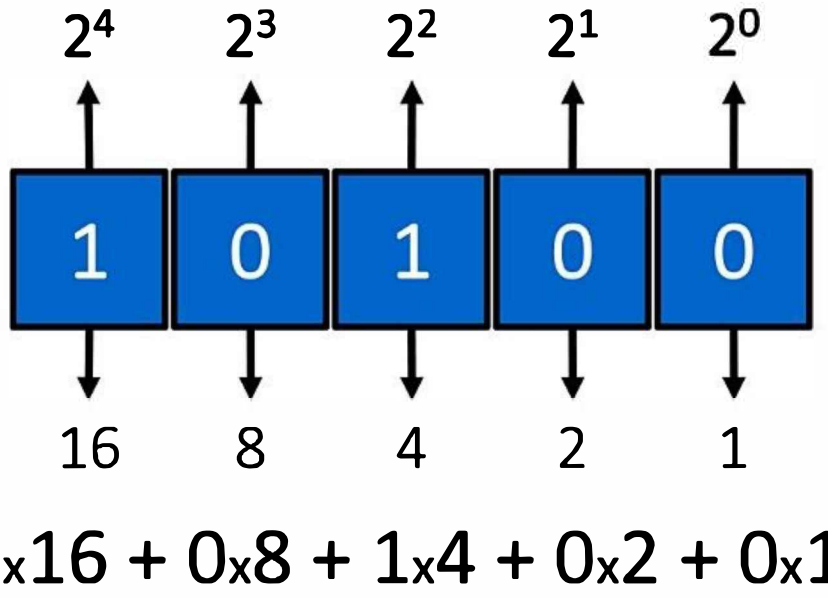


Figura 2.5: Exemplo de número binário

A figura 2.6 apresenta uma tabela com os valores de conversão de binário para decimal.

Binário	Decimal	Binário	Decimal
0_2	0_{10}	110_2	6_{10}
1_2	1_{10}	111_2	7_{10}
10_2	2_{10}	1000_2	8_{10}
11_2	3_{10}	1001_2	9_{10}
100_2	4_{10}	1010_2	10_{10}
101_2	5_{10}	1011_2	11_{10}

Figura 2.6: Tabela de conversão binário-decimal

Hexadecimal

A base binária representa como os números são armazenados e manipulados pelo processador. No entanto, um número com vários zeros e uns é de difícil assimilação pelo ser humano. Isto pode induzir a erros, principalmente quando estamos manipulando esses números.

Visando simplificar a escrita desses números, é comum utilizarmos a base hexadecimal. Esta base funciona com 16 algarismos, de modo que cada posição é representada por uma potência de 16.

$$N_{16} = \sum_{i=0}^{m-1} d_i 16^i \tag{2.6}$$

$$d^i \epsilon (0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F) \tag{2.7}$$

Como existem apenas 10 algarismos, utilizamos as seis primeiras letras do alfabeto para completar os algarismos. Deste modo, o caractere "A" representa a quantidade 10, o caractere "B" o valor 11 e assim por diante até o caractere "F" que representa 15. A figura 2.7 apresenta os valores de cada uma das posições num número hexadecimal.

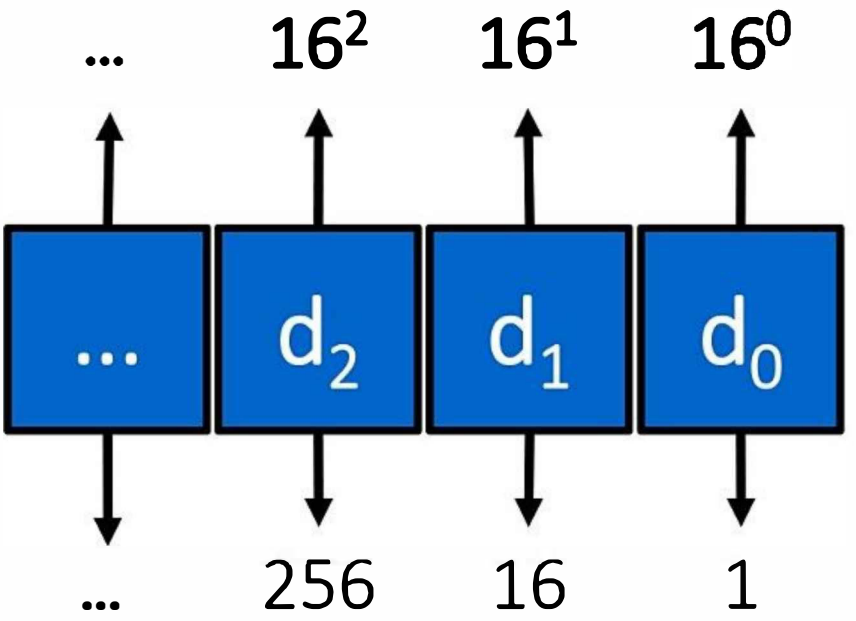


Figura 2.7: Valores dos algarismos num número hexadecimal

Tomemos por exemplo o número $F20C_{16}$. O caractere "C" indica que na primeira posição temos C (12) unidades de base 16, cujo valor de cada unidade é $16^0 = 1$. Já o caractere 2 indica que este número possui duas quantidades na terceira posição. Isto indica que este número vale 2×16^2, totalizando 512. A mesma coisa acontece para o caractere "F". A figura 2.8 apresenta essa informação de modo gráfico:

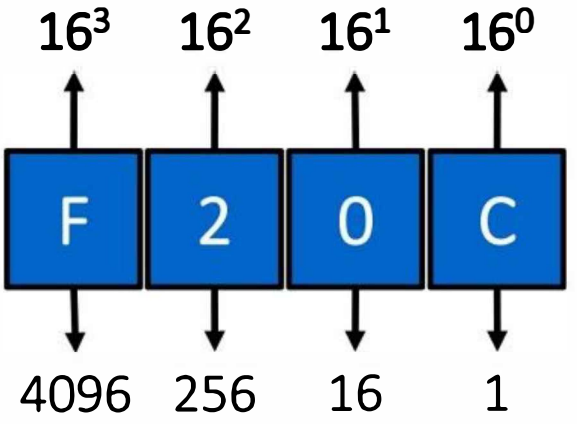


Figura 2.8: Exemplo de número hexadecimal

2.2 Conversão entre bases

Como já foi apresentado, as bases são apenas um modo de escrever a mesma quantidade. Veja a figura 2.9, por exemplo. Fazendo a contagem na base decimal, existem 8 pontos ou 8_{10}. Se quisermos armazenar a mesma quantidade na base binária, teremos o valor 100_2.

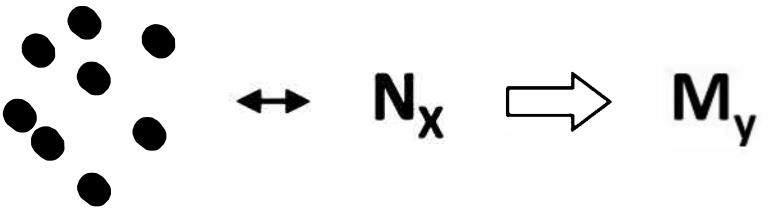


Figura 2.9: Processo de conversão entre bases

Todas as três bases apresentadas, binária, decimal e hexadecimal, possuem utilidade na hora de programar para um processador. É bastante comum que em um determinado tempo desejamos saber qual o valor de uma determinada quantidade em diferentes bases, como no caso dos pontos. Se estivermos conversando com alguém, diremos que existem 8_{10} pontos. No entanto, se formos armazenar esse número no computador, ele será salvo como o valor 100_2.

Para converter um número N, que está numa base decimal, para uma base Y, basta dividirmos N por Y sucessivas vezes enquanto anotamos o resto de cada divisão. Continuamos esse procedimento até o resultado ser zero.

Por exemplo, para converter 56_{10} na base 3 podemos dividir o número por 3 e anotar os resultados.

```
56/3 = 18 resto 2
18/3 = 9 resto 0
```

```
9/3 = 3 resto 0
3/3 = 1 resto 0
1/3 = 0 resto 1
```

Anotando todos os valores de resto, da última divisão para a primeira, temos o número 10002 na base 3, que equivale ao número 56 na base decimal. Portanto a conversão de decimal para binário, ou de hexadecimal para binário, pode ser feita através das divisões sucessivas. A figura 2.10 apresenta a conversão do número 231_{10} para binário e hexadecimal através deste processo.

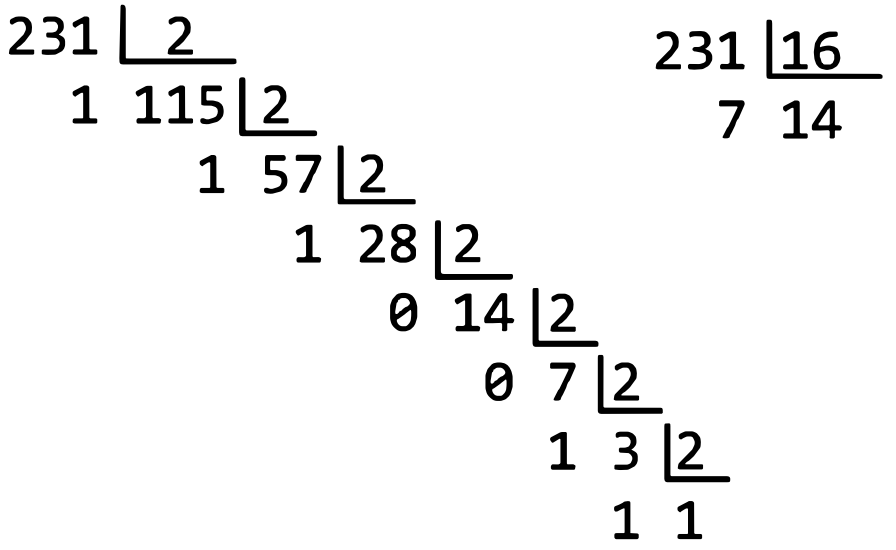


Figura 2.10: Exemplo de conversão utilizando divisões

O processo para converter um número N na base Y para a base decimal é feito somando-se os algarismos em cada posição do número N, levando-se em conta a base utilizada. Por exemplo, convertendo o número 10432 na base 5 para uma base decimal.

Se este número está na base 5, cada posição é um múltiplo de 5. A primeira posição indica a quantidade de valores 5^0, a segunda posição a quantidade de 5^1 e assim por diante.

A primeira posição, da direita para a esquerda, será o algarismo 2. $2 \times 5^0 = 5$. Já o algarismo 3 representa que o número em questão possui três unidades na posição dois, ou seja, 3×5^2. Deste modo, podemos expressar o número 10432_5 como:

$$10432_5 = (1 \times 5^4) + (0 \times 5^3) + (4 \times 5^2) + (3 \times 5^1) + (2 \times 5^0)$$

$$10432_5 = (1 \times 625) + (0 \times 125) + (4 \times 25) + (3 \times 5) + (2 \times 1)$$

$$10432_5 = 625 + 0 + 100 + 15 + 1 = 741_{10}$$

Para converter as outras bases para a base decimal podemos fazer o mesmo procedimento. Por exemplo, convertendo o número $13F4_{16}$ temos:

$$13F4_{16} = (1 \times 16^3) + (3 \times 16^2) + (F \times 16^1) + (4 \times 16^0)$$

$$13F4_{16} = (1 \times 4096) + (3 \times 256) + (15 \times 16) + (4 \times 1)$$

$$10432_5 = 4096 + 768 + 240 + 4 = 5108_{10}$$

Para o número binário 10010111_2 podemos fazer:

$$10010111_2 = (1 \times 2^7) + (0 \times 2^6) + (0 \times 2^5) + (1 \times 2^4) + (0 \times 2^3) + (1 \times 2^2) + (1 \times 2^1) + (1 \times 2^0)$$

$$10010111_2 = (1 \times 128) + (0 \times 64) + (0 \times 32) + (1 \times 16) + (0 \times 8) + (1 \times 4) + (1 \times 2) + (1 \times 1)$$

$$10010111_2 = 128 + 0 + 0 + 16 + 0 + 4 + 2 + 1 = 151_{10}$$

A conversão de binário para hexadecimal, ou de hexadecimal para binário, pode ser feita de um modo bem mais simples. Primeiramente, separamos os algarismos do número binário em grupos de 4 algarismos, da direita para a esquerda. Para cada grupo de 4 algarismos basta então utilizar a tabela 2.2 como ferramenta de conversão.

Tabela 2.2: Representação decimal – binária – hexadecimal

Decimal	*Binária*	*Hex.*	*Decimal*	*Binária*	*Hex.*
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	10	1010	A
3	0011	3	11	1011	B
4	0100	4	12	1100	C
5	0101	5	13	1101	D
6	0110	6	14	1110	E
7	0111	7	15	1111	F

Por exemplo, o número 18. Sabemos que este número na representação binária é dado por 10010_2. Utilizando o procedimento citado, podemos agrupá-lo em dois conjuntos 1 e 0010_2.

Da tabela 2.2 podemos verificar que 1_2 é equivalente a 1_{16} e que 0010_2 pode ser representado pelo número 2_{16}. Logo podemos definir que 10010_2 pode ser representado em hexadecimal por 12_{16}.

2.3 BCD e BCD compactado

Alguns dispositivos possuem modos diferentes de interpretar as informações armazenadas em base binária. Alguns destes dispositivos separam um byte, um conjunto de 8 bits, em dois nibbles, um conjunto de 4 bits. A figura 2.11 apresenta esta relação.

Cada nibble pode contar de zero a 1111_2, o que em decimal significa uma variação de zero a 15. No entanto, para simplificar a utilização dos dispositivos, optou-se por armazenar apenas valores de zero a 9. Este tipo de codificação é conhecido como BCD - Binary Coded Decimal, ou Decimal Codificado em Binário. A ideia é que os números em decimal possam ser armazenados diretamente em binário, onde cada posição da memória guardaria um algarismo decimal.

Apesar de facilitar a utilização em alguns casos, essa abordagem diminui a capacidade de armazenamento dos valores, pois cada byte, que antes podia contar até 255, agora só

armazena valores de zero a 9.

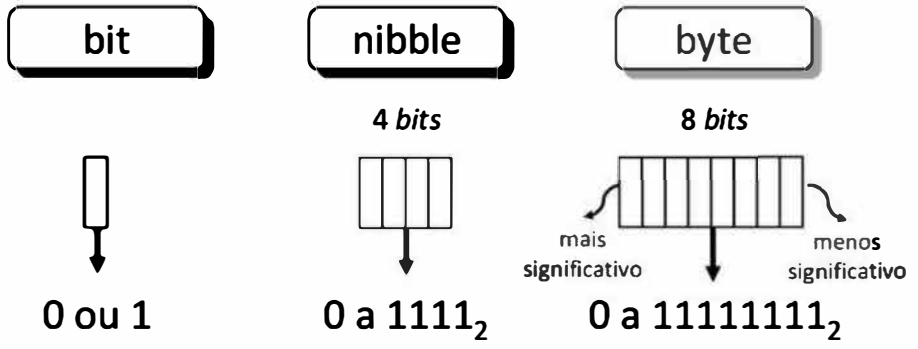


Figura 2.11: Relação entre bit, nibble e byte

Uma alternativa para melhorar essa estrutura é o BCD compactado. Para armazenar valores de zero a 9 são precisos apenas 4 bits. Por este motivo, em cada byte de um número em BCD, quatro bits ficavam sem utilização. O BCD compactado agrupa então dois números codificados em BCD utilizando apenas um byte, conforme figura 2.12:

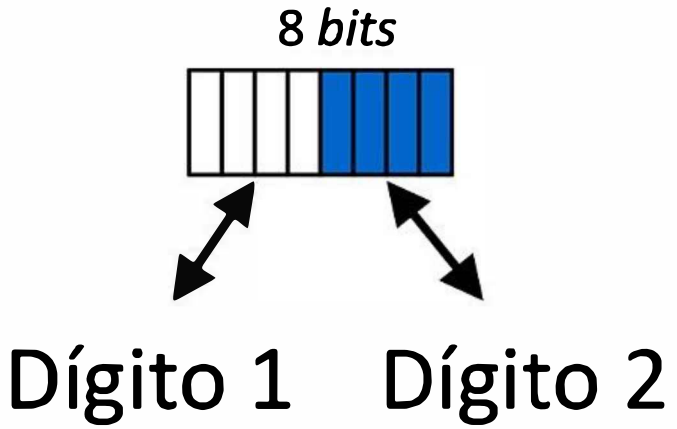


Figura 2.12: Representação de um número BCD compactado

Apesar de facilitar o uso dos dados em algumas situações, deve-se ter muito cuidado no uso de variáveis cujos valores representam BCD's compactados. Qualquer processo de cálculo envolvendo essas variáveis deve ser realizado levando-se em consideração as diferenças para uma variável regular.

Como cada nibble (conjunto de 4 bits) é utilizado para armazenar apenas um dígito, um byte, que normalmente pode contar de zero a 256, agora está em valores limitados de zero a 99.

2.4 Código Gray

Uma outra codificação bastante utilizada é o código Gray. Este código foi inicialmente utilizado para reduzir os erros que surgiam na transição dos encoders, equipamentos utilizados para medição de elementos rotativos. A ideia é criar uma sequência de valores, em binário, onde o próximo valor possua apenas um bit de diferença do anterior. Esta característica é muito útil ao se implementar sistemas que utilizem leitura de posições sequenciais, como nos encoders.

A figura 2.13 apresenta dois discos, codificados em binário. Neste exemplo poderiam ser utilizados até três sensores, um em cada uma das três faixas. O sensor retornaria o valor 1 se a área debaixo do sensor for branca e zero se a área for preta. Deste modo, podemos identificar em qual das oito posições o sensor se encontra.

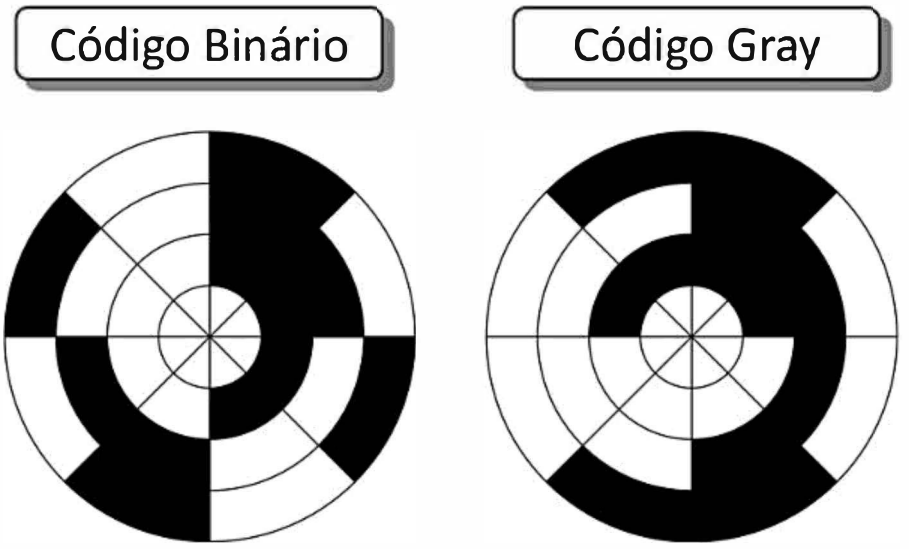


Figura 2.13: Disco rotativo em código binário e código Gray

Este é o funcionamento básico do encoder que atua como um sensor angular, como um controle de volume, por exemplo. Se for preciso aumentar a precisão, basta adicionar mais faixas e mais sensores.

A figura 2.14 apresenta a contagem de zero a 7 nos códigos binário e gray. Podemos perceber que no código gray apenas um bit é alterado a cada linha. Já o código binário chega a modificar até 3 bits de uma única vez.

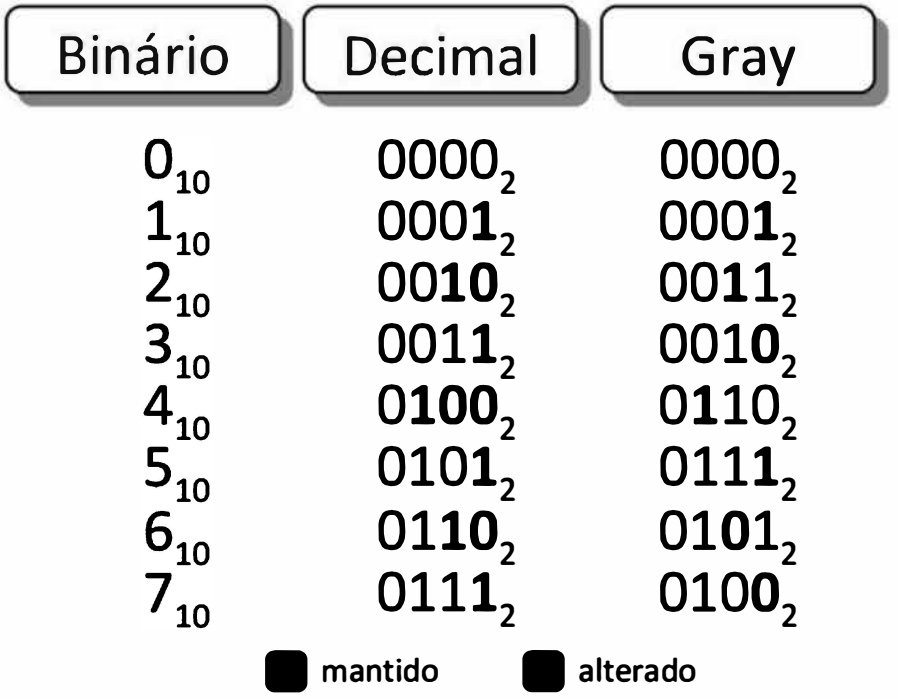


Figura 2.14: Tabela de conversão Binário-Decimal-Gray

Ambos os códigos, binário simples ou gray, conseguem identificar corretamente todas as áreas. A vantagem do código gray pode ser observada quando consideramos os erros que podem acontecer no sistema: problemas na impressão dos discos pintados, desalinhamento ou posicionamento dos sensores e até mesmo da vibração do equipamento enquanto estiver em uso.

A figura 2.15 apresenta novamente os dois discos dando ênfase nas bordas das impressões das áreas. Ambos os discos apresentam algum tipo de erro de impressão.

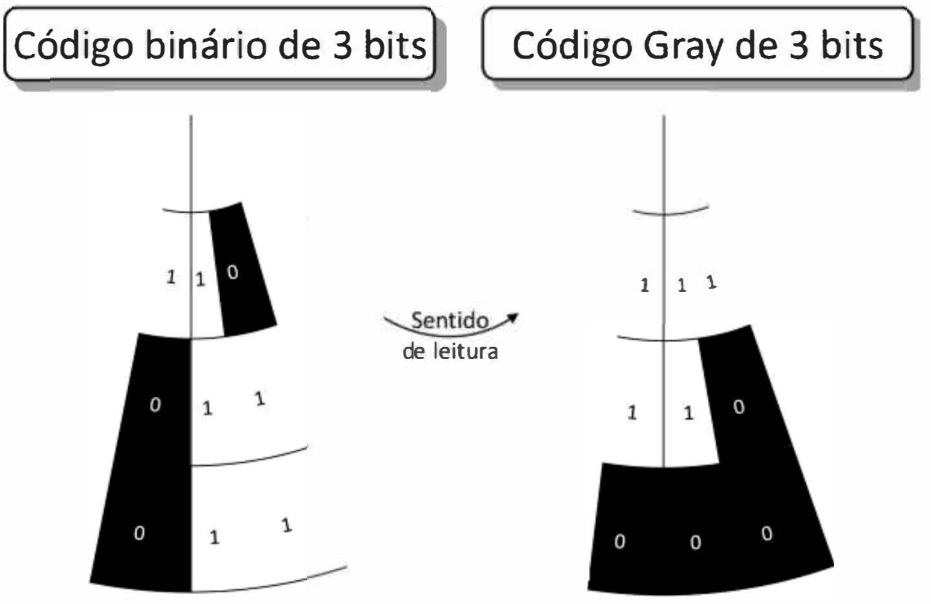


Figura 2.15: Disco rotativo em código Gray

Podemos notar que o disco 1 apresenta um valor errado entre os dois valores corretos. Já o disco codificado em gray apresenta apenas um atraso no valor correto, mas não apresenta um valor discrepante na mudança.

2.5 Codificação ASCII

Os computadores apenas armazenam e manipulam valores codificados em binário. Deste modo, qualquer tipo de informação, som, imagem ou texto tem que ser codificado em valores binários.

Para codificar um texto é preciso adotar um valor binário para cada letra do alfabeto. O código mais simples é o ASCII ou American Standard Code for Information Interchange.

O protocolo ASCII foi originalmente baseado no alfabeto inglês, sendo derivado dos primeiros códigos eletrônicos para transmissão de mensagens através de telégrafos. A principal motivação para o desenvolvimento desse código foi a necessidade de incluir letras minúsculas e marcações de pontuação nas mensagens.

Dado o espaço disponível, de 7 bits, foi possível codificar letras maiúsculas, minúsculas, pontuação, símbolos gráficos e comandos de controle. Os comandos eram ações especiais que seriam tomadas pelo equipamento que estivesse lendo a mensagem.

A primeira padronização deste código foi feita em maio de 1963 com a votação da inclusão das letras em minúsculas no padrão. A versão atual desse código é a ANSI X3.4, de 1986. A tab ela 2.3 relaciona os valores, em hexadecimal, e os símbolos gráficos definidos nesta norma:

Tabela 2.3: Codificação dos caracteres em ASCII

Hex	Cmd	Escape Code	Hex	Char	Hex	Char	Hex	Char
0	NUL	\0	20	(space)	40	@	60	‘
1	SOH		21	!	41	A	61	a
2	STX		22	“	42	B	62	b

3	ETX		23	#	43	C	63	c
4	EOT		24	$	44	D	64	d
5	ENQ		25	%	45	E	65	e
6	ACK		26	&	46	F	66	f
7	BEL	\a	27	'	47	G	67	g
8	BS	\b	28	(	48	H	68	h
9	HT	\t	29	)	49	I	69	i
0 A	LF	\n	2 A	*	4 A	J	6 A	j
0 B	VT	\v	2 B	+	4 B	K	6 B	k
0 C	FF	\f	2 C	,	4 C	L	6 C	l
0 D	CR	\r	2 D	-	4 D	M	6 D	m
0 E	SO		2 E	.	4 E	N	6 E	n
0 F	SI		2 F	/	4 F	O	6 F	o
10	DLE		30	0	50	P	70	p
11	DC1		31	1	51	Q	71	q
12	DC2		32	2	52	R	72	r
13	DC3		33	3	53	S	73	s
14	DC4		34	4	54	T	74	t
15	NAK		35	5	55	U	75	u
16	SYN		36	6	56	V	76	v
17	ETB		37	7	57	W	77	w
18	CAN		38	8	58	X	78	x
19	EM		39	9	59	Y	79	y
1 A	SUB		3 A	:	5 A	Z	7 A	z
1 B	ESC	\e	3 B	;	5 B	[	7 B	{
1 C	FS		3 C	<	5 C	\	7 C	|
1 D	GS		3 D	=	5 D	]	7 D	}
1 E	RS		3 E	>	5 E	^	7 E	
1 F	US		3 F	?	5 F	_	7 F	DEL

A criação da tabela levou em conta a melhor organização dos símbolos e suas funções. O espaço disponível, de zero a 127, foi dividido em 4 seções.

A primeira seção abriga os comandos como mudança de linha, quebra de página, entre outros. A única exceção é o comando DEL, que apaga a letra à direita do cursor, pois se encontra na última posição da tabela. Este comando foi padronizado muito tempo depois da criação da tabela original, e por isso havia sobrado apenas essa posição para ele.

Os comandos foram concebidos para controlar os primeiros equipamentos de impressão, sendo que vários deles não são mais utilizados. Entre os comandos ainda utilizados, temos o LF (**`0x0A`**), que funciona como indicador de mudança de linha. Alguns sistemas utilizam o LF em conjunto com o CR (**`0x0D`**) para indicar uma nova linha. Para utilizar esses comandos na linguagem C costumam-se utilizar códigos de escape para inserir esses caracteres especiais no texto.

A segunda seção abrange a grande maioria dos símbolos de pontuação e caracteres matemáticos, incluindo os números arábicos. Estes símbolos foram posicionados levando-se em conta a sequência em que aparecem nas antigas maquinas de escrever. Atualmente, algumas mudanças foram feitas nos teclados modernos, deste modo, a sequência física não

corresponde exatamente à sequência lógica da tabela, o que não é no entanto, um problema grave.

Por fim, a terceira seção abriga as letras maiúsculas e a quarta, as correspondentes minúsculas. Elas estão organizadas de modo que a diferença entre as maiúsculas e minúsculas é de apenas um bit, facilitando os algoritmos de ordenação de textos.

O código ASCII define apenas os caracteres da língua inglesa, ocupando as posições de 0 à $7F_{16}$ (127_{10}), não possuindo caracteres acentuados ou o alfabeto grego, o cirílico etc. Para dar suporte a estes caracteres outras codificações foram desenvolvidas. Os códigos mais utilizados atualmente são o UTF-8 e o ISO-8859, que possuem compatibilidade com o ASCII, ou seja, as primeiras 127 posições são as mesmas.

2.6 Exercícios

Ex. 2.1 — Qual a vantagem da base hexadecimal?

Ex. 2.2 — Onde é utilizado o código Gray?

Ex. 2.3 — Porque o código ASCII não apresenta suporte a acentos?

Ex. 2.4 — Converta os seguintes números da base decimal para a base binária:

- **`10`**
- **`255`**
- **`32`**
- **`1984`**

Ex. 2.5 — Converta os seguintes números da base hexadecimal para a base binária:

- **`0x125`**
- **`0xAA`**
- **`0x55`**
- **`0x15FB`**
- **`0x4B1D`**
- **`0xAA030586`**

Ex. 2.6 — Qual a diferença de um número codificado em BCD compactado ou em hexadecimal?

CAPÍTULO

3

Linguagem C

3.1 Processo de compilação

3.2 Organização dos programas em C

3.3 Padrão de escrita

3.4 Diretivas de pré-compilação

- Inclusão de arquivos
- Definição e expansão de macros
- Compilação condicional
- Diretiva pragma

3.5 Função main

- Começando com Wiring: Arduino e Chipkit

3.6 Entrada e saída de dados

3.7 Exercícios

"C é peculiar, cheia de falhas, e um enorme sucesso."

Dennis M. Ritchie

C (pronunciado como a letra c: /'se/) é uma linguagem de propósito geral, imperativa, com suporte à programação estruturada e recursão, possuindo variáveis de tipos definidos. O modo como foi projetada permite gerar códigos de máquina bastante eficientes, considerando as arquiteturas mais comuns de processadores. Além disso, apresenta ferramentas que facilitam o acesso a hardware. Por esses motivos, tem sido utilizada como substituta para aplicações que utilizavam a linguagem assembly.

Ela foi inicialmente desenvolvida por Dennis Ritchie, entre 1969 e 1973, na empresa AT&T Bell Labs. O primeiro grande projeto com essa linguagem, e que ajudou muito a sua popularização, foi a reimplementação do sistema operacional Unix.

Ela herdou muitas características da linguagem B, que era utilizada nos minicomputadores da época dentro dos laboratórios da Bell Labs. Por ser sucessora da linguagem B, chamaram-na simplesmente de linguagem C.

A linguagem C foi padronizada pelo instituto de padrões americanos, ANSI, em 1989, e pela ISO, desde 1990, sendo conhecida respectivamente como C89 ou C90. Uma revisão menor foi feita em 1995 pela ISO.

Em 1999 a ISO realizou mudanças na linguagem, como a possibilidade de comentários em uma única linha, melhor suporte a cálculos de ponto flutuante (IEEE754) e funções inline. A ANSI adotou esse padrão em 2000. É comum referenciarmos este padrão como C99.

O padrão em vigor é de 2011, sob o nome ISO/IEC 9899:2011. Este documento apresenta uma interface comum para criação de threads, um melhor suporte aos diferentes padrões de codificação de caracteres, macros para operação com números imaginários, entre outras novidades.

A tabela 3.1 apresenta o histórico do desenvolvimento dessa linguagem de programação. As linguagens BCPL e B constam na tabela por serem as linguagens que influenciaram no desenvolvimento da linguagem C.

Tabela 3.1: Histórico do desenvolvimento da linguagem C

Linguagem	Ano	Desenvolvida por
BCPL	1966	Martin Richards
B	1970	Ken Thompson, Dennis Ritchie
C	1972	Dennis Ritchie
K&R C	1978	Brain Kernighan and Dennis Ritchie
ANSI C	1989	Comitê da ANSI
ANSI/ISO C	1990	Comitê da ISO
ANSI/ISO C/NA1	1995	Comitê da ISO
C99	1999	Comitê da ISO
C11	2011	Comitê da ISO

Devido à popularidade que atingiu na época, a linguagem C influenciou várias linguagens atuais, incluindo C++, D, Go, Rust, Java, JavaScript, Limbo, LPC, C#, Objective-C, Perl, PHP, Python e Verilog. Estas linguagens possuem grande parte de suas estruturas de controle e outras caraterísticas básicas muito similares à C, com uma estrutura sintática muito parecida.

Vários fabricantes e comunidades de software livre possuem ferramentas de compilação em linguagem C para quase todas as arquiteturas de processadores e sistemas operacionais existentes. Os fabricantes de processadores e microcontroladores em geral também disponibilizam ferramentas de compilação para seus chips.

Todas essas vantagens, tanto de facilidade de acesso ao hardware, quanto de disponibilidade e portabilidade, consagraram a linguagem C como a ferramenta ideal para programação em sistemas embarcados.

3.1 Processo de compilação

O processo de compilação pode ser visualizado como a tradução de um texto. Se estivermos utilizando um compilador de linguagem C, a linguagem original é a C e a linguagem final é geralmente um código binário entendido pelo processador.

O arquivo de entrada contém o programa escrito pelo programador, também conhecido como código fonte. Este arquivo pode ser editado em qualquer ferramenta de edição de texto.

O arquivo de saída, ou código de máquina, indica ao processador como executar os comandos que foram definidos no arquivo texto. O arquivo binário depende da arquitetura de processador utilizado. O mesmo código em C gerará arquivos diferentes para processadores diferentes.

O processo de compilação é realizado em quatro etapas que, em geral, são sequenciais: pré-compilação, compilação, assemblagem e linkagem, conforme apresentado na figura 3.1:

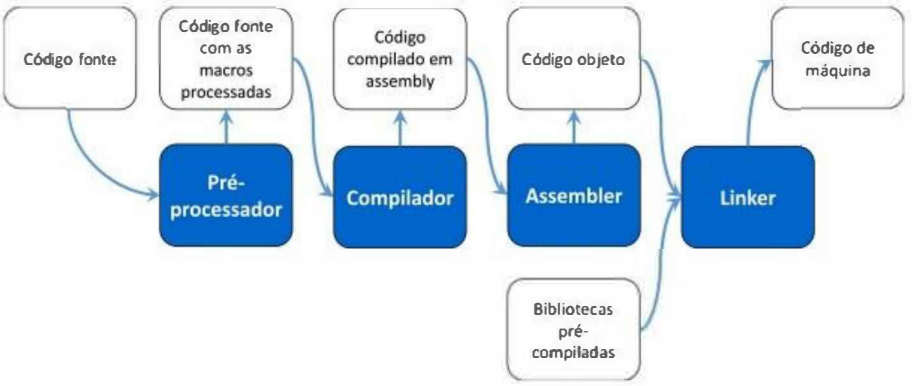


Figura 3.1: Processo de compilação de um programa em C

A etapa de pré-compilação é responsável por ler o código fonte e realizar substituições no código ou operações de configuração do compilador. Esta etapa processa os comandos incluídos no código especificamente para esta função, como #define, #include, #ifndef, #endif, #pragma, entre outros. Os comentários também são removidos nesta etapa. Em geral, os comandos aceitos na etapa de pré-compilação são iniciados pelo símbolo de hash "#". Os arquivos .C e .H são processados e arquivos temporários são gerados com as mudanças necessárias.

A compilação é a etapa mais complexa. A tradução efetiva do código C para assembly é realizada nesta etapa. Assembly é uma linguagem de baixo nível que representa de modo simbólico os comandos da linguagem de máquina. No entanto, ela ainda não é utilizável pois não se encontra em formato binário, além de não estar devidamente localizada na memória do processador. Nesta etapa também é executado o processo de otimização do código.

A otimização tem como objetivo reduzir a quantidade de código para diminuir o tamanho do arquivo final ou alterar o fluxo do programa para que este possa ser executado mais rapidamente. Em geral, esses dois objetivos são opostos, de modo que um programa menor provavelmente será mais demorado. Deve-se tomar cuidado com o processo de otimização, pois o compilador faz suposições que podem não ser verdadeiras para o ambiente embarcado, principalmente se estivermos operando com periféricos.

No fim da compilação para cada arquivo .C é gerado um arquivo intermediário em assembly.

A linkagem é o processo de reunir todos os arquivos intermediários, resultantes da compilação, num único arquivo. Neste momento, todas as referências são processadas e um arquivo final é montado contendo todo o código. As bibliotecas pré-compiladas, como as bibliotecas padrão da linguagem C (stdlib), são incorporadas estaticamente ao programa neste momento. O processo de linkagem dinâmica é pouco utilizado no ambiente embarcado, pois geralmente existe apenas um programa que fará uso da biblioteca, não havendo necessidade de compartilhá-la dinamicamente.

A assemblagem é a última etapa. Ela é responsável por transcrever o código intermediário em assembly para linguagem de máquina, substituindo as referências relativas por referências fixas de endereçamento e posicionamento na memória. O resultado desta etapa é o arquivo binário.

Dependendo do compilador utilizado, é possível que ele gere um arquivo binário com a extensão .s19. Este arquivo contém o código de máquina, mas em formato de texto. Isto facilita o processo de transmissão do arquivo para um gravador que vai convertê-lo em binário e

armazená-lo no chip, pois, além do código, ele contém valores de conferência para garantir que os dados não foram alterados na transmissão.

3.2 Organização dos programas em C

Um programa C é, na verdade, uma coleção de funções, criadas pelo próprio programador ou disponibilizadas em bibliotecas.

A maioria dos compiladores de linguagem C vem com uma grande quantidade de funções já criadas e compiladas em bibliotecas que são usadas dependendo da necessidade do programador. Parte destas bibliotecas se tornaram tão comuns que foram padronizadas pela linguagem.

Os componentes de um programa em C são:

- **Comentários**: podem e devem estar em qualquer ponto do programa. É aconselhável colocar um resumo logo no início do programa com algumas informações, tais como: nome do programa, nome do programador, período de elaboração, para que serve, se é parte de algum sistema maior, restrições de acesso por motivo de segurança, etc. Os comentários são escritos entre os delimitadores `/*` e `*/` e, desta forma, não são

considerados pelo compilador. Outra forma de inserir comentários é utilizar duas barras **`//`**. Qualquer coisa que esteja escrita após as duas barras até o fim da linha será interpretada como comentário.

- **Diretivas de compilação**: não são instruções próprias da linguagem C. As diretivas são mensagens que o programador envia ao compilador para que este execute alguma tarefa no momento da compilação. Deve haver uma linha inteira para cada diretiva, que é iniciada pelo caractere **`#`**. As diretivas mais comuns são **`#include`** e **`#define`**, respectivamente usadas para, especificar bibliotecas de funções a serem incorporadas na compilação, e macro substituições, como veremos mais adiante.

- **Definições globais**: normalmente são especificações de constantes, tipos e variáveis que serão válidas em todas as funções que formam o programa. Embora sejam de relativa utilidade, não é uma boa prática de programação definir muitas variáveis globais. Como elas podem ser acessadas em qualquer parte do programa, mesmo um breve descuido na alteração dos seus valores, pode provocar problemas em muitos outros locais.

- **Protótipos de funções**: são usados pelo compilador para fazer verificações durante a compilação: ver se as partes do programa que acionam as funções o fazem de modo correto, com o nome certo, com o número e tipo de parâmetros adequados, etc. Esta é uma boa prática para se programar em C.

- **Definições de funções**: são os blocos do programa onde são definidos os comandos a serem executados em cada função. A função pode, ou não, receber valores que serão manipulados em seu interior. Após o processamento, as funções podem, se necessário, retornar um valor. É obrigatório a presença de pelo menos uma função com o nome **`main`**, e esta será a função por onde começa a execução do programa. Não há ordem obrigatória para codificar as funções. No entanto, procuraremos sempre começar pela função **`main`**, para facilitar a tarefa de manutenção do programa. Todas as outras funções serão codificadas em seguida.

No código 3.1, é apresentado um exemplo de programa em linguagem C com todos os componentes.

Código 3.1: Exemplo de programa em linguagem C

```
 1 // Comentário utilizado como cabeçalho do arquivo
 2 // Programa 1
 3 // Função: descrição
 4 // Autor: nome
 5
 6 //Diretivas de compilação
 7
 8 //includes de bibliotecas locais
 9 #include "biblioteca.h"
10
11 //includes de bibliotecas padrões
12 #include <stdio.h>
13
14 //definições de tipo
15 #define macros
16 #define labels
17
18 //Seção de variáveis globais
19 char variavelGlobal;
20
21 //Seção de protótipos de funções
22 void funcao1(char var);
23 int funcao2(void);
24
25 //Função main
26 void main(void) {
27     //...
28 }
29
30 //Demais funções do programa
31 void funcao1(char var) {
32     //...
33 }
34
35 int funcao2(void) {
36     //...
37 return x;
38 }
```

3.3 Padrão de escrita

A linguagem C é bastante flexível quanto ao modo de escrita. Existem diversas opções sobre onde abrir uma chave, por exemplo: na mesma linha do **`if`** ou numa linha própria. O

conjunto de escolhas é denominado padrão de escrita.

Obedecer a um padrão na hora de programar facilita a visualização do código e pode ajudar a evitar erros mais simples. Nesta seção será apresentado o padrão utilizado neste livro.

O primeiro ponto importante de um padrão é a indentação dos blocos. Em linguagem C, os blocos são conjuntos de códigos delimitados por chaves: "{" e "}". Para indicar que os comandos pertencem ao mesmo código convenciona-se a adição de um espaço antes da linha, de modo a deslocar todo código interno ao bloco para a direita.

Código 3.2: Programa sem indentação em linguagem C

```
1 void main(void){
2 unsigned int i;
3 unsigned int newKey=0;
4 unsigned int key=0;
5 serialInit();
6 ssdInit();
7 lcdInit();
8 adInit();
9 for(;;){
10 ssdUpdate();
11 newKey = kpRead();
12 if (newkey != key){
13 newkey = key;
14 for(i=0;i<10;i++){
15 if (BitTst(key,i)){
16 lcdChar(i+'0');
17 }
18 }
19 }
20 for(i = 0; i < 1000; i++);
21 }
22 }
```

Se existir um segundo bloco dentro do primeiro, este deve ser deslocado duas vezes, indicando uma hierarquia no fluxo do programa. O código 3.2 é um exemplo de programa sem utilização de nenhuma indentação.

Por não estar indentado adequadamente, fica difícil entender quais comandos são executados dentro de cada loop. A indentação resolve este problema em particular, como

pode ser visto no código 3.3.

Código 3.3: Uso da indentação em linguagem C

```
 1 void main(void){
 2     unsigned int i;
 3     unsigned int newKey=0;
 4     unsigned int key=0;
 5     serialInit();
 6     ssdInit();
 7     lcdInit();
 8     adInit();
 9     for(;;){
10         ssdUpdate();
11         newKey = kpRead();
12         if (newkey != key){
13             newkey = key;
14             for(i=0;i<10;i++){
15                 if (BitTst(key,i)){
16                     lcdChar(i+'0');
17                 }
18             }
19         }
20         for(i = 0; i < 1000; i++);
21     }
22 }
```

Podemos notar, pelos códigos anteriores, que no indentado é mais fácil visualizar quais instruções/rotinas estão subordinadas às demais.

Outra característica de padronização está na criação de nomes de funções e de variáveis. Pela linguagem C, uma função ou variável pode ter qualquer nome, desde que seja iniciada por uma letra, maiúscula ou minúscula, e os demais caracteres sejam letras, números ou underscore "_".

A linguagem C permite que sejam declaradas duas variáveis com mesmo nome, caso possuam letras diferentes apenas quanto à caixa (maiúscula ou minúscula). Por exemplo: **var**

e **Var** são variáveis distintas, o que pode gerar erro no desenvolvimento do programa, causando dúvidas e erros de digitação.

Para evitar esses problemas é comum criar uma convenção no modo de escrita das variáveis. A opção adotada é fazer com que os nomes de variáveis sejam escritos apenas utilizando minúsculas. Caso o nome seja composto, utiliza-se uma letra maiúscula para cada nova palavra, como nas variáveis **contaPosicao** e **contaValorTotal**. Esta abordagem é conhecida como camelCase.

Os nomes de funções podem, por exemplo, ser escritos com a primeira letra maiúscula e, no caso de nome composto, cada inicial será grafada também em maiúscula, por exemplo: **InicializaTeclado()** ou **ParararSistema()**.

Os nomes utilizados nas "tags" ou "labels" de definições, utilizados na diretiva **#define**, serão grafados exclusivamente em maiúsculas como **NUMERO_DE_VOLTAS** ou **CONSTANTE_GRAVITACIONAL**. Para facilitar a leitura utilizaremos um underscore "_" entre as palavras.

Com relação à utilização das chaves, optou-se por abrir o bloco na mesma linha do comando e fechar o bloco numa linha própria, alinhando a chave com o comando que a abriu. Isto reduz a quantidade de linhas necessárias para escrever o programa. No entanto, não há nenhuma diferença prática na execução do programa, nem de velocidade ou de tamanho. Essa abordagem foi escolhida simplesmente pela economia de espaço na visualização dos códigos.

As regras apresentadas visam fornecer uma identidade visual ao código. Tais regras não são absolutas, servem apenas para o contexto deste livro. Em geral, cada instituição ou projeto possui seu próprio conjunto de normas. É importante ter conhecimento deste conjunto e aplicá-lo em seu código.

3.4 Diretivas de pré-compilação

As diretivas de pré-compilação são instruções inseridas nos arquivos de código que visam alterar o programa antes do processo de compilação. Essas instruções não existem no programa final. Todas as diretivas de compilação começam com o sinal **#**, conhecido como jogo da velha ou hash.

Inclusão de arquivos

A diretiva de compilação **#include** é a responsável por permitir que o programador utilize no seu código funções que foram implementadas em outros arquivos, seja por ele próprio ou por outras pessoas. O agrupamento de funções num arquivo para permitir seu uso futuro é chamado de biblioteca.

Os arquivos com a extensão .H são denominados de arquivos de cabeçalho (arquivos "Header") e geralmente possuem apenas os protótipos de função ou definição de tipos de variável disponibilizados pela biblioteca.

Se o código fonte for distribuído junto com a biblioteca ele possui, em geral, a extensão .C. Se for o caso, o programador não precisa distribuir o código fonte, podendo optar por entregar apenas o código objeto.

Deste modo, a biblioteca é geralmente composta por dois arquivos, um header e uma implementação, ou um header e um arquivo objeto.

Para fazer uso de uma biblioteca externa dentro de um arquivo, utiliza-se a diretiva **#include**. Existem dois modos de chamar esta diretiva, a diferença entre elas se encontra no modo como o compilador procura as bibliotecas:

```
1 #include <nome_arquivo.h> // arquivo do path do compilador
2 #include "nome_arquivo.h" // arquivo no mesmo diretório
```

A primeira opção, em que o nome do arquivo é escrito entre os sinais de maior e menor, indica ao compilador que ele deve procurar o arquivo citado nos diretórios conhecidos por ele. Esta opção é utilizada em geral para incluir as bibliotecas padrões da linguagem C.

A segunda opção indica que o arquivo a ser incluído pode estar no mesmo diretório que o arquivo atual.

Para simplificar o desenvolvimento é comum separar as funcionalidades em arquivos diferentes. Em sistemas embarcados, uma opção é separar por tipo de periférico, como portas digitais, comunicação serial ou um display de LCD.

Definição e expansão de macros

Outra funcionalidade do processo de pré compilação é poder efetuar troca de símbolos, valores ou textos antes de compilar o arquivo. Isto pode ser feito através da definição de macros, pela diretiva **`#define`**.

O procedimento de avaliar as macros e efetuar as trocas é conhecido como expansão de macros.

A definição pode ser feita de dois modos. No primeiro, define-se um identificador e, em seguida, o texto que substituirá o identificador cada vez que ele for utilizado no código:

```
#define nome <lista de comandos>
```

Onde:	`nome` É o nome (label) pelo qual a macro será conhecida ou chamada `lista de comandos` São os comandos que substituirão a macro

Um equívoco bastante comum é que a diretiva **`#define`** cria constantes. A utilização do **`#define`** permite criar identificadores ou nomes que serão substituídos pela lista de comandos. Na prática, isto permite inserir valores constantes no código de modo simplificado. No entanto, o processo de otimização do compilador pode pré-computar os valores de modo que o "valor constante" definido através deste método, nem mesmo apareça no código final. Por exemplo:

```c
#define SEGUNDOS_MINUTOS 60
#define MINUTOS_HORAS 60
#define HORAS_DIAS 24

int segundos para dias (int segundos){
    return segundos / (SEGUNDOS_MINUTOS * MINUTOS_HORAS * HORAS_DIAS);
}
```

Após a substituição dos valores, a função será escrita como:

```c
int segundos_para_dias (int segundos){
    return segundos / (60*60*24);
}
```

Como os três valores são constantes, durante o processo de compilação é possível que o compilador otimize esta linha para evitar recalcular o resultado, que será constante, toda vez que a função fosse executada. Deste modo, a função será reescrita como:

```c
int segundos_para_dias (int segundos){
    return segundos / (60*60*24);
}
```

Assim, os valores iniciais das "constantes" nem mesmo aparecem na função final. Eles serviram apenas como unidades de substituição.

O segundo modo de definição permite criar macros que fazem a substituição de um texto, mas mantendo algumas informações. Essas informações que serão mantidas após a substituição são escritas numa lista de parâmetros:

```
1 #define <nome>(<lista_parâmetros>) <lista_comandos>
```

Onde:	**nome** É o nome pelo qual a macro será conhecida ou chamada. **lista_parâmetros** São os "parâmeteros" que serão enviados para a lista de comandos. **lista_comandos** São os comandos que substituirão a macro fazendo uso dos parâmetros.

Esta diretiva é utilizada quando se deseja criar uma estrutura similar à função, mas que seja mais rápida. No entanto, existem algumas limitações. Não é possível criar variáveis locais nem fazer recursão. Além disso, o tamanho do arquivo final será maior, já que cada "chamada" de função terá o código copiado para onde a função foi chamada. Deste modo, o código aparecerá quantas vezes ele for utilizado, ao invés de uma única vez, como uma chamada de função real.

Apresentamos abaixo um exemplo de função com diretiva **#define** que calcula a média entre três valores:

```
1 #define Media(a, b, c) ((a+c+b)/3)
2 void main (void){
3     int x=10, y=20, z=30, resultado;
4     Resultado = Media(x,y,z);
5 }
```

Depois da etapa de pré-compilação, a função será substituída e o código do programa re-escrito da seguinte maneira:

```
1 void main (void){
2      int x=10, y=20, z=30, resultado;
3      Resultado = ((x,z,y)/3);
4 }
```

Notem que, ao substituir o código, o pré-compilador executa as mudanças obedecendo exatamente a estrutura definida. Por isso ele mantém as duas chaves, mesmo não precisando.

Notem que as posições das variáveis z e y estão trocadas depois da substituição. Isso acontece porque na definição da função as variáveis b e c aparecem trocadas. Como o pré-compilador apenas faz a substituição dos termos envolvidos, ele obedece exatamente o que for informado pelo programador.

Por fim, existe a diretiva **#undef**, que remove a definição de uma macro, caso ela exista. Isto permite modificar o valor de uma macro que já tenha sido definida anteriormente.

Uma outra função bastante comum da diretiva **#define** é criar um conjunto de referências que serão utilizadas ao longo do programa. Isto inclui as definições de hardware do dispositivo, como endereços de periféricos ou definições padrão.

Compilação condicional

Existem 6 diretivas que permitem condicionar a compilação de um código baseada numa premissa. São eles: **#ifdef**, **#ifndef**, **#else** e **#endif**.

O funcionamento destas diretivas é muito parecido com a estrutura de decisão da programação.

```
1 #if <cond1>
2     <código mantido se cond1 = verdadeira>
3 #elif <cond2>
4     <código mantido se cond1 = falso mas cond2 = verdadeira>
5 #else
6     <código mantido se ambas condições foram falsas>
7 #endif
```

A condição pode ser escrita utilizando constantes inteiras, caracteres, operações aritméticas, binárias ou lógicas. Macros também podem ser utilizadas. Neste caso, as macros são expandidas antes de executar a operação.

```
1 int simulaQueda(int altura){
2 #if (GRAVIDADE == 9.807)
3     CalculaQuedaComResistênciaAr();
4 #elif
5     CalculaQuedaSemResistenciaAr();
6 #endif
7 }
```

As diretivas **#ifdef** e **#ifndef** realizam decisões baseadas no fato de uma macro ter sido definida ou não. São bastante utilizadas para gerar a configuração do sistema.

Supondo uma empresa que possui três termômetros, uma versão mais barata teria apenas um alarme sonoro para indicar a condição de temperatura alta (sobretemperatura). Uma versão intermediária teria um display de LCD mostrando o valor atual. Uma versão completa teria tanto o LCD quanto o alarme sonoro.

Ao invés de criar três programas separados, os programadores optaram por criar apenas um código que pode ser parametrizado utilizando as macros e as diretivas de pré-compilação.

A tabela 3.2 apresenta o efeito das duas opções de **#define** depois da etapa de pré-compilação. Como podemos observar, apenas os códigos que estavam habilitados pelas macros permaneceram no documento final. Aqueles que não estavam habilitados foram removidos do código. As demais funções se mantiveram.

Código 3.4: Exemplo de uso das diretivas condicionais de compilação

```
1 #define LCD
2 #define SOM
3
4 void PrintTemp(char valor) {
5 #ifdef LCD
6     lcdPrint(valor);
7 #endif //LCD
8
9 #ifdef SOM
10    if (valor > 30) {
11        StartAlarm();
```

```
12      } else {
13          StopAlarm();
14      }
15 #endif //SOM
16 }
17
18 void main (void){
19      int temp;
20
21      temperatureInit();
22
23 #ifdef LCD
24      lcdInit();
25 #endif //LCD
26
27 #ifdef SOM
28      soundInit()
29 #endif //SOM
30
31      for(;;){
32          temp = ReadTemp();
33          PrintTemp(temp);
34      }
35 }
```

Algumas funções estão presentes em ambas as ocasiões. Podemos ver que o código que inicializa o LCD só será executado se a diretiva **#define** LCD estiver presente, e o mesmo acontece com o alarme de som. Assim podemos utilizar o mesmo código base para todos os produtos.

Diretiva pragma

A diretiva **#pragma** funciona de modo bastante diferente das outras diretivas de pré-compilação. Ela não altera o código fonte, mas fornece instruções especiais ao compilador alterando o processo de compilação ou especificando detalhes extra-código.

Tabela 3.2: Comparação entre as duas opções com #define

```
1 #define LCD
2 //a opção SOM não está definida
3 void PrintTemp(char valor) {
4     lcdNumber(valor);
5 }
6
7 void main (void){
8     int temp;
9     temperatureInit();
10    lcdInit();
11    for(;;){
12        temp = ReadTemp();
13        PrintTemp(temp);
14    }
15 }
```

```
1 //a opção LCD não está definida
2 #define SOM
3 void PrintTemp(char valor) {
4     if (valor > 30) {
5         StartAlarm();
6     } else {
7         StopAlarm();
8     }
9 }
10
11 void main (void){
12     int temp;
13     temperatureInit();
14     soundInit()
15     for(;;){
16         temp = ReadTemp();
17         PrintTemp(temp);
18     }
19 }
```

Essa diretiva depende da implementação do compilador, ou seja, o seu funcionamento, bem como as opções disponíveis, depende do fabricante do compilador e do processador para o qual o código está sendo compilado.

```
#pragma <definição> <opções>
```

Onde:	**definição** Define que tipo de comando pré-compilação será executado. **opções** São as opções passadas para o comando de pré-compilação.

A opção **CODE_SEG** é uma opção que pode não estar presente em todas as plataformas, ou pode assumir outros nomes. Ela indica ao linker em qual posição da memória ele deve armazenar as variáveis ou funções escritas depois do comando.

```
1 #pragma CODE_SEG __NEAR_SEG NON_BANKED
2 //a variável abaixo será armazenada em memória não paginável
3 int variável_nao_paginável;
4
5 #pragma CODE_SEG DEFAULT
6 //a variável abaixo está armazenada na região padrão
7 //da memória, podendo não estar mapeada caso haja uma
8 //troca de banco de memória
9 int variável_paginável;
```

Outro uso desta diretiva é configurar registros específicos do microcontrolador. Em geral, estes registros controlam as configurações básicas do processador como frequência de funcionamento, capacidade de debug, entre outras opções.

Para saber quais configurações são possíveis é necessário checar o datasheet do microcontrolador e a documentação do compilador/linker.

O código abaixo configura algumas informações de funcionamento de um microcontrolador utilizando a diretiva **#pragma**:

```
1 #pragma config OSC=HS        // Oscilador à cristal externo
2 #pragma config WDT=OFF       // Watchdog desligado
3 #pragma config DEBUG = OFF   // Desabilita debug
```

Existem ainda outras opções. Em geral são bastante específicas, sendo voltadas para cada chip, fabricante e compilador.

3.5 Função main

Todo sistema precisa ser iniciado de algum lugar. Os microcontroladores, assim que ligados, começam a executar os códigos a partir de um endereço pré-definido.

Para vários processadores, é necessário fazer um conjunto de configurações iniciais para que seja possível executar o programa em linguagem C.

Entre essas configurações temos a alocação do ponteiro de stack, configuração da página de memória ativa ou, até mesmo, o carregamento do código do programa para a memória RAM. Este procedimento é executado logo após o sistema ser ligado. O código para essa inicialização é feito geralmente em assembly e implementado pelo próprio compilador. Depois de sua execução essa rotina inicial é chamada a função **main()**.

A função **main()** é, portanto, a primeira função que pode executar algum código planejado pelo programador.

Por ser a primeira função a ser executada em sistemas embarcados, ela possui um formato um pouco diferente das implementações tradicionais, já que ela não recebe nem retorna nada, conforme o código a seguir:

```
1 void main(void) {
2       //aqui entra o código do programa
3 }
```

Outro ponto específico deste tipo de sistema é que são projetados para começarem a funcionar assim que ligados e apenas parar quando desligados. Como todas as funcionalidades são chamadas dentro da função **main()**, espera-se que o programa continue executando as instruções dentro dela até ser desligado ou receber um comando para desligar. Esse comportamento pode ser obtido através de um loop infinito.

Para se fazer um loop infinito basta utilizar uma estrutura de repetição que sempre seja verdadeira. A seguir estão as duas alternativas bastante utilizadas.

```
1 void main (void){
2     for(;;){
3         //aqui entra o
4         //código principal
5     }
6 }
```

```
1 void main (void){
2     while(1){
3         //aqui entra o
4         //código principal
5     }
6 }
```

Não existe diferença entre os dois modos de implementar o loop infinito.

Começando com Wiring: Arduino e Chipkit

As plataformas compatíveis com Arduino utilizam uma linguagem derivada da C++, chamada Processing, em conjunto com o framework Wiring. Nesta plataforma não temos acesso direto à rotina **main()**. A Wiring provê, no entanto, duas funções: a **setup()** e a **loop()**.

A função **setup()** é executada uma única vez quando o sistema é iniciado. Já a função **loop()** é executada ciclicamente enquanto o sistema permanecer ligado. A função **main()** é, portanto, escondida do programador, mas pode ser encontrada dentro da pasta do Arduino. A implementação completa da função **main()** do framework Wiring é apresentada no códig o 3.5:

Código 3.5: Função main() do framework Wiring

```
 1 int main(void){
 2     init();
 3 #if defined(USBCON)
 4     USBDevice.attach();
 5 #endif
 6     setup();
 7     for (;;) {
 8         loop();
 9         if (serialEventRun) serialEventRun();
10     }
11     return 0;
12 }
```

Assim que a placa é ligada a função **main()** é chamada. Ela executa uma inicialização básica do sistema e, se a placa faz uso de conexão USB, realiza a conexão com a USB para permitir a gravação e depuração da placa. Logo em seguida é executada a função **setup()** que foi implementada pelo programador.

Um loop infinito é criado pela instrução **for(;;)**. Dentro dele há duas funções: a **loop()**, que executa os comandos definidos pelo programador, e **serialEventRun()**, que executa comandos de eventos da serial. Ambas as funções são de responsabilidade do programador.

Na maioria dos exemplos deste livro podemos simplificar o funcionamento e entender a rotina de inicialização do Arduino como se ela fosse implementada conforme o código a seguir.

```
1 void main(void){
2     setup();
3     for (;;) {
4         loop();
5     }
6 }
```

3.6 Entrada e saída de dados

O foco deste livro é a apresentação da linguagem C como uma ferramenta para a programação de sistemas embarcados. Estes sistemas, em geral, possuem poucos recursos para entrada ou saída de dados.

A placa de desenvolvimento utilizada possui um caminho de comunicação serial que pode ser utilizado para receber ou enviar informações para o computador. Neste tipo de comunicação, todas as informações trafegam como textos, sendo necessários alguns passos para converter essa informação em números ou em outros tipos de dado.

Além da comunicação serial, a placa de desenvolvimento possui um LCD e um display de 7 segmentos com quatro dígitos para exibir informações e um teclado com 10 botões para receber informações. Todos os exemplos apresentados utilizarão algum destes dispositivos de exibição.

Para que estes dispositivos funcionem corretamente é necessário inicializá-los no começo do programa.

O teclado, além da inicialização, realiza um processo de leitura das teclas que deve ser constantemente executado através da função **`kpDebounce()`**. Por fim, para realizar a leitura de uma tecla basta utilizar a função **`kpReadKey()`**.

Para escrever no LCD podemos utilizar duas funções diferentes: uma função para escrever números **`lcdNumber()`** e uma para textos **`lcdString()`**.

A função **`lcdPosition()`**, por sua vez, posiciona o cursor segundo a linha e a coluna informada.

O código a seguir realiza a leitura das teclas do teclado e imprime no LCD, sempre na posição inicial.

```
 1 void main (void){
 2     int tecla;
 3     kpInit();
 4     lcdInit();
 5     for(;;){
 6         kpDebounce();
 7         tecla = kpReadKey();
 8         lcdPosition(0,0);
 9         lcdNumber(tecla);
10     }
11 }
```

Para as plataformas Arduino e Chipkit, baseadas na Wiring, as funções de inicialização são executadas dentro da função **`setup()`**. As demais funções ficam dentro da função **`loop()`**.

```
 1 void setup (void){
 2     kpInit();
 3     lcdInit();
 4 }
 5
 6 void loop(void){
 7     int tecla;
 8     kpDebounce();
 9     tecla = kpReadKey();
10     lcdPosition(0,0);
11     lcdNumber(tecla);
12 }
```

3.7 Exercícios

Ex. 3.1 — O que é uma linguagem de alto nível? Qual a sua diferença para uma linguagem de baixo nível?

Ex. 3.2 — Como é o processo de compilação de um programa?

Ex. 3.3 — Qual a função do compilador? E do linker?

Ex. 3.4 — Qual a necessidade de se criar um padrão de escrita para os programas?

Ex. 3.5 — O que é o erro de referência cíclica? Como evitá-lo?

Ex. 3.6 — Em que momento do processo de compilação as diretivas do tipo **`#pragma`** são executadas?

Ex. 3.7 — Quais os modos de entrada e saída disponíveis na placa de desenvolvimento? Como imprimir uma função corretamente?

Ex. 3.8 — Qual a relação entre as funções `setup()`, `loop()` e `main()`?

Ex. 3.9 — Porque em alguns sistemas embarcados a função `main()` não recebe nem retorna nenhum valor?

CAPÍTULO

4

Variáveis

4.1 Utilização de números e seus tipos

4.2 Declaração de variáveis

- Inicialização das variáveis
- Inicialização de conjunto de caracteres
- Vírgula

4.3 Conversão de tipos

- Promoção de tipos
- Perda de informação na conversão de tipos

4.4 Modificadores

- Modificadores de tamanho e sinal
- Modificadores de sinal
- Modificadores de acesso
- Modificador de armazenamento

4.5 Ponteiros

4.6 Exercícios

"A constante de um homem é a variável de outro."

Alan Perlis

Na linguagem C todas as informações são armazenas em estruturas denominadas variáveis. A declaração de uma variável é bastante simples:

```
<tipo> nomeDaVariavel = valorInicial;
```

Onde:

tipo É o tipo da variável, indica seu tamanho e modo de interpretação.

nomeDaVariavel É um identificador para a variável criada, deve ser único.

valorInicial É o valor que o programa dará inicialmente para a variável.

As variáveis são definidas com um nome e um tipo. Para criar o nome da variável podem ser utilizados quantos caracteres forem desejados, contanto que o primeiro caractere seja uma letra ou sublinhado. Importante lembrar que a linguagem C faz distinção entre maiúsculas e minúsculas. Sendo assim, **`matrix`** e **`MaTrIx`** são variáveis distintas.

Outra restrição é que a variável não pode ter o mesmo nome de uma palavra reservada em linguagem C. As palavras reservadas são:

- **`auto`**
- **`break`**
- **`case`**
- **`char`**
- **`const`**
- **`continue`**
- **`default`**
- **`do`**
- **`double`**
- **`else`**
- **`enum`**
- **`extern`**
- **`float`**
- **`for`**
- **`goto`**
- **`if`**
- **`int`**
- **`long`**
- **`register`**
- **`return`**
- **`short`**
- **`signed`**
- **`sizeof`**
- **`static`**
- **`struct`**
- **`switch`**
- **`typedef`**
- **`union`**
- **`unsigned`**
- **`void`**
- **`volatile`**
- **`while`**

O tipo define como a variável deve ser interpretada: se ela representa um número com vírgula, se ela pode armazenar valores positivos ou negativos, se ela indica um endereço ou se representa um caractere, entre outras características. Dependendo da escolha, ela pode ocupar uma quantidade maior ou menor de memória.

Existem quatro tipos básicos de dados na linguagem C. Estes tipos, bem como a faixa de valores que conseguem armazenar, são apresentados na tabela 4.1. A faixa de valor, bem como a quantidade de bits das variáveis, são dependentes do compilador utilizado.

Tabela 4.1: Tipos de dado e faixa de valores

Tipo	Bits	Bytes	Faixa de valores
`char`	8	1	-128 a 127
`int`	16	2	-32 .768 a 32 .767
`float`	32	4	$3,402 \times 10^{-38}$ a $3,402 \times 10^{38}$
`double`	64	8	$1,797 \times 10^{-308}$ a $1,797 \times 10^{308}$

As variáveis que ocupam mais espaço na memória podem armazenar valores maiores. Podemos perceber também que apenas os tipos **`float`** e **`double`** possuem casas decimais.

O tamanho do tipo **`char`** é definido nos padrões da linguagem C. Já o tipo **`int`** possui apenas uma restrição, que ele deve ser de pelo menos 16 bits. Deste modo, o tamanho real do inteiro depende do processador e do compilador utilizado. O mais comum é que ele possua o tamanho que o processador possa trabalhar naturalmente. Assim, é comum que os processadores de 64 bits possuam os inteiros com 64 bits, os de 32 bits com inteiros de 32 e os processadores de 16 bits com inteiros de 16. Os processadores de 8 bits possuem inteiros de 16 bits dada a restrição da norma.

Os tipos **`float`** e **`double`** não possuem uma definição de tamanho. A norma exige uma quantidade mínima de bits para esses dois tipos, não definindo um máximo. Para a maioria dos compiladores é comum adotar o padrão IEEE 754, que utiliza 4 bytes para precisão simples (**`float`**) e 8 bytes para precisão dupla (**`double`**).

Estes valores conseguem representar números com vírgulas de maneira similar à que escrevemos os números em notação científica: armazenando uma mantissa e um expoente. A f igura 4.1 apresenta a relação entre os bits e o número com precisão simples:

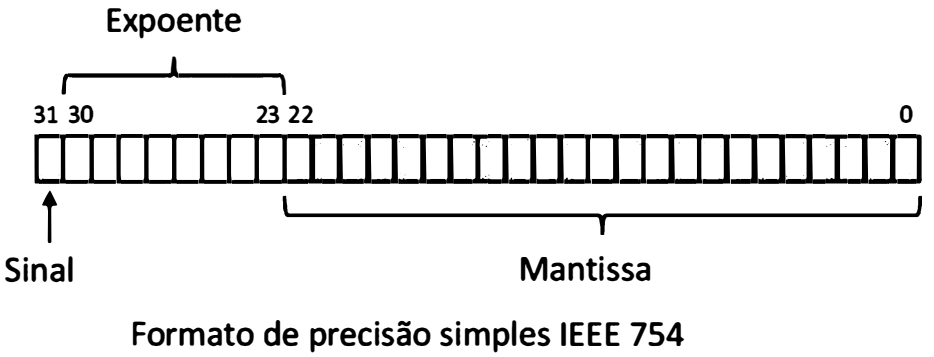


Figura 4.1: Formação do número do tipo **`float`** segundo padrão IEEE 754

Neste caso, o valor M (mantissa) é sempre um valor entre -1 e 1. Já o valor do expoente E são números inteiros, positivos ou negativos. A mantissa representa a precisão do número, já o expoente representa a faixa de valores que podem ser representados.

4.1 Utilização de números e seus tipos

A linguagem C trata, a princípio, de todo número que estiver escrito no meio do código como um valor do tipo inteiro e sinalizado. Por padrão, os números são interpretados como escritos no formato decimal. Durante o processo de compilação, estes números são transformados em binário para serem salvos na memória do microcontrolador.

Em algumas situações, pode ser mais simples, ou conveniente, escrever os números numa base diferente da decimal, seja a binária ou a hexadecimal. Isto pode ser feito utilizando os prefixos **`0b`**, para binário, ou **`0x`**, para hexadecimal, antes do número. Devemos lembrar que, no caso de números binários, só são aceitos os dígitos zero e 1. Para os hexadecimais são aceitos os dígitos de zero a 9 e as letras de A a F. As letras podem ser escritas em maiúscula ou minúscula.

```
1 // 200₁₀ = C8₁₆ = 11001001₂
2 unsigned int dec = 200;          //valor 200 escrito em decimal
3 unsigned int hex = 0xC8;         //valor 200 escrito em hexadecimal
4 unsigned int bin = 0b11001001; //valor 200 escrito em binário
5 //todas as três variáveis possuem o mesmo valor
```

Não importa em qual base o número foi inserido, todos serão convertidos para binário no momento do armazenamento. Deste modo, todas as comparações no código exemplo são verdadeiras, já que os números 200_{10}, $C8_{16}$ e 11001001_2 representam a mesma quantidade, apenas em bases diferentes.

A linguagem C ainda permite utilizar a base octal. Para isto, basta digitar o número começando com **0**.

```
1 int a = 200;     //valor 200 em decimal
2 int b = 0200;    //valor 200 em octal, representa 192 em decimal
3 int c = 0311;    //valor 311 em octal, representa 200 em decimal
4 int d = 311;     //valor 311 em decimal
```

A base octal não é muito comum nos programas atuais. Além disso, o modo de entrada dos valores pode confundir quem estiver lendo o código, visto que estamos treinados a ignorar os zeros à esquerda do número. Por esses motivos, essa base não será utilizada ao longo deste livro.

Para valores fracionados utiliza-se o ponto como separador decimal. A linguagem C pressupõe que todo número digitado com um ponto é do tipo **double**. Para garantir que o número digitado seja um **float** é preciso anexar a letra **f** como sufixo ao número.

```
1 float a = 3.05;   //3.05 é convertido e armazenado como float
2 double b = 3.05;  //3.05 é armazenado como double.
3
4 // por questões de precisão no armazenamento é possível
5 // que (a) possua um valor diferente de (b)
6 if(a == 3.05){
7     // a comparação é feita entre um valor float (a) e um valor double (3.05)
8 }
9 if(a == 3.05f){
10    // a comparação é feita entre um valor float (a) e um valor float (3.05f)
11 }
12 if(b == 3.05){
13    // a comparação é feita entre um valor double (b) e um valor double (3.05)
14 }
```

A primeira conversão pode ser interpretada como falsa, pois na conversão do número 3.05 para **`float`** há um arredondamento, de modo que o número armazenado pode não ser mais igual ao valor 3.05.

A segunda comparação é verdadeira, pois o número 3.05 é convertido antes da comparação, por causa do sufixo '**`f`**'. Deste modo, ambos os valores sofreram o mesmo tipo de arredondamento, possuindo o mesmo valor, que pode ser diferente de 3.05.

Por fim, a terceira comparação também é verdadeira, pois ambos os termos da comparação são do tipo **`double`**.

4.2 Declaração de variáveis

Uma declaração de variáveis é uma instrução que resulta em uma quantidade de memória para armazenar o tipo especificado.

Uma declaração de variável consiste no nome de um tipo, seguido do nome da variável:

```
<tipo> variavel;
```

Onde:	`<tipo>` Indica o tipo que será utilizado para criar a variável. `variavel` É o nome da variável.

Em alguns compiladores da linguagem C é necessário que todas as variáveis sejam declaradas antes de se executar qualquer função ou código. Deste modo, não é possível construir um loop **`for`** criando a variável na inicialização do loop.

Se as variáveis forem declaradas depois de alguma função, o programa acusa erro, como apresentado na tabela 4.2. Coloque a criação das variáveis sempre no início da função.

Tabela 4.2: Problema na ordem da declaração das variáveis

```
1  //Este código não compila
2
3  void delay(void){
4
5      for(int i; i<10; i++);
6  }
7
8  void main (void){
9
10     kbInit();
11     lcdInit();
12
13     for(;;){
14         kbDebounce();
15         int key;
16         key = kbReadKey();
17         lcdNumber(key);
18         delay();
19     }
20 }
```

```
1  //Este código compila sem problemas
2
3  void delay(void){
4      int i;
5      for(i; i<10; i++);
6  }
7
8  void main (void){
9      int key;
10     kbInit();
11     lcdInit();
12
13     for(;;){
14         kbDebounce();
15
16         key = kbReadKey();
17         lcdNumber(key);
18         delay();
19     }
20 }
```

Inicialização das variáveis

É possível combinar uma declaração de variável com o operador de atribuição, para que a variável tenha um valor ao mesmo tempo de sua declaração. A linguagem C não inicializa variáveis com valores padrão, deste modo, uma variável não inicializada pode conter "lixo". Como boa prática devemos sempre inicializar as variáveis no momento da declaração para evitar problemas.

Na linguagem C escrevemos os caracteres entre aspas simples, **'a'**, e números sem aspas, **97.6**. Notar que o separador da parte inteira e fracionária é o ponto "**.**", e não a vírgula "**,**".

Exemplo:

```
1 void main(void){
2     //declaração de variáveis com valor inicial
3     char inicial = 'A';
4     int idade = 18;
5     float peso = 60.5, salario = 300;
6
7     //utilização das variáveis
8     lcdInit();
9     lcdPrint(inicial);
10    lcdPrintNum(idade);
11    for(;;){
12    }
13 }
```

Inicialização de conjunto de caracteres

Quando uma variável é declarada na linguagem C, ela não contém valor e por isso uma boa prática é inicializá-la na declaração. Para inicializar uma variável com um conjunto de caracteres, devemos declará-la como se fosse um vetor de caracteres.

```
1 void main(void){
2     char nome[10] = "Jose";
3     lcdInit();
4     lcdPrintString(nome);
5     for(;;){
6     }
7 }
```

Um cuidado a ser tomado com a criação e inicialização de uma cadeia de caracteres é que o tamanho deve ser pelo menos uma unidade maior que a quantidade de caracteres.

Vírgula

A vírgula pode ser utilizada de dois modos dentro da linguagem C. O primeiro é como operador. Neste caso, as expressões entre vírgula são processadas da esquerda para a direita. Como a vírgula é o operador com a menor prioridade na linguagem C, todas as demais operações são executadas primeiro.

O uso da vírgula como operador não é muito comum. Sua utilização mais tradicional é a separação de informações, seja na criação de uma variável ou na passagem de parâmetros. Por exemplo:

```
 1 void main (void){
 2     int a=4, b=7;
 3     int c;
 4     c = a * b;
 5     lcdInit();
 6     lcdNumber(c);
 7     for(;;){
 8
 9     }
10 }
```

No código anterior, as variáveis **a** e **b** são criadas como sendo do tipo **int**.

4.3 Conversão de tipos

A linguagem C nos permite converter um valor de um tipo para um valor de outro tipo. Este processo é denominado typecast.

Para converter um valor de um tipo para outro basta escrever entre parênteses o nome do novo tipo antes do valor original.

```
(<novo_tipo>) variável/valor
```

Onde:	**<novo_tipo>** É o tipo para o qual a variável/valor será convertida. Fica entre parênteses. **variável/valor** É o valor a ser convertido. Esse valor pode vir de uma variável.

No código 4.1, é apresentado um exemplo com três conversões de tipo. Todo número digitado sem ponto é automaticamente entendido como inteiro. Se o valor está marcado com ponto decimal, na linguagem C o tipo deste valor é definido como **double**.

Código 4.1: Exemplo de conversão entre tipos

```
1 double calculaPreco(void){
2     int quantidade = 1;
3     double preco = 30.0;
4     float imposto = (float)0.05;
5     double preco_final = (double) preço * imposto * (double) quantidade;
6     return preco_final;
7 }
```

Na primeira conversão do código 4.1, o valor **0.05**, que é naturalmente entendido como um **double** pelo compilador, é convertido para **float** antes de ser armazenado na variável **imposto**.

A segunda conversão é feita com o valor da variável **preco**, uma variável do tipo **float**, transformando-a em um valor **double**. Nesta situação a variável **preco** não é alterada. O valor armazenado dentro da variável **preco** é copiado para uma memória temporária e então convertido para **float**. A variável original não é alterada.

Por fim, a última conversão do código 4.1 faz a leitura do valor armazenado na variável **quantidade** e o converte para o tipo **double**. Como as três grandezas estão agora em **double** o compilador executa o processo de cálculo e armazena o resultado na variável **preco_final** do tipo **double**.

Ao realizar um typecast tenha cuidado para não perder informação. Uma conversão de um tipo **float** para um tipo **double** pode fazer com que algumas casas decimais sejam ignoradas. Qualquer conversão de um tipo com mais bits para um tipo com menos bits pode causar estouro da variável.

Promoção de tipos

Em algumas situações, o compilador, percebendo que as variáveis ou os valores de uma operação são de tipos diferentes, efetua o typecast automaticamente. Esta situação é

denominada promoção de tipos.

A promoção acontece apenas quando a troca do tipo não causar perda de informação. Deste modo, o compilador sempre promove o valor com menor precisão. A ordem de promoção é dada por:

`char` → `int` → `float` → `double`

Perda de informação na conversão de tipos

Quando se converte uma variável com maior quantidade de bits para uma variável com uma quantidade menor, os bits mais significativos são descartados. O valor 1234, por exemplo, se armazenado numa variável do tipo **`unsigned char`**, ignorará os bits mais significativos, fazendo com que apenas o valor 57 seja armazenado. Supondo o tipo **`int`** com 16 bits, a figura 4.2 representa esta perda.

```
unsigned int  bigNum;
unsigned char num;
bigNum = 12345;
num = bigNum;
//num = 57
```

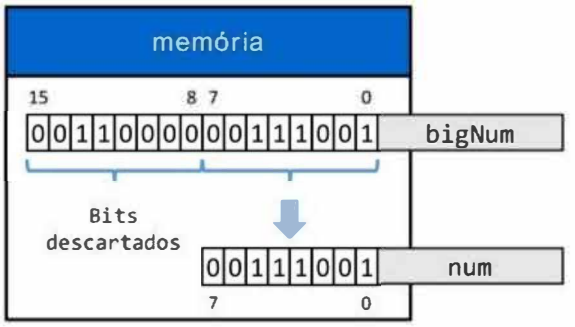


Figura 4.2: Perda de informação na conversão de um **`int`** para um **`char`**

É provável que o compilador gere um aviso (warning) quando este código for compilado indicando que pode haver perda de informação na atribuição. Para indicar ao compilador que estamos cientes disto e que esta é realmente a operação que desejamos fazer, podemos utilizar um typecast, conforme o código a seguir:

```
1 unsigned int  bigNum;
2 unsigned char num;
3 bigNum = 12345;
4 num = (unsigned char)bigNum;
5 //num = 57, mas não gera um aviso (warning) pelo compilador
```

4.4 Modificadores

Modificadores são palavras reservadas que, adicionadas aos tipos básicos de dados, têm a capacidade de alterar seu comportamento, tamanho ou modo de armazenamento. Estes modificadores são inseridos na variável no momento de sua declaração e acompanham a variável por todo tempo, não sendo possível alterá-la.

```
<modificador1> ... <modificadorN> <tipo> nome;
```

Onde:

<modificadorN>
É o nome do modificador a ser utilizado.

<tipo> É o tipo da variável, indica seu tamanho e modo de interpretação.

nomeDaVariavel
É um identificador para a variável criada, devendo ser único.

valorInicial
É o valor que o programa dará inicialmente para a variável.

A seguir serão apresentados os modificadores presentes na linguagem C.

Modificadores de tamanho e sinal

Existem dois modificadores de tamanho: **long** e **short**. Um tipo declarado com o modificador **long** pode ter tamanho *maior* ou *igual* ao tipo original. Um tipo declarado como **short** deve ter tamanho *menor* ou *igual* ao tipo original. A decisão de como os modificadores se comportam, cabe ao compilador. A tabela 4.3 mostra um exemplo para um processador onde o inteiro é de 16 bits.

Tabela 4.3: Impacto dos modificadores long e short numa variável inteira

Tipo	Bytes	Excursão máxima
int	2	-32 .768 a 32 .767
long int	4	-2 .147 .483 .648 a 2 .147 .483 .647
short int	2	-32 .768 a 32 .767

Como 16 bits é o menor tamanho para uma variável do tipo inteiro, o modificador **short** não faz efeito. Já o identificador **long** dobra o tamanho da variável, permitindo armazenar valores da ordem dos bilhões.

Modificadores de sinal

Existem dois modificadores de sinal: **signed** e **unsigned**. Os tipos declarados como **signed** possuem um bit reservado para o sinal. Deste modo, o valor máximo que podem atingir é menor.

Os tipos declarados como **unsigned** não podem assumir valores negativos, mas, em compensação, podem armazenar valores duas vezes maior que um tipo **signed**. A tabela 4.4 apresenta essas mudanças para as variáveis do tipo **int** e **char**.

Tabela 4.4: Impacto dos modificadores signed e unsigned nas variáveis int e char

Tipo	Bytes	Excursão máxima

unsigned char	1	0 a 255
signed char	1	-128 a 127
unsigned int	2	0 a 65 .535
signed int	2	-32 .768 a 32 .767

Se não forem utilizados os modificadores de sinal, por padrão o tipo inteiro será sinalizado. Deste modo, não é necessário escrever **signed int**, isto já está implícito.

No entanto, o tipo **char** não possui comportamento definido, podendo possuir ou não sinal, dependendo do compilador.

As variáveis do tipo **float** e **double** sempre possuem um bit de sinalização. A declaração de uma variável **unsigned double**, por exemplo, não existe.

Modificadores de acesso

Durante o processo de compilação existe uma etapa de otimização do programa. Durante esta etapa, o compilador pode retirar partes do código ou desfazer loops com períodos fixos. Por exemplo, o código abaixo:

```
1 //variável global
2 int x;
3
4 //inicio do programa
5 void main(void){
6     while (X==X);
7 }
```

Quando compilado, o loop do programa acima é transformado nos comandos em assembly como apresentado no código 4.2:

Código 4.2: Assembly para loop

```
1 // Starting pCode block
2 S_Teste__main   code
3 _main:
4     .line       19  // Teste.c  while (X==X);
5     BRA  _main
6     RETURN
```

Enquanto a variável **x** for igual a **x**, o programa permanece no loop. O compilador entende que esta condição sempre será verdadeira e, portanto, não há necessidade de se realizar o teste a cada rodada.

Deste modo, o compilador elimina o teste e insere a operação **BRA _main** que indica ao processador para voltar ao ponto inicial do loop. Isto acontece porque, para as variáveis comuns, seu valor só é alterado em atribuições diretas. O valor da variável não se altera sem o conhecimento do processador.

Entretanto existe uma condição onde a variável **x** pode alterar seu valor independentemente do programa. Se esta variável representar um endereço de memória associado a um periférico físico, seu valor pode mudar independentemente do fluxo do programa. A variável **X** pode indicar o resultado de um teclado, sendo que a tecla pode ser pressionada a qualquer momento do loop. Para indicar esse tipo de situação para o compilador utilizamos a **volatile**.

```
1 //variável global
2 volatile int x;
3
4 //inicio do programa
5 void main(void){
6     while (X!=X);
7 }
```

Compilando o código fonte com a palavra **volatile**, temos o código 4.3:

Código 4.3: Assembly para loop com variável volatile

```
1 // Starting pCode block
2 S_Teste__main   code
3 _main:
4 _00105_DS_:
5     .line       19              // Teste.c   while (X == X);
6     MOVLW       0x83            //primeira parte do endereço
7     MOVWF       r0x00
8     MOVLW       0x0f            //segunda parte do endereço
9     MOVWF       r0x01
10    MOVFF       r0x00, FSR0L
11    MOVFF       r0x01, FSR0H
12    MOVFF       INDF0, r0x00    //realiza primeira leitura
13    MOVLW       0x83            //primeira parte do endereço
14    MOVWF       r0x01
15    MOVLW       0x0f            //segunda parte do endereço
16    MOVWF       r0x02
17    MOVFF       r0x01, FSR0L
18    MOVFF       r0x02, FSR0H
19    MOVFF       INDF0, r0x01    //realiza segunda leitura
20    MOVF        r0x00, W
21    XORWF       r0x01, W
22    BZE         _00105_DS_      //faz o teste para igualdade
23    RETURN
```

Podemos perceber que, deste modo, o compilador é forçado a ler a variável **x** duas vezes. Após armazenar as duas leituras, ele realiza o teste para ver se ela permanece com o mesmo valor. A palavra **volatile** força, portanto, que o processador realize as operações na variável sem otimizações. Esta estrutura pode ser utilizada, por exemplo, para aguardar que um evento aconteça. O programa ficará travado no loop enquanto as leituras forem iguais. Se a primeira leitura for diferente da segunda, isto significa que o evento acabou de acontecer, e que, portanto, o programa pode continuar.

Em algumas situações é necessário indicar que algumas variáveis não podem receber valores pelo programa. Para isto utilizamos a palavra reservada **const**. Utilizamos este modificador para indicar que a variável representa um local que apenas pode ser lido e não modificado, como, por exemplo, uma porta para entrada de dados. Nesta situação é comum utilizar as palavras **volatile** e **const** juntas.

```
1 volatile const int X;
2 //início do programa
3 void main(void){
4     while (X == 3){
5         //realiza tarefa enquanto a variável não muda
6     }
7 }
```

Se em algum momento do código tentarmos modificar o valor da variável **X**, o compilador retornará sem sucesso à compilação com o seguinte erro:

```
main.c: error 33: Attempt to assign value to a constant variable (=)
```

Modificador de armazenamento

As variáveis em linguagem C possuem um tempo de vida que é condizente com o ambiente em que ela foi criada. Por exemplo, uma variável declarada dentro de uma função. Quando a função começa a ser executada ela reserva um espaço de memória para a variável em questão. Quando a função termina o espaço de memória que foi reservado para a variável é liberado automaticamente, fazendo com que o valor que estava armazenado na variável se perca. Estas variáveis são armazenadas na pilha de dados, conhecida como stack.

Já algumas variáveis possuem um espaço reservado de modo permanente. Durante todo o tempo em que a placa estiver ligada aquele espaço será utilizado exclusivamente para a variável.

Em geral, é trabalho do compilador definir onde a variável será armazenada. Para forçar uma variável a possuir um espaço reservado podemos utilizar o modificador de armazenamento **`static`**. Deste modo, as variáveis locais de uma função passam a armazenar o valor mesmo que a função termine, pois ela possui um endereço exclusivo para armazenar seu valor.

```
 1 //cria um contador persistente que é
 2 //incrementado a cada chamada de função
 3
 4 int ContadorPersistente(int reseta) {
 5     static char variável_persistente;
 6     if (reseta) {
 7         variável_persistente = 0;
 8     } else {
 9         return (variável_persistente++);
10     }
11     return -1;
12 }
```

Outro modificador de armazenamento é o **register**. Ele indica ao compilador que desejamos que a variável seja armazenada, se possível, nos registros do processador. Armazenar a variável num registro do processador aumenta muito a velocidade com que as operações realizadas com ela acontecem. No entanto, a quantidade de registros é muito limitada. Algumas arquiteturas de processador possuem um único registro.

Outro problema com o modificador **register** é que se a variável for efetivamente armazenada no registro, pode ser impossível recuperar o endereço dela para realizar qualquer operação com ponteiros, por exemplo.

Por fim, o modificador **register** é apenas uma sugestão para o compilador. Segundo a norma, o compilador não é obrigado a obedecer esta exigência do programador. Este modificador acaba funcionando como sugestão.

O último modificador de acesso é o **extern**. Esse modificador indica que a variável em questão já foi declarada em outro lugar, geralmente em outro arquivo. Este modificador visa permitir que diferentes arquivos acessem uma mesma variável. Toda variável em linguagem C declarada fora de uma função possui o modificador **extern** implícito.

4.5 Ponteiros

Toda variável criada é armazenada numa posição da memória. O número que identifica essa posição é chamado de endereço.

O endereço em que uma variável será salva depende de alguns fatores. Variáveis locais são, em geral, armazenadas na stack. Estas variáveis possuem endereços diferentes a cada execução da função, a menos que usem o modificador **static**.

Variáveis globais, declaradas fora das funções, possuem endereço fixo durante todo o tempo em que a placa permanecer ligada.

Para obter o endereço de uma variável podemos utilizar o operador **&**. O valor do endereço pode então ser utilizado como um valor normal.

Conhecer o endereço de uma variável é fundamental para criarmos um ponteiro para ela:

```
1 //cria a variável a num endereço de memória a ser
2 //decidido pelo compilador
3 int a = 0;
4 a = a + 1;
5 lcdNumber( a );   //imprime o valor 1
6 lcdNumber( &a );  //imprime o endereço de a (por exemplo 17821)
```

O ponteiro é uma variável que, geralmente, possui o mesmo tamanho de uma variável do tipo inteiro. Sendo assim, obedece as mesmas faixas de valores.

A diferença entre um ponteiro e um inteiro é o significado do valor armazenado. No inteiro o valor armazenado representa um número, uma quantidade que será utilizada para algum cálculo. Já no ponteiro o valor armazenado representa uma posição de endereço na memória.

Através do ponteiro é possível manipular o conteúdo de outra variável. A variável que será manipulada é definida pelo valor que está dentro do ponteiro. Para definir um ponteiro é preciso indicar ao compilador um tipo. Neste caso, o tipo não define o tamanho do ponteiro, mas sim o tamanho da variável que o ponteiro apontará.

A sintaxe para a declaração de um ponteiro é dada por:

```
<tipo> * nomeDaVariável;
```

Onde:	`<tipo>` Indica o tipo de variável que será apontado pelo ponteiro. `nomeDaVariavel` É um identificador para a variável criada, deve ser único.

Por exemplo, para se criar dois ponteiros, um apontando para um endereço onde existe um valor inteiro e outro para um endereço onde exite um valor float, podemos escrever:

```
1 int *ptrInt;
2 float *ptrFloat;
```

Deve-se tomar cuidado pois neste exemplo **apint** e **apfloat** são variáveis que armazenam endereços de memória, e não os valores do tipo **int** ou **float**. O lugar apontado pela variável **apint** é que armazena um inteiro, do mesmo modo que o lugar apontado por **apfloat** armazena um valor fracionário.

Assim, o ponteiro possui dois modos distintos de uso. Podemos manipular o valor da variável apontada pelo ponteiro ou podemos manipular o valor do próprio ponteiro. Isto significa manipular o endereço que o ponteiro armazena.

Se quisermos manipular o valor do endereço, utilizamos o nome do ponteiro normalmente, mas, se quisermos manipular o valor que está sendo apontado por ele, devemos usar um asterisco antes do nome da variável.

```
1 ptrInt = 1024;
2 *ptrInt = 254;
```

A primeira instrução indica ao compilador que queremos que o ponteiro **ptrInt** receba o valor 1024. Isto vai modificar o valor do próprio ponteiro. Depois dessa instrução o ponteiro passa a apontar para o endereço de memória 1024.

A segunda instrução armazena o valor 254 na região de memória apontada pelo ponteiro **ptrInt**. Como o ponteiro apontava para o endereço 1024, o valor 254 será armazenado lá. O valor 1024 dentro do ponteiro não será modificado pois o utilizamos apenas para servir de indicador.

Para trabalhar com ponteiros é preciso ter cuidado. Toda variável em linguagem C, ao ser criada, já possui um valor. Este valor pode ser resultado de operações antigas, sendo comumente chamado de lixo.

Ao ser definido, um ponteiro possui lixo como valor de endereço. Se tentarmos usar o ponteiro sem armazenar um valor correto, corremos o risco de manipular um valor que seja importante para o funcionamento do sistema e que, a princípio, não poderíamos ter alterado. Neste caso, podemos provocar danos aos dados, ao código do programa, ou mesmo travar a placa.

É necessário tomar cuidado ao inicializar os ponteiros. O valor atribuído a eles deve ser de um endereço disponível na memória.

Um exemplo bastante comum é criar um ponteiro que aponta para o endereço de uma variável já definida:

```
1 //definindo a variável ivar
2 int ivar;
3
4 //definindo o ponteiro iptr
5 int *iptr;
6
7 //o ponteiro iptr recebe o valor do endereço da variável ivar
8 iptr = &ivar;
9
10 // as próximas linhas são equivalentes
11 ivar = 421;
12 *iptr = 421;
```

Com sistemas embarcados, existem alguns endereços de memória que possuem características especiais. Estes endereços possuem registros de configuração, interfaces com o meio externo e variáveis importantes para o projetista. Eles são definidos no momento em que o microprocessador ou microcontrolador é projetado, portanto são fixos e dependentes da arquitetura utilizada. Através dos ponteiros é possível acessar esses endereços de maneira bastante simples, mesmo em linguagem C.

4.6 Exercícios

Ex. 4.1 — Quais são os tipos de dados básicos que podem armazenar valor na linguagem C?

Ex. 4.2 — Como os tipos das variáveis podem ser modificados?

Ex. 4.3 — Qual o impacto de uma variável com o modificador volatile?

Ex. 4.4 — É possível criar uma variável com os modificadores volatile e static ao mesmo tempo? Qual é o comportamento dessa variável?

Ex. 4.5 — Assinale os nomes de variáveis válidos na linguagem C:

( ) valor
( ) salario-liquido
( ) _b248
( ) (x2)
( ) nota*do*aluno
( ) a1b2c3
( ) 3 x 4
( ) Maria
( ) km/h
( ) xyz
( ) nome empresa
( ) sala_215
( ) “nota”

```
(     ) ah!
(     ) m{a}
```

Ex. 4.6 — Dê o tipo de cada um dos valores:

```
(          ) 613
(          ) -3.012 x 10
(          ) 613.0
(          ) "verdadeiro"
(          ) -613
(          ) 28.3 x 10-33
(          ) "613"
(          ) "constante"
(          ) falso
(          ) 21
(          ) 0,21
(          ) 17 x 1012
(          ) 15f
(          ) 613u
```

Ex. 4.7 — Supondo que as variáveis NB, NA, NMAT e SX sejam utilizadas para armazenar a nota do aluno, o nome do aluno, o número da matrícula e o sexo, declare-as corretamente, associando o tipo adequado ao dado que será armazenado.

Ex. 4.8 — Sendo P, Q, R e S variáveis cujos conteúdos são iguais a 2, 3, 12 e 4.5, respectivamente, quais os valores fornecidos pelas expressões aritméticas abaixo?

```
100 * Q / P + R = (          )
P + R % 5 - Q / 2 = (          )
R % (P+1) - Q * R = (          )
1 + (R + Q) / Q = (          )
2 * S % 3 + S = (          )
((20 / 3) / 3) + 2 = (          )
1 + (P*3+2*R)*(1/5) = (          )
2 + (R*10) / (1/S) = (          )
```

Ex. 4.9 — Sendo **soma**, **num** e **X** variáveis numéricas (**int**); **nome**, **cor**, **dia**, **tudo** variáveis literais (**char[]**), assinale os comandos de atribuição inválidos:

```
(     ) num = 5;
(     ) soma = num + 2 * X; (     ) tudo = soma;
(     ) cor = "Azul" - X; (     ) X = X + 1;
(     ) nome = "*ABC*";
(     ) dia = "Segunda";
(     ) soma + 2 = X * 2 - num;
```

Ex. 4.10 — Quais os valores armazenados em **soma** e **nome**, supondo-se que **num**, **x** e **dia** valem, respectivamente, 5, 2.5 , "Azul", "Terça"?

```
nome = dia;
soma = num * 2 / x + (1/x);
```

Ex. 4.11 — Dê o valor da variável **resultado** após a execução da seguinte sequência de operações (suponha que todas as variáveis sejam do tipo inteiro):

```
resultado = 3.0 * 6;
X = 2.0;
Y = 3.0;
resultado = X * Y - X;
resultado = 4;
X = 2;
resultado = resultado * X;
```

Ex. 4.12 — Suponha que **A**, **B**, **C**, **I**, **J** e **K** sejam variáveis do tipo **double**. Dados **A** = 4.0, **B** = 6.0 e **I** = 3, qual seria o valor final dos seguintes comandos de atribuição?

```
C = A * B - I;
K = I / 4 * 6;
C = B / A + 1.5;
```

Ex. 4.13 — Faça um programa que converta libras para newtons, sendo que há 4.9 newtons em uma libra.

Ex. 4.14 — Faça um programa que converta pés para metros, sendo que há 3.28 pés em um metro.

Ex. 4.15 — Faça um programa que converta milhas para quilômetros, sendo que há 1.61 quilômetros em uma milha.

Ex. 4.16 — Escreva um programa que, dado um número do tipo **double**, consiga separar a parte inteira da parte fracionária. Exemplo: Número: 237.48, Parte inteira: 237, Parte fracionária: 0.48

Ex. 4.17 — Escreva um programa em C que, dados dois números inteiros, imprima:
- raiz quadrada da soma dos números;
- resto da divisão do primeiro pelo segundo.

Ex. 4.18 — Escreva um programa que calcule a média aritmética de todos os números entre 13 e 25.

Ex. 4.19 — Uma revendedora de carros usados paga a seus funcionários vendedores, um salário fixo por mês, uma comissão, também fixa, para cada carro vendido e mais 5% do valor das vendas por eles efetuadas. Escreva um programa que calcule o salário de um funcionário que vendeu 5 carros de R$19.999,00, 2 de R$28.750,00 e 1 de R$46.700,00. O salário fixo é de R$1.500,00 e a comissão fixa é de R$99,95.

Ex. 4.20 — Escreva um programa para converter uma temperatura de graus Celsius para graus Fahrenheit, sendo que: $C = (F - 32)/18$

Ex. 4.21 — Escreva um programa que calcule quantas horas, minutos e segundos existem no intervalo de 67.702.556 segundos.

Ex. 4.22 — Escreva um algoritmo que, para uma determinada quantia em reais, calcule o menor número possível de notas de 100, 50, 10, 5 ou 1 em que o valor pode ser decomposto.

CAPÍTULO

5

Estruturas compostas

5.1 Estruturas homogêneas
- Vetor de char x Strings
- Matrizes

5.2 Estruturas heterogênas

5.3 Bit fields

5.4 Enumeradores

5.5 Definições de tipo

5.6 Exercícios

"Parece que alcançamos os limites do que é possível alcançar com a tecnologia do computador, embora se deva ter cuidado com tal declaração, já que elas tendem a soar muito tolas em 5 anos."

John von Neumann

5.1 Estruturas homogêneas

Um **vetor** é uma sequência de objetos do mesmo tipo. Os objetos são chamados elementos do vetor e são numerados consecutivamente a partir do zero (0, 1, 2, 3, . . .). Os números dos elementos do vetor são denominados valores índices. Os números localizam a posição do elemento dentro do vetor, possibilitando acesso direto ao vetor.

As variáveis que serão utilizadas para montagem do vetor podem ser de qualquer tipo de dado da linguagem C, incluindo estruturas definidas pelo usuário. A única restrição é que todas as posições possuam o mesmo tipo. Estas variáveis são criadas automaticamente quando o vetor é definido.

Se o nome do vetor é **v**, então **v[0]** é o nome do elemento que está na posição 0, **v[1]** é o nome do elemento que está na posição 1 e assim por diante.

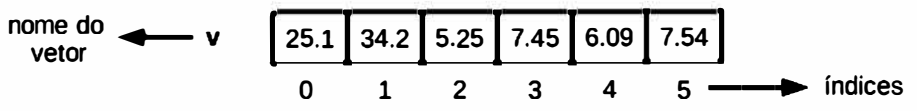


Figura 5.1: Exemplo de variável do tipo vetor

Assim como qualquer tipo de variável, deve-se declarar um vetor antes de utilizá-lo. A declaração do vetor é similar à declaração de outros tipos de dado. A única diferença é que devemos informar também a quantidade de elementos do vetor. Essa quantidade é também chamada de tamanho ou comprimento.

Para indicar o tamanho do vetor utilizamos números positivos e inteiros entre colchetes, após o nome do vetor.

A sintaxe para declaração de um vetor é:

```
<tipo> nomeVetor[numeroDeElementos];
```

Onde:

`tipo` É o tipo do dado de cada posição do vetor.

`nomeVetor` É um identificador para o vetor criado, deve ser único.

`numeroDeElementos` É a quantidade de elementos que o vetor consegue armazenar.

O tamanho do vetor deve ser uma constante inteira positiva. Não se podem utilizar outras variáveis como indiciativo do tamanho do vetor. É possível, no entanto, fazer uso de macros definidas com a diretiva **`#define`**.

Por exemplo, um vetor de 10 elementos do tipo **`int`** pode ser declarado como apresentado na figura 5.2:

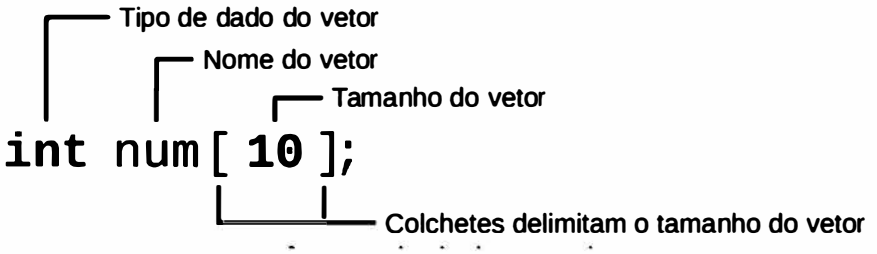


Figura 5.2: Sintaxe da declaração de um vetor

Essa declaração indica ao compilador que ele deve reservar espaço suficiente na memória para armazenar 10 elementos do tipo inteiro. Além do espaço de cada um dos elementos, o compilador também reserva espaço para um ponteiro que apontará para o primeiro elemento do vetor.

Grande parte da utilidade de um vetor provém do fato de ser possível acessar os elementos do vetor de forma individual, utilizando números para indicar a posição ou até mesmo variáveis inteiras. A figura 5.3 apresenta este conceito:

```
int idade[6];
```

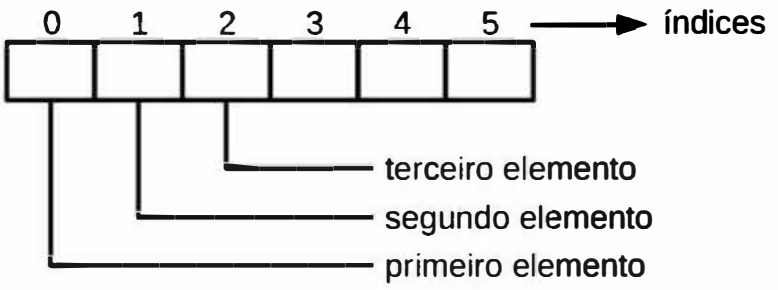


Figura 5.3: Acesso a elementos do vetor

Os elementos de um mesmo vetor são sempre armazenados em blocos contínuos na memória, de modo que o elemento **1** está localizado imediatamente depois do elemento **0** e antes do elemento **2**.

O código abaixo cria dois vetores. O vetor **num** consegue armazenar até cinco elementos do tipo inteiro. O vetor **codigo** armazena até cinco elementos do tipo **char**:

```
1 int num[5];
2 char codigo[5];
```

Se olharmos o posicionamento destes vetores na memória, teremos algo parecido com a figura 5.4:

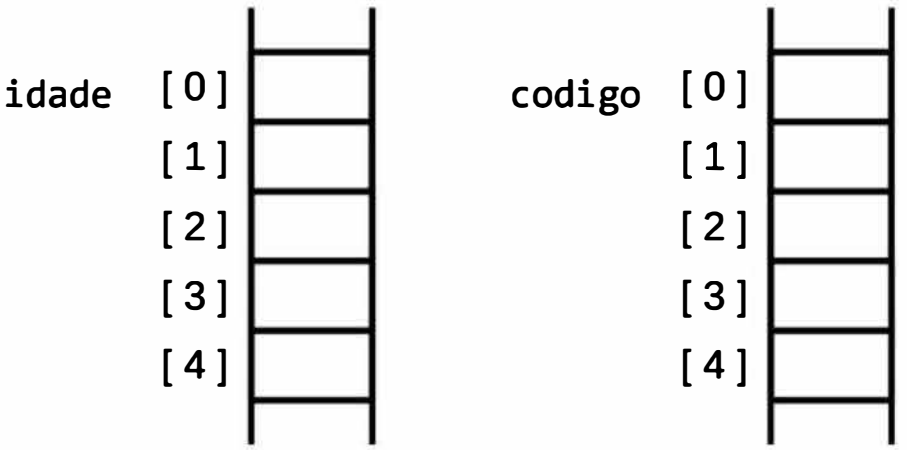


Figura 5.4: Posicionamento dos vetores na memória

Como todas as variáveis, cada uma das posições dos vetores começam com valores não definidos. Deste modo, para evitar fazer alguma conta ou processamento com esses valores

sem significado é importante atribuir valores a cada elemento do vetor. Isto pode ser feito da seguinte maneira:

```
1 nomeVetor[posicao] = novoValor;
```

Para inicializar o vetor idade podemos escrever, por exemplo, o seguinte código:

```
1 num[0] = 10;
2 num[1] = 20;
3 num[2] = 30;
4 num[3] = 40;
5 num[4] = 50;
```

A primeira sentença atribui o valor **10** à variável **num[0]** e o valor **20** à variável **num[1]**. Este método não é eficaz quando o vetor contém muitos elementos.

Outro modo de se inicializar os valores de um vetor é durante sua definição. Deste modo, é possível passar todos os valores para o vetor em uma única linha de código. A lista de valores são escritos entre chaves e separados por vírgulas, como apresentado na figura 5.5:

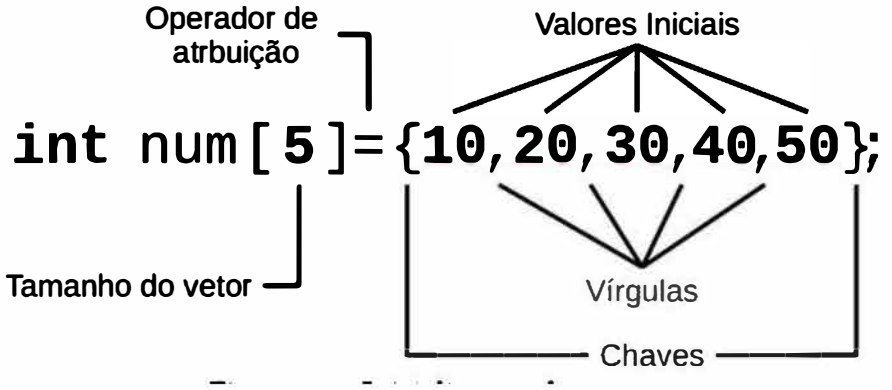


Figura 5.5: Inicialização de vetores

Quando os elementos do vetor são inicializados deste modo, não é preciso indicar o tamanho do vetor. O compilador contará o número de variáveis que foram digitadas entre as chaves e criará um vetor do tamanho adequado.

Exemplos:

```
1 //vetor de 5 elementos
2 int num[5] = {10, 20, 30, 40, 50};
3 //vetor de 3 elementos
4 int n[ ] = {3, 4, 5};
5 //vetor de 4 elementos
6 float k[ ] = {2.5, 2.1, 7.5, 5.6};
```

Vetor de char x Strings

O vetor de **char** é um tipo muito utilizado para armazenar um texto. Nesta situação cada letra do texto será armazenada numa posição do vetor.

Um problema com esse tipo de vetor é saber quanto o texto termina. A convenção mais utilizada é inserir o código **'\0'** como delimitador de fim de texto. Esse código é representado pelo número 0 (zero) na codificação ASCII, sendo também conhecido como terminador.

O tamanho de posições em um vetor de **char** que armazenará uma string deve ser uma unidade maior que a quantidade de letras que se deseja armazenar, sendo que a última posição possui o elemento **'\0'**.

A linguagem C define um modelo de inicialização para permitir que os textos sejam facilmente criados e armazenados num vetor de **char**. Para isso basta criar um vetor de **char** com o tamanho adequado e escrever o texto entre aspas duplas.

```
char nome[tamanho] = "texto";
```

Onde:

nome	É um identificador para o vetor criado, deve ser único.
tamanho	Indica a quantidade de elementos do vetor. O tamanho deve ser uma unidade maior que a quantidade de letras.
texto	É o texto a ser armazenado no vetor. Se o tamanho for omitido na declaração, o compilador utiliza esse texto para definir o tamanho.

Por estar entre aspas duplas, o compilador automaticamente insere o terminador no final do vetor. No código abaixo, por exemplo, existem 6 letras e 1 espaço, totalizando 7 caracteres. Inserindo o terminador **'\0'**, o tamanho final do vetor **msg** é de 8 caracteres.

```
1 char msg[] = "Bom dia"; //msg possui tamanho 8
```

Matrizes

As matrizes são variáveis muito semelhantes aos vetores, possuindo um conjunto de elementos do mesmo tipo armazenados sequencialmente na memória. A diferença é que as matrizes podem possuir mais de um índice. Cada índice controla uma dimensão da matriz. Os vetores podem ser entendidos então como matrizes unidimensionais, pois possuem apenas um índice e sua representação gráfica acontece ao longo de uma linha.

As matrizes bidimensionais se assemelham a tabelas, de modo que podemos interpretar os índices como suas linhas e colunas. Este formato de tabela é apenas uma interpretação, pois, na memória, os dados são alocados sequencialmente, do mesmo modo que o vetor.

A declaração da matriz segue o mesmo formato da declaração de um vetor, com números inteiros positivos, um para cada dimensão. Devemos tomar cuidado, pois uma simples matriz de 10 linhas por 10 colunas de números flutuantes consome 400 bytes, o que pode representar quase 25% da memória de um microcontrolador mais simples.

Por exemplo, uma variável do tipo matriz de duas dimensões pode ser declarada como:

```
<tipo> nomeMatriz[tamanhoL][tamanhoC];
```

Onde:

tipo	É o tipo de dado de cada posição da matriz.
nome	É um identificador para o vetor criado, deve ser único.
tamanhoL	Indica o tamanho da primeira dimensão da matriz (número de linhas).
tamanhoC	Indica o tamanho da segunda dimensão da matriz (número de colunas).

Da mesma maneira como ocorre com os vetores, os índices começam sempre em 0 (zero).

No exemplo abaixo, o código cria uma matriz chamada **a** contendo duas linhas (zero e 1) e 6 colunas (zero a 5), capaz de armazenar até dez números inteiros.

```
1 int a[2][6];
```

Representando essa matriz de modo gráfico, como uma tabela de duas linhas e seis colunas, temos a figura 5.6:

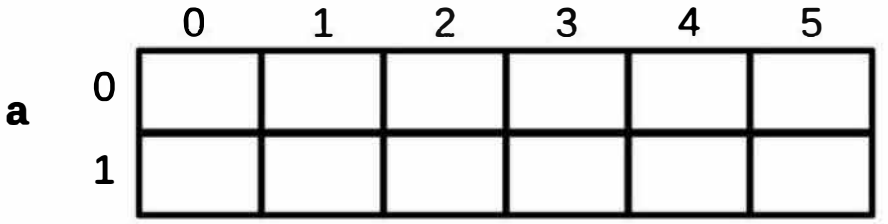


Figura 5.6: Matriz 2x6

Para armazenar valores na matriz basta indicar a linha e coluna em que queremos que o valor seja salvo.

```
1 a[1][4] = 10;
```

A figura 5.7 apresenta o estado da matriz após a inserção do valor 10 na linha 1, coluna 4:

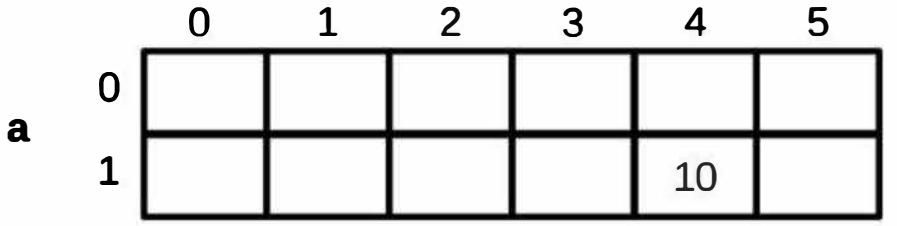


Figura 5.7: Atribuindo valores à matriz

As posições das matrizes que não forem inicializadas não estão vazias; na verdade, elas possuem valores que são resultado de operações anteriores. Estes valores não tem nenhum significado, sendo comumente chamados de lixo.

As matrizes podem ser inicializadas quando são declaradas. A inicialização consta de uma lista de constantes separadas por vírgulas do mesmo modo que na inicialização dos vetores.

Para efeitos de organização visual, pode-se separar cada linha com um par de chaves extras, inclusive colocando-as em linhas diferentes separadas por vírgulas.

Este último método permite apresentar os dados na inicialização com uma apresentação gráfica muito similar a uma tabela, facilitando a visualização dos dados:

```
1 //inicialização por lista contínua
2 int mat[2][3] =  {51, 52, 53,   54, 55, 56};
3 //inicialização por linhas
4 int mat[2][3] = {{51, 52, 53}, {54, 55, 56}};
5 //inicialização por linhas e com quebra de linha
6 int mat[2][3] = {{51, 52, 53},
7                  {54, 55, 56}};
```

Após a definição da variável e inicialização dos dados a matriz estará armazenada na memória conforme apresentado na figura 5.8:

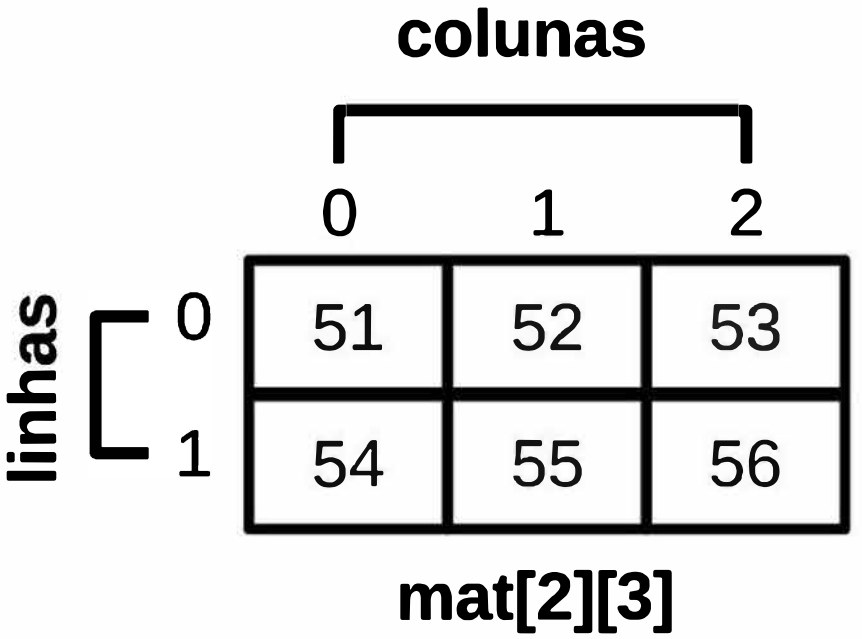


Figura 5.8: Matriz 2x3 inicializada

5.2 Estruturas heterogênas

Os vetores são estruturas de dados que contêm um número definido de elementos e todos eles possuem o mesmo tipo de dado. O fato de todos os elementos terem o mesmo tipo de dado é uma limitação quando necessitamos armazenar um conjunto de informações com elementos de tipos diferentes de dados.

A solução para esse problema é utilizar um tipo de dado chamado estrutura. Os elementos individuais de uma estrutura são chamados de membros. Cada membro (elemento) de uma estrutura pode conter dados de um tipo diferente dos outros membros.

Uma estrutura pode conter qualquer número de membros. Cada membro tem um nome que serve de identificador. Este nome deve ser único. Ao contrário dos tipos básicos, precisamos de duas etapas para poder criar uma estrutura.

A primeira é declarar o formato da estrutura. Isto é feito através da palavra reservada **`struct`**:

```
struct nomeEstrutura{
    tipoDadoMembro_1 nomeMembro_1;
    tipoDadoMembro_2 nomeMembro_2;
    //...
    tipoDadoMembro_n nomeMembro_n;
}; //end struct
```

Onde:

`nomeEstrutura`
É nome dado ao formato de estrutura recém-criado.

`tipoDadoMembro_n`
tipo da variável n (int, char, float, etc).

`nomeMembro_n`
nome da variável n.

Após definir o formato, podemos criar variáveis com esta estrutura através da palavra reservada **`struct`**:

```
struct nomeEstrutura nomeVar;
```

Onde:

`nomeEstrutura`
Nome da estrutura utilizada.

`nomeVar` Nome da variável criada.

Tomemos como exemplo uma estrutura para armazenar os dados de uma coleção de séries de TV. As informações relevantes para essa estrutura podem ser elencadas como os dados a seguir:

Nome do Membro	Tipo de dado
Título	vetor de tamanho 20
Produção	vetor de tamanho 30
Temporada	inteiro
Preço	ponto flutuante (float)
Data da compra	vetor de tamanho 8

O formato desta estrutura pode ser transcrito para código utilizando os comandos abaixo:

```
1 struct serieTv{
2       char titulo[30];
3       char producao[20];
4       int temporada;
5       float preco;
6       char dataCompra[8];
7 };//end struct
```

5.3 Bit fields

Uma necessidade comum em sistemas embarcados é a capacidade de manipular apenas uma certa quantidade de bits ou até mesmo um único bit de modo individual. Isto pode ser feito através das operações bitwise ou da criação de estruturas com variáveis cujo tamanho seja determinado pelo programador. Caso a variável criada possua uma quantidade menor de bits que a de uma variável normal, bits de "padding" serão adicionados.

Se duas variáveis forem criadas dentro de uma estrutura, elas serão alinhadas num mesmo espaço de memória, permitindo que o programador acione cada bit, ou região de bits. Esta técnica permite que bits sejam agrupados de acordo com os registros do microcontrolador.

Na figura 5.9 é apresentada a criação de duas variáveis. A primeira, **nibble**, cria uma estrutura composta de duas subseções de 4 bits cada. A segunda, **bits**, cria uma estrutura cujos dois primeiros bits podem ser acessados individualmente. Os demais bits não são utilizados e não podem ser acessados através da estrutura.

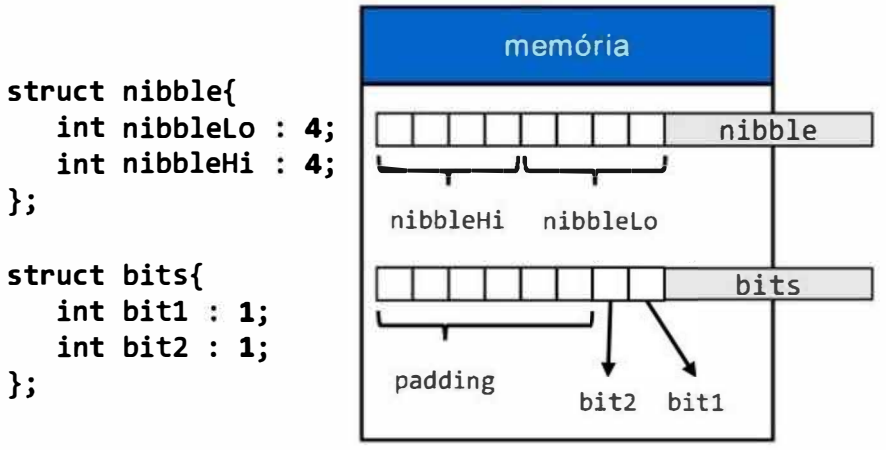


Figura 5.9: Criação de estruturas com bit fields

A sintaxe para a criação de uma estrutura com bit fields é dada por:

```
struct{
    tipo1 <nome_elemento1> : tamanho1;
    tipo2 <nome_elemento2> : tamanho2;
    tipo3 <nome_elemento2> : tamanho3;
    tipo4 <nome_elemento2> : tamanho4;
    //...
    tipoN <nome_elementoN> : tamanhoN;
};
```

Onde:

- `tipo` — Tipo base do elemento **char** ou **int**, com ou sem os modificadores de sinal.
- `<nome_elemento>` — Nome do elemento para posterior acesso pela estrutura.
- `tamanho` — Quantidade de bits reservados para o elemento.

O acesso a estas estruturas é feito do mesmo modo que as estruturas tradicionais. A diferença é que, como as variáveis possuem tamanho menor, é mais fácil acontecer um estouro, ou overflow.

```
1  struct nibble{
2      unsigned int nibbleLo : 4;
3      unsigned int nibbleHi : 4;
4  };
5
6      //criação de uma variável teste1 do tipo struct nibble
7      struct nibble teste1;
8
9      //como o nibbleLo possui apenas 4 bits, só pode contar até 15 = (2^4-1)
10     teste1.nibbleLo = 15;
11
12     teste1.nibbleLo++;
13     //como a variável estourou seu valor voltou a zero. No entanto a variável ↩
           nibbleHi não será alterada
14
15     //o LCD vai imprimir o número zero
16     lcdNumber(teste1.nibbleLo);
```

5.4 Enumeradores

Enumeradores são definições da linguagem C que visam facilitar a criação de referências textuais.

```
enum{LABEL1 = VALOR1,
     LABEL2 = VALOR2,
     //...
     LABELN = VALORN};
```

Onde:		
	LABEL	Nome textual que será utilizado para referência do valor.
	VALOR	especificação do valor numérico do label, é opcional.

Caso os valores sejam omitidos, a estrutura automaticamente incrementa uma unidade para cada elemento do **enum**. O código abaixo apresenta um exemplo de enumerador e sua correspondência com um conjunto de **#defines**.

```
1 enum{AZUL = 1,
2      VERDE, //como não foi definido valor, VERDE = AZUL+1
3      VERMELHO = 4};
4
5 //alternativa com defines
6 #define AZUL 1
7 #define VERDE 2
8 #define VERMELHO 4
```

5.5 Definições de tipo

Os tipos básicos da linguagem C nem sempre são suficientes para que possamos descrever as informações de aplicações reais, principalmente quando tratamos de estruturas mais complexas.

Nestes casos, podemos definir nossos próprios tipos. A declaração de um tipo novo é feita pela diretiva **typedef**, seguida do tipo base e do novo nome.

```
typedef <tipo> novoTipo;
```

Onde:		
	<tipo>	É o tipo base a ser utilizado na definição.
	novoTipo	É o nome dado ao novo tipo.

A linguagem C não possui a definição para o tipo **byte**. Podemos definí-lo como um **unsigned char**. Ou definir que o tipo **dinheiro** é baseado no tipo float.

```
1 typedef unsigned char byte;
2 typedef float dinheiro;
```

Estas definições ajudam a organizar o código e facilitam a portabilidade. O valor float, por exemplo, pode possuir 4 bytes numa arquitetura, mas em outra valer 8. Arrumando apenas a definição da variável dinheiro, garantimos que em todos os lugares em que esse tipo for utilizado as variáveis sejam alteradas corretamente.

Uma das grandes vantagem do **typedef** é criar novos tipos em conjunto com as diretivas **enum**, **struct** ou **union**.

Exemplo da criação do tipo bool:

```
1 //definição do tipo bool
2 typedef enum {false, true} bool;
3
4 //criação de uma variável do tipo bool
5 bool condition;
```

Para o **bool**, a palavra **false** passa a valer zero por estar na primeira posição. Já a palavra **true** passa a ter valor um. No entanto, as variáveis criadas com o tipo **bool** não possuem apenas 1 bit de tamanho. Elas possuem, em geral, o mesmo tamanho de um inteiro, que dependendo da plataforma pode ser 8, 16, 32 ou 64 bits.

Outro tipo comum de tipo para ser definido é a porta. Por porta entendemos um modo de acesso do microcontrolador ao mundo externo. Em geral, as portas representam através de bits os terminais físicos. Nestas situações, escrever o valor 1 (um) em um bit faz com que um determinado dispositivo ligue. Escrever o valor 0 (zero), por sua vez, faz com que ele desligue. O código 5.1 apresenta essa definição.

Código 5.1: Criação de uma struct para acesso individual a cada bit de uma porta

```
 1 typedef struct  {
 2     unsigned bit0:1 ;
 3     unsigned bit1:1 ;
 4     unsigned bit2:1 ;
 5     unsigned bit3:1 ;
 6     unsigned bit4:1 ;
 7     unsigned bit5:1 ;
 8     unsigned bit6:1 ;
 9     unsigned bit7:1 ;
10 } port;
11
12 //criação de uma variável do tipo struct port
13 port A;
14 //liga dispositivo da porta A, bit 0
15 A.bit0 = 1;
16 //desliga dispositivo da porta A, bit 0
17 A.bit0 = 0;
```

5.6 Exercícios

Ex. 5.1 — Qual a diferença entre um vetor de inteiros com 3 posições e uma estrutura com 3 membros inteiros?

```
1 int vetor[3];
2 struct triplo{
3     int v0;
4     int v1;
5     int v2;
6 };
```

Ex. 5.2 — Defina o tipo **`talher`**. Ele é descrito por um enumerador com 3 valores: colher, garfo e faca.

Ex. 5.3 — Crie uma estrutura composta por três membros de 2 bits e dois membros de 2 bits.

Ex. 5.4 — Construa uma matriz de tamanho 10 x 10 do tipo **`pontos`**. Os pontos são definidos por uma estrutura com dois inteiros: **`x`** e **`y`**.

CAPÍTULO

6

Operações binárias

6.1 Álgebra booleana
- Operação NÃO
- Operação E
- Operação OU
- Operação OU EXCLUSIVO

6.2 Operações binárias (bitwise)
- Bitwise NÃO
- Bitwise E
- Bitwise OU
- Bitwise OU EXCLUSIVO

6.3 Operação de deslocamento
- Shift circular ou rotacionamento de bits

6.4 Manipulando apenas 1 bit de cada vez
- Criando funções através de define's

6.5 Exercícios

"A virtude do binário é ser o modo mais simples de representar números. Qualquer outra coisa é mais complicada. Você pode pegar erros com ele, é inequívoco em sua leitura, há muitas coisas boas sobre o binário."

George M. Whitesides

Na programação de sistemas embarcados, algumas posições de memória servem para diferentes propósitos, não apenas para armazenar valores. Em algumas destas memórias, cada um dos bits possui um significado diferente, fazendo com que seja necessário conseguir manipulá-los individualmente ou em pequenos grupos.

Para efetuar essas operações binárias utilizamos conceitos específicos da lógica booleana implementados na linguagem C.

Este capítulo fará uma introdução à lógica booleana, bem como ao processo de manipulação de bits em linguagem C.

6.1 Álgebra booleana

A álgebra booleana é a porção da matemática que nos permite descrever circuitos lógicos. Por circuitos lógicos entendemos qualquer sistema que processe sinais cujos valores possuem apenas dois estados.

Estes sinais não representam necessariamente números, mas estados, como o estado lógico falso/verdadeiro, baixo/alto, zero/um. Uma variável booleana mantém um estado lógico, e não necessariamente um valor numérico.

Para a linguagem C toda variável que possui valor zero é considerada **`falsa`**. Toda variável que possui valor diferente de zero é **`verdadeira`**.

Por exemplo, um circuito elétrico composto por uma fonte, chaves mecânicas e lâmpadas. A fonte fornece energia, sendo a responsável por permitir que o sistema funcione. Ela garante o estado lógico verdadeiro. Para as chaves podemos convencionar que: quando ela estiver pressionada seu estado é verdadeiro, se estiver solta o estado lógico é falso. Para a lâmpada, se estiver acesa consideramos como verdadeiro e se estiver apagada o estado é falso.

A lógica de acendimento da lâmpada depende de como a fonte, as chaves e a lâmpada foram ligadas e das regras que utilizamos para definir o funcionamento do circuito.

Nos próximos exemplos utilizaremos como regras as leis da eletricidade básica, onde a corrente sai do positivo da fonte e caminha para o negativo, conforme apresentado na figura 6.1:

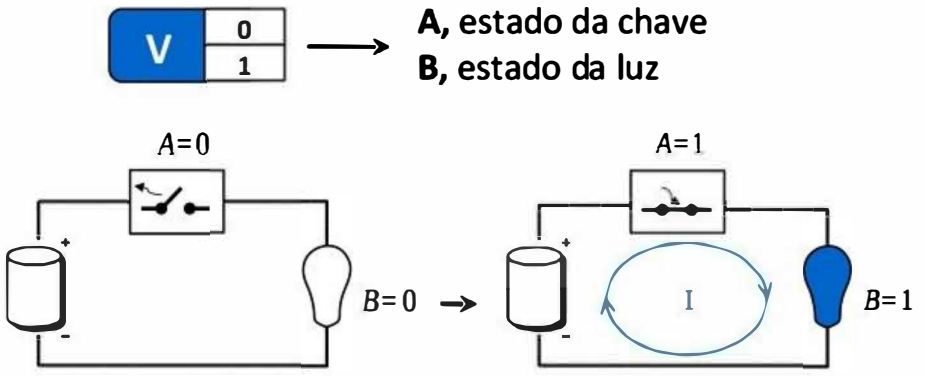


Figura 6.1: Chave alterando estado da lâmpada

Neste caso, podemos interpretar o estado da chave e o estado da lâmpada como variáveis binárias. Se a chave estiver solta, ou a lâmpada apagada, podemos considerar que elas estão com o estado zero. Se ela estiver pressionada ou a lâmpada acesa, o estado será interpretado como um.

Uma das diferenças entre a álgebra booleana e a álgebra que conhecemos são as operações. Para a álgebra comum, temos a soma, subtração, multiplicação etc. Para a álgebra booleana, são definidas três operações básicas: a operação OU, E e NÃO. Em circuitos lógicos, representamos essas operações através de símbolos que são conhecidos como portas lógicas. Estes símbolos estão apresentados na figura 6.2:

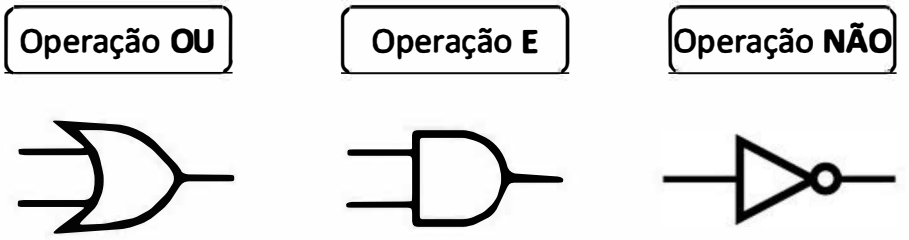


Figura 6.2: Operações binárias

As portas lógicas podem ser encadeadas para formar outros circuitos, que possuem funcionamento diferente. Algumas destas combinações formam circuitos bastante úteis e que são utilizados em diversos lugares.

Um exemplo é a formação da porta OU EXCLUSIVA. A figura 6.3 apresenta o circuito equivalente utilizado para implementar essa porta. Sua representação gráfica é similar à porta OU, com um traço extra.

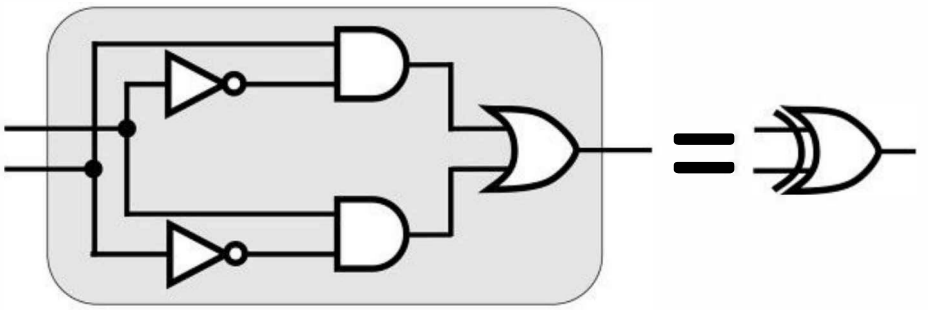


Figura 6.3: Circuito equivalente de uma porta OU EXCLUSIVA

O modo de operação de uma porta lógica, ou de um circuito lógico, pode ser representado através de uma tabela. Essa tabela é conhecida como tabela verdade e indica qual é a saída de um circuito para cada combinação de valores das entradas.

A figura 6.4 apresenta como transcrever um circuito digital para notação matemática e para tabela verdade. A caixa representa o circuito digital: qualquer combinação das entradas através de portas lógicas resultando em uma saída de informação.

A notação matemática faz uso da lógica booleana. Nesta lógica, as operações do tipo OU são representadas pelo símbolo de soma: "$A + B$". As operações do tipo E são representadas por um ponto, representando o símbolo de multiplicação: "$A \cdot B$".

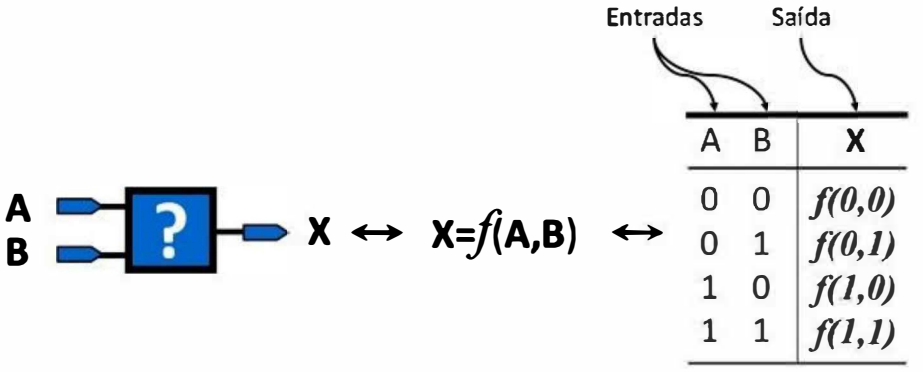


Figura 6.4: Transcrição de um circuito digital para tabela verdade

As operações do tipo negação são representadas por um traço em cima da variável ou operação a ser negada: "A".

A tabela verdade é uma ferramenta bastante simples de ser utilizada. Dados os valores de entrada para as variáveis, ela apresenta o valor esperado na saída.

Operação NÃO

A operação de negação é a operação mais simples da lógica digital. Ela funciona negando a informação de entrada. Se na entrada o estado for verdadeiro, na saída da porta o estado passa a ser falso.

Uma utilidade desta operação é indicar a execução de uma atividade apenas quando uma premissa for falsa. Um exemplo seria: "Se não estiver chovendo, vou sair de bicicleta". A ação de sair de bicicleta só será realizada se a premissa "estiver chovendo" for falsa.

Utilizando termos numéricos, a operação de negação inverte os valores em uma variável binária. Os bits que valem 1 passam a valer 0 (zero) e os bits que valem 0 (zero) passam a valer 1.

Outro modo de visualizar esta operação é através de um circuito com uma chave em paralelo com a lâmpada, conforme a figura 6.5:

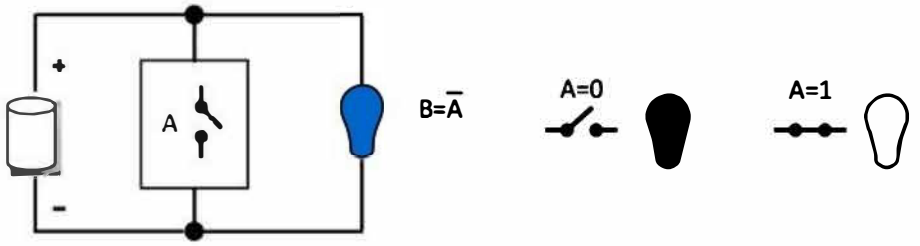


Figura 6.5: Chave funcionando como operação binária *não*

Se a chave A estiver aberta (estado zero), a lâmpada está ligada (estado 1). Se a chave estiver fechada (estado 1), o curto-circuito fará com que a lâmpada apague. Este é o funcionamento de uma operação lógica de negação. A tabela 6.1 apresenta este comportamento.

Tabela 6.1: Tabela verdade porta NÃO

A	*!A*
0	1
1	0

Na linguagem C de programação, a operação lógica de negação é feita pelo operador exclamação: "**!**". Lembrando que na linguagem C uma variável com valor zero representa falso e uma variável com valor diferente de zero representa verdadeiro.

```
1 char ini = 12;
2 char result;
3 // ini = 0b00001100
4 result = ! ini;
5 // result = 0
6 result = !(! ini);
7 // result = 1
```

A variável **ini** possui o valor **12** que, por ser diferente de zero, é interpretado como verdadeiro. A operação de negação transforma esse valor em falso, ou seja, a variável **result** terá o valor zero. Se duas operações de negação forem feitas em sequência, a primeira operação transformará o valor em zero e a segunda operação transformará o valor em 1.

O valor volta a ser verdadeiro, porém diferente do valor original. A utilização de duas operações de negação em sequência é bastante utilizada para garantir que a informação original fique restrita aos valores zero e um.

Operação E

A operação lógica E toma uma decisão baseada em duas informações de entrada. Esta operação lógica retorna um resultado verdadeiro apenas quando ambas as entradas são verdadeiras. Se alguma das entradas for falsa, a saída é falsa.

Esta operação é utilizada quando precisamos de duas premissas para poder realizar uma ação. Um exemplo seria: "Se eu tiver dinheiro E estiver com fome, comprarei um lanche". A ação de comprar um lanche só será realizada se ambas as premissas: "tiver dinheiro" e "estiver com fome" forem verdadeiras. Do ponto de vista matemático, seu funcionamento se assemelha à operação de multiplicação, onde, se algum dos valores for igual a zero, o resultado é zero. Por esse motivo, pode ser representada pelo sinal de multiplicação. A tabela 6.2 apresenta o comportamento desta operação.

Tabela 6.2: Tabela verdade porta E

A	*B*	*A&&B*
0	0	0
0	1	0
1	0	0
1	1	1

Esta operação pode ainda ser interpretada como duas chaves ligadas em série com uma lâmpada num circuito fechado, como demonstrado pela figura 6.6:

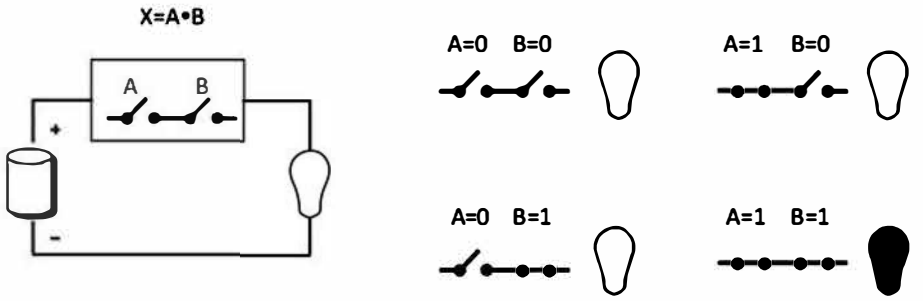


Figura 6.6: Chave funcionando como operação binária *E*

A lâmpada só estará ligada se ambas as chaves forem pressionadas ao mesmo tempo. Se apenas uma delas estiver pressionada, a corrente elétrica não terá um caminho fechado para circular, fazendo com que a lâmpada permaneça desligada.

Na linguagem C utiliza-se o operador **&&**, dois E comerciais, para realizar a operação lógica E. Nesta operação, os valores das variáveis **A** e **B** são avaliados como verdadeiros, diferentes de zero, ou falsos, iguais a zero.

```
1 char A = 8;
2 // A = 0b00001000
3 char B = 5;
4 // B = 0b00000101
5 result = A && B;
6 // result = 1
```

A variável **result** recebe o valor 1 como resultado da operação, pois as variáveis **A** e **B** são verdadeiras, ou seja, possuem valor diferente de zero.

Operação OU

A operação lógica OU trabalha com dois operadores de entrada. A saída dessa operação só apresentará o estado falso se ambas as entradas forem falsas. Se pelo menos uma das entradas for verdadeira, o estado da saída será verdadeiro. Isto é representado pela tabela 6.3.

Tabela 6.3: Tabela verdade porta OU

A	B	A | | B
0	0	0
0	1	1
1	0	1

1	1	1

O circuito equivalente que executa a mesma funcionalidade da operação lógica OU pode ser implementado com duas chaves em paralelo para acionar uma lâmpada, conforme mostrado pela figura 6.7:

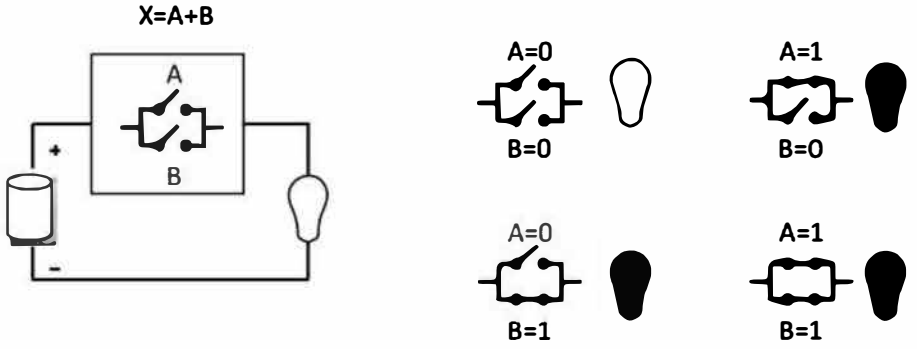


Figura 6.7: Chave funcionando como operação binária *OU*

Se qualquer uma delas estiver pressionada, o caminho entre a bateria e a lâmpada é fechado e a corrente que circula faz com que a lâmpada acenda. Se ambas as chaves estiverem pressionadas a lâmpada continua acesa, pois não há interferência de uma chave na segunda. Apenas quando ambas as chaves estão desligadas é que a lâmpada fica apagada. O operador utilizado é o `||`, duas barras paralelas, também conhecido como pipe. Assim como os demais operadores binários, o resultado da operação só pode ser os valores zero ou um.

```
1 char A = 8;
2 // A = 0b00001000
3 char B = 5;
4 // B = 0b00000101
5 result = A || B;
6 // result = 1
```

Operação OU EXCLUSIVO

A operação OU EXCLUSIVO verifica se apenas uma das entradas é verdadeira. Neste caso, a saída também é verdadeira. Se ambas forem iguais a zero, ou ambas iguais a um, a saída é zero, conforme tabela 6.4.

Tabela 6.4: Tabela verdade da porta OU EXCLUSIVO

A	B	A ⊕ B
0	0	0
0	1	1
1	0	1
1	1	0

Para exemplificar este comportamento precisamos de uma chave especial. Esta chave especial é composta de duas chaves simples: uma que está sempre fechada e uma que está sempre aberta. Estas duas chaves estão conectadas fisicamente, de modo que quando uma é pressionada a outra também se move. Deste modo, a chave que estava fechada se abre e a que estava aberta se fecha.

Este tipo de chave é comum em interruptores paralelos, também conhecidos como three-way. Estes interruptores são utilizados em pares, para poder ligar ou desligar uma lâmpada em dois locais diferentes, como nas duas pontas de um corredor.

A figura 6.8 apresenta este circuito. A linha tracejada indica a conexão física entre as duas chaves que formam o interruptor A e as duas chaves que formam o interruptor B:

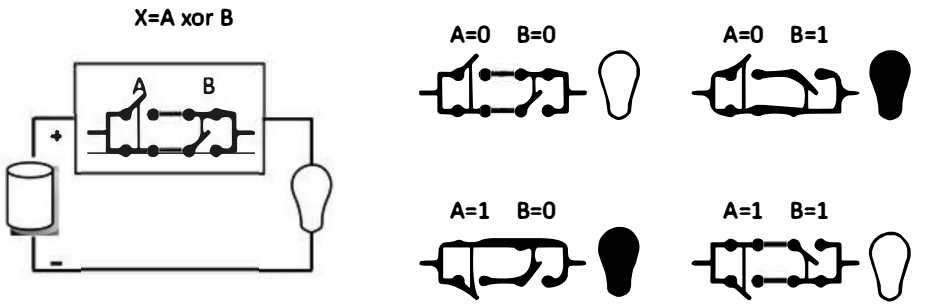


Figura 6.8: Chave funcionando como operação binária *OU EXCLUSIVO*

Quando ambas as chaves não estão pressionadas, não existe nenhum caminho para que a corrente circule da fonte até a lâmpada e esta permanece desligada. Se apenas a chave A for pressionada, o caminho de cima é completado e a corrente flui. A mesma coisa acontece se apenas a chave B for pressionada, só que, neste caso, o caminho se completa pela parte de baixo das chaves. Se ambas as chaves forem pressionadas a corrente para de circular pois o caminho é interrompido.

Para implementar essa lógica em linguagem C devemos utilizar a definição da própria operação ou exclusiva, visto que não há operador pronto.

```
1 char A = 8;
2 // A = 0b00001000
3 char B = 5;
4 // B = 0b00000101
5 result = (A && !B) || (!A && B);
6 // result = 1
```

6.2 Operações binárias (bitwise)

Nos sistemas microcontrolados todas as informações são codificadas em valores binários, mas a maioria das variáveis é composta por um conjunto de bits. Em geral, a manipulação da variável altera todos os bits ao mesmo tempo.

Existem, no entanto, algumas variáveis nas quais cada bit tem uma interpretação ou funcionalidade diferente. Por isso é necessário realizar algumas operações que modifiquem apenas um bit, mantendo os demais bits da variável inalterados.

Um exemplo são as portas de entrada e saída. O primeiro bit pode ser o responsável por controlar o acionamento de um motor para a esquerda, enquanto o segundo bit controla o acionamento para a direita. Nestes casos, é preciso conseguir ligar ou desligar cada um dos bits individualmente. No entanto, as operações lógicas estudadas até agora levam em conta o valor da variável como um todo.

A alternativa é utilizar as operações lógicas bitwise. Essas operações também trabalham com as variáveis, mas ao invés de interpretar a variável inteira como uma informação verdadeira ou falsa, elas trabalham com cada bit individualmente.

Bitwise NÃO

O operador "~" (til) executa uma operação de negação lógica. Por ser um operador bitwise a operação é realizada para cada um dos bits da variável e não para a variável como um todo. A quantidade de bits que será invertida pelo operador depende do tamanho da variável que está sendo negada.

Sua sintaxe de uso é:

```
resultado = ~variavel;
```

Onde:	**variavel** É a variável ou o valor a ser negado. **resultado** Variável que armazena o resultado da negação. O resultado pode ser utilizado sem ser armazenado numa variável.

Na tabela 6.5 é apresentada a comparação entre a operação lógica de negação e a operação bitwise de negação:

Tabela 6.5: Operações NÃO em linguagem C

Lógico

```
1 char A = 12; //0b00001100
2 char r;
3 r = !A; // r = 0
4 // A = 0b00001100
5
6 // r = 0b00000000
```

Bitwise

```
1 char A = 12; //0b00001100
2 char r;
3 r = ~A; // r = 243
4 // A = 0b00001100
5          ||||||||
6 // r = 0b11110011
```

Podemos notar que a operação lógica de negação inverte o estado da variável. Como o valor da variável **A** era diferente de zero, a variável foi interpretada como verdadeira. Assim, o valor invertido é falso, ou zero. Já a função bitwise troca todos os bits de modo individual. Assim, o valor inicial que era **12** passa a ser **243**.

Outra diferença entre as duas operações é que, se invertermos duas vezes a variável com a operação bitwise de negação, o valor volta ao seu estágio inicial, diferente da operação lógica.

Bitwise E

A operação bitwise E utiliza o operador **&**, conhecido como "&". Essa operação executa um E lógico para cada par de bits nas posições correspondentes das variáveis de entrada e armazena o resultado de cada uma das operações na variável de saída.

Sua sintaxe de uso é dada por:

```
resultado = variavel1 & variavel2;
```

Onde:	**variavelN** São as variáveis a serem operadas. **resultado** Variável que armazena o resultado da operação E. O resultado pode ser utilizado sem ser armazenado numa variável.

Na tabela 6.6 é apresentada a comparação entre a operação lógica E e sua correspondente bitwise.

Neste exemplo, ambos os valores originais são diferentes de zero. A operação lógica retorna o resultado verdadeiro, fazendo com que a variável **r** passe a ter o valor **1**. Já a operação bitwise executa o mesmo processo olhando cada um dos bits das variáveis. O bit na posição zero da variável **A** vale **0**, já o bit na posição zero da variável **B** vale **1.** Como em nenhuma posição das variáveis temos dois bits com valor **1**, em nenhuma posição do resultado haverá um valor **1**. Isto faz com que todos os bits sejam zerados no resultado final.

Tabela 6.6: Operações E em linguagem C

Lógico

```
1  char A = 8; //0b00001000
2  char B = 5; //0b00000101
3  char r;
4  r = A && B;
5  // r = 1
6  // A = 0b00001000
7
8  // B = 0b00000101
9
10 // r = 0b00000001
```

Bitwise

```
1  char A = 8; //0b00001000
2  char B = 5; //0b00000101
3  char r;
4  r = A & B;
5  // r = 0
6  // A = 0b00001000
7  ||||
8  // B = 0b00000101
9  ||||
10 // r = 0b00000000
```

Bitwise OU

O operador **|**, barra vertical ou pipe, executa uma operação do tipo OU lógica para cada par de bits das variáveis e coloca o resultado no bit correspondente.

Sua sintaxe de uso é:

```
resultado = variavel1 | variavel2;
```

Onde:

variavelN São as variáveis a serem operadas.
resultado Variável que armazena o resultado da operação OU. O resultado pode ser utilizado sem ser armazenado numa variável.

Na tabela 6.7 é apresentada a comparação entre a operação lógica OU e sua correspondente bitwise:

Tabela 6.7: Operações OU em linguagem C

Lógico

```
1  char A = 9; //0b00001001
2  char B = 3; //0b00000011
3  char r;
4  r = A || B;
5  // r = 1
6  // A = 0b00001001
7
8  // B = 0b00000011
9
10 // r = 0b00001011
```

Bitwise

```
1  char A = 9; //0b00001001
2  char B = 3; //0b00000011
3  char r;
4  r = A | B;
5  // r = 13
6  // A = 0b00001000
7                ||||
8  // B = 0b00000101
9                ||||
10 // r = 0b00001101
```

Bitwise OU EXCLUSIVO

Ao contrário das operações lógicas, a operação OU EXCLUSIVO em bitwise apresenta um operador dedicado a essa função o circunflexo (^). Ele executa uma XOR lógica para cada par de bits e coloca o resultado na posição correspondente.

Sua sintaxe de uso é:

```
resultado = variavel1 ^ variavel2;
```

Onde:

variavelN São as variáveis a serem operadas.
resultado Variável que armazena o resultado da operação OU exclusivo. O resultado pode ser utilizado sem ser armazenado numa variável.

Na tabela 6.8 é apresentada a comparação entre a operação lógica OU EXCLUSIVA e sua correspondente bitwise:

Tabela 6.8: Operações OU EXCLUSIVO em linguagem C

Lógico

```
1  char A = 12; //0b00001100
2  char B =  5; //0b00000101
3  char r;
4  r = (A && !B) || (!A && B);
5  // r = 0
6  // A = 0b00001000
7
8  // B = 0b00000101
9
10 // r = 0b00000000
```

Bitwise

```
1  char A = 12; //0b00001100
2  char B = 5;  //0b00000101
3  char r;
4  r = A ^ B;
5  // r = 9
6  // A = 0b00001100
7                |||
8  // B = 0b00000101
9                |||
10 // r = 0b00001001
```

Após a execução desta operação, as posições onde os bits das duas variáveis eram diferentes ficaram em **1**. As posições onde os bits eram iguais, ambos zero, apresentam valor **0**.

6.3 Operação de deslocamento

Outra operação com bits muito utilizada é o deslocamento dos bits dentro de uma variável. Esta é uma operação nativa dos processadores sendo representada em linguagem C pelos símbolos **»**, dois maiores, ou **«**, dois menores.

A operação de deslocamento permite que o programador controle o fluxo dos bits numa variável, sendo esse o procedimento utilizado para a criação de um sistema de comunicação serial.

As operações de shift em linguagem C podem deslocar os bits para a esquerda: operador **«**, ou direita: operador **»**. Essa operação apenas desloca os bits, ela não altera o valor da variável original. Para que o valor seja alterado e armazenado, é necessário atribuir o resultado a uma variável, que pode ser a própria variável original.

Além do operando (valor/variável original) e do operador (maior ou menor), é necessário indicar a quantidade de deslocamentos realizados. A sintaxe de uso destes operadores é:

```
//para direita
resultado = variavel >> vezes;

//para esquerda
resultado = variavel << vezes;
```

Onde:

- `variavel` É a variável ou valor que terá seus bits deslocados.
- `vezes` Indica a quantidade de vezes que o valor será deslocado.
- `resultado` Variável que armazena o resultado da operação de deslocamento. Se necessário o resultado pode ser utilizado sem ser armazenado numa variável.

Se o deslocamento for para a esquerda, o bit mais significativo será descartado e na posição do bit menos significativo será inserido o valor zero, como apresentado na figura 6.9:

```
unsigned char before;
unsigned char after;
before = 0xB9;
after = before << 1;
//after = 0x72
```

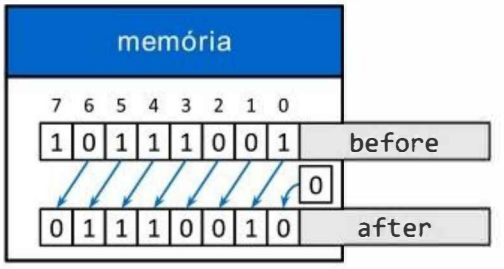


Figura 6.9: Deslocamento de bits para a esquerda

O valor resultante é, em termos numéricos, maior que o valor inicial. Para um deslocamento de uma posição, o valor será multiplicado por dois, contanto que o resultado não ultrapasse o máximo da variável original. Como a base é uma potência de dois, deslocar o número para a esquerda multiplica todas as posições por dois.

O shift para a direita possui uma peculiaridade. A norma da linguagem C não define se o operador » deve realizar um shift lógico ou aritmético.

O shift lógico para a direita efetua o deslocamento dos bits e no espaço vazio insere o número zero. O bit menos significativo da variável original é então descartado. A figura 6.10 apresenta esta operação.

```
unsigned char before;
unsigned char after;
before = 0xB9;
after = before >> 1;
//after = 0x5C
```

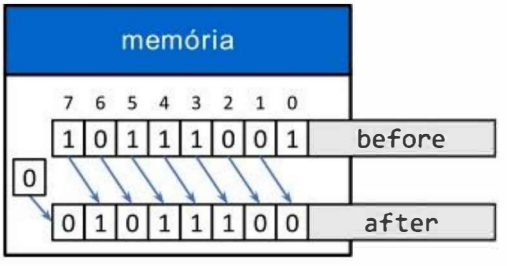


Figura 6.10: Deslocamento de bits para a direita (lógico)

Esta operação é chamada de shift lógico porque não leva em conta o resultado numérico da operação. Assim como o shift para a esquerda multiplica o valor por dois, cada deslocamento para a direita divide o valor por dois. Esta afirmação, no entanto, só é válida para números maiores que zero. Para números menores que zero, na representação de complemento de dois, o bit mais significativo tem que continuar valendo 1, mesmo após o deslocamento. Para este caso foi desenvolvido o shift aritmético.

O shift aritmético para a direita leva em conta, então, o valor do bit mais significativo da variável. Se ele for zero, será inserido zero no MSB. Se ele for 1, será inserido o valor 1. A figur a 6.11 apresenta esse comportamento:

```
char before;
char after;
before = 0xB9;
after = before >> 1;
//after = 0xDC
```

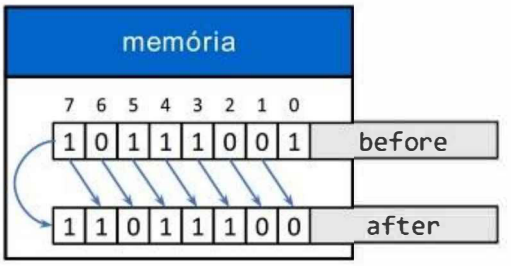


Figura 6.11: Deslocamento de bits para a direita (aritmético)

No entanto, deve-se tomar cuidado ao realizar a operação de shift para a direita. No padrão não existe nenhuma indicação quando ela será lógica ou aritmética, isso fica a cargo do programador.

Se a variável a ser deslocada não possuir sinal, unsigned char ou unsigned int, por exemplo, o shift executado será um shift lógico. Neste caso, sempre será inserido o valor zero no bit mais significativo.

Para variáveis com sinal, a documentação da linguagem C deixa a opção entre shift aritmético ou lógico para o desenvolvedor do compilador. Para saber qual foi a escolha é preciso procurar informações na documentação do compilador.

Devemos notar que o shift não altera o valor da variável original. Se quisermos alterá-la após a operação, precisamos salvar o valor em outra variável ou na própria variável original. A tabela 6.9 apresenta um exemplo de uso de cada uma das operações de shift.

Tabela 6.9: Operações de deslocamento em linguagem C

Lógico	*Bitwise*

`char A = 8; //0b00001000`
`char r;`
`r = A << 4;`
`// r = 128`
`// A = 0b00001000`
`|<<<|`
`// r = 0b10000000`
	`char A = 8; //0b00001000`
	`char r;`
	`r = A >> 3;`
	`// r = 1`
	`// A = 0b00001000`
	`|>>|`
	`// r = 0b00000001`

Shift circular ou rotacionamento de bits

Em algumas ocasiões, queremos rotacionar os bits de uma variável de modo cíclico, também conhecido como shift circular. Ao invés de descartar um bit no deslocamento, este bit é enviado para a outra extremidade da variável. As figuras 6.12 e 6.13 apresentam os dois modos de rotação.

```
unsigned char before;
unsigned char after;
before = 0x39;
after = ROT_RIGHT(before,1);
//after = 0x9C
```

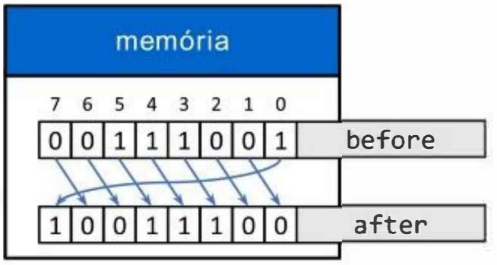


Figura 6.12: Rotação de bits para a direita

```
unsigned char before;
unsigned char after;
before = 0x39;
after = ROT_LEFT(before, 1);
//after = 0x72
```

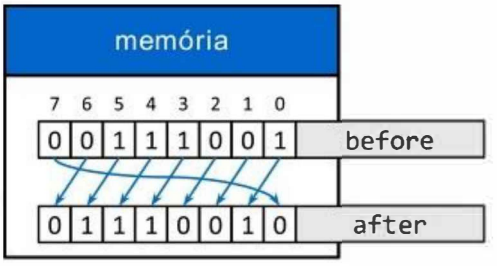


Figura 6.13: Rotação de bits para a esquerda

A linguagem C não dá suporte para executarmos essa operação diretamente. Para obter esse resultado, podemos separar a operação em 3 etapas.

Para efetuar a rotação *n* casas para a esquerda, primeiro deslocamos a variável *n* vezes à esquerda. A operação de shift insere zeros nas novas posições e descarta os valores deslocados para além da variável.

Num segundo momento, deslocamos a variável original *n* - 8 vezes para a direita, supondo uma variável de 8 bits, fazendo com que os bits que foram perdidos no primeiro deslocamento passem a ficar no lugar correto. Novamente, as posições novas são preenchidas com zeros.

Por fim, realizamos a operação OU entre esses dois valores intermediários e o resultado será o valor original rotacionado de *n* casas.

```
1 //para rotacionar X casas à esquerda uma variável de 8 bits
2 result = (variavel << X) | (variável >> (8 - X));
3
4 //para rotacionar X casas à direita uma variável de 8 bits
5 result = (variavel >> X) | (variável << (8 - X));
```

6.4 Manipulando apenas 1 bit de cada vez

Em diversas ocasiões é necessário que trabalhemos com os bits de maneira individual, principalmente quando estes bits representam saídas ou entradas digitais, como, por exemplo, chaves ou leds.

Suponha, por exemplo, que um sistema possua 8 leds ligados ao microcontrolador. Cada led é representado através de **1** bit de uma variável. Para ligarmos ou desligarmos apenas um led por vez, não alterando o valor dos demais, devemos nos utilizar de alguns passos da álgebra booleana.

Para ligar apenas um bit, utilizaremos uma operação OU. Supondo duas variáveis iniciais **A** e **B**. Se **A** possui o valor **1**, a operação **A|B** possui como resultado o valor 1, independentemente de **B**.

Se a variável **A** possui valor **0**, o resultado da operação **A|B** vai depender unicamente do valor de **B**.

Deste modo podemos utilizar a variável **A** como um sistema de sinalização para ligar um bit. Se **A** vale 1, o bit será ligado. Se **A** vale **0** o bit não será alterado.

Utilizando uma operação bitwise OU, podemos então controlar quais bits serão ligados na variável desejada.

O último passo para ligar apenas um bit dentro é criar uma variável de controle onde apenas o bit da posição que desejamos ligar possua valor um e as demais posições possuam valor **0**. Essa variável de controle é mais conhecida como máscara. A máscara é montada conforme a tabela a seguir, onde **X** é o bit desejado e **N** é a quantidade total de bits na variável.

Posição	N	...	X+1	X	X-1	...	0
Valor	0	...	0	1	0	...	0

Para criar uma máscara na qual todas as posições são **0** e apenas a posição de interesse seja **1**, basta deslocar o valor **1** **X** vezes para a esquerda, onde **X** é a posição do bit que queremos ligar.

Para ligar apenas o bit 2 de uma variável **PORTD** podemos utilizar o código 6.1:

Código 6.1: Ligar um bit sem alterar os demais

```
1 void main(void) {
2     char PORTD;
3     char mascara;              //variável que guarda a máscara
4     PORTD = 0x00;
5     //criar uma mascara com apenas o 1º bit ligado
6     mascara = 1;               // mascara = 0b00000001
7     // deslocando a mascara 2 posições
8     mascara = mascara << 2;    // bit = 0b00000100
9     //Ligando o bit 2
10    PORTD = PORTD | mascara;
11 }
```

Para conseguirmos desligar apenas um bit de uma variável, o procedimento é muito parecido com o que utilizamos para ligar um bit. Ao invés de utilizarmos uma operação bitwise OU, utilizaremos uma operação bitwise E.

Dadas duas variáveis, **A** e **B**, se **A** vale **0**, a operação **A&B** tem resultado **0** independentemente do valor de **B**. Se **A** vale **1**, o resultado da operação **A&B** é o próprio valor de **B**.

A geração da máscara para desligar um bit é realizado de maneira similar. No entanto, ela deve possuir todos os bits iguais a **1**, com exceção do bit X, o bit de interesse, que deverá valer **0**, conforme a tabela a seguir:

Posição	N	...	X+1	X	X-1	...	0
Valor	1	...	1	0	1	...	1

Para criar essa máscara podemos utilizar o mesmo procedimento para a criação da máscara para ligar um bit, mas no final inverteremos cada um dos bits. Isto pode ser feito através da operação bitwise de negação.

Por exemplo, para desligar apenas o bit 2 da variável **PORTD** podemos utilizar o código 6.2:

Código 6.2: Desligar um bit sem alterar os demais

```
1 void main(void) {
2     char PORTD;
3     char mascara;              //variável que guarda a máscara
4     PORTD = 0xFF;              //desliga todos os leds (lógica negativa)
5     //liga o primeiro bit da variável
6     mascara = 1;               // mascara = 0b00000001
7     // deslocando a mascara em 2 posições
8     mascara = mascara << 2;    // mascara = 0b00000100
9     // invertendo todos os bits da mascara
10    mascara = ~mascara;        // mascara = 0b11111011
11    //Desliga o bit 2
12    PORTD = PORTD & mascara;
13 }
```

Este procedimento para a criação da máscara é mais seguro, pois, a princípio não sabemos se a variável possui 8 ou 16 bits. Assim garantimos que todos os bits que não deverão ser desligados possuam valor **1**.

A alternativa seria inicializar a variável com apenas um **0** e fazer uma operação de deslocamento. No entanto, isto não funcionaria, pois, ao deslocar os bits, novos valores **0** são inseridos.

Para trocar o valor de um bit utilizaremos como artifício algébrico a operação bitwise OU EXCLUSIVA.

Nesta operação, o resultado da operação **A^B** será igual a **B**, se a variável **A** possuir o valor **0**, e será o inverso de **B** se a variável **A** possuir o valor **1**.

A variável **A** funciona como um sistema de controle, indicando se o bit da variável **B** deverá ter seu valor mantido ou se o bit deverá ter seu valor trocado.

Este comportamento da operação OU EXCLUSIVO permite que troquemos o valor de um bit em uma determinada posição, sem sabermos o valor original dele. Se o bit original estiver ligado, ele será desligado. Se o bit original estiver desligado, ele será ligado.

A máscara utilizada neste caso é a mesma para a operação de ligar um bit, conforme a tabela a seguir:

Posição	N	...	X+1	X	X-1	...	0
Valor	0	...	0	1	0	...	0

Por exemplo, para trocar os bits 2 e 6 da variável **PORTD** podemos executar o código 6.3:

Código 6.3: Trocar o valor de um bit sem alterar os demais

```
 1 void main(void) {
 2     char PORTD;
 3     char mascara;              //variável que guarda a mascara
 4     PORTD = 0xF0;              //desliga apenas os 4 primeiros bits
 5     mascara = 1;               // mascara = 0b00000001
 6     // deslocando a mascara em duas casas
 7     mascara = mascara << 2;    // mascara = 0b00000100
 8     //Inverte o valor do 2o bit
 9     PORTD = PORTD ^ mascara;   //PORTD = 0b11110010;
10     //liga o primeiro bit da variável
11     mascara = 1; // mascara = 0b00000001
12     // deslocando a mascara em seis casas
13     mascara = mascara << 6;    // mascara = 0b01000000
14     //Inverte o valor do 6o bit
15     PORTD = PORTD ^ mascara;   //PORTD = 0b10110010;
16 }
```

Percebemos através do exemplo que a utilização do procedimento apresentado troca o valor do bit escolhido. Foi utilizado o mesmo procedimento duas vezes. Na primeira, um bit foi ligado e, na segunda, outro foi desligado.

Para verificar se um determinado bit está ligado ou não, uma das opções é zerar todos os demais bits de modo que apenas o bit de interesse continue com seu valor original.

Para zerar todos os bits, com exceção do bit desejado, podemos utilizar a operação bitwise E com uma máscara onde apenas a posição de interesse tenha o valor 1. Esta máscara é a mesma da operação de ligar um bit, conforme tabela a seguir:

Posição	N	...	X+1	X	X-1	...	0
Valor	0	...	0	1	0	...	0

Ao executar uma operação bitwise E com a variável, o resultado da operação será **0** se o bit de interesse da variável original também for **0**. Se o bit da variável original for **1** a resposta será diferente de **0**. Como a linguagem C interpreta qualquer valor diferente de **0** como verdadeiro, este resultado pode ser utilizado para testes. O código 6.4 testa o bit 2 de uma variável **PORTD** e utiliza o teste para fazer uma estrutura de decisão:

Código 6.4: Testar o valor de um bit sem alterar os demais

```
1 void main(void) {
2     char PORTD;
3     char mascara;              //variável que guarda a mascara
4     PORTD = 0x00;              //desliga todos os bits
5     // cria uma variável onde APENAS o primeiro bit é 1
6     mascara = 1;               // mascara = 0b00000001
7     // deslocando a mascara em duas posições
8     mascara = mascara << 2;    // mascara = 0b00000100
9     //verificar se o bit 2 está ligado
10    if (PORTD & mascara) {
11        //o bit está ligado
12    } else {
13        //o bit está desligado
14    }
15 }
```

Muito cuidado pois o resultado da operação só será **1** se estivermos testando o bit **0**. O código abaixo será sempre falso, pois o teste com a porta B está verificando se o segundo bit está ligado. Se não estiver, o resultado de **(PORTD & 1«2)** será **0**. No entanto, se o segundo bit estiver ligado o resultado será o valor **2**, pois é a segunda posição que está ligada. Apesar deste resultado poder ser interpretado como verdadeiro, a comparação **((PORTD & 1«2) == 1)**, neste caso, será processada como **(2==1)** também retornando falso.

```
1 //o código abaixo sempre retorna falso
2 if ((PORTD & 1<<2) == 1) {
3     //o bit pode estar ligado, mas a condição nunca é verdadeira
4 } else {
5     //sempre entra aqui
6 }
```

Para testar o resultado da operação de um único bit devemos sempre comparar se o resultado é igual ou diferente de **0** (zero).

```
1 //o código abaixo funciona corretamente
2 if ((PORTD & 1<<2) != 0) {
3     //o bit está ligado
4 } else {
5     //o bit está desligado
6 }
```

Criando funções através de define's

Os define's podem criar macros de substituição que funcionam como funções simples. Estas funções são mais rápidas mas possuem algumas restrições. Estas restrições, no entanto, não são importantes em algumas ocasiões. O processo de ligar, desligar, trocar ou testar o valor de um bit, por exemplo, se beneficia muito da velocidade e simplicidade destas funções.

Para escrever a função utilizando a diretiva define, precisamos reescrever os exemplos acima para que possam ser executados em uma única linha de comando em linguagem C. Por exemplo, o código para ligar um determinado bit pode ser descrito como:

```
1 char bit = 2;          //bit a ser ligado
2 char mascara;          //máscara com apenas 1 bit ligado
3 char variavel;         //variavel que terá seu bit ligado
4 mascara = 1 << bit; //criação da mascara
5 variavel = variavel | mascara; //ligando o bit
```

Rearranjando os comandos acima, podemos reescrever o código para executar a mesma funcionalidade sem necessitar da variável máscara:

```
1 char bit = 2;
2 char variavel;
3 variavel = variavel | (1<<bit); //ligando o bit
```

Fazendo uso da contração dos operadores e escrevendo o bit na mesma linha, podemos fazer todos os comandos de uma única vez:

```
1 char variavel;
2 variavel |= (1<<2); //ligando o bit
```

Na criação da macro, dois parâmetros serão utilizados conforme o código abaixo:

```
#define BitSet(variavel, bit)  (variavel |= (1<<bit))
```

Onde:	**variavel**	Variável que terá um de seus bits alterados.
	bit	Posição do bit que será ligado. ● primeiro bit fica na posição zero.

As macros para desligar, trocar o valor de um bit ou testar se um bit está ligado são apresentadas no código 6.5:

Código 6.5: Macros para operação com bits

```
1 #define BitSet(variavel, bit)  (variavel |=  (1<<bit))
2 #define BitClr(variavel, bit)  (variavel &= ~(1<<bit))
3 #define BitFlp(variavel, bit)  (variavel &=  (1<<bit))
4 #define BitTst(variavel, bit)  (variavel |   (1<<bit))
```

A macro **BitTst** é a única que não utiliza um sinal de igual na sua definição, já que ela não altera o valor da variável, apenas prepara o valor para ser utilizado num teste.

A utilização destas macros no código são parecidas com a utilização de funções tradicionais, como pode ser visualizado no código 6.6.

Código 6.6: Utilização das macros de operação com bits

```
 1 void main (void){
 2     char PORT = 0x0F; //4 bits ligados e 4 desligados
 3     BitSet(PORT, 5);  //desliga o 4o bit.
 4     //PORT = 0b0010 1111
 5     BitClr(PORT, 1);  //liga o 1o bit.
 6     //PORT = 0b0010 1101
 7     BitFlp(PORT, 2)
 8     //PORT = 0b0010 1001
 9     BitFlp(PORT, 6)
10     //PORT = 0b0110 1001
11     if(BitTst(PORT,0) != 0){
12         //o bit está ligado, este código será executado
13     }
14 }
```

6.5 Exercícios

Ex. 6.1 — Descrever a operação e montar a tabela verdade das seguintes operações binárias:

- NÃO
- E
- OU
- OU EXCLUSIVO

Ex. 6.2 — Qual a diferença entre as operações lógicas e bitwise na linguagem C? Apresente o resultado das operações a seguir. Considere todos os números e variáveis como inteiros não sinalizados de 8 bits.

```
 1 ec0 = 0b10 &  0b01;
 2 ec1 = 0b10 && 0b01;
 3 ec2 = 0b10 |  0b01;
 4 ec3 = 0b10 || 0b01;
 5 ec4 = 0x77 &  0x88;
 6 ec5 = 0x77 && 0x88;
 7 ec6 = 0xAA |  0x55;
 8 ec7 = 0xAA || 0x55;
 9 ec8 = !0x0F;
10 ec9 = ~0x0F;
11 eca = !2003;
12 ecb = ~0b10010;
13 ecc = !0b10010;
14 ecd = 0xF0 ^ 0xAA;
15 ece = (0b10 | 0b01) & 0b01;
16 ecf = (0b10 & 0b01) | 0b01;
```

Ex. 6.3 — Para que serve a operação de bit-shift?

Ex. 6.4 — Qual a diferença do shift aritmético e do shift lógico?

Ex. 6.5 — Explique o conceito de máscaras. Para que são utilizadas?

CAPÍTULO

7

Estruturas condicionais

7.1 Comando condicional if
7.2 Comando condicional aninhado
7.3 Comando de seleção *switch...case*
7.4 Exercícios

"Tentar não. Faça, ou não faça. Tentativa não há."

Mestre Yoda

No desenvolvimento de um programa é comum existirem algumas ações que devem ser executadas apenas quando alguma condição for verdadeira. Nestas situações utilizamos estruturas de decisão.

As estruturas condicionais permitem que o programa escolha entre dois blocos de comando distintos para serem executados.

O bloco de comando é um conjunto de comandos que devem ser executados juntos. Um bloco de comandos pode ser utilizado em qualquer trecho de programa que se pode usar um comando C. O bloco é delimitado por chaves, tendo seu início marcado por um **"{"** e é finalizado por um **"}"**.

7.1 Comando condicional if

A estrutura condicional mais simples é a **`if`**. O comando **`if`** realiza a decisão de mudança do fluxo do código baseada numa expressão. Se a expressão for verdadeira os comandos serão executados. Se a expressão for falsa os comandos são ignorados.

A expressão condicional pode ser formada por qualquer tipo de código ou operação cujo resultado seja um valor. Se este valor for igual a zero, a expressão é definida como falsa. Para qualquer outro valor, diferente de zero, a expressão é avaliada como sendo verdadeira.

A sintaxe do **`if`** pode ser descrita como:

```
if (exp) {
    cmd;
}//end if
```

Onde:		
	exp	Representa a condição a ser avaliada como verdadeira ou falsa.
	cmd	lista de comandos a serem executados caso a expressão seja verdadeira.

A estrutura **if** também pode ser acompanhada de um segundo bloco de comandos que será executado apenas quando a expressão for falsa. Este segundo bloco é iniciado pela palavra reservada **else**, conforme a sintaxe:

```
if (exp) {
    cmd_1; //bloco verdade
} else {
    cmd_2; //bloco falso
}
```

Onde:		
	exp	Representa a condição a ser avaliada.
	cmd_1	Comandos a serem executados caso exp seja verdadeira.
	cmd_2	Comandos a serem executados caso exp seja falsa.

A expressão dentro dos parênteses do **if** é avaliada e se for verdadeira (valor de expressão diferente de zero), o primeiro bloco de comandos, o bloco verdade, é executado. Caso a expressão seja falsa (valor igual a zero), o segundo bloco de comandos, o bloco falso, é executado.

O código a seguir apresenta um exemplo de uso da estrutura de decisão:

```
1 if (temperature > 20){
2     turnOnCooler();
3     turnOffHeater();
4 }else{
5     turnOnHeater();
6     turnOffCooler();
7 }
```

A expressão avaliada é **(temperature > 20)**. Esta expressão é verdadeira sempre que o valor armazenado na variável **temperature** for maior que 20. Neste caso, o cooler será ligado e o aquecedor desligado, comandos pertencentes ao primeiro bloco de comandos. Se a expressão avaliada for falsa, ou seja, se a variável **temperature** for menor ou igual a 20, o segundo bloco será executado, desligando o cooler e ligando o aquecedor.

Apenas o código associado ao **if** ou o código associado ao **else** será executado, nunca ambos.

7.2 Comando condicional aninhado

Em alguns casos, apenas uma expressão não é suficiente para implementar o que desejamos que o sistema faça. Nestas situações, podemos encadear mais de uma operação if.

```
 1 if( expressao1 ){
 2     //bloco verdade do if 1
 3     lista de comandos;
 4
 5     if( expressao2 ){
 6         //expr.1 verdadeiraade e expr.2 verdade
 7         lista de comandos;
 8     }else{
 9         //expr.1 verdadeira e expr.2 falsa
10         lista de comandos;
11     }
12 }else{
13     //bloco falso do if 1
14     lista de comandos;
15
16     if( expressao3 ){
17         //expr.1 falsa e expr.3 verdadeira
18         lista de comandos;
19     } else{
20         //expr.1 falsa e expr.3 falsa
21         lista de comandos;
22     }
23 }
```

A sintaxe de um **if** aninhado é muito similar aos **if**'s normais. A diferença é que agora se podem tomar decisões que dependem do resultado de uma outra decisão.

Na sintaxe apresentada, existe um **if** aninhado em cada bloco de comandos do primeiro **if**. Isto não é obrigatório. Podemos colocar um **if** apenas no bloco verdade ou apenas no bloco falso. Isto fica a critério do programador.

Um exemplo seria um sistema de controle de temperatura. Além de ligar e desligar os sistemas de cooler e aquecimento, podemos ter condições que, devido a temperaturas muito altas ou muito baixas, exigem algum tipo de alarme. Se desejarmos ajustar a temperatura em 20 graus, com alarmes para temperaturas maiores que 30 ou menores que 10 graus, podemos codificar esse comportamento de acordo com o código 7.1.

Código 7.1: Exemplo de uso de if aninhado para controle de temperatura

```
 1 if (temperature > 20){
 2     turnOnCooler();
 3     turnOffHeater();
 4     if (temperature > 30){
 5         setAlarmHigh();
 6     }
 7 }else{
 8     turnOffCooler();
 9     turnOnHeater();
10     if (temperature < 10){
11         setAlarmLow();
12     }
13 }
```

7.3 Comando de seleção *switch...case*

Uma situação bastante comum é a necessidade de se comparar uma mesma variável com um grande conjunto de valores, de modo que cada valor execute uma ação diferente. Um exemplo deste comportamento é o processo de leitura dos botões de um sistema. Para cada botão pressionado queremos uma ação diferente: o botão "start" deve executar a ação selecionada, o botão "pra_esquerda" deve voltar a tela anterior, o botão "A" deve aumentar o valor da variável selecionada.

Para realizar essas diversas comparações com uma mesma variável utilizamos a estrutura **`switch...case`**. A comparação só pode ser feita com números e caracteres, sendo apenas comparações de igualdade, não sendo possível testar se uma variável é maior ou menor que um valor. Cada comparação **`case`** também e conhecida como um caso.

A sintaxe de utilização da estrutura **`switch...case`** é dada por:

```
switch( variavel ){
    case 1:
        // se variavel == 1
        lista de comandos 1;
        break;

    case 2:
        // se variavel == 2
        lista de comandos 2;
        break;

    // adiciona-se quantos cases forem necessários
    case 57:
        // se variavel == 57
        lista de comandos 57;
        break;

    default:
        //se for diferente de todos os valores acima
        lista de comandos default;
        break;
}//end switch
```

Onde:

`variavel`	É a variável a ser testada.
`case`$_n$	Executa uma lista de comandos caso a variável possua o valor n.
`default`	Se a variável não possuir nenhum dos valores anteriores a lista de comandos default será executada.

Na estrutura **`switch...case`** podemos criar quantas condições forem necessárias. No entanto, essas condições servem apenas para comparações com números ou letras, não sendo permitida a comparação da variável em teste com uma segunda variável.

Todas as estruturas **`case`** devem possuir um **`break`**. A ausência do **`break`** acionará um efeito conhecido como fallthrough. Nesta situação, o código continuará a ser executado até encontrar o próximo **`break`**.

Existe também um caso especial que é definido com a palavra **`default`**. Em português, **`default`** pode ser traduzido como padrão. Este é o caso executado quando nenhuma comparação for verdadeira.

Como boa prática de programação deve-se utilizar sempre um caso padrão **`default`**.

Um bom exemplo de uso desta estrutura é a execução de comandos que devem ser executados apenas sob uma dada entrada no teclado. Supondo o desenvolvimento de um jogo, poderíamos definir as funções dos botões do joystic como apresentado no código 7.2.

Apesar de neste exemplo não executarmos nenhuma ação quando não há teclas pressionadas, ainda assim é uma boa prática inserir o caso **`default`**.

Código 7.2: Exemplo de uso da estrutura case para leitura de um teclado

```
1 //Mapeando as teclas
2 // A – pulo, B – corrida, X – tiro, Y – tiro forte
3 // 'S'tart – pausa/resume
4 key = kbReadKey();
5 switch( key ){
6     case 'A':
7         jump();
8         break;
9     case 'B':
10        dash();
11        break;
12    case 'X':
13        shoot();
14        break;
15    case 'Y':
16        if(powerShootCount > 0){
17            powerShoot();
18            powerShootCount--;
19        }
20        break;
21    case 'S':
22        if(pauseStatus == 1){
23            unpause();
24            pauseStatus = 0;
25        }else{
26            pause();
27            pauseStatus = 1;
28        }
29        break;
30    default:
31        //nenhum comando acionado
32        break;
33 }//end switch
```

Para dois casos, "Y" e "S", inserimos uma estrutura **`if`** dentro do **`case`**. No "Y" esta estrutura verifica primeiro se o jogador não gastou todos os tiros especiais antes de executar a ação **`powerShoot()`**.

O botão "S" start pode realizar duas operações dependendo do estado atual do jogo: pausar ou continuar. Para isso, é utilizada uma variável auxiliar, **`pauseStatus`**, que indica o estado atual do jogo, servindo de comparação e controle para este botão.

7.4 Exercícios

Ex. 7.1 — Dados três pontos quaisquer: A(xA,yA), B(xB,yB), C(xC,yC), eles serão colineares se pertencerem a uma mesma reta. Para determinar se os 3 pontos são colineares, pode-se verificar se os coeficientes angulares das retas formadas pelos pontos AB e pelos pontos BC são iguais, ou seja, $yA - yB/xA - xB = yB - yC/xB - xC$. Construa um programa em linguagem C que, dados 3 pontos quaisquer, seja capaz de responder se são ou não colineares.

Ex. 7.2 — Um professor deseja automatizar seu sistema de pontuação. Ele aplica 5 (cinco) provas e calcula o conceito do aluno usando apenas as quatro maiores notas. Uma média de 9.0 ou mais ganha um grau A; 8.0 a 8.9, um grau B; 7.0 a 7.9, um grau C; e 0.0 a 6.9, um grau D. Escreva um programa que, dadas as 5 notas, encontre as quatro maiores, calcule a média e descubra o conceito do aluno.

Ex. 7.3 — Faça um programa em linguagem C para descobrir o menor valor entre três valores numéricos.

Ex. 7.4 — Uma equação quadrática é da forma $ax^2 + bx + c = 0$, onde $a! = 0$. As duas soluções para esta equação são dadas por $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$; onde a quantidade $(b^2 - 4ac)$ é denominada descriminante da equação. Escreva um programa que, dados os respectivos coeficientes (a, b e c), calcule o descriminante e imprima as soluções. Use as seguintes regras: descriminante = 0 raízes iguais; descriminante < 0 não há solução real; descriminante > 0 duas soluções reais distintas.

Ex. 7.5 — Faça um programa que, dado um número, calcule se este número é par ou ímpar e se é positivo ou negativo.

Ex. 7.6 — Faça um programa que, dado o ano de nascimento de uma pessoa, diga se ela poderá ou não votar este ano. No Brasil, a pessoa com mais de 16 anos pode votar. Não é necessário considerar o mês em que a pessoa nasceu.

Ex. 7.7 — As maçãs custam R$1, 30 cada, se forem compradas menos de uma dúzia, e R$1, 00, se forem compradas pelo menos 12. Escreva um programa que dado o número de maçãs compradas, calcule e escreva o custo total da compra.

Ex. 7.8 — Um dado sensor de corrente, que monitora a corrente de um motor, foi conectado ao microcontrolador através de um conversor analógico-digital. A corrente é medida em miliamperes e pode ser lida através do endereço $0x5f4$. Caso o motor trave, ou seja bloqueado por algum motivo, a corrente sobe demasiadamente. Para evitar a queima do sistema, o motor deve ser desligado nesta situação. Faça um programa cíclico que monitore a corrente do sistema. Quando essa corrente ultrapassar o valor de 512 miliamperes o sistema deve desligar o bit 2 da porta A, que, por sua vez, desliga o motor. A porta A pode ser acessada pela variável PORTA.

CAPÍTULO

8

Estruturas de repetição

8.1 Repetição com teste no início
8.2 Repetição com teste no final
8.3 Repetição com variável de controle
8.4 Comandos de desvio
8.5 Rotinas de tempo
8.6 Exercícios

"Para atingir a excelência em qualquer coisa na vida, é preciso repetir e treinar. Treinar e repetir, aprender a técnica de tal maneira que ela se torne intuitiva."

Paulo Coelho, *Aleph*

Existem estruturas que permitem a repetição de um trecho de código várias vezes. Estas estruturas são bastante utilizadas quando uma determinada sequência de ações ou eventos tem que acontecer um determinado número de vezes, ou enquanto uma determinada condição for verdadeira. Em computação é comum chamar estas estruturas de **`loops`**.

Todo loop **deve** possuir uma condição que indique quando ele deve terminar. Uma condição mal feita, que nunca é falsa, pode fazer com que o programa fique preso indeterminadamente. Esta é uma das causas mais comuns para o "travamento" das aplicações, chamada também de **loop infinito**.

8.1 Repetição com teste no início

A repetição com teste no início do loop é usada quando queremos que um determinado conjunto de comandos seja repetido enquanto uma condição for verdadeira. A sintaxe esta estrutura em linguagem C é dada por:

```
while( condicao ){
    comandos;
    alteracaoDaCondicao;
}//end while
```

Onde:

condição Indica ao processador se o loop deve continuar a ser executado.

comandos Lista de comandos a serem executados caso a condição seja verdadeira.

alteraçãoDaCondição Comando para alterar o valor da condição.

O controle do loop é feito através de uma condição. Para que o sistema não entre em "loop infinito" esta condição tem que ser alterada em algum momento dentro do loop.

Podemos utilizar esta estrutura para ficar aguardando o jogador pressionar algum botão conforme apresentado no código 8.1:

Código 8.1: Exemplo de uso da estrutura while para leitura de um teclado

```
1 x = kpRead();
2 //0 indica que não houve botão pressionado
3 while(x == 0){
4     x = kpRead();
5 }
```

8.2 Repetição com teste no final

Assim como a instrução **while**, a instrução **do...while** é utilizada para repetirmos um bloco do algoritmo diversas vezes. A diferença entre as duas é onde a verificação da condição é realizada. Na primeira, o teste é feito antes de entrar no loop. A segunda executa o loop uma vez e depois faz o teste.

Mesmo que a condição seja falsa desde o começo da execução do programa, na estrutura **do...while** o bloco é executado pelo menos uma vez.

A sintaxe desta estrutura é bastante similar à do **while**, como pode ser visto no código a seguir:

```
do{
    comandos;
    alteracao da condicao
}while( condicao )
```

Onde:

`condição` Indica ao processador se o loop deve continuar a ser executado.

`comandos` Lista de comandos a serem executados caso a condição seja verdadeira.

`alteraçãoDaCondição` Comando para alterar o valor da condição.

O controle do loop também é feito através de uma condição. Portanto, é necessário que essa condição seja alterada dentro do loop.

Como esta estrutura faz o teste apenas ao fim do loop, executando o código dentro do loop pelo menos uma vez, ela é uma alternativa mais simples para escrever o mesmo código de leitura do joystic, como pode ser visto no código 8.2:

Código 8.2: Exemplo de uso da estrutura do...while para leitura de um teclado

```
1 //não precisa executar a leitura antes como no while
2 do{
3     x = kpRead();
4     //zero indica que não houve botão pressionado
5 }while(x == 0)
```

8.3 Repetição com variável de controle

Em vários casos é preciso executar um conjunto de comandos uma certa quantidade de vezes. Uma maneira de se obter esse resultado é utilizar uma variável de controle em conjunto com as estruturas de repetição já apresentadas. A variável de controle será responsável por indicar quantas vezes o loop já foi realizado. O código 8.3 apresenta como implementar esse comportamento utilizando a estrutura **`while`**.

O código 8.3, a variável **`i`** é iniciada com o valor zero antes do loop começar. A condição de execução do loop é dada por **`i<10`**, que é verdadeira, já que o valor de **`i`** é zero.

Dentro do loop são executados os comandos necessários, **`doStuff()`** e a variável **`i`** é incrementada em uma unidade. Quando o loop voltar à instrução while, a condição continuará verdadeira, **`1<10`**, e o loop será executado novamente. Este processo se repetirá mais nove vezes.

Na décima rodada a variável **`i`** terá o valor nove. Como nove é menor que dez, o loop é executado mais uma vez. Ao fim do loop o valor da variável será aumentado para dez. Na

próxima execução do loop o resultado será falso, o que fará com que o loop seja encerrado e o programa continue.

Código 8.3: Repetição de um código dez vezes com a estrutura while

```
1 int i;
2
3 i=0; // começa a contagem
4 while(i<10){
5     doStuff();
6     i++;
7 }
```

Esta é uma estrutura bastante comum em diversas atividades, principalmente em operações com vetores. Por isso foi criada uma estrutura que simplifica a execução de loops controlados por uma variável. Esta estrutura é denominada **for** e sua sintaxe é dada por:

```
for (inicialização; condição; alteração) {
    comandos;
}
```

Onde:

inicialização É um conjunto de comandos a serem realizados antes de começar o loop.

condição Indica quando o loop deve parar de executar.

alteração É um conjunto de comandos a serem executados entre cada iteração do loop. Serve normalmente para alterar o valor da condição.

comandos Lista de comandos a serem executados dentro do loop.

Esta estrutura é uma forma de resumir o exemplo apresentado anteriormente com a estrutura **while** no código 8.3. Reescrevendo esse código com a estrutura **for**, temos o código 8.4.

Código 8.4: Repetição de um código dez vezes com a estrutura for

```
1 int i;
2 for (i=0; i<10; i++) {
3     doStuff();
4 }
```

Ambos os códigos são executados praticamente com a mesma velocidade e consumindo a mesma quantidade de memória. A vantagem da estrutura **for** é apresentar de maneira mais simples os valores de início, fim e alteração da variável de controle.

Podemos utilizar a estrutura **for** para imprimir cada posição de um vetor de letras no display de LCD. Para isso, definimos uma mensagem e utilizamos a função **lcdChar()**, que imprime um caractere de cada vez. O código 8.5 apresenta a implementação desta rotina.

Código 8.5: Repetição de um código dez vezes com a estrutura while

```
1 //o texto "Hello World" possui 11 caracteres
2
3 //a variável message possui 12 posições para armazenar também o caracter de ↩
      terminação '\0'
4 char message[] = "Hello World";
5
6 //como não é necessário imprimir o '\0', paramos a contagem no décimo primeiro ↩
      elemento
7
8 for (i=0; i<11; i++) {
9     lcdChar(message[i]);
10 }
```

8.4 Comandos de desvio

Existem dois comandos que podem modificar o andamento dos loops. Eles podem ser utilizados dentro do bloco de comandos de qualquer tipo de laço. São eles:

- **break** - provoca a saída do laço em que se encontra no momento.
- **continue** - termina a execução da lista de comandos e reinicia o loop. Se as condições continuarem válidas, uma nova iteração é realizada.

Os comandos **break** e **continue** permitem ao programador alterar o fluxo do programa dentro de um loop.

Ao utilizar o comando **break**, o loop é parado imediatamente, independentemente das condições. O programa tem sequência no primeiro comando depois do loop. Também é utilizado em conjunto com o comando **switch**.

Ao utilizar o comando **continue**, a iteração atual do loop para de ser executada no ponto onde o comando foi chamado e o loop recomeça. Se for usada dentro de um loop do tipo **for**,

o bloco de incremento é executado antes de recomeçar o loop.

8.5 Rotinas de tempo

Trabalhar com requisitos temporais é muito comum em sistemas embarcados. Algumas atividades devem ser executadas com uma certa cadência, com um tempo entre elas. Se estes tempos não forem obedecidos, não é possível fazer com que o sistema funcione corretamente.

Apesar de existirem periféricos dedicados à contagem de tempo, é muito comum utilizar uma estrutura de repetição para isso. A estrutura apenas fará com que o processador conte a partir de zero até um valor pré-definido, como no código a seguir:

```
1 unsigned char i;
2 for(i=0; i < 100; i++);
```

O código apresentado faz apenas a contagem, para a variável i, de zero a noventa e nove, totalizando cem repetições. Dependendo da velocidade do processador, do tipo da variável e dos valores programados, o loop pode demorar mais ou menos tempo para ser finalizado. Para as três plataformas, por exemplo, temos os tempos deste loop reunidos na tabela 8.1:

Tabela 8.1: Tempo gasto para execução de 100 loops por tipo de variável, em microssegundos

Plataforma	char	volatile char	int	volatile int	float	volatile float
Arduino Uno r3	29 ,0	70 ,1	42 ,2	120 ,8	1137 ,0	1268 ,1
Chipkit Uno32	0 ,8	14 ,4	0 ,8	12 ,1	108 ,7	170 ,3
Freedom KL05	74 ,1	83 ,0	70 ,4	70 ,6	887 ,2	887 ,5

Se precisarmos fazer contagens de tempo maiores podemos utilizar estruturas de repetição encadeadas. Deste modo, os valores são multiplicados. A plataforma Arduino, por exemplo, consome 29,0 uS para um loop com 100 repetições. Para gerar um atraso aproximado de 1 segundo podemos executar este loop 34.483 vezes ($34483 \times 29uS = 1,000007s$). Para conseguir contar até esse valor é necessário uma variável do tipo **unsigned int**.

```
1 void delay1s(void){
2     char i;
3     unsigned int j;
4     for(j=0; j < 34483; j++) {    //34483*29 = 1,000007s
5         for(i=0; i < 100; i++);   //100 repetições = 29uS
6     }
7 }
```

Executando o código acima na plataforma Arduino observamos um tempo de 17,37mS, muito diferente do tempo de 1 segundo calculado. Percebendo que o loop não executa nenhuma tarefa, o compilador otimiza o processo, omitindo alguns cálculos. Deste modo, o tempo consumido fica menor que o esperado.

Para evitar esse problema, podem ser utilizadas as variáveis com o modificador **volatile**. Assim, a contagem de tempo será bem mais próxima do esperado. Para a plataforma Arduino, como a contagem com uma variável do tipo **volatile char** consome 70,1 uS, devemos fazer um loop externo de 14.266 para atingir 1 segundo.

```
1 void delay1s(void){
2     volatile char i;
3     volatile unsigned int j;
4     for(j=0; j < 14266; j++) {   //14266*70,1 = 1,00005s
5         for(i=0; i < 100; i++);  //100 repetições = 70,1uS
6     }
7 }
```

No teste real o loop consumiu 1,011 segundos, um erro de apenas 1% do valor 1,00005 calculado anteriormente. As variáveis com o modificador **volatile** não são otimizadas durante o processo de compilação, possuindo um comportamento mais previsível.

Como pode ser visto, acertar os valores de tempo de modo exato, pode consumir muito tempo de desenvolvimento. Esta abordagem não é uma boa solução para contagem precisa de tempo. No entanto, é uma alternativa simples quando precisamos gerar um atraso, sem compromisso com a precisão, ou quando os hardwares dedicados (timers e RTC's) não estiverem disponíveis.

Para contagens mais simples, que não exijam uma precisão muito alta, pode-se criar uma função que permita gerar um atraso programável. Para isto, basta executar uma estrutura de repetição e ajustar os valores até obtermos uma base de tempo razoavelmente confiável. Depois disso, estrutura-se a função para repetir o código de acordo com a quantidade requisitada.

```
 1 void delay1ms(void){
 2     char i;
 3     unsigned int j;
 4     for(j=0; j < 15; j++) {   //15*70,1 ~ 1,05us
 5         for(i=0; i < 100; i++);  //100 repetições = 70,1uS
 6     }
 7 }
 8 void delay(unsigned int time) {
 9     while (time--){ //executa time vezes.
10         delay1ms();
11     }
12 }
```

8.6 Exercícios

Ex. 8.1 — Construa um programa em linguagem C que calcule a diferença entre a diagonal principal (células do canto superior esquerdo ao canto inferior direito) com a diagonal secundária (células do canto superior direito ao canto inferior esquerdo) de uma matriz de 5000 linhas por 5000 colunas.

Ex. 8.2 — Uma pista de atletismo circular é formada por uma quantidade de raias concêntricas (mesmo centro), consecutivas e com a mesma largura. A partir do valor do raio que marca o início da primeira raia, pode-se calcular o valor do raio que marca o início da segunda raia adicionando o valor da largura e assim sucessivamente. Construa um algoritmo em Portugol que possibilite calcular a distância que será percorrida para completar uma volta em cada marca de raia na pista definida pelo usuário. São dados: a fórmula para cálculo do perímetro da circunferência ($p = 2 \times \pi \times r$) e o valor do raio.

Ex. 8.3 — Sendo a matriz A de ordem "m x n" e a matriz B de ordem "n x r", o produto da matriz A por B será uma matriz C de ordem "m x r" cujos elementos são obtidos pela soma dos produtos das linhas de A pelas colunas de B. Assim, um elemento na matriz C é calculado por $c(i, j) = a(i, 1).b(1, j) + a(i, 2).b(2, j) + ... + a(i, n).b(n, j)$. Escreva um programa que calcule a multiplicação da matriz VA de ordem 100 x 30 e da matriz VB de ordem 30 x 50, armazenando o resultado em uma matriz VC.

Ex. 8.4 — Uma faculdade deseja saber quais são os alunos que cursam a disciplina Cálculo, mas não estão cursando a disciplina de Matemática Discreta. Existe disponível um vetor que armazena as matrículas dos alunos que fazem Cálculo (com 50 alunos) e um outro vetor que armazena as matrículas dos alunos de Matemática Discreta (com 80 alunos).

Ex. 8.5 — Em estatística, o conceito de variância é usado para descrever um conjunto de observações. Quando o conjunto de observações é uma população, é chamada de variância da população. Se o conjunto das observações é (apenas) uma amostra estatística, chamamos de variância amostral (ou variância da amostra). Faça um algoritmo que, dado um vetor V com

20 elementos, calcule a variância da amostra (*Var*) usando a seguinte fórmula: $Var(X) = \sum_{i=1}^{n} p_i \cdot (v_i - \mu)^2$, onde: v_i é o elemento i do vetor V; n é o número de elementos; e μ é a média de todos os elementos, que deve ser calculada antes.

Ex. 8.6 — Apresentar o total da soma dos cem primeiros números pares (2 + 4 + 6 + ... + 98 + 100).

Ex. 8.7 — Faça um programa que calcule o número de divisores inteiros de um dado número.

Ex. 8.8 — Um número primo é qualquer inteiro positivo divisível apenas por si mesmo e por 1. Escreva um programa que calcule se um dado número é primo ou não.

Ex. 8.9 — Dado um número natural n, faça um programa para determinar e escrever o número harmônico Hn definido por: $Hn = 1 + 1/2 + 1/3 + 1/4 + ...1/n$.

Ex. 8.10 — Faça um programa que dado um valor N inteiro e positivo, calcule o fatorial de N ($N!$).

Ex. 8.11 — Faça um programa que calcule o resultado da expressão de potenciação, dado o valor de uma base e da potência.

Ex. 8.12 — Faça um programa que calcule todos os números múltiplos de 7 entre 1 e N, sendo N um valor definido no começo do programa. Por exemplo: 7, 14, 21, 28, 35.

Ex. 8.13 — Construa um programa em C que, dados dois vetores X e Y, encontre entre estes 1000 pontos quais são os 3 pontos cuja soma entre suas distâncias seja o valor máximo. A fórmula para calcular a distância entre dois pontos é: $dist = \sqrt[2]{((x_a^2 - x_b^2) + (y_a^2 - y_b^2))}$. A soma entre 3 pontos A, B e C é dada por dist(A,B) + dist(B,C) + dist(C,A). Essa soma deve ser a maior entre os 1000 pontos disponíveis nos vetores.

Ex. 8.14 — Dois apostadores A e B apostaram cartelas na mega-sena. Cada cartela na mega-sena contém 6 números distintos. O apostador A está concorrendo com 150 cartelas e o apostador B está concorrendo com 95 cartelas. Para facilitar, suponha que as cartelas já estão preenchidas, não existem números repetidos na mesma cartela nem cartelas repetidas para o mesmo jogador. No entanto, os valores dos números de uma cartela não obedece nenhuma ordem. O programa deve descobrir quantas das cartelas do apostador A são iguais à do apostador B. Devem-se utilizar as matrizes a e b definidas abaixo para armazenar as cartelas do apostador A e do apostador B.

```
1    int a[150][6];
2    int b[95][6];
```

Ex. 8.15 — Faça um programa que verifica se um dado número faz parte da sequência de Fibonacci. Os dois primeiros números na sequência de Fibonacci são 1 e 1. O n-ésimo número é dado pela seguinte fórmula: $F_n = F_{n-1} + F_{n-2}$, para $n > 2$. Por exemplo, os oitos primeiros números da sequência são 1, 1, 2, 3, 5, 8, 13 e 21. Assim, o número 7 não faz parte da sequência de Fibonacci, bem como o número 13 faz parte.

Ex. 8.16 — A diretoria de uma faculdade deseja saber se existem alunos cursando as disciplinas de Algoritmos e Linguagens de Programação, mas que não estão cursando Estrutura de Dados. Existe disponível um vetor que armazena as matrículas dos alunos que fazem Algoritmos (50 alunos); um outro vetor que armazena as matrículas dos alunos de Linguagens de Programação (80 alunos); e um outro vetor que armazena as matrículas dos alunos de Estrutura de Dados (80 alunos). Faça um programa que realize a busca utilizando os três vetores dados.

```
1 int algo[50];
2 int prog[80];
3 int ed[80];
```

Ex. 8.17 — Dado um conjunto de 100 valores numéricos, faça um programa que calcule o somatório dos 100 valores dados obedecendo a fórmula a seguir:
$S = (b1 + b100)^2 + (b2 + b99)^2 + ... + (b50 + b51)^2$

Ex. 8.18 — Faça um programa que armazene em um vetor de dimensão 60, os números que representam as notas dos alunos. Depois descubra qual é a média e a maior nota.

Ex. 8.19 — Faça um programa para corrigir 30 provas de múltipla escolha. Cada prova tem 10 questões, cada questão valendo um ponto. O primeiro conjunto de dados será o gabarito para a correção da prova. Os outros dados serão respostas dos 30 alunos.

Ex. 8.20 — Faça um programa que inicialize duas matrizes de inteiros A e B, cada uma de duas dimensões com 5 linhas e 3 colunas. Construa uma matriz C de mesma dimensão, onde C é formada pela soma dos elementos da matriz A com os elementos da matriz B.

Ex. 8.21 — Faça um programa que inicialize uma matriz identidade de ordem 1000. A matriz identidade é uma matriz onde todos os elementos são zero, com exceção dos elementos da diagonal principal que valem 1.

Ex. 8.22 — Crie um programa que leia um valor de distância de um sensor ultrassônico na entrada analógica e apresente a distância no barramento de LEDs (porta D). Este sensor consegue identificar distâncias de 40 cm à 200 cm. Quando há um objeto a 50 cm, a saída do sensor apresenta 0 volts (0 no AD). Quando o objeto está à distância de 200 cm, a saída fica saturada em 5 volts (1024 no AD). O sensor possui uma resposta linear. O programa deve deixar todos os leds acesos quando o sensor indicar objeto a 50 cm e apagar um led a cada 20 cm, de modo que todos os leds fiquem apagados quando o objeto estiver a 200 cm ou mais. Pode-se fazer uso da biblioteca adc.h.

```
1    //adc.h
2    void InicializaADC(void);
3    unsigned int LeValorAD(void);
```

Ex. 8.23 – Um modelo simples de cifragem é a utilização de uma operação XOR dos dados a serem cifrados com um número. Para realizar a decifragem dos dados basta fazer novamente a operação XOR com o mesmo número. Este número pode ser entendido como uma senha. Este sistema apresenta um nível de segurança muito ruim. No entanto, é bastante rápido, sendo útil em algumas situações. Supondo que a região onde os dados estão gravados na memória vai de 0x100 até 0x2FE, faça um programa que cifre estes dados, realizando a operação de XOR com o número 0*xEF*, e armazene os dados cifrados na região de memória de 0*x*300 até 0*x*4*FE*.

```
1 //exemplo de cifragem
2 dadoCifrado = dadoNãoCifrado ^ 0xEF;
```

Ex. 8.24 – Existe uma variável não sinalizada, com 16 bits (**`unsigned int`**) e com o nome "TECLADO". Cada bit desta variável representa uma tecla física. Se a tecla está pressionada seu valor é **`1`**. Se estiver solta o valor do bit é **`0`**. Construa um programa cíclico que verifique quais teclas estão pressionadas e exiba no LCD. Para isso, pode ser utilizada a função **`lcdNumber(int n)`**.

Ex. 8.25 – Num dado sistema, a biblioteca do GPS é composta pelos arquivos "gps.h" e "gps.c" e a biblioteca de controle do quadricóptero é "quad.h" e "quad.c". Na biblioteca GPS existem apenas três funções: "void InicializaGPS(void)", que inicializa todas as configurações necessárias, "int LerLatitude(void)", que retorna o valor da latitude, e "int LerLongitude(void)". Caso não haja caractere, a função retorna o valor 0 (zero). Na biblioteca quad também temos três funções: "void VelNorteSul(int vel)", que altera a velocidade do quadricóptero na direção norte, se o valor for positivo, e para a direção sul, se for negativo. A função "void VelLesteOeste(int vel)", recebe um valor indicando em qual velocidade o quad se deslocará na direção leste (positivo) e oeste (negativo). Por último, a função "void InicializaQuad(void)", responsável por configurar o quadricóptero. Supondo que o lugar que o quadricóptero será manobrado não possui nenhum obstáculo, construa um programa cíclico que verifique a posição atual do quad e controle-o para que chegue ao seu destino. A posição destino já está definida em duas variáveis: "int longDest;" e "int latDest;".

CAPÍTULO

9

Funções e bibliotecas em linguagem C

9.1 Criando funções
- Chamada de função
- Protótipo de funções

9.2 Bibliotecas
- Referência circular
- Padrão para um header
- Projetando uma biblioteca

9.3 Driver ou biblioteca?

9.4 Composição de bibliotecas

9.5 Exercícios

"As bibliotecas padrão evitam que os programadores precisem reinventar a roda."

Bjarne Stroustrup

A modularização é um recurso muito importante apresentado nas linguagens de programação, onde um programa pode ser dividido em sub-rotinas específicas. A linguagem C possibilita a modularização por meio das funções.

As funções permitem que um determinado código possa ser reutilizado em diversos locais sem a necessidade de se copiar todo o algoritmo diversas vezes. Além disso, por meio dos parâmetros, é possivel executar o código com diferentes valores de entrada. Para melhor organizar o código é comum reunir as funções com mesma funcionalidade num mesmo arquivo, criando uma biblioteca.

A biblioteca, portanto, é um conjunto de funcionalidades reunidas de modo a facilitar a utilização por parte do programador. Estas funcionalidades podem ser agrupadas de diversas maneiras. Em geral, optamos por separá-las por tipo de funcionalidade (matemática, operação com textos, tempo, etc.) ou por tipo de hardware (lcd, teclado, serial, etc.).

Além das funções, a biblioteca pode conter definições de tipos, valores, macros e até armazenar valores dentro de si.

9.1 Criando funções

Um programa escrito em C possui no mínimo uma função chamada **main()**, que é o ponto inicial de execução do programa. Ela é a única função obrigatória na programação em

linguagem C. Existem outras funções pré-definidas que podem ser acessadas através das bibliotecas padrões.

A função pode ser definida como um conjunto de sentenças que podem ser chamadas de qualquer parte de um programa.

As funções não podem ser declaradas dentro de outras funções. Esta é uma limitação da linguagem C.

Cada função deve realizar uma tarefa definida pelos códigos implementados no corpo da função. O comando **`return`** é utilizado para indicar que a função deve ser finalizada naquele ponto. Com o fim da função, o fluxo do código retorna para o ponto em que a função foi chamada, seja no programa principal (**`main()`**) ou dentro de outra função.

A sintaxe da criação de uma função é dada por:

```
tipoRetorno nomeDaFunção(listaDeParametros){
    corpoDaFuncao;
    return expressão;
}
```

Onde:

`tipoRetorno`
tipo de valor devolvido pela função.

`nomeDaFunção`
identificador ou o nome dado à função.

`listaDeParametros`
são as variáveis passadas para a função. Quando a função recebe mais de um parâmetro, esses são separados por vírgula.

`corpoDaFuncao`
tem que estar definido entre o abre chaves { e o fecha chaves }. Não há ponto-e-vírgula depois da chave de fechamento.

`expressão` indica o valor que a função vai devolver para o programa.

A figura 9.1 apresenta um exemplo de função para somar dois números inteiros. Neste exemplo, as partes que compõem a função estão destacadas.

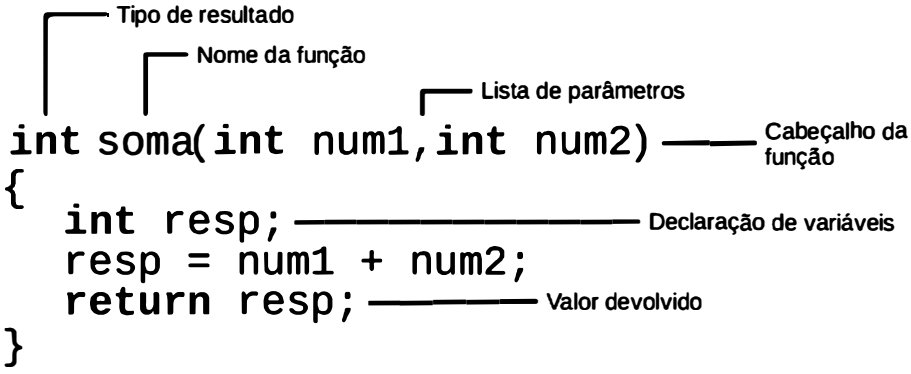


Figura 9.1: Declaração de uma função

As constantes, tipos de dados e variáveis declaradas dentro da função podem ser usadas apenas dentro da função e não podem ser lidas ou acessadas fora da função.

Um nome de função começa com uma letra ou um "_" (underscore), podendo conter tantas letras, números ou "_" quanto deseje o programador. As letras maiúsculas e minúsculas são distintas para efeito do nome da função, assim como na declaração das variáveis.

```
1 int max(int x, int y)                 //nome da funcao max
2 double media(double x1, double x2) //nome da funcao media
```

O usuário deve especificar qual o tipo de dado que a função vai retornar. O tipo de dado deve ser um dos tipos de C, por exemplo: **int**, **double**, **float** ou **char**; ou então um tipo definido pelo usuário. O tipo **void** serve para indicar que a função não retorna nenhum valor.

Por exemplo:

```
1 int max(int x, int y)                 //retorna um valor inteiro
2 double media(double x1, double x2) //retorna um double
3 float soma(int numElem)               //retorna um float
4 void tela(void)                       //não retorna nada
```

A função pode devolver um único valor, e o resultado é mostrado como uma sentença **return**. O valor retornado por uma função deve seguir as mesmas regras que são aplicadas a um operador de atribuição.

Por exemplo, o valor **int** não pode ser retornado se o tipo de retorno da função for um **char**. Uma função pode ter qualquer número de comandos **return**.

Sempre que o programa encontra uma instrução **return**, retorna para a instrução que originou a chamada para a função. A execução de uma chamada à função também termina se não encontrar nenhuma instrução **return**; e, nesse caso, a execução continua até a chave final do corpo da função.

Chamada de função

Para que uma função seja executada, é necessário que a função seja chamada. Qualquer expressão pode conter uma chamada à função, a qual redirecionará o controle do programa para a função chamada. Para realizar a chamada basta escrever o nome da função acrescido de parênteses no final. Dentro dos parênteses são escritos os valores que devem ser passados para a função, se forem necessários.

Normalmente, a chamada a uma função é realizada pela função **main()**, mas a chamada pode ser feita dentro de outra função.

Quando a função termina sua execução, o controle do programa volta para a função que a chamou, seja a **main()** ou outra qualquer.

Por exemplo, para criar uma função que imprima 10 asteriscos no LCD, podemos implementar o seguinte código:

```
 1 #include "io.h"
 2 #include "lcd.h"
 3
 4 void desenha(void){
 5     int i; //variavel local
 6     for(i=0; i<10; i++){
 7         lcdChar('*');
 8     }
 9 }
10 void main(void){
11     systemInit();
12     lcdInit();
13     desenha(); //chamada para funcao desenha
14     for(;;);
15 }
```

Modificando o código anterior, é possivel criar uma função que, ao invés de imprimir uma quantidade fixa de asteriscos, possa imprimir uma quantidade definida pelo programador. Deste modo, a função pode ser utilizada em diversas ocasiões.

Para isso, basta fazer com que a função receba um parâmetro do tipo **int**, que será responsável por indicar quantos asteriscos deverão ser impressos. Esta função pode ser reimplementada como:

```
 1 #include "io.h"
 2 #include "lcd.h"
 3
 4 //o parâmetro nAst indica a quantidade de asteriscos
 5 void desenha(int nAst){
 6     int i; //variavel local
 7     //o loop agora é dinâmico e obedece o valor recebido por parâmetro
 8     for(i=0; i<nAst; i++){
 9         lcdChar('*');
10     }
11 }
12 void main(void){
13     systemInit();
14     lcdInit();
15
16     desenha(5); //chamada para funcao desenha, imprime 5 asteriscos
17
18     desenha(2); //chamada para funcao desenha, imprime 2 asteriscos
19
20     for(;;);
21 }
```

Protótipo de funções

O compilador da linguagem C realiza a leitura do documento do início para o fim. Por isso, é imprescindível que a função seja declarada antes de ser utilizada. Deste modo, o compilador consegue identificar se o código está coerente com relação ao uso das funções.

Isto pode gerar alguns problemas. Se a função A faz uso da função B, e a função B faz uso da função A, não existe uma ordem que permita escrever A antes de B e B antes de A. Para solucionar esses problemas foi criada a estrutura de protótipos de função.

Além disso, os protótipos são utilizados nos arquivos de cabeçalho (.h) para indicar ao programador quais funções estão disponíveis dentro da biblioteca.

Os protótipos servem, portanto, para declarar uma função sem a sua implementação. O protótipo de uma função possui o mesmo cabeçalho da função, com a diferença que os protótipos terminam com ponto-e-vírgula.

Um protótipo é composto dos seguintes elementos: o tipo de retorno, o nome da função, os parâmetros (que devem estar entre parênteses e são opcionais) e um ponto-e-vírgula no final.

A sintaxe de um protótipo é dada por:

```
tipoRetorno nomeDaFunção(listaDeParametros);
```

Onde:

tipoRetorno
tipo de valor devolvido pela função.

nomeDaFunção
identificador ou o nome dado à função.

listaDeParametros
são as variáveis passadas para a função. Quando a função recebe mais de um parâmetro, esses são separados por vírgula.

Um protótipo declara uma função e passa ao compilador as informações necessárias para que ele verifique se a função está sendo chamada corretamente, com relação à quantidade e tipo dos parâmetros, bem como o tipo de retorno.

Uma boa prática de programação é sempre definir todos os protótipos no início do programa, antes da definição do **`main()`**.

O compilador utiliza os protótipos para validar que o número e os tipos de dado dos argumentos na chamada da função, sejam os mesmos que aparecem na declaração formal da função chamada. Caso encontre alguma inconsistência, uma mensagem de erro é visualizada.

Uma vez que os protótipos foram processados, o compilador conhece quais são os tipos de argumentos que ocorreram. Quando é feita uma chamada para a função, o compilador confirma se o tipo de argumento na chamada da função é o mesmo definido no protótipo. Se não forem os mesmos, o compilador gera uma mensagem de erro.

9.2 Bibliotecas

Uma biblioteca em linguagem C é um conjunto de funcionalidades reunidas de modo a facilitar a utilização por parte do programador. Estas funcionalidades podem ser agrupadas de diversas maneiras. Em geral, optamos por separá-las por tipo de funcionalidade (matemática, operação com textos, tempo etc.) ou por tipo de hardware (lcd, teclado, serial etc.).

A biblioteca funciona como modo de organização do código. À medida que o projeto cresce em complexidade, manter todas as funções num mesmo arquivo pode dificultar uma futura manutenção ou atualização do programa.

Outra vantagem do uso de uma biblioteca é a capacidade de reutilização de código. Uma vez criada a biblioteca matemática, seu código pode ser utilizado em vários outros projetos. De fato, algumas funcionalidades são tão básicas que as bibliotecas que suprem essa necessidade são padronizadas e disponibilizadas diretamente com o compilador de linguagem C. Essas bibliotecas formam o conjunto das bibliotecas padrão, ou standard libs.

Para a primeira versão da linguagem C, havia 15 bibliotecas padronizadas, que proviam desde funções matemáticas, como seno e cosseno, passando pela manipulação de tempo e strings até o controle de alocação de memória ou geração de números aleatórios. Em 1995, três novas bibliotecas foram adicionadas, em geral para gerenciar operações com caracteres. Em 1999, seis novas bibliotecas, a maioria para operações numéricas, foram inseridas. Em 2011,

foram adicionadas mais cinco, totalizando 29. A relação destas bibliotecas pode ser encontrada na tabela 9.1.

Em geral, a biblioteca é composta de um arquivo de cabeçalho, ou header, com extensão .H, e um arquivo de código, com extensão .C. O arquivo .H funciona como um índice ou resumo, explicando quais funcionalidades foram implementadas no arquivo de código. Em geral, são disponibilizados apenas os protótipos de função e definição de tipos no header. As funções são implementadas no arquivo .C, bem como o armazenamento de informações.

Tabela 9.1: Bibliotecas padrão da linguagem C

Nome	Definida em	Descrição
<assert.h>		Auxilia no teste e depuração do programa.
<complex.h>	C99	Funções para manipulação de números complexos.
<ctype.h>		Funções para manipulação de caracteres.
<errno.h>		Definições dos códigos de erros reportados pelas bibliotecas.
<fenv.h>	C99	Conjunto de funções para controlar o comportamento dos números com ponto flutuante.
<float.h>		Define contantes com macros para explicitar as propriedades dos números flutuantes.
<inttypes.h>	C99	Define tipos de variáveis inteiras com tamanhos definidos e como formatá-las para impressão/leitura.
<iso646.h>	95	Macros para expressar tokens padronizados. Para programar com caracteres do tipo ISO 646.
<limits.h>		Define contantes com macros para explicitar as propriedades dos números inteiros.
<locale.h>		Utilizada para garantir acentuação correta de acordo com o padrão de acentuação do sistema.
<math.h>		Define as funções matemáticas mais comuns.
<setjmp.h>		Declara as macros setjmp e longjmp.
<signal.h>		Define funções para tratamento de eventos/sinais.
<stdalign.h>	C11	Especifica o alinhamento dos objetos/variáveis na memória.
<stdarg.h>		Para funções com quantidade de argumentos variáveis.
<stdatomic.h>	C11	Para simplificar operações atômicas em objetos compartilhados entre threads.
<stdbool.h>	C99	Define o tipo booleano.
<stddef.h>		Define um conjunto extenso de macros.
<stdint.h>	C99	Define tipos de variáveis inteiras com tamanhos definidos.
<stdio.h>		Funções para entrada e saída de dados.
<stdlib.h>		Cria funções para conversão numérica, geração de números aleatórios, alocação de memórias e controle de processos.
<stdnoreturn.h>	C11	Estruturas para especificar funções que nunca retornam.
<string.h>		Funções para manipulação de strings.
<tgmath.h>	C99	Funções matemáticas para tipos genéricos.
<threads.h>	C11	Funções para gerenciamento de múltiplas threads.
<time.h>		Funções para manipulação de hora e data.
<uchar.h>	C11	Funções para manipulação de caracteres padrão Unicode.
<wchar.h>	95	Funções para manipulação de string com caracteres Unicode.
<wctype.h>	95	Funções para classificar e converter caracteres Unicode.

Referência circular

Um problema relacionado à criação de bibliotecas é a possibilidade de referência circular.

Para exemplificar esse problema podemos supor duas bibliotecas, uma responsável pela comunicação serial (serial.h) e a outra responsável pelo controle de temperatura (temp.h). Um determinado projeto pode exigir que a temperatura seja controlada pela porta serial e que toda vez que a temperatura passar de um determinado valor, seja enviado um alerta pela porta serial.

A implementação do arquivo header da biblioteca da porta serial (serial.h) tem as seguintes funções:

```
1 // arquivo serial.h
2 // header da biblioteca de comunicação serial
3
4 char LerSerial(void);
5 void EnviaSerial(char val);
```

O arquivo de controle da temperatura (temp.h), por sua vez, é composto das seguintes funções:

```
1 // arquivo temp.h
2 // header da biblioteca de controle de temperatura
3
4 char LerTemperatura(void);
5 void AjustaCalor(char val);
```

Em seu funcionamento, toda vez que a função **`LerTemperatura()`** for chamada, ela deve fazer um teste e se o valor for maior que um limite pré-definido, ela deve chamar a função **`EnviaSerial()`** com o código 0*x*30. Para isso, o arquivo temp.h deve incluir o arquivo serial.h. Portanto, o header do arquivo temp.h passa a ser:

```
1 // arquivo temp.h
2 // header da biblioteca de controle de temperatura
3
4 #include "serial.h"
5
6 char LerTemperatura(void);
7 void AjustaCalor(char val);
```

Já a função **LerSerial()**, realiza a leitura de um caractere da serial e, se ele for um número, ela deve chamar a função **AjustaCalor()** e repassar o valor. Portanto, para funcionar corretamente, o arquivo serial.h deve incluir o arquivo temp.h:

```
1 // arquivo serial.h
2 // header da biblioteca de comunicação serial
3
4 #include "temp.h"
5
6 char LerSerial(void);
7 void EnviaSerial(char val);
```

O problema é que, deste modo, é criada uma referência circular: o compilador lê o arquivo serial.h e percebe que tem que inserir o arquivo temp.h. Inserindo o arquivo temp.h percebe que tem que inserir o arquivo serial.h. Inserindo o arquivo serial.h percebe que tem que inserir o arquivo temp.h. Esta sequência de inclusões segue indefinidamente, conforme pode ser visto na figura 9.2:

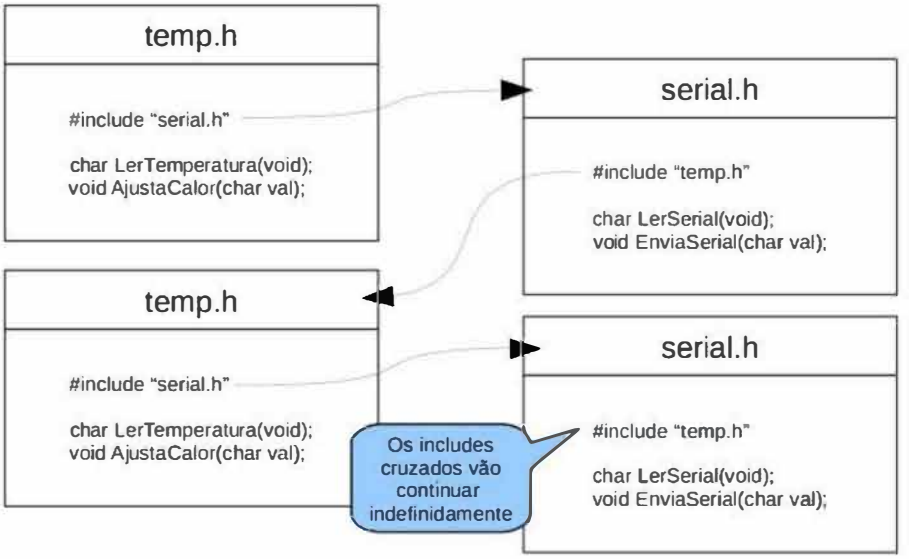


Figura 9.2: Problema das referências circulares

A solução é criar um dispositivo que permita que o conteúdo do arquivo seja lido apenas uma vez. Este dispositivo é implementado através da estrutura apresentada no código 9.1:

Código 9.1: Estrutura de header

```
1 #ifndef TAG_CONTROLE
2 #define TAG_CONTROLE
3 //todo o conteúdo do arquivo vem aqui.
4
5 #endif //TAG_CONTROLE
```

Segundo o código 9.1, o conteúdo que estiver entre o **#ifndef** e o **#endif**, só será mantido se a tag "**TAG_CONTROLE**" não estiver definida. Como isto é verdade durante a primeira leitura, o pré-compilador lê o arquivo normalmente. Se acontecer uma referência cíclica, na segunda vez que o arquivo for lido, a tag "**TAG_CONTROLE**" já estará definida impedindo assim que o processo cíclico continue, conforme pode ser visto na figura 9.3.

Geralmente, usa-se como tag de controle o nome do arquivo, já que ele é único para cada arquivo e biblioteca.

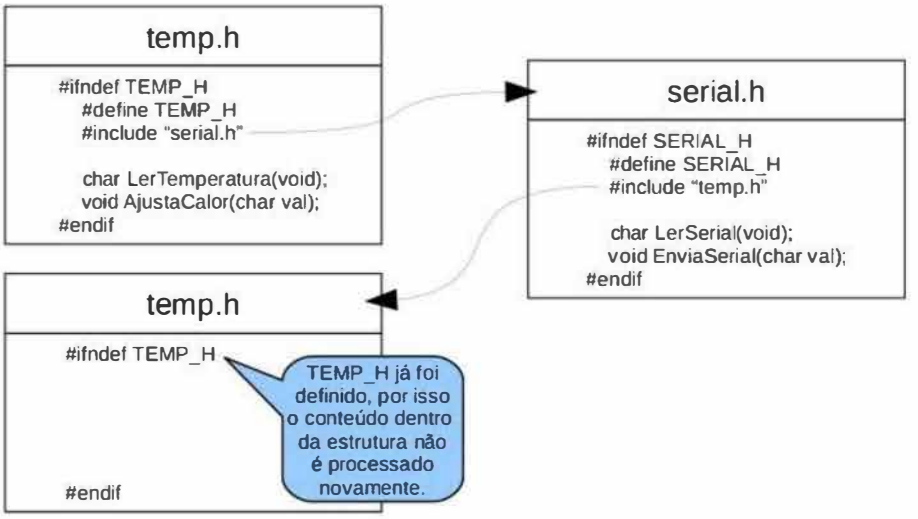


Figura 9.3: Solução das referências circulares com **`#ifndef`**

Padrão para um header

Para evitar o problema de referência circular, bem como padronizar a criação das bibliotecas, podemos utilizar uma estrutura de pré-compilação conforme o modelo a seguir:

```
#ifndef NOME_ARQUIVO_H
#define NOME_ARQUIVO_H
    //definição de novos tipos (typedef)
    typedef tipoNovo tipoAntigo;
    //protótipos de função
    void PrototypeFunction(void);
#endif
```

Onde:

`NOME_ARQUIVO_H`
É o nome do arquivo ou da biblioteca.

`typedef` Comando para definição dos novos tipos de variáveis da biblioteca.

`PrototypeFunction`
Lista das funções disponibilizadas pela biblioteca.

As funções que estiverem com seu protótipo definido no header, podem ser utilizadas pelos arquivos que incluírem a biblioteca. Já aquelas que estiverem declaradas apenas no arquivo de código não poderão ser acessadas externamente.

A biblioteca é utilizada para organizar o código. Isto facilita, inclusive, a reutilização das funções em outros projetos. Em geral, as bibliotecas reúnem funções que provêm o mesmo tipo de recurso, ou trabalham com o mesmo periférico.

Esta diferença é bastante útil quando a biblioteca possui algumas funções que não devem ser utilizadas fora da biblioteca, por exemplo, a biblioteca de controle do LCD. O código 9.2 apresenta o conteúdo dos arquivos lcd.h e lcd.c:

Código 9.2: Exemplo de biblioteca: LCD

```
1  //#######################################
2  //header: lcd.h
3
4  #ifndef LCD_H
5  #define LCD_H
6      void lcdInit(void);
7      void lcdChar(char letter);
8      void lcdNumber(int value);
9      //a rotina de delay não é disponibilizada para os demais arquivos
10 #endif
11
12 //#######################################
13 //código: lcd.c
14
15 //É uma boa pratica o arquivo de código incluir o header
16 #include "lcd.h"
17
18 void lcdInit(void){
19     //inicialização do lcd
20     //segue o protocolo definido pelo tipo de LCD utilizado
21 }
22
23 void lcdChar(char letter){
24     //envio de um caracter para o lcd
25     //segue o protocolo definido pelo tipo de LCD utilizado
26 }
27
28 void lcdNumber(int value){
29     //decompõe o número num conjunto de caracteres antes de imprimir cada algarismo
30     //segue o protocolo definido pelo tipo de LCD utilizado
31 }
32
33 void delay(int microsseconds){
34     //rotina para geração de um atraso de microssegundos
35     //geralmente implementada como um loop for
36 }
```

A função **`delay()`** está declarada apenas no arquivo de código. Deste modo, não é possível chamar esta função dentro do main.c ou de qualquer outro arquivo que inclua o lcd.h. A biblioteca LCD não foi projetada para prover a funcionalidade de contagem de tempo para o programador. A função **`delay()`** foi feita unicamente para gerar os atrasos necessários para que a comunicação com o display de LCD funcione corretamente. Ela é uma função necessária para a biblioteca LCD mas não é o foco da mesma. Por este motivo, o projetista da biblioteca resolveu defini-la apenas no arquivo de código.

Projetando uma biblioteca

Ao projetar uma biblioteca, devemos primeiramente definir a necessidade que será atendida por ela. Um exemplo, seria uma biblioteca para cálculos em ambientes tridimensionais.

Num ambiente tridimensional, um elemento p tem sua localização representada por um conjunto de três números: x_p, y_p e z_p. Estes números também podem ser representados entre parênteses, $p = (x, y, z)$. Os números representam a distância que o elemento p está da referência, que é definida por $r = (0, 0, 0)$.

Existem diversos cálculos de distância que podem ser feitos: a distância cartesiana (menor distância representa por uma linha reta), a diferença de altitude dos pontos (representada pela diferença entre os valores de z), a distância de Manhattan (distância percorrida por um carro do ponto A ao ponto B numa cidade com todas as ruas perpendiculares).

Definidas as funções que serão disponibilizadas pela biblioteca, podemos montar o header como apresentado no código 9.3:

Código 9.3: Header da biblioteca 3D

```
1 //#####################################
2 //arquivo de header: dist3d.h
3 //funções de cálculo de distância
4
5 #ifndef DIST3D_H
6     #define DIST3D_H
7
8     //define o tipo posição como uma struct com 3 floats
9     typedef struct{
10        float x;
11        float y;
12        float z;
13    } position;
14
15    //calculo de distâncias entre 2 pontos
16    float CartesianDistance(position A, position B);
17    float ManhatanDistance(position A, position B);
18    float AltitudeDifference(position A, position B);
19
20 #endif
```

A implementação das funções de cálculo 3d é mostrada no código 9.4.

Podemos notar que existem dentro do arquivo seis funções. As três apresentadas no header e três funções novas: **quad()**, **mod()** e **sqrt()**. Estas funções servem de base de cálculo para as operações em 3D. No entanto, estas funções não são apresentadas no header. Esta decisão foi tomada pois a precisão apresentada por essas funções pode não ser suficiente em outras situações.

Outro motivo foi evitar problemas com o processo de compilação. Supondo que todas as funções estivessem no header. Um projetista poderia fazer uso das operações em seu

programa. Numa alteração posterior, a biblioteca de cálculo 3D poderia ser trocada por uma biblioteca que efetue o calculo utilizando um hardware específico, como uma placa de vídeo. Neste momento, o programa deixaria de funcionar, pois a nova biblioteca não apresentaria as mesmas funções, já que tais operações não são importantes para ela.

Código 9.4: Implementação das funções da biblioteca 3D

```
1 //#####################################
2 //arquivo de código: dist3d.c
3 //funções de cálculo 3d
4 #include "dist3d.h"
5
6 //Calcular o quadrado de um número
7 float quad (float val){
8     return val * val;
9 }
10
11 //retorna o módulo
12 float mod(float a){
13     if(a>0) return a;
14     return -a;
15 }
16
17 //Calcular a raiz quadrada de um número
18 //Adaptado da Fast_inverse_square_root() do jogo Quake
19 float sqrt( float number ){
20     long i;
21     float x2, y;
22     const float threehalfs = 1.5F;
23     x2 = number * 0.5F;
24     y  = number;
25     i  = * ( long * ) &y;
26     i  = 0x5f3759df - ( i >> 1 );
27     y  = * ( float * ) &i;
28     y  = y * ( threehalfs - ( x2 * y * y ) );
29     return (1/y);
30 }
31
32 float CartesianDistance(position A, position B){
33     return sqrt(quad(A.x - B.x) + quad(A.y - B.y) + quad(A.z - B.z));
34 }
35
36 float ManhatanDistance(position A, position B){
37     return mod(A.x - B.x) + mod(A.y - B.y) + mod(A.z - B.z);
38 }
39
40 //Altitude leva em conta apenas a cota Z
41 float AltitudeDifference(position A, position B){
42     return mod(A.z - B.z);
43 }
```

Uma opção, neste caso, seria reunir todas as operações matemáticas numa única biblioteca. Isto é o que a biblioteca padrão "math.h" da linguagem C faz. No exemplo apresentado, as três funções poderiam ser substituídas pela math.h através de um **`#include`**.

9.3 Driver ou biblioteca?

Um driver é um componente de software que realiza a interface entre o hardware e o programador. Em geral, funciona como uma camada de abstração do funcionamento do periférico fazendo com que o programador possa utilizá-lo de modo mais simples. Em linguagem C, o driver pode ser implementado como uma biblioteca.

A diferença do driver para uma biblioteca geral é que o driver possui uma estrutura definida, permitindo a troca do periférico sem que o programa principal precise ser alterado. Em algumas situações, o driver também deve implementar uma interface permitindo que o programador faça sua gestão de um modo padronizado, principalmente nas rotinas de inicialização, finalização e detecção de erros.

Os drivers possuem uma estrutura intimamente ligada aos periféricos. O desenvolvimento de um driver exige que o programador conheça como os dispositivos eletrônicos estão conectados ao microcontrolador, bem como o procedimento exigido pelo processador para realizar acesso a estes periféricos.

Na segunda parte deste livro, serão apresentados os conceitos dos periféricos mais comuns, bem como o desenvolvimento das bibliotecas de acesso para cada um. Na terceira parte do livro, estas bibliotecas serão padronizadas de modo que um sistema operacional possa gerenciar a sua utilização, transformando-as em drivers.

9.4 Composição de bibliotecas

Até agora, as bibliotecas foram definidas como um conjunto de funcionalidades reunidas num mesmo arquivo. Estas bibliotecas não dependiam de nenhuma outra biblioteca, sendo todas elas implementadas usando apenas os comandos básicos da linguagem C.

Em algumas situações, no entanto, é possível que a funcionalidade a ser desenvolvida seja bastante complexa, como, por exemplo, um sistema digital de controle de temperatura.

Este sistema depende, pelo menos, da capacidade de realizar a leitura de sinais de temperatura e de atuar num sistema de aquecimento ou resfriamento do ambiente. Estruturar uma biblioteca para realizar todas essas atividades pode fazer com que ela fique muito complexa. Uma opção é quebrar esta estrutura em duas camadas.

A primeira camada será responsável por realizar a interface com o mundo externo. Essa camada possui duas bibliotecas: AD e PWM. A biblioteca AD é responsável por fazer acesso a um periférico de leitura de sinais elétricos. Já a biblioteca PWM recebe um valor digital e o converte num sinal de saída. Por serem periféricos distintos é comum separá-los em duas bibliotecas. Os headers destes arquivos são apresentados no código 9.5.

Código 9.5: Header das bibliotecas AD e PWM

```
1 //#####################################
2 //header: pwm.h
3 #ifndef PWM_H
4     #define PWM_H
5     void pwmInit(void);
6     void pwmSet(int value);
7 #endif
8
9 //#####################################
10 //header: ad.h
11 #ifndef AD_H
12     #define AD_H
13     void adInit(void);
14     int adRead(void);
15 #endif
```

A biblioteca de controle de temperatura faz uso dos dois periféricos, no entanto, ela própria não faz acesso direto ao hardware. Sua principal atividade é realizada através da função **tempSystemUpdate()**, que faz a leitura da temperatura atual, através do AD, e a compara com o valor de referência. Este valor está armazenado dentro da biblioteca. Se a temperatura atual estiver mais baixa, ela utiliza o PWM para ligar o aquecedor. Esta biblioteca fica numa segunda camada de software, pois provê uma funcionalidade que é baseada em bibliotecas anteriores. Uma possível implementação para ela pode ser vista no código 9.6.

A programação por camadas tem o benefício de simplificar cada vez mais a utilização das bibliotecas por parte do programador, no entanto ela implica numa sobrecarga, também conhecida como overhead, tanto de processamento quanto de consumo de memória. Em geral, este overhead é baixo, principalmente quando comparado com as vantagens na programação, tanto em questão de velocidade, ao reaproveitamento de código e à organização do sistema. Por exemplo a utilização da biblioteca de temperatura pode ser feita como no código 9.7.

Pelo exemplo, podemos perceber que a configuração e o funcionamento do controle de temperatura são bastante simples, pois o programador não precisa se preocupar com as implementações de bibliotecas de acesso ao hardware. Outra vantagem é que se houver mudança na estrutura do sistema embarcado e, por exemplo, o modelo de aquecedor mudar, basta reconfigurar a biblioteca **PWM**. Se a mudança for um pouco mais drástica, é possível que a biblioteca **temperature** também seja modificada. A função **main()**, no entanto, permanece inalterada garantindo que as demais funcionalidades do software não sejam impactadas com essa mudança.

Código 9.6: Header e código da biblioteca temperature

```
1 //#########################################
2 //header: temperature.h
3 #ifndef TEMPERATURE_H
4     #define TEMPERATURE_H
5     void tempInit(void);
6     void tempSetReference(int value);
7     void tempSystemUpdate(void);
8 #endif
9
10 //#########################################
11 //código: temperature.c
12
13 #include "temperature.h"
14 #include "pwm.h"
15 #include "ad.h"
16
17 int reference; //variável que armazena a temperatura de referência
18 void tempInit(void){
19     pwmInit();
20     adInit();
21 }
22 void tempSetReference(int value){
23     reference = value;
24 }
25 void tempSystemUpdate(void){
26     int temp;
27     temp = adRead();
28     if (temp < reference){
29         pwmSet(100);    //liga o aquecedor
30     }
31 }
```

Código 9.7: Utilização da biblioteca temperature

```
1 //include com as funções da biblioteca temperature
2 #include "temperature.h"
3
4 void main (void)
5 {
6     tempInit();
7     //configura valor de referência da temperatura em 18 graus
8     tempSetReference(18);
9     for(;;)
10    {
11        //executa a atividade de controle de temperatura
12        tempSystemUpdate();
13    }
14 }
```

9.5 Exercícios

Ex. 9.1 — Crie uma biblioteca matemática com o nome "math" com funções que façam as seguintes operações:

- Calcule o quadrado de um número.
- Calcule o fatorial de um número. Não utilizar recursividade.
- Faça a conversão de um valor em graus para radianos.Utilize *pi* = 3.14159265359. A fórmula da conversão é dada por: $a_{graus} = a_{radianos} * 2 * \pi/360$.
- Calcule o seno de um número. A função seno pode ser aproximada por $sen(x) = (x) - (x^3)/3! + (x^5)/5! - (x^7)/7!$ (x em radianos).
- Calcule as raízes de uma equação de segundo grau. A resposta deve ser retornada pelos parâmetros da função.

Ex. 9.2 — Não existe maneira simples de gerar números aleatórios na linguagem C. O mais próximo é a geração de números pseudo-aleatórios. Estes números formam uma sequência de sorteios bem definida, mas que sob certas circunstâncias pode ser utilizado como um número aleatório. O processo de geração de números pseudo-aleatórios é feito através de uma função que envolve operações lógicas e/ou aritméticas. Faça uma biblioteca "random" com uma função que retorne um número aleatório através de uma função do tipo LCG. Neste tipo de função, o próximo número é baseado no número anterior, segundo a equação: $X_{n+1} = (aX_n + c) \mod m$. O número inicial é conhecido como seed. Faça duas funções na biblioteca:

- Uma função que receba um valor **`unsigned long int`** e armazene-o numa variável dentro da biblioteca. Essa variável deve ser chamada "seed".
- Uma função que retorne um **`unsigned long int`**. O valor de retorno deve ser calculado segundo a fórmula passada. Após o cálculo, e antes de retornar o valor, deve-se atualizar a variável seed com o novo valor.

Use os seguintes valores: a = **`16807`**, c = **`0`** e m = **`2 147 483 647`**.

Ex. 9.3 — Crie uma biblioteca chamada "BarramentoLeds". Esta biblioteca deve realizar operações com o barramento de leds que está localizado no endereço 0xf902. Crie funções que sejam capazes de:

- Ligar os leds de acordo com um parâmetro recebido. Este parâmetro será uma variável unsigned char onde os leds que devem ser ligados possuem valor 1 no seu bit respectivo. Ex: 0x81: ligar os leds 0 e 7; 0x36: ligar os leds 1, 2, 5 e 6. Os demais leds não podem ser alterados.
- Desligar os leds de acordo com um parâmetro recebido. O funcionamento é similar ao Ligar, com a diferença que os leds que devem ser desligados possuem valor zero.
- Retornar o estado atual dos leds.

Ex. 9.4 — Crie uma biblioteca "alarme" que será responsável por armazenar internamente os limites superiores e inferiores de alarme de temperatura. Esta biblioteca deve possuir as seguintes funções:

- **`void ConfiguraAlarmeAlto(int valor)`** que atualizará o valor do limite superior do alarme. A função deve garantir que o valor não seja mais baixo que o limite inferior.
- **`void ConfiguraAlarmeBaixo(int valor)`** que atualizará o valor do limite inferior do alarme. A função deve garantir que o valor não seja mais alto que o limite superior.
- **`int ChecaAlarme(int temperatura)`** que receberá o valor de temperatura e, comparando com os limites, retornará se o valor está dentro da faixa (retornando o valor 0), acima do limite superior (retornando o valor **`1`**) ou abaixo do limite inferior (retornando o valor **`-1`**).

Ex. 9.5 — Crie uma biblioteca em linguagem C que controle a temperatura de um sistema. A biblioteca deve possuir três funções: uma para ligar o sistema, uma para configurar a temperatura desejada e uma para atualizar a operação. É possível ler a temperatura através da função **`int LerTemperatura(void)`** que retorna um valor de 0 a +100 graus Celsius. Existe um aquecedor que é ligado com a função **`void AtivarAquecedor(int val)`** que deve receber um valor de 0 a 100%. Abaixo apresentamos um exemplo de arquivo main.c que usa a biblioteca.

```
 1 #include "temp.h"
 2
 3 void main (void)
 4 {
 5     LigarSistTemp();
 6     ConfiguraTemperatura(50);
 7     for(;;)
 8     {
 9         AtualizaOperaçãoTemperatura();
10     }
11 }
```

Ex. 9.6 — Construa uma biblioteca chamada "alarmes" que permita ao programador configurar 2 tipos de alarmes: temperatura e pressão. Os alarmes são ativados pelos bits 3 (temperatura) e 6 (pressão) da porta E. Para ativar os alarmes basta ligar os respectivos bits. Para desativar é suficiente desligar os bits. A biblioteca deve ter quatro funções:

- uma para configurar o valor limite de temperatura, que recebe um **`int`** com o valor limite de temperatura e não retorna nada;
- uma função para configurar o valor limite de pressão, que recebe um **`int`** com o valor limite de pressão e não retorna nada;
- uma função para inicializar o sistema de alarme;
- e uma função que faz a leitura dos valores de pressão e tensão atuais e aciona as saídas caso algum limite tenha sido ultrapassado.

A leitura dos valores de pressão e temperatura pode ser feita através da biblioteca "adc.h" que possui duas funções: **`void InicializaADC(void)`** que inicializa o ADC, e **`int LerValorAD(int canal)`**; que lê o canal desejado e retorna seu valor. O sensor de temperatura está no canal 3 e o de pressão, no canal 6.

CAPÍTULO

10

Planejando o software embarcado

10.1Primeiro modelo: o loop infinito
10.2A evolução do loop no tempo

"Você pode usar uma borracha na prancheta de desenho ou uma marreta na construção."

Frank Lloyd Wright

Vários dos equipamentos embarcados são desenvolvidos para funcionar continuamente, monitorando sinais de entrada, tomando decisões e atuando nos dispositivos de saída. Eles são planejados para que suas atividades sejam executadas continuamente, enquanto estiverem ligados.

Outra característica destes equipamentos é que, geralmente, eles não têm contato com a parte do usuário, seja um teclado ou comunicação serial. Mesmo que o equipamento tenha essas interfaces, estes eventos podem não acontecer. Assim sendo, os programas são pensados para continuarem sua execução independentemente da interação do usuário. Se um equipamento embarcado ficar aguardando uma informação de um usuário, ele pode negligenciar outras funções.

Além destas peculiaridades de seu funcionamento, devemos levar em conta a necessidade de gerenciar tanto os periféricos de entrada e saída quanto a aplicação em si. O programador tem a responsabilidade de primeiro garantir que o sistema esteja funcionando, com todas as suas características básicas, para só então desenvolver a aplicação. É necessário primeiro se certificar de que todas as bibliotecas e drivers funcionam corretamente antes de programar as funcionalidades do sistema.

Por fim, é necessário, nestes sistemas, que o programador se preocupe com as questões temporais. Os sistemas embarcados possuem informações que se alteram constantemente, mesmo sem a interferência do usuário. Além disso, algumas atividades possuem requisitos de execução que, se não atendidos, podem causar problemas, desde simples falhas no sistema até a destruição do equipamento.

Levando todas essas questões em consideração, o funcionamento do programa para um sistema embarcado é diferente do de um programa para desktop, para internet ou até mesmo para um celular. É necessário planejar o programa levando em conta todas essas peculiaridades.

Existem algumas técnicas/modelos que facilitam o desenvolvimento de programas para sistemas embarcados. Boa parte delas está relacionada a estruturas de máquinas de estado ou fluxogramas. O que todos esses modelos têm em comum é a implementação de um loop infinito e o conceito de evolução temporal do código.

10.1 Primeiro modelo: o loop infinito

Todo software embarcado possui ao menos um loop infinito. Este loop normalmente se localiza dentro da função **main()** como no código 10.1:

Código 10.1: O loop infinito de um sistema embarcado

```
1 //includes
2 void main (void){
3     //inicialização do sistema
4     for(;;){
5         //código a ser executado
6     }
7     //o sistema não deve chegar neste ponto
8 }
```

O código apresenta duas seções bem definidas: uma antes do loop, que será executada uma única vez assim que o sistema for ligado, e uma dentro do loop, que será executada indefinidamente enquanto o sistema estiver ligado.

Na inicialização da placa, devem-se configurar todos os periféricos que forem utilizados na aplicação, como timers, portas de I/O, comunicações seriais, entre outros. Deve-se tomar cuido com a inicialização dos periféricos externos ao chip. Estes periféricos podem possuir rotinas complexas de inicialização. Além disso, eles dependem de periféricos internos, de modo que estes têm que ser inicializados primeiro.

As funções inseridas no loop serão repetidas diversas vezes ao longo do tempo, enquanto o sistema estiver ligado. Devemos tomar o cuidado para lembrar do loop quando formos projetar essas funções. Se fizermos uma função que fique travada esperando algum comando, todas as demais funções do sistema vão parar de funcionar.

De fato, as funções projetadas para o loop infinito se baseam justamente no fato de que serão repetidas indefinidamente para funcionar. Por isso, não podemos criar funções que façam com que este fluxo de repetição seja bloqueado.

Esta abordagem é a mesma utilizada pelas plataformas Arduino e Chipkit. Ambas utilizam o framework Wiring, que implementa a função **main()** mas não a expõe para o programador. Ela fica escondida dentro das bibliotecas do framework. Em vez da função **main()**, ele apresenta duas outras funções: **loop()** e **setup()**.

A função **setup()** é responsável por fazer toda a inicialização do sistema, já a função loop será executada indefinidamente enquanto a placa estiver ligada. A função **main()** utilizada pelo Arduino é implementada da seguinte maneira:

```
 1 int main(void){
 2     init();
 3     initVariant();
 4 #if defined(USBCON)
 5         USBDevice.attach();
 6 #endif
 7     setup();
 8     for (;;) {
 9         loop();
10         if (serialEventRun) serialEventRun();
11     }
12     return 0;
13 }
```

As funções **init()**, **initVariables()** e **USBDevice.attach()** são responsáveis por inicializar as funções mínimas da placa e permitir que a placa receba o código novo a ser gravado. Depois ela chama a função **setup()**, que é responsabilidade do programador.

Dentro do loop infinito é executada a função **loop()**. Deste modo, todo o código implementado segue o fluxo temporal e é reexecutado continuamente. Já a função **serialEventRun()** apenas é executada se a placa tiver recebido alguma mensagem na serial. Isso simplifica o processo de comunicação serial para o programador.

Neste livro faremos uso das placas compatíveis com Wiring, mas vamos ignorar as funcionalidades deste framework, visto que o objetivo é justamente ensinar como programar um sistema embarcado pensando diretamente no hardware e nos periféricos.

10.2 A evolução do loop no tempo

Uma ideia pouco clara quando começamos a programar para sistemas embarcados é que, enquanto o loop infinito é executado, o mundo externo passa por mudanças. Assim, deve-se entender o loop não como uma simples execução cíclica de tarefas, mas como um bloco de comandos sendo executados em instantes diferentes no tempo. O loop infinito pode ser interpretado como um conjunto infinito de repetições do mesmo código, como podemos ver na figura 10.1:

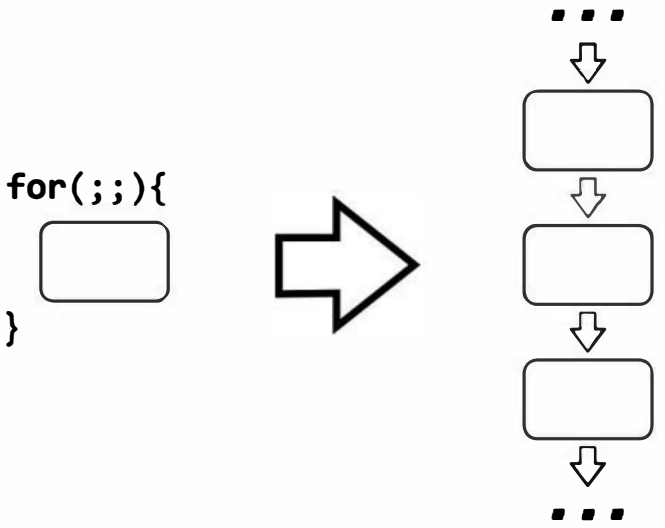


Figura 10.1: Interpretação de um loop infinito como uma sequência de comandos sequenciais

Quando criarmos um comando dentro do loop que verifica uma informação de teclado, por exemplo, devemos lembrar que essa verificação será reexecutada a cada rodada do loop. Por isso não convém ficar aguardando um determinado evento, podemos simplesmente perguntar se as condições para aquele evento são verdadeiras e, caso não sejam, esperamos uma próxima rodada do loop para perguntar novamente.

Vejamos o exemplo de relógio com um cronômetro de 60 segundos. O programa deve verificar se o botão de start foi pressionado para iniciar a contagem. Se durante a contagem o botão de reset for pressionado, o contador deve voltar para zero e parar. Neste exemplo utilizaremos quatro funções prontas: **`print()`**, que escreve um valor inteiro no LCD; **`getStartStatus()`**, que devolve o estado do botão start; **`getResetStatus()`**, que devolve o estado do botão reset; **`delaySecond()`**, que gera um atraso de 1 segundo. Segue uma primeira implementação desse algoritmo:

```
1 int cont = 0;
2 while (getStartStatus() == 0); //espera apertar o botão
3 for{cont = 0; cont<60;cont++){
4     lcdNumber(cont);
5     timerDelay(1000);
6     if (getResetStatus() == 1){
7         break;
8     }
9 }
```

Esta implementação realiza todas as operações necessárias seguindo a lógica de funcionamento de um cronômetro. O problema é que ela não é cíclica. Isto faz com que o

cronômetro funcione apenas uma vez e depois saia da rotina. A primeira providência a ser tomada é criar um loop infinito para que o programa funcione continuamente. Conforme o código 10.2:

Código 10.2: Código para controle de um cronômetro

```
1 int count = 0;
2 for(;;){
3     while (getStartStatus() == 0); //espera apertar o botão
4     for{cont = 0; count<60;cont++){
5         lcdNumber(cont);
6         timerDelay(1000);
7         if (getResetStatus() == 1){
8             break;
9         }
10    }
11 }
```

A transição de uma rotina normal para uma rotina cíclica, que possui um loop infinito, não pode ser feita do modo apresentado. Vários problemas podem aparecer por esse motivo.

Como o exemplo é simples, apenas um problema apareceu nesse caso: o estado do botão reset só é levado em consideração depois da rotina **timerDelay()**. Durante todo o tempo em que o programa estiver dentro da rotina **timerDelay()**, qualquer pressionamento do botão não resetará o valor da contagem. Para evitar esse problema, deveríamos testar o botão reset dentro da rotina **timerDelay()**.

Outra situação bastante comum é a criação de botões para controle de variáveis ou eventos, como, por exemplo, um botão de aumentar o volume. Supondo que tenhamos uma função **getVolumeUp()**, que retorna o valor **0** (zero) se o botão estiver solto e **1** se o botão estiver pressionado, poderíamos tentar criar o seguinte código para aumentar o volume:

```
1 unsigned int volume;
2 for(;;){
3     if (getVolumeUp() == 1){
4         volume++;
5     }
6     //restante do código
7 }
```

O código testa se a o botão de aumentar volume está pressionado e, se estiver, a variável volume é aumentada. No entanto, este código é executado dentro de um loop infinito. Isto faz com que a cada loop do código a variável seja aumentada em uma unidade. Mesmo nos sistemas embarcados mais simples, este loop pode executar dezenas ou centenas de vezes por segundo. Isto faz com que mesmo um pressionamento rápido da tecla, faça com que a variável seja aumentada em várias unidades.

Para evitar estes problemas, devemos planejar o desenvolvimento do software levando em conta as reexecuções do loop, lembrando que, para a maioria dos sistemas, a velocidade de execução do loop chega a centenas ou milhares de vezes por segundo.

Uma boa abordagem para iniciar o planejamento é desenhar o sistema explicitando as entradas e saídas do dispositivo, bem como as principais características do sistema a ser desenvolvido. Para um cronômetro, podemos chegar ao diagrama da figura 10.2:

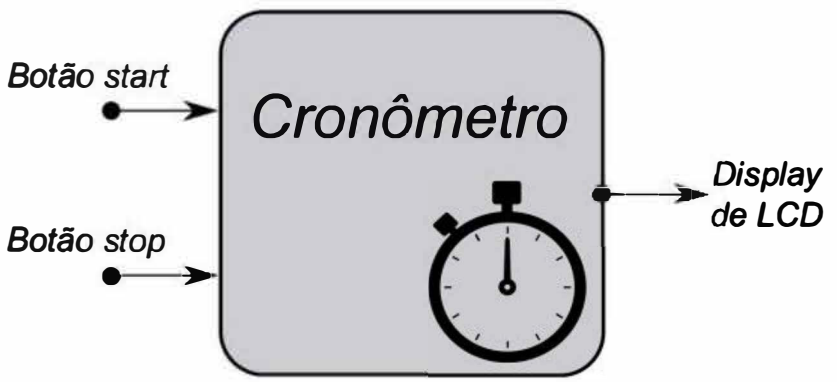


Figura 10.2: Diagrama de entradas e saídas de um cronômetro

De posse desse diagrama, devemos pensar quais funcionalidades o sistema deve executar. É importante também listar as atividades que devem ser executadas quando cada uma das entradas for ativada e se existem atividades que serão executadas em condições específicas.

Uma boa abordagem para a implementação do código é montar um ciclo que lerá todas as entradas definidas no diagrama, verificar se existe alguma ação a ser realizada e atualizar as saídas.

No caso do cronômetro, a principal funcionalidade é realizar a contagem do tempo. Com relação à lista de atividades, temos:

- Habilitar a contagem se o botão **`start`** for pressionado.
- Reiniciar a contagem se o botão **`reset`** for pressionado.
- Quando a contagem chegar em 60 o cronômetro deve parar.

Isto pode ser transcrito para programação como apresentado no código 10.3:

Código 10.3: Código para controle de um cronômetro explicitando as entradas e saídas

```
1 for(;;){
2     //*** leitura das entradas e tomada de decisão
3     if(getStartStatus()==1){
4         enable_counting = 1;
5     }
6     if(getResetStatus()==1){
7         count = 0;
8         enable_counting = 0;
9     }
10    //*** processamento das funcionalidades
11    //contagem do tempo
12    if(enable_counting == 1){
13        count++;
14    }
15    if(count >= 60){
16        count = 0;
17        enable_counting = 0;
18    }
19    timerDelay(1000);
20
21    //*** acionamento das saídas
22    lcdNumber(cont);
23 }
```

O código 10.3 é bastante diferente do código 10.2, apresentado inicialmente para controlar o cronometro. Devemos olhar o código como um conjunto de atividades que será reexecutado sequencialmente enquanto o sistema estiver ligado. Assim, a cada ciclo do loop verificamos se houve pressionamento de algum dos botões, fazemos a contagem, se necessário, e atualizamos as saídas. Nenhuma atividade é deixada de lado enquanto estamos verificando alguma entrada ou aguardando algum evento.

CAPÍTULO

11

Debug de sistemas embarcados

11.1 Externalizar as informações

Usando os terminais de entrada e saída

11.2 Programação incremental

11.3 Cuidado com a otimização de código

11.4 Reproduzir e isolar o erro

11.5 Crie rotinas de teste

11.6 Criação de uma biblioteca para debug

"Na mente do iniciante existem muitas possibilidades; na do especialista existem poucas."

Shunryu Suzuki

Quando se encontra um erro no funcionamento de um programa, chamamos esse erro de "bug", inseto em português. O procedimento de remover os bugs do programa é conhecido como "debug".

Como os sistemas embarcados geralmente apresentam restrições físicas e computacionais, é difícil visualizar o que está acontecendo dentro do sistema sem alterar o seu funcionamento ou até mesmo pausá-lo. Este tipo de sistema possui ainda vários dispositivos agregados ao chip e à placa que funcionam independentemente do processador. Isto dificulta a compreensão do que está acontecendo.

Mesmo que o software esteja correto, é possível que problemas no hardware atrapalharem o funcionamento do sistema. Bouncing, tempo de chaveamento de transistores, velocidade na transmissão de dados e interferências eletromagnéticas são exemplos de problemas em hardware que podem levar o programador a pensar que o software não está funcionando bem.

Com todas essas questões, o processo de debug de sistemas embarcados pode ser mais complexo que sistemas comuns. Neste capítulo serão abordadas ferramentas e conceitos que podem ajudar o programador a procurar os erros e consertá-los.

11.1 Externalizar as informações

A primeira necessidade é saber o que está acontecendo dentro do sistema. Na programação em computadores tradicionais é comum imprimirmos mensagens na tela que notificam o que o programa está fazendo.

```
1 #include "stdio.h"
2 #include "serial.h"
3 //início do programa
4 int main(int argc, char* argv[]){
5     int resp;
6     printf("Inicializando sistema");
7     a = RecebeCaracter();
8     if (a == NULL){
9         printf("Debug: caracter com problema");
10    } else {
11        printf("Debug: caracter lido corretamente");
12    }
13    return 0;
14 }
```

Estas mensagens servem como uma informação quando algo não funciona, ou podem ser utilizadas simplesmente para informar se o sistema está funcionando corretamente. Um ponto importante é lembrar de remover esses alertas quando o programa estiver pronto.

Quando não temos uma tela disponível, podemos fazer uso de portas de comunicação serial.

A comunicação serial permite que o programador realize um comando de impressão, como um **`printf()`**, por exemplo, mas, em vez de ser exibido numa tela ligada ao sistema embarcado, o texto será enviado para um computador, que fará a leitura e o exibirá na sua tela. É possível também enviar comandos do computador para o sistema embarcado se algo der errado.

Em vários sistemas, no entanto, não temos uma tela para exibir as mensagens nem um caminho de comunicação serial. É possível também que, mesmo tendo uma comunicação serial disponível, não queiramos utilizá-la, uma vez que a impressão na serial pode atrasar o funcionamento do código. Nestas situações, podemos utilizar um LED como indicador.

```
1 //início do programa
2
3 void main(void) {
4     int lastKey;
5     //configurando o teclado
6     kpInit();
7     LigaLed(1);
8     serialInit();
9     bufferInit();
10    LigaLed(2);
11    for (;;) {
12        TrocaLed(3);
13        kpDebounce();
14        Processing();
15        lastKey = kpRead();
16        ExecuteCommand(lastKey);
17    }
18 }
```

O código acima utiliza 3 leds para indicar o andamento do programa. Os dois primeiros acendem à medida que os dispositivos forem inicializados. Se algum led não acender, quer dizer que houve um erro e o programa travou. Se o led 1 não acender, é provável que a função **`InicializaTeclado()`** não tenha executado corretamente. Já o led 2 indica que as funções de inicialização da serial e do buffer terminaram.

Caso o segundo led não acenda, alguma das duas funções está com problema. A primeira providência seria deslocar a função que liga o led 1 para depois da função de inicialização da serial. Assim podemos verificar se o erro está na serial ou no buffer.

```
 1
 2 void main(void) {
 3     int lastKey;
 4     //configurando o teclado
 5     kpInit();
 6     serialInit();
 7     LigaLed(1);       //Mudado de posição
 8     bufferInit();
 9     LigaLed(2);
10     for (;;) {
11         //código não foi alterado
12     }
13 }
```

Como o led 1 foi deslocado, se ele acender, sabemos que o erro está na inicialização do buffer, se não acender, o erro provavelmente está na função de inicialização da serial. Lembrando que isto só é verdade se tivermos certeza que a inicialização do teclado está correta. Como no primeiro teste o led havia acendido, essa afirmação é coerente.

O último led realiza duas funções: ele indica que o programa conseguiu entrar no loop; e troca de valor a cada vez que o loop é executado. Em geral, a velocidade com que ele pisca é muito alta, não sendo possível ser visto a olho nu. Uma opção interessante é utilizar um osciloscópio para ver o sinal elétrico. Ler a velocidade com que o led está piscando e verificar se o sistema está funcionando de acordo com o esperado.

Esse procedimento de externalizar as informações também é conhecido como instrumentação do código, ou seja, adicionamos instrumentos (exibição de texto, leds, etc.) para indicar o bom andamento da execução do código.

Usando os terminais de entrada e saída

A estrutura mais simples para externalizar informações é a utilização de terminais do microcontrolador. No primeiro exemplo, foram utilizados 3 leds para apresentar o estado da placa: dois para inicialização e um para a execução do loop principal. No entanto, o desenvolvedor pode precisar externalizar mais informações do que apenas 3 estados. A solução mais simples é utilizar mais terminais, o que pode ser inviável do ponto de vista do projeto de hardware.

A segunda solução é utilizar todos os pinos disponíveis como um conjunto de informações, e não como bits isolados.

No exemplo, temos três bits disponíveis e, se utilizados em conjunto, podem apresentar até 8 estados diferentes: **`000`**, **`001`**, **`010`**, **`011`**, **`100`**, **`101`**, **`110`** e **`111`**. Neste caso, a junção dos

valores dos terminais indicará a informação desejada. Cada estado será responsável por passar uma mensagem, e, assim, temos 8 possíveis combinações para apresentar uma informação, em vez de apenas 3 como no exemplo original. O código **000**, por não apresentar nenhuma mudança na saída dos terminais, pode ser escolhido como representação de que nenhum código foi enviado.

Se utilizarmos um led RGB, estes estados serão visualizados na placa através de diferentes cores, como apresentado na tabela 11.1:

Tabela 11.1: Cor visualizada através da combinação de um led RGB

R	G	B	Cor visualizada
0	0	0	Apagado
0	0	1	Azul
0	1	0	Verde
0	1	1	Ciano/Azul claro
1	0	0	Vermelho
1	0	1	Rosa
1	1	0	Amarelo
1	1	1	Branco

Pensando nos três terminais, podemos criar um conjunto de macros para facilitar a utilização do debug. O primeiro conjunto de macros serve para definir quais terminais são utilizados. No exemplo, criaremos macros para os três leds.

```
1 //definindo 1o pino de debug (red)
2 #define DEBUG_PIN_1 2
3 //definindo 1o pino de debug (green)
4 #define DEBUG_PIN_2 3
5 //definindo 1o pino de debug (blue)
6 #define DEBUG_PIN_3 4
```

Num segundo momento, definimos quais são os códigos para cada estado, de acordo com as definições apresentadas pela tabela 11.1.

```
1 #define DEBUG_NO_MSG 0b000
2 #define DEBUG_BLUE   0b001
3 #define DEBUG_GREEN  0b010
4 #define DEBUG_CYAN   0b011
5 #define DEBUG_RED    0b100
6 #define DEBUG_PINK   0b101
7 #define DEBUG_YELLOW 0b110
8 #define DEBUG_WHITE  0b111
```

Por fim, criamos uma função que será responsável por atualizar as saídas de acordo com o código desejado:

```
#ifdef DEBUG
    #define debugPrint(a)    debugPrintFunction(a)
#else
    #define debugPrint(a)
#endif

void debugPrintFunction(int debugState){
    if(debugState & 0x01) {
        DEBUG_PIN_1 = 1;
    } else {
        DEBUG_PIN_1 = 0;
    }
    if(debugState & 0x02) {
        DEBUG_PIN_2 = 1;
    } else {
        DEBUG_PIN_2 = 0;
    }
    if(debugState & 0x04) {
        DEBUG_PIN_3 = 1;
    } else {
        DEBUG_PIN_3 = 0;
    }
}
```

A função **debugPrint()** será implementada apenas se o label **DEBUG** for definido. Caso contrário, todas as chamadas da função serão removidas do código.

Como agora temos 7 diferentes estados, podemos instrumentar o código de modo mais preciso:

```
1 #define DEBUG
2 void main(void) {
3     //configurando todos os pinos como saídas
4     kbInit();
5     debugPrint(DEBUG_BLUE);
6     serialInit();
7     debugPrint(DEBUG_GREEN);
8     bufferInit();
9     debugPrint(DEBUG_YELLOW);
10    for (;;) {
11        debugPrint(DEBUG_RED);
12        kbDebounce();
13        debugPrint(DEBUG_CYAN);
14        Processing();
15        lastKey = readKey();
16        if (lastKey > 10){
17            debugPrint(DEBUG_WHITE);
18        }
19        ExecuteCommand(lastKey);
20    }
21 }
```

Com o aumento de estados, podemos monitorar cada passo do código, verificando se todos os casos foram executados corretamente. Quando o programa estiver concluído, basta remover a diretiva **`#define DEBUG`** que todas as funções de debug não serão compiladas.

Por fim, uma boa prática é deixar disponível na placa um conector com um conjunto de terminais que poderão ser utilizados para depuração. Isto permite conectar a placa rapidamente a um osciloscópio ou analisador lógico.

11.2 Programação incremental

Uma boa técnica de programação é desenvolver o menor código possível, mesmo que ele não tenha todas as funcionalidades no início. Esse código será a base do desenvolvimento.

Partindo desse código, devemos adicionar funcionalidades e testar o sistema de modo gradual.

Outro ponto importante é não alterar regiões diferentes do código ao mesmo tempo. Concentre a alteração em um único arquivo ou função por vez. Assim, se alguma coisa não funcionar, é provável que o erro se encontre perto de onde as alterações foram feitas. Neste caso, basta instrumentar o código, utilizando leds ou displays para verificar qual linha não está executando, ou que está executando erroneamente.

11.3 Cuidado com a otimização de código

Entende-se por otimização o processo de modificar o código rearranjando a sequência ou estrutura dos comandos, de modo que ele seja executado mais rapidamente. Isto é feito, em geral, quando o sistema não consegue processar todas as atividades programadas no tempo aceitável para a aplicação.

A otimização deve ser o último recurso. Existem algumas abordagens que devem ser utilizadas antes. Primeiro verifique se é possível aumentar o clock do processador. Grande parte dos microcontroladores permite que o programador altere a frequência de trabalho; outros precisam da troca do cristal de oscilação. Em ambos os casos, o custo dessa troca é zero. Trabalhar com clock mais alto pode gerar dois tipos de problema: maior consumo de energia, situação crítica para sistemas que operam com bateria, e aumento de ruído nos outros sinais da placa. Isto pode ser minimizado com um layout de trilhas adequado.

Um segundo recurso bastante interessante é tentar entender melhor o problema que se está tentando resolver e verificar se não existem algoritmos mais adequados para serem utilizados nessa etapa. Um exemplo são os algoritmos de ordenação de números numa lista. Há diversos algoritmos, alguns mais simples, que demoram mais tempo para terminar, e outros mais rebuscados, que terminam a mesma atividade em menos tempo. A área da computação que analisa os algoritmos neste sentido chama-se complexidade de algoritmo.

Se mesmo assim, for necessário otimizar o código, verifique primeiro onde realizar a otimização. Coloque informações de debug em pontos definidos do código e verifique qual região consome mais tempo.

Uma função grande com muito código, gasta mais tempo para ser executada. No entanto, mesmo que ela demore muito tempo para terminar, ela pode não ser o problema do sistema. Uma função menor, que é chamada com maior frequência, pode ser a maior responsável pelo consumo de tempo de processamento. Focar na função ou região crítica, é fundamental para conseguir uma boa economia de tempo.

Outra opção é utilizar uma ferramenta de profiler. Os profilers instrumentam o código automaticamente e devolvem informações sobre qual processo está consumindo mais recursos de processamento.

11.4 Reproduzir e isolar o erro

Quando um erro for percebido ou informado, o primeiro passo é conseguir reproduzi-lo. Tente criar um pequeno guia que explique passo a passo quais ações devem ser feitas, e em que ordem, para que o erro aconteça.

A reprodução permite que o programador entenda melhor onde está acontecendo o erro e corrija o código de maneira eficiente. Depois de resolvido o erro, podemos usar o guia para ver se a correção funcionou.

Se não conseguirmos reproduzir o erro de modo consistente, é mais difícil garantir que o erro foi eliminado.

No processo de instrumentação do código, podemos colocar um loop infinito dentro de uma condição de teste para que, se acontecer alguma coisa que não era esperada, o sistema trave naquele ponto e indique o problema.

```
1     // começo do trecho de debug
2     if (tecla >= 10){       //a variável tecla deveria ir só até 9.
3         LigaLed(3);         //liga o led 3
4         for(;;);            //trava o programa
5     }
6     // fim do trecho de debug
```

11.5 Crie rotinas de teste

Rotinas de testes são funções ou procedimentos que visam verificar se um código está funcionando corretamente. A melhor maneira de se verificar se uma função está correta é passar um valor conhecido para ela e verificar se o resultado bate com o esperado.

```
1 int powerConsumption(int current, int voltage){
2     //realiza o calculo de potencia
3 }
```

A função **powerConsumption()** recebe os valores de corrente e tensão de um sensor e faz o cálculo da potência consumida. Nesta caso, devemos elencar alguns testes que conhecemos os resultados e verificar se eles estão corretos.

Entre os valores que serão utilizados como parâmetros da função, devemos usar aqueles que tendem a verificar situações extremas. No caso do cálculo de potência, podemos utilizar valores negativos e verificar se o resultado ainda é maior que zero, já que não faz sentido obtermos consumo de potência negativa em equipamentos sem geração de energia.

```
1 void tests (void){
2     if (powerConsumption ( 0, 0) != 0) { printf("erro"); }
3     if (powerConsumption ( 0, 1) != 0) { printf("erro"); }
4     if (powerConsumption ( 1, 0) != 0) { printf("erro"); }
5     if (powerConsumption ( 1, 1) != 1) { printf("erro"); }
6     if (powerConsumption (-1, 1)  < 0) { printf("erro"); }
7     if (powerConsumption ( 1,-1)  < 0) { printf("erro"); }
8     if (powerConsumption (-1,-1)  < 0) { printf("erro"); }
9 }
```

Ao longo do desenvolvimento, a função **powerConsumption()** pode ser alterada para utilizar outro algoritmo que seja mais rápido ou mais preciso. Nestas alterações, é possível que algum erro apareça. Manter uma rotina de testes ajuda a detectar esse tipo de erro.

A linguagem C possui uma biblioteca desenvolvida com esse intuito: a **assert.h**. Essa biblioteca implementa apenas uma função e uma macro.

A macro **assert()** realiza um teste se o parâmetro enviado é verdadeiro. Se o resultado estiver errado, ela executa a função **_assert()**, que imprime o nome do arquivo e o número da linha em que o erro aconteceu, além de travar o programa num loop infinito.

```
1 #define assert(x) ((x) == 0 ? _assert(#x, __FILE__, __LINE__):(void)0)
2
3 void _assert(char *expr, const char *filename, unsigned int linenumber)
4 {
5     printf("Assert(%s) failed at line %u in file %s.\n",
6     expr, linenumber, filename);
7     while(1);
8 }
```

Como apresentado anteriormente, alguns sistemas embarcados não possuem saídas de texto, deste modo seria interessante reescrever a função para imprimir um código de erro. Para isso, podemos utilizar os códigos de acesso aos leds, por exemplo, e reescrever a macro **assert**, como no código a seguir.

```
1 #define assert(condition,errCode) ((condition) == 0 ? debugPrint(errCode):(void)0)
2 #define DEBUG
3 void main(void) {
4     kbInit();
5     for (;;) {
6         kbDebounce();
7         lastKey = kbRead();
8         assert(lastKey > 10, DEBUG_WHITE);
9         //executa código baseado na tecla pressionada
10        ExecuteCommand(lastKey);
11    }
12 }
```

No exemplo dado, se em algum momento a tecla pressionada for maior que 10, o programa indicará um erro.

11.6 Criação de uma biblioteca para debug

Como cada placa possui um conjunto de particularidades, uma solução é reunir num único lugar todas as macros e funções relacionadas ao processo de depuração. Para unificar o

acesso aos terminais físicos, é utilizada a biblioteca **io.h**, que reúne as informações dos periféricos.

```
1 //Arquivo: Debug.h
2 //macros para exibição dos códigos e para testes
3
4 #include "io.h"
5
6 //utiliza-se o #ifdef pois assim o programador pode remover todo o sistema de ↩
      debug de uma única vez
7 #ifdef DEBUG
8     #define debugPrint(a)        debugPrintFunction(a)
9     #define assert(cond,errCode) ((cond) == 0 ? debugPrint(errCode):(void)0)
10 #else
11     #define debugPrint(a)        ((void)0)
12     #define assert(cond,errCode) ((void)0)
13 #endif
14
15 void debugPrintFunction(int debugState);
16
17 //códigos de debug
18 #define DEBUG_NO_MSG  0b000
19 #define DEBUG_MSG_1   0b001
20 #define DEBUG_MSG_2   0b010
21 #define DEBUG_MSG_3   0b011
22 #define DEBUG_MSG_4   0b100
23 #define DEBUG_MSG_5   0b101
24 #define DEBUG_MSG_6   0b110
25 #define DEBUG_MSG_7   0b111
26
27 //definindo os terminais disponíveis para debug a partir dos defines do arquivo io.h
28 #define DEBUG_PIN_1 PIN_LED_BLUE
29 #define DEBUG_PIN_2 PIN_LED_GREEN
30 #define DEBUG_PIN_3 PIN_LED_RED
```

No arquivo de código, apenas a função de debug é implementada. Ela utiliza as definições dos pinos de saída para gerar o código binário que será apresentado nos leds.

```
1 //Arquivo: debug.c
2 #include "debug.h"
3
4 //implementação da função de impressão
5 void debugPrintFunction(int debugState){
6     if(debugState & 0x01){
7         digitalWrite(DEBUG_PIN_1,HIGH);
8     }else{
9         digitalWrite(DEBUG_PIN_1,LOW);
10    }
11    if(debugState & 0x02){
12        digitalWrite(DEBUG_PIN_2,HIGH);
13    }else{
14        digitalWrite(DEBUG_PIN_2,LOW);
15    }
16    if(debugState & 0x04){
17        digitalWrite(DEBUG_PIN_3,HIGH);
18    }else{
19        digitalWrite(DEBUG_PIN_3,LOW);
20    }
21 }
```

Parte II

Controlando periféricos de sistemas embarcados

12 ---Introdução a microcontroladores
13 ---Programação dos periféricos
14 ---Saídas digitais
15 ---Display de 7 segmentos
16 ---Entradas digitais
17 ---Display LCD
18 ---Comunicação serial
19 ---Conversor analógico digital
20 ---Saídas PWM
21 ---Temporizadores
22 ---Interrupção
23 ---Watchdog

CAPÍTULO

12

Introdução a microcontroladores

12.1A unidade de processamento

12.2Memória

12.3Mapeando periféricos na memória

12.4Clock e tempo de instrução

12.5Microcontroladores

Atmel ATMega328

NXP KL05z

Microchip PIC32MX320F128

12.6Registros de configuração do microcontrolador

12.7Requisitos elétricos do microcontrolador

12.8Exercícios

"As pessoas que realmente levam a sério o software devem fazer o seu próprio hardware."

Alan Kay

A maioria dos circuitos digitais é desenvolvida de modo a realizar uma operação específica. O tamanho destes circuitos depende das funcionalidades a ele agregadas. Desenvolver um novo circuito dedicado a cada atividade é um problema que consome muito tempo de projeto e o produto final, geralmente um chip, não pode ser reutilizado em outros projetos. Em alguns casos, eles podem se tornar inviáveis tecnicamente devido à grande quantidade de transistores.

Tentando resolver esses problemas e gerar um chip que pudesse ser utilizado em diversos projetos, sem que fosse necessário reconstruir o hardware, o engenheiro Frederico Faggin, da Intel, desenvolveu um circuito que possuía um conjunto de operações que poderiam ser executadas em uma sequência definida por um roteiro armazenado numa memória. Passava-se de um circuito combinacional, em que o valor de saída só depende dos valores das entradas, para um circuito sequencial, em que a saída depende das entradas, como também das operações realizadas em passos anteriores.

O primeiro produto desenvolvido com essa tecnologia foi a calculadora BUSICOM 141-PF, que utilizava o chip 4004, o primeiro processador comercial. A figura 12.1 apresenta este processador em sua primeira versão produzida:

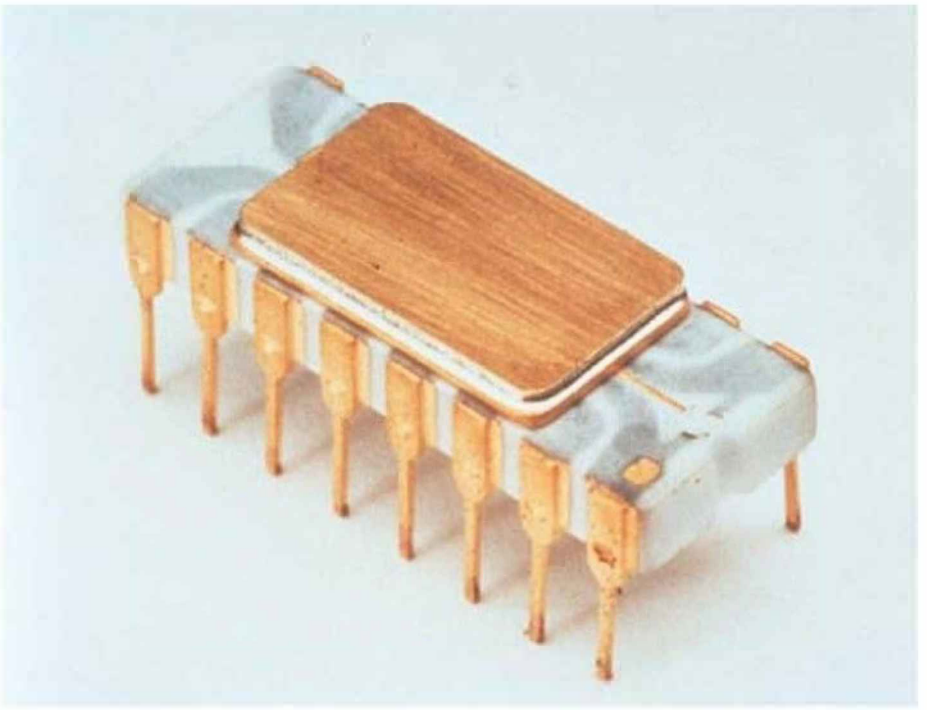


Figura 12.1: Microprocessador 4004 (Fonte: Intel)

O microprocessador 4004 foi lançado em 1971, possuindo uma arquitetura de 4 bits e 45 instruções diferentes. Sua frequência de operação máxima era de 740kHz, chegando ao número de até 92.600 instruções por segundo.

No entanto, apenas com o microprocessador não era possivel desenvolver um produto completo. Eram necessários outros circuitos para que o processador pudesse interagir com o meio externo. Em conjunto com o 4004 foram desenvolvidos mais três circuitos auxiliares: uma memória ROM de 2048 bits (4001), uma memória RAM de 320 bits (4002) e um registrador de deslocamento de 10 bits (4003), utilizado para realizar a interface com dispositivos externos.

O problema com a utilização de diversos circuitos separados é que a construção de uma placa acaba sendo mais complexa. Visando reduzir essa complexidade, várias empresas começaram a incluir dentro de um mesmo chip tanto o processador quanto a memória e até alguns periféricos de entrada e saída. Estes chips são chamados de microcontroladores, pois, além de possuir as estruturas computacionais, também conseguem realizar o controle de elementos externos.

Em geral, os periféricos de entrada e saída são tratados da mesma forma que a memória. Isso quer dizer que, para o processador, não existe diferença se estamos tratando com um valor guardado na memória RAM ou com valores externos de chaves ou leds.

Isto é possível porque existem circuitos eletrônicos que criam essa abstração em hardware fazendo com que todos os dispositivos apareçam como endereços de memória. Dizemos, nestes casos, que os dispositivos estão mapeados na memória. A figura 12.2 apresenta este modelo. Podemos notar que as memórias e os periféricos estão todos conectados no mesmo barramento interno.

Como todos os dispositivos utilizam um único barramento de dados para enviar e receber mensagens, é necessário que alguém controle o acesso. Há diversos modos de fazer esse

controle e um dos mais simples é fornecer autorização ao dispositivo através de um outro barramento, conhecido como barramento de endereço.

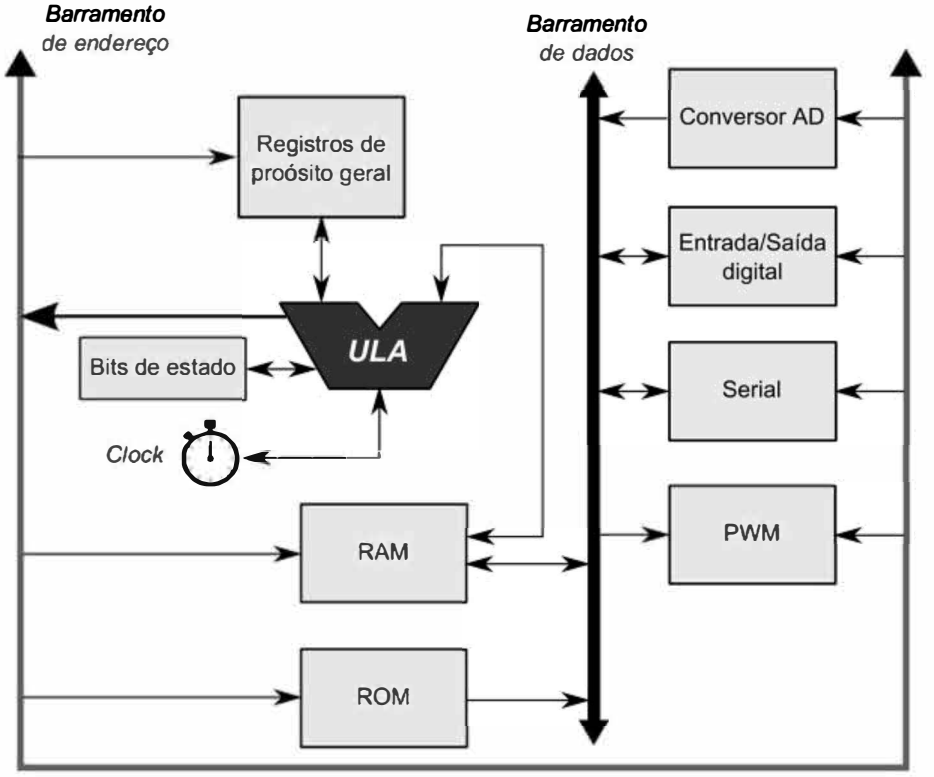


Figura 12.2: Arquitetura interna de um microcontrolador genérico

Esse segundo barramento é o responsável por indicar quem deve ler ou escrever no barramento de dados. Em geral, apenas a unidade de processamento (ULA) possui controle sobre esse barramento. Assim sendo, os acessos são todos organizados por ela.

Por fim, é necessário um sistema de geração de clock para sincronizar os eventos e permitir que a unidade de processamento execute suas tarefas.

12.1 A unidade de processamento

A unidade de processamento, ou processador, é o centro de um microcontrolador. É ela que coordena, executa e gerencia todos os recursos disponíveis.

O processador é um circuito digital sequencial. Isto quer dizer que o estado atual e os dados de entrada definem o próximo estado em que o circuito se encontrará. A mudança entre um estado para o próximo é comandada por um circuito de sincronismo, também chamado de clock. O tempo entre dois estados é chamado de ciclo.

O roteiro com as entradas, que indicarão ao processador o que ele deve fazer, são listadas em um conjunto de comandos. Cada comando é chamado de instrução, sendo que cada instrução pode estar relacionada a mais de uma operação a ser efetuada pelo microprocessador. O tempo que o processador leva para executar uma instrução é medido pela quantidade de ciclos necessários para terminar a instrução, bem como o tempo de cada ciclo. Alguns comandos podem realizar operações com valores armazenados na memória. Esses valores são também chamados de dados.

A figura 12.3 apresenta o modelo resumido do funcionamento de um processador:

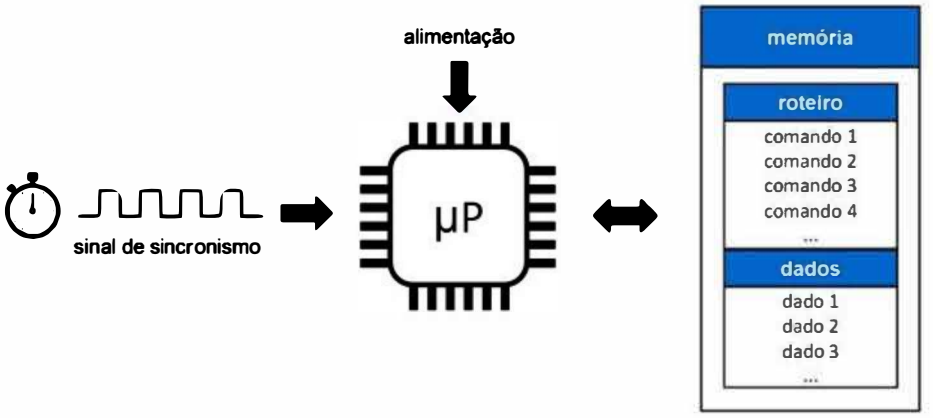


Figura 12.3: Mapeamento de tarefa para operação

Com relação ao modo de funcionamento, a grande maioria dos processadores atuais se divide em duas arquiteturas: Harvard e Von Neumman.

Na arquitetura Von Neumman existe apenas um barramento de dados. O barramento de dados é o caminho pelo qual os valores são transportados. Por haver apenas um barramento, os dados e as instruções do programa fazem o mesmo percurso para chegar ao processador, conforme a figura 12.4.

Nos processadores Von Neumman, as memórias são diferenciadas através do barramento de endereços. A memória de programas e a memória de dados recebem, cada uma, uma faixa de valores distintos. Uma opção bastante comum é reservar os primeiros endereços para a memória de dados e os últimos para a memória de programa. Isto no entanto, é uma opção do desenvolvedor do microcontrolador.

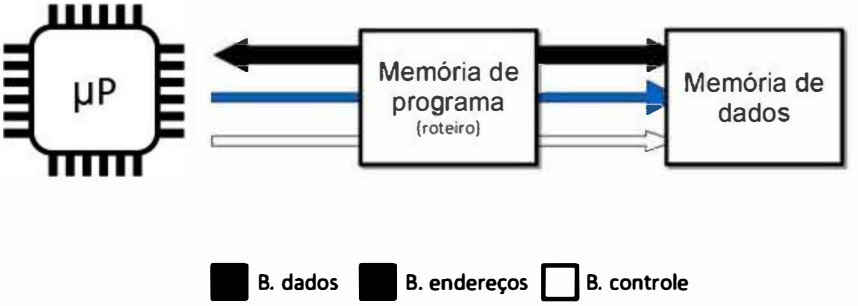


Figura 12.4: Arquitetura de processador Von Neumman

O projetista do chip insere um hardware dedicado que verifica qual é o endereço que está chegando para a memória e, estando de acordo, ele habilita ou não o envio de informações entre a memória e o barramento de dados.

Na arquitetura Harvard, os barramentos são separados. Neste caso, a memória de dados pode ser acessada independentemente da memória de programa. Isso permite executar duas

ações ao mesmo tempo, dado que os barramentos são distintos, como apresentado na figura 1 2.5:

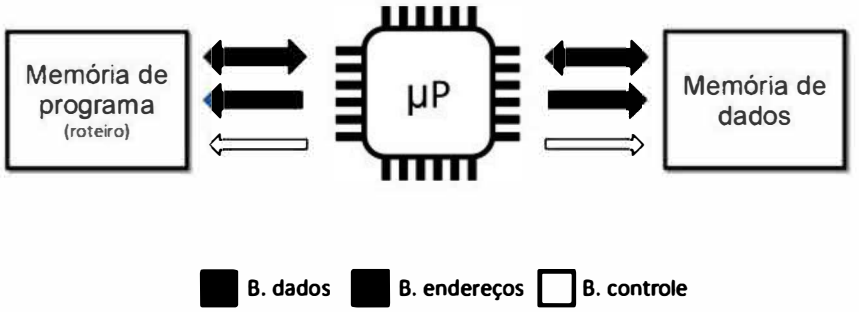


Figura 12.5: Arquitetura de processador Harvard

Algumas arquiteturas mais novas possuem elementos híbridos, com uma arquitetura externa do tipo Von Neumman mas possuindo memórias internas com barramentos dedicados. Com relação ao funcionamento interno, o processador pode ser apresentado como uma estrutura de quatro unidades: execução, controle, decodificação e controle de barramento.

A unidade de controle do barramento é a responsável por gerenciar todas as interações de informação entre os barramentos e as unidades internas. É ela a responsável por buscar as instruções na memória de programa e ler ou armazenar valores nas memórias de dados.

A unidade de decodificação é a responsável por interpretar as informações recebidas no barramento de dados e indicar às demais unidades qual atividade deve ser realizada.

Na unidade de execução é onde as atividades decodificadas são efetivamente executadas. Ela processa os comandos que chegam pelo barramento e o resultado dessas operações pode modificar os registros de operação. Os registros, no entanto, armazenam os resultados das operações de modo temporário. Se for desejado pelo programador, a unidade de execução pode salvar estes resultados novamente na memória, de modo que não se percam.

Por fim, a unidade de controle é a responsável por realizar o sincronismo entre todas as atividades das demais unidades. A velocidade com que o processador trabalha é definida por ela. Dentro desta unidade é o lugar onde o clock é definido. Alguns microcontroladores podem gerar o sinal de sincronismo, ou clock, dentro da própria unidade de controle.

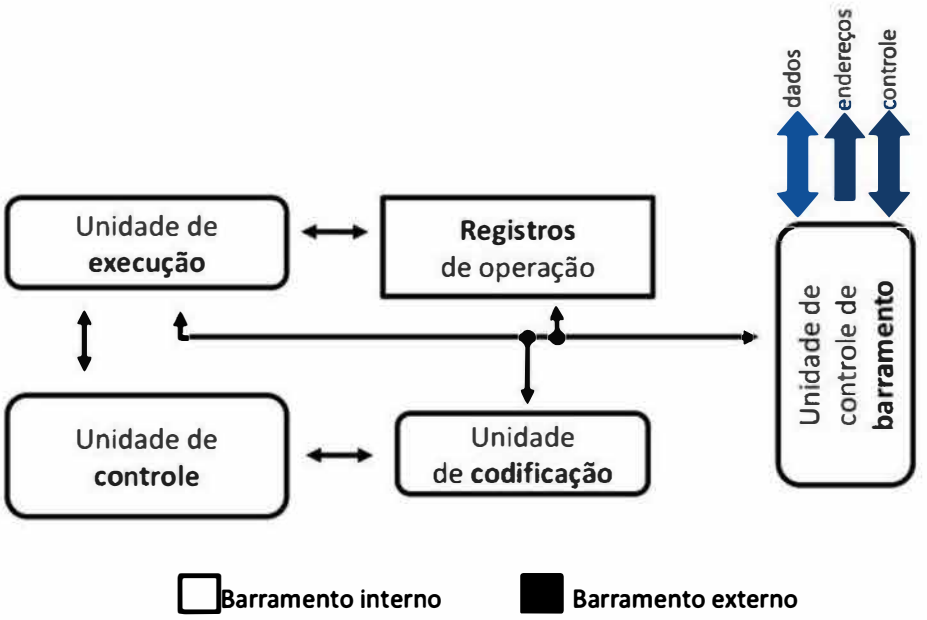


Figura 12.6: Organização básica de um processador

Os registros de operação possuem diferentes funções. Existem pelo menos 4 tipos distintos de registos, como apresentado na figura 12.7.

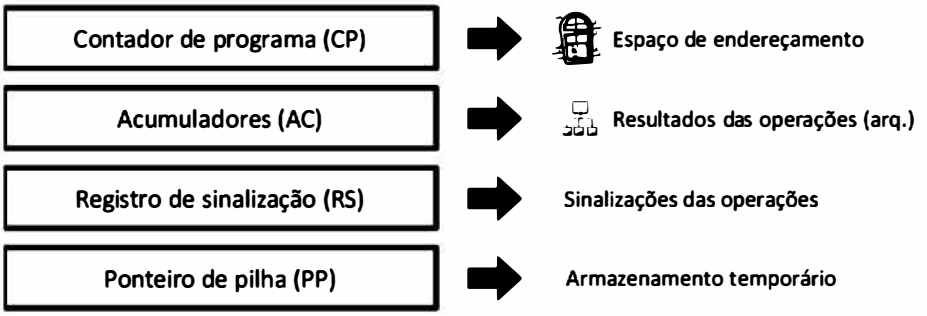


Figura 12.7: Registros internos de um processador

O contador de programa tem como responsabilidade indicar em qual região da memória o processador deverá realizar a leitura dos dados de programa. Modificando este contador, é possível controlar qual parte do código será executada. Este comportamento é utilizado, por exemplo, na instrução da linguagem C **`if`**, para definir qual de dois blocos de comandos será executado.

O registro de sinalização serve para indicar qual a condição atual do processador, de acordo com as operações realizadas até o momento. Entre os sinais monitorados por ele podemos ter a indicação se o resultado da última operação, foi negativo, zero ou positivo, se as interrupções estão habilitadas, se houve algum problema de estouro, ou overflow, em alguma operação matemática, entre outras informações que o projetista do processador julgou necessárias.

O ponteiro de pilha funciona como um indicador de memória. Seu funcionamento é muito similar ao de um ponteiro. Através dele é possível acessar um outro endereço de memória. A

diferença é que as instruções de leitura ou escrita através do ponteiro de pilha são acompanhadas por uma mudança de endereço. Isto facilita o armazenamento de valores de modo temporário e permite criar uma estrutura para chamadas de função ou interrupções de maneira bastante simples e eficiente.

Por fim, os acumuladores são os registros que armazenam os dados que serão utilizados nos cálculos. Em geral, são estes registros que definem o tamanho de bits do processador.

Para fazer leituras ou escrita de informações na memória, o processador utiliza três barramentos, conforme apresentado na figura 12.8.

O primeiro deles é o barramento de dados. É por meio desse barramento que as informações de dados, ou do programa, são transferidas da memória para o processador, ou do processador para a memória.

O barramento de endereço serve para indicar a posição de memória que o processador está acessando no momento. Quando utilizamos ponteiros na linguagem C, o valor de endereço armazenado no ponteiro será utilizado no barramento de endereços para que a memória correspondente envie seu valor pelo barramento de dados.

Assim, o barramento de controle é o responsável por controlar as memórias. Num microcontrolador convencional existem várias memórias distintas, por este motivo é necessário controlar quais delas estão ativas para um determinado endereço.

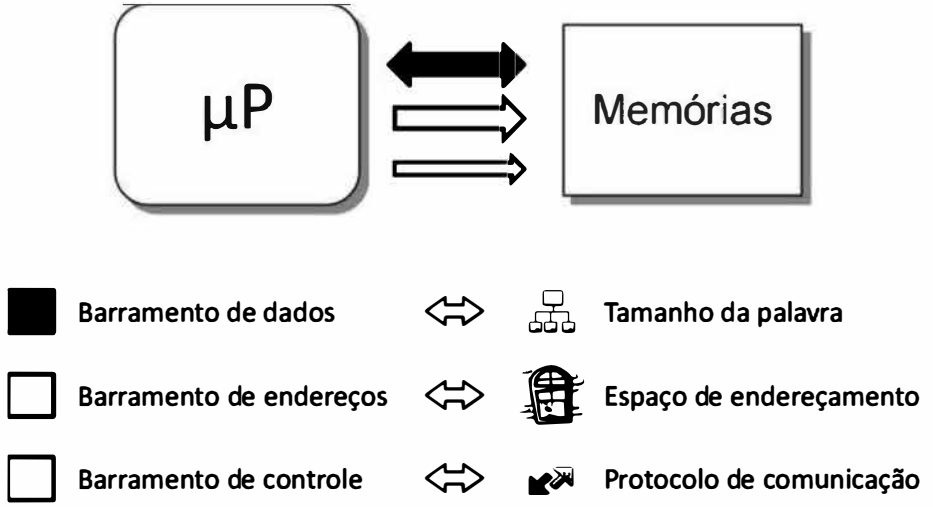


Figura 12.8: Barramentos em um processador

Os barramentos, a princípio, são estruturas bastante simples, formados por conjuntos de trilhas internas que, no entanto, definem muito da arquitetura do sistema. O barramento de dados indica o tamanho da palavra que o processador consegue operar na memória. Já o barramento de endereços define o espaço de endereçamento; em termos práticos, a quantidade máxima de memória com que o processador consegue trabalhar. Desta maneira, o barramento de controle define o protocolo de comunicação lógico e elétrico entre os dispositivos, bem como os ciclos de escrita e leitura de dados. Estes barramentos definem, portanto, a forma de se comunicar com o microprocessador.

A figura 12.9 mostra como podemos identificar o espaço de memória observável pelo processador através da quantidade de bits do barramento de endereço.

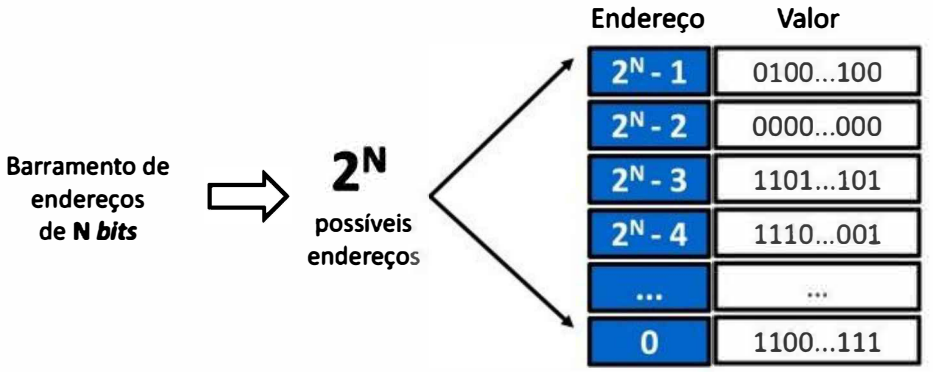


Figura 12.9: Espaço de endereçamento

12.2 Memória

A quantidade de memória disponível que um microcontrolador pode acessar depende basicamente do tamanho do barramento de endereço. Este tamanho indica quantas posições de memória o processador consegue endereçar.

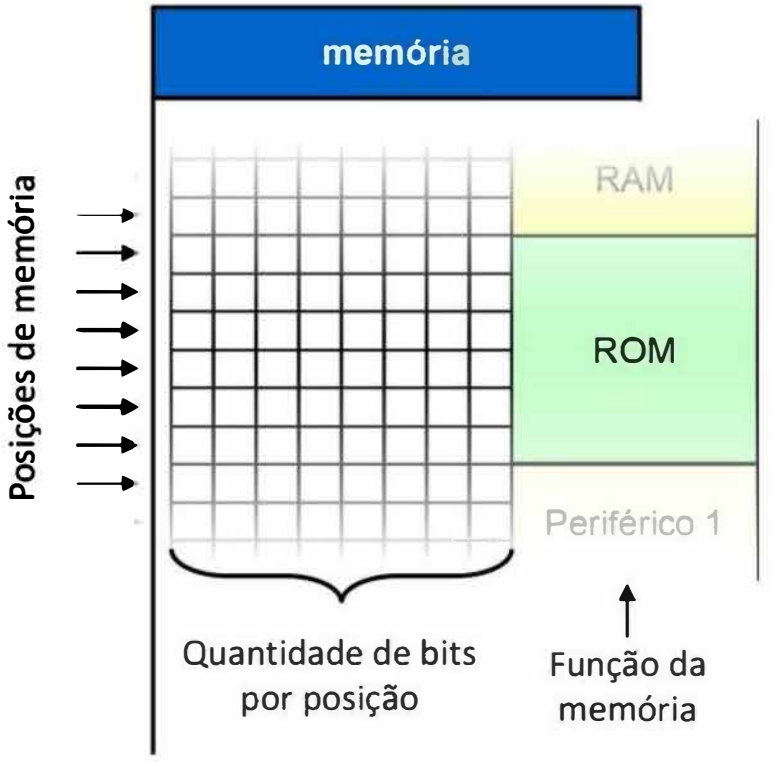


Figura 12.10: Espaço de memória

Por exemplo, um microcontrolador cujo tamanho da palavra de endereço é de 8 bits possui a capacidade de acessar uma memória de até $2^{(tamanho_do_endereco)} = 2^8 = 256$ bytes.

Para a maioria das memórias, cada posição armazena 8 bits de dados, mesmo para arquiteturas de 16, 32 ou 64 bits. Portanto, a quantidade máxima de memória para um barramento com N bits é de 2^N bytes. Um sistema com barramento de endereço com 10 bits pode armazenar até 1024 bytes. Por esse motivo os sistemas operacionais de 32 bits não conseguem fazer uso efetivo de memórias RAM com mais de 2^{32} = 4 gigabytes.

Mesmo o processador podendo alcançar toda essa extensão, nem sempre existe memória física em cada uma destas posições para armazenar dados. Deste modo a memória pode possuir espaços vazios. Se um ponteiro for apontado para um destes espaços, uma operação de escrita será desperdiçada. Apesar de tentar escrever, os valores serão perdidos, pois não há memória para armazenar a informação. Uma operação de leitura retornará lixo do barramento de dados, não sendo possível prever qual será o valor retornado.

12.3 Mapeando periféricos na memória

A memória digital pode ser entendida como um armário. Num armário com 6 espaços vazios, um marceneiro poderá fabricar até 6 gavetas para serem instaladas. O número da gaveta será o endereço da variável e o conteúdo da gaveta, a informação armazenada naquele endereço.

Para guardar um objeto no armário precisamos saber qual posição está disponível. No entanto, só será possível armazenar o objeto corretamente se a gaveta estiver encaixada no espaço dedicado a ela. Se a gaveta não estiver presente, não é possível afirmar com certeza o que acontecerá com o objeto armazenado ali.

Em vez de construir gavetas regulares, um marceneiro pode projetar outros "sistemas de armazenamento" nos espaços de um armário. Alguns destes sistemas podem permitir que o usuário enxergue o que está armazenado, mas não permitir que se mexa nele, como uma vitrine. Alguns sistemas permitem que o usuário coloque objetos, mas não permitir que ele os retire, como um cofre boca-de-lobo. Outros, ainda, permitem que a pessoa retire objetos mas não permite que ela os reponha, como dispensadores de latas de refrigerante. A figura 12.11 apresenta essa analogia de modo gráfico.

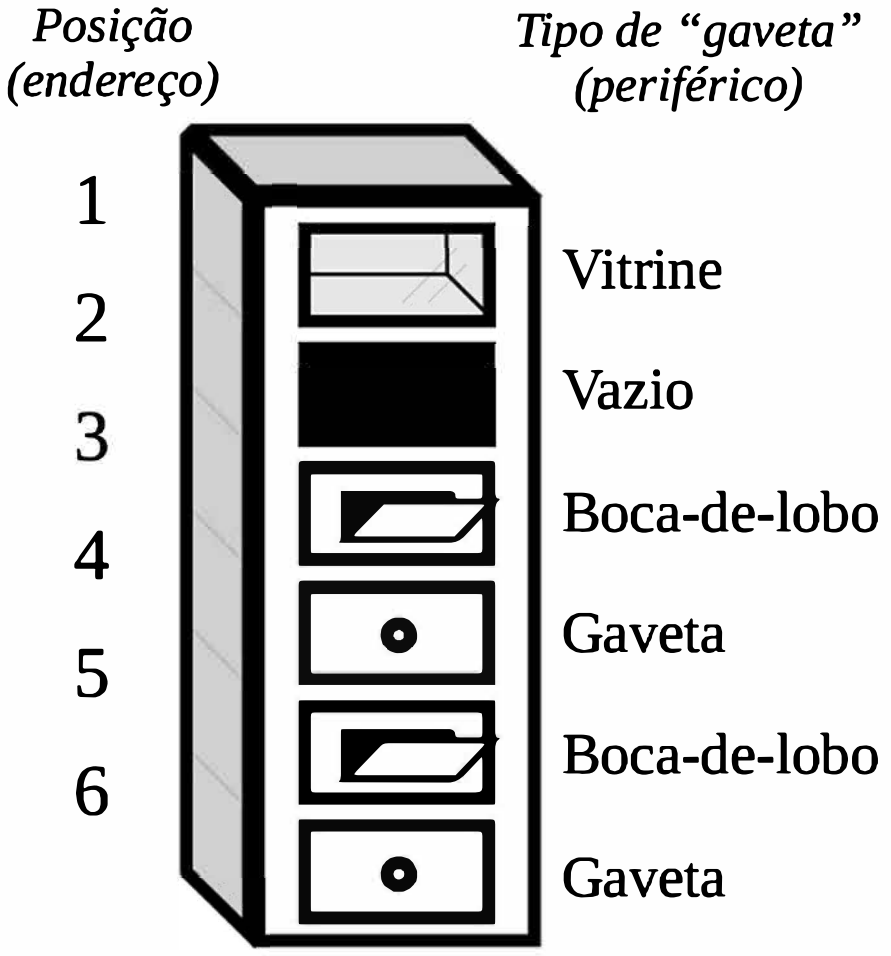


Figura 12.11: Memória e periféricos como um armário

Estes vários sistemas de armazenamento representam a variedade de tipos de memória e de interfaces de periféricos que podem estar ligados diretamente à memória do processador. A figura 12.12 apresenta esse modelo. Todos os periféricos estão conectados nos mesmos barramentos de dados e de endereços. O barramento de controle é o responsável por indicar qual dos periféricos deve responder aos comandos do microprocessador.

A identificação da posição de memória é feita através de um valor de endereço. Por estarmos tratando de sistemas digitais, o valor do endereço é codificado em binário.

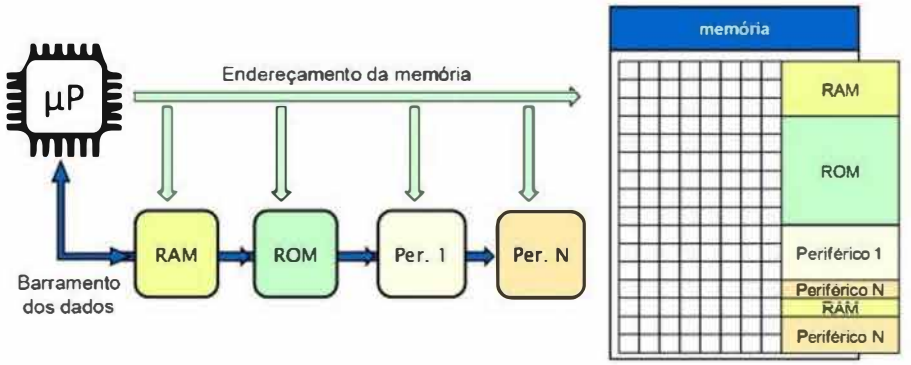


Figura 12.12: Mapa de memória e circuitos correspondentes

Em geral, escrevemos os valores em hexadecimal para evitar erros na transcrição ou indicação dos dados.

Os dispositivos que são mapeados na memória podem ocupar uma região do espaço total da memória. A quantidade de endereços depende do tipo de dispositivo. Por exemplo: um processador com barramento de endereço com 16 bits pode enxergar até 2^{16} ou 65.535 posições. Estas posições são numeradas de 0 a 65.535, ou em hexadecimal, de 0000_{16} a $FFFF_{16}$. Uma memória RAM de 2 kilobytes possui 2048 endereços, podendo ser numerados de 000_{16} a $3FF_{16}$, por exemplo. Para que essa memória seja lida corretamente pelo processador é necessário que cada um dos endereços da memória seja mapeados nos endereços do processador. Uma abordagem bastante comum é conectar os bits menos significativos e utilizar os bits mais significativos para identificar o chip.

Quando utilizamos vários dispositivos ao mesmo tempo, a identificação de cada um deles deve ser feita de modo único. Isso acontece com todos os tipos de memória e todos os periféricos. Se for preciso salvar uma variável na memória, é preciso tomar o cuidado de utilizar endereços que estão ligados em memórias do tipo RAM. Se quisermos ler alguma informação gravada na memória ROM, é preciso conhecer a localização exata antes de realizar a leitura.

Os microcontroladores que implementam arquitetura Harvard de processamento possuem diferentes barramentos de memória para os dados e para os programas. O microcontrolador da Atmel ATmega 328 com core AVR possui essa arquitetura.

A figura 12.13 apresenta o espaço de endereços do ATmega328 com arquitetura Harvard. Neste microcontrolador existem 2 regiões de memória: a de dados e a de programa.

A linguagem C foi desenvolvida para sistemas com apenas uma região de memória, cujos valores de endereço possuem apenas uma faixa. Na estrutura Harvard, podemos observar que existem duas posições de memória com o mesmo endereço. Deste modo, um ponteiro que aponte para o endereço **`0xF83`** pode estar apontando para qualquer uma das memórias. Para evitar esse problema, os compiladores utilizam palavras especiais para indicar para qual das regiões de memória o ponteiro está apontando. É comum utilizar os modificadores **`near`** e **`far`** nestas ocasiões.

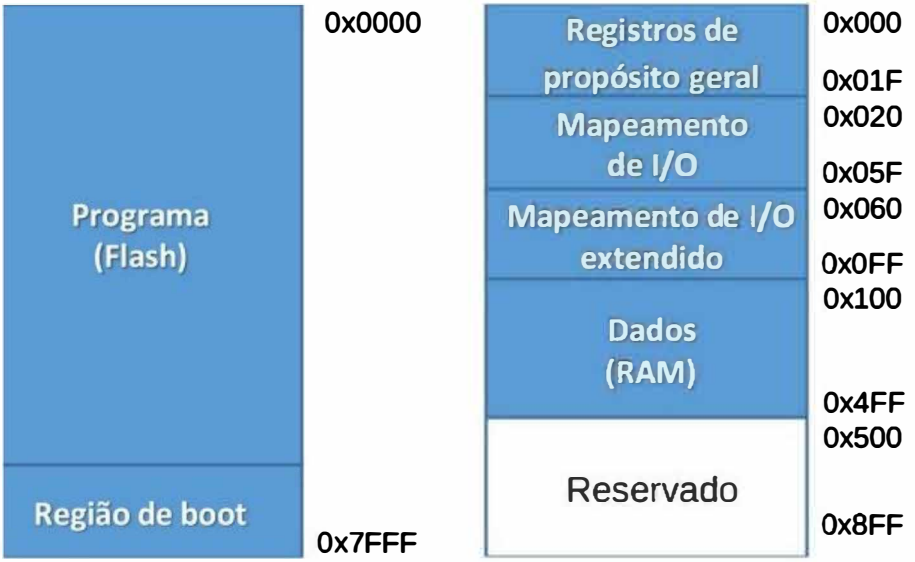


Figura 12.13: Regiões de memórias disponíveis no Atmel ATmega 328

A figura 12.14 apresenta o espaço de memória dos microcontroladores da Microchip e NXP. Estes microcontroladores utilizam arquitetura Von Neumman, com apenas um espaço de endereçamento. Por isso as memórias de programa (flash) e de dados (RAM) estão intercaladas.

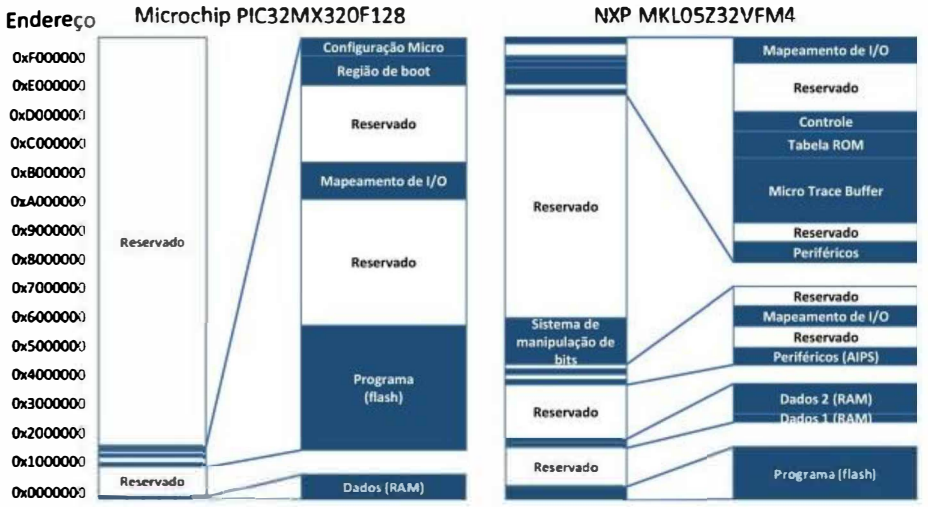


Figura 12.14: Regiões de memórias disponíveis no Microchip PIC32 e NXP KL05Z

Os mapas de memória apresentados foram baseados nos valores fornecidos pelos fabricantes através de seus datasheets. Cada modelo de processador pode possuir diferenças consideráveis. Em geral, as regiões marcadas como "reservado" são utilizadas nos diferentes modelos para abrigar mais periféricos ou mais memórias, sejam RAM ou Flash.

12.4 Clock e tempo de instrução

O microcontrolador é um circuito digital controlado por um sinal de sincronismo. Esse

sinal é denominado de clock. A execução de uma atividade compreende pelo menos 3 etapas: buscar da próxima operação na memória (fetch), decodificar a instrução (decode) e executar a operação. O conjunto destas três operações é chamado de ciclo de máquina.

Alguns processadores possuem um ciclo de máquina com mais instruções por ciclo. Assim, a velocidade de execução de tarefas do processador é uma fração da velocidade do clock.

Além desta redução, a velocidade efetiva de uma instrução varia de acordo com a tarefa que a instrução realiza. Algumas instruções podem ser mais demoradas, precisando de mais de um ciclo de máquina para ser executado.

Alguns processadores são desenvolvidos para que todas as instruções consumam o mesmo tempo. Estes processadores são também conhecidos como RISC. Já os processadores CISC são desenvolvidos com um conceito chamado microinstrução, que permite que o processador saiba fazer instruções mais complexas, no entanto essas instruções são mais demoradas.

Um exemplo é o caso da multiplicação de dois números. Para números inteiros a multiplicação é feita diretamente. A soma de números em notação científica, no entanto, possui passos extras: é necessário igualar as potências antes de realizar a multiplicação, fazer a multiplicação dos números, fazer a soma dos expoentes e depois ajustar as casas decimais para o formato científico. Este é um dos motivos que faz a segunda operação ser mais demorada que a primeira. Na tabela 12.1 é apresentado um exemplo numérico.

O cálculo de operações com ponto flutuante é especialmente caro para os processadores tradicionais. Eles podem demorar até 10 vezes mais para terminar do que as operações inteiras. Para evitar esse atraso, alguns processadores possuem embutido em seu hardware um coprocessador matemático, capaz de efetuar as operações com ponto flutuante de modo muito mais rápido.

Conhecer quanto tempo o código leva para ser executado permite ao desenvolvedor saber de maneira determinística qual é a exigência, em termos de capacidade de processamento, que o sistema embarcado em desenvolvimento precisa.

Tabela 12.1: Diferença na quantidade de operações com diferentes representações de números

Soma de inteiros	Soma de fracionários

`A = 12321;`
`B =  5550;`
`C = A + B;`
`//C = 17871`
 	
`//1. Somar`
`//      12321`
`// +     5550`
`//      17871`
	`A = 1.2321 x 10 ^ 4`
	`B = 5.55   x 10 ^ 3`
	`C = A + B`
	`//C = 1.7871 x 10 ^ 4`
	
	`//1. Converter para os`
	`//   mesmos expoente`
	`//      12.321 x 10 ^ 3`
	`//       5.55  x 10 ^ 3`
	
	`//2. Somar os números`
	`//   mantendo a mantissa`
	`//      12.321 x 10 ^ 3`
	`// +     5.55  x 10 ^ 3`
	`//      17.781 x 10 ^ 3`
	
	`//3. Corrigir quantidade`
	`//   de casas decimais`
	`//   e o valor da mantissa`
	`//  1.7781 x 10 ^ 4`

12.5 Microcontroladores

Os microcontroladores são circuitos eletrônicos que reúnem num único chip: a unidade de processamento, as memórias e um conjunto de periféricos de entrada e saída.

Isto facilita o desenvolvimento de produtos eletrônicos, pois, com um único componente, podemos desenvolver aplicações bastante complexas. Além disso, o projeto das placas é simplificado, visto que a quantidade de componentes e ligações fica reduzida.

Quando programamos sistemas embarcados em linguagem C, grande parte das diferenças entre os processadores é abstraída pelo compilador. Deste modo, não é preciso se preocupar com a arquitetura do processador, com a quantidade de bits nos barramentos de dados ou no barramento de endereços.

No entanto, com relação aos periféricos de entrada e saída, quase todo o procedimento de acesso e configuração fica a cargo do programador. O compilador da linguagem C não provê nenhuma ferramenta de acesso aos periféricos. As configurações também diferem para cada modelo de microcontrolador. O programador deve então ficar atento ao chip utilizado para realizar corretamente o processo de configuração dos periféricos.

O intuito deste livro é apresentar o conceito por trás desses periféricos e o modo de operação de cada um deles, sem ficar preso a uma arquitetura específica. Em seguida, serão apresentadas as três arquiteturas das placas utilizadas: AVR, ARM e MIPS.

Atmel ATMega328

Os microcontroladores da família Atmel 8-bit são baseados na arquitetura AVR, um modelo de processador RISC. Essa arquitetura possui 8 bits e barramentos separados de dados

e programas (Harvard). Esta arquitetura foi desenvolvida em 1996, sendo um dos primeiros microcontroladores a fazerem uso de memória flash para armazenar o código do programa. Isso permitiu que os chips fossem reprogramados de modo simples, facilitando o processo de desenvolvimento do produto. A figura 12.15 apresenta o digrama de blocos do microcontrolador ATmega328, que é a base da placa de controle Arduino.

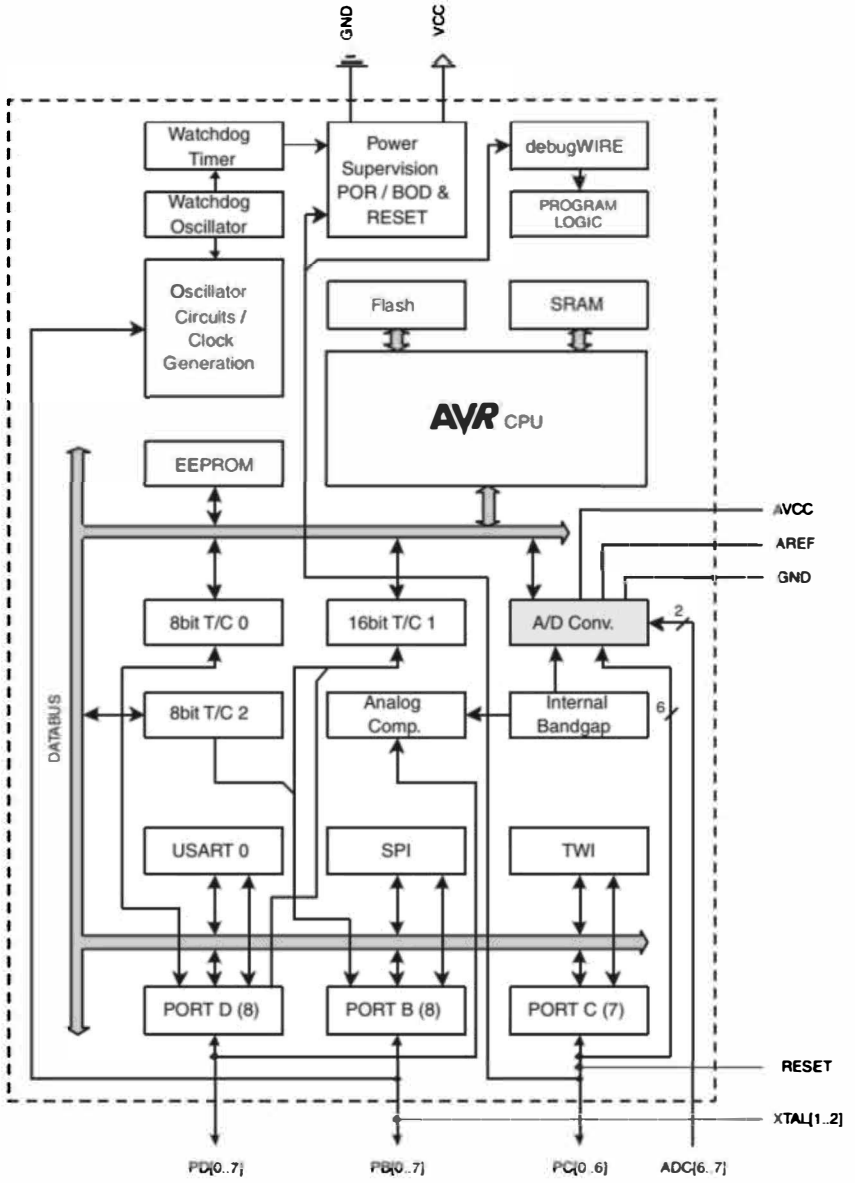


Figura 12.15: Diagrama de blocos do ATmega328
Fonte: datasheet do microcontrolador

O ATMega328 possui 23 terminais de I/O, 6 canais de entrada analógica com 10 bits de precisão, 6 saídas com modulação PWM, comunicação serial em UART, SPI, I2C, três timers configuráveis e cinco modos de economia de energia. O clock de operação pode alcançar 20MHz. Pode ser alimentado com tensões de 1,8 até 5,5 volts.

Com relação à memória, o ATmega possui uma memória flash de 32 kbytes e uma memória RAM do tipo SRAM de 2 kbytes. Ele possui ainda, uma memória EEPROM de 1 kbyte. Esta última pode ser utilizada para armazenar variáveis mesmo quando a energia for interrompida.

NXP KL05z

Os microcontroladores da família Kinetis L Series KL0x MCU são baseados na arquitetura ARM de processador. Essa arquitetura possui 32 bits e barramentos de dados e programas juntos (Von-Neumann). A figura 12.16 apresenta o digrama de blocos da família de microcontroladores KL05, que é a base da placa de controle Freedom.

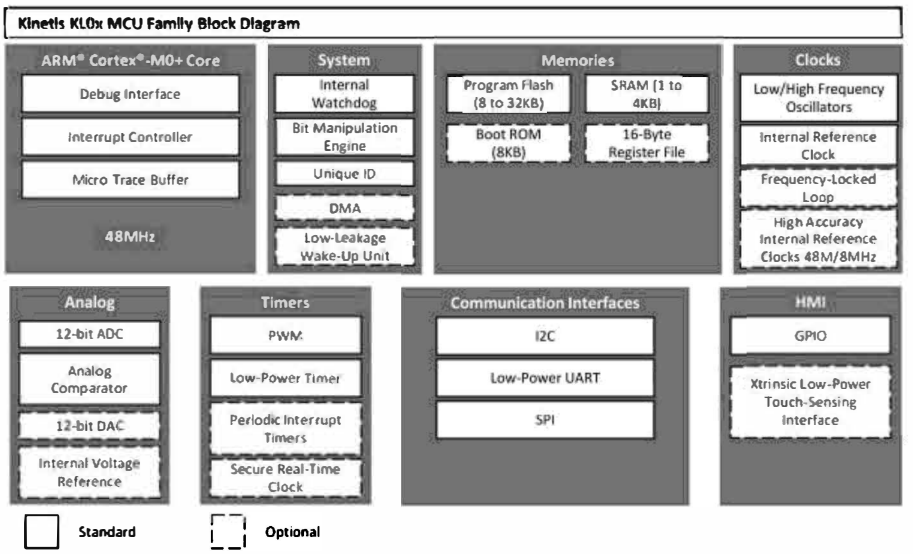


Figura 12.16: Diagrama de blocos do KL05
Fonte: datasheet do microcontrolador

O MKL05Z32VFM4 possui 28 terminais de I/O, 12 canais de entrada analógica com 12 bits de precisão, 8 saídas com modulação PWM, comunicação serial em UART, SPI, I2C, três timers configuráveis e nove modos de economia de energia. O clock de operação pode alcançar 48MHz. Pode ser alimentado com tensões de 1,7 até 3,6 volts.

Com relação à memória, o MKL05Z possui uma memória flash de 32 kbytes e uma memória RAM do tipo SRAM de 4 kbytes.

Microchip PIC32MX320F128

Os microcontroladores da família PIC32 são baseados na arquitetura MIPS de processador. A figura 12.17 apresenta o digrama de blocos do microcontrolador PIC32, que é a base da placa de controle Chipkit.

FIGURE 1-1: BLOCK DIAGRAM

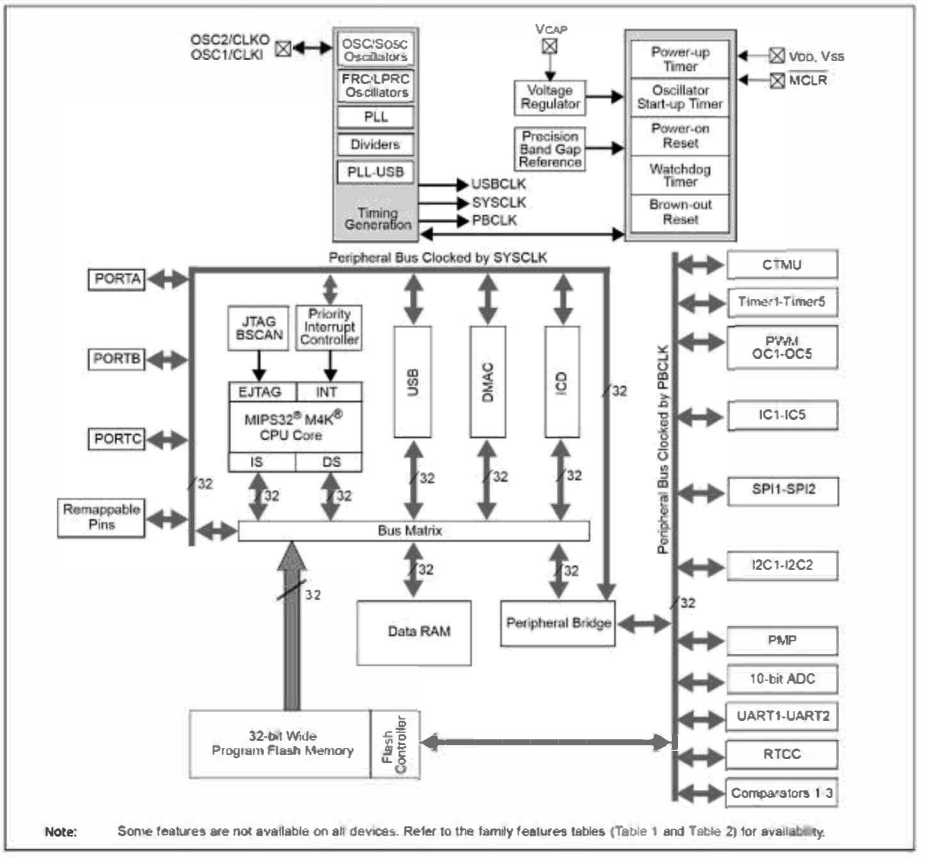


Figura 12.17: Diagrama de blocos do PIC32MX
Fonte: datasheet do microcontrolador

O modelo de microcontrolador utilizado é o PIC32MX110. Ele possui 21 terminais de IO, 10 canais de entrada analógica, 5 saídas com modulação PWM e comunicação serial em UART, SPI, I2C.

O PIC32MX320F128 possui 21 terminais de I/O, 16 canais de entrada analógica com 10 bits de precisão, 8 saídas com modulação PWM, comunicação serial em UART, SPI, I2C, cinco timers configuráveis e três modos de economia de energia. O clock de operação pode alcançar 80 MHz. Pode ser alimentado com tensões de 2,3 até 3,6 volts.

Com relação a memória, o PIC32MX320F128 possui uma memória flash de 128 kbytes e uma memória RAM do tipo SRAM de 16 kbytes.

12.6 Registros de configuração do microcontrolador

Visando serem úteis para mais de uma aplicação, é comum que os microcontroladores possuam terminais que podem ser utilizados de mais de um modo.

Antes de utilizá-los, devemos fazer a configuração destes terminais do modo desejado. A escolha de qual será a função de cada um dos terminais fica a cargo do projetista do hardware no momento em que ele faz as conexões entre o microcontrolador e os periféricos externos. Do ponto de vista do software, devemos descrever o funcionamento de acordo com o que foi planejado pelo hardware. Essa descrição é feita utilizando espaços de memória pré-definidos.

Estes espaços são chamados de registros, e os valores que podem assumir variam de processador para processador e de periférico para periférico. As definições corretas podem ser pesquisadas nos documentos conhecidos como datasheets, dos microcontroladores.

Para as plataformas Arduino e Chipkit, a maioria das configurações já estão prontas ou armazenadas nos chips. Para a plataforma Freedom, é necessário fazer algumas definições para que os demais periféricos consigam funcionar corretamente.

A primeira é a definição da fonte de clock a ser utilizada. A Freedom pode utilizar cristais externos como referência ou um circuito oscilador interno. Ela possui ainda vários modos internos de oscilação, com capacidade de modificar sua velocidade. Para simplificar a utilização da placa, ela será configurada para utilizar o oscilador interno com uma frequência fixa de 24MHz. Para isso, deve-se configurar o registro **C4** do periférico **MCG**, ligando-se o bit 7.

```
1 MCG_BASE_PTR ->C4 |= 0x80;
```

12.7 Requisitos elétricos do microcontrolador

O projeto inicial do hardware se dá pela escolha do microcontrolador. Após a escolha, devemos adicionar os demais componentes e sistemas que permitam que o microcontrolador seja ligado e gravado. Por fim, adicionamos o sistema de clock.

O projeto eletrônico que define como os dispositivos estão conectados é denominado esquemático. O esquemático será utilizado ao longo do livro para descrever de modo gráfico os componentes utilizados.

O circuito que liga o microcontrolador é chamado de circuito de alimentação. O fator mais importante é o nível de tensão utilizado. Atualmente, boa parte dos microcontroladores trabalha com níveis de 5 volts, mas a cada dia cresce o número daqueles que precisam de apenas 3,3 volts. Ligar um microcontrolador de 3,3 numa alimentação de 5 volts pode levá-lo a queimar e, até mesmo, danificar os demais componentes da placa. Alguns microcontroladores suportam uma faixa de alimentação, como o NXPKL05z, por exemplo, que pode ser alimentado com qualquer tensão contínua entre 1,71 e 3,6 volts.

Os microcontroladores podem ser bastante sensíveis a alterações na tensão, principalmente se fizerem uso de conversores analógicos para digitais. Um dos modos de reduzir o ruído que chega pelo ambiente é fazer uso de um pequeno capacitor. Este capacitor deve ficar o mais próximo dos terminais de alimentação do microcontrolador, um no positivo e outro no terra.

Alguns microcontroladores possuem mais de um par de terminais de alimentação. Assim, coloca-se um capacitor em cada par desses terminais. Em geral, utiliza-se capacitores cerâmicos de valores próximos a 100nF. Para saber o valor recomendado para seu microcontrolador, procure informações com o fabricante no datasheet do componente.

O circuito de gravação depende do modelo de microcontrolador a ser utilizado. Em geral, a maioria dos circuitos permite a gravação do microcontrolador sem precisar tirá-lo da placa. Este tipo de gravação é chamada de in-circuit. Um dos protocolos de gravação mais comuns é o JTAG, ou Joint Test Action Group. Já a Microchip utiliza o ICSP, ou In-Circuit Serial Programming.

Com relação ao sinal de sincronia, ou clock, é possível utilizar diversos circuitos diferentes. Três são mais comuns: osciladores do tipo RC, osciladores à cristal e fonte de sinal externa.

Os osciladores do tipo RC são, em geral, mais baratos. Eles utilizam uma malha com um resistor (R) e um capacitor (C) como base de tempo. O resistor funciona como um regulador de corrente, limitando a quantidade de energia que flui para o capacitor. Este por sua vez, leva um tempo até ser carregado. Depois que ele está carregado, um circuito interno inverte o sentido da corrente e começa a descarregar o capacitor. De posse dessa onda de carga/descarga é utilizado um comparador para gerar uma onda quadrada de sincronismo.

Vários microcontroladores possuem um circuito RC interno, reduzindo o custo com componentes externos.

O maior problema com a malha RC é a sua precisão. Quando utilizados componentes externos, estes podem apresentar até 20 % de variação na frequência obtida. Além disso, o valor da capacitância pode se alterar com o tempo, fazendo com que a frequência de operação também se altere.

Os osciladores à cristal são um pouco mais caros que a malha RC, mas apresentam uma precisão extremamente alta, chegando a atrasar apenas um segundos a cada 30 anos. O funcionamento é similar à malha RC, no entanto, o circuito ressonante é baseado na frequência de vibração do cristal de quartzo utilizado.

Por fim, é possível utilizar uma fonte de clock externa. Esta alternativa pode ser utilizada quando queremos fazer o controle de velocidade de processamento externamente ou para sincronizar o trabalho de vários processadores.

12.8 Exercícios

Ex. 12.1 — O que são microcontroladores? Qual a diferença para microprocessadores?

Ex. 12.2 — Como os registros dos periféricos podem ser acessados?

Ex. 12.3 — Como é dividida a memória de um microcontrolador?

Ex. 12.4 — Qual a principal diferença entre as arquiteturas de processador Harvard e Von Neumman?

Ex. 12.5 — O que é um barramento de dados?

CAPÍTULO

13

Programação dos periféricos

13.1Controlando os terminais do microcontrolador

Mapeando os terminais nas placas de controle

13.2Configuração dos periféricos

13.3Exercícios

"Em teoria, não há diferença entre teoria e prática; mas, na prática, sim."

Chuck Reid

No microcontrolador, o processador é responsável por executar a lista de códigos, incluindo as abstrações lógicas e aritméticas. As demais atividades são carregadas por dispositivos dedicados chamados periféricos.

Os periféricos podem ser internos ao microcontrolador, embutidos no mesmo chip, servirem de interface entre as estruturas internas do chip e o mundo externo ou serem circuitos externos com funções próprias.

Os periféricos internos podem executar atividades dedicadas de modo independente do processador, como converter um sinal analógico num valor digital, realizar a contagem de um tempo ou preparar uma mensagem para ser enviada por um sistema serial.

Os periféricos de interface fazem com que os sinais gerados pelo processador cheguem aos terminais físicos do microcontrolador. Eles funcionam como periféricos de saída. Eles podem, ainda, fazer o caminho contrário, recebendo valores dos terminais físicos e adequando-os para que o processador seja capaz de interpretar e manipular essa informação.

Os periféricos externos adicionam funcionalidades ao produto desenvolvido, funcionando como interfaces de entrada e saída para o usuário, como um teclado ou um display de LCD. Estes tipos de periféricos podem também complementar as funções internas do chip, provendo recursos que o processador não apresenta, como relógios de tempo real, memória extra de armazenamento, conversão de protocolos de comunicação, entre outros.

Este livro toma como base um conjunto de periféricos que visam compreender os principais conceitos de acionamento, gerenciamento e programação de periféricos para sistemas embarcados.

Com relação aos periféricos externos de exibição, serão abordados: um display de LCD, um display de leds de 7 segmentos e um led RGB. O led RGB será controlado por uma saída PWM.

Como periféricos externos de entrada serão utilizados um teclado matricial, com duas linhas e cinco colunas, um sensor de luminosidade e um potenciômetro, os dois últimos

através de um conversor ADC.

Para exemplificar os periféricos de comunicação, serão abordados um sistema de comunicação assíncrono, utilizando uma UART, e um sistema síncrono, comunicando-se com um relógio de tempo real via I2C.

Por fim, para estudo dos periféricos internos, serão abordados os sistemas de PWM, ADC, temporização, interrupção e watchdog.

13.1 Controlando os terminais do microcontrolador

A maioria dos microcontroladores que disponibilizam terminais de entrada e saída, acaba organizando esses terminais através de conjuntos, denominados portas. As portas são registros que repassam a informação da memória para os terminais físicos. Nestas portas cada um dos bits representa um terminal. Deste modo, ao ligar o bit, um dos terminais passa a ter um nível de tensão alto. Quando o bit é desligado, o terminal correspondente passa ao nível de tensão zero.

O primeiro passo é identificar qual registro controla cada uma das portas, bem como quais terminais correspondem aos bits do registro. A figura 13.1 apresenta um exemplo onde cada terminal é mapeado para um bit na memória.

No caso da figura 13.1, os oito bits estão diretamente conectados aos terminais físicos de número 33 a 40. Isto que dizer que, ligando o bit de número 4, o terminal 37 apresentará nível alto de tensão, 3,3 ou 5 volts (dependendo do microcontrolador utilizado). Desligando esse bit, o terminal 37 apresentará zero volts em sua saída. Deste modo, podemos controlar o que estiver conectado ao terminal utilizando as operações com os bits.

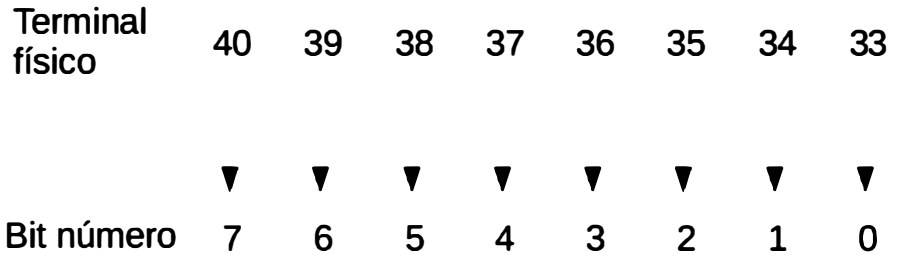


Figura 13.1: Mapeamento de terminais físicos a bits da memória

Algumas portas podem, ainda, ter seus terminais trabalhando como entrada ou como saída de sinais, sendo necessário como primeiro passo configurá-las antes de sua utilização. A figura 13.2 apresenta a representação simplificada de um dos terminais do microcontrolador:

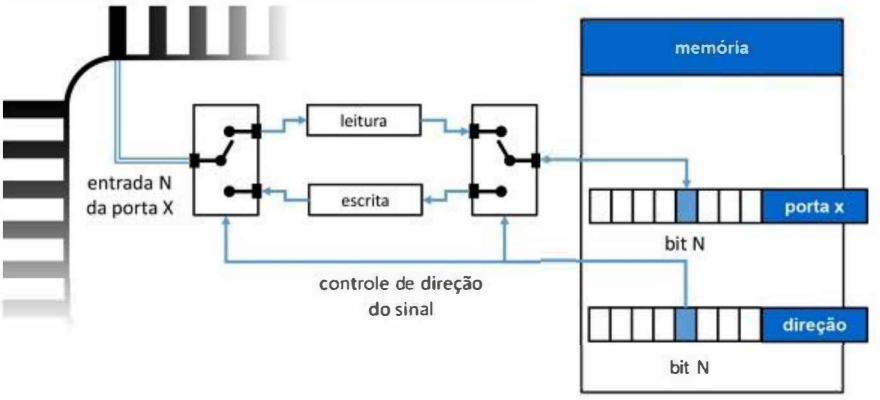


Figura 13.2: Processo de leitura/escrita de um terminal a partir de um bit na memória.

Em geral, as portas possuem dois registros: um para realizar a leitura/escrita da porta e outro para controlar a direção do sinal. Algumas arquiteturas separam os registros de escrita e leitura. Deste modo, para ler um determinado valor, devemos primeiro configurar a porta como entrada e realizar a leitura no registro correto. Para a escrita, devemos reconfigurar a porta como saída e depois escrever em outro registro. O comportamento exato do periférico deve ser observado no datasheet do microcontrolador utilizado.

Para realizar a leitura, o nível de tensão que está conectado ao pino externo é convertido para uma informação lógica **`0`** (zero) ou **`1`** (um), dependendo do nível de tensão. Este valor binário é então levado para uma posição pré-definida de memória, comumente chamado de porta.

Para conseguir mudar o valor de tensão externo é preciso primeiramente configurá-lo como saída. Este controle é feito através de um bit em outra região de memória que controla a direção dos sinais. Dependendo do microcontrolador, escrevendo zero neste bit faz com que o pino se torne uma saída e, escrevendo um, ele se torna uma entrada. Depois de configurado com saída basta acionar o bit correspondente na porta para que o terminal tenha sua tensão controlada pelo valor do bit.

Uma mesma porta pode possuir parte de seus terminais configurados como entrada e parte como saída, sem interferir no funcionamento do programa.

Com relação aos três microcontroladores utilizados, seus terminais foram conectados de modo que ficassem compatíveis com o modelo utilizado pela placa do Arduino. Deste modo, 14 terminais digitais do microcontrolador foram ligados ao lado direito da placa. Seis terminais analógicos foram conectados ao lado esquerdo da placa. As figuras 13.3, 13.4 e 13.5 apresentam as três placas com suas conexões, nomes dos terminais e relações com os registros de configuração das portas, conforme a tabela 13.1:

Tabela 13.1: Endereços de memória dos registros de acesso às portas

Registro	*Endereço*	*Microcontrolador*
DDRB	**`0x04`**	ATmega328
PORTB	**`0x05`**	ATmega328
DDRC	**`0x07`**	ATmega328
PORTC	**`0x08`**	ATmega328
DDRD	**`0x0A`**	ATmega328
PORTD	**`0x0B`**	ATmega328

TRISB	**0xBF886040**	PIC32 MX320
PORTB	**0xBF886050**	PIC32 MX320
TRISD	**0xBF8860C0**	PIC32 MX320
PORTD	**0xBF8860D0**	PIC32 MX320
TRISF	**0xBF886140**	PIC32 MX320
PORTF	**0xBF886150**	PIC32 MX320
TRISG	**0xBF886180**	PIC32 MX320
PORTG	**0xBF886190**	PIC32 MX320
PORTA_PDOR	**0x4004F000**	KL05 z32
PORTA_PDIR	**0x4004F010**	KL05 z32
PORTA_PDDR	**0x4004F014**	KL05 z32
PORTB_PDOR	**0x4004F040**	KL05 z32
PORTB_PDIR	**0x4004F050**	KL05 z32
PORTB_PDDR	**0x4004F054**	KL05 z32

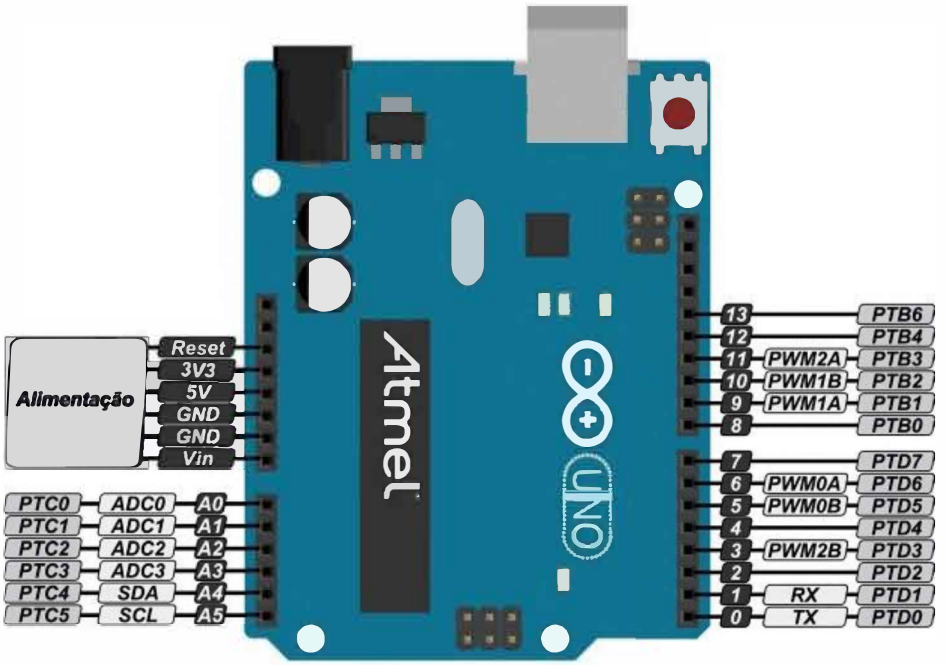


Figura 13.3: Pinagem da placa Arduino com ATmega328
Fonte: Imagem produzida com Fritzing/Inkscape

Como mencionado, as portas utilizadas em cada um dos microcontroladores estão mapeadas em alguma região da memória. A tabela 13.1 apresenta os endereços de cada um dos registros das portas, bem como dos registros de configuração de direção. Os nomes de cada um dos registros variam de acordo com o fabricante.

Para os microcontroladores da Microchip, os registros de configuração de direção são chamados de TRISx e as portas, de PORTx, onde x é a letra da porta. Para a Atmel, as portas também são chamadas de PORTx, mas os registros de direção possuem o nome DDRx. A NXP utiliza a nomenclatura GPIOA_PDDR para os registros de configuração de direção. A NXP

também realiza uma divisão entre o registro de leitura da porta (GPIOA_PDIR) e de escrita (DGPIOA_PDOR). Deste modo, no microcontrolador KL05 existem três registros por porta.

Para acessar os terminais, portanto, basta utilizar o valor que está no endereço listado na tabela. Se quisermos ligar todas as saídas da porta C no microcontrolador ATmega328, devemos colocar o valor **`0xFF`** no endereço de memória **`0x08`**. Para que o valor seja efetivamente enviado para a porta, devemos configurar o **`DDRC`** com o valor **`0x00`**.

Um dos modos de se fazer isso é criar um ponteiro para esse endereço de memória de modo a acessá-lo indiretamente.

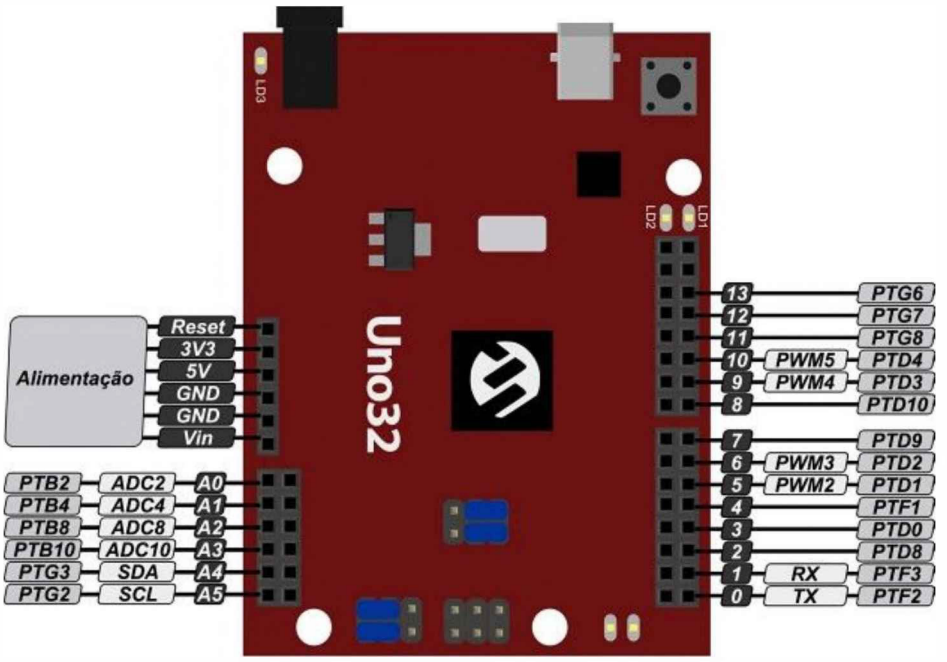


Figura 13.4: Pinagem da placa Chipkit com PIC32MX320
Fonte: Imagem produzida com Fritzing/Inkscape

Por exemplo, o código a seguir faz a inicialização dos terminais digitais de zero à sete, através da porta D no microcontrolador ATMega328.

```c
//início do programa
void main(void){
    //Para que o ponteiro para a porta D e Tris D funcione
    //eles são definidos como:
    //a) unsigned char: pois os 8 bits representam valores
    //b) volatile: as variáveis podem mudar a qualquer momento
    volatile unsigned char *PORTD = 0x0A;
    volatile unsigned char *DDRC  = 0x0B;

    //configurando todos os pinos como saídas
    // 0 = entrada (Input)
    // 1 = saída   (Output)
    *DDRD  = 0b11111111;
    //liga apenas os quatro últimos terminais
    *PORTD = 0b11110000;

    //mantém o sistema ligado indefinidamente
    for(;;);
}
```

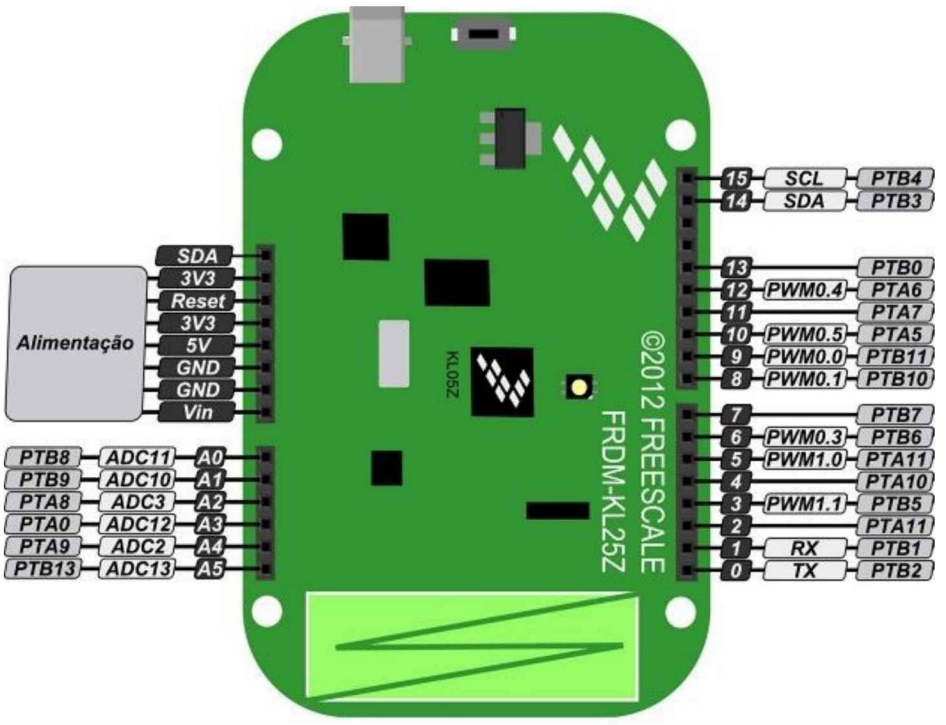


Figura 13.5: Pinagem da placa Freedom com KL05z
Fonte: Imagem produzida com Fritzing/Inkscape

Podemos notar que, por se tratar de ponteiros, as variáveis **PORTD** e **DDRD** devem sempre ser manipuladas com um asterisco, indicando que estamos trabalhando com o endereço indicado pelo ponteiro.

Uma outra maneira de manipular as portas é criar define's que fazem o uso de uma conversão do número da posição da variável para um ponteiro. A mesma estrutura define e dereferência o ponteiro, fazendo com que, para o programador, o registro funcione como uma variável normal.

A utilização desta abordagem, utilizando macros, é preferida em relação a de ponteiros pois simplifica o modo de uso das portas. Outra vantagem é que esta definição precisa ser feita apenas uma vez em um arquivo e todas as demais partes do código podem fazer uso dela.

O código 13.1 apresenta a utilização da porta D através de defines, de modo que os registros **PORTD** e **DDRD** são utilizados como se fossem variáveis comuns.

Código 13.1: Acesso direto aos terminais de entrada e saída

```
1 //define's para portas de entrada e saída
2 #define PORTD (*(volatile unsigned char*)0x0A)
3 #define DDRD  (*(volatile unsigned char*)0x0B)
4
5 //início do programa
6 void main(void){
7
8     //configurando todos os pinos como saídas
9     TRISD = 0b11111111;
10
11    //liga apenas os quatro últimos terminais
12    PORTD = 0b11110000;
13
14    //mantém o sistema ligado indefinidamente
15    for(;;);
16 }
```

Na prática, não é comum fazer este arquivo a mão, já que grande parte dos fabricantes entrega um header com todas essas definições prontas. Essas definições fazem uso de estrutura mais ajustadas e conhecidas pelo compilador, gerando códigos mais otimizados.

Nas três plataformas utilizadas, os arquivos que contêm essas definições são dados através de cada um dos includes a seguir:

```
1 //definições dos registros do Atmel ATmega
2 #include <avr/io.h>
3
4 //definições dos registros do Microchip PIC32MX
5 #include <plib.h>
6
7 //definições dos registros do NXP KL05
8 #include <MKL05Z4.h>
```

Mapeando os terminais nas placas de controle

Em geral, os terminais dos microcontroladores podem ser configurados de diversas formas. Um mesmo terminal pode funcionar como uma entrada de um conversor analógico ou um canal de recepção de dados seriais, ou até mesmo uma saída de um sinal PWM. É necessário realizar a configuração do terminal antes de utilizá-lo.

A escolha de qual das funcionalidades disponíveis será utilizada, depende do projetista. Para as placas de controle, os projetistas já definiram as funcionalidades de modo que os terminais ficassem compatíveis com o formato dos terminais do Arduino.

No padrão do Arduino, exitem 14 terminais digitais, onde alguns podem ser utilizados como saída PWM, e dois em comunicação serial assíncrona. Outros 6 terminais estão disponibilizados como entrada analógica, onde dois podem ser usados como comunicação serial síncrona.

Para compatibilizar os três microcontroladores estudados, os projetistas das placas Arduino, Chipkit e Freedom projetaram as conexões de modo que os terminais dos barramentos fossem compatíveis.

A placa base foi projetada de modo que, independentemente da placa de controle utilizada, a funcionalidade implementada pelos terminais fosse a mesma. A figura 13.6 apresenta essas conexões.

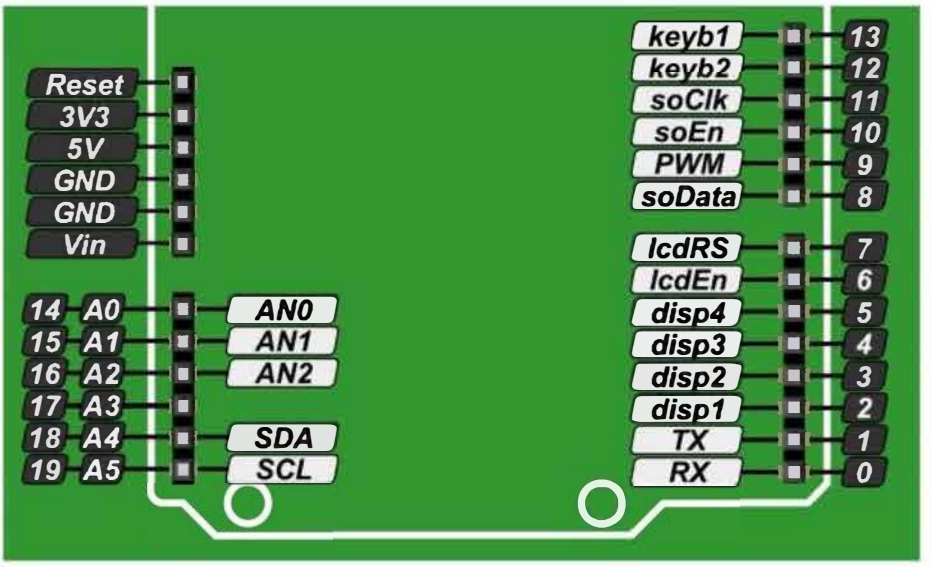


Figura 13.6: Footprint da placa base com a funcionalidade dos terminais
Fonte: Imagem produzida com Fritzing/Inkscape

A tabela 13.2 apresenta os terminais de entrada e saída como são numerados nos barramentos, a função implementada na placa base com cada um destes terminais e os terminais originais em cada uma das três plataformas utilizadas.

Tabela 13.2: Correspondência entre terminais e portas dos microcontroladores

Terminal	Função	Direção	Freedom	Chipkit	Arduino
D0	RX	Entrada	PTB2	PTF2	PTD0
D1	TX	Saída	PTB1	PTF3	PTD1
D2	disp1	Saída	PTA11	PTD8	PTD2
D3	disp2	Saída	PTB5	PTD0	PTD3
D4	disp3	Saída	PTA10	PTF1	PTD4
D5	disp4	Saída	PTA12	PTD1	PTD5
D6	lcdEn	Saída	PTB6	PTD2	PTD6
D7	lcdRS	Saída	PTB7	PTD9	PTD7
D8	soData	Saída	PTB10	PTD10	PTB0
D9	PWM	Saída	PWM0 .0	PWM4	OC1 A
D10	soEn	Saída	PTA5	PTD4	PTB2
D11	soClk	Saída	PTA7	PTG8	PTB3
D12	keyb1	Entrada	PTA6	PTG7	PTB4
D13	keyb2	Entrada	PTB0	PTG6	PTB5
A0 /D14	AN0	Entrada	ADC11	ADC2	ADC0
A1 /D15	AN1	Entrada	ADC10	ADC4	ADC1
A2 /D16	AN2	Entrada	ADC3	ADC8	ADC2
A3 /D17	–	–-	–-	–	–
A4 /D18	SDA	Entrada/Saída	PTA9	PTG3	PTC4
A5 /D19	SCL	Saída	PTB13	PTG2	PTC5

13.2 Configuração dos periféricos

O processo de configuração dos periféricos é dependente do microcontrolador utilizado e das conexões realizadas. A plataforma Arduino traz uma camada de abstração que nos permite utilizar os periféricos sem precisar conhecer os detalhes de configuração destes. Essa camada é especializada para cada tipo de processador utilizado. O modo de utilizá-la é dado por:

```
pinMode(pin,mode);
```

Onde:	`pin`	Terminal físico a ser configurado (de 1 à 18)
	`mode`	Modo de operação(`INPUT` ou `OUTPUT`)

O código 13.2 apresenta, simplificadamente, como a função **`pinMode()`** é implementada segundo o framework Wiring.

Código 13.2: Implementação da função pinMode pelo framework wiring

```
1 void pinMode(uint8_t pin, uint8_t mode){
2     uint8_t bit = digitalPinToBitMask(pin);
3     uint8_t port = digitalPinToPort(pin);
4     volatile uint8_t *reg, *out;
5
6     reg = portModeRegister(port);
7     out = portOutputRegister(port);
8
9     if (mode == INPUT) {
10        *reg &= ~bit;
11        *out &= ~bit;
12    } else if (mode == OUTPUT){
13        *reg |= bit;
14    }
15 }
```

Podemos notar que a função recorre a duas outras funções para mapear o terminal do microcontrolador com o terminal indicado pelo barramento do Arduino. A primeira,

digitalPinToBitMask(), retorna qual é o bit relacionado ao terminal escolhido. Já a função **digitalPinToPort()** retorna qual é a porta em que o terminal se encontra. O resultado é o endereço do registro na memória onde a porta está mapeada.

A plataforma exige que o usuário defina qual é o modelo de placa que está em uso, para fazer as configurações corretas. Isto facilita o processo de programação mas insere uma quantidade de código considerável. A utilização de um framework facilita a programação, no entanto, pode esconder detalhes importantes do modo de uso e levar a problemas no funcionamento do hardware.

Para a placa Freedom, no entanto, o framework Wiring não está disponível. Deste modo é preciso criar as funções de mapeamento dos terminais.

O primeiro passo é conhecer quais bits de quais portas acionam os terminais físicos. Para a Freedom, eles são mapeados de acordo com a tabela 13.3.

Tabela 13.3: Terminais padrão Arduino x Freedom

Terminal	0	1	2	3	4	5	6	7	8	9	10	11	12	13	18	19
Porta/Bit	B2	B1	A11	B5	A10	A11	B6	B7	B10	B11	A5	A7	A6	B0	A9	B13

Para simplificar o uso dos terminais na placa, eles foram mapeado através do número do terminal físico, utilizando um **#define**. Isto facilita a criação das bibliotecas. Caso haja necessidade de mudança da placa de controle, a atualização dos códigos para acesso aos terminais é feita apenas num local.

Para essa arquitetura, o registro que controla a direção da porta é o **PDDR** - Port Data Direction Register. Para configurar um terminal como saída, é preciso colocar o valor **1**, sendo o valor **0** utilizado para entrada. Além disso, para garantir que o terminal será um dispositivo de I/O digital, é necessário colocar o valor **0x140** no registro PCR de cada terminal.

Para ligar ou desligar um terminal, depois que ele for configurado como saída, utiliza-se a função **digitalWrite()** no framework Wiring do Arduino ou do Chipkit. Para compatibilizar os códigos com a Freedom, esta função foi desenvolvida para essa plataforma com o mesmo funcionamento. Ela recebe o número do terminal e faz uma operação **bitClr()** para desligar a saída, ou uma operação **bitSet()** para ligar. O registro que controla a escrita de dados é o **PDOR**, Port Data Output Register.

O processo de leitura do terminal, quando configurado como entrada, é muito parecido. Basta realizar uma operação de **bitTst()** no bit da porta correspondente ao terminal. O registro que controla a escrita de dados é o **PDIR**, Port Data Input Register.

Estas funções foram reunidas numa biblioteca denominada **io**. O header e a implementação estão apresentados nos códigos 13.3 e 13.4. Além das funções, essa biblioteca cria as definições de **INPUT**, **OUTPUT**, **HIGH** e **LOW** para igualar o funcionamento das funções com o framework Wiring.

Código 13.3: Header da biblioteca de IO para Freedom

```
1 #ifndef IO_H_
2 #define IO_H_
3
4 #define bitSet(arg,bit) ((arg) |= (1<<bit))
5 #define bitClr(arg,bit) ((arg) &= ~(1<<bit))
```

```
 6 #define bitFlp(arg,bit) ((arg) ^= (1<<bit))
 7 #define bitTst(arg,bit) ((arg) & (1<<bit))
 8
 9 #define OUTPUT 0
10 #define INPUT  1
11 #define LOW 0
12 #define HIGH 1
13
14 //definição das funções dos terminais físicos
15 #define SCL_PIN      19
16 #define SDA_PIN      18
17 #define keyb1        13
18 #define keyb2        12
19 #define SO_CLK_PIN   11
20 #define SO_EN_PIN    10
21 #define PWM           9
22 #define SO_DATA_PIN   8
23 #define LCD RS PIN    7
24 #define LCD_EN_PIN    6
25 #define DISP4_PIN     5
26 #define DISP3_PIN     4
27 #define DISP2_PIN     3
28 #define DISP1_PIN     2
29 #define TX            1
30 #define RX            0
31 //Os led compartilham os terminais dos displays
32 #define LED_R_PIN DISP1_PIN
33 #define LED_G_PIN DISP2_PIN
34 #define LED_B_PIN DISP3_PIN
35
36     void pinMode(int pin, int type);
37     void digitalWrite(int pin, int value);
38     int digitalRead(int pin);
39     void systemInit(void);
40 #endif /* IO_H_ */
```

Código 13.4: Código da biblioteca de IO para Freedom

```
#include "io.h"
#include "derivative.h"

#define PORTA_PDDR (PTA_BASE_PTR ->PDDR)
#define PORTA_PDOR (PTA_BASE_PTR ->PDOR)
#define PORTA_PDIR (PTA_BASE_PTR ->PDIR)

#define PORTB_PDDR (PTB_BASE_PTR ->PDDR)
#define PORTB_PDOR (PTB_BASE_PTR ->PDOR)
#define PORTB_PDIR (PTB_BASE_PTR ->PDIR)

#define PRC_V (PORT_PCR_MUX(1) | PORT_PCR_DSE_MASK)

void systemInit(void) {
    //init clock das portas
    SIM_SCGC5 |= (SIM_SCGC5_PORTA_MASK | SIM_SCGC5_PORTB_MASK);
    //configura para usar clock interno em 24MHz
    MCG_BASE_PTR ->C4 |= 0x80;
    //portb 5, remover o NMI
    PORTB_PCR(5) = PRC_V;
}
void pinMode(int pin, int type) {
```

```
23      if (type == OUTPUT) {
24          switch (pin) {
25          case 0: PORTB_PCR(2)  = PRC_V; bitSet(PORTB_PDDR, 2);  break;
26          case 1: PORTB_PCR(1)  = PRC_V; bitSet(PORTB_PDDR, 1);  break;
27          case 2: PORTA_PCR(11) = PRC_V; bitSet(PORTA_PDDR, 11); break;
28          case 3: PORTB_PCR(5)  = PRC_V; bitSet(PORTB_PDDR, 5);  break;
29          case 4: PORTA_PCR(10) = PRC_V; bitSet(PORTA_PDDR, 10); break;
30          case 5: PORTA_PCR(12) = PRC_V; bitSet(PORTA_PDDR, 12); break;
31          case 6: PORTB_PCR(6)  = PRC_V; bitSet(PORTB_PDDR, 6);  break;
32          case 7: PORTB_PCR(7)  = PRC_V; bitSet(PORTB_PDDR, 7);  break;
33          case 8: PORTB_PCR(10) = PRC_V; bitSet(PORTB_PDDR, 10); break;
34          case 9: PORTA_PCR(11) = PRC_V; bitSet(PORTA_PDDR, 11); break;
35          case 10: PORTA_PCR(5) = PRC_V; bitSet(PORTA_PDDR, 5);  break;
36          case 11: PORTA_PCR(7) = PRC_V; bitSet(PORTA_PDDR, 7);  break;
37          case 12: PORTA_PCR(6) = PRC_V; bitSet(PORTA_PDDR, 6);  break;
38          case 13: PORTB_PCR(0) = PRC_V; bitSet(PORTB_PDDR, 0);  break;
39
40          case 18: PORTA_PCR(9) = PRC_V; bitSet(PORTA_PDDR, 9);  break;
41          case 19: PORTB_PCR(13)= PRC_V; bitSet(PORTB_PDDR, 13); break;
42          default: break;
43          }
44      }
45      if (type == INPUT) {
46          switch (pin) {
47          case 12: PORTA_PCR(6)  = PRC_V; bitClr(PORTA_PDDR, 6);  break;
48          case 13: PORTB_PCR(0)  = PRC_V; bitClr(PORTB_PDDR, 0);  break;
49          case 18: PORTA_PCR(9)  = PRC_V; bitClr(PORTA_PDDR, 9);  break;
50          case 19: PORTB_PCR(13) = PRC_V; bitClr(PORTB_PDDR, 13); break;
51          default:  break;
52          }
53      }
54  }
```

```
55 void digitalWrite(int pin, int value) {
56     if (value) {
57         switch (pin) {
58         case 0:bitSet(PORTB_PDOR, 2);break;
59         case 1:bitSet(PORTB_PDOR, 1);break;
60         case 2:bitSet(PORTA_PDOR, 11);break;
61         case 3:bitSet(PORTB_PDOR, 5);break;
62         case 4:bitSet(PORTA_PDOR, 10);break;
63         case 5:bitSet(PORTA_PDOR, 12);break;
64         case 6:bitSet(PORTB_PDOR, 6);break;
65         case 7:bitSet(PORTB_PDOR, 7);break;
66         case 8:bitSet(PORTB_PDOR, 10);break;
67         case 9:bitSet(PORTB_PDOR, 11);break;
68         case 10:bitSet(PORTA_PDOR, 5);break;
69         case 11:bitSet(PORTA_PDOR, 7);break;
70         case 12:bitSet(PORTA_PDOR, 6);break;
71         case 13:bitSet(PORTB_PDOR, 0);break;
72 
73         case 18:bitSet(PORTA_PDOR, 9);break;
74         case 19:bitSet(PORTB_PDOR, 13);break;
```

```
75          default:break;
76          }
77      } else {
78          switch (pin) {
79          case 0:bitClr(PORTB_PDOR, 2);break;
80          case 1:bitClr(PORTB_PDOR, 1);break;
81          case 2:bitClr(PORTA_PDOR, 11);break;
82          case 3:bitClr(PORTB_PDOR, 5);break;
83          case 4:bitClr(PORTA_PDOR, 10);break;
84          case 5:bitClr(PORTA_PDOR, 12);break;
85          case 6:bitClr(PORTB_PDOR, 6);break;
86          case 7:bitClr(PORTB_PDOR, 7);break;
87          case 8:bitClr(PORTB_PDOR, 10);break;
88          case 9:bitClr(PORTB_PDOR, 11);break;
89          case 10:bitClr(PORTA_PDOR, 5);break;
90          case 11:bitClr(PORTA_PDOR, 7);break;
91          case 12:bitClr(PORTA_PDOR, 6);break;
92          case 13:bitClr(PORTB_PDOR, 0);break;
93
94          case 18:bitClr(PORTA_PDOR, 9);break;
95          case 19:bitClr(PORTB_PDOR, 13);break;
96          default:break;
97          }
98      }
99  }
```

```
100 int digitalRead(int pin) {
101     switch (pin) {
102     case 0:return bitTst(PORTB_PDIR, 2);
103     case 1:return bitTst(PORTB_PDIR, 1);
104     case 2:return bitTst(PORTA_PDIR, 11);
105     case 3:return bitTst(PORTB_PDIR, 5);
106     case 4:return bitTst(PORTA_PDIR, 10);
107     case 5:return bitTst(PORTA_PDIR, 12);
108     case 6:return bitTst(PORTB_PDIR, 6);
109     case 7:return bitTst(PORTB_PDIR, 7);
110     case 8:return bitTst(PORTB_PDIR, 10);
111     case 9:return bitTst(PORTB_PDIR, 11);
112     case 10:return bitTst(PORTA_PDIR, 5);
113     case 11:return bitTst(PORTA_PDIR, 7);
114     case 12:return bitTst(PORTA_PDIR, 6);
115     case 13:return bitTst(PORTB_PDIR, 0);
116
117     case 18:return bitTst(PORTA_PDIR, 9);
118     case 19:return bitTst(PORTB_PDIR, 13);
119     default:break;
120     }
121     return -1;
122 }
```

13.3 Exercícios

Ex. 13.1 — Um mesmo terminal pode ser configurado como entrada e saída? De que forma?

Ex. 13.2 — O que é necessário fazer para que o processador possa ler informações de um terminal físico?

Ex. 13.3 — Crie as definições para acessar as portas A, B, C e D de um microcontrolador. Essas portas se encontram nos endereços **`0x9EFDE`**, **`0x9EFDF`**, **`0x9EFE0`** e a maior, no **`0x9EFE1`**.

Ex. 13.4 — Qual a função dos registros TRIS/DDR/PDDR? Desenhe seu esquema de funcionamento.

Ex. 13.5 — Crie um programa cíclico que realize a leitura de 3 chaves na porta B, nos bits 1, 2 e 3. Em seguida, este programa deverá acender uma quantidade de leds correspondente ao somatório dos números das chaves. Ex: Se as chaves 1 e 3 estiverem pressionadas, 4 leds serão acesos. Se as chaves 1, 2 e 3 estiverem pressionadas, 6 leds serão acesos. Se nenhuma das chaves estiver pressionada, todos os leds devem ser apagados. As chaves podem ser lidas através da variável PORTB e os leds se encontram na variável PORTD. Os registros de configuração de entrada/saída estão localizados no DDRB e DDRD.

Ex. 13.6 — Uma prensa industrial possui vários sistemas de segurança para evitar que seja acionada de maneira errada. Um deles é um sensor de fim de curso que fica ativo quando a porta está fechada. Um segundo sensor indutivo fica dentro do equipamento e verifica se a peça foi colocada corretamente. Por fim, o acionamento é feito utilizando dois botões. Estes botões devem ser pressionados ao mesmo tempo. Faça um programa que verifique se os sensores de segurança (fim de curso no bit 4 da porta E e indutivo no bit 7 da porta E) e acione a prensa (bit 5 da porta E) quando os dois botões forem pressionados. Caso os botões forem pressionados em tempos diferentes, o programa deve aguardar os botões serem soltos antes de verificar novamente. Os botões estão nos bits 2 e 3 da Porta E.

CAPÍTULO

14

Saídas digitais

14.1Acionamentos
- Leds
- Transistor
- Relé
- Relé de estado sólido
- Ponte H

14.2Controle de Led RGB
- Criação da biblioteca

14.3Expansão de saídas
- Conversor serial-paralelo
- Criação da biblioteca

14.4Exercícios

"A revolução digital é muito mais significativa do que a invenção da escrita, ou mesmo da prensa de impressão."

Douglas Engelbart

Diversos componentes eletrônicos possuem apenas dois tipos de estados: ligado ou desligado. Um exemplo bastante simples é uma lâmpada, que pode estar acesa ou apagada. Para realizar o controle destes dispositivos são utilizadas saídas digitais.

As saídas digitais apresentam dois estados distintos. Isto é feito para permitir que o programador consiga ligar ou desligar os componentes. O controle deste tipo de saída é bastante simples. Na maioria das vezes, estas saídas estão mapeadas em uma determinada região da memória do microcontrolador. Ao acessar essa memória e fazer com que o bit tenha o valor **1**, a saída passa para um estado alto. Fazendo o bit receber o valor **0**, faz com que o estado da saída seja baixo.

A maioria das portas de entrada ou de saída digitais possui mais de um terminal conectado à mesma posição de memória. Deste modo, para acionar um bit devemos tomar cuidado para não alterar os demais. Para isso, podemos utilizar as rotinas **`bitClr()`** e **`bitSet()`**.

Por estado alto entendemos que o terminal físico apresentará uma tensão positiva em relação ao terra na saída. No estado baixo, a saída apresentará o mesmo potencial que o terra.

Para os sistemas eletrônicos, a saída de nível alto depende da tecnologia e do microcontrolador utilizado. Os níveis mais comuns são 3,3 e 5,0 volts.

O Arduino fornece tensões de 5v nos seus terminais digitais, com uma capacidade de corrente de 20mA por terminal. A Freedom e a Chipkit operam com 3,3 volts, a primeira permitindo até 4mA por terminal e a segunda, 12mA. A tabela 14.1 apresenta estas e outras informações elétricas das placas.

Tabela 14.1: Capacidade de corrente e tensão

Placa	Tensão	Corrente (por terminal)	Corrente (máximo da placa)
Arduino Uno	5, 0V	20mA	200mA
Chipkit Uno	3, 3V	12mA	200mA
Freedom KL05	3, 3V	4mA	200mA

A placa de desenvolvimento foi projetada para aceitar comandos tanto de 3,3 quanto de 5 volts. Deste modo, não há diferença no método de acionamento. O nível de tensão tem que ser considerado pelo projetista de hardware.

14.1 Acionamentos

A capacidade de acionamento de uma saída digital de um microcontrolador é relativamente baixa, cerca de 10 miliamperes em 3,3 ou 5 volts. Para conseguir acionar cargas maiores, é preciso se utilizar de um circuito de amplificação. Estes circuitos são conhecidos como drivers de acionamento.

Do ponto de vista computacional, o acionamento de um led ou de um chuveiro é o mesmo. A diferença está nos drivers de acionamento utilizados e, consequentemente, na capacidade de corrente de cada um.

Nesta seção serão apresentados os circuitos de drivers mais comuns em pequenos e médios acionamentos.

Leds

Um dos componentes mais simples de ser acionado digitalmente são os led's. Led é a abreviação de light emitting diode, ou diodo emissor de luz. Os led's são componentes semicondutores bastante similares aos diodos. Quando uma corrente elétrica circula por um led, ele passa a emitir luz. A intensidade da luz é proporcional à corrente que passa pelo led.

Por possuir uma estrutura similar à do diodo, o led só funciona se a corrente passar num sentido. Se a corrente tentar passar pelo sentido contrário, ela é barrada pelas propriedades do semicondutor, fazendo com que o led não acenda. O led é representado pelo mesmo símbolo do diodo, com duas setas, indicando a luz que sai do componente. Os leds podem ser encontrados praticamente em qualquer coloração.

O circuito mais simples para o acionamento do led é colocá-lo em série com um resistor e aplicar a tensão desejada. O resistor funciona como limitador de corrente para evitar que o led consuma muita corrente e queime. A figura 14.1 apresenta o circuito deste acionamento.

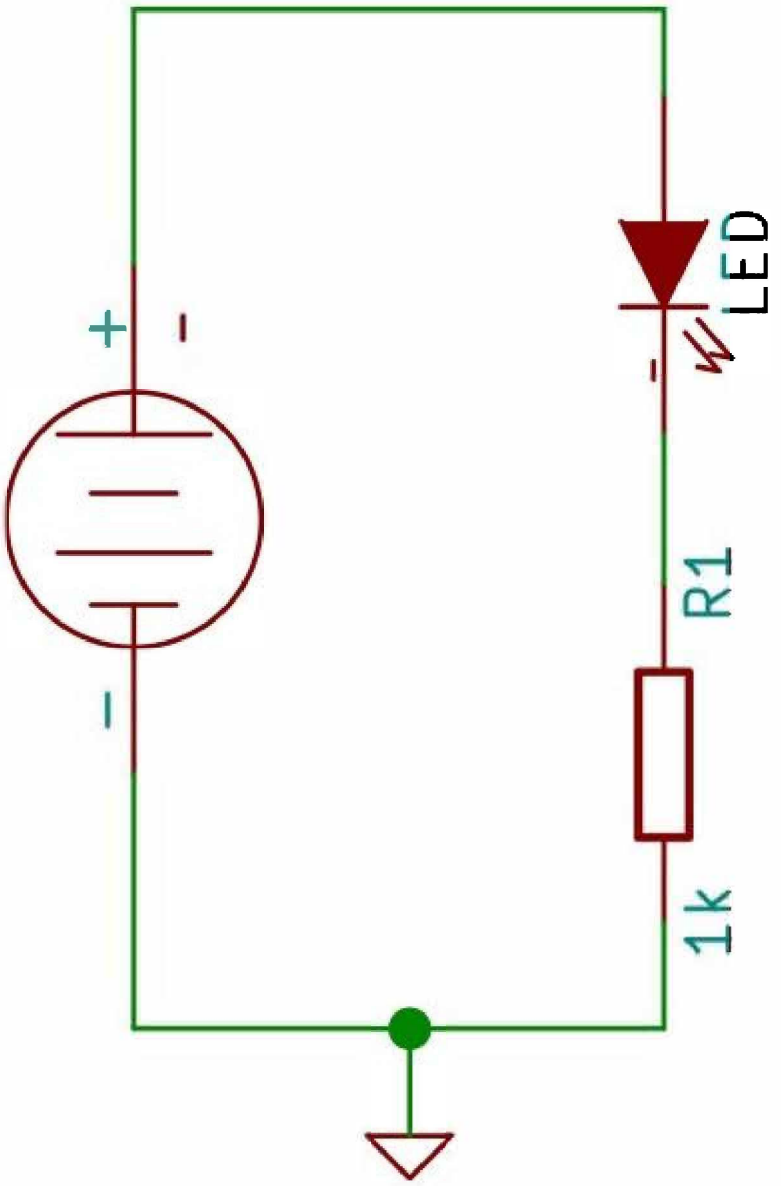


Figura 14.1: Circuito de acionamento de um LED

Para permitir que através de um único componente seja possível gerar qualquer cor de luz, foi desenvolvido um led com as três cores primárias de emissão: vermelho, verde e azul. Este componente na realidade é composto por três led's montados num mesmo encapsulamento. A figura 14.2 apresenta este componente.

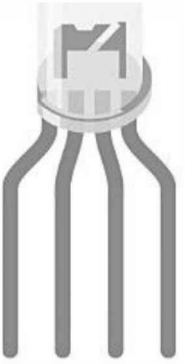


Figura 14.2: Led RGB
Fonte: Imagem produzida com Fritzing/Inkscape

As cores primárias de emissão são diferentes das cores primárias de reflexão: azul, amarelo e vermelho. Na emissão, a formação da luz amarela é feita através da adição da luz vermelha e da luz verde.

Transistor

O transistor é um componente eletrônico que pode funcionar tanto como um amplificador quanto como uma chave. No modo amplificador, ele possui a capacidade de ampliar o nível de tensão ou corrente de um sinal. Como chave, ele permite que através de um pequeno sinal de entrada possamos ligar cargas maiores em sua saída.

Eles podem vir em diversos tamanhos e modelos (figura 14.3). Esta variação depende principalmente da capacidade de corrente de cada transistor.

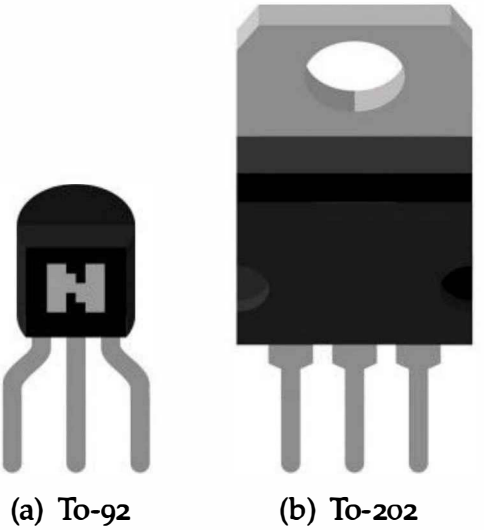


Figura 14.3: Modelos de encapsulamento de transistores
Fonte: Imagens produzidas com Fritzing/Inkscape

O transistor possui três terminais. Os transistores do tipo bipolar têm seus terminais denominados base, emissor e coletor. Para funcionar como chave, basta que a tensão aplicada na base seja maior, cerca de 1 volt a mais, que a tensão no emissor.

Nos circuitos de acionamento que operam como chave, em geral, utiliza-se o terminal emissor conectado ao terra, de modo que a tensão gerada pelo microcontrolador, mesmo os de 3,3v, seja suficiente para que o transistor ligue.

Com o transistor ligado, a corrente consegue então circular da tensão positiva até o terra através da carga e do transistor, como no circuito da figura 14.4.

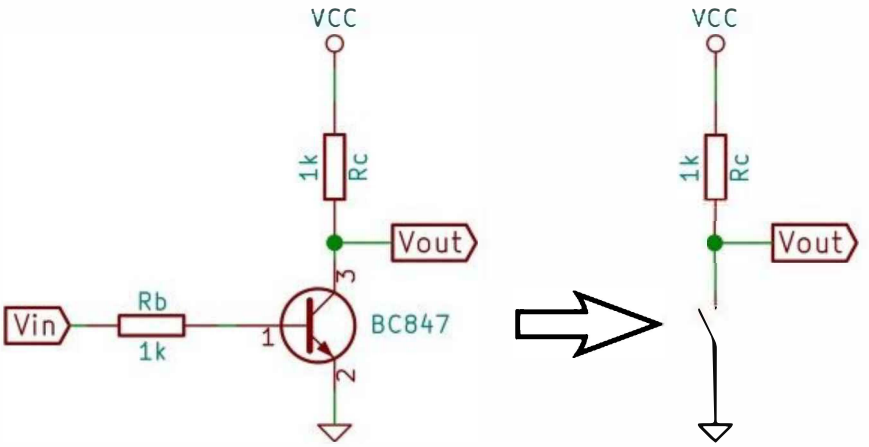


Figura 14.4: Transistor operando como chave

Esta estrutura é o modelo utilizado para fazer a ligação do display quádruplo de 7 segmentos da placa base, dado que o microcontrolador não conseguiria fornecer energia para todos os 32 leds.

Relé

Os transistores de baixo custo possuem algumas limitações com relação ao acionamento. Em geral, não conseguem manipular correntes muito altas, nem mesmo operar em corrente alternada.

A energia elétrica é, em sua maioria, gerada, transmitida e distribuída em corrente alternada. Isto impede o uso de transistores simples para acionamento de boa parte dos equipamentos. Uma opção neste caso é fazer uso de dispositivos eletromecânicos, conhecidos como relés (figura 14.5).

Figura 14.5: Relé eletromecânico
Fonte: Imagem produzida com Fritzing/Inkscape

Os relés possuem pequenas chaves elétricas, muito parecidas com os interruptores residenciais. A diferença é que esses interruptores não são acionados através de um botão, mas sim de um campo eletromagnético que atrai a chave, fazendo com que ela seja acionada. O campo é formado através da circulação de uma corrente em um fio enrolado. A ação de fechar e soltar a chave é que produz o som característico do relé, parecido com um estalo.

Se conseguirmos controlar a passagem dessa corrente por esse fio, poderemos ligar e desligar o relé. Este, por sua vez, funciona como uma chave que pode ligar ou desligar qualquer tipo de equipamento. O problema é que os relés possuem uma corrente de acionamento muito alta para os microcontroladores. Uma opção é fazer uso do acionamento por um transistor. A figura 14.6 apresenta o circuito utilizado como interface entre um microcontrolador e um relé.

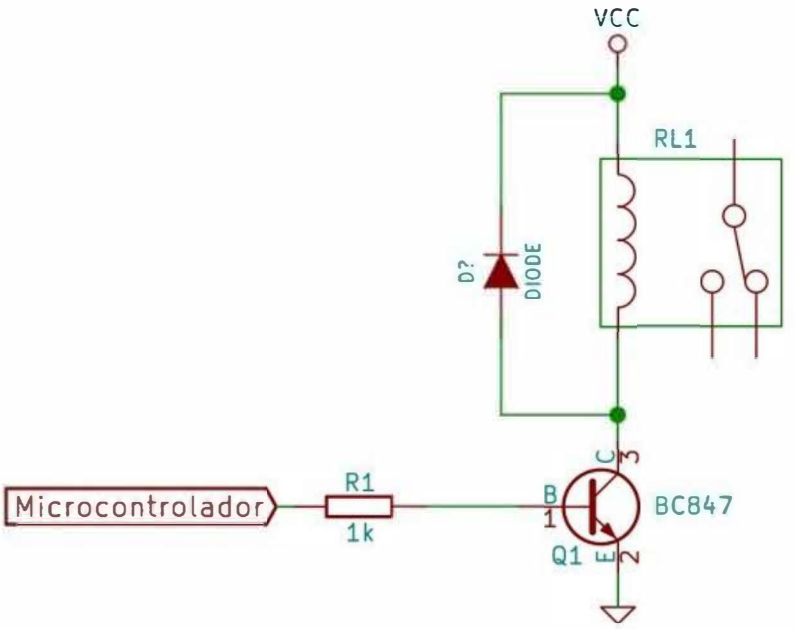


Figura 14.6: Acionamento de um relé eletromecânico

Este circuito possui uma outra vantagem: não existe ligação elétrica entre a carga e o microcontrolador. O relé garante isolação elétrica, de modo que o acionamento das cargas é bastante seguro. Qualquer problema que aconteça na rede elétrica ou no equipamento, seja um curto ou uma sobrecorrente, não afetará o sistema embarcado.

O diodo inserido em paralelo com o relé serve para proteger o relé contra surtos de tensão.

Relé de estado sólido

Os relés de estado sólido funcionam do mesmo modo que os relés eletromecânicos, com a diferença de não possuírem partes móveis, como a chave (figura 14.7).

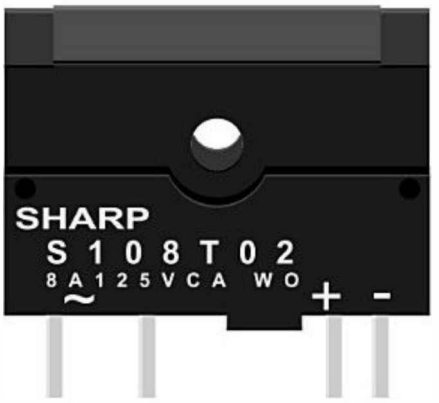


Figura 14.7: Relé de estado sólido
Fonte: Imagem produzida com Fritzing/Inkscape

Seu acionamento é mais simples que o dos relés eletromecânicos, pois não necessita utilizar um transistor.

Em geral, estes sistemas possuem o acionamento interno realizado através de uma interface ótica. Deste modo, conseguimos o mesmo efeito de isolação elétrica dos relés eletromecânicos, enquanto se consome pouca corrente na entrada. A figura 14.8 apresenta o circuito de acionamento:

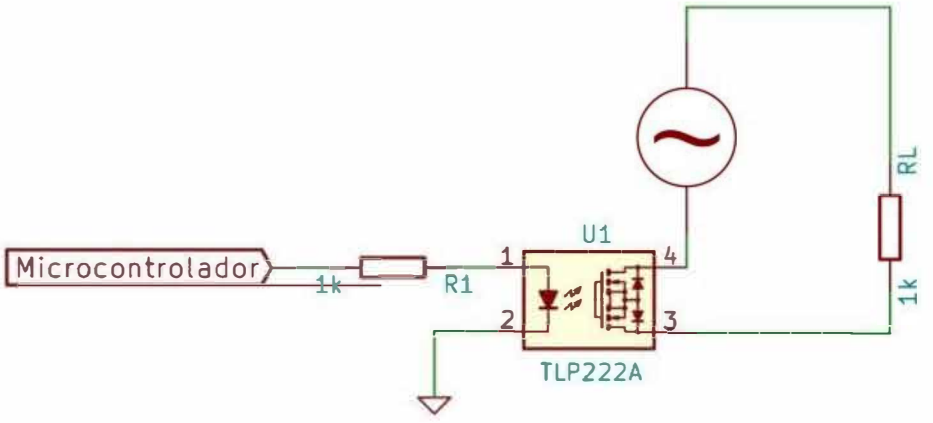


Figura 14.8: Circuito de acionamento de um relé de estado sólido

A entrada de controle deve se manter entre os níveis especificados pelo fabricante. Para o MP120D2, por exemplo, esse valor pode variar de 3 a 24 volts. Com uma faixa tão extensa, esses circuitos podem ser utilizados em sistemas com lógica de 3,3 ou 5 volts. Na sua saída, ele consegue acionar cargas de 127 volts e 2 amperes.

Ponte H

Uma aplicação muito comum é a utilização das chaves já mencionadas (transistores, relés ou relés de estado sólido) para efetuar o controle de motores de corrente contínua (figura 14.9).

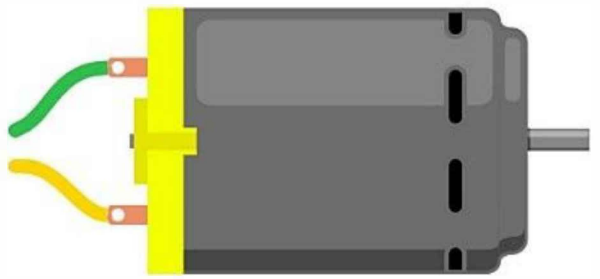


Figura 14.9: Motor DC
Fonte: Imagem produzida com Fritzing/Inkscape

O funcionamento destes motores é bastante simples: se uma tensão positiva for aplicada no terminal 1, ele gira em sentido horário, se for aplicada no terminal 2, ele gira em sentido anti-horário.

Para facilitar o processo de controle destes motores, foi desenvolvida uma topologia de controle com chaves chamadas ponte H. Deste modo, é possível inverter o sentido da corrente sem precisar mudar as ligações físicas do circuito, como apresentado na figura 14.10.

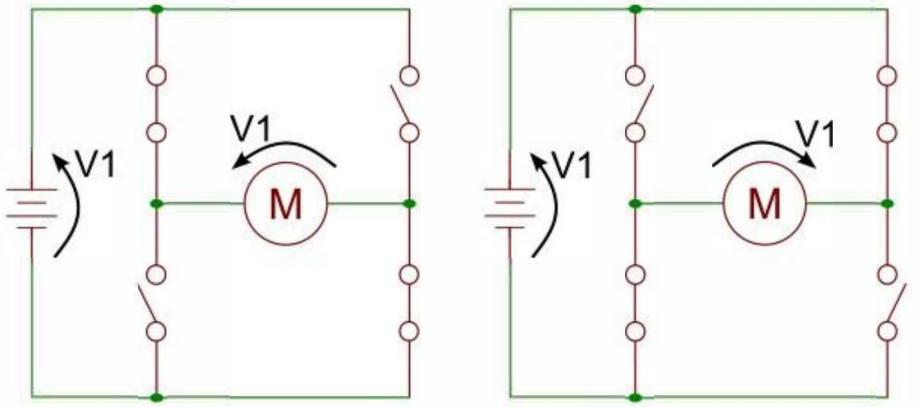


Figura 14.10: Motor DC controlado por ponte H

Para realizar o acionamento deste circuito precisamos ter controle das quatro chaves. Um dos modos é ligar estas chaves nos terminais do microcontrolador através dos sistemas com transistores ou relés. Supondo que as chaves estejam ligadas aos terminais digitais 3, 4, 5 e 6, podemos criar a rotina para acionamento do motor conforme o código 14.1.

Código 14.1: Código para acionamento de uma ponte H transistorizada

```
#define swtich1A 3
#define swtich1B 4
#define swtich2A 5
#define swtich2B 6

void initMotorControl(void){
//configura os quatro terminais como saida
    pinMode(swtich1A, OUTPUT);
    pinMode(swtich1B, OUTPUT);
    pinMode(swtich2A, OUTPUT);
    pinMode(swtich2B, OUTPUT);
}

void motorOff(void){
    digitalWrite(swtich1A, LOW);
    digitalWrite(swtich1B, LOW);
    digitalWrite(swtich2A, LOW);
    digitalWrite(swtich2B, LOW);
}

void motorOnLeft(void){
    //sempre desliga primeiro
    digitalWrite(swtich2A, LOW);
    digitalWrite(swtich2B, LOW);
    digitalWrite(swtich1A, HIGH);
    digitalWrite(swtich1B, HIGH);
}

void motorOnRight(void){
    //sempre desliga primeiro
```

```
31      digitalWrite(swtich1A, LOW);
32      digitalWrite(swtich1B, LOW);
33      digitalWrite(swtich2A, HIGH);
34      digitalWrite(swtich2B, HIGH);
35  }
```

Com essa topologia, devemos tomar cuidado para não ligar ao mesmo tempo as duas chaves da esquerda (1a e 1b) ou as duas da direita (2a e 2b). Se isto acontecer, a corrente passará diretamente da alimentação para o terra, gerando um curto-circuito.

Uma maneira de se evitar esse problema é utilizar um circuito dedicado de acionamento, como o DVR8833 ou o SN754410. Eles realizam todas as proteções e facilitam a utilização dos motores. Ambos os circuitos permitem acionar até dois motores com duas pontes H distintas. O circuito de acionamento pode ser simplificado para o da figura 14.11.

O acionamento, do ponto de vista computacional, também é simplificado, pois existem apenas dois terminais de controle de direção e um terminal que habilita/desliga o motor. Com relação aos terminais de direção, o primeiro liga o motor para a esquerda e o segundo para a direita. Se ambos os terminais forem ligados, o circuito de interface realiza a proteção do sistema e evita o curto-circuito. Para simplificar ainda mais este acionamento, podemos criar funções específicas para ligar o motor para a esquerda ou para a direita, como no código 14.2.

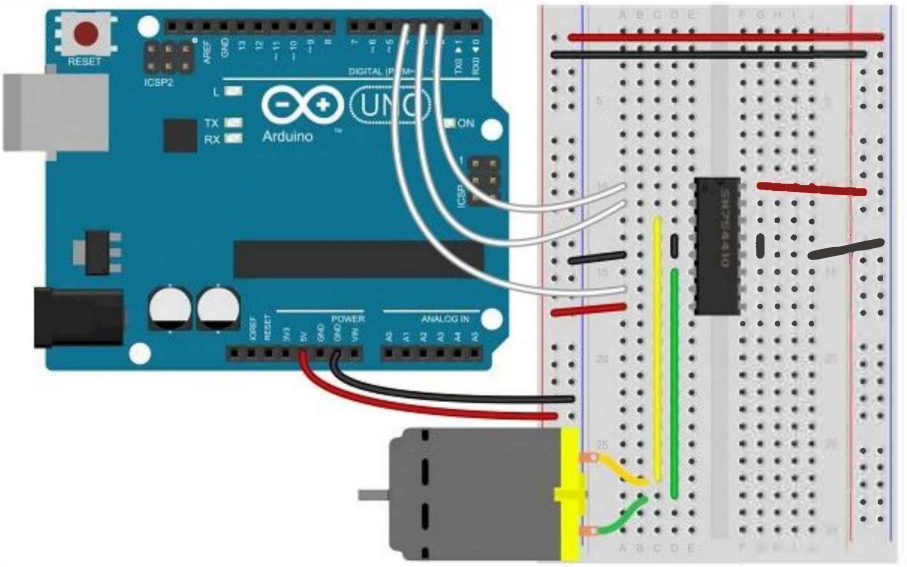


Figura 14.11: Ponte H microcontrolada
Fonte: Imagem produzida com Fritzing/Inkscape

Código 14.2: Código para acionamento de uma ponte H

```
#define motor1Pin  3
#define motor2Pin  4
#define enablePin  2

void initMotorControl(void){   //configura os três terminais como saida
    pinMode(motor1Pin, OUTPUT);
    pinMode(motor2Pin, OUTPUT);
    pinMode(enablePin, OUTPUT);
}
void motorOff(void){
    digitalWrite(enablePin, LOW);
}
void motorOnLeft(void){
    digitalWrite(enablePin, HIGH);
    digitalWrite(motor1Pin, LOW);
    digitalWrite(motor2Pin, HIGH);
}
void motorOnRight(void){
    digitalWrite(enablePin, HIGH);
    digitalWrite(motor1Pin, HIGH);
    digitalWrite(motor2Pin, LOW);
}
```

14.2 Controle de Led RGB

Na placa base, existe um led RGB conectado aos terminais 2, 3 e 4. Estes terminais estão em portas diferentes, dependendo da placa de controle utilizada (Arduino, Chipkit ou Freedom).

O circuito de conexão do led na placa é apresentado na figura 14.12. São utilizados três resistores de 1k para limitar a quantidade de corrente de cada um dos leds.

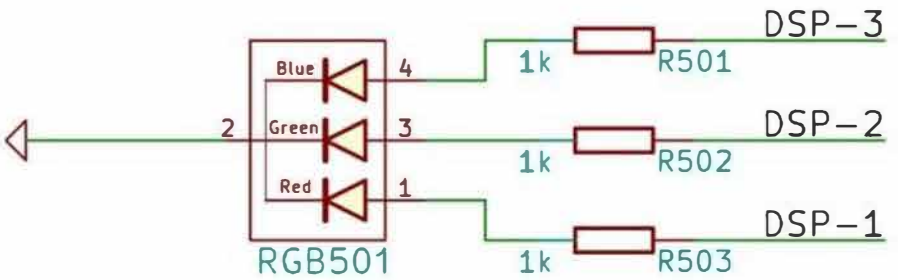


Figura 14.12: Circuito de conexão do Led RGB

Para ligar ou desligar o led, é necessário configurar primeiro os terminais como saídas digitais. Em cada placa os terminais são distintos, mas como a função **pinMode()** e

`digitalWrite()` fazem o mapeamento correto entre cada uma das placas e os terminais digitais, o acesso aos leds pode ser feito de maneira idêntica em todas elas.

Criação da biblioteca

A ideia de se desenvolver uma biblioteca para o led RGB é simplificar o processo de utilização do periférico. Como o led pode acender cada uma das cores individualmente ou misturá-las, podemos criar uma rotina que receba como parâmetro qual é a cor que o usuário deseja exibir.

Para isto, será criada uma lista de definições das cores disponíveis. Essas definições serão descritas no arquivo de header para ser disponibilizado ao programador. O header está apresentado no código 14.3 e a implementação das funções no código 14.4.

Código 14.3: Header da biblioteca de Led RGB

```
 1 #ifndef RGB
 2 #define RGB
 3
 4 //todos desligados
 5 #define OFF    0
 6
 7 //cores primárias
 8 #define RED    1
 9 #define GREEN  2
10 #define BLUE   4
11
12 //cores secundárias
13 #define YELLOW (RED+GREEN)
14 #define CYAN   (GREEN+BLUE)
15 #define PURPLE (RED+BLUE)
16
17 //todos acesos
18 #define WHITE  (RED+GREEN+BLUE)
19
20     void rgbColor(int led);
21     void turnOn(int led);
22     void turnOff(int led);
23     void rgbInit(void);
24 #endif
```

Os códigos das cores foram desenvolvidos com base em cada uma das cores primárias. O vermelho ocupa o primeiro bit, o verde, o segundo e o azul, por sua vez, o terceiro. Deste modo, um valor binário **0b111** representa todos os três leds acesos. As funções levam isso em conta no momento de exibir a cor desejada.

Código 14.4: Código da biblioteca de Led RGB

```
#include "io.h"

void rgbColor(int led) {
    if (led & 1) {
        digitalWrite(LED_R_PIN, HIGH);
    } else {
        digitalWrite(LED_R_PIN, LOW);
    }
    if (led & 2) {
        digitalWrite(LED_G_PIN, HIGH);
    } else {
        digitalWrite(LED_G_PIN, LOW);
    }
    if (led & 4) {
        digitalWrite(LED_B_PIN, HIGH);
    } else {
        digitalWrite(LED_B_PIN, LOW);
    }
}
void turnOn(int led) {
    if (led & 1) {
        digitalWrite(LED_R_PIN, HIGH);
    }
    if (led & 2) {
```

```
25          digitalWrite(LED_G_PIN, HIGH);
26      }
27      if (led & 4) {
28          digitalWrite(LED_B_PIN, HIGH);
29      }
30 }
```

```
31 void turnOff(int led) {
32      if (led & 1) {
33          digitalWrite(LED_R_PIN, LOW);
34      }
35      if (led & 2) {
36          digitalWrite(LED_G_PIN, LOW);
37      }
38      if (led & 4) {
39          digitalWrite(LED_B_PIN, LOW);
40      }
41 }
42 void rgbInit(void) {
43      pinMode(LED_R_PIN, OUTPUT);
44      pinMode(LED_G_PIN, OUTPUT);
45      pinMode(LED_B_PIN, OUTPUT);
46 }
```

As definições dos terminais são dadas pelo header do arquivo io.h. Devemos lembrar que os leds estão ligados juntos dos terminais de controle do display de 7 segmentos. O correto funcionamento do display de sete segmentos impede o uso eficiente dos leds.

14.3 Expansão de saídas

Um dos quesitos relacionados ao custo de um microcontrolador é a quantidade de saídas disponíveis. Dependendo do custo do projeto, é comum que o microcontrolador dentro do preço, não possua saídas suficientes para a aplicação desejada.

Nestes casos, podemos utilizar circuitos específicos para a expansão de saídas. Esta expansão pode ser feita de diversos modos. Os mais comuns são os conversores de serial para paralelo, expansores de IO, multiplexação temporal dos terminais ou multiplexação em frequências diferentes.

Cada abordagem acarreta algum tipo de custo no sistema: financeiro, atraso na resposta ou aumento da complexidade no acionamento.

Conversor serial-paralelo

Um conversor serial-paralelo muito utilizado é o chip LM74HC595 (figura 14.13). Este conversor recebe os sinais de modo serial e os transforma em um conjunto de 8 bits em paralelo.

Para que este dispositivo funcione, existem três sinais de controle: shift register clock (SHCP), que controla a velocidade do envio dos bits, storage register clock (STCP), que repassa os dados recebidos até o momento para a saída paralela, e output enable (OE), que habilita a saída dos dados.

Os bits são enviados através do terminal SHCP. Para que o bit seja recebido pelo 74HC595, é necessário enviar um sinal do tipo pulso no terminal STCP.

Figura 14.13: Conversor serial-paralelo 74HC595
Fonte: Imagem produzida com Fritzing/Inkscape

Um sinal do tipo pulso é um sinal em que o terminal, estando em nível baixo, sobe para o nível alto durante um tempo e volta para o nível baixo. Ao longo do tempo, a forma desse sinal é parecida com a apresentada na figura 14.14.

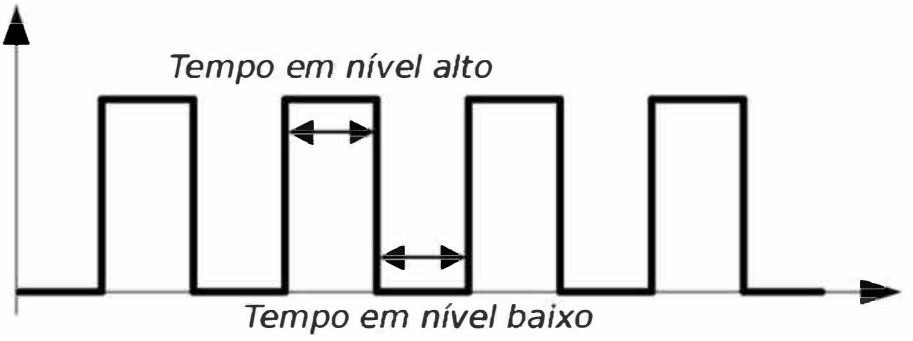


Figura 14.14: Pulso de clock

O código para gerar esse tipo de pulso é dado pelo código 14.5. A primeira função gera um pulso de clock no terminal STCP e a segunda no terminal RCLK.

Código 14.5: Geração de pulso de clock

```
1 void PulseEnClock(){
2     digitalWrite(SO_EN_PIN, HIGH);
3     digitalWrite(SO_EN_PIN, LOW);
4 }
5
6 void PulseClockData(){
7     digitalWrite(SO_CLK_PIN, HIGH);
8     digitalWrite(SO_CLK_PIN, LOW);
9 }
```

Para enviar os oito bits, precisamos inicialmente definir de qual extremo binário iremos começar, pelo bit mais significativo ou pelo bit menos significativo. Pela estrutura do 74HC595, para que o bit 7 apareça na saída Q7, devemos começar pelo bit mais significativo.

O código para enviar cada um dos bits utiliza um loop **for** que testa cada um dos bits. Este bit é então colocado no terminal SCHP e um comando de pulso é enviado. Esta função pode ser visualizada no código 14.6.

Código 14.6: Envio de dados serializados para o 74hc595

```
 1 void soWrite(int value) {
 2     int i;
 3     digitalWrite(SO_CLK_PIN, LOW);
 4     for (i = 0; i < 8; i++) {
 5         digitalWrite(SO_DATA_PIN, value & 0x80);
 6         PulseClockData();
 7         value <<= 1;
 8     }
 9     PulseClock();
10 }
```

A última linha da função envia um pulso no terminal OE. Este terminal é o responsável por atualizar as saídas, pois, enquanto os bits são enviados, são armazenados na memória interna do 74HC595. Assim, os bits seriais que foram armazenados são disponibilizados nos terminais paralelos de saída.

O circuito complexo da ligação do 74HC595 com o processador é apresentado na figura 14. 15.

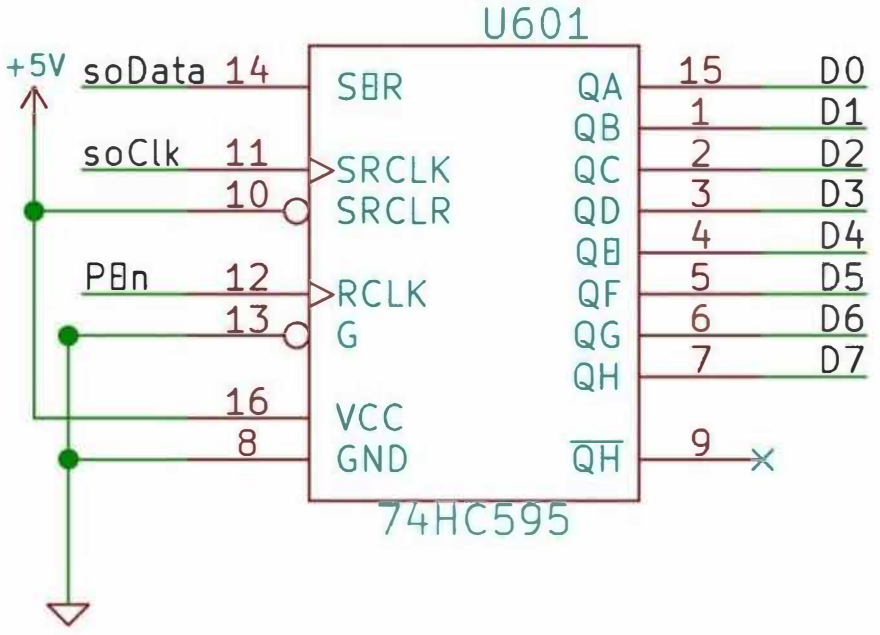


Figura 14.15: Ligações do 74HC595 com a placa de controle

Criação da biblioteca

O conversor serial para paralelo pode ser utilizado como uma nova porta de saída de dados, sendo composta por oito terminais. Deste modo, será criado um barramento de dados novo que terá a capacidade de enviar até 8 bits de informação para um destes periféricos. Para facilitar a utilização deste barramento para o programador, é interessante criar uma biblioteca que simplifique o envio dessas informações. O header está apresentado no código 14.7 e a implementação das funções, no código 14.8.

Código 14.7: Header da biblioteca serial-paralela

```
1 #ifndef SO_H_
2 #define SO_H_
3
4     void soInit (void);
5     void soWrite(int value);
6
7 #endif
```

Código 14.8: Código da biblioteca serial-paralela

```
#include "io.h"

void soInit(void) {
    pinMode(SO_EN_PIN, OUTPUT);
    pinMode(SO_CLK_PIN, OUTPUT);
    pinMode(SO_DATA_PIN, OUTPUT);
}
//pulso de clock para habilitar os dados na sáida
void PulseEnClock(void){
    digitalWrite(SO_EN_PIN, HIGH);
    digitalWrite(SO_EN_PIN, LOW);
}
//pulso de clock para enviar um bit
void PulseClockData(void){
    digitalWrite(SO_CLK_PIN, HIGH);
    digitalWrite(SO_CLK_PIN, LOW);
}
void soWrite(int value) {
    char i;
    digitalWrite(SO_CLK_PIN, LOW);
    for (i = 0; i < 8; i++) {
        digitalWrite(SO_DATA_PIN, value & 0x80);
        PulseClockData();
        value <<= 1;
    }
    PulseEnClock();
}
```

Este periférico vai ser utilizado para acionar outros circuitos da placa, principalmente o display de LCD, o display de 7 segmentos e o teclado. Isto foi feito pois as placas de controle não possuem terminais suficientes para todos os dispositivos.

14.4 Exercícios

Ex. 14.1 — Num dado equipamento, a porta D possui oito terminais com um led cada. Dado o programa abaixo, indique quais leds estão ligados. No equipamento em questão, os leds ligam com nível alto.

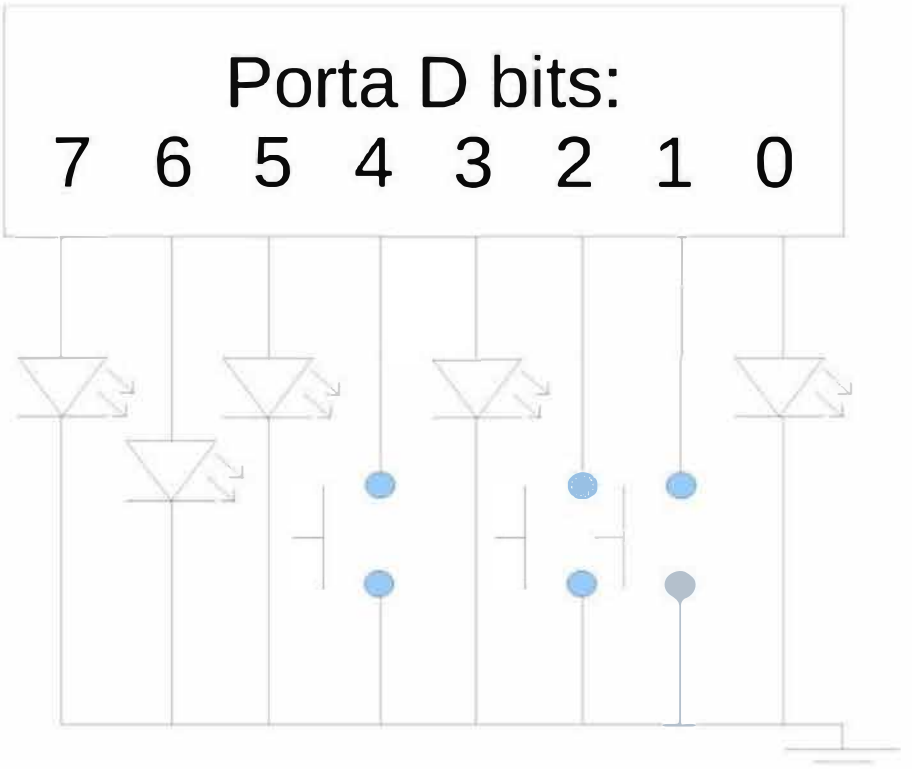
```
1 #include "config.h"
2 #include "basico.h"
3 void main(void){
4       TRISD = 0xE3;
5       PORTD = 0xAA;
6       for(;;);
7 }
```

Ex. 14.2 — Crie uma biblioteca chamada "BarramentoLeds". Esta biblioteca realizará operações com o led RGB. Crie funções que sejam capazes de:

- Ligar os leds de acordo com um parâmetro recebido. Este parâmetro será uma variável unsigned char onde os leds que devem ser ligados possuem valor 1 no seu bit respectivo. Ex: 0x03 - ligar os leds 0 e 1; 0x06 - ligar os leds 1 e 2. Os demais leds não podem ser alterados.
- Desligar os leds de acordo com um parâmetro recebido. O funcionamento é similar ao Ligar, com a diferença que os leds que devem ser desligados possuem valor zero.
- Inicializar o sistema.
- Retornar o estado atual dos leds

Ex. 14.3 — Crie um programa que utiliza a biblioteca criada no exercício anterior para gerar um efeito de acendimento de cada uma das cores de modo sequencial.

Ex. 14.4 — O bit na posição i da variável não sinalizada de 8 bits TRISD controla o sentido dos dados da porta D. Um bit com número 0 (zero) significa que aquele terminal é de saída. Um bit com número 1 significa que aquele terminal é de entrada. Configure TRISD para o circuito ao lado. Apresente o valor de TRISD em binário, hexadecimal e decimal. Considere que os leds são dispositivos de saída e as chaves, dispositivos de entrada.



Ex. 14.5 — Construa uma biblioteca chamada "controle_led" que permita ao programador acessar cada um dos leds individualmente através de duas fuções: "LigaLed", que recebe como parâmetro um char indicando qual posição deve ser ligada e "DesligaLed", que também recebe um char indicando a posição a ser desligada. Além disso, crie a função InicializaLED(), que recebe como parâmetro dois endereços (ambos unsigned int). O primeiro indica onde estão os leds. Este endereço deverá ser armazenado num ponteiro de char para ser usado depois pelas outras duas funções. O segundo endereço representa o controle de entrada e saída de onde estão os leds. Utilize-o para configurar todos os oito bits como saída (valor zero). Lembre-se de fazer os dois arquivos da biblioteca.

Ex. 14.6 — Crie um programa que leia um valor de distância de um sensor ultrassônico na entrada analógica e apresente a distância no barramento de leds (porta D). Este sensor consegue identificar distâncias de zero cm a 80 cm. Quando há um objeto a zero cm, a saída do sensor apresenta 0 volts (0 no AD). Quando o objeto está à distância de 80 cm, a saída fica saturada em 5volts (1024 no AD). O sensor possui uma resposta linear. Pode-se utilizar a biblioteca "adc.h". O programa deve deixar todos os leds acesos quando o sensor indicar objeto a zero centímetros e apagar um led a cada 10 cm, de modo que todos os leds fiquem apagados quando o objeto estiver a 80 cm ou mais. Os leds acendem com valor zero (lógica invertida).

```
1 //adc.h
2 //inicializa o conversor
3 void adInit(void);
4 //devolve o valor lido pelo conversor
5 unsigned int readAD(void);
```

CAPÍTULO

15

Display de 7 segmentos

15.1 Multiplexação de displays
Criação da biblioteca
15.2 Projeto: Relógio
15.3 Exercícios

"A luz do mundo vem, principalmente, de duas fontes: o sol e a lâmpada do estudante."

Christian Nestell Bovee

As soluções para exibição de informações em sistemas embarcados são dispositivos que tem a capacidade de transformar os sinais elétricos que saem do microcontrolador em alguma representação visível ao ser humano.

O sistema mais simples de exibição são os leds. Através da corrente gerada pelos terminais do microcontrolador é possível controlar o estado do led, permitindo apresentar ao usuário uma informação do tipo ligado ou desligado.

Procurando desenvolver soluções que conseguissem passar ao usuário um conjunto maior de informações que simplesmente ligado/desligado, foram desenvolvidos os displays de 7 segmentos.

Os displays de 7 segmentos (figura 15.1) são os componentes optoeletrônicos mais simples que podem ser utilizados para apresentar informações numéricas ao usuário.

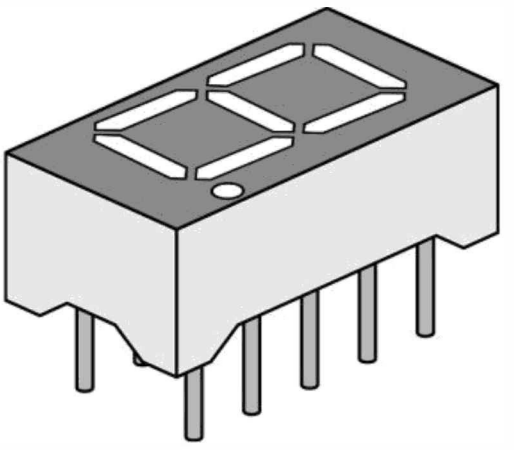


Figura 15.1: Display de 7 segmentos
Fonte: Imagem produzida com Fritzing/Inkscape

Estes displays foram concebidos com o intuito de gerar os dez algarismos arábicos: **0**, **1**, **2**, **3**, **4**, **5**, **6**, **7**, **8**, **9** a partir de sete comandos digitais. Cada comando digital é responsável por ligar ou desligar um dos segmentos. Estes segmentos são iluminados por leds, trazendo a vantagem de um baixo consumo de energia.

Além dos algarismos, é possível representar apenas algumas letras: as maiúsculas **A**, **C**, **E**, **F**, **H**, **J**, **L**, **I**, **O**, **P**, **S**, **U**, **Z** e as minúsculas **b**, **c**, **d**, **h**, **i**, **n**, **o**, **r**, **t**, **u**. Deve-se, no entanto, tomar cuidado pois algumas letras podem ser confundidas com os números, como a letra maiúscula **O** e número **0**.

Alguns displays também apresentam um oitavo segmento: o ponto decimal. Este led pode ser utilizado para indicar onde fica o separador da parte inteira do número com a parte fracionária. Os displays utilizam por padrão o ponto como separador ao invés da virgula por causa do padrão americano de escrita.

Os displays podem ser encontrados nos mais diferentes modelos, cores e tamanhos. Com relação à estrutura de funcionamento, eles podem ser divididos em dois tipos: cátodo comum ou ânodo comum. Esta diferença está relacionada ao modo de ligação dos leds. O led é um elemento polarizado, de forma que ele precisa que a corrente circule num determinado sentido. Para facilitar o projeto, dentro dos displays, um dos lados de todos os leds são conectados num único ponto, como apresentado na figura 15.2. Para os dispositivos cátodo comum, este ponto comum deve ser ligado no terra e os leds são ligados com tensão positiva. Para os dispositivos ânodo comum, o terminal comum deve ser ligado na tensão positiva e os leds são ligados colocando-se o valor zero.

Os segmentos são nomeados utilizando as letras **a**, **b**, **c**, **d**, **e**, **f**, **g** e **dp**. Eles são nomeados em sentido anti-horário a partir do segmento mais alto.

Para desenhar o número **2** no display é necessário acender os segmentos **a**, **b**, **g**, **e** e **d**, e desligar os demais segmentos. Se estivermos utilizando um display com cátodo comum, precisamos colocar um nível alto para ligar o led, ou seja, o led liga com valor **1** (um) e desliga com valor **0** (zero).

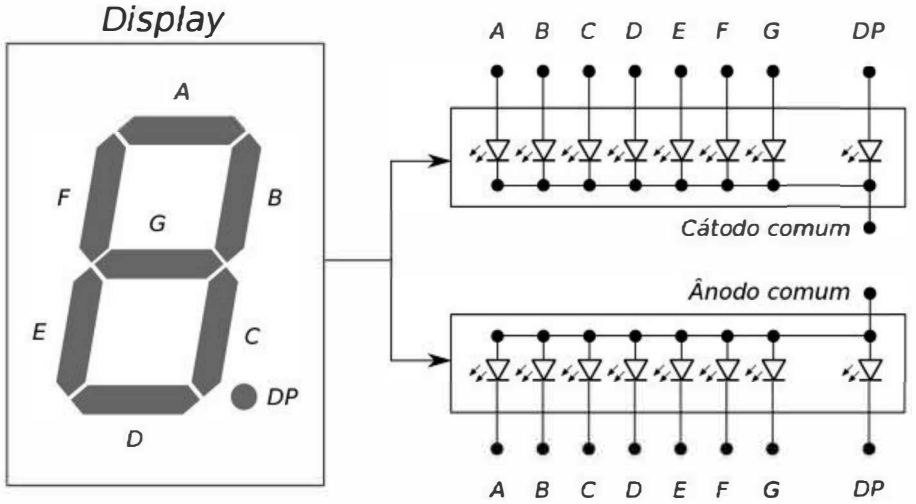


Figura 15.2: Diagrama elétrico de um display de 7 segmentos, ânodo e cátodo comum

Na placa de controle, os leds são acionados através do conversor série-paralelo 74HC595. Por este motivo faremos a inclusão da biblioteca **so.h**, para enviar os valores de acionamento para cada led do display de 7 segmentos.

A tabela 15.1 apresenta os valores para cada dígito hexadecimal a ser exibido. Dentre as letras disponíveis, estão apresentadas apenas os caracteres **A**, **b**, **C**, **d**, **E**, **F**. Elas foram escolhidas por serem utilizadas para apresentar valores em hexadecimal nos displays.

Tabela 15.1: Conversão binário - hexadecimal para displays de 7 segmentos

Dígito	e	d	0	c	g	a	f	b	Hex (ed0 cgafb)
0	1	1	0	1	0	1	1	1	D7
1	0	0	0	1	0	0	0	1	11
2	1	1	0	0	1	1	0	1	CD
3	0	1	0	1	1	1	0	1	5 D
4	0	0	0	1	1	0	1	1	1 B
5	0	1	0	1	1	1	1	0	5 E
6	1	1	0	1	1	1	1	0	DE
7	0	0	0	1	0	1	0	1	15
8	1	1	0	1	1	1	1	1	DF
9	0	1	0	1	1	1	1	1	5 F
A	1	0	0	1	1	1	1	1	9 F
b	1	1	0	1	1	0	1	0	DA
C	1	1	0	0	0	1	1	0	C6
d	1	1	0	1	1	0	0	1	D9
E	1	1	0	0	1	1	1	0	CE
F	1	0	0	0	1	1	1	0	8 E

Devemos notar que os valores apresentados na tabela 15.1 só têm validade se a ligação entre os leds e o barramento forem idênticos aos utilizados na placa de desenvolvimento. Outros modelos de placas podem possuir ligações diferentes, sendo necessário readequar as posições das colunas para gerar os valores hexadecimais.

Para simplificar a utilização deste tipo de display, podemos criar um vetor, de modo que na posição *i* seja salvo o valor hexadecimal responsável por desenhar o caractere *i* no display de 7 segmentos. O código 15.1 apresenta esta solução:

Código 15.1: Exibição de caracteres no display de 7 segmentos

```
#include "so.h"
#include "io.h

void main(void){
    //vetor que armazena a conversão dos algarismos para o display 7 seg
    static const char conv[] = {0xD7, 0x11, 0xCD, 0x5D, 0x1B, 0x5E, 0xDE, 0x15, ←
        0xDF, 0x5F, 0x9F, 0xDA, 0xC6, 0xD9, 0xCE, 0x8E};
    unsigned int var;
    float time;
    initSystem();
    soInit();
    pinMode(DISP1_PIN,OUTPUT);
    digitalWrite(DISP1_PIN,HIGH);
    for(;;){
        for (var = 0; var < 16; var++){
            //coloca os caracteres em sequência na saída
            soWrite(conv[var]);
            //apenas para contar tempo entre os digitos
            for (time = 0; time < 10000; time++);
        }
    }
}
```

15.1 Multiplexação de displays

Cada display exige pelo menos 7 terminais configurados como saída pelo microcontrolador. Para o desenvolvimento de um relógio que exiba as horas e os minutos seria necessário utilizar 4 displays, exigindo 28 terminais de saída. Esta quantidade de terminais pode fazer com que o custo do produto aumente, inviabilizando projetos mais simples. Os microcontroladores que possuem mais terminais têm um custo maior, mesmo possuindo as mesmas características de memória e periféricos disponíveis.

Uma técnica que pode ser utilizada para reduzir essa necessidade é a multiplexação dos displays. Esta técnica leva em conta um efeito biológico conhecido como percepção retiniana. As imagens mais claras ficam gravadas na retina durante um determinado tempo. É por este motivo que quando olhamos para uma lâmpada, ou qualquer objeto que emita uma quantidade grande de luz, e fechamos os olhos, continuamos "vendo" o objeto. Isto acontece por causa do tempo necessário para sensibilizar e dessensibilizar as células oculares. Este fato faz com que o olho humano seja incapaz de perceber mudanças superiores a 16 imagens por segundo.

Este efeito permite que liguemos um display por um determinado tempo de modo que a imagem se fixe na retina do observador. Ao desligarmos este display, o cérebro continuará percebendo a imagem durante um tempo. Se durante este tempo ligarmos o próximo display, enquanto o efeito de percepção retiniana estiver ativo, para nosso cérebro, ambos os displays estarão ligados simultaneamente.

Fazendo com que esse processo seja cíclico, e suficientemente rápido, podemos fazer com que o usuário tenha a percepção de que todos os displays estão constantemente ligados.

Esta é uma abordagem bastante comum, tanto que existem displays que são desenvolvidos para serem utilizados deste modo. O display da figura 15.3, utilizado na placa de

desenvolvimento, apresenta 4 dígitos de 7 segmentos conjugados num único dispositivo, exibindo a mensagem "**diSP**":

Figura 15.3: Display de 7 segmentos quádruplo
Fonte: Imagem produzida com Fritzing/Inkscape

Esta abordagem permite a redução da quantidade de cabos e terminais utilizados, além de simplificar o projeto da placa. Utilizando 4 displays separados e com acionamentos distintos, são necessários 32 terminais. Com os displays reunidos, ou utilizando uma arquitetura com o barramento compartilhado, reduzimos a quantidade de terminais necessários para 12, uma economia de 20 terminais.

As desvantagens desta solução são o aumento da complexidade do código de acionamento do o display e o maior consumo de processamento para executar todas as atividades.

A figura 15.4 apresenta o circuito utilizado na placa de desenvolvimento. Podemos notar que os terminais responsáveis pelos segmentos (a, b, c, d, e, f, g, dp) estão conectados aos pinos D0 a D7, saída do conversor serial-paralelo. Já os terminais de acionamento dos displays estão ligados a um circuito transistorizado, que é acionado pelos terminais Disp1 a Disp4.

Os transistores são componentes eletrônicos que funcionam como chaves, ligando ou desligando cada um dos displays de modo independente. Este componente é utilizado pois permite que o microcontrolador acione dispositivos que precisam de uma maior quantidade de corrente, que para 8 leds pode chegar a mais de 100mA.

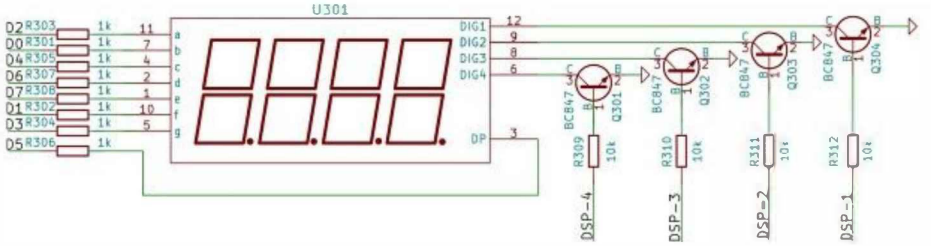


Figura 15.4: Ligação do display quádruplo na placa de desenvolvimento

Para que o display funcione, não basta enviar o valor correto para os leds através do conversor serial-paralelo. Se fizermos isto, todos os displays vão apresentar o mesmo número. Para conseguir exibir números diferentes em cada um dos displays, é necessário utilizar um algoritmo mais complexo, que leva em conta o efeito da percepção retiniana para causar o efeito de que eles estão exibindo números diferentes. O algoritmo pode ser descrito como:

1.colocar no barramento de dados o valor a ser mostrado no display *X*;
2.ligar o display *X* através da linha de comando;
3.esperar um tempo adequado para evitar flicker;

4.desligar o display *X*;

5.escolher o próximo display $X = X + 1$.

No código 15.2 é apresentada a implementação deste algoritmo. Como cada display possui um bit de acionamento diferente, optou-se por criar uma estrutura de case para simplificar a ligação de cada display.

Código 15.2: Exibição de caracteres no display de 7 segmentos

```
1 #include "so.h"
2 int n1, n2, n3, n4; //numeros a serem exibidos
3 const char conv[] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F, ←
      0x77, 0x7C, 0x39, 0x5E, 0x79, 0x71};
4 int display;
5 void ssdUpdate(){
6     float t;
7     switch(display){
8         case 1:                          //1o display
9             soWrite(conv[n1]);
10            digitalWrite(DISP1_PIN,HIGH);
11            for(t=0;t<10;t++);
12            digitalWrite(DISP1_PIN,LOW);
13            break;
14        case 2:                          //2o display
15            soWrite(conv[n1]);
16            digitalWrite(DISP2_PIN,HIGH);
17            for(t=0;t<10;t++);
18            digitalWrite(DISP2_PIN,LOW);
19            break;
```

```
20          case 3:                             //3o display
21              soWrite(conv[n1]);
22              digitalWrite(DISP3_PIN,HIGH);
23              for(t=0;t<10;t++);
24              digitalWrite(DISP3_PIN,LOW);
25              break;
26          case 4:                             //4o display
27              soWrite(conv[n1]);
28              digitalWrite(DISP4_PIN,HIGH);
29              for(t=0;t<10;t++);
30              digitalWrite(DISP4_PIN,LOW);
31              break;
32          default:
33              display = 1;
34              break;
35      }
36      display++; //prepara para o próximo
37  }
38
39  void main (void){
40      soInit();
41      //config terminais de controle dos displays
42      n1 = 1; n2 = 2; n3 = 3; n4 = 4;
43      for(;;){
44          ssdUpdate();
45          //restante do código.
46      }
47  }
```

Este código, apesar de funcionar corretamente, apresenta um problema grave para sua utilização com outras bibliotecas. Gasta-se muito tempo com os loops para exibição dos números dentro da função **`ssdUpdate()`**. Este tempo é necessário para que o cérebro humano consiga enxergar todos os dígitos acesos. No entanto, se outras funcionalidades forem adicionadas ao **`main`**, o sistema pode passar a gastar muito tempo entre os acionamentos, o que fará com que os displays permaneçam mais tempo desligados, atrapalhando a visualização de todos os dígitos.

Para evitar esse problema, é possível modificar o algoritmo invertendo a sequência, de modo que o tempo possa ser deslocado para fora da função. Como o algoritmo é cíclico, alterar a ordem não impactará no funcionamento do algoritmo. Podemos então descrevê-lo como:

1.desligar o display *X*;
2.escolher o próximo display *X* = *X* + 1;
3.colocar no barramento de dados o valor a ser mostrado no display *X*;
4.ligar o display *X* através da linha de comando;
5.esperar um tempo adequado para evitar flicker.

Deste modo, o novo algoritmo começa no antigo passo de número 4 e a etapa de esperar um tempo com o display ligado passa a ser o último elemento da lista. Os quatro primeiros passos são então implementados na função e o último é deixado para que o programador o realize fora da função.

Código 15.3: Exibição de caracteres no display de 7 segmentos com o tempo fora da função ssdUpdate()

```
1 void ssdUpdate(void){
2     //desliga todos os displays
3     digitalWrite(DISP1_PIN,LOW);
4     digitalWrite(DISP2_PIN,LOW);
5     digitalWrite(DISP3_PIN,LOW);
6     digitalWrite(DISP4_PIN,LOW);
7
8     switch(display){ //liga apenas o display da vez
9         case 0:
10            soWrite(valor[v0]);
11            digitalWrite(DISP1_PIN,HIGH);
12            display = 1;
13            break;
14        case 1:
15            soWrite(valor[v1]);
16            digitalWrite(DISP2_PIN,HIGH);
17            display = 2;
18            break;
19        case 2:
20            soWrite(valor[v2]);
21            digitalWrite(DISP3_PIN,HIGH);
22            display = 3;
23            break;
24        case 3:
25            soWrite(valor[v3]);
26            digitalWrite(DISP4_PIN,HIGH);
27            display = 0;
28            break;
```

```
29          default:
30              display = 0;
31              break;
32      }
33  }
34
35  void main (void){
36      float t;
37      soInit();
38      //config terminais de controle dos displays
39      n1 = 1; n2 = 2; n3 = 3; n4 = 4;
40      for(;;){
41          ssdUpdate();
42          for(t=0;t<10;t++);
43          //restante do código.
44      }
45  }
```

Como podemos ver, agora a responsabilidade de criar um tempo para que o display fique ligado é do programa principal. A responsabilidade da função **`ssdUpdate()`** é desligar o display atual, atualizar o número e ligar o próximo.

A adição de mais código ao programa principal não causará tantos problemas, o número exibido apenas ficará mais tempo visível. Isto pode levar o sistema a apresentar o problema do flicker. Neste caso, basta diminuir o tempo de espera no loop principal. Num sistema real é comum não precisar gastar tempo desnecessário apenas para manter o funcionamento. Nestes casos, o tempo de processamento pode ser utilizado para realizar atividades úteis para o funcionamento da placa.

Código 15.4: Rotina principal sem necessidade de delay

```
 1 void main (void){
 2     soInit();
 3     //config terminais de controle dos displays
 4     for(;;){
 5         ssdUpdate();
 6         //O processamento gasta tempo suficiente
 7         //para que o número seja visto pelo usuário
 8         processamentoUtil();
 9     }
10 }
```

Criação da biblioteca

O programa 15.6 apresenta um exemplo de código para criar uma biblioteca para os displays de 7 segmentos utilizando o conversor serial-paralelo como barramento de dados. O programa 15.5 apresenta o header desta biblioteca. É interessante notar as variáveis **v0** a **v3**, que armazenam o valor a ser exibido de modo que o programador não precise ficar atualizando essa informação a cada execução da atualização do display.

Código 15.5: ssd.h

```
1
2 #ifndef DISP7SEG_H
3 #define DISP7SEG_H
4     void ssdDigit(char position, char value);
5     void ssdUpdate(void);
6     void ssdInit(void);
7 #endif
```

A utilização da biblioteca é bastante simples. A função **ssdInit()** inicializa todos os recursos de hardware necessários para o funcionamento do display. A função **ssdDigit()** é a responsável por modificar o dígito que será exibido em cada posição do display. Por fim, a função **ssdUpdate()** realiza o procedimento de multiplexação, trocando o display ativo de acordo com as informações armazenadas pela função **ssdDigit()**.

Código 15.6: ssd.c

```
#include "ssd.h"
#include "so.h"
#include "io.h"

//vetor para armazenar a conversao do display :edOcgafb
static const char valor[] = {0xD7, 0x11, 0xCD, 0x5D, 0x1B, 0x5E, 0xDE, 0x15, 0xDF, 0x5F, 0x9F, 0xDA, 0xC6, 0xD9, 0xCE, 0x8E};
static char display;  //armazena qual e o display disponivel
//armazena os valores a serem enviados ao display
static char v0=0, v1=0, v2=0, v3=0;

void ssdDigit(char position, char value){
    if (position == 0){ v0 = value; }
    if (position == 1){ v1 = value; }
    if (position == 2){ v2 = value; }
    if (position == 3){ v3 = value; }
}

void ssdUpdate(void){
    //desliga todos os displays
    digitalWrite(DISP1_PIN,LOW);
    digitalWrite(DISP2_PIN,LOW);
    digitalWrite(DISP3 PIN,LOW);
    digitalWrite(DISP4 PIN,LOW);
    switch(display){ //liga apenas o display da vez
    case 0:
        soWrite(valor[v0]); digitalWrite(DISP1 PIN,HIGH); display = 1;
        break;
    case 1:
        soWrite(valor[v1]); digitalWrite(DISP2_PIN,HIGH); display = 2;
        break;
    case 2:
        soWrite(valor[v2]); digitalWrite(DISP3_PIN,HIGH); display = 3;
        break;
    case 3:
        soWrite(valor[v3]); digitalWrite(DISP4_PIN,HIGH); display = 0;
        break;
    default:
        display = 0;
        break;
    }
}

void ssdInit(void){
    soInit();
    //configurao dos pinos de controle
    pinMode(DISP1_PIN,OUTPUT);
    pinMode(DISP2 PIN,OUTPUT);
    pinMode(DISP3_PIN,OUTPUT);
    pinMode(DISP4_PIN,OUTPUT);
}
```

De acordo com a solução adotada, a responsabilidade de inserir um tempo para que o display permaneça ligado é da função principal. O código 15.7 apresenta um exemplo de uso levando em conta este tempo:

Código 15.7: Utilizando a biblioteca ssd

```
 1 #include "ssd.h"
 2 void main(void){
 3     unsigned int time, count;
 4     initSystem();
 5     ssdInit();
 6     for(;;){
 7         count++;
 8         //separa cada algarismo da variável count
 9         ssdDigit(0, (count     ) %10);
10         ssdDigit(1, (count/10  ) %10);
11         ssdDigit(2, (count/100 ) %10);
12         ssdDigit(3, (count/1000) %10);
13         ssdUpdate();
14         //gasta um tempo para evitar o efeito flicker
15         for(time=0;time<1000;time++);
16     }
17 }
```

15.2 Projeto: Relógio

Os displays de 7 segmentos funcionam muito bem para exibição de informações numéricas. Um exemplo que pode ser realizado com a placa de desenvolvimento é um relógio digital com horas e minutos.

Primeiramente, é necessário termos uma base de tempo confiável. Em geral, utilizamos timers dedicados do processador para isto. Como os timers só serão vistos nos próximos capítulos, será utilizada uma base aproximada. Esta base fará com que cada iteração do loop consuma 10 milissegundos.

De posse deste loop temporizado, podemos criar um contador que será incrementado a cada loop. Quando esse contador atingir 1 segundo, podemos reiniciá-lo e incrementar a variável de segundos. Quando a variável de segundos chegar a 60, devemos reiniciá-la e incrementar uma variável que armazenará os minutos. O procedimento é o mesmo para incrementar as horas. Esta aplicação é apresentada no código 15.8.

Código 15.8: Projeto do relógio com display de 7 segmentos

```
 1 #include "io.h"
 2 #include "ssd.h"
 3
 4 void main(void){
 5     int cont=0, seg=0, min=0, hora=0;
 6     initSystem();
 7     ssdInit();
 8     for(;;){
 9     //o loop demora cerca de 7ms. 7ms*142 ~ 1s
10         if(cont >= 142){
11             seg++;
12             cont = 0;
13         }
14         if(seg>=60){
15             min++;
16             seg = 0;
17         }
18         if(min >= 60){
19             hora++;
20             min = 0;
21         }
22         if(hora >= 24){
23             hora = 0;
24         }
25         ssdDigit(3, (hora/10)%10);
26         ssdDigit(2, hora%10);
27         ssdDigit(1, (min/10)%10);
28         ssdDigit(0, min%10);
29         cont++;
30         ssdUpdate();
31     }
32 }
```

15.3 Exercícios

Ex. 15.1 — Desenhe o esquema elétrico de um display de 7 segmentos e conecte-o a uma porta com 8 bits do microcontrolador.

Ex. 15.2 — Desenvolva o fluxograma do processo de multiplexação de 2 displays de 7 segmentos.

Ex. 15.3 — Modifique a biblioteca apresentada no livro e crie a função **`ssdDotOn`**. Esta função será responsável por permitir a ligação do ponto decimal. A função recebe como parâmetro o display que deve ter seu ponto decimal ligado. Lembre-se que a função **`ssdUpdate`** é a responsável por ligar/desligar efetivamente os leds. A função **`ssdDotOn`** deve apenas armazenar em variáveis internas a necessidade, ou não, de se ligar cada um dos pontos decimais. A função **`ssdUpdate`**, em seu processamento, irá realizar o teste destas variáveis e atuar de modo coerente. Crie também a função **`ssdDotOff`**.

Ex. 15.4 — Crie um calendário com o display de 7 segmentos da placa de controle utilizando a biblioteca "ssd.h". Este calendário deve exibir o dia e o mês com dois dígitos cada. Configure o programa para contabilizar os meses com 28, 30 e 31 dias corretamente. Não se preocupe com os anos bissextos.

Ex. 15.5 — Crie um programa que apresente a sequência apresentada na figura a seguir nos 4 displays de 7 segmentos. Não use a biblioteca "ssd.h". Os displays 1, 2, 3 e 4 são habilitados através dos terminais 2, 5, 3 e 1 da porta E. Os segmentos (leds) são ligados com "1" e são acionados pelos bits da porta C, seguindo a sequência **`gfedcba`**. Para a contagem de tempo, utilize um loop **`for`** entre os acionamentos.

CAPÍTULO

16

Entradas digitais

16.1Debounce por hardware
16.2Debounce por software
Debounce para mais de uma entrada
16.3Arranjo matricial
16.4Criação da biblioteca
16.5Detecção de eventos
16.6Aplicações
Reed Switch
Reed Switch
16.7Exercícios

"*Supostamente deveria ser automático, mas, na verdade, você tem que apertar este botão.*"

John Brunner

Os periféricos que se utilizam de entradas digitais são aqueles que geram um tipo de informação binária: verdadeira ou falsa; pressionado ou solto; presente ou ausente.

Entre estes tipos de periférico, temos chaves, botões, sensores de presença, reedswitches, encoders, entre outros.

O funcionamento da maioria destes sensores se dá pela geração de um sinal de tensão que varia entre dois estados. O sinal baixo, ausente ou falso é normalmente representado por 0v (zero volts) e o sinal alto, presente ou verdadeiro é dado pela tensão de alimentação do circuito de 3,3 ou 5 volts, dependendo do tipo da placa de controle.

Os periféricos que transformam efeitos mecânicos em elétricos se utilizam de uma chave que pode estar aberta ou fechada. É o princípio de funcionamento das microswitches, os pequenos botões comumente utilizados em placas eletrônicas (figura 16.1).

Figura 16.1: Chave do tipo microswitch
Fonte: Imagem produzida com Fritzing/Inkscape

A transformação da posição da chave (aberta ou fechada) para um sinal elétrico é feita através de um arranjo da própria chave, um resistor e uma fonte de alimentação. Existem dois arranjos possíveis: a chave conectada na alimentação e o resistor no terra (pull-down); a chave conectada ao terra com o resistor na tensão de alimentação (pull-up), conforme os circuitos da figura 16.2.

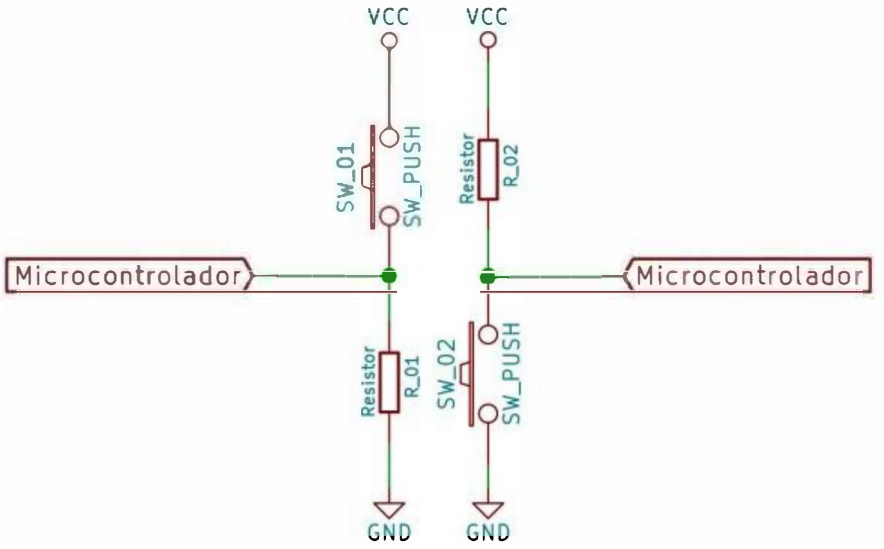


Figura 16.2: Circuito de leitura de chave com resistor de pull-down e pull-up

O circuito com um resistor de pull-down apresenta o funcionamento da chave diretamente similar ao estado encontrado na entrada digital. Isto é, com chave pressionada o sinal apresenta nível lógico alto, igual à tensão de alimentação. Com a chave solta o sinal é igual ao nível lógico baixo, ou zero volts. Esse comportamento é obtido pela posição da chave ligada à alimentação e do resistor ao terra, quando a chave está solta o resistor força a tensão de entrada do microcontrolador a zero volts.

No circuito de pull-up, o funcionamento é o inverso do anterior: a chave pressionada gera um sinal de zero volts, a chave livre faz o resistor elevar a saída para a tensão de alimentação. Isto acontece pois não existe corrente circulando, deste modo a queda de tensão no resistor é zero, e a tensão consegue chegar no microcontrolador. Quando a chave é pressionada, o terminal do microcontrolador passa a ficar aterrado e desta maneira, a tensão do terminal cai para zero, caracterizando o nível lógico baixo no terminal.

O procedimento de leitura deste tipo de sinal é muito simples quando conectado a uma entrada digital: inicializa-se o terminal como entrada e fazemos a leitura do bit correspondente, como no código 16.1.

Código 16.1: Fazendo a leitura de um terminal de entrada diretamente dos registros

```
1 void main (void){
2     //inicialização como entrada
3     BitClr(TRISD,0);
4     for(;;){
5         //teste do bit
6         if(BitTst(PORTD,0)){
7             //A chave está pressionada
8         }else{
9             //A chave está solta
10        }
11    }
12 }
```

O funcionamento deste tipo de circuito, apesar de simples, apresenta um problema. As chaves mais simples e baratas podem apresentar um efeito de oscilação no sinal quando a tecla é pressionada. Este ruído é conhecido como bouncing. A figura 16.3 apresenta a leitura do sinal de uma chave com pull-up ao longo do tempo, evidenciando o problema da oscilação.

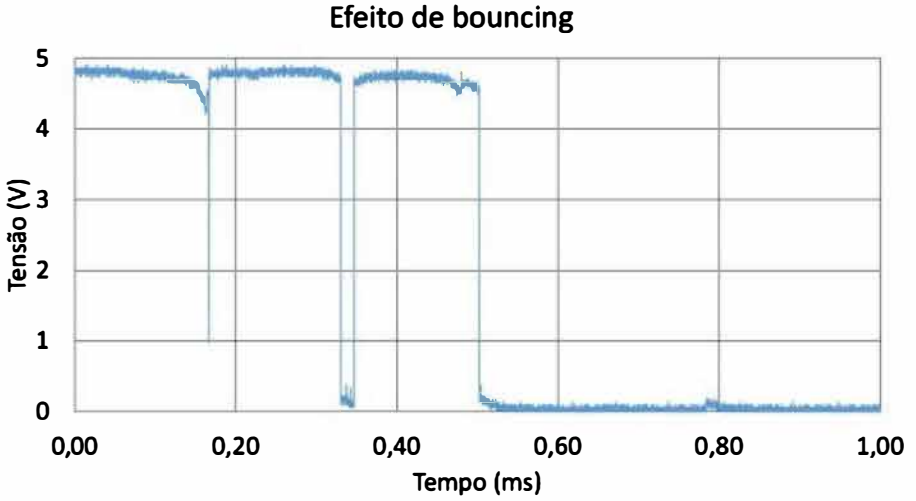


Figura 16.3: Oscilação do sinal no momento do chaveamento

Quando a chave é pressionada, a tensão cai para zero volts. No entanto, a chave é um elemento mecânico que pode vibrar no momento do choque, do mesmo modo que uma bola, que, quando cai no chão, quica algumas vezes antes de parar.

Estas oscilações podem ser interpretadas pelo microcontrolador como vários pressionamentos da chave. Para evitar este problema, podemos utilizar técnicas de debounce, por hardware ou software.

16.1 Debounce por hardware

A opção de debounce por hardware consiste em adicionar elementos eletrônicos que consigam amortecer ou zerar as oscilações.

O circuito de debounce mais simples consiste na adição de um capacitor em paralelo com o terminal do microcontrolador (figura 16.4).

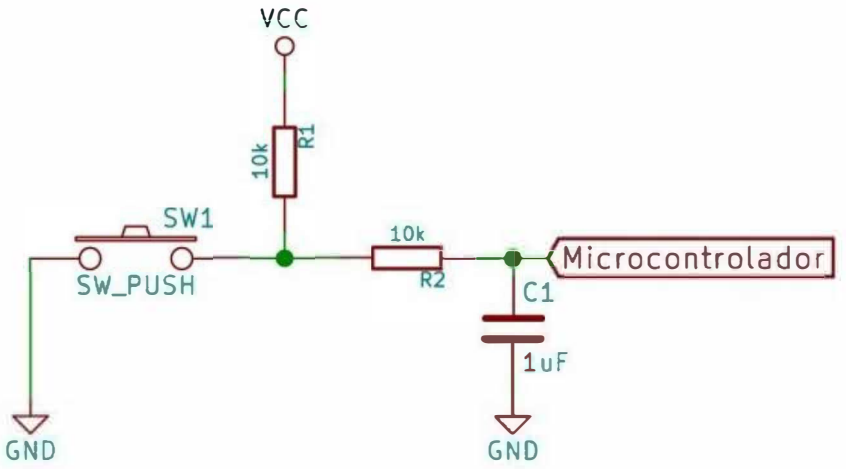


Figura 16.4: Circuito de debounce

O capacitor levará um tempo para ser carregado ou descarregado. Isto fará com que mudanças rápidas no sinal sejam amortecidas ou até mesmo eliminadas. A capacidade de amortecer as mudanças depende da capacitância do capacitor. Capacitores maiores serão capazes de aumentar o amortecimento. Na figura 16.5, é utilizado um capacitor de 1nF.

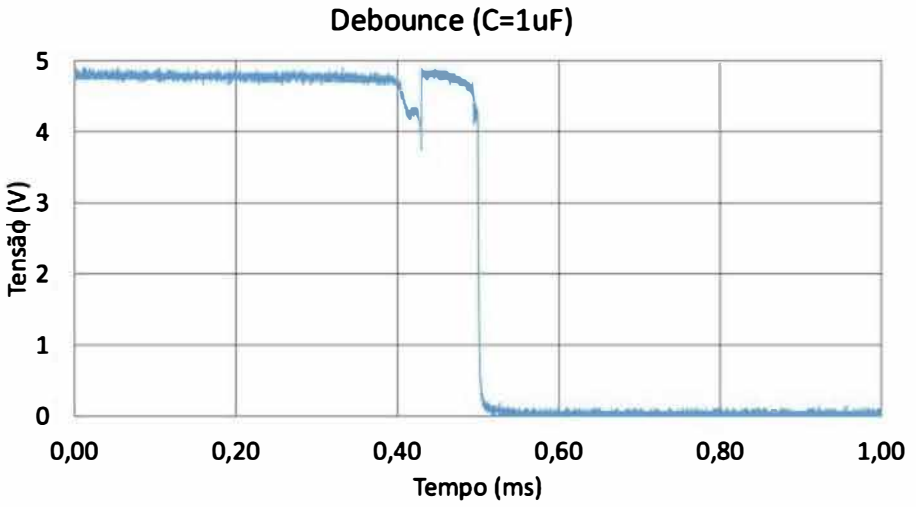


Figura 16.5: Utilização de filtro RC para debounce do sinal

Podemos notar que o nível do sinal filtrado ainda sofre algumas alterações, mas não chega a zerar instantaneamente.

O maior problema dessa abordagem é o custo de produção das placas. Mesmo possuindo um custo relativamente baixo, os capacitores de debounce ainda podem impactar no custo total da placa, principalmente se for levada em conta a produção em escala. Esta solução pode ser inadequada nestas situações.

Outro problema da adição do capacitor é que ele atrasa a detecção do sinal. Após o pressionamento da chave, é preciso esperar que o sinal abaixe de um certo nível para que o sistema detecte a entrada como valor zero. A figura 16.6 apresenta o efeito de um capacitor com maior capacitância, bem como o atraso gerado no tempo de descarga.

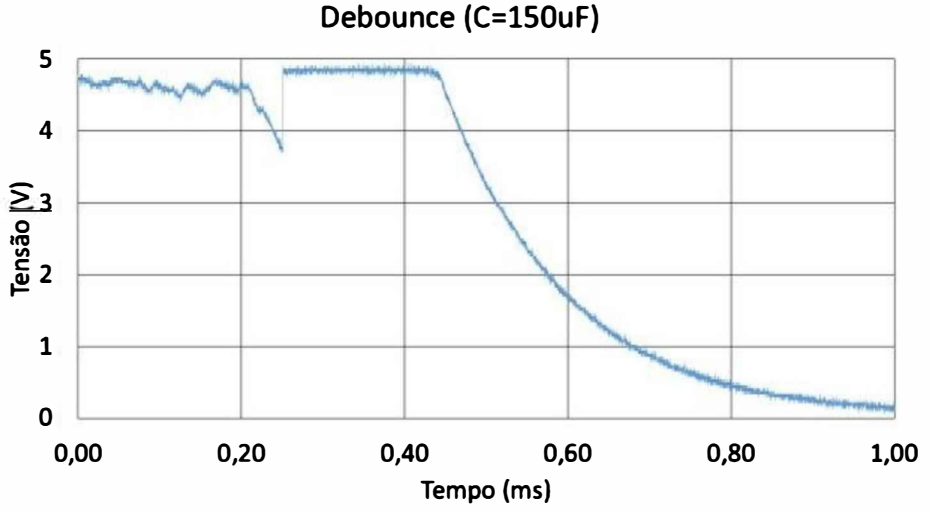


Figura 16.6: Utilização de filtro RC para debounce do sinal

Quando estamos tratando de sistemas sem necessidades muito restritivas para a detecção do acionamento, uma opção é realizar o debounce por software.

16.2 Debounce por software

O debounce por software tenta copiar o efeito do debounce por hardware sem trazer os custos associados à adição de componentes.

A lógica por trás do debounce é aguardar uma mudança no valor da porta. Se este valor se mantiver constante durante um determinado tempo, podemos confiar que o sinal se estabilizou. No entanto, se o sinal mudar novamente antes do tempo de estabilização, devemos reiniciar a contagem. O código 16.2 apresenta o algoritmo de debounce para uma única chave.

Código 16.2: Exemplo de debounce de um sinal digital

```
1 void main (void){
2     int oldValue;
3     int t;
4     //inicialização como entrada
5     pinMode(13, INPUT);
6     //incializa o valor antigo
7     oldValue = digitalRead(13);
8     for(;;){
9         //verifica se houve mudança
10        if (oldValue != digitalRead(13)){
11            //atualiza o sinal;
12            oldValue == digitalRead(13);
13            //se passar por 100 testes o sinal está estável
14            for(t=0; t<100; t++){
15                //se o valor mudar, reinicia a contagem
16                if(oldValue != digitalRead(13)){
17                    t = 0;
18                }
19            }
20        }
21        //aqui o valor da chave é estável
22    }
23 }
```

A grande desvantagem da utilização de uma abordagem de debounce por software é que ela insere um atraso na detecção do sinal, visto que devemos aguardar a estabilização do mesmo.

Um requisito para realizar o debounce por software é conhecer quanto tempo o sinal gerado pela chave leva para se estabilizar. Este tempo varia para cada modelo de chave.

Uma das maneiras de se obter esse tempo é através de um teste com osciloscópio, como na figura 16.3, apresentada anteriormente. Ela mostra o sinal de uma chave sem filtro de bouncing. Com este sinal, podemos inferir o tempo que o software precisa esperar para garantir que a chave estabilizou, realizando corretamente a rotina de debounce.

Debounce para mais de uma entrada

Ao invés de criarmos uma rotina de debounce para cada uma das entradas podemos fazer o debounce de um conjunto de entradas de uma única vez.

A vantagem é que podemos reduzir a necessidade de processamento fazendo o debounce de uma única vez. A maior desvantagem é que, para sistemas cujas entradas se modificam rapidamente, esse processo pode atrasar os sinais ainda mais que o debounce regular.

Para simplificar o processo, uma solução interessante é adicionar todos os sinais numa única variável e então verificar a estabilidade da variável. Se a variável permanecer com o mesmo valor durante o período mínimo do debounce, podemos considerar que todos os sinais estão estáveis.

Código 16.3: Debounce para múltiplas entradas

```
1 int readKeys(void){ //função para ler e agregar as teclas numa única variável
2     int newValue = 0;
3     if (digitalRead(5)) {
4         BitSet(newValue,0);
5     } else {
6         BitClr(newValue,0);
7     }
8     if (digitalRead(9)) {
9         BitSet(newValue,1);
10    } else {
11        BitClr(newValue,1);
12    }
13    if (digitalRead(3)) {
14        BitSet(newValue,2);
15    } else {
16        BitClr(newValue,2);
17    }
18    return newValue;
19 }
20 void main (void){//Rotina principal
21    int oldValue;
22    int t;
23    //inicialização como saida
24    pinMode(INPUT,3);
25    pinMode(INPUT,5);
26    pinMode(INPUT,9);
27    for(;;){
28        //verifica se houve mudança
29        if (oldValue != readKeys()){
30            //atualiza o sinal;
31            oldValue == readKeys();
32            //se passar por 100 testes o sinal está estável
33            for(t=0; t<100; t++){
34                //se o valor mudar, reinicia a contagem
35                if(oldValue != readKeys()){
36                    t = 0;
37                }
38            }
39        }
40        //podemos utilizar o valor das chaves sem problemas
41    }
42 }
```

O processo de debounce até aqui descrito, apresenta o mesmo problema do display de sete segmentos, ou seja, é necessário que o programa fique um determinado tempo aguardando a

estabilização do sinal. Isto pode fazer com que as outras atividades deixem de ser cumpridas.

O procedimento para permitir que a função de debounce não gaste tempo demais em uma só execução é fazer com que a função em si só execute um teste de cada vez. O programa principal será responsável por chamar a função várias vezes. Como o processo de debounce será realizado em várias chamadas, é necessário criar uma variável que indique quantas vezes a função já foi chamada, mantendo uma contagem do tempo decorrido. Também será utilizada uma variável temporária para armazenar o novo valor e verificar sua estabilidade.

Código 16.4: Debounce otimizado

```
1 int debouncedKeys;
2 int tempKey;
3 int debounceCount;
4
5 int readKeys(void){
6     //função para agregar os sinais em uma única variável
7     return debouncedKeys;
8 }
9 void debounce(void){
10    //verifica se houve mudança
11    if(tempKey != readKey()){
12        debounceCount = 10;
13        tempKey = readKey();
14    }
15    //se o valor é igual decrementa o contador
16    if(tempKey == readKey()){
17        if(debounceCount>0){
18            debounceCount--;
19        }
20    }
21    //se o contador chegou a zero pode atualizar pois o sinal está estável
22    if(debounceCount == 0){
23        debouncedKeys = tempKey;
24    }
25 }
26 void main (void){
27    //inicialização como saída
28    for(;;){
29        //realiza o processo de debounce, se houver mudança o valor é atualizado ←
              depois de 10 loops
30        debounce();
31    }
32 }
```

16.3 Arranjo matricial

Em cada tecla ou botão que é inserido no projeto, é necessário um terminal de entrada

digital do microcontrolador. Para um teclado maior, é possível que o microcontrolador não possua terminais disponíveis em quantidade suficiente, ou que o modelo de microcontrolador seja caro demais para o projeto.

Do mesmo modo que nos displays de sete segmentos, é possível multiplexar as entradas de modo que a quantidade de chaves a serem lidas seja maior que a quantidade de terminais disponíveis. Novamente, este ganho em termos de hardware, aumenta a complexidade para o software e, além disso, aumenta o custo em termos de tempo de processamento.

Uma das técnicas mais eficientes para a leitura de um teclado é o arranjo em formato matricial. Com esta configuração podemos, com N terminais, ler até $(N/2)^2$ chaves. Existem até mesmo teclados prontos fabricados em formato matricial.

Neste formato, as chaves são arranjadas em linhas e colunas. Existe apenas uma chave que consegue conectar uma determinada coluna a uma determinada linha.

Um dos problemas com esta estrutura é que pode gerar falsos positivos. Isto faz com que, mesmo que algumas chaves não estejam pressionadas, o valor de saída das linhas seja positivo. Este problema pode ser evitado ao se utilizar um diodo em cada chave para evitar o retorno da corrente e gerar esses falsos positivos. A figura 16.7 apresenta o circuito utilizado na placa de desenvolvimento.

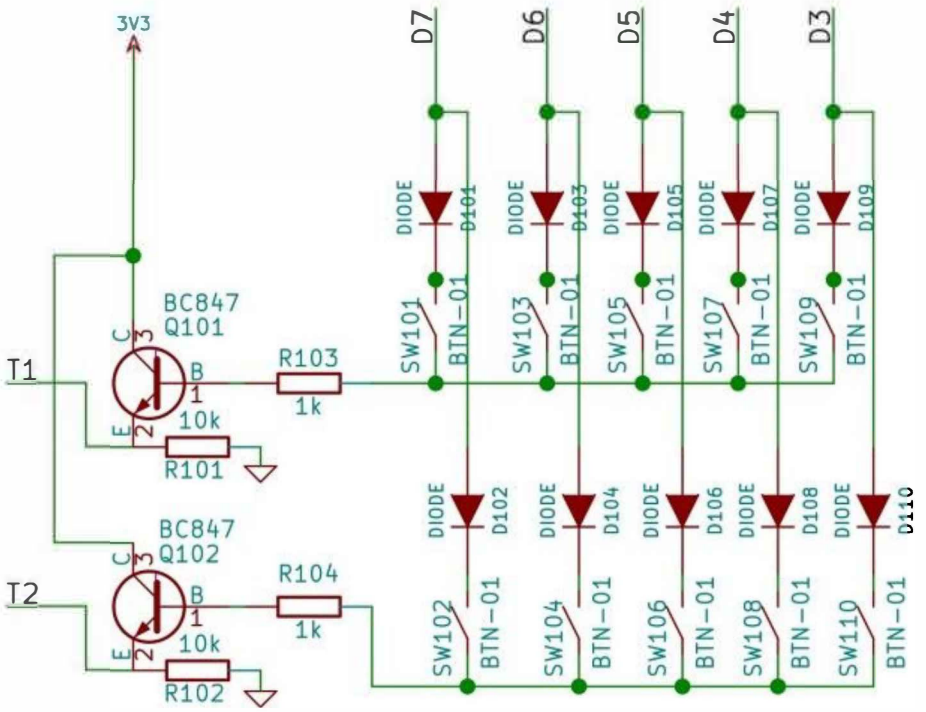


Figura 16.7: Esquemático do teclado matricial

Na saída das linhas existe um circuito transistorizado. Este circuito é um conversor de níveis de tensão. Quando o valor na base (B) do transistor é zero volts, a saída no emissor (E) se mantém em zero volts. No entanto, se a entrada for uma tensão entre 1 e 5 volts, a saída é dada exclusivamente pela tensão de alimentação do transistor, no caso do circuito 3,3 volts.

Este circuito permite que as placas de controle utilizadas sejam tanto de 5 ou 3,3 volts, sem perigo de queimar alguma entrada dos microcontroladores.

O procedimento para realizar a leitura deste tipo de teclado é conhecido como varredura. O circuito matricial é composto de entradas e saídas. No caso da placa base, as cinco colunas

são operadas como terminais de saída e apenas as duas linhas são configuradas como terminais de entrada.

O processo pode ser descrito como:

1.Desligar todas as colunas;

2.Ligar apenas a coluna X;

3.Aguardar um tempo para estabilização dos sinais;

4.Realizar leitura nos terminais de entrada;

5.Passar para a próxima coluna $X = X + 1$.

O acionamento das colunas é feito através do barramento do conversor serial-paralelo. Este barramento é compartilhado com outros periféricos, devendo-se tomar cuidado com seu uso. O código 16.5 apresenta o procedimento de leitura por varredura.

Código 16.5: Leitura do teclado matricial por varredura

```
 1 #include "so.h"
 2
 3 void main(void){
 4     unsigned char coluna;
 5     unsigned char chave[5][2];
 6     soInit();
 7     pinMode(INPUT,12);
 8     pinMode(INPUT,13);
 9     for(;;){
10         for(coluna = 0; coluna < 5; coluna++){
11             //ligar apenas a coluna indicada
12             soWrite(1<<coluna);
13             //teste da 1a linha
14             if (digitalRead(12)){
15                 chave[coluna][0] = 1;
16             } else {
17                 chave[coluna][0] = 0;
18             }
19             //teste da 2a linha
20             if (digitalRead(13)){
21                 chave[coluna][1] = 1;
22             } else {
23                 chave[coluna][1] = 0;
24             }
25         }
26     }
27 }
```

O código 16.5 trata cada tecla como uma posição da matriz **chave**. Como existem apenas 10 chaves, uma solução mais interessante é utilizar apenas uma variável do tipo **int**, reduzindo o gasto de memória.

É importante notar que o código 16.5 não apresenta debounce em software para as teclas.

16.4 Criação da biblioteca

O código 16.7 apresenta um exemplo de código para criar uma biblioteca para um teclado de 10 teclas com leitura matricial. O header pode ser visto no código 16.6.

Código 16.6: keypad.h

```
1 #ifndef TECLADO_H
2 #define TECLADO_H
3     unsigned int kpRead(void);
4     void kpDebounce(void);
5     void kpInit(void);
6 #endif
```

Código 16.7: keypad.c

```c
#include "keypad.h"
#include "so.h"
#include "io.h"

static unsigned int keys;

//vetor com o "nome" dos botões
//U -> up, L -> left, D -> doum, R -> right
//S -> start, s -> select
//a ordem é referente a posição dos botões
static const char charKey[] = {'U','L','D','R','S','s','Y','B','A','X'};

unsigned int kpRead(void) {
    return keys;
}
char kpReadKey(void){
    int i;
    for(i=0;i<10;i++){
        if (bitTst(keys,0)){
            return charKey[i];
        }
    }
    //nenhuma tecla pressionada
    return 0;
}
```

```
26 void kpDebounce(void) {
27     int i;
28     static unsigned char tempo;
29     static unsigned int newRead;
30     static unsigned int oldRead;
31     newRead = 0;
32     for(i = 0; i<5; i++){
33       soWrite(1<<(i+3));
34       if(digitalRead(13)){
35         bitSet(newRead,i);
36       }
37       if(digitalRead(12)){
38         bitSet(newRead,(i+5));
39       }
40     }
41     if (oldRead == newRead) {
42         tempo--;
43     } else {
44         tempo = 4;
45         oldRead = newRead;
46     }
47     if (tempo == 0) {
48         keys = oldRead;
49     }
50 }
51 void kpInit(void) {
52   soInit();
53   pinMode(12, INPUT);
54   pinMode(13, INPUT);
55 }
```

16.5 Detecção de eventos

É muito comum utilizarmos os teclados ou qualquer tipo de entrada digital como sinalizadores de eventos. Em geral, queremos fazer um código que responda a estes eventos: "quando X for pressionado, realizar a tarefa Y". O código 16.8 faz uso da biblioteca do teclado para aumentar um contador quando o botão **0** for pressionado.

Este código funciona do seguinte modo: quando o usuário pressionar o botão zero e o debounce estiver terminado, a função **kpRead()** retornará um valor cujo bit **0** está ligado. Assim, podemos identificar se o botão foi pressionado.

O problema com esse código é que devemos levar em conta a velocidade de execução do processador e o tempo dos eventos reais. Um pressionamento de botão, mesmo rápido, pode levar algumas centenas de milissegundos, tempo em que o loop principal terá sido executado várias vezes. Para o exemplo, isto significa que, a cada pressionamento, a variável **cont** será incrementada diversas vezes, quando a ideia original era que fosse incrementada apenas uma vez a cada pressionamento.

Código 16.8: Leitura do teclado matricial por varredura

```
 1 #include "keypad.h"
 2
 3 void main(void){
 4     int cont=0;
 5     kpInit();
 6     for(;;){
 7         kpDebounce();
 8         if (bitTst(kpRead(),0)){
 9             //executa a atividade
10             cont++;
11         }
12     }
13 }
```

O que estamos querendo fazer neste caso é somente entrar no **if** quando houver mudança na variável. Este evento é conhecido em eletrônica como borda de subida: é o momento onde o sinal que estava desligado passa ao estado ligado.

Para detectar este tipo de evento, precisamos saber o estado atual da chave, bem como o estado anterior. Como o loop principal é cíclico, podemos armazenar a informação do estado da chave para futuras comparações.

Quando o loop reiniciar, comparamos o valor armazenado na rodada anterior com o valor atual. Se eles estiverem diferentes, quer dizer que entre a execução anterior e a atual o valor se alterou. Neste momento, podemos verificar qual foi essa alteração e executar as ações programadas. O código 16.9 apresenta essa função com o uso da biblioteca do teclado.

Código 16.9: Leitura do teclado matricial por varredura

```
 1 #include "keypad.h"
 2 #include "io.h"
 3 #include "ssd.h"
 4
 5 void main(void){
 6     int cont=0;
 7     int lastValue=0;
 8     int actualValue=0;
 9     int time;
10     initSystem();
11     kpInit();
12     ssdInit();
13     for(;;){
14         kpDebounce();
15         actualValue = kpRead();
16         //houve algum evento?
17         if (actualValue != lastValue){
18             //processa os eventos
19             if (bitTst(actualValue,0)){
20                 //executa atividade
21                 cont++;
```

```
22                  if (cont>0xf){
23                      cont = 0;
24                  }
25                  ssdDigit(0,cont);
26              }
27              //armazena a mudança para a próxima rodada
28              lastValue = actualValue;
29          }
30          ssdUpdate();
31          //tempo pra evitar flicker
32          for(t=0;t<100;t++);
33      }
34  }
```

16.6 Aplicações

Além dos botões, existem diversos sensores que fazem uso de entradas digitais dos microcontroladores para enviar sinais e informações.

Reed Switch

As reed switches (figura 16.8) são chaves que podem ser "pressionadas" por um campo magnético. São bastante utilizadas como sensores de presença, sem necessidade de contato físico.

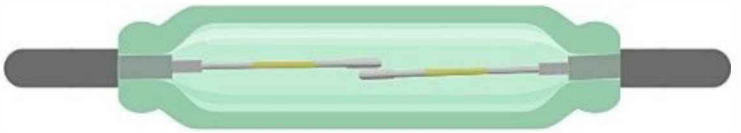


Figura 16.8: Reed switch
Fonte: Imagem produzida com Fritzing/Inkscape

O seu funcionamento é exatamente igual ao de um botão, necessitando de um circuito de pull-up ou pull-down, bem como um sistema de debounce.

Para gerar o campo magnético, é comum se utilizar de pequenos imãs de neodímio. Uma aplicação bastante comum para este sistema é o monitoramento de rotação.

O maior problema em monitorar a rotação de qualquer sistema mecânico é que não é simples inserir dispositivos eletrônicos na parte física que se movimenta, principalmente para enviar energia ou receber os sinais. Com as chaves do tipo reed, podemos simplesmente fixar

um imã permanente no elemento móvel e monitorar com a chave reed, que estará colocada em alguma parte fixa próxima à trajetoria do imã. Este sistema é bastante utilizado para medir velocidade/rotação em bicicletas.

A medição de velocidade é feita verificando-se quanto tempo se passou entre duas ativações consecutivas da chave.

Encoder

Os encoders (figura 16.9) são componentes que possuem um elemento rotativo, denominado cursor, que pode ser movimentado em ambas as direções. Através de circuitos internos dedicados, é possível identificar a posição ou o sentido de rotação do eixo.

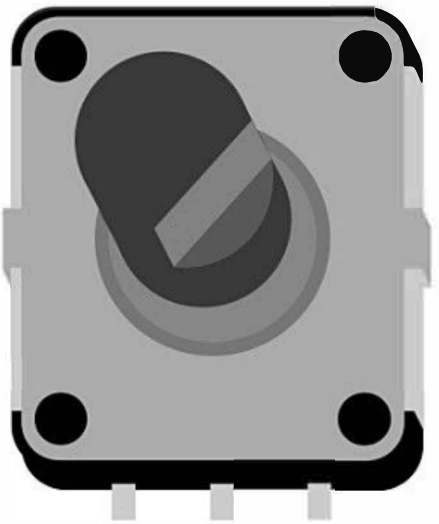


Figura 16.9: Encoder
Fonte: Imagem produzida com Fritzing/Inkscape

A capacidade de identificar posição ou sentido depende do tipo de encoder: absoluto ou relativo.

Os encoders absolutos possuem um sistema interno de contatos em um disco que permite saber em que posição o encoder se encontra. Este disco pode estar codificado em binário ou em código gray.

Cada faixa do disco é lida por um sistema de contato e apresentada no terminal do encoder. No exemplo dado com três faixas, é possivel dividir o encoder em 8 posições diferentes, cada uma representada por uma sequência binária distinta com 3 bits cada.

Por exemplo, se nos terminais do encoder constar o valor **`010`**, podemos afirmar que o cursor está apontado para o número **`2`**. Existem encoders com diversos níveis de precisão, alguns sendo capazes de monitorar variações de centésimos de grau.

Os encoders relativos possuem apenas duas trilhas, não permitindo que possamos identificar a posição real do cursor. No entanto, essas duas trilhas são capazes de indicar o sentido de rotação bem como o ângulo que o cursor andou. Isto é possível por causa do formato do disco, como podemos ver na figura 16.10.

Os quadrados escuros estão deslocados nas duas faixas. Ao se mover ao longo da faixa superior, da esquerda para a direita, percebemos que toda vez que acontece uma transição do

quadrado escuro para o claro na faixa de cima, na faixa de baixo o cursor está em cima de um quadrado escuro.

No sentido contrário, da direita para a esquerda, toda vez que há uma transição de um quadrado escuro para um quadrado claro na faixa de cima, na faixa de baixo o cursor está em cima de um quadrado claro.

Podemos utilizar então a faixa de cima para indicar quantas transições aconteceram, e a faixa de baixo para fornecer a informação se a transição foi em sentido horário ou anti-horário.

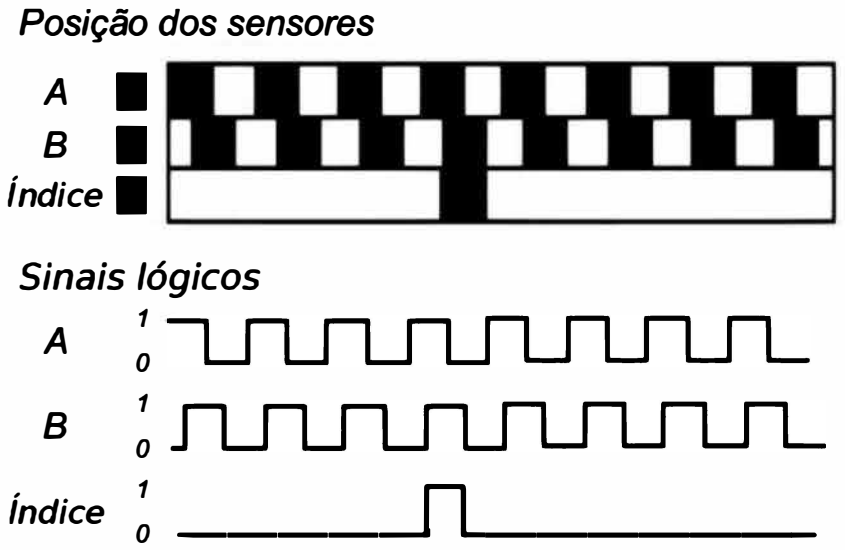


Figura 16.10: Disco de leitura de um encoder relativo

16.7 Exercícios

Ex. 16.1 — Crie um programa que leia e realize o debounce dos valores lidos na porta B e apresente estes valores na porta D. Não use nenhuma biblioteca pronta. Os bits de 0 (zero) a 3 possuem problema com bounce na ordem de 5ms e os bits de 4 a 7 possuem bouncing na ordem de 10ms. Não é permitido usar o tempo de 10ms de debounce para todos os bits.

Ex. 16.2 — Utilizando as bibliotecas "ssd.h" e "keypad.h", crie um programa que faça a leitura das teclas e apresente qual está pressionada utilizando o display de 7 segmentos.

Ex. 16.3 — Altere a biblioteca "keypad.h" apresentada na apostila para que ela possa ler 64 teclas. Estas teclas estão em formato matricial 8 x 8. Os 8 terminais de entrada se encontram nos terminais digitais **`D0`** à **`D7`**. Os 8 terminais de saída são acionados pelo conversor serial-paralelo através da biblioteca "so.h". Altere também a função **`kpRead()`** para que ela receba um parâmetro. Este parâmetro indica qual tecla desejamos verificar e a função retorna 0 (zero) se a tecla estiver desligada, e 1 (um) se estiver pressionada.

Ex. 16.4 — Um sistema embarcado possui as 9 chaves ligadas conforme a figura abaixo (adotar a porta C como saída e porta B como entrada). As chaves não apresentam problemas de oscilação (bouncing). Monte duas funções: 1) "void VarreduraChaves(void)": a função fará a varredura das chaves e salvará o estado de cada chave numa variável "unsigned int Teclas" dentro da biblioteca. 2) "unsigned char LeTecla(void)": a função deve retornar o número da chave pressionada ou **`0`** (zero) se nenhuma estiver apertada. Usar como referência a variável "Teclas".

Ex. 16.5 — Na porta D estão ligadas 8 teclas, uma em cada terminal do microcontrolador. Quando soltas, o valor lido é **`0x00`**. Quando todas são pressionadas, o valor da porta D é **`0xFF`**. Faça uma função que realize o debounce das teclas e retorne qual está pressionada: 1 para a tecla no terminal 0 (zero), 2 para a tecla no terminal 1, e assim até a última tecla. A rotina de debounce deve testar dez vezes se a saída está estável antes de retornar a chave pressionada.

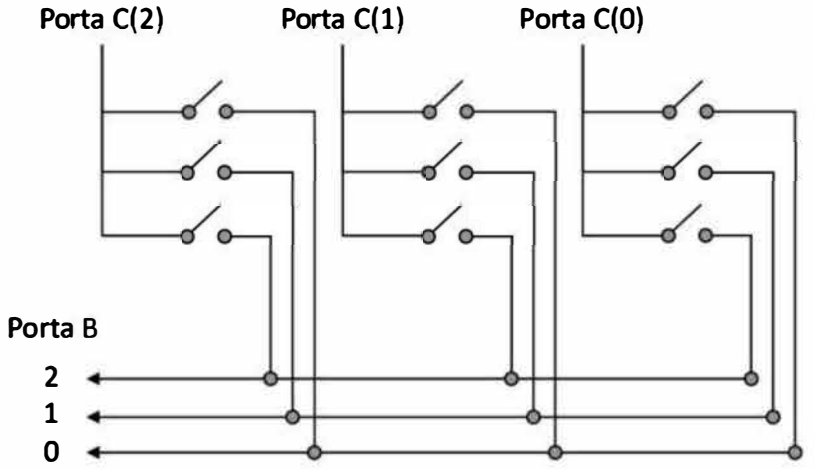


Figura 16.11: Teclado matricial 3x3

Ex. 16.6 — Crie um programa cíclico que realize a leitura de 3 chaves nos terminais **`D0`**, **`D1`** e **`D2`**. Em seguida, este programa deverá acender uma quantidade de leds correspondente ao somatório dos números das chaves. Ex: Se as chaves 1 e 3 estiverem pressionadas, 4 leds serão acesos. Se as chaves 1, 2 e 3 estiverem pressionadas, 6 leds serão acesos. Se nenhuma das chaves estiver pressionada, todos os leds devem ser apagados. As chaves podem ser lidas através da função **`digitalRead()`** e os leds se encontram ligados no conversor serial-paralelo. Utilize a biblioteca "so.h" para acionar os leds.

CAPÍTULO

17

Display LCD

17.1 Circuito de conexão

17.2 Comunicação com o display

- Comandos
- Posicionando os caracteres no LCD

17.3 Criação da biblioteca

17.4 Desenhar símbolos personalizados

17.5 Criando um console com displays de LCD

17.6 Exercícios

> "*O que me deixou orgulhoso foi que usei poucas peças para construir um computador que poderia realmente exibir palavras numa tela e digitar palavras num teclado e rodar uma linguagem de programação que poderia executar jogos. E eu fiz tudo isso sozinho.*"
>
> Steve Wozniak

O display de LCD é um dispositivo de saída que faz uso de cristais líquidos que tem a capacidade de polarizar a luz. Estes cristais permitem ou impedem a passagem da luz, dependendo da excitação elétrica a que são submetidos. Os displays podem ser fabricados em vários formatos diferentes. O mais comum é criar um conjunto de pixels em formato matricial que facilita a construção de várias formas.

Sistemas mais simples optam por formatar o cristal de modo similar aos displays de 7 segmentos, reduzindo a complexidade para sua utilização. Outros modelos criam símbolos/desenhos para simplificar ainda mais a transmissão da mensagem, como pequenos ícones descritivos. Devido à variedade, ao custo e à versatilidade, estes dispositivos são bastante utilizados em sistemas embarcados.

Os modelos mais comuns são os displays alfanuméricos, onde os caracteres são divididos em linhas e colunas. Quase todos os displays que se enquadram nesta definição possuem controladores internos compatíveis com o HD44780. A figura 17.1 apresenta um modelo deste display que possui 16 colunas e duas linhas, isto é, 16x2.

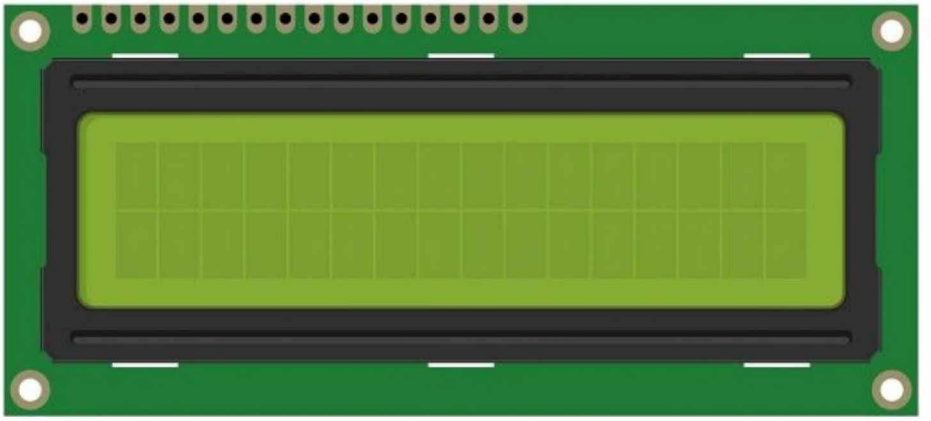


Figura 17.1: Display Alfanumérico LCD 2x16
Fonte: Imagem produzida com Fritzing/Inkscape

Existem versões compatíveis com o controlador HD44780 que possuem de 1 a 4 linhas e 8 a 40 colunas. Para cada formato de display, pode haver algumas mudanças nos protocolos de comunicação, principalmente em relação às posições de memória para armazenamento dos caracteres enviados.

Os displays compatíveis com o HD44780 codificam os caracteres no padrão ASCII. Neste padrão, os valores de 32 a 127 representam os caracteres do alfabeto latino, alguns sinais gráficos de pontuação e os algarismos arábicos. O padrão ASCII reserva os valores de 0 a 32 para caracteres de controle, que não são processados pelo LCD. Já o espaço de valores entre 128 e 255 podem armazenar diversos símbolos diferentes, mas depende do fabricante e da versão do display.

O padrão HD44780 reserva as posições de memória iniciais, de 0 a 8, para armazenar caracteres ou símbolos que podem ser customizados e definidos pelo programador.

A figura 17.2 apresenta numa tabela os caracteres disponíveis nos displays de LCD HD44780 com memória ROM do tipo A00. Como pode ser visto na figura, todos os caracteres ASCII estão presentes. No lugar dos caracteres acentuados são disponibilizados um conjunto de caracteres japoneses (Katakana) e alguns caracteres gregos.

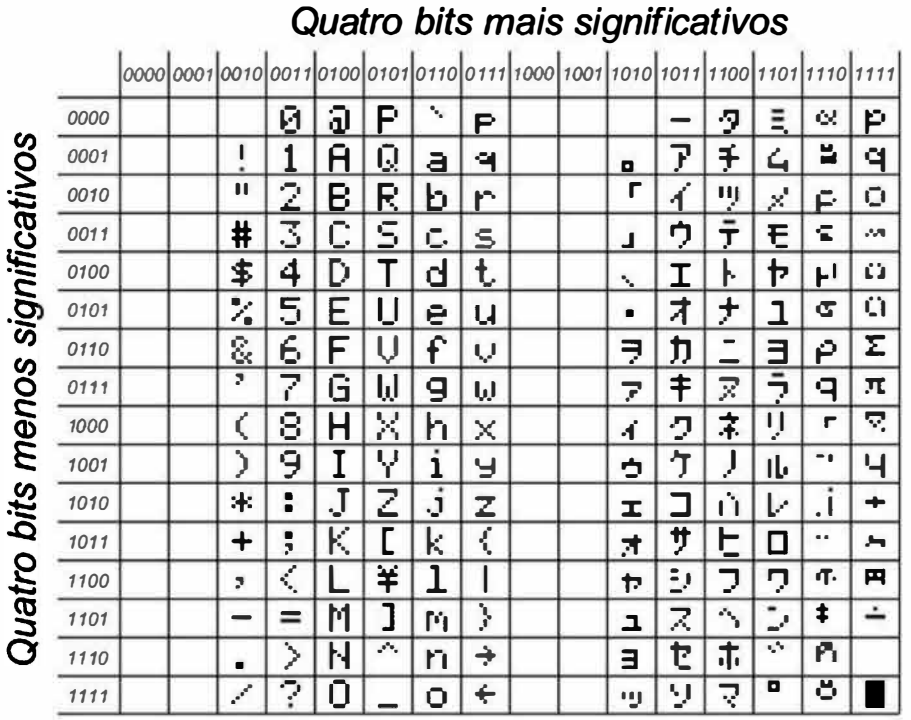



Figura 17.2: Caracteres disponíveis para ROM A00

17.1 Circuito de conexão

Os displays compatíveis com o HD44780 apresentam obrigatoriamente 14 pinos, 2 de alimentação, 1 de controle de contraste, 3 de comando e 8 de dados. Alguns possuem iluminação própria, nestes casos dois pinos extras são disponibilizados. Os terminais são numerados e podem ser identificados como:

1. Terra
2. VCC (+5 V)
3. Ajuste de contraste
4. Seleção de registro(RS)
5. Read/Write (RW)
6. Clock, Enable (EN)
7. Bit **0**
8. Bit **1**
9. Bit **2**
10. Bit **3**
11. Bit **4**
12. Bit **5**
13. Bit **6**
14. Bit **7**
15. VCC, Backlight (opcional)
16. GND, Backlight (opcional)

Os displays podem operar com oito ou quatro bits de dados. No modo de operação de oito bits é possível enviar um caractere por vez. No modo de quatro bits é necessário enviar o

caractere em duas partes.

O modo de quatro bits, apesar de mais lento, permite reduzir a quantidade de terminais utilizados, reduzindo o custo e simplificando o projeto da placa. O conjunto de terminais utilizado para enviar os bits de dados é comumente chamado de barramento de dados.

O terminal RW (read or write) indica ao display se estamos enviando ou lendo alguma informação. Na maioria dos projetos não é necessário realizar nenhum tipo de leitura no LCD, fazendo com que os projetistas liguem este terminal diretamente no terra. Nesta situação, devemos apenas garantir que os tempos para o envio dos dados será respeitado.

O terminal RS (register select) é utilizado para indicar ao display se a informação inserida no barramento contém um comando a ser executado com o valor **0**, ou uma informação para ser exibida com o valor **1**.

Por fim, o terminal EN (enable) é responsável por sincronizar o envio da informação, indicando quando o display pode fazer a leitura dos dados ou a execução da informação que está no barramento.

Na placa de desenvolvimento, os terminais de dados estão ligados no barramento de dados disponibilizado pelo conversor 74HC595, partilhados também com o display de 7 segmentos e o teclado. Para que ambos funcionem juntos é necessário multiplexá-los no tempo, tomando cuidado para que as rotinas das bibliotecas não interfiram nos outros dispositivos. Os terminais de controle RS e EN estão ligados aos terminais digitais 7 e 6, conforme o esquemático da figura 17.3:

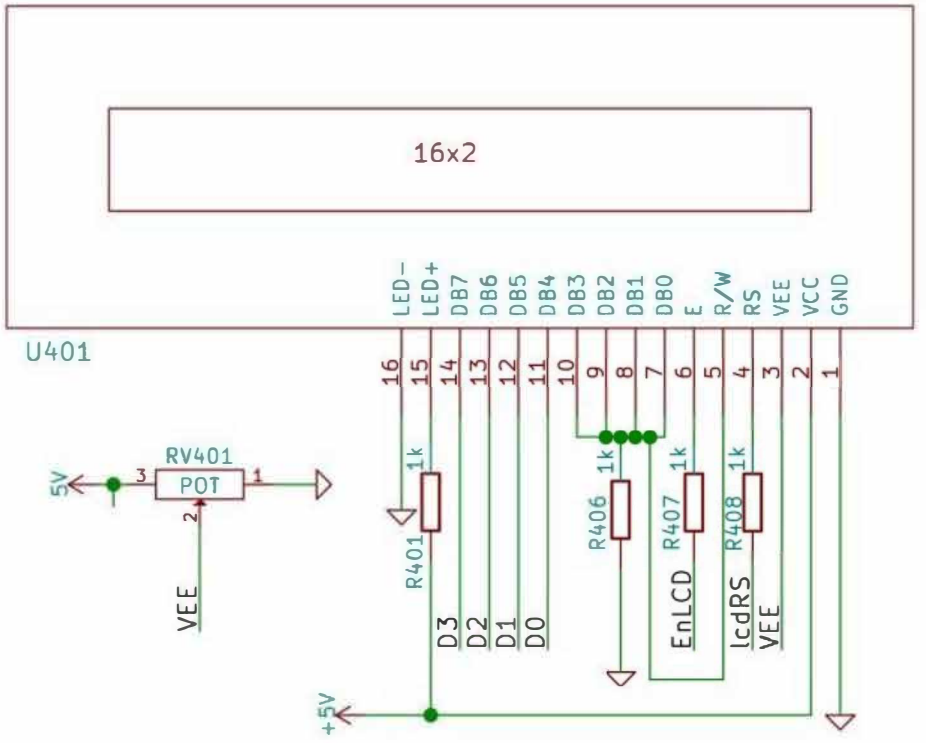


Figura 17.3: Display de LCD alfanumérico

17.2 Comunicação com o display

O processo de comunicação do microcontrolador com o display é controlado pelo pino EN. Quando esse terminal passa do valor **1** para o valor **0**, os dados que estiverem no

barramento são lidos pelo display. Como a comunicação será feita utilizando apenas 4 bits, é necessário enviar primeiro os quatro bits mais significativos e depois os quatro bits menos significativos.

O display pode interpretar os oito bits que foram recebidos de duas formas: como um caractere que deve ser impresso ou como um comando. Para diferenciar entre os dois modos, utilizamos o pino RS (register select). Se **RS=1**, o LCD interpreta os dados como um comando. Se **RS=0**, os dados são interpretados como um caractere codificado em ASCII. Como, por padrão, a linguagem C codifica os caracteres em ASCII, não é necessário realizar nenhuma conversão.

Tanto no envio dos comandos quanto no envio dos caracteres, o display precisa de um tempo para processar o comando atual antes de receber o próximo. É possível perguntar para o display se ele já está pronto para receber o próximo comando. Para isso, é preciso colocar o display em modo de leitura **RW=1**. No circuito apresentado, o pino **RW** permanece todo o tempo em **0**. Por isso é importante respeitar os tempos do display para garantir que não serão enviados comandos ou caracteres rápido demais.

O tempo entre dois comandos ou dois caracteres exigido pelo display pode variar de fabricante para fabricante. Para a maioria dos modelos o tempo de 40µs é suficiente. O comando de limpar o display exige um tempo maior, em torno de 2ms. É possível implementar essas rotinas de tempo com loops, conforme o código 17.1:

Código 17.1: Rotina de Delay para o LCD

```
1 void delayMicro(int a){
2     //utilizar volatile pra evitar otimizações do compilador
3     volatile int i;
4     //1uS é aproximadamente dois ciclos na Freedom
5     for(i=0;i<(a*2);i++);
6 }
7
8 void delayMili(int a){
9     volatile int i;
10    for(i=0;i<a;i++){
11        delayMicro(1000);
12    }
13 }
```

O processo de envio de um caractere é apresentado no código 17.2. Utiliza-se o conversor serial-paralelo para enviar as informações para o barramento de dados. Note que a rotina de tempo só é chamada quando os oito bits já foram enviados.

Código 17.2: Rotina para envio de caractere

```
1 void EscreveChar(char valor){
2     char hi_nibble;
3     char low_nibble;
4
5     hi_nibble = (valor>>4) & 0x0F;
6     low_nibble = (valor) & 0x0F;
7     //configura como envio de caracter
8     digitalWrite(LCD_RS_PIN,HIGH);
9
10    //envia parte alta (mais signficativa)
11    soWrite(hi_nibble);
12    pulseEnableBit();
13
14    //envia parte baixa (menos signficativa)
15    soWrite(low_nibble);
16    pulseEnableBit();
17
18    delayMicro(80);
19 }
```

Para os comandos, a rotina é praticamente a mesma. As duas diferenças estão no valor terminal **RS** que agora recebe valor **1**, e na rotina de delay que deve aguardar de 2ms, para garantir que qualquer tipo de comando seja executado corretamente.

Comandos

O display de LCD possui a capacidade de executar algumas funções, que podem ser acessadas via comandos específicos. Estes comandos permitem que o programador altere o comportamento do LCD, bem como modifique o processo de escrita da informação. A formação dos comandos obedece ao protocolo do HD44780.

Os comandos reconhecidos pelo display são apresentados na Tabela 17.1. As posições com números, zeros ou uns, devem ser obedecidas para a geração do comando. As posições com traços indicam bits que não interferem no comando, podendo ser tanto zero quanto um. As posições que possuem letras indicam opções de escolha na configuração, como ligar/desligar o cursor, modo de deslocamento, nova posição do cursor, entre outras.

Tabela 17.1: Lista de comandos aceitos pelo LCD

Instrução	Barramento de dados (bit)							
	7	6	5	4	3	2	1	0
1) Limpa o display	0	0	0	0	0	0	0	1
2) Reinicia variáveis internas	0	0	0	0	0	0	1	-
3) Configura modo de entrada	0	0	0	0	0	1	**Id**	**S**
4) Configura exibição do display	0	0	0	0	1	**D**	**C**	**B**
5) Configura modo de escrita	0	0	0	1	**Sc**	**Rl**	-	-
6) Configura funcionamento	0	0	1	**Dl**	**N**	**F**	-	-
7) Configura caracteres especiais	0	1	Endereço					
8) Deslocamento do cursor	1	**L**	0	0	Coluna			

Onde:

Id	Configura deslocamento 1 – Incrementa, 0 – Decrementa
S	Configura deslocamento 1 – O display acompanha o deslocamento
D	Estado do display: 1 – ligado, 0 – desligado
C	Estado do cursor: 1 – visível, 0 – invisível
B	O cursor fica piscando: 1 – sim, 0 – não
Sc	Quando recebe uma letra, desloca: 1 – o display, 0 – o cursor
Rl	A próxima letra será escrita: 1 – à direita, 0 – à esquerda
Dl	Quantidade de bits na comunicação: 1 – 8, 0 – 4
N	Quantidade de linhas do display: 1 – 2, 0 – 1
F	Tamanho dos caracteres: 1 – 5x10, 0 – 5x8
L	Nova linha do cursor: 1 – segunda linha, 0 – primeira linha
Coluna	Nibble indicando a posição da nova coluna do cursor
Endereço	Posição do caractere customizado a ser gravados

O display exige um processo de inicialização bem definido para ser utilizado. É necessário aguardar um tempo específico para que o display possa inicializar suas rotinas internas e, em seguida, devemos configurar seu modo de operação.

Como a comunicação será feita em 4 bits, é necessário inicialmente configurar o LCD através de uma rotina de envio de comandos. Este procedimento é apresentado no código 17.3.

Código 17.3: Rotina de inicialização do LCD

```
1 void lcdInit () {
2     pinMode(lcdEnPin,OUTPUT);
3     pinMode(lcdRSPin,OUTPUT);
4     soInit();
5     delayMili(15);          //Não sabemos o estado da comunicação do LCD
6     pushNibble(0x03,LOW);
7     delayMili(5);
8     pushNibble(0x03,LOW);
9     delayMicro(160);
10    pushNibble(0x03,LOW);
11    delayMicro(160);        //Aqui o LCD está, com certeza, em 8bits
12    pushNibble(0x02,LOW);   //Mudando comunicação para 4 bits
13    delayMicro(100);
14    //configurando o display
15    lcdCommand(0x28);       //8bits, 2 linhas, 5x8
16    lcdCommand(0x06);       //Modo incremental
17    lcdCommand(0x0C);       //Display e cursor on, com blink
18    lcdCommand(0x03);       //Reiniciar variáveis internas
19    lcdCommand(0x01);       //Limpar display e vai para posição inicial
20 }
```

É necessário configurar o LCD primeiro para 8 bits, pois, a princípio, não sabemos em que estado de comunicação o LCD se encontra. Internamente, três estados são possíveis:

1.Comunicação em 8 bits

2.Comunicação em 4 bits, aguardando primeiro nibble do comando

3.Comunicação em 4 bits, aguardando segundo nibble do comando

Se o LCD estiver no primeiro estado, ao receber o comando **0x3**, ele permanece no primeiro estado, já que os outros 4 bits do comando serão lidos dos terminais **D0** a **D3**. Independentemente do valor destes terminais, o comando será entendido como "mudar para comunicação em 8 bits". Nesta situação, os outros comandos **0x3** não afetarão o LCD.

Se o LCD estiver no segundo estado, ele receberá o valor 0x3, sendo este valor interpretado como a primeira parte do comando. O segundo valor **0x3** será composto ao primeiro, formando o comando **0x33**, que será entendido como "mudar para comunicação em 8 bits". Neste ponto, o terceiro comando **0x3** não surtirá efeito, pois que o LCD já se encontra na comunicação em 8 bits.

Se o LCD estiver no terceiro estado, o primeiro valor **0x3** será composto com o nibble já armazenado no LCD, que não sabemos de antemão qual é o valor. Portanto, esse primeiro comando não surtirá nenhum efeito. A partir deste momento, o LCD passa para o estado 2. Neste ponto os próximos dois comandos **0x3** farão com que o LCD mude a comunicação para 8 bits.

Ao fim dos três comandos **0x3**, temos certeza que o LCD está na comunicação de 8 bits. A partir deste momento, podemos enviar o comando **0x2** tendo certeza que o LCD mudará para a comunicação em 4 bits.

Depois da comunicação em 4 bits se estabelecer, se dá inicio a configuração das propriedades do LCD, de acordo com o modelo do LCD e com o projeto do sistema.

Posicionando os caracteres no LCD

Cada vez que um caractere é enviado para o LCD, o cursor se desloca para a próxima posição. No entanto, é possível informar a posição do cursor desejada, antes de escrever o caractere. Isto permite reescrever apenas uma determinada região da tela, mantendo o restante inalterado.

Este comando pode variar dependendo do fabricante. Para o display de 16 colunas e 2 linhas é utilizado o seguinte formato: **0b1x00yyyy**, onde **x** representa a linha de escrita (**0** = primeira linha, **1** = segunda linha), e **yyyy** é um número entre zero e 15 representando a coluna de escrita. Deste modo, a função do código 17.4 permite a escolha da linha e da coluna para a escrita.

Código 17.4: Escolha da linha e coluna para escrita

```
1 void lcdPosition(char line, char col){
2     if (line == 0){
3         sendCommand(0x80+col);
4     }
5     if (line == 1){
6         sendCommand(0xC0+col);
7     }
8 }
```

17.3 Criação da biblioteca

Para facilitar o controle do display, as funções apresentadas foram reunidas em uma biblioteca.

O header desta biblioteca é apresentada no código 17.5. Podemos notar que as funções de delay não foram colocadas no header. Essa funções são utilizadas apenas nas rotinas internas, não devendo ser disponibilizadas para outras bibliotecas, principalmente por não serem rotinas de tempo precisas.

Código 17.5: lcd.h

```
1 #ifndef LCD
2 #define LCD
3   void lcdCommand(char value);
4   void lcdChar(char value);
5   void lcdString(char msg[]);
6   void lcdNumber(int value);
7   void lcdPosition(int line, int col);
8   void lcdInit(void);
9 #endif
```

As funções para enviar um caractere e para enviar um comando são muito similares. A única diferença entre elas é o estado do bit **RS**: **0** para comando e **1** para caractere.

Além dessas funções, um conjunto de rotinas foi desenvolvido para simplificar o uso da biblioteca de LCD.

O envio de um texto através de uma string é feito pela **lcdString()**. Essa função imprime todos os caracteres do vetor até encontrar o fim da string, indicado pelo caractere **'\0'**.

A impressão de números obedece uma estrutura similar, com a diferença que os algarismos têm que ser separados. Para isso, é utilizada a operação resto de divisão por dez: **%10**. O nome dessa função é **lcdNumber()**.

Por fim, para facilitar o posicionamento do cursor no LCD, foi criada a função **lcdPosition()**, que, baseada na linha e na coluna requisitadas, envia o comando correto para o LCD.

Código 17.6: lcd.c

```
1 #include "so.h"
2 #include "io.h"
3 #include "lcd.h"
4
5 void delayMicro(int a) {
6     volatile int i;
7     for (i = 0; i < (a * 2); i++)
8         ;
9 }
```

```
9  }
10 void delayMili(int a) {
11     volatile int i;
12     for (i = 0; i < a; i++) {
13         delayMicro(1000);
14     }
15 }
16 //Gera um clock no enable
17 void pulseEnablePin() {
18     digitalWrite(LCD_EN_PIN, HIGH);
19     delayMicro(5);
20     digitalWrite(LCD_EN_PIN, LOW);
21     delayMicro(5);
22 }
23 //Envia 4 bits e gera um clock no enable
24 void pushNibble(char value, int rs) {
25     soWrite(value);
26     digitalWrite(LCD_RS_PIN, rs);
27     pulseEnablePin();
28 }
29 //Envia 8 bits em dois pacotes de 4
30 void pushByte(char value, int rs) {
31     soWrite(value >> 4);
32     digitalWrite(LCD_RS_PIN, rs);
33     pulseEnablePin();
34
35     soWrite(value & 0x0F);
36     digitalWrite(LCD_RS_PIN, rs);
37     pulseEnablePin();
38 }
```

```
39 void lcdCommand(char value) {
40     pushByte(value, LOW);
41     delayMili(2);
42 }
43 void lcdPosition(int line, int col) {
44     if (line == 0) {
45         lcdCommand(0x80 + (col % 16));
46     }
47     if (line == 1) {
48         lcdCommand(0xC0 + (col % 16));
49     }
50 }
51 void lcdChar(char value) {
52     pushByte(value, HIGH);
53     delayMicro(80);
54 }
55 //Imprime um texto (vetor de char)
56 void lcdString(char msg[]) {
57     int i = 0;
58     while (msg[i] != 0) {
59         lcdChar(msg[i]);
60         i++;
61     }
62 }
63 void lcdNumber(int value) {
64     int i = 10000; //Máximo 99.999
65     while (i > 0) {
66         lcdChar((value / i) % 10 + 48);
67         i /= 10;
68     }
69 }
70 // Rotina de incialização
71 void lcdInit() {
72     pinMode(LCD_EN_PIN, OUTPUT);
73     pinMode(LCD_RS_PIN, OUTPUT);
74     soInit();
75     delayMili(15);
76     // Comunicação começa em estado incerto
77     pushNibble(0x03, LOW);
78     delayMili(5);
79     pushNibble(0x03, LOW);
```

```
80      delayMicro(160);
81      pushNibble(0x03, LOW);
82      delayMicro(160);
83      // Mudando comunicação para 4 bits
84      pushNibble(0x02, LOW);
85      delayMili(10);
86      // Configura o display
87      lcdCommand(0x28);          //8bits, 2 linhas, fonte: 5x8
88      lcdCommand(0x08 + 0x04);   //display on
89      lcdCommand(0x01);          //limpar display, voltar p/ posição 0
90  }
```

17.4 Desenhar símbolos personalizados

A maioria dos LCDs permite a criação de caracteres personalizados. Para os displays compatíveis com a controladora HD44780 é possível armazenar 8 caracteres customizados diferentes, a partir do endereço 0x40, conforme a figura 17.4:

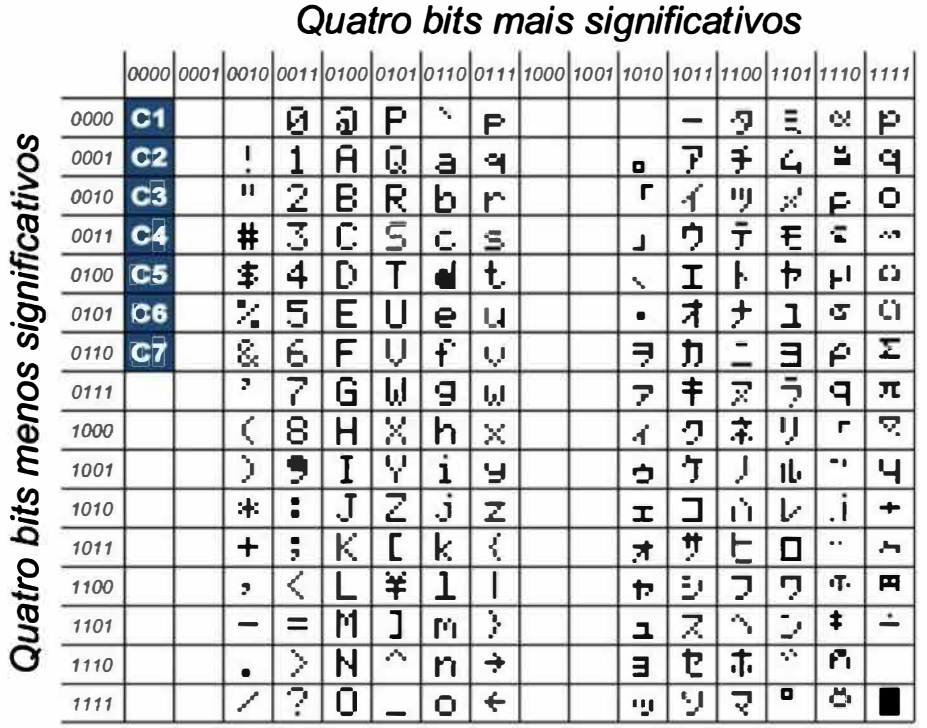



Figura 17.4: Posição dos símbolos personalizados

Cada caractere personalizado é formado pela matriz binária de 8 linhas e 5 colunas (8*5), como exemplificado na figura 17.5.

O uso deste recurso de criação de caracteres próprios é extremamente simples. No código 17.7 é demonstrada a montagem de um novo caractere através do uso da matriz binária de

oito elementos, com 5 bits cada.

Código 17.7: Criando um caractere customizado

```
1 //Cada linha é representada por um caracter
2 char sino[8] =  {0x04, 0x0E, 0x0E, 0x0E, 0x0E, 0x1F, 0x00, 0x04};
3 //Configura para a primeira posição de memória
4 lcdCommand(0x40);
5 //Envia cada uma das linhas em ordem
6 for(i=0; i<8; i++){
7    lcdChar(sino[i]);
8 }
```

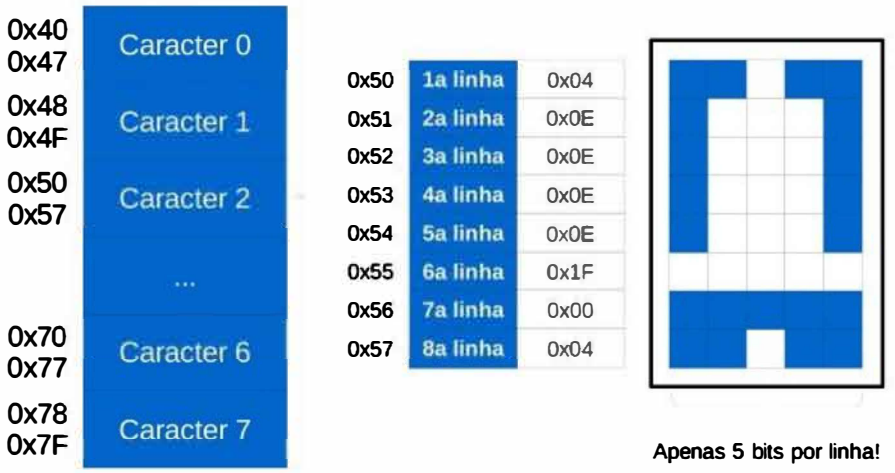
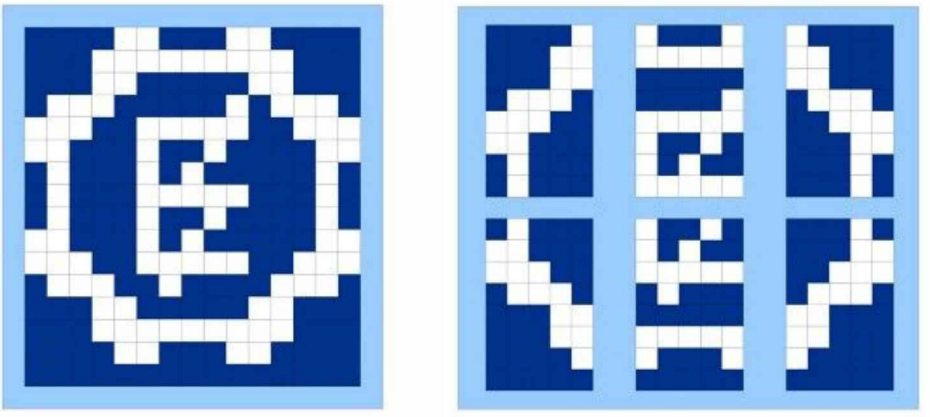


Figura 17.5: Criação de um símbolo personalizado

É possível criar um desenho/imagem de até 20*16 pixels (4*2 caracteres). A imagem será binária, composta de apenas pixels brancos e pretos. Existe uma separação física do LCD entre os caracteres, que pode prejudicar de certa maneira a imagem em questão. Os passos para geração de uma imagem e exibição no LCD são:

1.Criar uma imagem binária com o desenho desejado, figura 17 .6 (a);
2.Segmentar a imagem em retângulos de 8 x5 , figura 17 .6 (b);
3.Transcrever cada linha em binário/hexadecimal, figura 17 .7 ;
4.Gerar o código fonte, código 17 .8 .



(a) Imagem binária (b) Imagem segmentada

Figura 17.6: Criar uma imagem binária com o desenho desejado

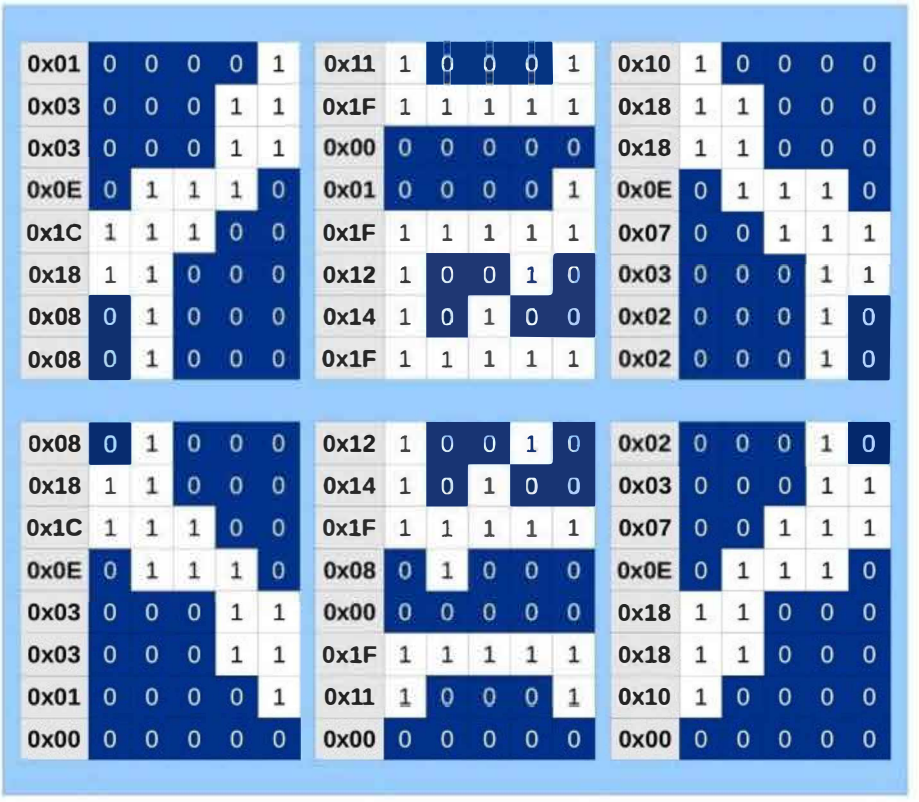


Figura 17.7: Transcrição de cada linha da imagem em binário/hexadecimal Código 17.8: Código fonte da imagem

Código 17.8: Código fonte da imagem

```
1 //Cada linha é representada por um caracter
2 char logo[48] = {
3     0x01, 0x03, 0x03, 0x0E, 0x1C, 0x18, 0x08, 0x08, //0,0
4     0x11, 0x1F, 0x00, 0x01, 0x1F, 0x12, 0x14, 0x1F, //0,1
5     0x10, 0x18, 0x18, 0x0E, 0x07, 0x03, 0x02, 0x02, //0,2
6     0x08, 0x18, 0x1C, 0x0E, 0x03, 0x03, 0x01, 0x00, //1,0
7     0x12, 0x14, 0x1F, 0x08, 0x00, 0x1F, 0x11, 0x00, //1,1
8     0x02, 0x03, 0x07, 0x0E, 0x18, 0x18, 0x10, 0x00  //1,2
9 };
10
11 //Configura para armazenar na primeira posição de memória disponível
12 lcdCommand(0x40);
13
14 //Envia cada um dos bytes dos caracteres em ordem
15 for(i=0; i<48; i++){
16     lcdChar(logo[i]);
17 }
```

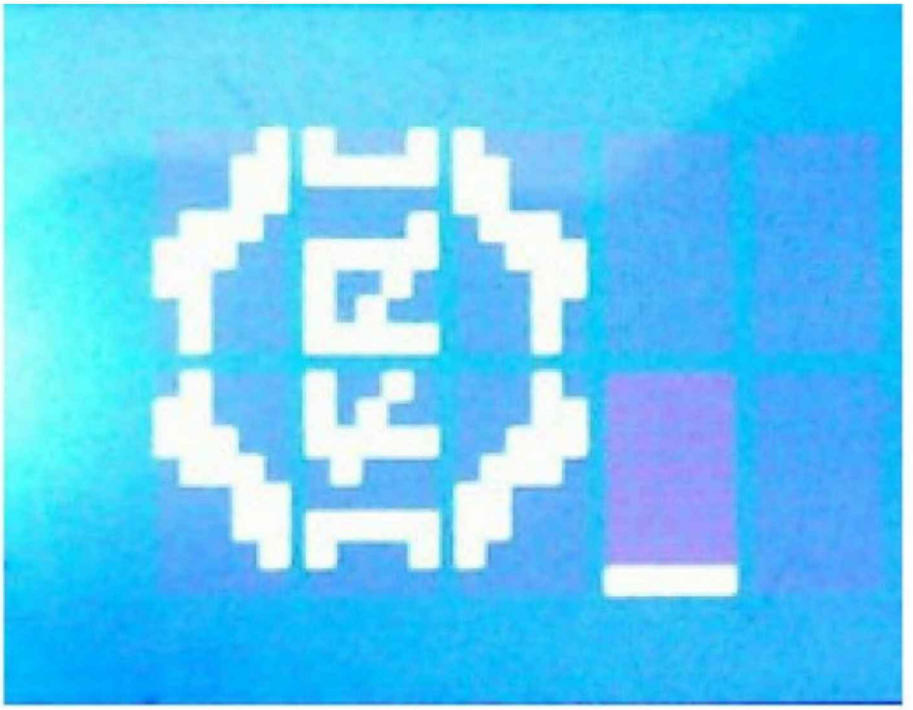


Figura 17.8: Resultado da imagem gerada

17.5 Criando um console com displays de LCD

Para fazer com que um display de LCD funcione de modo similar a um console é importante armazenar internamente os dados enviados para o LCD de modo a recuperá-los quando preciso, visto que não é possível perguntar ao LCD o que está escrito nele.

Criando uma matriz com uma quantidade de linhas maior do que a suportada pelo LCD é possível recuperar as linhas escritas, montando um histórico dos textos exibidos pelo LCD. Essa matriz funcionará como um buffer, permitindo que a biblioteca controle qual posição será exibida. A figura 17.9 mostra a formação desse buffer e sua relação com o que é exibido. Na figura, o buffer possui 4 linhas enquanto o LCD consegue exibir 2 ao mesmo tempo.

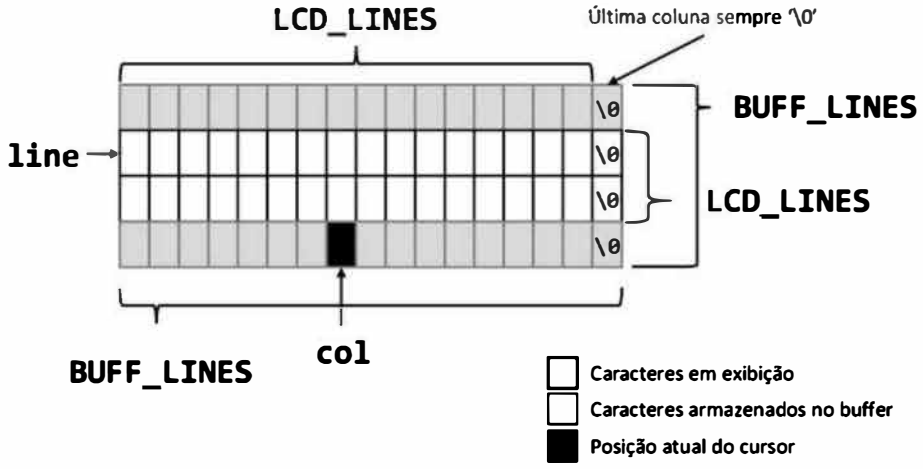


Figura 17.9: Buffer de caracteres e variáveis de controle

Esta estrutura foi implementada na biblioteca console. No header, apresentado no código 1 7.9, são exibidas as 4 funções básicas do console.

Código 17.9: console.h

```
1 #ifndef CONSOLE_H
2 #define CONSOLE_H
3     void consoleInit();
4     void consolePrint(char* vet);
5     void consoleUpdate(void);
6     void consoleMoveLine(int relativeMove);
7 #endif
```

A função central do código é a **consolePrint()**, que, recebendo um vetor de caracteres, os formata para serem inseridos no buffer. Esta função apenas insere os caracteres no buffer mas não faz nenhuma impressão.

A função **consoleUpdate()** é responsável por fazer a exibição da seção do buffer indicada pela variável **line**. Modificando essa variável é possível exibir diferentes linhas do histórico.

Para modificar as linhas que serão exibidas, o programador pode fazer uso da função **consoleMoveLine()**, que recebe um inteiro indicando se a tela deve ser movimentada para cima (valor positivo) ou para baixo (valor negativo).

Por fim, a função **consoleInit()** faz a inicialização do LCD bem como de todas as variáveis internas.

O código 17.10 apresenta a implementação completa das funções bem como do buffer de caracteres. Na biblioteca, existe uma função que não é disponibilizada no header: a **newLine()**. Essa função serve para movimentar o histórico uma linha para cima, do mesmo modo quando a tecla "enter" é pressionada num editor de texto. Para isso, cada caractere deve ser levado para a linha de cima, na mesma posição, e a última linha deve ser limpa.

Código 17.10: console.c

```
 1 #include "lcd.h"
 2
 3 #define LCD_COLS 16
 4 #define LCD_LINES 2
 5 #define BUFF_LINES 4
 6
 7 //mais um pra guardar o '\0' no fim de cada linha
 8 #define BUFF_COLS (LCD_COLS+1)
 9
10 char buffer[BUFF_LINES][BUFF_COLS];
11 int line;
12 int col;
13
14 void consoleInit() {
15     int i, j;
```

```
16      lcdInit();
17      for (i = 0; i < BUFF_LINES; i++) {
18          for (j = 0; j < BUFF_COLS; j++) {
19              buffer[i][j] = '\0';
20          }
21      }
22      line = (BUFF_LINES - LCD_LINES);
23      col = 0;
24  }
25
26  void newLine(void) {
27      int l;
28      int c ;
29      //sobe cada linha do buffer em uma posição
30      for (l = 1; l < BUFF_LINES; l++) {
31          for (c = 0; c < BUFF_COLS; c++) {
32              buffer[l - 1][c] = buffer[l][c];
33          }
34      }
35      //limpa a última linha
36      for (l = 0; l < BUFF_COLS; l++) {
37          buffer[BUFF_LINES - 1][l] = '\0';
38      }
39  }
40
```

```
41 void consoleUpdate(void) {
42     int i, j;
43     lcdCommand(0x01);
44     for (i = 0; i < LCD_LINES; i++) {
45         lcdPosition(i,0);
46         for (j = 0; j < BUFF_COLS; j++) {
47             if ((buffer[line + i][j] == '\0')) {
48                 break;
49             } else {
50                 lcdChar(buffer[line + i][j]);
51             }
52         }
53     }
54 }
55 
56 void consolePrint(char* vet) {
57     char i, j;
58     int currentPos = 0;
59     //enquanto a string não terminar continua processando
60     while (vet[currentPos] != '\0') {
61         //se chegou uma nova linha termina a atual e passa pra próxima
62         if (vet[currentPos] == '\n') {
63             buffer[BUFF_LINES - 1][col] = '\0';
64             col = 0;
65             newLine();
66         } else {
67             //se chegou uma letra normal armazena no buffer
```

```
68              buffer[BUFF_LINES - 1][col] = vet[currentPos];
69              col++;
70              //se encheu a linha, passa para próxima
71              //(-1 por causa do espaço para o '\0')
72              if (col >= (BUFF_COLS - 1)) {
73                  buffer[BUFF_LINES - 1][BUFF_COLS - 1] = '\0';
74                  col = 0;
75                  newLine();
76              }
77          }
78          currentPos++;
79      }
80      //armazena '\0' na última posição
81      buffer[BUFF_LINES - 1][col] = vet[currentPos];
82  }
83
84  //muda a posição da linha que deve ser exibida
85  void consoleMoveLine(int relativeMove) {
86      if (relativeMove < 0) {
87          if (line > 0) {
88              line--;
89          }
90      }
91      if (relativeMove > 0) {
92          if (line < BUFF_LINES - LCD_LINES) {
93              line++;
94          }
95      }
96  }
```

Há ainda algumas melhorias que se pode realizar nesse console, principalmente ligar o cursor para indicar onde está acontecendo a inserção dos dados e aceitar os comandos de delete e backspace.

17.6 Exercícios

Ex. 17.1 — Faça um programa que realize a leitura do teclado e apresente a tecla correspondente no LCD. Lembre-se de efetuar a conversão para código ASCII antes de enviar os dados ao LCD.

Ex. 17.2 — Crie uma biblioteca "lcd8bits" que implemente as mesmas funcionalidades que a biblioteca LCD, mas usando 8 bits de dados, ao invés de 4, na comunicação.

Ex. 17.3 — Crie uma rotina para desenhar os símbolos de seta para esquerda, direita, cima e baixo no display de LCD e armazenar na posição referente aos primeiros 4 caracteres.

Ex. 17.4 — Crie um programa que controle o cursor no LCD. Ele deve fazer a leitura das teclas através da função **`kpReadKey()`**. As teclas de movimentação devem reposicionar o cursor do LCD através da função **`lcdPosition()`**. As teclas **`A`**, **`B`**, **`X`** e **`Y`** inserem os caracteres 'A', 'B', 'C' e 'D' no display na posição atual. As teclas **`S`** e **`s`** limpam o display e retornam o cursor para a posição inicial.

Ex. 17.5 — Construa um relógio/calendário que exiba a hora na primeira linha do LCD e a data na segunda. A rotina de tempo pode ser feita com um loop **`for`**. Leve em conta o ano bissexto. Para saber se o ano atual é bissexto, use o seguinte algoritmo:

```
1 if ((year % 400) == 0){
2     //ano bissexto
3 }else if ((year % 100) == 0){
4     //não é bissexto
5 }else if ((year % 4) == 0){
6     //ano bissexto
7 }else{
8     //não é bissexto
9 }
```

CAPÍTULO

18

Comunicação serial

18.1 I²C
- Soft I²C
- Relógio de tempo real

18.2 SPI

18.3 CAN

18.4 RS232
- RS232 ou UART?

18.5 USB
- Serial sobre USB

18.6 Serial sem fios

18.7 Leitura e processamento de protocolos
- O protocolo NMEA de GPS

18.8 Exercícios

> "*Empresas gastam milhões em firewalls, criptografia e dispositivos de segurança e é dinheiro desperdiçado, porque nada disto resolve o elo mais fraco na cadeia de segurança: as pessoas que usam, administram, operam e cuidam dos sistemas que contêm informações protegidas.*"
>
> Kevin Mitnick

Em geral, a comunicação entre dois dispositivos eletrônicos é realizada de modo serial, isto é, as informações são passadas bit a bit do transmissor para o receptor. Este tipo de comunicação possui algumas vantagens em relação à comunicação paralela, na qual a palavra de 8 bits (byte) é enviada toda de uma vez.

A primeira vantagem é a simplificação do hardware. Como os dados são enviados um a um, a quantidade de fios envolvidos na transmissão é menor.

A segunda vantagem é a maior taxa de transmissão, o que, à primeira vista, pode parecer inconsistente, já que a comunicação paralela envia mais de um bit ao mesmo tempo. Mas, para frequências muito altas, nem sempre o envio das informações são sincronizadas em todos os fios. Existe também o problema do crosstalking, onde o campo elétrico ou magnético gerado por um cabo induz uma tensão no cabo adjacente, atrapalhando a comunicação. Estes

problemas aumentam com a frequência, limitando, assim, a máxima transferência possível pelo barramento paralelo. Foi este o motivo que levou os projetistas de hardware a desenvolverem o protocolo serial SATA, em detrimento do IDE, paralelo, para comunicação entre o HD e a placa-mãe.

Existem diversas alternativas de protocolo de comunicação serial para sistemas embarcados. Nestes sistemas é comum a utilização de protocolos mais simples, principalmente por questões de custo de implementação. Estes protocolos conseguem atender grande parte das demandas em termos de segurança e taxas de transmissão dos componentes eletrônicos envolvidos.

O funcionamento básico de qualquer protocolo de comunicação serial consiste num sistema que consiga enviar os bits de modo sequencial através de um terminal do microcontrolador. A velocidade com que os bits são enviados pode ser configurada. O sinal de velocidade (ou clock), pode ou não ser enviado junto com os dados. Em geral, existe um registro específico para serializar um byte, que faz a transmissão dos dados, e um registro para armazenar os bits que chegam. Estas relações estão expressas na figura 18.1.

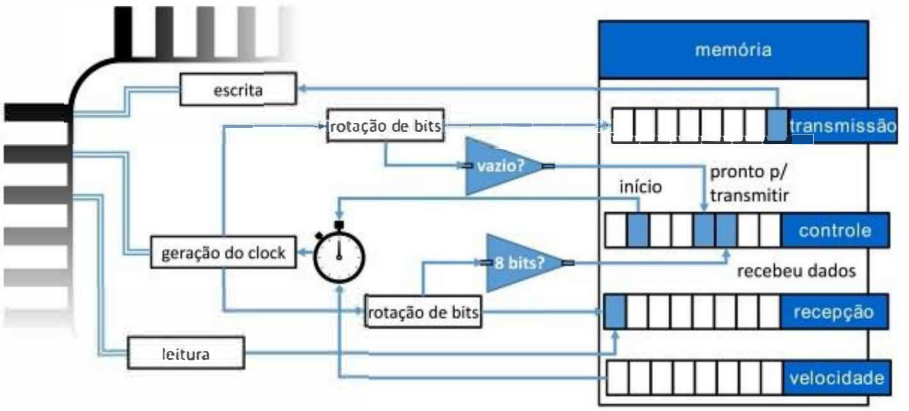


Figura 18.1: Relação entre os terminais de uma comunicação serial e os registros de memória

18.1 I²C

O protocolo I²C foi desenvolvido pela Phillips na década de 1980, para permitir que componentes eletrônicos de uma mesma placa pudessem se comunicar de modo simples e eficiente. O protocolo foi desenvolvido para suportar uma taxa de comunicação de 100kbps (100.000 bits por segundo). Em sua última versão, 4.0 de 2012, permite que os componentes atinjam taxas de até 5Mbps.

- 1982 - Protocolo I²C criado pela Phillips (100 kHz)
- 1992 - Versão 1.0: Adicionada a frequência de 400 kHz (Fast mode) e endereçamento de 10 bits (1024 endereços para dispositivos)
- 1998 - 2 .0 : Frequência de 3 .4 MHz (High-speed mode)
- 2007 - 3 .0 : Frequência de 1 .0 MHz (Fast mode plus)
- 2012 - 4 .0 : Frequência de 5 .0 MHz (Ultra fast mode plus) e criada tabela com identificadores de fabricantes

Este é um protocolo serial síncrono, ou seja, o clock é enviado junto com o sinal, permitindo ao receptor ler os sinais do barramento no momento certo.

Se um dispositivo possui em sua descrição que é compatível com I²C 3.0, isto indica que ele pode se comunicar com velocidades de até 1.0MHz de clock. Isto, no entanto, não impede

que ele trabalhe com velocidades mais baixas.

A especificação do padrão I²C define que o protocolo é do tipo mestre/escravo, permitindo mais de um dispositivo escravo no barramento.

Como existem diversos dispositivos, cada componente recebe um identificador para evitar erros no envio das informações. Deste modo, o mestre pode enviar as mensagens para o dispositivo correto, bem como saber qual é o elemento que está devolvendo a resposta.

Como todos os dispositivos são conectados fisicamente ao mesmo barramento I²C, o desenvolvimento do hardware se torna simplificado, reduzindo inclusive a necessidade de um microcontrolador com muitos terminais. A figura 18.2 mostra um exemplo de um barramento I²C com um mestre e três escravos:

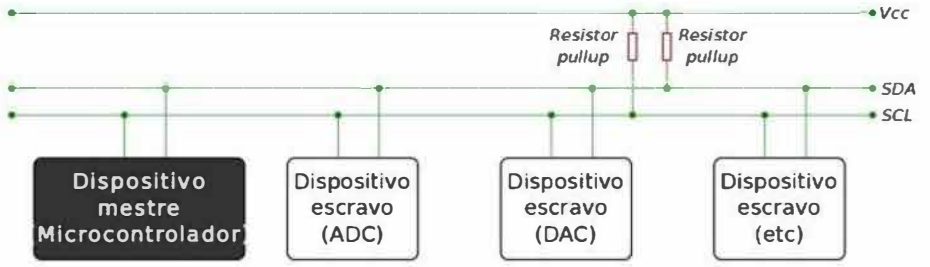


Figura 18.2: Barramento I²C com vários dispositivos

Um ponto importante para garantir que este barramento funcione de maneira adequada é a estrutura eletrônica escolhida para as conexões: o coletor aberto, como mostra a figura 18.3.

A estrutura de coletor aberto permite que mais de um dispositivo se conecte ao barramento, sem que haja perigo de curtos-circuitos. Se um componente estiver enviando um sinal alto, de 5 volts, por exemplo, e o outro está enviando um sinal desligado, de zero volts, a estrutura evita que aconteça um curto entre os sinais.

O envio e a recepção de dados são sempre iniciados pelo mestre, sempre em grupos de 8 bits. Há também alguns "bits" especiais, que marcam o início e o fim da transmissão. Existe ainda uma estrutura de confirmação para permitir que o dispositivo indique que a mensagem chegou corretamente.

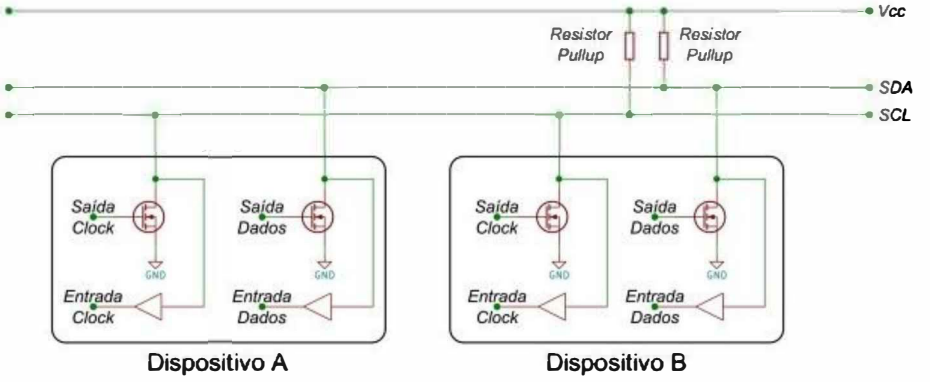


Figura 18.3: Conexão com coletor aberto

O "bit" de início de transmissão ocorre quando o mestre altera o valor do terminal de dados de alto para baixo sem alterar o valor do terminal de clock.

Após o início, o mestre envia o primeiro byte. Este byte é responsável por indicar se o mestre deseja realizar uma leitura ou escrita, bem como identificar qual é o dispositivo que irá

responder ao comando. O primeiro bit enviado é o que indica a leitura ou escrita, os sete restantes apresentam a identificação.

Após o envio do primeiro byte o mestre fica aguardando o sinal de recebimento. Se a operação for uma operação de escrita, o segundo byte é enviado pelo mestre.

Se a operação for de leitura, o byte é enviado pelo escravo e lido pelo mestre.

O envio das informações é bastante parecido com o envio de dados para os displays de LCD. A diferença é que, neste caso, apenas um bit é enviado por vez. O processo pode ser descrito como:

1.SDA e SCL começam em nível alto;
2.SDA é levado para nível baixo como sinal de início;
3.SCL é levado para nível baixo;
4.o primeiro bit (menos significativo) é colocado em SDA;
5.SCL é levado para nível alto, aguarda-se o tempo do sistema e abaixa-se novamente SCL;
6.repete-se o processo do item 3 até o fim da transmissão;
7.SDA é levado para nível alto indicando fim de transmissão.

Esta sequência de atividades, bem como os níveis dos sinais SDA e SCL estão apresentados na figura 18.4, a seguir:

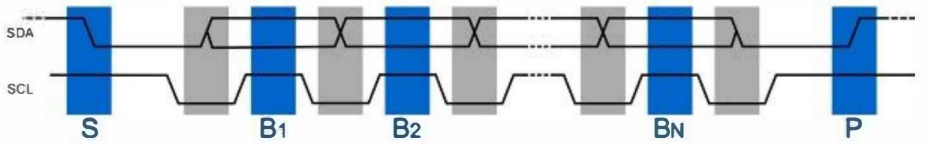


Figura 18.4: Estado dos terminais SDA e SCL no envio de um dado

Todas essas operações são sincronizadas pelo clock do mestre, mesmo quando o byte é enviado pelo escravo. Segundo a norma do protocolo, o valor na linha de dados deve ser sempre válido quando a linha de clock estiver alta.

Diversos microcontroladores possuem este protocolo implementado em hardware. Isto permite que todos os detalhes da comunicação sejam tratados pelo periférico de I²C e o programador fica responsável apenas por definir a operação.

Uma solução muito utilizada é implementar o protocolo inteiramente em software. Para isso, basta ter acesso a dois terminais digitais do microcontrolador.

Soft I²C

A implementação de uma comunicação I²C por software, pode ser feita utilizando apenas dois terminais digitais, sendo que um deles deve ser capaz de ser alterado entre entrada e saída.

Como este barramento opera com uma saída de coletor aberto, é possível forçar o nível baixo na saída, no entanto, o nível alto só pode ser obtido através dos resistores de pull up. Por isso, enviar um sinal zero (**0**) corresponde a colocar o terminal como saída e colocar o valor zero no bit da porta correspondente. Para enviar um sinal um (**1**) o terminal deve ser configurado como uma entrada, fazendo com que os resistores consigam levar o valor para um. Deve-se tomar cuidado para não colocar o terminal como saída e fornecer uma tensão alta (5v ou 3,3v), pois isto, além de prejudicar a comunicação, pode trazer problemas físicos aos componentes envolvidos.

Para simplificar esse processo, podemos criar um conjunto de macros para manipular os terminais de dados, **SDA**, e de clock, **SCL**.

```
1 //macros para controlar os terminais
2 #define SDA_OFF()   digitalWrite(SDA_PIN,LOW)
3 #define SDA()       digitalRead(SDA_PIN)
4 #define SDA_IN()    pinMode(SDA_PIN,INPUT)
5 #define SDA_OUT()   pinMode(SDA_PIN,OUTPUT)
6
7 #define SCL_OFF()   digitalWrite(SCL_PIN,LOW)
8 #define SCL()       digitalRead(SCL_PIN)
9 #define SCL_IN()    pinMode(SCL_PIN,INPUT)
10 #define SCL_OUT()   pinMode(SCL_PIN,OUTPUT)
```

Através destas macros podem ser criadas funções para: enviar um valor zero (**clear_SDA()** e **clear_SCL()**), enviar um valor um (**set_SDA()**), bem como retornar o estado dos terminais (**read_SDA()** e **read_SCL()**). Estas funções são apresentadas no códig o 18.1.

É possível, que em algumas situações, o dispositivo escravo não tenha terminado de processar algum evento mesmo com o intervalo de tempo proporcionado pelo mestre. O protocolo prevê que o escravo pode "segurar" o sinal de clock em nível baixo durante o tempo que for necessário. Isto é conhecido como alongamento do clock, ou clock stretching.

Código 18.1: Manipulação de bits para o protocolo I²C

```
1 //configura SCL como entrada e retorna o valor do canal
2 int read_SCL(void){
3     pinMode(SCL_PIN,INPUT);
4     return digitalRead(SCL_PIN);
5 }
6 //Envia um bit zero
7 void clear_SCL(void){
8     pinMode(SCL_PIN,OUTPUT);
9     digitalWrite(SCL_PIN,LOW);
10 }
11 //configura SDA como entrada e retorna o valor do canal
12 int read_SDA(void){
13     pinMode(SDA_PIN,INPUT);
14     return digitalRead(SDA_PIN);
15 }
16 //Envia um bit um
17 void set_SDA(void){
18     //o bit um é feito colocando o terminal como entrada e permitindo que o ↩
           pull-up leve o terminal para 1
19     pinMode(SDA_PIN,INPUT);
20 }
21 //Abaixa o nivel do canal de clock
22 void clear_SDA(void){
23     pinMode(SDA_PIN,OUTPUT);
24     digitalWrite(SDA_PIN,LOW);
25 }
```

Para marcar o início e o fim das mensagens, são utilizadas transições especiais nos barramentos.

Na transmissão de um bit, o valor do canal de dados (SDA) só pode variar enquanto o sinal de clock (SCL) estiver com valor baixo. Deste modo, antes de trocar o valor do SDA, todos os dispositivos escravos devem verificar se SCL está baixo; caso contrário, ninguém altera o valor de SDA.

Para indicar o início de uma mensagem, o mestre muda o valor do SDA de alto para baixo com o valor de SCL alto. Para indicar o fim de uma mensagem, o valor de SDA é levado de um nível baixo para um alto, com SCL ainda alto.

Existe também um bit conhecido como repeated start. Ele acontece quando um bit de start é enviado antes do bit de stop correspondente. Estes sinais são apresentados na figura 18.5.

Como o bit de repeated start é uma variação do start, optou-se por utilizar a mesma estrutura. Para diferenciar entre as duas execuções optou-se por criar uma variável temporária **`started`**. Esta variável indica se é a primeira vez, ou não, que o comando start está sendo enviado. Depois de enviado uma primeira vez, seu valor passa a ser **`1`**. A variável volta ao estado normal, **`0`**, quando é enviado um sinal de stop. As funções que implementam isso são apresentadas no código 18.2.

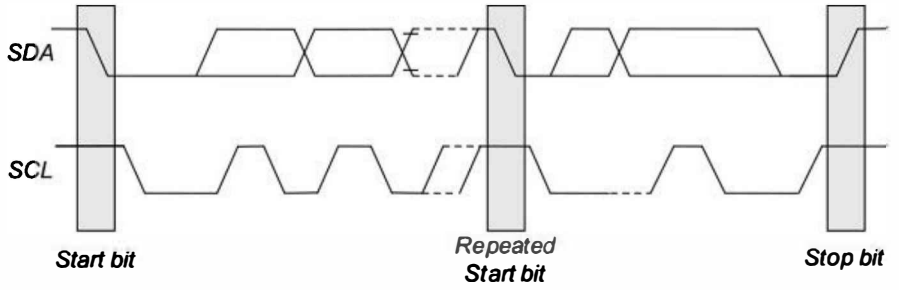


Figura 18.5: Bits de start, repeated start e stop

Código 18.2: Geração dos bits de start e stop

```
 1 int started = 0; // primeira vez
 2 void i2c_start(void) {
 3     //se já estiver iniciado, prepara para reenviar o bit de start
 4     if (started) {
 5         set_SDA();
 6         I2C_delay();
 7         //Aguarda o clock ficar disponível (clock strecching)
 8         while (read_SCL() == 0);
 9         I2C_delay();
10     }
11     // SCL está alto, mudar SDA de 1 para 0
12     clear_SDA();
13     I2C_delay();
14     clear_SCL();
15     started = true;
16 }
17 void i2c_stop(void){
18     // set SDA to 0
19     clear_SDA();
20     I2C_delay();
21     // Aguarda o clock ficar disponível (clock strecching)
22     while (read_SCL() == 0);
23     I2C_delay();
24     // SCL está alto, mudar SDA de 0 para 1
25     set_SDA();
26     I2C_delay();
27     started = false;
28 }
```

O código 18.3 apresenta a implementação dos processos de escrita e de leitura em duas funções, ambas para um único bit. Deve-se lembrar que a cada bit enviado é preciso verificar se o escravo liberou o barramento, através do sinal de clock, considerando assim a possibilidade do clock stretching.

Código 18.3: Envio dos bits via comunicação I²C

```
1 void i2c_write_bit(unsigned char bit) {
2     if (bit) {
3         read_SDA();
4     } else {
5         clear_SDA();
6     }
7
8     I2C_delay();
9     while (read_SCL() == 0) ; // Clock stretching
10
11    if (bit && read_SDA() == 0) {
12        arbitration_lost();
13    }
14    I2C_delay();
15
16    clear_SCL();
17    I2C_delay();
18 }
19
20 unsigned char i2c_read_bit(void) {
21    unsigned char bit;
22    read_SDA();
23
24    I2C_delay();
25    while (read_SCL() == 0) / // Clock stretching
26
27    bit = read_SDA();
28    I2C_delay();
29    clear_SCL();
30
31    I2C_delay();
32    return bit;
33 }
```

O protocolo para a transmissão de um byte completo é bastante simples. Sua principal atividade é a serialização dos dados. Este processo pode ser implementado com os seguintes passos:

1.Colocar o valor do bit a ser enviado na linha de DADOS (SDA);
2.Gerar um pulso de clock na linha de CLOCK (SCL);
3.Havendo mais bits para enviar, voltar ao passo 1 .

Além disso, pode ser necessário enviar um start e/ou um stop bit em cada mensagem.

Para simplificar a utilização do protocolo pelo programador, a função de enviar byte **`i2cWriteByte()`** recebe como parâmetros, além do próprio byte, dois valores: **`send_stop`** e **`send_start`**. Eles servem para indicar para a função se os bits especiais de start e end devem ou não ser enviados.

A função de ler um byte **`i2cReadByte()`** também recebe como parâmetros dois valores: **`nack`** e **`send_stop`**. O nack indica se, ao fim da leitura, o mestre deve enviar um sinal de not acknowledge ou não. Em geral, esse sinal indica se o mestre vai requisitar mais dados do escravo ou não. As duas funções podem ser encontradas no código 18.4.

Código 18.4: Envio de bytes via comunicação I²C

```
1 unsigned char i2cWriteByte(unsigned char send_start, unsigned char send_stop, ↩
      unsigned char byte) {
2     unsigned char bit;
3     unsigned char nack;
4     if (send_start) {
5         i2c_start();
6     }
7     for (bit = 0; bit < 8; bit++) {
8         i2c_write_bit((byte & 0x80) != 0);
9         byte <<= 1;
10    }
11    nack = i2c_read_bit();
12    if (send_stop) {
13        i2c_stop();
14    }
15    return nack;
16 }
17
18 unsigned char i2cReadByte(unsigned char nack, unsigned char send stop) {
19    unsigned char byte = 0;
20    unsigned bit;
21    for (bit = 0; bit < 8; bit++) {
22        byte = (byte << 1) | i2c_read_bit();
23    }
24    i2c_write_bit(nack);
25    if (send_stop) {
26        i2c_stop();
27    }
28    return byte;
29 }
```

Por fim, a última função necessária para o funcionamento da biblioteca é a de inicialização do sistema. Como a biblioteca não faz nenhuma comunicação em si, ela apenas presta serviço para o programador, não existe a necessidade de inicializar nenhum protocolo, apenas os dois terminais de comunicação. Estes terminais começam, de praxe, como entradas.

```
1 void i2cInit(void) {
2     SDA_IN();
3     SCL_IN();
4 }
```

A maioria das funções foram implementadas para simplificar o processo de programação da biblioteca. O programador precisa apenas de três: a de inicialização, a de escrita de bytes e de leitura de bytes. Portanto, o header da biblioteca fica como no código 18.1.

```
1 #ifndef I2C_H
2 #define I2C_H
3     void i2cInit(void);
4     unsigned char i2cWriteByte(unsigned char send_start, unsigned char send_stop, ↩
          unsigned char byte);
5     unsigned char i2cReadByte(unsigned char nack, unsigned char send_stop);
6 #endif
```

Relógio de tempo real

O dispositivo DS1307 (figura 18.6) é um relógio de tempo real (RTC) com um protocolo baseado no I²C. Um RTC é um dispositivo especializado em manter a contagem de tempo correta para longos períodos de tempo.

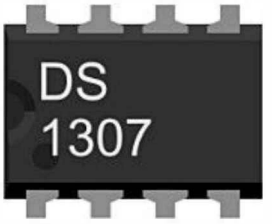


Figura 18.6: Relógio de tempo real DS1307
Fonte: Imagem produzida com Fritzing/Inkscape

Este é um dispositivo externo, portanto é necessária a conexão deste por meio de 2 caminhos de comunicação (SCL e SDA) com o microcontrolador. Além disso, o RTC exige um cristal próprio para que possa realizar a contagem de tempo de modo independente do

microcontrolador. Por se tratar de uma comunicação do tipo I²C, dois resistores de pullup são adicionados nos terminais de comunicação.

Por fim, para que o DS1307 possa continuar a contagem de tempo mesmo quando a placa for desligada, pode ser adicionada uma bateria ao seu circuito.

O esquemático completo da ligação do DS1307 ao micro é dado pela figura 18.7.

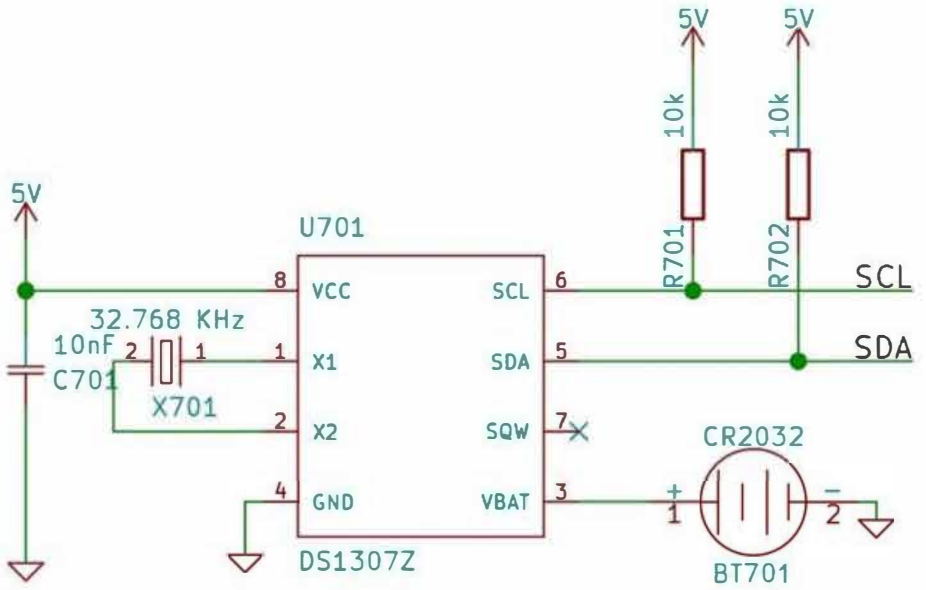


Figura 18.7: Conexões do DS1307

A comunicação do dispositivo segue o padrão I²C, onde o primeiro byte identifica o endereço do dispositivo, com 7 bits, e o último bit explicita se o comando é de leitura ou escrita.

Se for um processo de escrita, o segundo byte identifica o endereço do registro interno do RTC que será sobrescrito. O terceiro byte indica, por fim, o valor que será escrito no endereço enviado.

Se o procedimento for de leitura, o segundo byte será enviado pelo escravo para o mestre. O seu conteúdo depende do endereço que estava ativo no RTC.

Estes procedimentos são apresentados na figura 18.8:

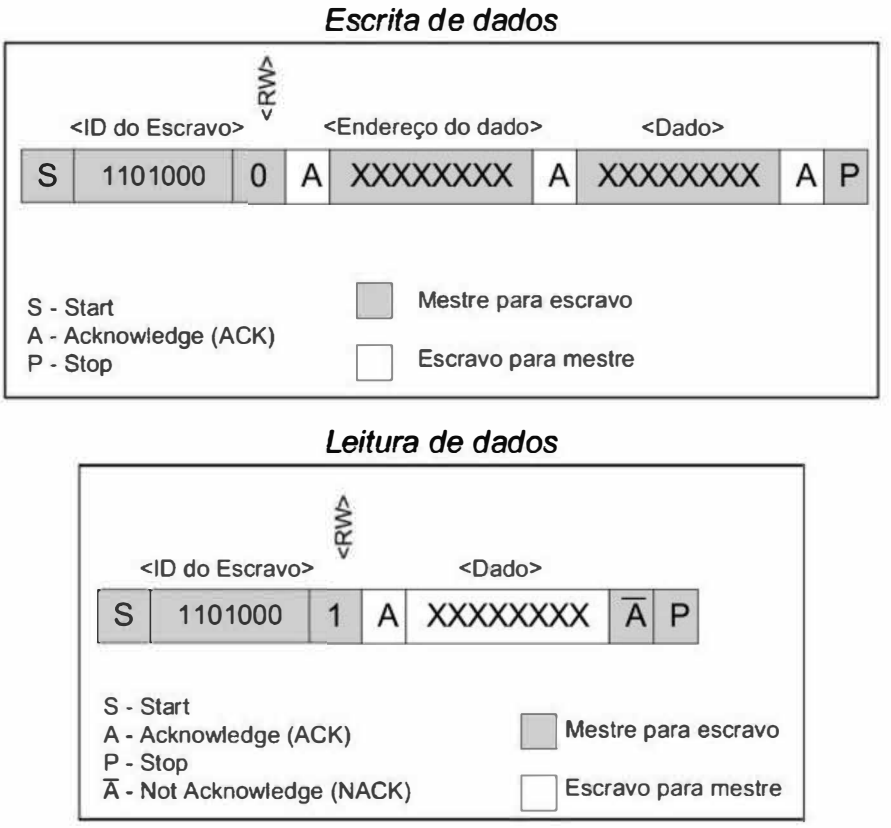


Figura 18.8: Escrita/leitura de dados para o DS1307

Para conseguir ler de um determinado endereço do RTC é necessário primeiro enviar um comando para escrever o endereço, seguido de um comando de leitura de um valor, conforme a figura 18.9:

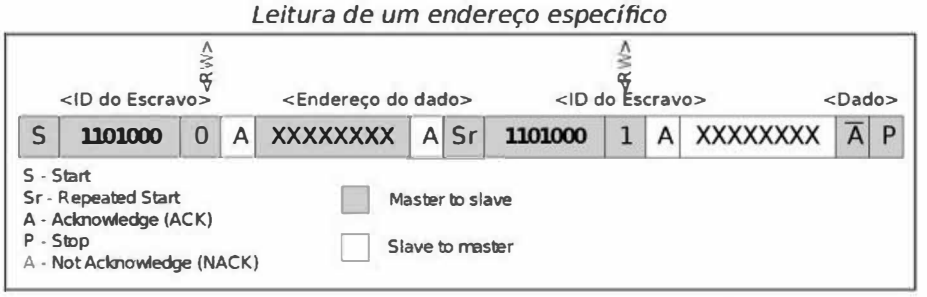


Figura 18.9: Leitura de um endereço específico do DS1307

O RTC possui 8 registros internos, com endereços de **`0x00`** a **`0x07`**, que fazem a contagem dos segundos, minutos, horas, dias, meses, anos e dia da semana. Por fim existe ainda um registro de configuração. Os valores destes endereços estão codificados em BCD compactado, para facilitar a passagem dos valores. Estes registros são apresentados na figura 18.10. Existe ainda uma região de 56 bytes de RAM, que estão disponíveis do endereço **`0x08`** a **`0x3F`**.

Endereço	BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0	Função	Faixa
00h	CH	Dezena de segundos			Segundos				Seconds	00–59
01h	0	Dezenas de minutos			Minutos				Minutes	00–59
02h	0	12(1) ou 24(0)	10 Horas PM/ AM	10 Horas	Horas				Horas	1–12 +AM/PM 00–23
03h	0	0	0	0	0	Dia/Semana			Dia/Semana	01–07
04h	0	0	Dezenas de Dias		Dias				Dias	01–31
05h	0	0	0	10 Meses	Meses				Meses	01–12
06h	Dezenas de Anos				Anos				Anos	00–99
07h	OUT	0	0	SQWE	0	0	RS1	RS0	Controle	—
08h–3Fh	Byte de dados (RAM)								RAM 56 x 8	00h–FFh

***0** = Sempre é lido como zero*
***CH** = Clock Halt (habilita contagem)*
***SQWE** = Habilita saida do clock*
***RS** = Configura velocidade do clock de saída*

Figura 18.10: Registros internos do DS1307

O código 18.5 apresenta a implementação das funções da biblioteca de acesso ao DS1307. Esta biblioteca utiliza as funções da biblioteca I²C apresentadas anteriormente para implementar a comunicação completa com o dispositivo.

Código 18.5: Código da biblioteca DS1307

```
 1 #include "i2c.h"
 2 #include "ds1307.h"
 3
 4 //endereço do dispositivo, deslocado por causa do bit de RW
 5 #define DS1307_CTRL_ID (0x68<<1)
 6 #define I2C_WRITE 0
 7 #define I2C_READ  1
 8
 9 int dec2bcd(int value) {
10     return ((value / 10 * 16) + (value % 10));
11 }
12 int bcd2dec(int value) {
13     return ((value / 16 * 10) + (value % 16));
14 }
15 void dsInit(void) {
16     i2cInit();
17 }
```

```
18 void dsStartClock(void) {
19     int seconds;
20     seconds = dsReadData(SEC);
21     dsWriteData(0x7f & seconds,SEC);
22     return;
23 }
24 void dsWriteData(unsigned char value, int address) {
25     i2cWriteByte(1,0, DS1307_CTRL_ID|I2C_WRITE);
26     i2cWriteByte(0,0,address);
27     i2cWriteByte(0,1,value);
28 }
29 int dsReadData(int address) {
30     int result;
31     i2cWriteByte(1,0,DS1307_CTRL_ID | I2C_WRITE);
32     i2cWriteByte(0,0,address);
33     i2cWriteByte(1,0, DS1307_CTRL_ID | I2C_READ);
34     result = i2cReadByte(1,1 );
35     return result;
36 }
```

Pelo código, podemos ver duas funções não muito relacionadas com o DS1307: a **dec2bcd()** e a **bcd2dec()**. Estas funções fazem a conversão de números codificados em decimal para números codificados em BCD, e vice-versa.

O código18.6 apresenta o header com os protótipos das funções bem como três conjuntos de macros. O primeiro define os endereços de cada um dos registros dos dados do DS1307. O segundo e o terceiro grupos utilizam macros para criar funções de acesso aos registros, de modo mais simples para o programador.

Código 18.6: Header da biblioteca DS1307

```
#ifndef DS1307RTC_h
#define DS1307RTC_h

//definição dos endereços
#define SEC     0
#define MIN     1
#define HOUR    2
#define WEEKDAY 3
#define DAY     4
#define MONTH   5
#define YEAR    6

    //funções do DS1307
    void dsInit(void);
    void dsStartClock(void);
    void dsStopClock(void);
    int dec2bcd(int value);
    int bcd2dec(int value);
    void dsWriteData(unsigned char value, int address);
    int dsReadData(int address);
```

```
21
22 //funções de leitura/escrita simplificadas
23 #define getSeconds()  (bcd2dec(dsReadData(SEC)& 0x7f))
24 #define getMinutes()  (bcd2dec(dsReadData(MIN)& 0x7f))
25 #define getHours()    (bcd2dec(dsReadData(HOUR)& 0x5f))
26 #define getWeekDay()  (bcd2dec(dsReadData(WEEKDAY)& 0x07))
27 #define getDays()     (bcd2dec(dsReadData(DAY)& 0x5f))
28 #define getMonths()   (bcd2dec(dsReadData(MONTH)& 0x3f))
29 #define getYears()    (bcd2dec(dsReadData(YEAR)& 0xff))
30
31 #define setSeconds(v) (dsWriteData(dec2bcd(v,SEC)))
32 #define setMinutes(v) (dsWriteData(dec2bcd(v,MIN)))
33 #define setHours(v)   (dsWriteData(dec2bcd(v,HOUR)))
34 #define setWeekDay(v) (dsWriteData(dec2bcd(v,WEEKDAY)))
35 #define setDays(v)    (dsWriteData(dec2bcd(v,DAY)))
36 #define setMonths(v)  (dsWriteData(dec2bcd(v,MONTH)))
37 #define setYears(v)   (dsWriteData(dec2bcd(v,YEAR)))
38
39 #endif
```

18.2 SPI

O barramento de interface serial para periférico, em inglês Serial Peripheral Interface (SPI), foi desenvolvido pela Motorola em 2000 e transformou-se em um padrão para dispositivos embarcados. Este barramento é uma interface de comunicação serial síncrona de curta distância, normalmente utilizada em sensores, unidades de armazenamento, interfaces de entrada e saída para usuário etc. Sua principal vantagem, em relação ao I²C, é a maior velocidade de transmissão e o volume de dados flexível, não limitado em apenas palavras de 8 bits. O modo de ligação é apresentado na figura 18.11:

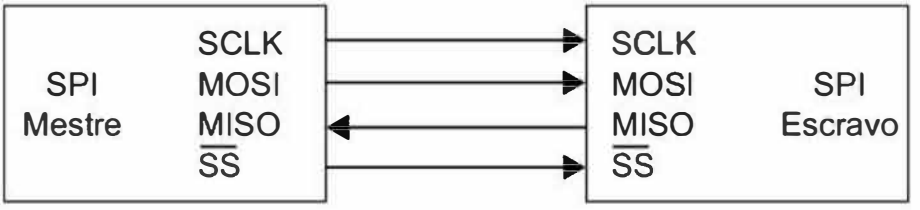


Figura 18.11: Barramento SPI em comunicação ponto a ponto, mestre e escravo.

A versão original do protocolo SPI é bidirecional (full duplex) e utiliza a arquitetura mestre-escravo, similar ao I²C, com apenas um mestre por barramento e vários escravos

independentes. Possui comunicação síncrona graças a uma via de clock no barramento chamada SCK (Serial Clock), que garante ao chip receptor a sequência correta de bits transmitida, sem necessidade de bits de sincronismo ou sinalizadores.

No entanto, as semelhanças desta versão do SPI com o I²C acabam aqui, pois os anais de comunicação de dados deste barramento são separados: a via de transmissão do mestre é chamada de MOSI (Master Output, Slave Input) e a via de recepção do mestre é chamada MISO (Master Input, Slave Output). Além dos canais de comunicação existe um canal que habilita o funcionamento do escravo, chamado SS (Slave Select).

Cada escravo possui, em geral, uma via SS, independente, ligada ao mestre. Isto causa uma certa desvantagem nesta comunicação, pois a cada novo escravo independente é necessário inserir uma nova via SS ao mestre e ao barramento, necessitando um uso bem maior de pinos no dispositivo mestre que o I²C, como mostrado na figura 18.12:

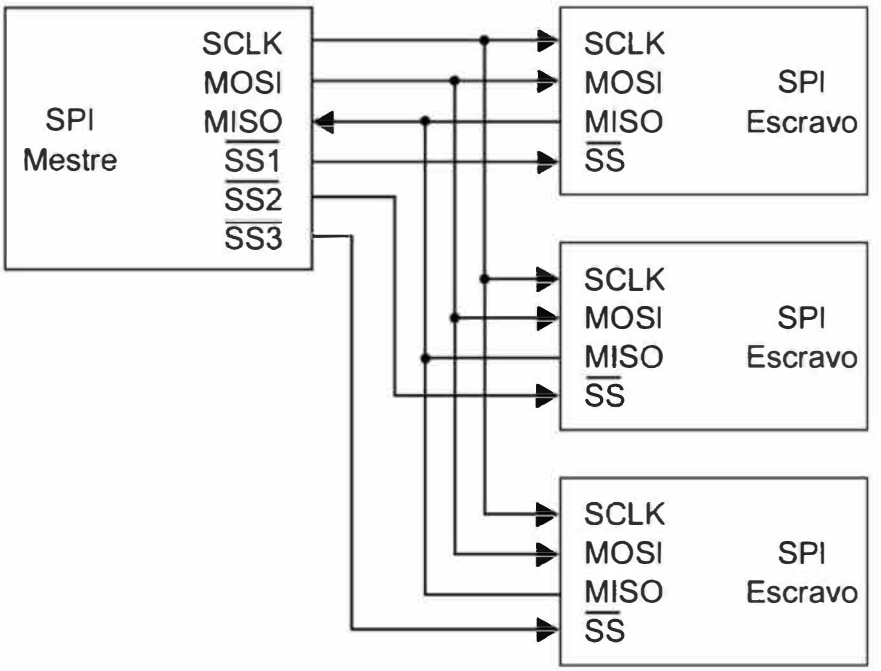


Figura 18.12: Versão original do SPI com três escravos independentes

Existe uma outra versão de SPI presente em alguns chips, chamada Daisy Chain ou comunicação em anel. Nesta estrutura de ligações, não é necessário uma via de seleção (SS) para cada escravo do barramento. Os escravos funcionam em modo cooperativo, onde a informação do mestre trafega por cada escravo até chegar ao destino. Forma-se um anel ou loop de comunicação, com um funcionamento similar a um registrador de deslocamento (shift register) entre os escravos, como pode ser observado na figura 18.13. Isto simplifica o barramento, pois não existirá mais que quatro vias de comunicação entre os chips, barateando o custo do projeto. Mas os escravos deixam de ser independentes nesta versão de SPI e, no caso de queima de um deles, todos os demais perderão a comunicação reduzindo, assim, a confiabilidade do sistema. Os principais sistemas que utilizam esta versão de SPI são o Serial GPIO (SGPIO) e o JTAG.

A operação de transmissão de dados no SPI é inicializada pelo dispositivo mestre configurar uma frequência de clock na via SCK suportada pelos demais chips escravos. O mestre seleciona o escravo que será comunicado inserindo o nível lógico baixo na via SS do

dispositivo desejado, habilitando a comunicação entre eles. Dependendo do tipo de tarefa do escravo, o mestre aguarda um tempo predeterminado em código, antes de iniciar a troca de dados após a mudança da via SS, para garantir que o escravo tenha realizado suas tarefas antes da comunicação. Assim, o mestre envia seu comando pela via MOSI para o escravo, sincronizada pela SCK. Ao receber o comando, o escravo aguarda a sequência de pulsos de clock fornecida pelo mestre para retornar seus dados.

Todo este processo é apresentado na figura 18.14. É importante salientar que o mestre sabe o tamanho da palavra de resposta do escravo para gerar corretamente o número de ciclos corretos com o tamanho da informação de resposta, pois em software é definida a estrutura de comandos e respostas dos escravos com o mestre, própria para cada tipo de chip utilizado.

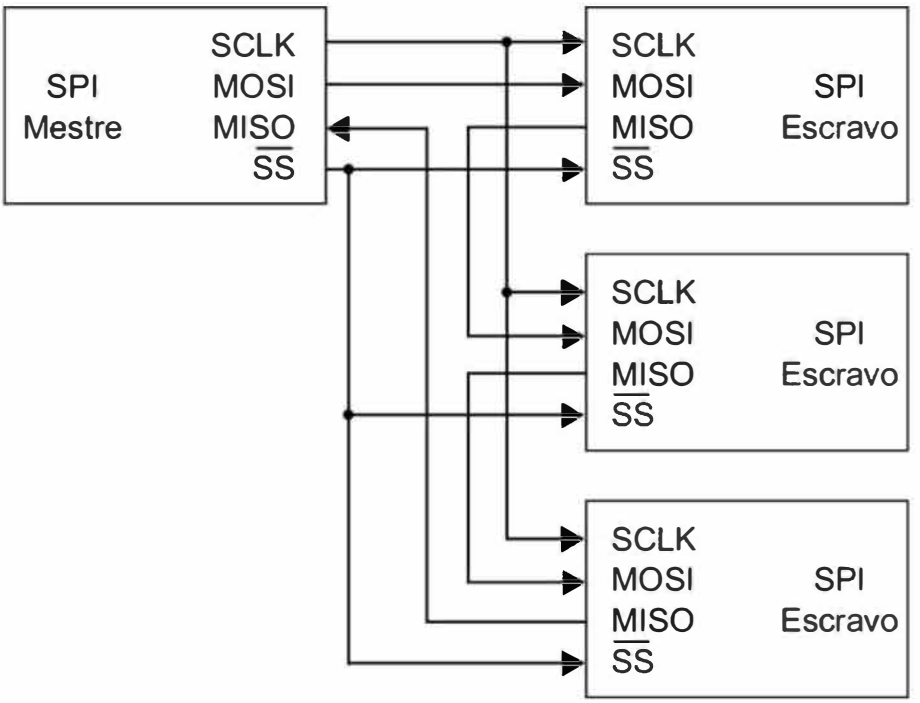


Figura 18.13: Versão em anel do SPI com três escravos cooperativos

O SPI suporta a comunicação bidirecional (full duplex) porque suas vias de transmissão (MOSI) e recepção (MISO) são independentes. Desta forma, em algumas situações é possível transmitir e receber do escravo simultaneamente, como exemplo um sensor enviando a resposta do comando do mestre e, paralelamente, recebendo um novo comando. Mas é necessário verificar no manual do componente se é possível utilizar essa situação.

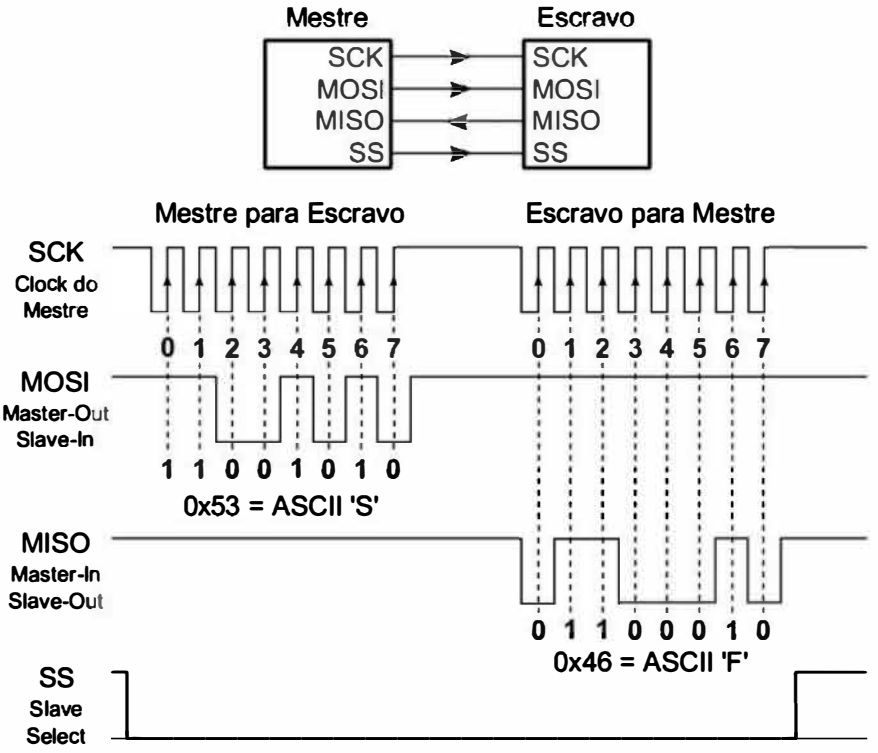


Figura 18.14: Comunicação SPI entre mestre e escravo

18.3 CAN

O protocolo CAN (Controller Area Network) é um protocolo de comunicação serial com suporte eficiente para controle distribuído em tempo real e alto nível de segurança. Seu desenvolvimento iniciou-se em 1983 pela empresa alemã Robert Bosch GmbH para uso em barramentos automotivos, onde não é necessária uma unidade central de gerenciamento. Sendo um protocolo baseado em mensagens, obteve seu primeiro uso em 1988, em um veículo BMW, onde realizava a multiplexação das informações dos múltiplos sensores presentes no automóvel.

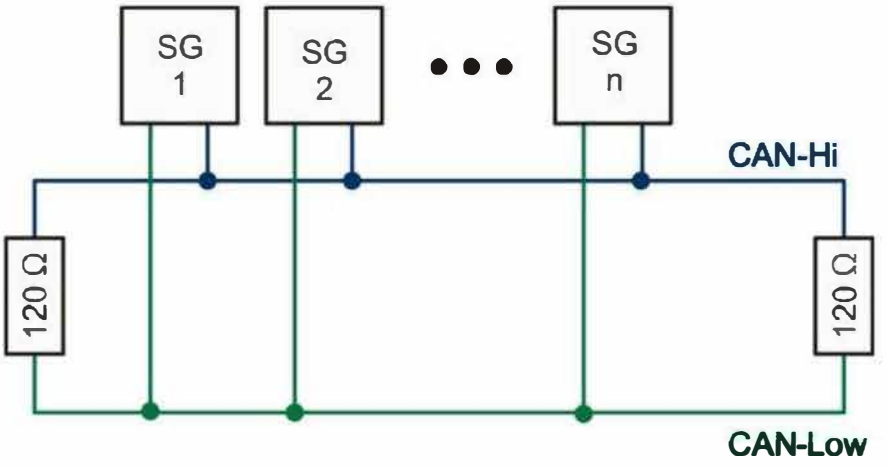


Figura 18.15: Ligação elétrica da camada física do CAN, com os terminadores resistivos

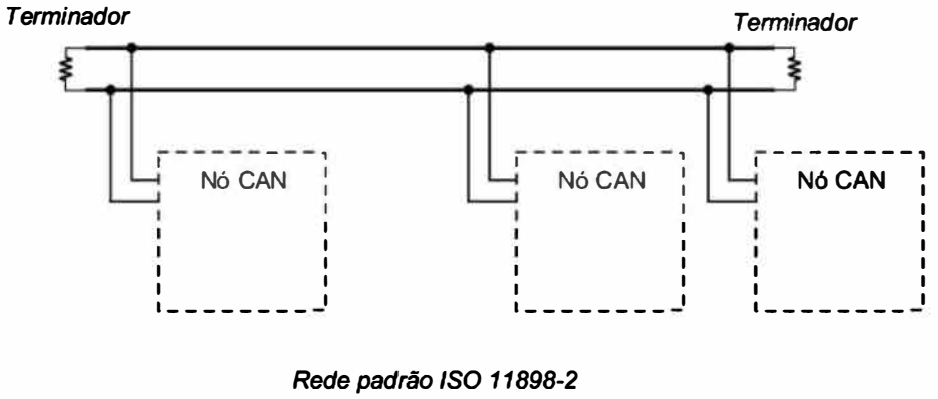


Figura 18.16: Exemplo de rede de alta velocidade CAN no padrão ISO 11898-2

Durante os anos, várias versões do protocolo CAN foram lançadas e padronizadas. Em 1991 foi apresentado pela Bosch o CAN 2.0 de até 1Mbps com duas partes de especificação: a padrão, com um identificador de 11-bits (CAN 2.0A) e a estendida com um identificador de 29-bits (CAN 2.0B). No ano de 1993 foi entregue o padrão CAN chamado ISO 11898, onde foi reestruturado em duas partes novamente: o ISO 11898-1, para a camada de comunicação para a troca de dados, e o ISO 11898-2, para a camada física, isto é, o sistema elétrico que garante a comunicação em alta velocidade. Posteriormente, foi mostrado o padrão ISO 11898-3 para a camada física de comunicação em baixa velocidade e tolerante a falhas. Os padrões ISO 11898-2 e 3 não fazem parte das especificações do antigo padrão CAN 2.0 da Bosch. Ainda foram padronizados protocolos derivados, como o CANopen e DeviceNet, para aplicações de automação e aeronáuticas.

A rede CAN é classificada como um barramento serial de múltiplos mestres para a conexão de inúmeras unidades de controle eletrônico (ECUs), chamadas de nós (nodes). Assim, dois ou mais nodes podem ser designados para a comunicação simultaneamente, ligando nodes simples, como dispositivos de entrada ou saída a microcontroladores ou a uma interface CAN, e um programa sofisticado. Além da presença de conversores que transferem as informações circulantes do CAN para redes Ethernet ou USB para computadores normais.

Cada node presente no CAN possui uma unidade de processamento para identificação de mensagens a serem lidas e transmitidas; um controlador padrão ISO 11898-1 como parte dessa unidade de processamento, responsável pelos buffers seriais de transmissão e recepção, identificando quando o barramento está livre para transmissão; um transceptor padrão ISO 11989-2/3 para interpretar os níveis de tensão do barramento e convertendo para os valores de tensão do controlador, além de desempenhar o papel de circuito de proteção contra surtos no barramento.

Diferente, do I²C e do SPI, o barramento CAN não possui uma via de sincronismo, sua comunicação é sincronizada em cada nova amostra de bit enviada no barramento.

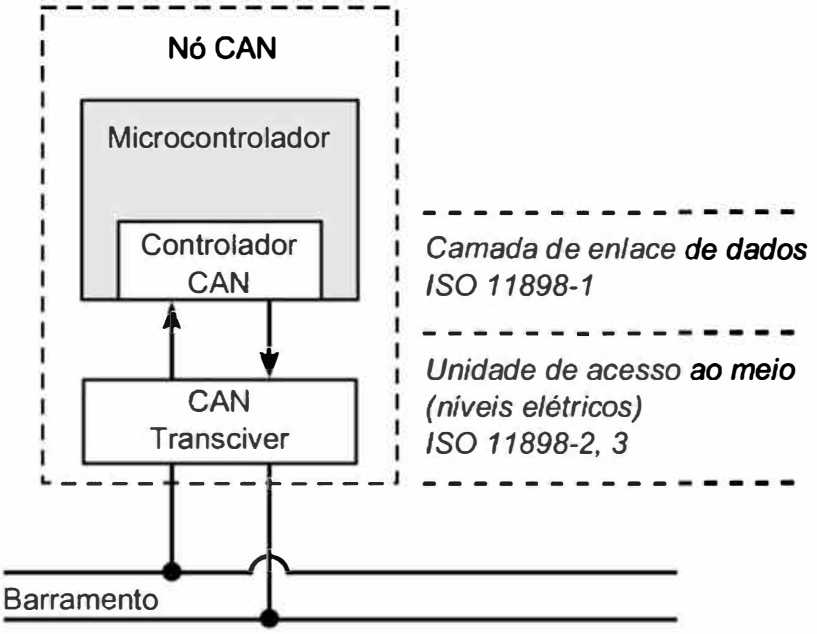


Figura 18.17: Partes integrantes de um node CAN ligado ao barramento

Isto é chamado de sincronismo de CAN, mas não é um termo preciso pois os dados são transmitidos sem um sinal de clock em um formato assíncrono. Os bits que formam as palavras de transmissão são organizados em estruturas ou mensagens, inicialmente definidas por um identificador, representando a prioridade da mensagem. Para garantir a integridade dos bits, um valor de CRC (Cyclic Redundancy Check) também faz parte da mensagem.

Caracterizada como uma rede ponto a ponto (peer-to-peer), ela não possui um gerenciamento central entre os nodes para acesso de escrita ou leitura no barramento. Quando um node está pronto para transmitir dados é verificado se está ocupado o barramento, então transmite-se a mensagem. A mensagem transmitida não possui um endereço ou um node de destino, ao invés disso, um identificador do tipo de informação é vinculado à mensagem (pressão, combustível, torque, temperatura) então os demais nodes do barramento decidem se irão ou não aceitar a mensagem. Um exemplo disso é o sensor do motor enviar a mensagem de temperatura no barramento CAN e tanto o node da injeção eletrônica quanto o node do painel do veículo se interessarem em capturar essa mensagem na rede para utilizarem em suas finalidades.

No caso de múltiplos envios simultâneos para a transmissão de mensagens, o node com maior prioridade, isto é, o valor de identificador mais baixo, ganha automaticamente o acesso ao barramento. Nodes de baixa prioridade precisam aguardar até que o barramento esteja disponível novamente antes de retransmitir. Logo, sensores mais importantes como temperatura e nível do óleo, têm o uso prioritário do barramento CAN em relação a sensores gerais, como o sensor lambda de mistura de gases ou o sensor de nível de combustível.

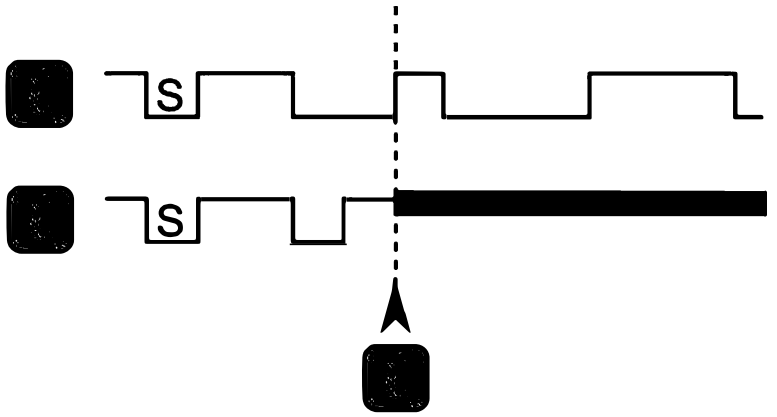


Figura 18.18: Controle de prioridades no barramento CAN

18.4 RS232

O protocolo de comunicação RS232 (Recommended Standard 232) é muito utilizado para comunicação entre dispositivos que transmitem ou recebem pouca quantidade de informação. É um dos protocolos mais antigos ainda em uso, tendo seu primeiro uso em 1962 em máquinas eletromecânicas de escrever. O padrão RS232 revisão C é datado de 1969. Em 1986 aparece a revisão D pela EIA (Electronic Industries Alliance). A versão atual do protocolo é datada de 1997 pela TIA (Telecommunications Industry Association) sendo chamada TIA-232-F.

Este é um protocolo ponto a ponto assíncrono. Isto quer dizer que apenas dois dispositivos podem conversar usando o mesmo caminho de comunicação. O assincronismo vem do fato de que não existe o envio de um clock para sincronizar o envio dos bits.

Como o sincronismo não existe, é necessário que ambos os dispositivos que vão conversar operem na mesma frequência de comunicação. Esta frequência define quanto tempo cada um dos bits deve estar disponível no barramento. Para facilitar a comunicação foi criado um "bit" que indicia o início da transmissão.

Por exemplo uma frequência de 1Hz: cada bit deve estar disponível durante exatamente 1 segundo. Assim que o dispositivo percebe que o bit de início foi enviado, ele espera 1 segundo e realiza a primeira medição. Mais um segundo e o microcontrolador faz a leitura do segundo bit. Este procedimento se repete até que todos os bits tenham sido lidos. Se ambos os lados forem configurados corretamente, as leituras e escritas serão realizadas de maneira alinhada, permitindo que a mensagem seja recebida corretamente.

RS232 ou UART?

Em diversos microcontroladores, a porta de comunicação padrão é denominada UART, Universal Asynchronous Receiver/Transmitter, ou Receptor/Transmissor Universal Assíncrono. Ela é uma versão generalizada dos protocolos EIA, RS-232, RS-422 e RS-485 e tem como objetivo fornecer um periférico customizável que possa ser utilizado em qualquer um destes protocolos.

Para conseguir realizar a comunicação entre dois microcontroladores utilizando UART, é necessário que o sinal do terminal de transmissão do primeiro esteja conectado no terminal de recepção do segundo e a transmissão do segundo na recepção do primeiro. Por fim, precisamos garantir que ambos os microcontroladores utilizam a mesma referência como terra, como mostrado na figura 18.19.

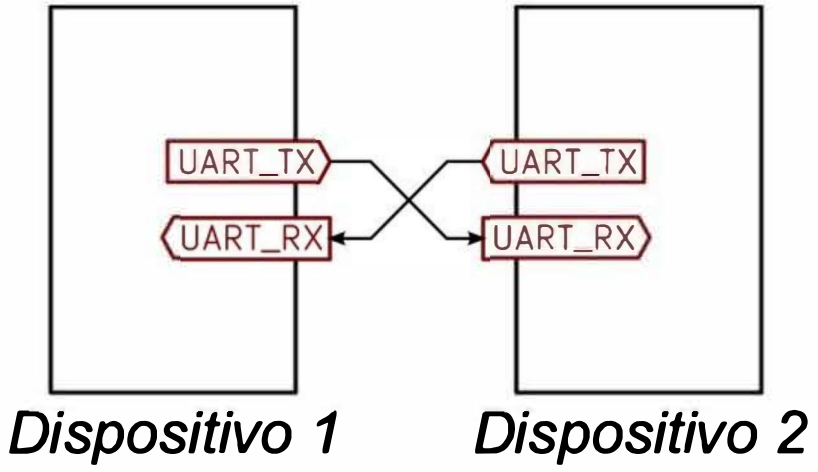


Figura 18.19: Comunicação UART

Uma das grandes diferenças entre a UART e o RS232, ou o RS485, está nos níveis de tensão utilizados. Enquanto os protocolos utilizam tensões positivas e negativas, com uma faixa ampla de valores aceitos, as tensões no protocolo UART são compatíveis com os níveis de tensão utilizados pelo microcontrolador: seja 5 ou 3,3 volts.

Para fazer a conversão dos sinais podemos utilizar chips dedicados, como os apresentados nas figuras 18.20(a) e 18.20(b).

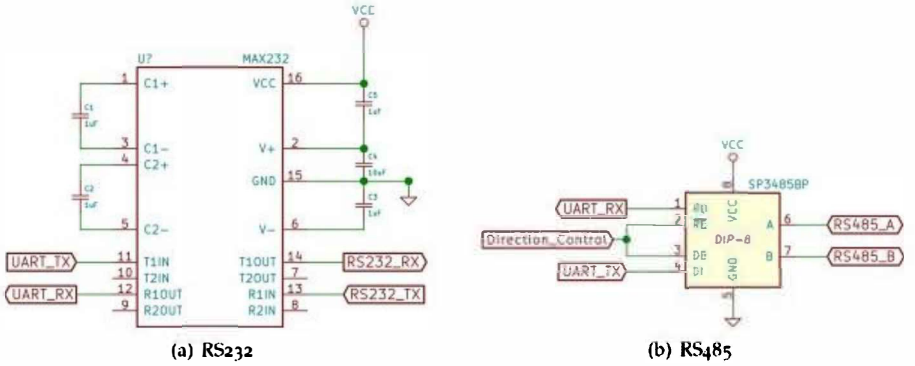


Figura 18.20: Chips de conversão de nível utilizando UART

O procedimento de envio de um valor pela serial através do padrão RS232 pode ser visto como uma operação de bit-shift.

No protocolo RS232 a transmissão começa pelo bit menos significativo. Este bit permanece durante um determinado tempo, baseado na velocidade de transmissão.

Depois deste tempo, realiza-se um shift para a direita e o "novo" bit menos significativo é "reenviado". Esta operação é repetida oito vezes, uma para cada bit.

Por fim, é enviado um último bit indicando o fim da transmissão.

Para o protocolo RS232, o nível alto ou **1** (um) é aquele com tensões positivas entre -3 e -15. O nível lógico baixo ou **0** (zero) é interpretado entre +3 e +15 volts.

A figura 18.21 apresenta o sinal elétrico enviado ao longo do tempo para a letra "**K**" maiúscula. A região em branco, que se estende entre +3 e -3, indica a região de tensão cujo valor de bit não está definido. Caso a tensão lida esteja entre estes limiares, seja devido a ruídos ou outros problemas, o sistema de recepção não entenderá a mensagem e os dados podem ser perdidos ou corrompidos. Em ASCII, a letra **K** é codificada como $76_{10} = 11010010_2$.

Antes de iniciar a transmissão dos bits, é enviado um bit que marca o início da transmissão, chamado start bit. Este bit não é contado como informação útil.

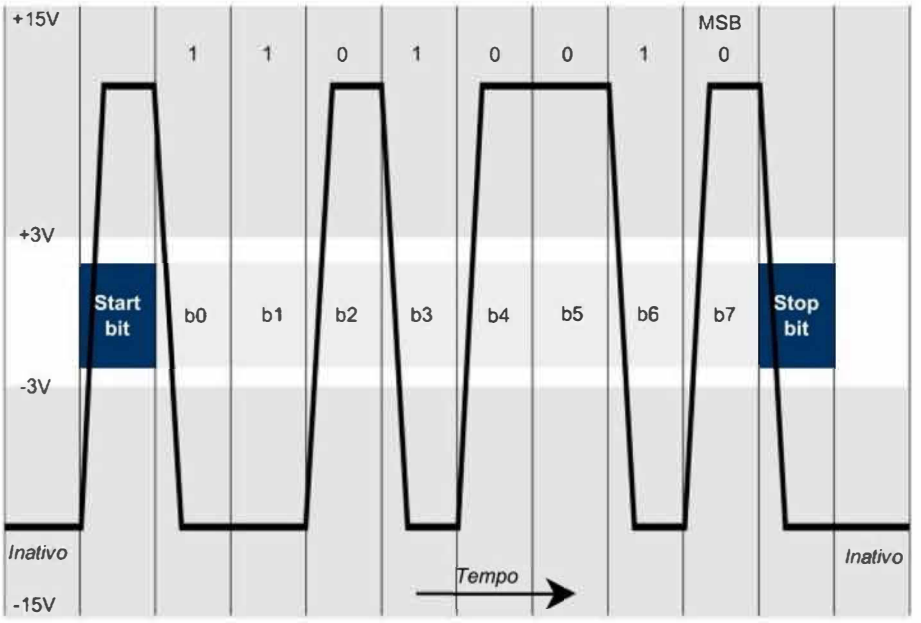


Figura 18.21: Sinal serializado para transmissão em RS232
Fonte: Adaptado de Samuel Tardieu

Para que o protocolo funcione corretamente, é necessário que todos os valores estejam configurados corretamente em ambos os dispositivos que forem conversar. Isto significa definir a codificação utilizada (ASCII, UTF-8 etc.), especificar o fluxo de caracteres (quantidade de bits por caractere, tamanho do stop bit, uso ou não de paridade) bem como a taxa de transmissão desejada.

Do mesmo modo que no protocolo I²C, os fabricantes de microcontroladores desenvolveram periféricos dedicados a tratar dos detalhes. Isto permite que o programador fique focado no desenvolvimento da aplicação, não se preocupando com os processos de serialização dos dados ou de temporização dos sinais.

O código 18.7 apresenta o processo de configuração do periférico UART para a plataforma Freedom.

Código 18.7: Configuração do periférico de UART

```
1 void serialInit(void) {
2     unsigned int sbr_val;
3
4 //1 – Desliga a UART antes de configurá-la
5     UART0_C2 &= ~(UART0_C2_TE_MASK | UART0_C2_RE_MASK);
6
7 //2 – Configuração dos terminais
8     //Configura o pino D0 como transmissor serial
9     PORTB_PCR1 = PORT_PCR_MUX(0x2);
10    //Configura o pino D1 como receptor serial
11    PORTB_PCR2 = PORT_PCR_MUX(0x2);
12
13 //3 – Configuração do oscilador da serial
14    //Seleciona o clock da serial a partir do clock principal da CPU
15    SIM_SOPT2 |= SIM_SOPT2_UART0SRC(1);
16    //habilita o clock para o periférico da UART
17    SIM_SCGC4 |= SIM_SCGC4_UART0_MASK;
18    //A velocidade de transmissão é dada por: baudrate = sys_clock/(OSR x sbr)
19    // Configurando OSR para 10x
20    UART0_C4 = 0x09;
21
22 //4 – Configurando o valor do baudrate
23    //sbr_val = (sys_clock) / (115200 * 10));
24    sbr_val = (unsigned int) (24000000 / (115200 * 4));
25    //o valor de sbr tem que ser armazenado em dois registros diferentes
26    //os 5 bits mais altos vão para o UART0_BDH
27    UART0_BDH = ((sbr_val & 0x1F00) >> 8);
28    //os 8 mais baixos vão para UART0_BDL
29    UART0_BDL = sbr_val & 0x00FF;
30
31 //5 – Liga o transmissor e o receptor
32    UART0_C2 |= (UART0_C2_TE_MASK | UART0_C2_RE_MASK);
33 }
```

Cada plataforma possui sua própria configuração, bem como cada microcontrolador diferente apresenta registros e configurações distintas. Para saber como configurar a UART para uma outra arquitetura, o primeiro passo é entender, a partir do datasheet, quais registros estão relacionados ao periférico. Em geral, os datasheets apresentam as configurações padrões para o funcionamento do periférico. Em geral, o algoritmo de configuração é dado por:

1. Desligamento da UART antes de configurá-la;
2. Configuração dos terminais físicos: tipo de periférico, RX como entrada e TX como saída;
3. Configuração da velocidade de funcionamento do periférico: de onde vem o sinal de clock, qual oscilador utilizar e os divisores de frequência;
4. Configuração do valor de baudrate: a velocidade de transmissão/recepção de dados deve ser a mesma do periférico que será conectado ao microcontrolador. Em geral, existe um registro que faz a configuração do tempo baseado no clock do periférico (configurado no passo anterior);
5. Ligação do transmissor e do receptor.

Para as placas Arduino e Chipkit, quando utilizando o framework Wiring, algumas dessas atividades são realizadas pelo bootloader. As demais podem ser facilmente executadas através da função **`Serial.begin()`**, passando o valor de baudrate desejado. Além da configuração

do periférico, em sua inicialização, existem duas outras funções que devem ser implementadas: **`serialRead()`** e **`serialSend()`**.

Em geral existem registros que permitem acessar facilmente os dados que foram recebidos pela serial, bastando que o programador faça a leitura do registro correto. Antes de ler o valor, no entanto, devemos verificar se o periférico recebeu realmente alguma informação; caso contrário, o valor lido será apenas lixo da memória.

Para enviar os dados, o processo é parecido. Para transmitir os bits basta escrever o byte num registro adequado. Como os bits são enviados um a um, é possível que o periférico ainda não tenha terminado de mandar os 8 primeiros bits quando o programa quiser enviar o segundo byte. Para evitar de corromper a transmissão, devemos checar se o barramento está disponível.

O arquivo de header da biblioteca de comunicação serial é apresentado no código

Código 18.8: serial.h

```
1 #ifndef SERIAL_H
2 #define SERIAL_H
3     void serialSend(char c);
4     char serialRead(void);
5     void serialInit(void);
6 #endif
```

Código 18.9: serial.c

```
#include "serial.h"
#include "io.h"
#include "derivative.h"

void serialSend(char c) {
    while (!(UART0_S1_REG(UART0_BASE_PTR) & UART0_S1_TDRE_MASK));
    UART0_D_REG(UART0_BASE_PTR) = c;
}
char serialRead(void) {
    //Verificar se há algo disponível
    if ((UART0_S1_REG(UART0_BASE_PTR) & UART0_S1_RDRF_MASK)) {
        //Lê o registro da serial
        return UART0_D_REG(UART0_BASE_PTR) ;
    } else {
        //Código para "não há caracter disponível"
        return 0xff;
    }
}

void serialInit(void) {
    unsigned int sbr_val;
    //Configura os terminais D0 e D1
    PORTB_PCR1 = PORT_PCR_MUX(0x2);
    PORTB_PCR2 = PORT_PCR_MUX(0x2);
    //Seleciona o clock da serial a partir do clock principal da CPU
    SIM_SOPT2 |= SIM_SOPT2_UART0SRC(1);
    //habilita o clock para o periférico da UART
    SIM_SCGC4 |= SIM_SCGC4_UART0_MASK;
    // Desliga a UART antes de configurá-lo
    UART0_C2 &= ~(UART0_C2_TE_MASK | UART0_C2_RE_MASK);
    //A velocidade de transmissão é dada por: baudrate = sys_clock/(OSR * sbr)
    // Configurando OSR para 10x
    UART0_C4 = (10-1);//sempre 1 unidade a menos
    //escolhendo o valor de sbr para um baudrate de 115200
    //sbr_val = (sys_clock) / (baudrate * OSR));
    sbr_val = (unsigned int) (24000000 / (115200 * 10));
    //o valor de sbr tem que ser armazenado em dois registros diferentes
    //os 5 bits mais altos vão para o UART0_BDH
    UART0_BDH = ((sbr_val & 0x1F00) >> 8);
    //os 8 mais baixos vão para UART0_BDL
    UART0_BDL = sbr_val & 0x00FF;
    //Liga o transmissor e o receptor
    UART0_C2 |= (UART0_C2_TE_MASK | UART0_C2_RE_MASK);
}
```

18.5 USB

O padrão USB, ou Universal Serial Bus, foi desenvolvido inicialmente em 1994 por sete empresas: Compaq, DEC, IBM, Intel, Microsoft, NEC e Nortel. O padrão visa estabelecer modelos de cabos, conectores, protocolos de comunicação e sistemas de alimentação para comunicação entre dispositivos.

Este padrão simplificou a comunicação e integração de dispositivos, principalmente entre periféricos e computadores, substituindo uma grande quantidade de modelos de conectores por apenas 2 modelos: tipo A e tipo B. Com a criação de novas versões do padrão, outros modelos foram criados, principalmente pela necessidade de miniaturizar os plugs e receptáculos.

No entanto, uma das maiores vantagens do padrão foi a padronização do protocolo de comunicação, que gerou um sistema de identificação bastante complexo capaz de abrigar quase todos os tipos de periféricos. Deste modo, praticamente qualquer tipo de equipamento pode ser interconectado utilizando o padrão USB. Na tabela 18.1 são apresentadas algumas das classes de dispositivos reconhecidos pelo padrão.

Tabela 18.1: Dispositivos reconhecidos pelo padrão USB

Classe	Descrição	Exemplo
01h	Áudio	Auto-falantes, microfones, porta MIDI
02h	Comunicação	Modem, Adaptador de ethernet, Wi-Fi, RS232.
03h	Dispositivo de interface humana (HID)	Teclado, mouse, joystic
05h	Dispositivo de interface físico (PID)	Joystic com forcefeedback
06h	Imagem	Webcam, scanner
07h	Impressora	Impressoras laser/tinta, máquinas de CNC
08h	Armazenamento em massa	Pen drives, leitores de cartões de memória, drivers externos
0Dh	Sistemas de segurança	Leitores de impressão digital
0Eh	Vídeo	Webcam
0Fh	Dispositivos de saúde pessoal	Monitores cardíacos
FEh	Específico para aplicação	Ponte de comunicação infravermelha, Porta de atualização de dispositivo
FFh	Definido pelo fabricante	Indica que precisa de drivers específicos

Com a padronização dos tipos de periféricos, não é mais necessário possuir um driver para cada dispositivo diferente. Por exemplo, todos os mouses USB (com ou sem fio), são reconhecidos pelo sistema operacional do mesmo modo, não sendo necessário instalar um drive diferente para cada tipo de mouse conectado.

O padrão 1.0 permite atingir uma velocidade de transmissão de até 12 Mbit por segundo no modo "Full speed". Os padrões seguintes tiveram um impacto muito grande na taxa de transmissão. A velocidade atingiu o patamar de 480Mbit por segundo na USB 2.0 e até 5Gbit por segundo no 3.0.

A comunicação USB utiliza um modelo de transmissão físico chamado diferencial. Neste modelo, a informação é enviada através de 2 fios, ao invés de 1. O sinal nestes dois fios é transmitido de modo invertido, ou diferencial. Este tipo de transmissão traz uma grande vantagem: aumento da imunidade a ruídos.

Isto acontece por causa do processo de decodificação do sinal. Quando o sinal chega ao receptor, a primeira providência é subtrair os dois sinais para obter o sinal original. Se, durante a transmissão, algum sinal eletromagnético gerar um ruído nos dados transmitidos, esse ruído será sentido igualmente nos dois terminais, pois eles estão juntos no cabo. Quando o receptor fizer a subtração o ruído será eliminado. A figura 18.22 apresenta o modelo de transmissão de um pacote no padrão 1.0.

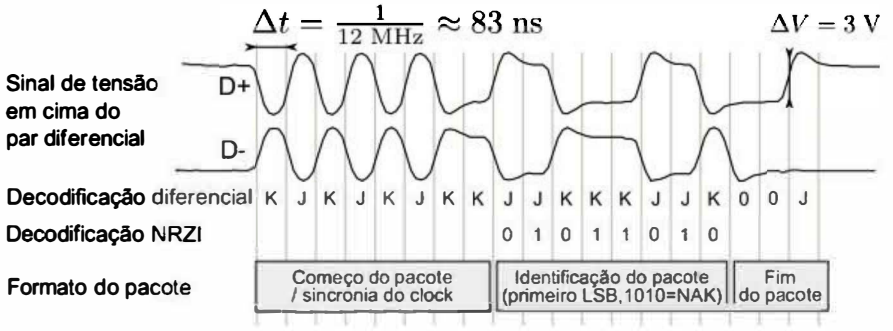


Figura 18.22: Sinal serializado da comunicação USB

Os dados são transmitidos utilizando a codificação NRZI, ou Non-return-to-zero inverted. Nesta codificação, a transmissão de um bit **0** se dá invertendo o sinal anterior. A transmissão de um bit **1** acontece mantendo o sinal anterior. O padrão 3.0 possui outras opções de codificação.

Um pacote USB começa com um byte de sincronização com o valor 0x01. Como podemos ver na figura 18.22, os sete primeiros bits são zero, havendo mudança nos sinais e o último bit vale um, por isso o sinal não se altera. Os protocolos 2 e 3 apresentam outros modelos possíveis de sincronização/início de transmissão.

Para implementar um protocolo USB em um dispositivo embarcado é necessário que o microcontrolador utilizado possua suporte embutido no chip. Não é viável implementar todo o protocolo USB apenas em software.

Caso o dispositivo não possua uma comunicação USB nativa, existem algumas alternativas. Uma das mais utilizadas são os conversores UART/USB, ou RS232/USB. São esses os conversores que permitem a comunicação entre a maioria das placas compatíveis com Arduino e os computadores.

Serial sobre USB

Um dos modos mais simples de conectar um microcontrolador a um computador é fazer uso de um conversor USB/Serial. Um dos modelos mais comuns é o FT232. O esquemático para ligação é apresentado na figura 18.23:

A vantagem de se utilizar esse tipo de conversor é que os sistemas operacionais já estão preparados para trabalhar com esse dispositivo. Isto reduz o trabalho necessário para se desenvolver um aplicativo para o computador que consiga ler os sinais do microcontrolador.

Para melhorar a comunicação entre o computador e o microcontrolador, é interessante criar algum protocolo de comunicação, onde os valores são passados obedecendo um padrão. Um exemplo de comunicação é apresentado na seção de leitura de protocolos.

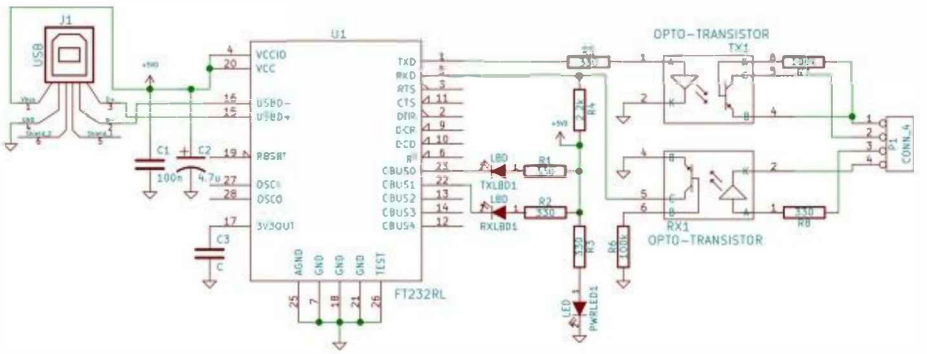


Figura 18.23: Circuito de comunicação USB/Serial

18.6 Serial sem fios

O conceito Wireless (sem fio) tornou-se comum para a maioria das aplicações embarcadas, de tal forma que existe uma certa desvalorização dos dispositivos domésticos processados, que necessitam de uma comunicação via cabos, como TVs inteligentes (SmartTVs), equipamentos de som, impressoras, headfones etc.

A forma mais simples e barata de transmitir uma informação digital pelo ar é utilizar o conceito de comunicação serial sem fios, sendo regida pelo padrão IEEE 802 para a transmissão de dados em redes locais e áreas metropolitanas, define vários protocolos conhecidos, como o ZigBee no IEEE 802.15.4, o Bluetooth com o IEEE 802.15.1 ou a rede WiFi no padrão IEEE 802.11.

Os protocolos de transmissão ZigBee e Bluetooth são utilizados em aplicações de baixo custo e baixa potência. O módulo Zigbee é utilizado em sistemas de controle sem fio e aplicações de monitoramento de rede em malha (mesh), seus transceptores possuem microcontroladores que realizam tarefas simples de entrada e saída, além de uma porta serial para transmissão a outros módulos. No caso do Bluetooth, seu uso é comum em sistemas de baixo alcance (de 1 a 100 metros) em dispositivos de uso doméstico, como equipamentos de som, celular, som automotivo ou entre computadores, onde a baixa velocidade de comunicação não é um problema.

Todos os protocolos citados neste tópico (ZigBee, Bluetooth e WiFi) trabalham na faixa de 2,4GHz, mas raramente ocorrem problemas de interferência entre eles, pois trabalham com um número razoável de canais que são alterados de forma automática para faixas livres de ruídos, como nas versões mais recentes do protocolo ZigBee. As interferências podem surgir quando a faixa de frequências é próxima a 2,4GHz, que já está saturada por um uso não otimizado de equipamentos sem fio em um mesmo local.

Do mesmo modo que a comunicação USB, é possível criar dispositivos que utilizem esses protocolos de modo nativo ou façam uso de conversores. A utilização de conversores simplifica o projeto e acelera o desenvolvimento do produto, mas aumenta o custo final. Por outro lado, projetar diretamente com esses protocolos é um processo um pouco mais demorado, mas traz o benefício de um projeto com menos componentes, consequentemente mais barato e, até certo ponto, mais otimizado para a aplicação. No entanto, esses pontos estão além do escopo deste livro.

18.7 Leitura e processamento de protocolos

Na maioria dos casos de comunicação entre dois dispositivos, é necessário enviar um conjunto de dados de modo organizado para conseguir transmitir uma mensagem. As mensagens, por sua vez, são geralmente compostas de vários bytes, onde cada um deles pode representar uma informação distinta: início/fim de mensagem, ID do transmissor/receptor, tipo de mensagem, valores de variáveis, códigos de checagem etc. Um conjunto de bytes formando uma mensagem é comumente chamado de pacote. Para conseguir realizar o processamento dos protocolos, é muito importante armazenar todos os bytes do pacote primeiro. Se algum dos bytes for perdido, toda a mensagem pode ser comprometida.

Outro problema com a comunicação é que não se pode esperar que a mensagem chegue corretamente, nem que o dispositivo que está enviando a mensagem funcione corretamente. Por esse motivo é necessário criar uma rotina de leitura que contemple estes possíveis problemas e possa se recuperar no caso de uma falha na recepção dos dados.

Uma boa estrutura é criar um buffer que armazenará todos os bytes recebidos na mesma ordem em que chegarem. A cada novo byte, o buffer será processado para saber se existe um pacote completo no buffer. Em caso afirmativo, o pacote é processado e as providências podem ser tomadas. Se não, o sistema simplesmente aguarda o próximo byte.

Em geral, os pacotes possuem tamanhos pré-definidos e/ou bytes de início e fim. Estas informações são úteis para averiguar possíveis erros. Se um byte de início chegou antes de completar o pacote anterior, provavelmente algum byte do pacote anterior foi perdido e, portanto, a mensagem anterior não tem validade. Se a quantidade de bytes recebidos for maior do que o tamanho pré-definido do pacote, é provável que um byte de início ou fim foi perdido e dois pacotes diferentes se misturaram, invalidando ambos. Em todos esses casos podemos reiniciar o buffer.

A verificação da existência de um pacote completo pode ser baseada também em dois fatores: a quantidade de bytes recebidos ou o byte de fim de mensagem, dependendo de como o protocolo é implementado. O código 18.10 apresenta um exemplo de rotina que realiza o armazenamento dos bytes, detecta a recepção de um pacote, processa o pacote de acordo com o protocolo e identifica possíveis erros.

Código 18.10: Modelo para leitura e processamento de protocolo

```
unsigned char pos=0; //posição atual do buffer
char buffer[100];    //buffer de armazenamento dos dados

//supondo uma mensagem com 50 bytes
#define MSG_SIZE    50
#define START_BYTE '$'
#define END_BYTE   '\n'

void main (void){
    for(;;){
        //recebimento dos dados
        data = serialRead();

        //teste de inicio de mensagem (por starbyte)
        if(data == START_BYTE){
            //se chegou start byte pode ignorar os dados anteriores
            pos=0;
        }

        //se chegou algum byte armazena no buffer
        if (data != 0xff) {
            buffer[pos] = data;
            pos++;
        }

        //se ultrapassou o tamanho máximo, aconteceu erro, reseta o buffer
        if (pos >= 100) {
            pos = 0;
        }

        //teste de fim de mensagem: por tamanho de bytes
        if(pos == MSG_SIZE){
            //verifica validade da mensagem e processa o comando
        }
        //teste de fim de mensagem: por recepção do endByte
        if(data == END_BYTE){
            //verifica validade da mensagem e processa o comando
        }

        //teste de fim de mensagem: por tamanho e endByte, mais seguro
        if((pos == MSG_SIZE) && (data == END_BYTE)){
            //verifica validade da mensagem e processa o comando
        }
    }
```

Na próxima seção será analisado um caso em particular, exemplificando o processo de leitura de dados via comunicação serial.

O protocolo NMEA de GPS

O protocolo NMEA é um formato de comunicação de dispositivos de navegação marinha, incluindo sonares e receptores de sinal GPS. A maioria dos dispositivos utilizados no NMEA 0183 utilizam como protocolo serial o padrão EIA-422, sendo que alguns equipamentos possuem saídas RS232.

Neste protocolo, os dados são codificados em ASCII. Isto faz com que seja necessário convertê-los em números para efetuar qualquer tipo de processamento ou análise de valores. Para a simples exibição de dados, os bytes podem ser enviados diretamente para o display de LCD.

Dentro do protocolo NMEA existem várias mensagens. Cinco estão relacionadas com dados de GPS:

- GPGSA - Precisão e quantidade de satélites ativos
- GPGCA - Dados fixados (lat/long/alt/diferencial)
- GPRMC - Dados mínimos (lat/long/alt)
- GPVTG - Velocidade e deslocamento
- GPGSV - Dados dos satélites visíveis

Cada mensagem possui um tamanho distinto, com diferentes informações. Em todas elas os campos são divididos pelo caractere de vírgula. A mensagem do tipo GPRMC, por exemplo, é formada por 14 campos:

1. ID da mensagem
2. Tempo (UTC)
3. Rastreando (R) ou Aceitável (A)
4. Latitude
5. Norte/Sul
6. Longitude
7. Leste/Oeste (E/W)
8. Velocidade (magnitude)
9. Velocidade (ângulo)
10. Data (UTC)
11. Variação magnética (ângulo em graus)
12. Variação magnética (direção, E/W)
13. Modo de operação (N não válido, A autônomo, D diferencial, E estimado, M manual, S simulação)
14. Checksum

Todos os dados são codificados em ASCII, onde cada dígito do número representa um byte. Este procedimento faz com que a mensagem ocupe mais memória para ser armazenada, no entanto, evita problemas para a definição do valor dos bytes de início e fim, além de simplificar o processo de visualização das mensagens.

A estrutura esperada de um pacote GPRMC é dada por:

```
$GPRMC,000000,X,0000.00,X,00000.00,X,000.0,000.0,000000,000.0,????\r\n
```

Os zeros indicam posições ocupadas por números, as letras **'X'** por caracteres e as interrogações indicam o valor do CRC.

O CRC é um código que indica se os dados estão corretos ou não. Durante a transmissão dos dados, é possível que algum bit tenha seu valor trocado, fazendo com que a informação seja inválida. Para minimizar as possibilidades de erro, cada byte da mensagem é somado em uma variável temporária e o resultado da soma é enviado ao fim da mensagem. Quando o receptor receber a mensagem, ele refaz a conta e compara com o resultado recebido. Se eles forem iguais a mensagem provavelmente está integra. O CRC utilizado neste protocolo é o CCITT16, que envolve, no lugar da soma dos bytes, uma operação um pouco mais complexa.

Uma mensagem do tipo GPRMC traz basicamente informações de posição e velocidade. A seguir, temos um exemplo de uma mensagem real, onde cada posição da mensagem é um byte a ser recebido pela comunicação serial.

```
$GPRMC,220516,A,5133.82,N,00042.24,W,173.8,231.8,130694,004.2,W*70\r\n
```

Para processar a mensagem corretamente é necessário primeiro receber todos os bytes da mensagem. Quando a mensagem estiver completa no buffer temporário, podemos analisar o pacote e tomar providências do que fazer.

A mensagem possui o caractere cifrão, **'$'**, como byte de início. Para byte de fim, o protocolo usa dois caracteres, comumente utilizados como fim de linha: <cr> e <lf>. Na linguagem C, esses caracteres são representados por **'\n'** e **'\r'**, correspondendo aos números 13 e 10 da tabela ASCII.

Quando a mensagem estiver completa, devemos inicialmente verificar se é o pacote desejado: **GPRMC**. Depois podemos percorrer cada caractere do buffer procurando pelas vírgulas. A cada vírgula encontrada, sabemos que estamos em um novo campo de dados. No c ódigo 18.11, é apresentado todo o processo de recepção e processamento da mensagem. O algoritmo de recepção é um pouco distinto do apresentado anteriormente, pois apenas estaremos testando o fim de mensagem através do byte de fim. O processo de ler cada um dos bytes da mensagem e processar seu conteúdo também é conhecido como parser.

Código 18.11: Parser do protocolo NMEA para exibição em LCD

```
 1 #include "serial.h"
 2 #include "lcd.h"
 3 #include "io.h"
 4 char buffer[100];    //buffer temporário
 5
 6 void main(void) {
 7     unsigned char pos=0; //posição atual do buffer
 8     char data;
 9     systemInit();
10     serialinit();
11     lcdInit();
```

```
12      for(;;) {
13          //recebimento dos dados
14          data = serialRead();
15          if (data != 0) {
16              buffer[pos] = data;
17              pos++;
18          }
19          if (pos >= 100) {
20              pos = 0;
21          }
22          //teste de fim de mensagem: pelos dois bytes de fim
23          if((pos > 2) && (buffer[pos-2] == 13) &&  (buffer[pos-1] == 10)) {
24              pos = 0;
25              //Verifica se é a mensagem correta para processar os dados
26              if( (buffer[0] == '$') && (buffer[1] == 'G') &&
27                  (buffer[2] == 'P') && (buffer[3] == 'R') &&
28                  (buffer[4] == 'M') && (buffer[5] == 'C')) {
29                  //Parser começa já no 2o campo, depois do ID
30                  pos = 7;
31                  while(buffer[pos] != ','){      //2 - Hora UTC
32                      //processa bytes da hora
33                      pos++;
34                  }
35                  pos++;//pula a vírgula
36                  while(buffer[pos] != ','){      //3 - Sinal pronto? Sim(A), Não(R)
37                      //processa dados de estabilidade do sinal
38                      pos++;
39                  }
40                  pos++;//pula a vírgula
```

```
41
42                  lcdCommand(0x80);
43                  lcdString("Lat");
44
45                  while(buffer[pos] != ','){      //4 - Latitude
46                      //envia dados de latitude para o LCD
47                      lcdChar(buffer[pos]);
48                      pos++;
49                  }
50                  pos++;//pula a virgula
51                  while(buffer[pos] != ','){      //5 - Norte(N)/Sul(S)
52                      lcdChar(buffer[pos]);
53                      pos++;
54                  }
55                  pos++;//pula a virgula
56                  lcdCommand(0xC0);
57                  lcdString("Lon")
58                  while(buffer[pos] != ','){      //6 - Longitude
59                      //envia dados de longitude para o LCD
60                      lcdChar(buffer[pos]);
61                      pos++;
62                  }
63                  pos++;//pula a virgula
```

```
64                  while(buffer[pos] != ','){      //7 - Leste(E)/Oeste(W)
65                      lcdChar(buffer[pos]);
66                      pos++;
67                  }
68                  pos++;//pula a virgula
69                  //..continua para os demais campos
70              }
71          }
72      }
73  }
```

18.8 Exercícios

Ex. 18.1 — O que é clock stretching?

Ex. 18.2 — Faça um programa que receba um caractere via comunicação serial e, se este valor for uma letra, o exiba no LCD. Caso este caractere seja um número, ele deve ser exibido no display de 7 segmentos. Utilize as bibliotecas “ssd.h” “lcd.h” e “serial.h”.

Ex. 18.3 — Crie um programa que controle o led RGB da placa, de acordo com o caractere recebido pela porta serial. Se o caractere 'R' for recebido, o led vermelho deve ser aceso e assim sucessivamente para os caracteres 'G' para verde, 'B' para azul, 'C' para ciano, 'Y' para amarelo, 'P' para roxo e 'W' para branco. Para qualquer outro caractere recebido, o led deve ser desligado.

Ex. 18.4 — Grande parte dos protocolos de comunicação serial se utilizam da checagem de integridade para garantir que a mensagem está correta. Uma delas é realizar a soma de todos os valores da mensagem e enviar esse resultado por último. Para indicar o começo de uma mensagem é comum se utilizar de um valor padrão como, por exemplo, o número 42, e um segundo número padrão é utilizado para marcar o final da mensagem como, por exemplo, 24. Utilizando essas informações, podemos verificar, por exemplo, que a primeira mensagem abaixo está correta, pois o somatório dos números entre o 42 e o 24, 01+02+03+04+05+06+07, é igual a 28, conforme escrito na posição depois do número 24. Já a segunda mensagem está errada, pois 00+01+02+03+04+05+06=21, e não 22, como representado. Mensagem 1:

```
[42 01 02 03 04 05 06 07 24 28]
```

Mensagem 2:

```
[42 00 01 02 03 04 05 06 24 22]
```

Dado que a biblioteca "serial.h' permite ao programador ler qualquer informação que for recebida pela serial, faça um programa que realize, ciclicamente, a leitura dos valores recebidos pela serial e calcule se a mensagem está correta ou errada. O tamanho da mensagem pode variar. As únicas informações confiáveis são que: a mensagem sempre começa com um número 42 e termina com um número 24 seguido de um número com o somatório dos números intermediários, conforme o exemplo dado.

Ex. 18.5 — Construa um programa que realize a leitura da serial e envie os caracteres recebidos para o LCD através da biblioteca "console.h". Caso seja detectado o caractere '**`w`**', o console deverá subir uma linha na exibição do histórico. Se for recebido o caractere '**`s`**', ele deve descer uma linha na exibição do histórico.

Ex. 18.6 — Faça um programa que receba comandos via serial. Quando chegar a palavra "IDE" pela serial, o programa deve escrever no LCD a versão da placa. Lembre-se: a função **`serialRead()`** retorna o valor **`0xff`** quando não há nada a receber na serial; a palavra "IDE" é composta de 3 caracteres, que devem ser recebidos em sequência; não é possível ter certeza de quando um caractere será recebido, portanto o programa deve ficar esperando constantemente. Podem ser utilizas as funções do lcd.h e serial.h.

CAPÍTULO

19

Conversor analógico digital

19.1 Elementos sensores
- Divisor resistivo
- Sensores ativos
- Sensores da placa de desenvolvimento

19.2 O conversor eletrônico
- Canais e multiplexação de entradas analógicas

19.3 Processo de conversão
- Criação da biblioteca

19.4 Aplicação

19.5 Exercícios

"As pessoas estão tão focadas na gravação digital agora que elas esqueceram quão fácil a gravação analógica pode ser."

Dave Grohl

Um conversor analógico digital é um circuito eletrônico capaz de transformar um valor de tensão numa informação codificada em formato digital. Os conversores são muito utilizados como interfaces de entrada entre sensores e o microcontrolador.

Os conversores podem ser encontrados separados ou integrados aos microcontroladores. Os conversores separados, em geral, possuem algum tipo de comunicação serial para enviar ao microcontrolador a informação já em formato digital. Naqueles que são integrados, o resultado é disponibilizado em um registro na memória.

Os conversores são definidos basicamente por três fatores: excursão máxima do sinal de entrada, resolução (quantidade de bits) e velocidade de conversão.

Para fazer uso correto dos conversores é importante entender os detalhes do sensor que será utilizado. Grande parte das rotinas e funções são dependentes da dinâmica do sensor, bem como de seu funcionamento interno.

Neste capítulo será apresentada uma breve introdução sobre os tipos de elementos sensores, bem como sobre o tipo de resposta que eles geram. Em seguida será apresentado o processo de conversão analógico digital e as rotinas computacionais para trabalhar com estes valores.

19.1 Elementos sensores

Nos sistemas embarcados, por diversas vezes temos a necessidade de realizar a leitura de algum tipo de informação, como aceleração, pressão, temperatura, acidez, luminosidade ou qualquer outra grandeza física. Para isto, podemos utilizar os elementos sensores.

O sensor é um dispositivo desenvolvido para apresentar uma resposta mensurável a um estímulo físico, químico ou biológico.

Os sensores eletrônicos apresentam, como resposta ao estímulo, a geração de um sinal elétrico, geralmente em tensão, podendo ser também em resistência ou corrente.

Para conseguir processar as medições realizadas é necessário utilizar o conversor analógico digital. A maioria dos conversores fazem leitura apenas de sinais de tensão. Deste modo, precisamos converter os sensores resistivos e de corrente para tensão.

A conversão dos sensores com saída resistiva é feita utilizando a topologia de divisor resistivo.

Os sensores que entregam sinais de corrente podem ser convertidos com o uso de uma resistência em paralelo com a leitura.

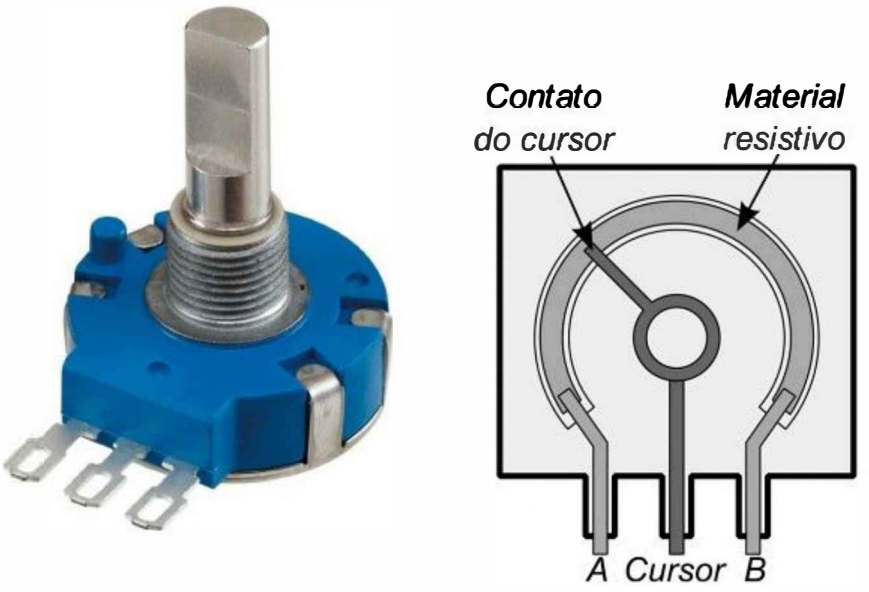
Os sensores, cuja saída já é dada em tensão, podem ser ligados diretamente no conversor, se as tensões forem compatíveis.

Divisor resistivo

O divisor resistivo é dado por duas resistências ligadas em série e alimentadas por uma fonte de tensão. Diversos elementos sensores são baseados na variação de um valor de resistência.

Na placa de desenvolvimento, dois sensores fazem uso desta estrutura: um sensor de ângulo, o potenciômetro, e um sensor de luminosidade, o LDR.

O potenciômetro (figura 19.1) é constituído por uma resistência distribuída ao longo de um círculo. Em cima desta trilha é adicionado um cursor, que pode ser movimentado através de um eixo. A resistência entre o início e o fim da trilha é fixo, podendo ter praticamente qualquer valor de resistência, de poucos ohms até dezenas de milhões.



(a) Imagem (*Fonte: Iain Fergusson*) (b) Resistência

Figura 19.1: Potenciômetro

O cursor divide a resistência em duas parcelas, cuja soma continua sendo constante. Estas parcelas podem ser interpretadas como duas resistências ligadas em série. O valor de cada uma das resistências pode variar de zero até o máximo. A capacidade de variar os valores das resistências é o que permite transformar esta estrutura num elemento sensor de ângulo. A figu ra 19.2 apresenta a equivalência entre o potenciômetro e as resistências.

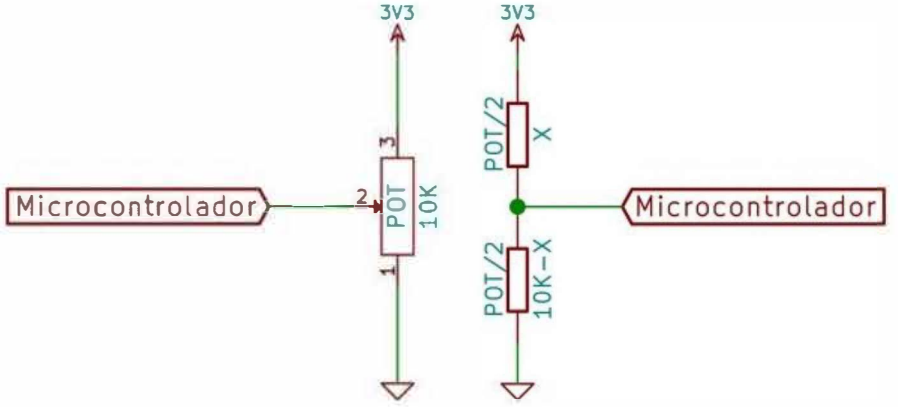


Figura 19.2: Potenciômetro como divisor de tensão

Os conversores precisam de uma entrada em valores de tensão. Para transformar um divisor resistivo numa variação de tensão basta utilizar uma fonte de tensão nos extremos dos resistores e realizar a leitura no terminal central. O valor de tensão no meio dos resistores é dado por:

$$V_{out} = \frac{V_S * R_2}{R_1 + R_2} = V_S * (\frac{R_2}{R_{Total}})$$

Se a trilha do potenciômetro for construída de modo que a variação da resistência varie linearmente em relação ao ângulo do cursor, podemos identificar a posição em que ele se encontra baseando-nos na tensão de saída. A tensão poderá variar de zero até a tensão da fonte utilizada.

Outros elementos também têm sua resistência variável, como o LDR, light dependent resistor, resistor dependente de luz ou fotorresistor (figura 19.3).

Figura 19.3: LDR/Fotorresistor - Sensor de luminosidade
Fonte: Imagem produzida com Fritzing/Inkscape

O LDR é, a princípio, uma resistência que tem seu valor alterado dependendo da quantidade de luz que incide sobre ele. Em geral, possuem valores de 100kOhms no escuro (cerca de 10 lumens), reduzindo para valores próximos de zero sob a incidência de luminosidade.

Para conseguir transformar essa variação de resistência numa variação de tensão é necessário construir uma estrutura de divisor de tensão. Neste caso, o LDR funcionará com um dos dois resistores, sendo que o segundo resistor do divisor será uma resistência fixa. Uma opção é utilizar um segundo resistor com o mesmo valor que o do LDR, de 10kOhm.

Utilizando o LDR na parte inferior do circuito, a tensão de saída irá variar de metade da tensão até o valor zero. Se utilizarmos o LDR na parte superior do divisor de tensão, o valor da saída variará da metade da fonte até o máximo. Apesar desta topologia conseguir transformar a variação da resistência numa variação de tensão, a saída não consegue passar do valor zero até o valor máximo. Isto pode ser corrigido com o uso de amplificadores. Para algumas aplicações, no entanto, isto pode não ser um problema.

Sensores ativos

Visando facilitar a utilização dos sensores, alguns fabricantes produzem chips que já fazem a conversão da grandeza a ser medida, como temperatura ou luminosidade, e entregam uma saída de tensão.

Essa conversão pode ser acompanhada de circuitos internos de condicionamento de sinal, que permitem que a saída seja linear com a grandeza medida.

Os sensores que possuem circuitos de amplificação e condicionamento do sinal, embutidos no mesmo chip, que o elemento sensor são comumente chamados de sensores ativos. Um sensor ativo possui, normalmente, 3 terminais: 2 para alimentação e um terminal para saída da informação.

Um exemplo de sensor ativo é o LM35, que é um sensor de temperatura. Ele é disponibilizado no formato TO-92, como na figura 19.4.

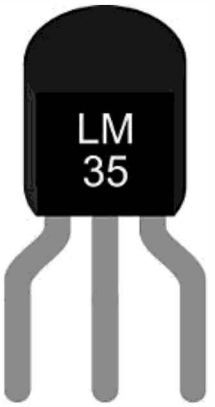


Figura 19.4: Circuito integrado LM35
Fonte: Imagem produzida com Fritzing/Inkscape

Os sensores ativos são mais caros que os elementos sensores simples, no entanto, são mais estáveis e mais simples de serem utilizados.

As características de alimentação e de saída do sensor variam de acordo com o modelo. Grande parte dos sensores ativos é alimentada com 5 volts.

A tensão de saída obedece uma fórmula, cujo valor é derivado da grandeza que está sendo medida. Em geral, esta fórmula é dada por uma equação linear, ou seja, a saída é proporcional ao valor medido. Deste modo basta medir a tensão V na saída do sensor e realizar o seguinte cálculo:

$$Valor = V \times k + c \qquad (19.1)$$

Os valores de k e c são dados pelo dispositivo usado. Para o LM35, o valor de k é $100^{o}\ C/V$ e o valor de c é zero. Desta forma, uma tensão de 1,000 volt representa uma temperatura de $100^{o}\ C$ e uma temperatura de $36{,}5^{o}\ C$ gerará uma tensão de 0,365 volts.

Sensores da placa de desenvolvimento

A placa de desenvolvimento possui três sensores conectados às suas entradas analógicas: um potenciômetro, um LDR e um LM35. Os circuitos de acionamento, bem como as portas às quais eles estão conectados, são descritos na figura 19.5:

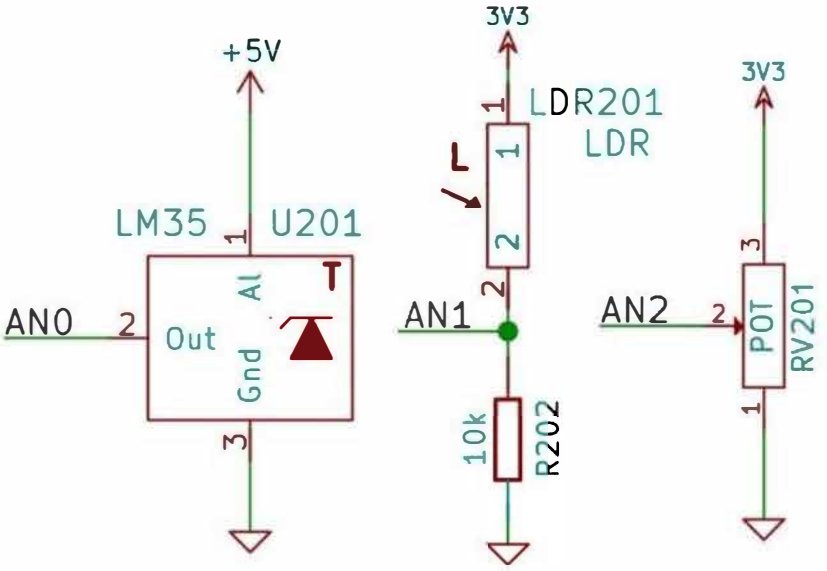


Figura 19.5: Sensores da placa de desenvolvimento

Os dois sensores baseados em sistemas de divisor de tensão por malha resistiva, o potenciômetro e o LDR, estão alimentados com 3,3 volts. Isto foi feito para permitir que estes sensores possam ser lidos pelas placas com 3,3 volts sem que suas entradas queimem.

Já o LM35 é alimentado com 5 volts. Este valor é definido pelo fabricante do sensor. Entretanto, sua saída não atinge valores maiores que 1,5 volts, pois ele não é capaz de medir temperaturas maiores que 150º *C*. Para medição de temperaturas ambientes, sua saída não deve ser maior que 500 milivolts, dado que é incomum temperaturas maiores que 50º *C*.

Quando estes sensores forem lidos nas placas cuja tensão é 5 volts, haverá uma perda de resolução, visto que o microcontrolador poderia ler tensões de até 5 volts, mas a tensão máxima é menor que esse valor.

19.2 O conversor eletrônico

Os circuitos que realizam a conversão de um sinal analógico para um sinal digital são chamados de conversores analógico-digitais, ou ADCs da sigla em inglês Analog to Digital Converter.

O modo mais simples para a converter o sinal é utilizar circuitos comparadores. Os comparadores são circuitos eletrônicos baseados em amplificadores operacionais. Estes circuitos realizam a comparação de um sinal com uma referência. A saída deste circuito é dada em dois estados. Para a aplicação desejada, podemos configurar estes circuitos para possuírem saída de zero volts, quando a tensão analisada for mais baixa que a referência ou de cinco volts, quando a tensão analisada for mais alta que a referência. Este circuito é apresentado na f igura 19.6:

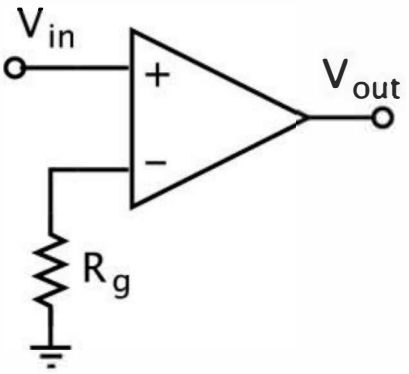


Figura 19.6: Circuito comparador utilizando amplificador operacional

O circuito dado consegue transformar o sinal de entrada, analógico, num sinal de saída digital utilizando apenas 1 bit de sinal. Se o valor for maior que a referência, a saída vale **1**, se for menor, a saída vale **0**.

Usar apenas um bit para a conversão faz com que o sinal perca muita informação. O ideal é utilizar mais bits para garantir que as informações sejam transpostas de maneira adequada do mundo analógico para o mundo digital. A quantidade de bits necessárias depende da aplicação.

No processo de conversão é preciso configurar o circuito para que cada conversor seja ligado num nível de tensão diferente. As tensões de referência são geralmente escolhidas de modo que a tensão máxima seja dividida em intervalos iguais.

Por exemplo, um conversor com quatro comparadores poderá gerar 4 resultados diferentes. Para codificar essa informação em um valor digital podem ser utilizados 2 bits. Se os 4 níveis forem separados igualmente entre os valores de 0 a 5 volts, cada intervalo será de 1,25 volts. A tabela 19.1 apresenta os valores digitais de saída, a faixa de entrada e o valor médio de tensão para cada faixa.

Tabela 19.1: Faixa de tensão dos comparadores

Representação com 2 bits	Faixa de tensão	Valor médio
00	0 .00 - 1 ,25	0 .625
01	1 ,25 - 2 ,50	1 .875
10	2 ,50 - 3 ,75	3 .125
11	3 ,75 - 5 ,00	4 .375

Durante a conversão do sinal existe perda de informação. O sinal é comparado com a faixa em que se encaixa e o sinal digital é gerado. Para o exemplo do conversor de 2 bits, se a entrada possuir uma tensão de 500 ou 1.200 milivolts, ambas serão convertidas para o valor **00**. Esse erro, que é inserido pelo processo de conversão, é chamado de erro de quantização. A figura 19.7 apresenta de modo gráfico as faixas de valores de tensão e suas saídas correspondentes.

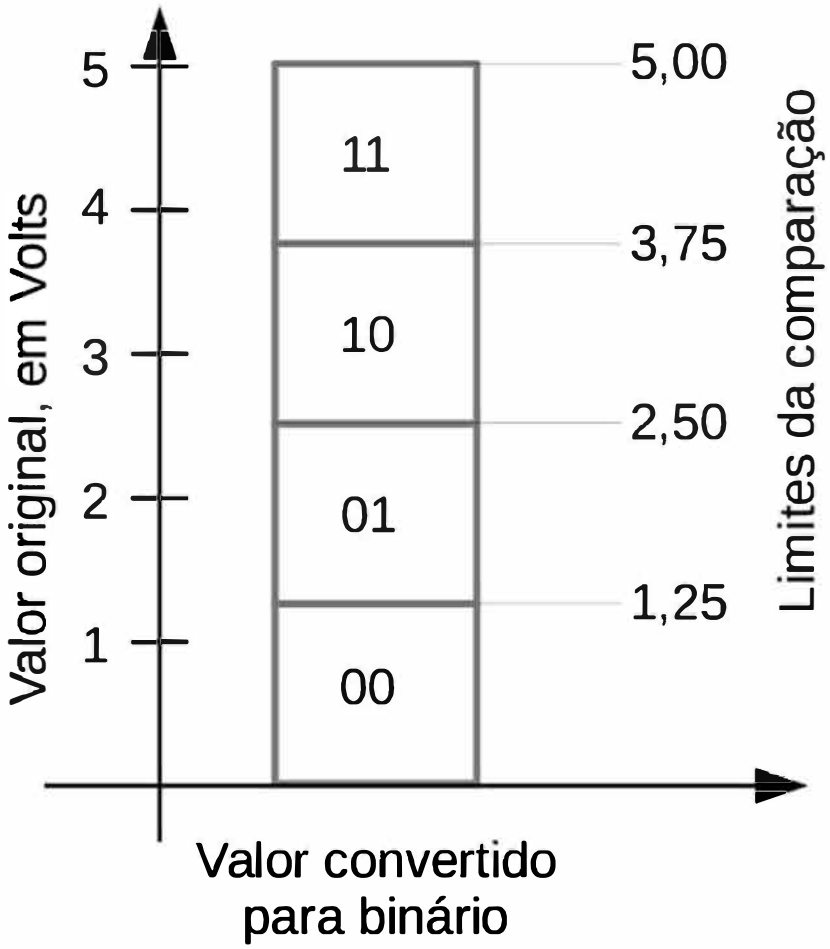


Figura 19.7: Definição de faixa de valores para AD de 2 bits

A figura 19.8 apresenta o circuito completo de um conversor analógico-digital de 2 bits. Ele é conhecido como conversor do tipo flash, onde cada nível de tensão possui seu próprio comparador. Isto faz com que o processo de conversão seja bastante rápido. No entanto, os comparadores são componentes eletrônicos relativamente caros. Para reduzir o custo dos conversores foram desenvolvidas outras abordagens que minimizam o uso de comparadores. Essas abordagens acabam inserindo um atraso no processo de conversão. O tempo de atraso depende do tipo de circuito utilizado e da quantidade de bits a ser convertida. Isto deve ser levado em conta quando for desenvolvida a rotina de programação.

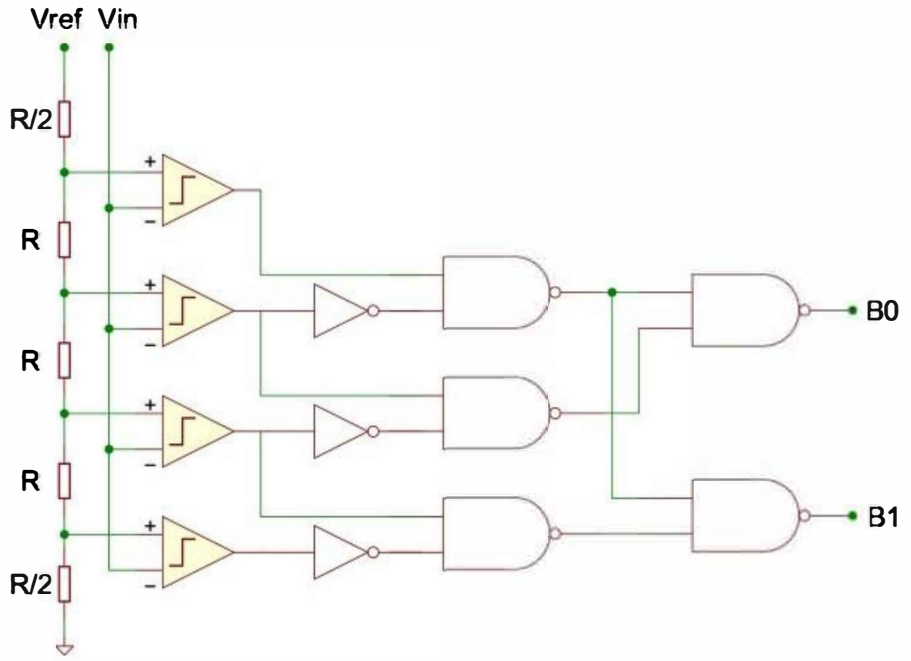


Figura 19.8: Conversor analógico-digital de 2 bits
Fonte: Jon Guerber (Modificado)

Canais e multiplexação de entradas analógicas

Os conversores são periféricos relativamente caros. Para reduzir o custo e ainda assim permitir que o desenvolvedor consiga fazer a leitura de vários sinais analógicos, é comum fazer uso de um multiplexador analógico, como na figura 19.9.

O problema com a utilização de um multiplexador é que os sinais de entrada não podem ser convertidos ao mesmo tempo. Isto faz com que o tempo para converter todas as entradas seja maior fazendo com que a velocidade de leitura dos sinais seja mais baixa. Deve-se tomar cuidado para que a velocidade seja suficiente para a aplicação desejada.

Outro problema acontece quando existe a necessidade de obtermos dois valores simultaneamente. Para se calcular a potência que está sendo consumida por um equipamento, por exemplo, é necessário realizar a leitura tanto da tensão quanto da corrente no mesmo instante. Se essas duas medidas forem tomadas em instantes de tempo distintos, os resultados obtidos não serão coerentes com a realidade.

Sistemas de controle de motores, por exemplo, exigem que essas medidas sejam corretas. Nestes casos, os fabricantes desenvolvem chips dedicados para estas aplicações, contendo dois, três ou mais conversores.

O multiplexador seleciona uma das entradas e a conecta ao conversor. A seleção é realizada através de um registro específico para cada placa utilizada.

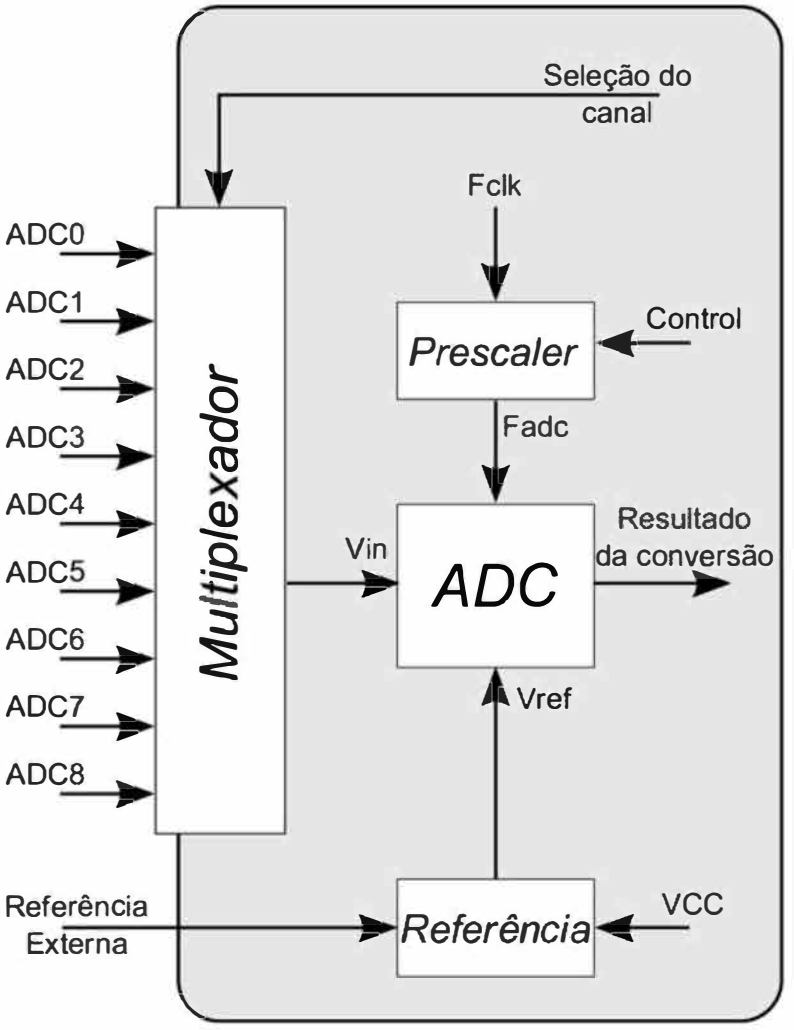


Figura 19.9: Multiplexador para seleção de canal analógico

19.3 Processo de conversão

Para aumentar a flexibilidade dos conversores, os fabricantes disponibilizam diversas características que podem ser configuradas como a quantidade de bits, a velocidade de conversão, o canal de conversão e até mesmo o modo de aproximação do conversor. Deste modo, é necessário realizar a inicialização antes de começar a conversão dos sinais. A figura 19 .10 apresenta a relação entre os terminais analógicos e os registros de memória.

Toda conversão leva um determinado tempo que depende da arquitetura que estamos utilizando, da qualidade do conversor e, algumas vezes, do valor de tensão que queremos converter. Em geral, os microcontroladores disponibilizam algum sistema que permite ao programador verificar quando a conversão terminou corretamente.

O procedimento geral para utilizar um conversor AD pode ser dado por:

1.Configurar o conversor;

2.Iniciar a conversão;

3.Monitorar o final da conversão;

4.Ler o valor no registro indicado.

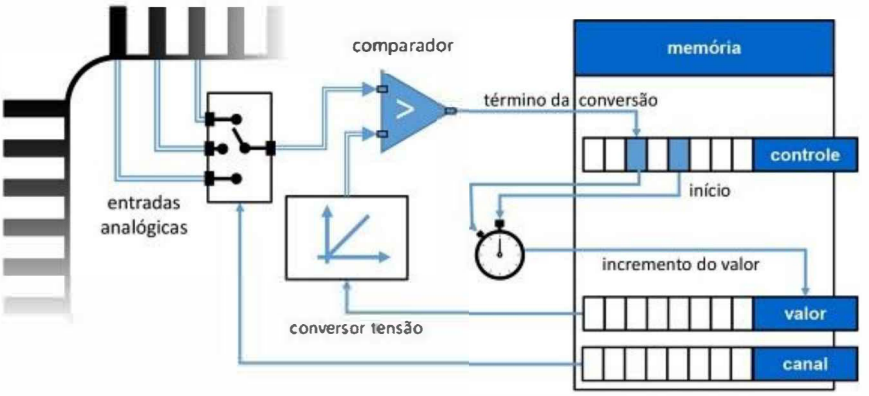


Figura 19.10: Relação entre os terminais de uma comunicação serial e os registros de memória

Criação da biblioteca

O procedimento de uso dos conversores nas três plataformas é muito similar. Para facilitar o projeto da biblioteca serão geradas apenas duas funções: uma para inicializar o conversor e uma para realizar a conversão e a leitura do sinal. A segunda função receberá como parâmetro um valor indicando qual é o canal a ser lido. Deste modo, a função deverá configurar o multiplexador, inicializar a conversão e aguardar o resultado. O código 19.1 apresenta o cabeçalho da biblioteca, que será o mesmo para as três plataformas.

Código 19.1: ad.h

```
1 #ifndef ADC_H
2 #define ADC_H
3     void adInit(void);
4     int adRead(int channel);
5 #endif
```

O código 19.2 apresenta o arquivo .c da biblioteca para conversores analógico-digital no microcontrolador Freedom. Foi utilizado o mesmo padrão de funções do framework Wiring, no entanto, acessando os registros que fazem o controle do periférico. O código 19.3 apresenta um programa exemplificando o uso da biblioteca criada.

Código 19.2: ad.c

```
#include "io.h"
#include "ad.h"
#include "derivative.h"

void adInit(void) {
    // Habilita o clock para o conversor
    SIM_SCGC6 |= SIM_SCGC6_ADC0_MASK;
    //configura o sistema de conversão: velocidade e modo de disparo
    ADC0_CFG1 = ADC_CFG1_ADIV(2) | ADC_CFG1_MODE(1) | ADC_CFG1_ADLSMP_M
        ADC_CFG1_ADICLK(1);
    //Configura terminal 8 da porta B como entrada analógica canal 11
    PORTB_BASE_PTR ->PCR[8] = (PORT_PCR_MUX(0) | PORT_PCR_DSE_MASK);
    bitSet(PTB_BASE_PTR ->PDDR, 8);
    //Configura terminal 9 da porta B como entrada analógica canal 10
    PORTB_BASE_PTR ->PCR[9] = (PORT_PCR_MUX(0) | PORT_PCR_DSE_MASK);
    bitSet(PTB_BASE_PTR ->PDDR, 9);
    //Configura terminal 8 da porta A como entrada analógica canal 3
    PORTA_BASE_PTR ->PCR[8] = (PORT_PCR_MUX(0) | PORT_PCR_DSE_MASK);
    bitSet(PTA_BASE_PTR ->PDDR, 8);
}
int adRead(int channel) {
    //Primeiro configura o canal correto
    //isso já inicializa a conversão.
    if (channel == 0) {
        ADC0_SC1A = 11;
    }
    if (channel == 1) {
        ADC0_SC1A = 10;
    }
    if (channel == 2) {
        ADC0_SC1A  = 3;
    }
    //aguarda a conversão
    while ((ADC0_SC1A & ADC_SC1_COCO_MASK) == 0);
    //retorna o valor convertido
    return ADC0_RA;
}
```

Código 19.3: Exemplo de uso da biblioteca de conversores AD

```
 1 #include "io.h
 2 #include "ssd.h"
 3 #include "ad.h"
 4 //início do programa
 5 void main(void){
 6     float time;
 7     int value = 0;
 8     systemInit();
 9     adInit();
10     ssdInit();
11     for(;;){
12         //0 – Temperatura
13         //1 – Luminosidade
14         //2 – Potênciometro
15         value = adRead(0);
16         ssdDigit((value / 1000) %10,0);
17         ssdDigit((value /  100) %10,1);
18         ssdDigit((value /   10) %10,2);
19         ssdDigit((value       ) %10,3);
20         ssdUpdate();
21         for(time = 0; time < 1000; time++);
22     }
23 }
```

19.4 Aplicação

O conversor analógico para digital pode ser demonstrado em uma aplicação simples de controle de nível de um reservatório através de uma boia ligada a um potenciômetro. A altura da boia dentro do reservatório faz com que o eixo do potenciômetro se desloque mudando sua resistência, respectivamente a sua tensão de saída, que seria ligada à entrada do conversor analógico para digital. Cada terminal da extremidade do potenciômetro seria ligado a cada polo de alimentação, terra e 3,3 volts. Sua saída varia conforme sua posição: reservatório cheio em 3,3 volts e vazio com terra na saída.

Esta aplicação tem como saída um LED RGB, onde cada cor indicaria uma região no eixo do potenciômetro: vermelho para reservatório vazio, verde para metade do volume do reservatório e azul para cheio.

A conversão utilizada ainda pode sofrer variações conforme a placa utilizada nesta

aplicação. A tensão de referência do conversor pode ser 3,3 ou 5 volts, seguindo a própria tensão de alimentação do microcontrolador. A resolução pode variar entre 10 bits (0 a 1023), como o AtMega328 e o PIC32MX320F128, e 12bits (0 a 4095), como o MKL05Z32VFM4.

O programa desta aplicação, pode ser verificado no código 19.4, que executa uma rotina simples de teste da entrada analógica AN0, onde é coletado o sinal do potenciômetro de 0 a 3,3 volts. Três estruturas de decisão, escolhem qual a situação do nível de tensão convertido para digital em que a entrada se encontra: um terço do valor máximo, entre um terço e dois terços, e, finalmente, acima de dois terços do valor máximo. Acendendo o tom vermelho, verde e azul, de acordo com os níveis encontrados: baixo, médio e alto.

Código 19.4: Aplicação de controle de nível exibido em LED RGB

```
 1 #include "io.h"
 2 #include "ad.h"
 3 #include "rgb.h
 4
 5 //Valor máximo do potenciômetro
 6 int MaxPot = 675;     //ATMega238 5V
 7 //int MaxPot = 1023; //PIC32MX320F128 3.3V
 8 //int MaxPot = 4095; //KL05z32VF4 3.3V
 9
10 void main(void){
11     int adValue;
12     systemInit();
13     rgbInit();
14     adInit();
15     for(;;){
16         //Leitura do potenciômetro
17         adValue = analogRead(AN0);
18         //Nível baixo (frio)
19         if (adValue < (MaxPot/3)){
20             rgbColor(BLUE);
21         }
22         //Nível medio (ok)
23         else if (adValue < ((MaxPot*2)/3)){
24             rgbColor(GREEN);
25         }
26         //Nível alto (quente)
27         else{
28             rgbColor(RED);
29         }
30     }
31 }
```

19.5 Exercícios

Ex. 19.1 — Construa um programa que envie o valor da tensão capturada na entrada **AN0** para o display LCD.

Ex. 19.2 — Utilizando a entrada **AN1**, determine a intensidade luminosa no LDR e escreva um valor de 0 a 99 no display de 7 segmentos, que corresponda ao valor mínimo e máximo do LDR.

Ex. 19.3 — Monte um programa para comparar o valor das entradas **AN0** e **AN1**. Se **AN0** for menor que **AN1**, o led RGB deve ficar azul. Quando for maior deve ficar verde. Se forem exatamente iguais, deve ficar vermelho.

Ex. 19.4 — Os sensores analógicos podem apresentar vários problemas com ruídos. Alguns destes problemas podem ser resolvidos com filtros digitais. Um modelo de filtro digital é o filtro de média móvel. Este filtro é implementado a partir de uma média ponderada entre o valor atual do filtro com seus valores antigos. Crie um programa que realize e armazene as últimas 5 leituras de um sinal analógico. De posse dessas leituras, ele deve aplicar o seguinte filtro e exibir o resultado no display de LCD.

$$V_{filtrado} = (V_0 + V_{-1} + V_{-2} + V_{-3} + V_{-4})/5$$

Onde V_{-n} representa a enésima amostra anterior.

CAPÍTULO

20

Saídas PWM

20.1 Conversor digital-analógico usando um PWM

20.2 Soft PWM

20.3 O periférico do PWM

20.4 Criação da biblioteca

20.5 Aplicações

- Servomotores
- Controle da frequência e emissão de sons

20.6 Exercícios

"Quando uma bobina é operada com correntes de frequência muito elevadas, belos efeitos podem ser produzidos na escova, mesmo que a bobina tenha dimensões relativamente pequenas. O experimentador pode variá-los de muitas maneiras e, mesmo que não fossem nada mais, eles oferecem uma visão agradável."

Nikola Tesla

As saídas do tipo PWM, *pulse width modulation*, são saídas digitais que possuem um sistema de chaveamento acoplado. Estas saídas possuem um nível de tensão que fica alterando entre nível alto (5 ou 3,3 volts) e nível baixo (zero volts) várias vezes por segundo, num formato conhecido como onda quadrada.

Neste tipo de saída é comum que a frequência de trocas de níveis digitais seja fixa, de modo que o sinal se repita em intervalos constantes de tempo.

A característica mais marcante das saídas PWM é, mantendo a frequência constante, conseguir alterar o tempo em que o sinal permanece com nível alto. A razão entre o tempo que o sinal permanece no nível alto sobre o tempo de repetição do ciclo é conhecido como ciclo de trabalho ou *duty cycle*. Em geral apresentamos esse valor como uma porcentagem.

A figura 20.1 apresenta 3 sinais PWM com a mesma frequência, mas com *duty cycles* diferentes:

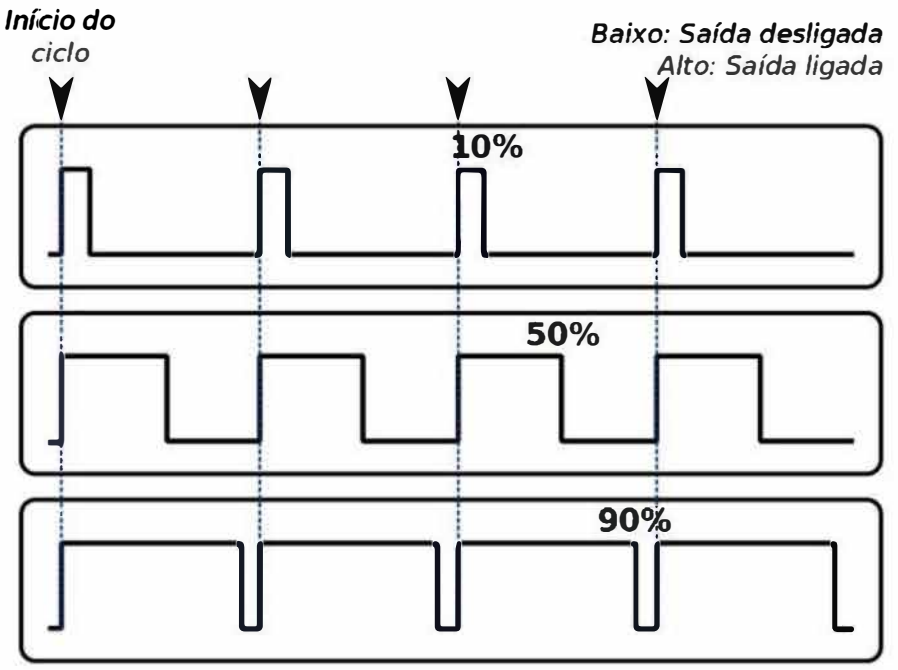


Figura 20.1: Sinais PWM com variação do duty cycle

A grande vantagem de se utilizar uma saída PWM é que, dependendo do sistema que está sendo controlado, ela pode funcionar como uma saída analógica.

Supondo uma saída PWM ligada a um resistor como na figura 20.2. Quando a saída estiver em nível alto, ocorre a passagem de uma corrente elétrica e, consequentemente, a resistência libera calor para o ambiente.

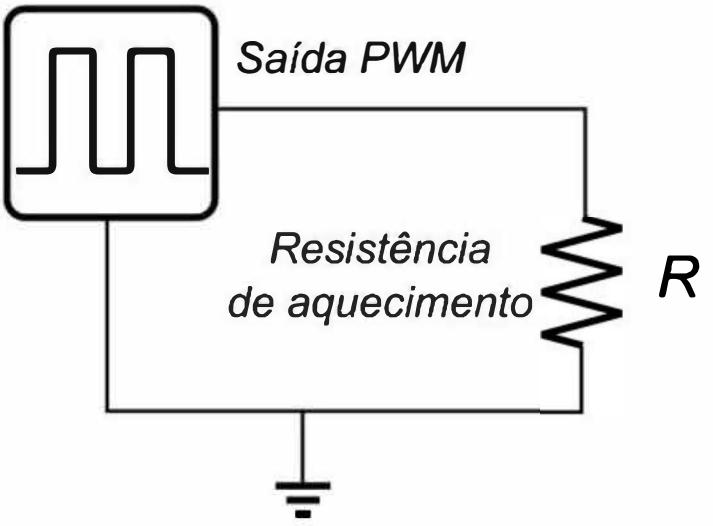


Figura 20.2: Resistência R sendo controlada por saída PWM

A quantidade de calor liberada depende do valor da resistência, da intensidade da corrente e do tempo que o sistema estiver ligado. Como a corrente depende apenas do valor

da resistência e da tensão de alimentação, podemos calcular a quantidade de calor em joules pela fórmula seguinte:

$$Q_{alto} = \frac{V^2}{R} \times t_{alto} \qquad (20.1)$$

Onde V é a tensão do nível alto da saída PWM, R é o valor da resistência e t_{alto} é o tempo em que o sinal ficou no nível alto.

Quando a saída PWM está em nível baixo, não há passagem de corrente e, consequentemente, não há liberação de calor.

Durante um ciclo completo da saída PWM, a resistência passa um tempo t_{alto} ligada e um tempo t_{baixo} desligada. O tempo total é o próprio tempo de duração do ciclo do PWM t_{pwm}. A quantidade máxima de calor que pode ser liberada durante o ciclo é atingida quando o resistor passa todo o tempo ligado, ou seja, $t_{alto} = t_{pwm}$, fazendo com que t_{baixo} seja igual, a zero.

Como a saída PWM permite controlar o tempo que o resistor permanece ligado, podemos controlar, de modo bastante simples, a quantidade de calor que será gerada pelo resistor. Deste modo, através de uma saída digital é possível controlar uma grandeza física de modo que seu valor seja variável. Esta estrutura permite utilizar a saída PWM como uma saída analógica.

No entanto, para que isso aconteça, é necessário que o ciclo de trabalho seja rápido o suficiente. Ligar o resistor durante meia hora e o desligar durante meia hora fará com que o ambiente receba apenas metade do calor máximo. O problema é que durante a primeira meia hora o ambiente será aquecido e durante a segunda hora terá sua temperatura reduzida. Para que a saída consiga controlar a temperatura, é interessante ligar o resistor durante alguns segundos e desligá-lo durante outros segundos. Repetindo esse procedimento de modo rápido, a impressão, para o usuário, é que o resistor está configurado para gerar apenas metade da energia que ele consegue.

O algoritmo para gerar um sinal do tipo PWM é bastante simples:

1. Inicialização do terminal de saída;
2. Escolha do tempo de ciclo;
3. Escolha do *duty cycle*;
4. Liga-se a saída e inicia-se contagem de tempo;
5. Quando o tempo for maior que o *duty cycle* desliga a saída;
6. Quando o tempo acabar retorna ao passo 4 .

A figura 20.3 apresenta um modelo de como as informações de tempo de um relógio interno podem ser utilizadas para gerar o sinal de PWM.

No modelo existem dois registros, o primeiro configura o tempo máximo de contagem, após o qual o relógio é reiniciado ou resetado. Este registro é que define a frequência do PWM. O segundo registro deve possuir um valor de, no mínimo, zero e de, no máximo, igual ao primeiro registro. Este valor é que será utilizado para definir quanto tempo o sinal permanecerá ligado e quanto tempo permanecerá desligado.

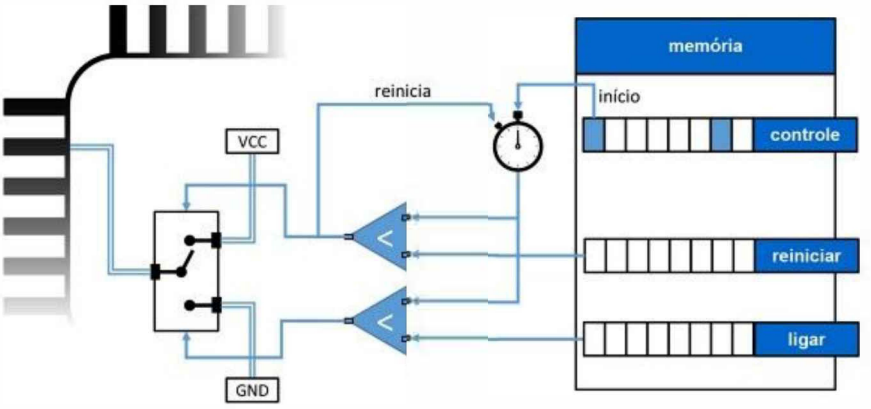


Figura 20.3: Relação entre os terminais de uma saída PWM e os registros de memória

20.1 Conversor digital-analógico usando um PWM

Para transformar a saída PWM numa saída analógica é preciso que a frequência de operação seja muito superior às constantes de tempo do sistema físico. Sistemas térmicos, em geral, possuem constantes de tempo bastante lentas, mas sistemas óticos ou eletrônicos, no entanto, são bem mais rápidos.

Dependendo do circuito a ser acionado é interessante já entregar o sinal de modo analógico, transformando o sinal quadrado num valor constante e proporcional ao tempo que o PWM está ligado. Para isso, podemos fazer uso de circuitos de filtro do tipo passa-baixas.

O filtro passa-baixa visa eliminar frequências altas. Por definição, o sinal do tipo PWM possui uma frequência fixa. Por se tratar de uma onda quadrada, outras frequências aparecem misturadas a esse sinal, no entanto todas elas são maiores que a frequência base. Utilizando o filtro passa-baixas é possível remover a frequência do PWM e o que sobra é apenas o valor de tensão média.

O exemplo de filtro passa-baixas mais simples é a utilização de um resistor em série com um capacitor. A constante de tempo desse circuito é dada pela multiplicação entre o valor da resistência em Ohms pelo valor da capacitância em Faradays: $t = R \times C$. É importante que esse valor seja menor que o ciclo do PWM. Quanto maior a diferença menos ruído haverá no sinal filtrado. Um valor 10 vezes menor já apresenta um bom resultado. A figura 20.4 apresenta o circuito para a conversão da saída PWM num sinal analógico.

É importante notar que transformar a saída PWM numa analógica com uma rede RC, gera alguns problemas. O primeiro é com relação a velocidade de alteração do sinal. Como o PWM é um sistema baseado numa frequência, é impossível mudar o valor da saída analógica mais rápido que a velocidade de mudança do PWM.

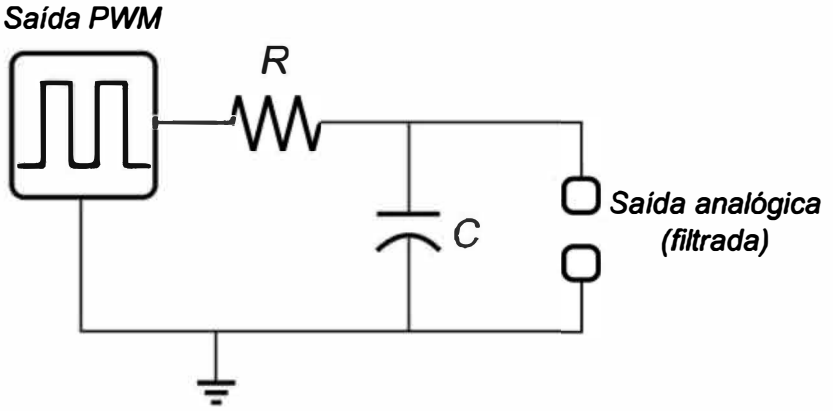


Figura 20.4: Utilização de filtro RC em saída do tipo PWM

O segundo problema é com relação ao acionamento de cargas. A saída filtrada não tem capacidade de acionar cargas mantendo a tensão. Qualquer corrente enviada para a saída causará uma queda de tensão no resistor R. Deste modo, é indicado a utilização de um buffer para o acionamento de cargas.

A figura 20.5 apresenta os sinais de um PWM antes, canal 1 sinal mais alto, e depois, canal 2 sinal mais baixo, do filtro RC com diferentes *duty cycles*. Nestes exemplos foi utilizado um circuito RC com um tempo cem vezes menor que a frequência do PWM, por isso quase não há oscilação no sinal de saída.

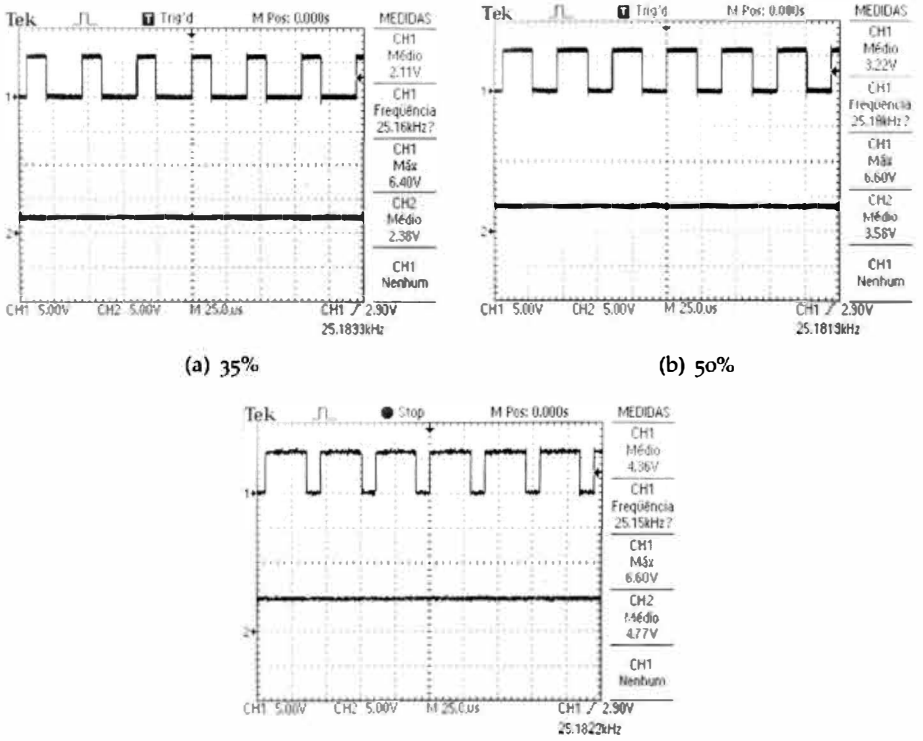


Figura 20.5: Sinais analógicos gerados a partir de uma saída PWM com filtro passa-baixas

Existem circuitos de filtros passa-baixas mais robustos, que permitem trabalhar com frequências mais altas. Em geral, estes circuitos são ativos e fazem uso de amplificadores operacionais, aumentando o custo do projeto, mas permitindo um uso mais amplo e simples de saídas analógicas derivadas de PWM.

20.2 Soft PWM

É possível gerar uma saída do tipo PWM por software, sendo necessário apenas a disponibilidade de uma porta digital. Isto reduz o custo, mas insere uma sobrecarga considerável no processamento.

A implementação mais simples é através de um loop **for**, como no código 20.1.

Código 20.1: Soft PWM

```
 1 #include "pwm.h"
 2 #include "io.h"
 3 #include "rgb.h"
 4
 5 #define PWM_CYCLE_TIME    1000
 6 #define DUTY_CYCLE         100
 7
 8 void main(void){
 9     int t;
10     systemInit();
11     rgbInit();
12     for(;;){
13         rgbColor(OFF);
14         for(t=0;t<PWM_CYCLE_TIME;t++){
15             if(t==DUTY_CYCLE){
16                 rgbColor(RED);
17             }
18         }
19     }
20 }
```

A frequência do PWM depende da velocidade com que o microcontrolador executa o loop de contagem da variável **t**. Se este loop for executado a cada 1µs, para a contagem de t até **1000**, teremos um tempo de ciclo de 1ms, o que representa uma frequência de 1kHz. Diminuir o valor de **PWM_CYCLE_TIME**, fará com que a frequência aumente, mas a resolução, ou capacidade ajuste fino da saída, será menor. Neste exemplo, é possível ajustar a saída em

múltiplos de 0,1%. Reduzindo **PWM_CYCLE_TIME** para **100** fará com que a frequência aumente para 10kHz mas a resolução caia para múltiplos de 1%.

O código possui, portanto, uma restrição que é baseada principalmente na capacidade de processamento do microcontrolador utilizado. O micro pode não ter capacidade de processamento suficiente para gerar um sinal com a frequência necessária para a aplicação desejada. Lembrando que, para acionar cargas de modo similar a uma saída analógica, o PWM tem que possuir uma frequência alta.

Além disso, a adição de outras funcionalidades neste código pode fazer com que a frequência caia ainda mais, pois as atividades devem ser adicionadas dentro do loop de contagem da variável **t**. Qualquer atividade adicionada depois do loop aumentará o tempo que o PWM está desligado, fazendo com que ele nunca atinja o valor de 100%, reduzindo a possibilidade de uso dessa técnica.

A melhor solução nestes casos é utilizar um periférico dedicado que consiga implementar esta funcionalidade em hardware, liberando o processador para outras atividades.

20.3 O periférico do PWM

Em geral, o funcionamento dos periféricos de PWM segue uma mesma estrutura para sua configuração.

1.Configurar os terminais físicos como saída
2.Configurar o clock do periférico
3.Escolher a velocidade do ciclo do PWM
4.Habilitar o funcionamento do periférico

Como todos os demais periféricos que precisam de uma base de tempo, o PWM pode escolher entre diferentes fontes de clock do sistema. O mais comum é utilizar a mesma fonte de clock do oscilador principal.

Em geral, a fonte de clock do oscilador principal é bastante alta, cerca de algumas dezenas de MHz. Existem opções que permitem reduzir essa velocidade. Esses dispositivos são os divisores de frequência, conhecidos como prescallers.

O funcionamento básico do PWM, como apresentado no modelo do soft PWM, consiste em executar um ciclo num tempo T determinado. Durante esse tempo podemos escolher uma fração do tempo T, durante o qual uma saída permanecerá ligada. No restante do tempo a saída ficará desligada. Um dos modos de se fazer isso é utilizar um relógio que será incrementado de acordo com o clock escolhido. Esse valor será comparado constantemente com dois outros valores: um para ligar a saída e outro para desligar.

Quando a saída for desligada, o contador de tempo será resetado e o ciclo se reiniciará.

Cada plataforma apresenta sua especificidade. Vamos apresentar o exemplo com a Freedom. O cálculo da frequência de trabalho do PWM é dada por:

$$Freq._{PWM} = \frac{F_{OSC}}{resetCounter \times (PWM_{Prescaler})}$$

Onde: F_{OSC} é a frequência de oscilação da placa (24 MHz), $PWM_{Prescaler}$ é o divisor utilizado pelo PWM (1 :1) e *resetCounter* é o valor utilizado para reinicializar o contador de tempo.

O duty cycle (em porcentagem) é calculado de acordo com a fórmula abaixo:

$$DutyCycle_{PWM} = \frac{matchValue}{resetValue} \times 100$$

Onde: *matchValue* é o valor, entre zero e *resetValue*, para o qual a saída será levada para **1**, quando este for igual ao contador de tempo. Quando o contador de tempo se igualar ao valor de *resetValue*, o próprio contador é reinicializado e a saída retorna para **0**.

20.4 Criação da biblioteca

Para configurar as saídas PWM devemos especificar a frequência de trabalho através do registro **TPM0_MOD**. No registro **PORTB_PCR11** o terminal D9 é configurado como uma saída do tipo PWM. O prescaler foi desabilitado, de modo a obter a maior frequência de trabalho possível.

No código 20.3 é apresentado um exemplo de como criar as rotinas de operação do PWM. O header desta biblioteca é apresentado no código 20.2. Por fim, o código 20.4 apresenta um exemplo de utilização desta biblioteca.

Código 20.2: pwm.h

```
1 #ifndef PWM_H
2 #define PWM_H
3
4     void pwmInit(void);
5     void pwmBuzzer(unsigned int frequency);
6     void pwmFrequency(unsigned int frequency);
7     void pwmDutyCycle(float percentage);
8
9 #endif //PWM_H
```

Código 20.3: pwm.c

```
#include "pwm.h"
#include "io.h"
#include "derivative.h"
//liga o buzzer com uma frequência de som
//duty cycle configurado em 50%
void pwmBuzzer(unsigned int frequency){
    int period;
    //para prescaler = 1/1 (sem prescaler)
    if ((frequency>0) && (frequency < 40000)){
        period = 24000000/frequency;
        TPM0_MOD = period;
        TPM0_C0V = (period/2);
    }
```

```
14 }
15 //define uma frequência de trabalho
16 void pwmFrequency(unsigned int frequency){
17     //para prescaler = 1/1 (sem prescaler)
18     if ((frequency>0) && (frequency < 40000)){
19         TPM0_MOD = 24000000/frequency;
20     }
21 }
22 //configura a saída como um valor de 0 à 100%
23 void pwmDutyCycle(float percentage){
24     if ((percentage>=0) && (percentage <= 100)){
25         TPM0_MOD = (unsigned int)((percentage*TPM0_MOD)/100);
26     }
27 }
28 void pwmInit(void){
29     //Habilita as saídas dos terminais
30     SIM_SCGC5 |= SIM_SCGC5_PORTB_MASK | SIM_SCGC5_PORTA_MASK;
31     //Habilita o clock para o periférico do PWM
32     SIM_SCGC6|=( SIM_SCGC6_TPM0_MASK | SIM_SCGC6_TPM1_MASK);
33     //escolhe a fonte de clock do periférico como o oscilador principal do sistema
34     SIM_SOPT2 |= SIM_SOPT2_TPMSRC(1); //
35     //configura o terminal D9 (portb 11) como uma saída do tipo PWM
36     PORTB_PCR11  = (0|PORT_PCR_MUX(2));
37     //configura o valor máximo para o contador de tempo
38     TPM0_MOD  = 1000;
39     //Configura timer para acioanr o PWM, ligando quando a compararção for ←
           verdadeira e desligando quando resetar o contador
40     TPM0_C0SC = TPM_CnSC_MSB_MASK | TPM_CnSC_ELSA_MASK;
41     //configura o registro de contador para aumentar a cada incremento do timer, ←
           sem prescaler(1/1)
42     TPM0_SC   = TPM_SC_CMOD(1) | TPM_SC_PS(0);
43 }
```

Código 20.4: Exemplo de uso da biblioteca das saídas PWM

```
 1 #include "io.h"
 2 #include "pwm.h"
 3 #include "adc.h"
 4 //início do programa
 5 void main(void){
 6     int freq;
 7     systemInit();
 8     adInit();
 9     pwmInit();
10     for(;;) {
11         //lê o valor do potênciometro
12         freq = adRead(2);
13         //ajustando a frequência de acordo com entrada analógica
14         pwmBuzzer(temp);
15     }
16 }
```

20.5 Aplicações

Servomotores

Um servomotor (figura 20.6) é um sistema composto basicamente de três elementos: uma unidade motora, uma unidade sensora (posição, velocidade ou aceleração) e uma unidade de controle.

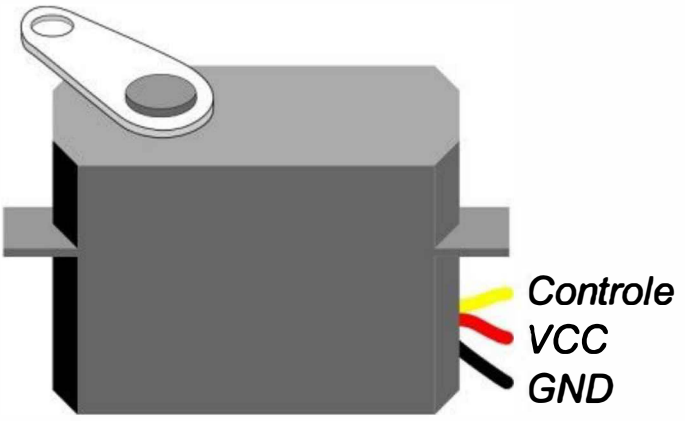


Figura 20.6: Servomotor
Fonte: Imagem produzida com Fritzing/Inkscape

O servomotor tem como objetivo estabilizar uma das grandezas do motor, geralmente o ângulo de seu eixo ou a velocidade de rotação. Para isso, ele realiza uma medida através do sensor e, se a grandeza medida, posição ou velocidade, estiver errada, ele aciona uma saída para corrigi-la.

Existem alguns modos de comunicação entre os servomotores e os microcontroladores. Um dos mais comuns é através de um sinal de tensão similar a uma saída PWM.

Esse sinal possui um período de 20ms (50Hz), onde o valor do tempo alto pode variar de 1 a 2ms. Para um servomotor com controle de posicionamento e abertura de 180 graus, seu controle varia conforme a figura 20.7.

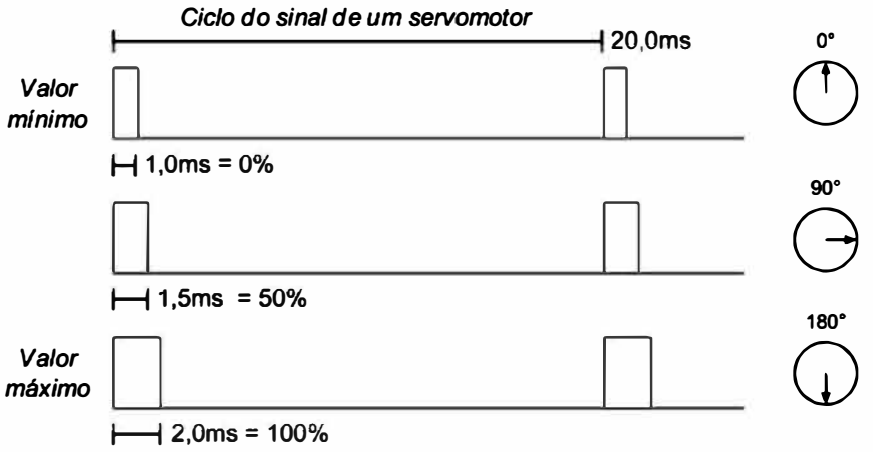


Figura 20.7: Forma de onda de controle de um servomotor

Para fazer o controle precisamos então utilizar valores entre 1ms/20ms = 5% e 2ms/20ms= 10%. Para a Freedom, o valor de leitura do potênciometro através do conversor analógico/digital é de 0 à 4095. No código 20.5 é apresentado o programa que faz a conversão e o controle de um servomotor a partir do ângulo do potênciometro.

Código 20.5: Controle do ângulo de um servomotor pelo potenciômetro

```
#include "io.h"
#include "pwm.h"
#include "adc.h"
//início do programa
void main(void){
    float ang;
    systemInit();
    adInit();
    pwmInit();

    //configurando o valor da frequencia de trabalho do pwm para 50Hz.
    pwmFreq(50);
    for(;;) {
        //lê o valor do potênciometro
        //o potênciometro varia de 0 à 4095 na freedom
        ang = adRead(2);

        //convertendo para um valor entre 0 e 1
        ang = (ang/4095);

        //convertendo para um valor entre 5 e 10
        ang = (ang+1)*5;

        //ajustando o valor do ângulo de acordo com o potênciometro
        pwmDuty(ang);
    }
}
```

Controle da frequência e emissão de sons

É possível utilizar o PWM para controlar a frequência de um sinal, ajustando-a conforme a necessidade do projeto. Uma aplicação muito comum é ajustar a frequência base de modo que ela coincida com os valores das notas musicais. Ligando esta saída PWM a um sistema de geração de som, como um alto-falante ou piezelétrico, é possível reproduzir as próprias notas musicais.

O piezoelétrico é um cristal que possui a propriedade de gerar uma tensão elétrica quando pressionado. O inverso também é possível: na presença de uma tensão elétrica ele modifica seu tamanho. Inserido um sinal de tensão que varia ao longo do tempo, é possível fazer com que o cristal vibre na mesma frequência do sinal. Conectando esse cristal a um diafragma podemos gerar ondas sonoras com a frequência desejada. O conjunto de cristal + diafragma é conhecido como *buzzer*.

O nome buzzer vem da característica do som produzido, parecido com um zunido (“bzzzzzz”). Este é o tipo de som utilizado em grande parte dos primeiros videogames ou nos toques de celulares conhecidos como monofônicos.

Como as saídas do microcontrolador podem não possuir capacidade de corrente suficiente para acionar um buzzer, é comum utilizar um circuito de amplificação transistorizado. Como o sinal do PWM é uma onda quadrada, utilizar um o transistor operando como chave é suficiente para que o buzzer funcione. A figura 20.8 apresenta o circuito utilizado na placa de desenvolvimento.

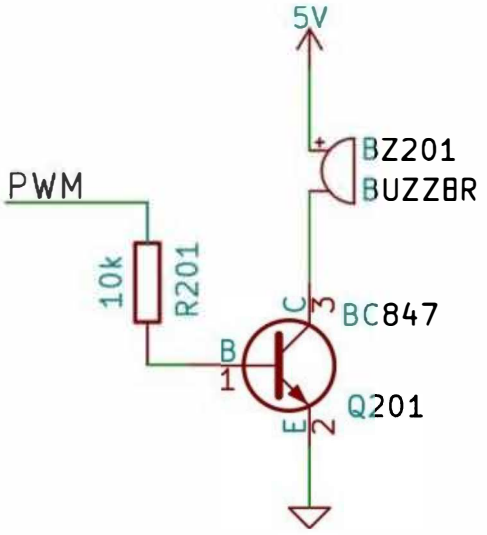


Figura 20.8: Circuito transistorizado de acionamento de Buzzer

Do ponto de vista do software é necessário criar as definições das frequências das notas musicais. O código 20.6 apresenta estas definições.

Código 20.6: Reprodução de sons

```
1 //frequência das notas musicais
2 #define C  523
3 #define CS 554
4 #define D  587
5 #define DS 622
6 #define E  659
7 #define F  698
8 #define FS 740
9 #define G  784
10 #define GS 830
11 #define A  880
12 #define AS 932
13 #define B  987
14 //próximas oitavas são múltiplos das frequencias base por exemplo:
15 #define C2 C*2
```

Para inserir um período de silêncio existem duas opções: desligar o PWM ou gerar uma frequência inaudível, acima de 20kHz. A segunda opção é mais simples por não precisar desligar/ligar constantemente o PWM.

20.6 Exercícios

Ex. 20.1 — Crie uma biblioteca "alarmeSonoro" que utiliza a biblioteca "pwm" como base. Esta biblioteca deve ter 3 funções. A primeira, que não recebe nem retorna nada, faz a inicialização dos periféricos. A segunda gera um alarme de tom (frequência) único. Ela recebe dois parâmetros: um para frequência do som e um indicando a duração do alarme. A terceira função também gera um alarme, mas com dois tons diferentes (similar ao som de uma ambulância). Essa função recebe 3 parâmetros: a duração do sinal de alarme, a frequência grave e a frequência aguda.

Ex. 20.2 — Num deteminado sistema embarcado existe um ventilador com motor DC conectado a uma saída PWM. Faça um programa que controle a velocidade desse ventilador através da comunicação serial. Se chegar um caractere numérico pela serial, ele deve ser usado para ajustar o PWM com o valor adequado: caso chegue o caractere '0', a potência deve ser regulada para 0%, para o caractere '1', a potência deve ser 10%, seguindo-se essa sequência até o caractere '9' para 90%. Utilize as bibliotecas "pwm.h" e "serial.h". Para o ventilador funcionar corretamente a frequência do PWM deve estar entre 10 e 20 kHz.

Ex. 20.3 — Um determinado sistema embarcado foi desenvolvido para manter uma mesa de controle nivelada numa embarcação. A mesa possui um acelerômetro como sensor de angulação e um servomotor como atuador. O acelerômetro retorna valores de -180 a 180 graus, com uma faixa de valores de 0 a 4095 no AD. O servomotor está montado de forma que consiga movimentar a angulação da mesa de -180 a 180 graus, com uma forma de onda PWM de 1ms a 2ms em 20ms, com dutycycle de 5 a 10%. Faça um programa que leia o valor do

acelerômetro através do canal **`AN0`** e controle o servomotor com um ângulo oposto, de modo a nivelar a mesa. Utilize as bibliotecas “ad.h” e “pwm.h”.

Ex. 20.4 — Vários controles remotos utilizam um protocolo de comunicação sem fio baseado em leds e sensores infravermelhos. A transmissão dos dados é feita através de pulsos de luz, geralmente configurados para frequências entre 30 e 60 kHz, sendo a mais comum a de 38kHz. Os bits zero ou um são transmitidos de acordo com o tempo que o sistema fica ativo. Os valores corretos de tempo dependem do protocolo de cada dispositivo. Monte um programa que faça a transmissão dos 8 bits de uma variável **`unsigned char`**. A transmissão de um bit zero é feita desligando a saída PWM por 1ms e ligando a saída PWM por 1ms, com a frequência de 38kHz. A transmissão de um bit 1 é feita desligando a saída por 1ms e ligando por 2ms. Utilize a biblioteca “pwm.h”.

CAPÍTULO

21

Temporizadores

21.1 Criação da biblioteca

21.2 Aplicação

- Geração de uma base de tempo
- Contador de frequência de eventos
- Relógio/Calendário
- Reprodução de melodias

21.3 Exercícios

"A única razão para o tempo é para que tudo não aconteça de uma só vez."

Albert Einstein

Um dos requisitos mais importantes para o desenvolvimento de sistemas embarcados é a capacidade de executar eventos em tempos pré-definidos. Estas requisições vêm das restrições nos acionamentos de dispositivos eletrônicos, para evitar o flicker, por exemplo, às características de resposta de protocolos de comunicação ou a questões de agendamento de atividades. Em qualquer uma destas situações é necessário possuir uma base de tempo confiável.

O processo de contagem de tempo em microcontroladores pode ser implementado de modo bastante simples. Basta utilizar uma variável que será incrementada em intervalos regulares. O valor desta variável passa então a indicar quanto tempo decorreu desde a última vez em que ela foi zerada. O código a seguir apresenta um exemplo deste procedimento. A base de tempo para a geração dos intervalos regulares é implementada pelo último loop **`for`**, fazendo com que cada ciclo do loop consuma exatamente 1ms.

```
 1 #include "lcd.h"
 2 #include "io.h
 3
 4 void main (void){
 5     //time armazena quanto tempo o sistema está ligado
 6     int time=0;
 7     int dummyCount;
 8
 9     lcdInit();
10     systemInit();
11
12     for(;;){
13         //incrementa a contagem de tempo
14         time++;
15
16         //executa as atividades
17         DoStuff();
18
19         //imprime a hora atual
20         timePrint(time);
21
22         //tempo extra para acertar o loop de 1ms
23         for(dummyCount=0;dummyCount<134;dummyCount--);
24     }
25 }
```

O problema com essa abordagem é encontrar o valor correto para o loop de modo que o tempo seja constante. Além disso, a função **DoStuff()** pode consumir mais ou menos tempo, dependendo das estruturas de condição utilizadas internamente, fazendo com que seja impossível garantir que todos os ciclos serão idênticos.

Para gerar uma base de tempo confiável os microcontroladores possuem um circuito dedicado para realizar contagem. Quando este circuito é alimentado por um sistema temporizado, ou clock, a contagem obedece a uma constante de tempo. A junção do circuito de contagem com o sistema de clock é chamada de temporizador.

Os temporizadores, ou timers, possuem uma estrutura bastante simples para serem utilizados. Em geral, basta inicializar o contador e aguardar que a contagem atinja o valor desejado. A figura 21.1 apresenta um modelo desta tipo de periférico.

A maioria dos temporizadores utilizam a mesma frequência de clock que o processador. Para a maioria das aplicações esta frequência pode ser muito alta. A solução adotada, por grande parte dos fabricantes, é utilizar um circuito de preescaler para reduzir a frequência, fazendo com que o timer rode mais devagar.

Como a contagem é feita através de um circuito dedicado ela não é afetada por nenhuma outra operação do processador.

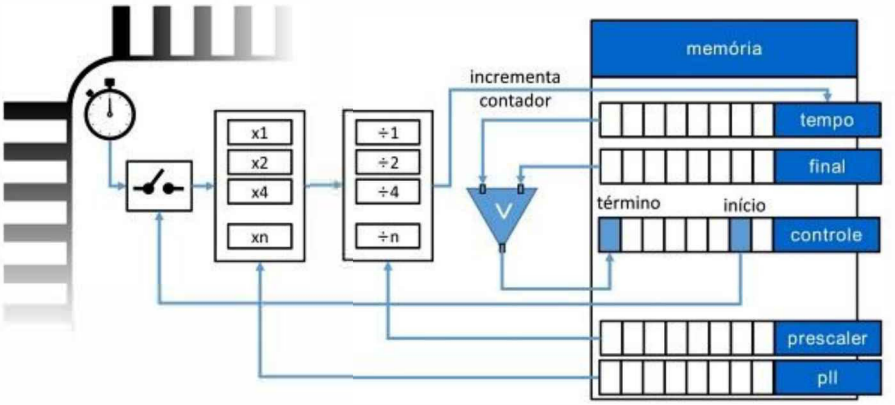


Figura 21.1: Modelo de temporizador por hardware

21.1 Criação da biblioteca

A utilização mais simples de um timer pode ser feita em três funções: uma para inicializar o periférico, **`timerInit()`**, uma para configurar o tempo de contagem, **`timerStart()`** e uma última que indica se a contagem do tempo especificado já terminou, **`timerFinished()`**. Uma variação da última função é a **`timerWait()`**, que ao invés de apenas checar se o timer já terminou, fica aguardando dentro da função até que a contagem acabe.

A partir das funções apresentadas é possível gerar uma quinta função para a biblioteca: **`timerDelay()`**. Esta função configura o timer em um valor pré-definido e aguarda o tempo desta contagem. Assim, é possível criar um atraso de tempo com o tamanho desejado.

Outra funcionalidade interessante de um timer é permitir a contagem de tempo entre dois eventos. Uma solução neste caso é deixar com que o timer fique rodando livremente, ou free running clock, e permitir que o programador tenha acesso ao valor atual do timer.

O programador poderá então verificar o tempo decorrido entre dois eventos. Para isso, podemos chamar a função **`timerStart()`**, com qualquer valor, apenas para inicializar a contagem do timer a partir do zero. A função **`timerRead()`** retornará o valor atual da contagem.

Código 21.1: timer.c

```c
#include "derivative.h"
#include "timer.h"

void timerStart(unsigned int count_val){
    // desliga timer para mudar a configuração de tempo e resetar contador
    LPTMR0_CSR=0;
    // Configura o tempo de contagem, como o timer usa um clock de 1KHz, cada unidade equivale 1ms
    // Subtrai duas unidades, descontando o overflow e a reinicialização
    LPTMR0_CMR = count_val-2;
    //Liga o timer
    LPTMR0_CSR |= 1;
    return;
}
unsigned int timerRead(void){
    return LPTMR0_CNR;
}
void timerWait(void){
    //Aguarda final da contagem
    while (!(LPTMR0_CSR & LPTMR_CSR_TCF_MASK));
    //Desliga o contador
    LPTMR0_CSR &= ~LPTMR_CSR_TEN_MASK;
}
int timerFinished(void){
    //Verifica se terminou de contar
    if (LPTMR0_CSR & LPTMR_CSR_TCF_MASK){
        //Se terminou desliga o contador
        LPTMR0_CSR &= ~LPTMR_CSR_TEN_MASK;
        return 1;
    }else{
        return 0;
    }
}
//gera um atraso de (time) milissegundos
void timerDelay(unsigned int time){
    timerStart(time);
    while(!timerFinished());
}
void timerInit(void){
    //Liga sistema de clock do timer
    SIM_SCGC5 |= SIM_SCGC5_LPTMR_MASK;
    //Utiliza oscilador de 1kHz sem prescaller
    LPTMR0_PSR = LPTMR_PSR_PCS(1)|LPTMR_PSR_PBYP_MASK;
}
```

Código 21.2: timer.h

```
1 #ifndef TIMER_H
2 #define TIMER_H
3     void timerStart(unsigned int count_val);
4     unsigned int timerRead(void);
5     void timerWait(void);
6     int  timerFinished(void);
7     void timerDelay(unsigned int time);
8     void timerInit(void);
9 #endif
```

21.2 Aplicação

Os timers podem ser utilizados de diversos modos. Aqui são apresentados quatro modelos de utilização.

Geração de uma base de tempo

Utilizando a biblioteca, podemos resolver o problema de garantir a temporização do loop principal permitindo que uma base de tempo confiável possa ser gerada e armazenada numa variável. Essa abordagem é apresentada no código 21.3:

Código 21.3: Geração de loop temporizado baseado em timer

```
 1 #include "lcd.h"
 2 #include "timer.h"
 3 void main (void){
 4     //time armazena quanto tempo o sistema está ligado
 5     long int timeVar=0;
 6     int dummyCount;
 7     lcdInit();
 8     timerInit();
 9     for(;;){
10         //configura o tempo do timer em 1ms
11         timerStart(1000);
12         timeVar++;
13         //executa as atividades
14         DoStuff();
15         //imprime a quantidade de segundos
16         lcdNumber(timeVar/1000);
17         //aguarda o tempo restante para 1ms
18         timerWait();
19     }
20 }
```

No código, iniciamos a contagem com **`timerStart()`** no início do loop, fornecendo o tempo desejado. Ao final do loop, utilizamos a função **`timerWait()`** para aguardar o tempo restante para atingir a contagem especificada.

A vantagem com essa abordagem é que, mesmo que as funções no meio do loop gastem mais ou menos tempo, o timer continuará a contagem independentemente das rotinas. Ao final do loop, o timer aguardará apenas o tempo necessário para completar o ciclo, no caso do exemplo de 1ms.

Deve-se, no entanto, tomar cuidado com esta solução. O código garante que o loop consumirá **pelo menos** 1ms. Não é possível, sem o uso de estruturas mais complexas, garantir que todos os loops gastarão apenas 1ms. Se as funções **`DoStuff()`** ou **`lcdNumber()`** demorar mais de 1ms para ser executadas, a base de tempo será perdida.

Existem algumas soluções para esse problema. A mais simples é utilizar uma base que deixe uma certa folga para as funções serem executadas. Outra solução é mover todas as atividades críticas com relação ao tempo para interrupções baseadas em timer. Por fim, pode-se optar pelo uso de um sistema operacional preemptivo de tempo real.

Contador de frequência de eventos

Em algumas situações, é necessário conhecer a frequência com que um determinado evento acontece. Isto pode ser feito de dois modos: definir um determinado intervalo de tempo e contar a quantidade de eventos ou medir o tempo entre dois eventos. O código 21.4 apresenta a primeira abordagem:

Código 21.4: Contagem de eventos por período de tempo

```
#include "timer.h"
#include "keypad.h"
void main (void){
    //time armazena quanto tempo o sistema está ligado
    int lastkey;
    kpInit();
    timerInit();
    for(;;){
        //configura o tempo do timer em 1ms
        timerSetCounter(1000);
        kpDebounce();

        //executa as atividades
        if ((lastkey!= kpRead()) && (kpRead() == 'A')){
            lastkey== kpRead();
            count++;
        }
        //aguarda o tempo restante para 1ms
        timerWait();
        //neste ponto count representa a frequencia de eventos

        //zera-se a variável para a próxima contagem
        count = 0;
    }
}
```

Para a contagem de tempo decorrido entre dois eventos, é necessário utilizar a estrutura de leitura de teclas por rampa de subida, fazendo uso de uma variável temporária, como no código 21.5.

Código 21.5: Contagem de tempo entre eventos

```
#include "timer.h"
#include "keypad.h"
void main (void){
    //time armazena quanto tempo o sistema está ligado
    int lastkey;
    int started;
    int timeEvents;
    systemInit();
    kpInit();
    timerInit();
    for(;;){
        kpDebounce();
        //verifica se houve algum evento (tecla A por exemplo)
        if ((lastkey!= kpReadKey()) && (kpReadKey() == 'A')){
            lastkey== kpReadKey();
            //testa se a contagem ainda não foi iniciada
            if(!started){
                //apenas para iniciar a contagem, o valor final não será utilizado.
                timerStart(0);
                started = 1;
            }else{
                //contagem já iniciada no evento anterior, agora realiza a ↩
                    leitura do tempo
                timeEvents = timerRead();
            }

        }
    }
}
```

O cuidado com a segunda abordagem é para que o relógio não estoure. Se muito tempo passar entre os dois eventos, é possível que o contador do relógio sofra um overflow e o valor passe a não ser mais válido.

Relógio/Calendário

Para se construir um relógio sem a utilização de um RTC, o primeiro passo é a geração de uma base de tempo confiável. Isto pode ser conseguido através de um loop temporizado com um timer.

A cada loop uma variável é incrementada. Esta variável servirá de contador inicial. Toda vez que seu valor representar a quantidade referente a 1 segundo ela será reiniciada e a variável de minutos será aumentada. Este procedimento se repete para cada uma das outras variáveis de contagem: hora, dia, mês e ano.

Para simplificar o uso deste procedimento, podemos criar uma biblioteca que mantenha as variáveis consistentes, bem como realize a contagem e as comparações. Para que a biblioteca funcione adequadamente, basta que a função de contagem seja chamada em intervalos pré-definidos. As funções são apresentadas no código 21.6.

Código 21.6: Biblioteca de contagem/armazenamento de tempo

```
1  int count;
2  int baseTick;
3  int seconds, minutes, hours, days, months, years;
4
5  void clockInit(int newBaseTick){
6      count=0;
7      seconds=0; minutes = 0; hours=0;
8      days=0; months=0; years=0;
9      //define a velocidade de contagem em ms
10     baseTick = newBaseTick;
11 }
12
13 //deve ser chamada a cada baseTick
14 void clockTick(void){
15     //aumenta o tempo pré-configurado
16     count+=baseTick;
17     if (count >= 1000){
18         //incrementa a quantidade de segundos adequada
19         seconds++;
20         //se baseTick não for divisor de 1000, pode haver sobra a cada loop.
21         //armazena a sobra para o próximo ciclo.
22         count=count%1000;
23     }
24     if (seconds>=60){
25         minutes++;
26         seconds = 0;
27     }
28     if (minutes>=60){
29         hours++;
30         minutes = 0;
31     }
32     if (hours>=24){
33         days++;
34         hours = 0;
35     }
36     //continua para meses e anos
37 }
38
39 //As funções para ler/escrever os minutos, as horas, os dias, os meses e os anos ↩
       são similares às de segundos
40 void clockSetSeconds(int val){
41     second = val;
42 }
43 int clockGetSeconds(void){
44     return val;
45 }
```

A biblioteca funciona de modo muito similar à do RTC, Real Time Clock, permitindo que o programador configure ou leia cada uma das grandezas de tempo.

A função de inicialização recebe um parâmetro indicando qual é a velocidade em que ela será chamada. Isso é útil para configurar o funcionamento da biblioteca com diferentes velocidades de execução, sem necessidade de reescrever o código da biblioteca.

Caso a velocidade não seja múltipla de 10, as comparações da variável **`count`** poderiam levar a erros de arredondamento. Para evitar esse problema, essa variável não é simplesmente zerada. Utilizamos o resto da divisão para garantir que qualquer possível sobra continue sendo utilizada para a próxima contagem.

Supondo que o relógio tenha sido configurado para ser incrementado a cada 700ms, isto quer dizer que, na segunda chamada, o valor de clock será de 1400ms. Isto indica que a variável segundos deverá ser aumentada em uma unidade, mas existem 400ms que devem ser mantidos para garantir a sincronia do sistema.

Código 21.7: Uso da biblioteca de contagem/armazenamento de tempo

```
 1 #include "clock.h"
 2 #include "lcd.h"
 3 #include "timer.h"
 4
 5 //início do programa
 6 void main(void){
 7     unsigned char cont=0;
 8     unsigned char pos=0;
 9     systemInit();
10     timerInit();
11     lcdInit();
12
13     //configurando para ser chamado a cada 10ms
14     clockInit(10);
15
16     for(;;) {
17         //Configura timer para 10ms
18         timerRest(10000);
19
20         //imprime as informações
21         lcdPosition(0,1);
22         lcdNumber(clockGetSeconds());
23
24         //aumenta o tick e aguarda próxima rodada.
25         clockTick();
26         timerWait();
27     }
28 }
```

Reprodução de melodias

Uma melodia é o encadeamento de notas e pausas musicais de maneira ordenada. Numa partitura, documento que descreve como uma música deve ser tocada, é possível encontrar esse encadeamento e, mais especificamente, três informações: a nota (frequência), a duração (tempo), e a intensidade (volume).

Através do buzzer e do PWM utilizados, é possível controlar a frequência do sinal sonoro. Utilizando o timer podemos controlar a duração das notas. Com base nestes dois periféricos podemos construir um sistema de reprodução musical. O sistema será alimentado com dois vetores, o primeiro indicando os tempos da nota, e o segundo, a frequência. O código 21.8 apresenta a estrutura básica para efetuar a leitura dos vetores e reproduzir o som.

Código 21.8: Reprodução de melodias monofônicas

```
1
2 //início do programa
3 void main(void){
4     unsigned char cont=0;
5     unsigned char pos=0;
6     //Ground Theme – Koji Kondo (Super Mario Bros.)
7     unsigned char tempo[] = {15, 5, 15, 7, 30, 15, 30, 30, 30, 30, 30, 30, 15, 30, ←
          15, 30, 15, 30, 30, 15, 30, 22, 15, 15, 30, 15, 30, 30, 15, 15, 30, 15, ↩
          30, 15, 30, 15, 30, 15, 30, 30, 15, 30, 22, 15, 15, 30, 15, 30, 30, 15, ↩
          15, 30, 30, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 30, ↩
          15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 30, 15, 15, 15, 15, 15, 15, ↩
          15, 15, 15, 15, 15, 15, 15, 15, 30, 15, 30, 15, 30, 15};
8     unsigned int notas[] = {E2, v, E2, v, E2, C2, E2, G2, v, G, v, C2, v, G, v, E, ←
          v, A, B, AS, A, G, E2, G2, A2, F2, G2, E2, C2, D2, B, v, C2, v, G, v, E, ↩
          v, A, B, AS, A, G, E2, G2, A2, F2, G2, E2, C2, D2, B, v, G2, F2S, F2, ↩
          D2S, v, E2, v, G2, A, C2, v, A, C2, D2, v, G2, F2S, F2, D2S, v, E2, v, ↩
          C3, v, C3, C3, v, G2, F2S, F2, D2S, v, E2, v, GS, A, C2, v, A, C2, D2, v, ←
          D2S, v, D2, v, C2};
9     systemInit();
10    pwmInit();
11    timerInit();
12
13    //inicializa com a primeira nota
14    pos = 0, cont = 0;
15    pwmBuzzer(notas[0]);
16    for(;;) {
17        //cada ciclo consome 10ms
18        timerRest(10000);
19        cont ++;
20        if (cont >= tempo[pos]){
21            //passado o tempo muda a nota
22            pos++;
23            pwmBuzzer(notas[pos]);
24            cont=0;
25        }
26        //verifica se terminou de percorrer o vetor de notas
27        if(pos>=100){
28            pos = 0;
29        }
30        timerWait();;
31    }
32 }
```

21.3 Exercícios

Ex. 21.1 — Crie um relógio com o uso do timer que conte os segundos minutos e horas. Os valores devem ser exibidos no LCD. O programa deve ainda fazer a leitura do teclado de modo que o valor dos segundos possa ser alterado pelas teclas A e B, os minutos pelas teclas X e Y e as horas pelas teclas U e D. Utilize as bibliotecas "timer" e "keypad".

Ex. 21.2 — Um determinado sistema embarcado realiza o monitoramento das rotações de um motor através de uma chave tipo reed switch. Acoplado, ao eixo do motor, se encontram 16 imãs uniformemente distribuídos ao longo da circunferência. Deste modo, o motor gera 16 pulsos a cada rotação. Faça um programa que meça a quantidade de rotações por minuto e a exiba no display de LCD.

Ex. 21.3 — Faça um programa que controle as luzes de um semáforo que controla a saída dos carros de uma garagem. Com relação ao tempo de acionamento, a luz verde deve ficar acesa por 30 segundos, a amarela por 5s e a vermelha por 60s. Utilize a biblioteca "timer" para ter precisão no tempo. Lembre-se de que as funções da biblioteca não conseguem medir intervalos de tempo superiores a 65.535 microssegundos.

Ex. 21.4 — Um dos modos de se construir um radar de velocidade é utilizando dois sensores de presença. Os sensores são dispostos a uma distância conhecida. Assim que o objeto passa pelo primeiro sensor, é iniciada a contagem de tempo. Quando o objeto passa pelo segundo sensor, a contagem é pausada e a velocidade é calculada como a divisão do espaço, pré-definido, e o tempo gasto. Crie um programa que faça a leitura de dois sensores, conectados aos terminais digitais **D4** e **D5**, e calcule e exiba a velocidade no display de LCD, sabendo que a distância entre os sensores é de 1 metro.

CAPÍTULO

22

Interrupção

22.1Fonte de interrupção
22.2Acessando a rotina de serviço da interrupção
22.3Compartilhando informações
22.4Exercícios

"Quando eu vou numa biblioteca e vejo a bibliotecária em sua mesa, tenho medo de interrompê-la, mesmo que ela se sente lá especificamente para que ela seja interrompida, mesmo que ser interrompida por razões como esta, por pessoas como eu, seja exatamente seu trabalho."

Aaron Swartz

Até o momento, todos os programas que foram desenvolvidos seguem um fluxo sequencial, sendo alterado apenas por chamadas de funções, estruturas de decisão ou de repetição. Quando fazemos a leitura de alguns periféricos é necessário aguardar que as atividades terminem. O fluxo sequencial não permite que façamos uma outra atividade enquanto os periféricos estão trabalhando.

Por exemplo, o conversor analógico-digital. Em seu funcionamento, é preciso executar um comando para inicializar a conversão. Esta atividade demanda uma certa quantidade de tempo, que pode inclusive variar de acordo com o valor do sinal. A solução adotada até agora é utilizar um loop que fica constantemente verificando se a conversão terminou. Esta técnica é conhecida como pooling.

O problema de se realizar a leitura de algum periférico por pooling, é que o processador perde tempo atoa para verificar se algum evento aconteceu.

A solução ideal é possuir um sistema que monitorasse o evento desejado sem necessidade do processador. Este sistema identificaria quando o evento aconteceu e avisaria ao processador para que ele tomasse as devidas providências. Este sistema é conhecido como interrupção. No entanto, para se beneficiar deste sistema, é necessário que o periférico seja capaz de gerar interrupções. Esta capacidade é implementada em hardware pelo fabricante do microcontrolador.

A geração da interrupção depende do acontecimento de um evento, como o fim da conversão para o AD, a chegada de informação na serial ou a mudança de valor de algum dos terminais em uma porta.

Em seu funcionamento, quando o evento acontece, o periférico que gerencia a interrupção pausa o programa que estiver em execução no processador. Pode haver um pequeno atraso para a conclusão de atividades que não podem ser interrompidas. Em seguida, todas as informações importantes para o programa e que estão no processador são salvas. Estas informações vão para uma região de memória que permite que elas sejam facilmente recuperadas quando a interrupção terminar.

Com o programa pausado e as informações salvas, o periférico faz a chamada de uma rotina. Esta rotina possui seu endereço pré-definido. Ela é comumente chamada de ISR, *interrupt service routine*, ou rotina de serviço da interrupção.

Após o fim da execução da função, o periférico que gerencia a interrupção recarrega as informações que foram salvas e volta a executar o programa principal exatamente no ponto onde estava antes da interrupção.

22.1 Fonte de interrupção

É possível transformar qualquer tipo de evento numa interrupção. Para isso, o desenvolvedor do hardware deve projetar o periférico para tal. As fontes de interrupção mais comuns vêm dos periféricos mais utilizados nos microcontroladores:

- Conversor AD: Fim de conversão
- Timer: Overflow da contagem
- Timer: Comparação de valor (match)
- Portas de I/O: Mudança no valor dos terminais
- Serial: Fim de transmissão
- Serial: Recepção de valor
- Serial: Colisão de mensagens
- Memória: Fim de escrita

Para flexibilizar a utilização das interrupções é comum os microcontroladores possuírem canais de interrupção externa.

As interrupções internas funcionam exatamente como as internas. A diferença é que os eventos são definidos por atividades externas, em geral, por outros circuitos que precisem avisar ao microcontrolador de algum evento.

Isto permite que outros tipos de eventos sejam adicionados ao projeto: botões de emergência, transferência de dados, variação de luminosidade, bateria baixa, entre outros. A interrupção externa precisa apenas que essa interface seja realizada de modo digital em um terminal pré-definido.

22.2 Acessando a rotina de serviço da interrupção

A rotina de serviço da interrupção pode ser implementada como uma função que não recebe nenhum parâmetro nem retorna informação. Em geral, o nome da função também pode ser escolhido pelo programador.

```
1 void ISR_serial(void){
2     //código ...
3 }
4 void ISR_adc(void){
5     //código ...
6 }
```

Existem microcontroladores que possuem várias rotinas de interrupção, uma para cada tipo de evento. Noutros há apenas 1 rotina, e os periféricos têm que compartilhar a mesma função. Nestes microcontroladores, o programador deve, dentro da função, verificar qual foi o evento que chamou a rotina. Isto pode ser feito através de bits dedicados a essa função, chamados de flags.

```
1 void ISR_common(void){
2     if(BitTst(SERIAL_REGISTER,SERIAL_INT_FLAG)){
3         //código ...
4     }
5     if(BitTst(ADC_REGISTER,ADC_INT_FLAG)){
6         //código ...
7     }
8 }
```

O endereço dos registros, bem como a posição do bit de flag, varia para cada arquitetura e para cada periférico. No entanto, estas informações podem ser facilmente obtidas no datasheet do microcontrolador.

O segundo passo para implementar a rotina é indicar ao compilador qual função deverá ser chamada quando acontecer a interrupção. Cada compilador possui sua maneira de fazer esse link. Para os compiladores baseados no GCC, por exemplo, basta adicionar a expressão **`interrupt N`** após a declaração da função, onde **`N`** indica qual é o número da interrupção a ser tratada.

```
1 // Do manual do compilador temos:
2 //     4 -> trata as interrupções do AD
3 //     7 -> trata as interrupções da serial
4
5 void NomeDaFuncao(void) __interrupt 4 {
6     //código para tratar a interrupção AD
7 }
8
9 void NomeDaFuncao(void) __interrupt 7 {
10    //código para tratar a interrupção da serial
11 }
```

Alguns compiladores precisam que o programador indique o endereço físico onde a função deve ser armazenada. Por exemplo, para o compilador C18 da Microchip, é necessário utilizar a diretiva **#pragma** para armazenar um pequeno código em assembly numa região pré-definida. Esse código, por sua vez, faz a chamada da função.

A diretiva **#pragma** é utilizada mais uma vez para indicar que a função criada será chamada via interrupção. Existem algumas diferenças entre chamar a função de algum ponto do código ou chamar a função para servir uma interrupção. Deste modo o compilador faz os ajustes necessários.

```
1 //Indicar a posição da rotina
2 #pragma code high_vector=0x08
3 void interrupt_at_high_vector(void){
4     _asm GOTO Interrupcao _endasm
5 }
6 #pragma code
7
8 #pragma interrupt ISR_function
9 void ISR_function(void){
10    //código ...
11 }
```

Para a plataforma Freedom, as funções que tratam a interrupção estão pré-definidas, bastando criá-las no **`main()`** e ativar as flags correspondentes.

No código 22.1 é apresentado um exemplo de sistema de eco da serial. Todos os bytes recebidos serão devolvidos pela serial. A grande vantagem dessa abordagem é que o loop principal fica livre para executar qualquer outra atividade.

Código 22.1: Exemplo de resposta a eventos da serial pela interrupção na Freedom

```
1 //função de serviço da interrupção da serial
2 void UART0_IRQHandler(void) {
3     serialSend(serialRead());
4 }
5
6 void main(void) {
7     systemInit();
8     serialInit();
9
10    //Habilita a geração de uma interrupção pelo periférico
11    UART0_C2 |= 1 << 5;
12
13    // Habilita o microccontrolador a atender a interrupção gerada pela serial
14    NVIC_ICPR |= 1 << (12 % 32);
15    NVIC_ISER |= 1 << (12 % 32);
16
17    for (;;) ;
18 }
```


22.3 Compartilhando informações

As interrupções servem para atender aos eventos que acontecem sem que o processador fique aguardando esses eventos enquanto poderia estar fazendo algo útil.

Após atender as interrupções é comum querermos deixar alguma informação disponibilizada para que o processador possa utilizar o resultado do evento. Como a interrupção não devolve nenhum valor é preciso criar um sistema de comunicação.

O sistema mais simples para a comunicação entre a interrupção e o código principal é a utilização de uma variável global. No entanto, compartilhar variáveis entre a interrupção e o código principal pode ser perigoso.

Duas condições agravam esta situação: se ambos fizerem, além de leituras, gravações de valor na variável; e se a variável não puder ser manipulada de uma única vez pelo processador, como uma variável de 64 bits numa placa de 16 bits. O código 22.2 apresenta esta situação.

Código 22.2: Problema no compartilhamento de informações

```
1 //Interrupção externa da chave de emergência
2 void ISR_Extern (void){
3     //liga o freio de emergência
4     BitSet(PORTC,2);
5 }
6 void main (void){
7     int t;
8     //Apenas troca o valor de um led para indicar que o sistema está funcionando
9     for(;;){
10        BitSet(PORTC,1);
11        for(t=0;t<10000;t++);
12        BitSet(PORTC,1);
13        for(t=0;t<10000;t++);
14    }
15 }
```

No código apresentado, tanto a interrupção quanto o programa principal gravam informações em **PORTC**. É possível então que a informação dessa variável seja corrompida. Para entender o problema podemos começar com a operação **BitSet(PORTC,1)**. Esta operação na verdade é composta de 4 passos:

1.Ler o valor da porta C e o salvar numa variável temporária;

2.Criar uma a máscara para o bit 1 ;

3.Realizar a operação bitwise OU entre a máscara e a variável temporária;

4.Salvar o valor da variável temporária na porta C.

O problema do compartilhamento acontece pois a interrupção pode parar o processamento dessa função a qualquer momento, inclusive entre os passos.

Supondo que a porta C possui o valor **0x00** e o programa começou a rotina para ligar o led. Entre os passos 2 e 3 acontece a interrupção. Durante o passo 1, uma variável temporária foi criada e recebeu o valor **0x00**. No passo 2, a máscara **0x01** é criada. Neste momento, a rotina de interrupção é chamada e liga o bit 2 da porta C. Como a porta C tinha o valor **0x00**, com o segundo bit sendo ligado, ela passa a ter o valor **0x02**. No entanto, a variável temporária, que foi criada pelo programa principal, não foi alterada. Ela continua com o valor **0x00**.

Voltando ao programa principal, o passo 3 é executado e a variável temporária passa a ter o valor **(0x00|0x01) == 0x01**. No passo 4 a porta C, que tinha o valor **0x02**, por causa da interrupção, receberá o valor **0x01**, **desligando** o bit responsável pelo acionamento do freio mesmo após a detecção do acionamento do botão de emergência.

Para evitar esse problema, uma opção é não realizar operações de escrita na variável dentro do programa principal, apenas de leitura. Isto pode ainda levar a problemas quando for feita a leitura de mais de uma variável ao mesmo tempo, como um vetor. Para o compartilhamento de variáveis simples é uma boa alternativa.

Se for necessário realizar a escrita da variável em vários locais, uma opção é proteger a escrita desabilitando a interrupção durante o processo.

Em geral, para desabilitar uma interrupção, é necessário escrever um comando em assembly, visto que a linguagem C não apresenta suporte nativo a esta funcionalidade. É comum criar um macro para facilitar o uso destas instruções.

```
1 //As macros dependem da arquitetura e modelo de processador
2 #define INT_DISABLE() __asm CLI
3 #define INT_ENABLE()  __asm SLI
4 void main (void){
5     int t;
6     for(;;){
7         INT_DISABLE();
8         BitSet(PORTC,1);        //Escrita protegida
9         INT ENABLE();
10        for(t=0;t<10000;t++);
11        INT_DISABLE();
12        BitSet(PORTC,1);        //Escrita protegida
13        INT_ENABLE();
14        for(t=0;t<10000;t++);
15    }                                    352
16 }
```

É recomendado desligar o sistema apenas durante o tempo necessário para fazer a escrita. Deixar o sistema desligado durante muito tempo pode fazer com que as detecções dos eventos sejam atrasadas, fazendo com que o sistema não funcione como desejado.

Existem opções mais seguras para fazer o compartilhamento de informações entre as interrupções e o programa principal como a utilização de filas de mensagem.

As filas de mensagem são bibliotecas que conseguem fazer o armazenamento de mensagens inteiras sem o problema de corromper as variáveis. O problema da interrupção é tratado internamente à biblioteca, deixando transparente para o programador seu funcionamento, geralmente fazendo uso de semáforos ou mutexes.

Tanto as filas de mensagens como os sincronizadores semáforo e mutex não serão abordados neste livro.

22.4 Exercícios

Ex. 22.1 — Quando uma interrupção é chamada?

Ex. 22.2 — Porque existem diferentes modos de se definir qual é a função responsável por atender uma interrupção?

Ex. 22.3 — Escreva uma função que faça o tratamento da interrupção de recepção de dados da serial. Esta função deve ler o valor recebido da serial no registro RX_DATA e armazená-lo em uma variável global. A interrupção de serial é definida pela função **`SerialISR()`**. A função só deve atualizar a variável global se o dado recebido for um algarismo entre **`0`** e **`9`**. Para qualquer outro valor recebido a variável global deve receber **`0xFF`**.

Ex. 22.4 — Num determinado sistema, uma interrupção de timer, que acontece a cada milissegundo, incrementa a variável global **`unsigned long int tick`** em uma unidade.

Crie um programa que utilize essa variável para realizar a contagem de tempo e exibir a informação no LCD.

CAPÍTULO

23

Watchdog

23.1Modo de uso

"Tecnologia é uma palavra para descrever uma coisa que ainda não funciona."

Douglas Adams

Um dos primeiros resultados obtidos no campo da computação teórica é o Teorema de Parada. Nele, Alan Touring enuncia ser impossível provar que um determinado programa está correto. Deste modo, por mais que sejam realizados testes, não é possível garantir de modo determinístico que não haverá erro na execução do código.

Além dos problemas de codificação, ou bugs, existem também os problemas físicos, que podem gerar erros no funcionamento do sistema. Estes erros podem ser permanentes, como a queima de algum componente, a desconexão de algum cabo, ou intermitentes, como problemas de contato, surtos de tensão ou ruídos eletromagnéticos. Os problemas físicos podem ser tão ou mais graves que os erros de codificação.

Pensando nestas possíveis falhas, algumas ferramentas foram desenvolvidas para garantir que o sistema, caso encontre algum problema, possa se recuperar.

Uma destas ferramentas faz uso de uma interrupção, que é chamada sempre que o processador encontre algum código que ele não consegue executar. Isto permite que o programa seja informado do erro e tome as providências necessárias.

Outro tipo de interrupção voltado para a correção do erro são as os eventos gerados por erro em cálculos matemáticos, como divisão por zero, operações com infinito, entre outros.

O watchdog também é uma ferramenta desenvolvida com o objetivo de permitir ao sistema se recuperar de erros. No entanto, seu funcionamento é bastante diferente e, num primeiro contato, pode parecer estranho.

O watchdog é constituído de um temporizador implementado em hardware e um circuito de reset. Quando um tempo pré-definido se esgota, o circuito de reset é acionado. Sua única função é reiniciar, ou resetar, o sistema quando sua contagem terminar.

Esta ferramenta consegue aumentar a segurança do sistema por ser independente do sistema principal. Sua contagem continua mesmo que haja algum problema no software. Além disso, ele possui uma estrutura que permite ao programador resetar sua contagem.

Contanto que o programador reinicie o watchdog em intervalos regulares, ele não terminará sua contagem e o sistema continuará em funcionamento. Se acontecer algum problema com o programa, seja um loop infinito, um deadlock ou até mesmo um invasor que impeça o fluxo normal do programa, quando o watchdog terminar sua contagem ele reiniciará a placa, dando uma chance do programa se reinicializar corretamente.

A maioria dos sistemas de watchdog, uma vez ligados, só podem ser desligados reinicializando-se o microcontrolador.

23.1 Modo de uso

Para utilizar o watchdog corretamente é preciso primeiramente conhecer o programa que será executado. Para sistemas embarcados, é comum possuir um código onde as funções são executadas de modo cíclico, ou até mesmo uma função que tenha algum requisito temporal crítico.

No código 23.1, a função **`DebounceTeclas()`** precisa ser executada constantemente para que o teclado funcione corretamente. No entanto, na linha 6, é utilizada uma leitura do teclado por pooling, onde o sistema aguarda enquanto nenhuma tecla for pressionada. Isso faz com que o programa fique travado. Sem executar a função de debouce, a tecla não é atualizada; sem atualizar a tecla o programa não sairá do loop.

Código 23.1: Problema (loop inifito) debounce mau utilizado

```
1 void main(void){
2     kpInit();
3     lcdInit();
4     for(;;){
5         kpDebounce();
6         while (kpRead() == 0);
7         lcdPrint(KpRead());
8     }
9 }
```

Para resolver este problema, basta fazer com que, ao invés de aguardar uma resposta para depois continuar, o sistema apenas imprima a resposta se houver alguma tecla pressionada, como no código 23.2.

Código 23.2: Solução do problema de debounce mau utilizado

```
 1 void main(void){
 2     systemInit();
 3     kpInit();
 4     lcdInit();
 5     for(;;){
 6         kpDebounce();
 7         if (kpRead() != 0){
 8             lcdNumber(kpRead());
 9         }
10     }
11 }
```

O problema do loop infinito, bem como sua solução neste exemplo, é bastante simples e até mesmo óbvia. No entanto, estes problemas podem estar escondidos em outras funções ou operações, fazendo com seja muito difícil percebê-lo. O problema pode ainda possuir requisitos bastante complexos para acontecer (se o aquecedor estiver ligado e o sensor de temperatura marcar mais de 35 graus e o botão de emergência for pressionado de modo sincronizado com a recepção de dados da serial, etc.). Isto faz com que seja difícil rastrear todas as opções.

Adicionar o sistema de watchdog não faz com que esses problemas sejam resolvidos, mas, quando acontecerem, a placa consiga tentar se recuperar sozinha, sem interferência humana, ao invés de simplesmente ficar travada. Adicionando ao watchdog ao exemplo anterior, temos o código 23.3.

Código 23.3: Programa com watchdog

```
 1 void main(void){
 2     systemInit();
 3     kpInit();
 4     lcdInit();
 5     watchdogInit();
 6     for(;;){
 7         kpDebounce();
 8         if (kpRead() != 0){
 9             lcdNumber(kpRead());
10         }
11         watchdogFeed();
12     }
13 }
```

A partir do momento em que o watchdog é inicializado, ele deve ser "alimentado" constantemente. Chamamos de alimentar o watchdog o ato de reinicializar seu contador.

Enquanto o sistema estiver funcionando corretamente, o watchdog será alimentado em intervalos regulares, não resetando a placa. Caso o sistema trave em algum ponto, o watchdog atuará e o sistema será reiniciado.

No momento em que o sistema é reiniciado, é possível verificar se a reinicialização está acontecendo por condições normais (a placa acabou de ser ligada) ou se foi o watchdog que forçou a reinicialização. Neste último caso, é possível planejar para que o sistema tome decisões e informe ao usuário que alguma coisa aconteceu.

Outro uso para o watchdog é garantir que a placa está funcionando obedecendo todos os tempos planejados. Em sistemas embarcados, não executar as tarefas na velocidade programada pode ser tão ruim quanto a placa parar de funcionar completamente. Para isso, podemos ajustar o valor do watchdog de modo que, se o tempo de processamento ultrapassar o permitido, a placa será reinicializada.

No código 23.4, a função **`pwmProcess()`** deve ser executada a cada 1ms. Se este tempo não for obedecido, o sistema de controle não funcionará corretamente, podendo levar a uma condição de risco para a placa ou até mesmo para o usuário.

Código 23.4: Protegendo o programa com watchdog

```
 1 void main(void){
 2     systemInit();
 3     pwmInit();
 4     lcdInit();
 5     serialInit();
 6     watchdogInit();
 7     for(;;){
 8         serialProcess();
 9         pwmProcess();        //executar a cada 1ms
10         watchdogFeed();
11     }
12 }
```

Se a função **serialProcess()** demorar muito para terminar, por estar processando uma mensagem muito longa, por exemplo, o watchdog irá reinicializar o processador, que poderá, por sua vez, informar que a placa apresenta algum problema e não deve mais ser utilizada. O código 23.5 apresenta um modelo de inicialização levando em conta o acionamento do watchdog.

Código 23.5: Testando reinicialização por watchdog

```
1 void main(void){
2     systemInit();
3     //primeiro teste antes da continuidade do programa
4     if(watchdogProblem()){
5         lcdString("Problema 01");
6         for(;;); //não pode continuar execução
7     }
8     pwmInit();
9     serialInit();
10    lcdInit();
11    watchdogInit();
12
13    for(;;){
14        serialProcess();
15        pwmProcess();       //executar a cada 1ms
16        watchdogFeed();
17    }
18 }
```

Parte III

Arquiteturas para desenvolvimento de software embarcado

24 ---Arquiteturas de software embarcado
25 ---Desenvolvimento de um kernel cooperativo
26 ---Projeto de kernel com soft realtime
27 ---Controladora de dispositivos

CAPÍTULO

24

Arquiteturas de software embarcado

24.1 One-single-loop

24.2 Sistema controlado por interrupções

24.3 Multitask cooperativo

- Fixação de tempo para execução dos slots
- Utilização do tempo livre para interrupções

24.4 Kernel

24.5 Sistemas operacionais

- Processo
- Escalonadores

24.6 Exercícios

> "*Restringidos por limitações de memória, requisitos de desempenho, considerações físicas e de custos, cada projeto de sistemas embarcados exige uma plataforma adaptadas precisamente às suas necessidades, recursos não utilizados ocupam espaço de memória preciosa, enquanto os recursos em falta devem ser acrescentados.*"
>
> Richard Soley

Sistemas embarcados são caracterizados por um hardware e um software que formam um componente de um sistema maior, onde o funcionamento deve acontecer sem a intervenção humana. Os sistemas embarcados constituem grande parte do destino final dos processadores e componentes produzidos pela indústria de semicondutores.

Os sistemas embarcados tornam-se mais complexos à medida que a própria tecnologia de semicondutores permite a implementação de aplicações mais complexas. As restrições impostas a esses sistemas (como desempenho, consumo de energia, custo, confiabilidade e tempo de desenvolvimento) estão cada vez mais rigorosas. Grande parte da dificuldade do projeto se deve ao fato de que as restrições impostas induzem a um projeto integrado de software e hardware.

A implementação deste projeto pode ocorrer em várias arquiteturas diferentes, podendo ser através de microcontroladores, processadores digitais de sinais (DSP) ou dispositivos de lógica programável (FPGA). Devido à complexidade da arquitetura de um sistema embarcado, contendo múltiplos componentes de hardware e software em torno de uma

estrutura de comunicação, e à grande variedade de soluções possíveis (desempenho, consumo de potência), é essencial que o projeto do sistema apresente níveis de abstração elevados.

No desenvolvimento de um sistema de maior porte, é importante definir o tipo de arquitetura que será utilizada. A escolha deve ser baseada no tipo de dispositivo a ser desenvolvido. Várias características podem influenciar na escolha: a complexidade do sistema, a capacidade de processamento, a quantidade ou possibilidade de subprodutos (derivados), a necessidade de garantia de tempo real, a quantidade de periféricos, a criticidade, entre outras. Outro agravante na escolha é que, em geral, não existe solução ótima, nem uma solução geral que possa ser aplicada a todos os projetos.

A seguir, neste capítulo, serão apresentadas quatro arquiteturas existentes.

24.1 One-single-loop

Neste tipo de arquitetura, o software possui apenas um loop. Este loop faz a chamada de sub-rotinas que, por sua vez, gerenciam uma parte do hardware ou do software.

Dentro da função principal **`main()`** é criado um loop infinito que fica responsável por executar as tarefas que formarão a aplicação. O exemplo do código 24.1 a seguir utiliza essa abordagem:

Código 24.1: Exemplo de arquitetura single-loop

```
1 //seção de includes
2 #include "keypad.h"
3 #include "ssd.h"
4 //função principal
5 void main (void){
6     //declaração das variáveis
7     int  ia, ib, ic;
8     float fa, fb, fc;
9     //inicialização dos periféricos
10    kpInit();
11    ssdInit();
12    //loop principal
13    for(;;){
14        //chamada das tarefas
15        kpDebounce();
16        ia = kpRead();
17        ssdUpdate(); //tem que ser executado pelo menos a cada 10ms
18    }
19 }
```

A vantagem de utilizar esta abordagem é a facilidade de se iniciar um projeto, devido à simplicidade de sua implementação.

O uso dessa arquitetura para o desenvolvimento de sistemas maiores não é aconselhada devido à dificuldade de se coordenar um conjunto maior de tarefas e ainda garantir que a execução delas em tempo hábil, geralmente com requisitos de tempo determinísticos.

Outro problema é a modificação ou ampliação do software. Geralmente a inserção de uma função no meio do loop pode gerar erros em outras funções devido a restrições de tempo dos periféricos associados. A seguir, no código 24.2 é apresentado um trecho de um programa que insere algumas atividades de cálculo e de recepção de dados da serial ao programa anterior. Estas novas atividades podem atrapalhar o tempo de atualização do display de sete segmentos, gerando flicker para o usuário.

Código 24.2: Problema na sincronia de tempo para o single-loop

```
1     //loop principal
2     for(;;) {
3         //chamada das tarefas
4         kpDebounce();
5         ia = kpReadKey();
6         ssdUpdate(); //tem que ser executado pelo menos a cada 10ms
7         ic = serialRead();
8         fa = 2.0 * ic / 3.14;
9         serialSend(fa & 0x00FF);
10        serialSend(fa >> 8);
11    }
```

Quando a plataforma a ser utilizada apresenta poucos recursos, principalmente no quesito de memória, a utilização de um sistema operacional pode ser inviabilizada. É comum os sistemas operacionais consumirem alguns kb de memória. O FreeRTOS, por exemplo, exige um mínimo de 5 kb. Alguns sistemas mais complexos, como o VxWorks, podem alcançar dezenas ou centenas de kb, inviabilizando seu uso em sistemas com poucos bytes de ROM/Flash.

Apesar das vantagens aparentes, este é um modelo que deve ser usado com cautela. A velocidade de execução do loop principal é dependente das funções implementadas e qualquer alteração pode modificar este tempo. Assim, é praticamente impossível utilizar a frequência de execução para realizar alguma tarefa temporal.

Outro problema é a possibilidade de travamento do sistema. Se alguma das funções entrar num loop infinito, ou em qualquer condição de deadlock, o sistema inteiro ficará paralisado.

A arquitetura one-single-loop pode ser utilizada com sucesso, contanto que a aplicação seja simples e não exija requisitos rigorosos em relação ao tempo de execução. Além disso, é uma abordagem interessante para realizar provas de conceito e testes iniciais em novas plataformas. Lembre-se apenas de documentar e separar as funções em arquivos diferentes, agrupadas pela similaridade ou por tipo de periférico que elas acessam. Isto facilitará a mudança do sistema para arquiteturas mais complexas quando houver a necessidade de fazê-lo.

24.2 Sistema controlado por interrupções

A arquitetura one-single-loop é bastante rápida e praticamente não insere sobrecarga de processamento ou memória. No entanto, o maior problema com esta arquitetura é a criação de rotinas que envolvem leituras de informação ou execuções periódicas. Em geral, as leituras envolvem algum tipo de atraso, como a conversão de um sinal analógico ou a recepção de uma mensagem. O problema com as execuções periódicas, é que nem sempre o loop tem tempo constante, podendo variar seu período a cada execução. Mesmo quando o loop apresenta uma constância, o cálculo deste tempo não é simples de ser realizado, além de alterar-se com cada nova adição ou remoção de código e funcionalidade.

Entre as soluções disponíveis para resolver estes problemas, uma se destaca: as interrupções. Alguns sistemas embarcados são controlados por interrupções, ou seja, as tarefas desempenhadas pelo sistema são provocadas por diferentes tipos de evento. Uma interrupção pode ser gerada por um temporizador em uma frequência pré-definida, ou por um controlador de porta serial. É possível programar um sistema de modo que ele responda aos eventos e não apenas execute operações de modo sequencial. No entanto, é necessário que o hardware tenha suporte às interrupções. Entre as interrupções mais comuns implementadas pelos fabricantes, tem-se:

- Temporizadores/relógios: Acontece em tempos pré-definidos.
- Entradas digitais: Ná mudança no valor de uma porta ou de um terminal.
- Entradas analógicas: Quando o processo de conversão terminou.
- Comunicação: Na recepção de uma mensagem/byte ou quando o sistema está disponível para enviar uma informação.

Além das interrupções geradas pelos periféricos internos, alguns fabricantes oferecem a possibilidade de gerar uma interrupção por um evento externo, disponibilizando um terminal especificamente para isso.

No desenvolvimento orientado para interrupções, as funcionalidades do sistema são codificadas em funções que são executadas como resposta aos eventos. Esta abordagem reduz drasticamente a latência da resposta aos eventos. No entanto, alguns cuidados devem ser tomados.

A interrupção pausa o fluxo do programa principal, executando a função pré-definida. Com exceção do tempo transcorrido, o programa principal não percebe a pausa. Se a interrupção for algo recorrente, como uma função de controle temporizada, o tempo gasto na interrupção pode fazer com que o loop principal seja executado lentamente ou, até mesmo, com que não seja executado.

De modo geral, é importante que as funções que serão executadas nas interrupções sejam leves e simples. Sempre que possível, deixar o processamento pesado no programa principal.

A seguir, é apresentado um exemplo, onde é implementado um interpretador de comandos via comunicação serial. Com o uso da arquitetura one-single-loop pode-se projetar o sistema do seguinte modo:

```
1 #include "serial.h"
2 void main (void){
3     char buffer[50];
4     char pos=0;
5     char data=0;
6     serialInit();
7     for(;;){
8         //aguarda o recebimento de um byte
9         do{
10            data = readSerial();
11        }while(data == 0);
12        //salva o novo byte no buffer
13        buffer[pos] = data;
14        pos++;
15        //teste para evitar overflow
16        if (pos >= 50){
17            pos = 0;
18        }
19        //se chegou o caracter de fim de linha executa a ação
20        if (data == '\n'){
21            //verificação se não houve erros na recepção da mensagem
22            if (crcCheck(buffer)){
23                doStuff(buffer);
24            }
25            pos = 0;
26        }
27    }
28 }
```

O problema com essa abordagem é que as funções **`crcCheck()`** e **`doStuff()`** podem consumir tempo demais, fazendo com que alguns dados recebidos pela comunicação serial sejam perdidos. Isso pode ser evitado com o uso da interrupção de chegada de byte na comunicação serial.

```
1 #include "serial.h"
2 #include "interrupcao.h"
3
4 //o buffer é global para que tanto a interrupção quanto o programa principal ↩
      possam acessá-lo
5 static char buffer[50];
6
7 //indica qual a proxima posição disponivel no buffer
8 static char pos=0;
9
10 //essa função é executada pela interrupção de recepção da serial, portanto temos ↩
      certeza de possuir um byte válido no registro do microcontrolador
11 void UART0_IRQHandler(void) {
12     buffer[pos] = serialRead();
13     if (pos >= 50){
14         pos = 0;
15     }
16 }
17
18 void main (void){
19     systemInit();
20     serialInit();
21
22     //Habilita a geração de uma interrupção pelo periférico
23     UART0_C2 |= 1 << 5;
24     // Habilita o microccontrolador a atender a interrupção gerada pela serial
25     NVIC_ICPR |= 1 << (12 % 32);
26     NVIC_ISER |= 1 << (12 % 32);
27
28     for(;;){
29         //a verificação de fim de mensagem agora depende do último caracter recebido
30         if (buffer[pos] == '\n'){
31             //verificação se não houve erros na recepção da mensagem
32             if (crcCheck(buffer)){
33                 doStuff(buffer);
34             }
35             pos = 0;
36         }
37         lowPowerMode();
38     }
39 }
```

O problema de perder algum dado da comunicação serial é resolvido, porém surge um outro problema. Como as duas funções (principal e interrupção) operam com o buffer, é

possível que esse buffer seja alterado na interrupção enquanto a função principal calcula o CRC da mensagem. Isto pode levar a um resultado errado da função **`crcCheck()`** ou, pior, a função **`doStuff()`** vai executar uma tarefa de maneira errada.

Existem algumas soluções que podem ser implementadas para evitar esse tipo de problema. Algumas dessas soluções são o uso de mutexes, semáforos ou filas de mensagens para passar o evento para o programa principal. A solução mais simples é a utilização de um flag indicando que a mensagem está em processamento e que portanto, a interrupção deve evitar alterar a mensagem.

Outra grande vantagem do uso da interrupção é a possibilidade de se utilizar o modo de baixo consumo de energia de modo simples. Como grande parte das ações são executadas apenas quando há uma interrupção, o sistema pode pausar o processamento enquanto não houver uma mensagem disponível no buffer. Se o sistema for inteiramente orientado a eventos que geram interrupção, ele automaticamente sairá do modo de baixo consumo sempre que algum evento importante acontecer.

O desenvolvimento do sistema pode ser, então, simplificado em 3 etapas: atender as interrupções; processar o resultado das interrupções no loop principal; se não houver nada a ser processado, entrar em modo de baixo consumo de energia.

24.3 Multitask cooperativo

Nas seções anteriores foram apresentadas a arquitetura one-single-loop, e o sistema com controle de interrupções. No entanto, nestas arquiteturas, a adição ou modificação de uma tarefa do sistema pode impactar de forma negativa nos requisitos temporais ou, até mesmo, inserir erros não previstos por causa das relações entre as operações realizadas.

Uma alternativa é o uso de uma arquitetura multitarefa que é muito parecida com a arquitetura one-single-loop, exceto pelo loop, que é escondido em uma API. O programador define uma séria de tarefas e cada uma das tarefas tem seu próprio ambiente de execução.

Uma máquina de estados pode ser usada para indicar quais são as tarefas, quais as ligações entre elas e as ações que devem ser tomadas em cada estado do sistema. Cada estado dessa máquina representa uma situação onde o sistema realiza uma determinada tarefa, ou um conjunto de tarefas. A mudança do estado é dada quando alguma condição for satisfeita. Se a mudança de tarefas for extremamente rápida, o efeito resultante, para o ser humano, é de que todas as tarefas estão sendo executadas simultaneamente.

Por exemplo, um sistema que possua um teclado, um display e uma comunicação serial. O display precisa ser atualizado periodicamente e, por isso, as atualizações são intercaladas com as outras operações. Este sistema pode ser modelado conforme mostrado na figura 24.1:

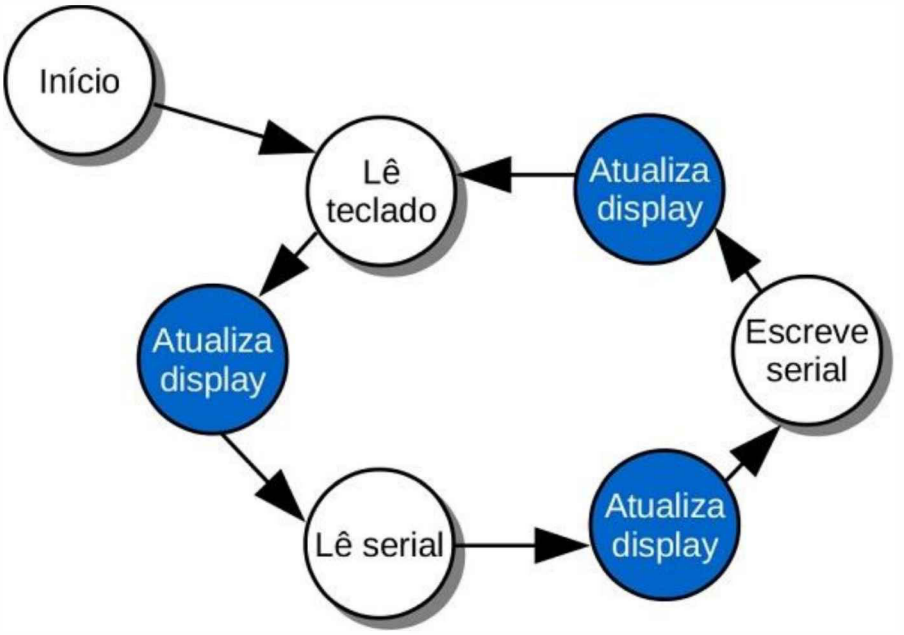


Figura 24.1: Exemplo de máquina de estados

Pode-se notar, que neste modelo, após a fase de inicialização, o sistema entra em um ciclo, como na abordagem one-single-loop. Isto é comum em sistemas sequenciais. No entanto, nesta modelagem, é possível que a saída de um estado possa ser baseada em alguma condição, gerando mais de um caminho de saída. A estrutura da figura 24.1 pode ser alterada para que a escrita da serial só aconteça quando houver algum comando (do teclado ou da serial). Dessa forma, tem-se uma nova máquina de estados, como pode ser visto na figura 24.2:

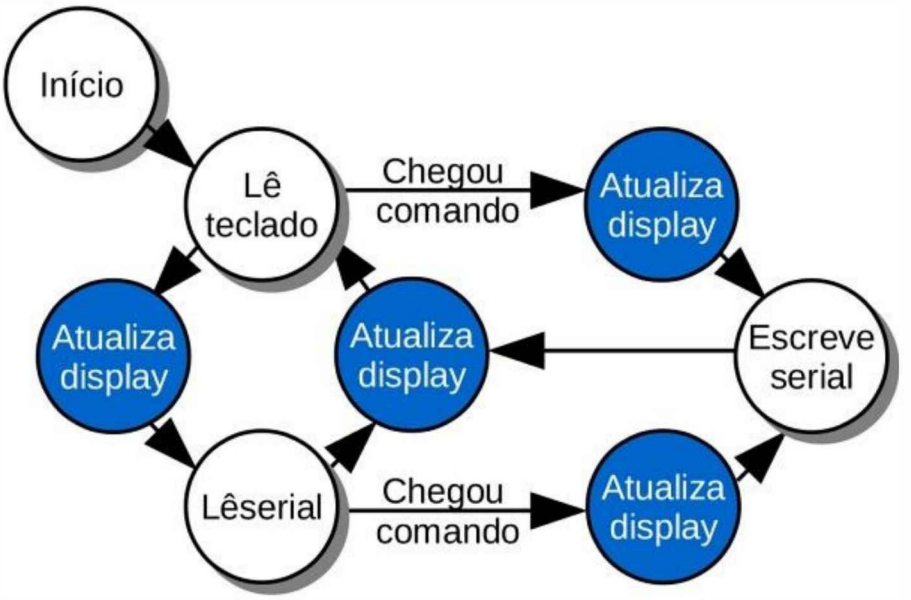


Figura 24.2: Exemplo de máquina de estados com condições

A transposição de uma máquina de estados para o código em C pode ser feita facilmente através de uma estrutura switch-case. Cada estado da máquina é representado por um case e a mudança para o novo estado é baseada nas condições de saída do estado atual, como mostrado no trecho de código 24.3.

Com esta arquitetura, a inserção de uma nova tarefa é feita de maneira simples, bastando adicionar outro estado, ou seja, basta inserir no código implementado um novo **`case`** com a tarefa desejada. Se este estado possuir mais de uma saída, é necessário fazer o teste de condição e indicar qual é o próximo estado a ser executado.

Como a máquina de estados está dentro do loop infinito, a cada vez que o programa passar pelo case, ele executará apenas um estado (atual). No entanto, esta estrutura de código gera um efeito interessante.

Como pode ser visto no código 24.3, naturalmente surgem duas regiões: o top-slot e o bottom-slot. Se algum código for colocado nesta região ele será executado toda vez, de modo intercalado, entre os slots. Pela figura da máquina de estados, percebe-se que é exatamente este o comportamento requerido para a função **`ssdUpdate()`**. Deste modo, pode-se então remodelar o código fazendo a alteração, como visto no código 24.4.

Código 24.3: Máquina de estados em linguagem C

```
1 void main(void){
2     //declaração das variáveis
3     char slot;
4     //funções de inicialização
5     serialInit();
6     kpInit();
7     ssdInit();
8     for(;;){    //início do loop infinito
9     //*************** início do top-slot ******************
10    //***************** fim do top-slot  ******************
11    //************ início da máquina de estado ************
12        switch(slot){
13            case 0:
14                if( kpRead() != 0){
15                    slot = 4; //chegou comando
16                }else{
17                    slot = 1;
18                }
19            break;
20            case 1:
21                ssdUpdate();
22                slot = 2;
23            break;
24            case 2:
25                if( serialRead() != 0){
26                    slot = 4; //chegou comando
27                }else{
```

```
28                      slot = 3;
29                  }
30              break;
31              case 3:
32                  ssdUpdate();
33                  slot = 0;
34              break;
35              case 4:
36                  ssdUpdate();
37                  slot = 5;
38              break;
39              case 5:
40                  serialSend('-');
41                  slot = 1;
42              break;
43              default:
44                  slot = 0;
45              break;
46          }
47      //************ fim da máquina de estado **************
48      //*************** início do bottom-slot ******************
49      //***************** fim do bottom-slot  ******************
50      } //fim for(;;)
51  }
```

Código 24.4: Uso do top slot/bottom slot

```
1 void main(void){
2     //declaração das variáveis
3     char slot;
4     //funções de inicialização
5     serialInit();
6     kpInit();
7     ssdInit();
8     for(;;){    //início do loop infinito
9     //*************** início do top-slot ******************
10    ssdUpdate();
11    //**************** fim do top-slot  ******************
12    //************ início da máquina de estado ************
13        switch(slot){
14            case 0:
15                if( kpRead() != 0){
16                    slot = 2; //chegou comando
17                }else{
18                    slot = 1;
19                }
20            break;
21            case 1:
22                if( serialRead() != 0){
23                    slot = 2; //chegou comando
24                }else{
25                    slot = 0;
26                }
27            break;
28            case 2:
29                serialSend('-');
30                slot = 1;
31            break;
32            default:
33                slot = 0;
34            break;
35        }
36    //************ fim da máquina de estado **************
37    //*************** início do bottom-slot ****************
38    //**************** fim do bottom-slot  *****************
39    } //fim for(;;)
40 }
```

Esta abordagem é conhecida também como cooperative multitasking, onde as funções podem demorar o tempo que quiserem e apenas quando terminam sua execução, a próxima

função começa a ser executada. Além disso, a frequência com que cada função será executada não pode ser determinada facilmente. É necessário, portanto, conhecer o tempo de execução de cada função do sistema. É possível remodelar a máquina de estados de acordo com a figura 24.3, explicitando o top-slot fora do loop, já que ele é executado toda rodada.

No entanto, é possível transformar esta arquitetura em um modelo temporizado com o uso de um timer. Assim, as frequências de execução podem ser facilmente definidas.

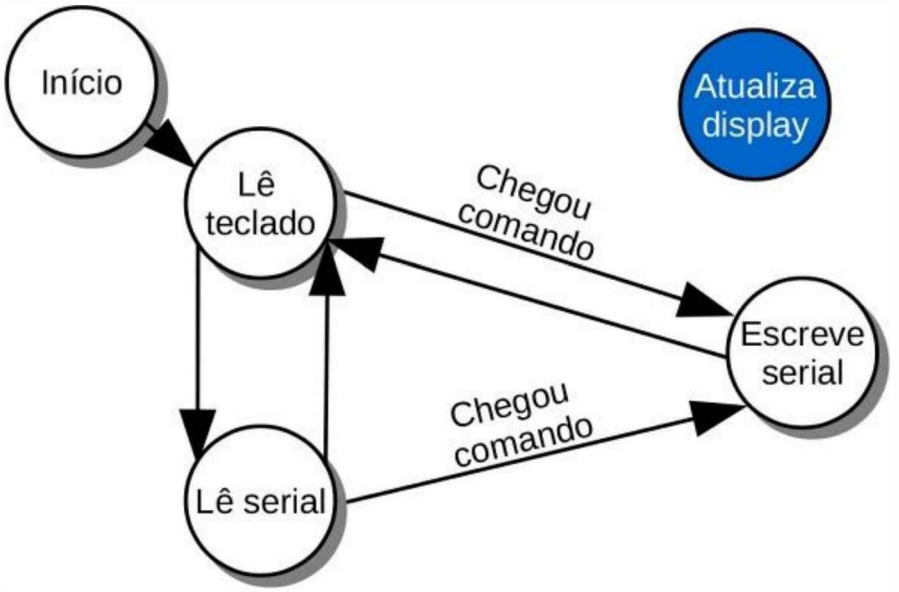


Figura 24.3: Exemplo de máquina de estados condicional com *top-slot*

Fixação de tempo para execução dos slots

No modelo de máquina de estados, assim que uma tarefa termina, o processador automaticamente passa para a próxima tarefa, independentemente de questões temporais. No entanto, não se tem qualquer garantia de tempo de execução, pois, os tempos variam de acordo com o tamanho da tarefa. Um bom exemplo é a recepção de comandos pela serial. Nem sempre o sistema recebe bytes e, mesmo quando recebe, ele precisa aguardar todos os bytes chegarem para que a mensagem seja processada. Assim, a função de recepção tem pelo menos três durações diferentes: quando não há nada na serial; quando há informação a ser salva na serial; e quando há uma mensagem para ser processada.

Uma característica desejada nos sistemas embarcados é que as funções tenham um tempo determinado de execução. Deste modo, todo o sistema se tornaria mais previsível.

A maneira mais simples de realizar este procedimento é fazer com que, nas vezes que a função for executada e terminar antes, o sistema faça com que ela aguarde até um determinado período de tempo. Este período tem que ser maior que o pior caso de execução da função.

Apesar de não ser o ideal, uma vez que o processador irá ficar tempo ocioso apenas para padronizar os tempos de execução das funções, o determinismo atingido por esse método traz imensas vantagens ao desenvolvimento. Outra vantagem é a possibilidade de criação de uma arquitetura que faça essa garantia consumindo poucos recursos, ou seja, esta é uma

abordagem possível de se implementar em sistemas que não são capazes de suportar um sistema de tempo real, como o FreeRTOS.

Para que essa implementação seja possível, basta ter acesso a um temporizador em hardware. É necessário inicializar a contagem no top-slot da arquitetura de máquina de estados e aguardar o fim da contagem no bottom-slot. Deste modo, toda vez que um slot terminar, o sistema ficará aguardando o tempo definido antes de iniciar o próximo slot.

Código 24.5: Máquina de estados temporizada

```
 1 void main(void){
 2     //declaração das variáveis
 3     char slot;
 4     //funções de inicialização
 5     ssdInit();
 6     kpInit();
 7     timerInit();
 8     for(;;){    //início do loop infinito
 9     //*************** início do top-slot ******************
10         timerStart(5000);   //5-ms para cada slot
11         ssdUpdate();
12     //**************** fim do top-slot  ******************
13     //*********** início da máquina de estado ************
14         switch(slot){
15             case 0:
16                 kpRead();
17                 slot = 1;
18             break;
19             case 1:
20                 serialRead();
21                 slot = 2;
22             break;
23             case 2:
24                 serialSend('-');
25                 slot = 0;
26             break;
27             default:
28                 slot = 0;
29             break;
30         }
31     //************ fim da máquina de estado **************
32     //*************** início do bottom-slot ****************
33         timerWait();
34     //**************** fim do bottom-slot  *****************
35     } //fim for(;;)
36 }
```

No exemplo apresentado no código 24.5, foi inserida a função **`timerWait()`** no bottom-slot de modo que a próxima função só executará em 5ms.

Pode-se notar que, se a função ultrapassar 5ms, todo o cronograma será afetado, sendo necessário garantir que todo e cada slot será executado em menos de 5ms. Isto pode ser ser feito através de testes de bancada.

Supondo que a tarefa 1, **`LeTeclado()`**, gaste um tempo de 2.0ms, a tarefa 2 por sua vez,

`RecebeSerial()`, consuma 3.1ms e a tarefa 3, **`EnviaSerial()`**, apenas 1.2ms; o top-slot, por sua vez, 0.5ms e o bottom-slot requeira um tempo de 0.3ms para ser executado, nesta situação, a representação da linha temporal de execução do sistema pode ser vista na figura 24 .4.

Pode-se notar que, para o ciclo do primeiro slot, são gastos 0.5 + 2.0 + 0.3 = 2.8ms. Deste modo, o sistema fica aguardando na função **`timerWait()`** durante 2.2ms sem realizar nenhum processamento útil. Para o segundo slot, tem-se um tempo livre de (5 - (0.5 + 3.1 + 0.3)) = 1.1ms. O terceiro slot é o que menos consome tempo de processamento, possuindo um tempo livre de (5 - (0.5 + 1.2 + 0.3)) = 3.0ms.

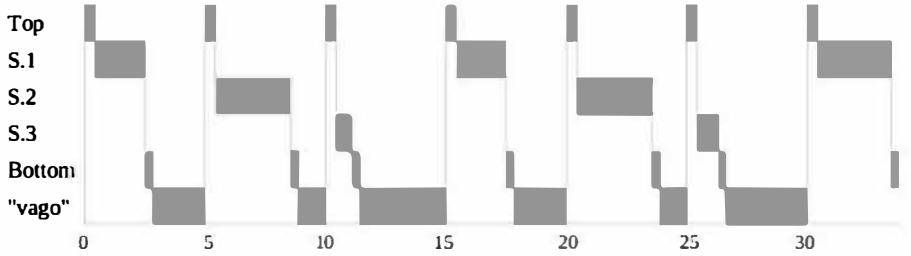


Figura 24.4: Exemplo da mudança de slots no tempo

Utilização do tempo livre para interrupções

Dependendo do tempo escolhido para o slot e do "tamanho" da função, podem existir espaços vagos na linha de tempo do processador. A figura 24.5 apresenta uma linha de tempo de um sistema que possui apenas 1 slot:

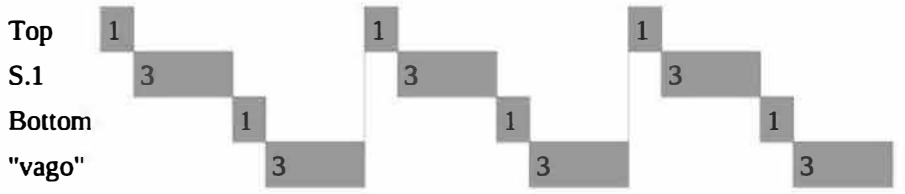


Figura 24.5: Linha de tempo de um sistema com 1 slot

A cada ciclo "sobram" 3ms. Este tempo pode ser considerado perdido caso exista apenas a tarefa S.1 no sistema. Nesta situação, é possível diminuir o tempo do temporizador para 5ms. Isto permitiria um acréscimo à frequência de execução da tarefa S.1, que era de 125 (Hz) para 200 (Hz), ou 60% de ganho.

No entanto, não havendo tempo livre, qualquer alteração acabaria com o determinismo, além de impossibilitar o uso de interrupção sem atrapalhar a frequência de execução. Portanto, esse tempo livre é importante por dois motivos: evitar que pequenas alterações não esperadas na execução das funções impactem na taxa de execução e, mais importante ainda, permitir que as interrupções aconteçam sem causar problema à temporização das tarefas.

A figura 24.6 mostra o comportamento do mesmo sistema sendo interrompido através de interrupções assíncronas. Neste caso, a interrupção "rouba" 1ms a cada iteração da máquina de estados.

Como cada interrupção gasta um tempo de 1ms e tem-se um tempo livre de 3ms em cada ciclo, basta garantir que os eventos que geram a interrupção não ultrapassem a frequência de 3

eventos a cada 8ms. Esta análise deve ser feita para assegurar que o sistema, apesar de perder tempo de processamento, tenha um modo simples de garantir o determinismo na execução das funções.

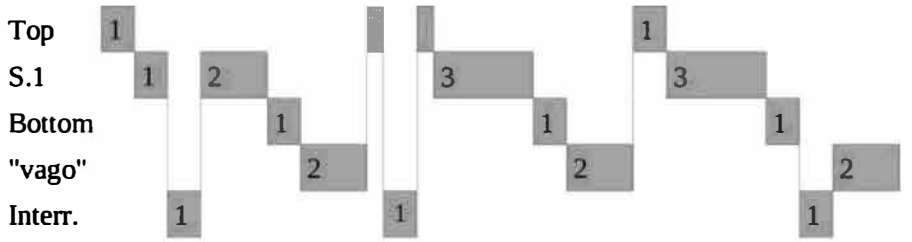


Figura 24.6: Comportamento da linha de tempo com interrupções

O tempo que o processador fica aguardando no bottom-slot é uma ótima oportunidade para colocar o sistema em baixo consumo de energia e permitir que ele seja acordado com a interrupção do timer. Através dessa economia de energia, o objetivo passa de ter uma alta taxa de execução sem tempo ocioso para possuir o máximo de tempo livre, reduzindo efetivamente o consumo.

24.4 Kernel

Em ciência da computação, o kernel é a camada de software de um sistema responsável por implementar a interface e gerenciar o hardware e a aplicação. Os recursos de hardware mais críticos a serem gerenciados são o processador, a memória e os drivers de entrada/saída (I/O).

Outra função comumente desempenhada pelo kernel é o gerenciamento de processos. Tal tarefa possui maior importância no contexto de sistemas embarcados, no qual, em geral, os processos possuem limitações de tempo de execução. Quando não há kernel, a responsabilidade de organizar os processos, o hardware e as aplicações é do programador.

Em geral, um kernel possui três principais responsabilidades:

1)Gerenciar e coordenar a execução dos processos através de algum critério.

Esse critério pode ser o tempo máximo de execução, prioridade, criticidade de um evento, sequência de execução, entre outros. É esse gerenciamento que diferencia um kernel conhecido como preemptivo de um cooperativo. No kernel preemptivo, cada processo possui tempo máximo para ser executado; se este limite for ultrapassado, o processo seguinte é iniciado e, quando este é finalizado ou estoura a faixa de tempo, o anterior volta a ser executado no ponto em que foi interrompido. Ja no kernel cooperativo, cada processo é executado completamente, de modo que um novo processo é chamado somente quando o anterior finalizar. Sendo responsável por gerenciar processos, o kernel deve possuir funções que permitam incluir novos processos ou remover antigos.

Como cada processo utiliza internamente uma certa quantidade de memória para suas variáveis, o kernel deve gerenciá-la. Esta é a segunda responsabilidade do kernel.

2)Manusear a memória disponível e coordenar o acesso dos processos a ela.

O kernel deve também ser capaz de informar ao processo quando uma função de alocação de memória não for executada. Além da memória, os processos precisam acessar os recursos de entrada e saída do computador/microcontrolador, como portas seriais, displays LCD, teclados, etc. A responsabilidade de permitir ou negar o acesso dos processos aos dispositivos de hardware é do kernel. Esta é a sua terceira responsabilidade.

3)Intermediar a comunicação entre os drivers de hardware e os processos.

O kernel deve fornecer uma API (Application Programming Interface, traduzida literalmente como Interface de Programação de Aplicativos), pela qual os processos podem seguramente acessar a informação disponível no hardware, tanto para operações de leitura como para operações de escrita.

Construir o próprio kernel pode facilitar o desenvolvimento das aplicações e, ainda, fornecer ao desenvolvedor total controle sobre o código gerado.

Com uma arquitetura de execução única (one-single-loop), é necessário refazer os testes a cada vez que se reutiliza o código. Quando o kernel está totalmente testado, não há problema no reuso. Até mesmo aplicações possuem uma maior chance de reaproveitamento em kernels que mantêm a camada de abstração de hardware, independentemente da mudança de um chip.

Quando se planeja utilizar um kernel no desenvolvimento de um novo sistema, é necessário sempre considerar todas as alternativas, tanto pagas quanto gratuitas ou com licenças do tipo opensource.

Existem muitas opções para migrar de um sistema baremetal, sem kernel, para um com kernel. Soluções pagas possuem benefícios, especialmente pelo suporte técnico oferecido. As soluções opensource, por sua vez, podem apresentar uma comunidade bastante ativa, com diversos exemplos prontos.

Tipos de kernel

O kernel é o componente central do sistema operacional da maioria dos computadores. Ele serve de ligação entre os aplicativos e o processamento de dados feito em nível de hardware. As responsabilidades do kernel incluem: gerenciar os dispositivos, a memória, os processos e as chamadas do sistema; ou seja, gerenciar a comunicação entre os componentes de hardware e software.

Como um componente básico do sistema operacional, um kernel pode oferecer uma camada de abstração de baixo nível para os recursos (processadores e dispositivos de I/O) que os aplicativos de software devem controlar para realizar alguma função. O kernel torna essas facilidades disponíveis para os processos de aplicativos através de mecanismos de comunicação entre processos e chamadas de sistema.

É possível classificar os tipos de kernel em 4 categorias.

`Monolítico`: é o modelo em que a maioria dos seus recursos é executada pelo próprio kernel, no espaço reservado para carregar o kernel e para que o kernel realize suas funções. Em comum com as outras arquiteturas, o kernel define uma camada de alto nível de abstração sobre o hardware do computador, com um conjunto de primitivas ou chamadas de sistema para implementar os serviços do sistema operacional, como o gerenciamento de processos, concorrência e gestão de memória, em um ou mais módulos.

Mesmo que cada módulo de manutenção dessas operações seja separado de uma forma geral, é muito difícil fazer o código de integração entre todos estes módulos, e, uma vez que todos os módulos executam num mesmo espaço de endereçamento, um erro em um módulo pode derrubar todo o sistema.

Uma vantagem do kernel monolítico é uma melhor segurança e um melhor desempenho, uma vez que seus recursos residem dentro do próprio kernel. Por isso, seus recursos estarão sempre em execução, do momento em que ligar o computador até o momento de desligá-lo, consumindo recurso do hardware.

A principal característica do kernel monolítico é permitir que funções como rede, vídeo e acesso a outros periféricos sejam possíveis através do kernel space. Isso é possível através do uso de módulos. O que significa que um módulo, apesar de não estar no mesmo código do kernel, é executado no espaço de memória do kernel. Sendo assim, apesar de modular, o

kernel monolítico continua sendo único e centralizado. Isso pode levar a considerações errôneas sobre o conceito. Exemplos de kernel monolítico: o Linux, BSD e alguns Windows.

Microkernel: é uma arquitetura cujas funcionalidades são quase todas executadas fora do kernel, em oposição a um kernel monolítico. É um kernel minúsculo, que trabalha somente com o mínimo de processos possíveis, essenciais para manter o sistema em funcionamento, executando-os no kernel space. No kernel space, os aplicativos têm acesso a todas as instruções e a todo o hardware. Todos os demais processos são executados por daemons, conhecido como servidores de forma isolada e protegidos no user space.

O microkernel consiste em definir abstrações simples sobre o hardware, com um conjunto de primitivas ou chamadas de sistema, para implementar serviços mínimos do sistema operacional, como gerenciamento de memória, multitarefas e comunicação entre processos. Outros serviços, incluindo aqueles normalmente fornecidos por um kernel monolítico como rede, são implementados em programas de user space, conhecidos como servidores. Microkernels são mais fáceis de manter em relação ao kernel monolítico, mas um grande número de chamadas de sistemas de trocas de contexto pode desacelerar o sistema porque eles geralmente geram mais degradação no desempenho do que simples chamadas de função.

O sistema de um microkernel é dividido da seguinte forma: servidor I/O, servidor de memória, servidor de gerenciamento de processos, servidor de sistema de arquivos, servidor de device drivers. Esses servidores se comunicam com o microkernel; o sistema monitora continuamente cada um destes processos e, se uma falha for detectada, ele substitui automaticamente este processo defeituoso, sem reiniciar a máquina, ou seja, sem perturbar os outros processos em execução e, principalmente, sem que o usuário perceba.

Um microkernel permite a implementação das partes restantes do sistema operacional, como aplicativos normais escritos em linguagens de programação de alto nível, e o uso de diferentes sistemas operacionais sobre o mesmo kernel não modificado. Ele também torna possível alternar dinamicamente entre sistemas operacionais e manter mais de um deles ativos simultaneamente São exemplos de microkernel o Hurd e o Minix.

Nanokernel: um nanokernel delega virtualmente todos os serviços, incluindo até os mais básicos, como controlador de interrupções ou o temporizador, para drivers de dispositivo a fim de tornar o requerimento de memória do kernel ainda menor do que o dos tradicionais microkernels.

Exokernel: é um tipo de kernel que não abstrai hardware em modelos teóricos. Ao invés disso, ele aloca recursos físicos de hardware, como o tempo de um processador, páginas de memória e blocos de disco, para diferentes programas. Um programa rodando em um exokernel pode ligar para uma biblioteca do sistema operacional que usa o exokernel para simular as abstrações de um sistema operacional conhecido, ou ele pode desenvolver abstrações específicas para aquele aplicativo para um desempenho superior.

24.5 Sistemas operacionais

O sistema operacional, SO, é um conjunto de códigos que funciona como uma camada de abstração do hardware provendo funcionalidades para as aplicações de alto nível. Este isolamento permite que a aplicação não sofra alteração quando há mudança no hardware. Esta é uma característica muito desejada em sistemas embarcados, onde existe uma pluralidade de tipos de periférico, dificultando a reutilização de código.

De modo geral, os sistemas operacionais possuem três principais responsabilidades:

- manusear a memória disponível e coordenar o acesso dos processos a ela;
- gerenciar e coordenar a execução dos processos através de algum critério;
- intermediar a comunicação entre os periféricos de hardware e os processos.

Estas responsabilidades se relacionam com os três recursos fundamentais de um sistema computacional: o processador, a memória e os dispositivos de entrada e saída. A figura 24.7 ilustra estes recursos, bem como o papel de interface que um sistema operacional deve realizar.

A ausência de um sistema operacional implica que toda a responsabilidade de organizar o andamento dos processos, os acessos ao hardware e o gerenciamento da memória seja do programador. Este aumento de responsabilidade, a baixa capacidade de reutilização de código, e a consequente necessidade de recriar os códigos e rotinas, podem ser causadores de erros nos programas.

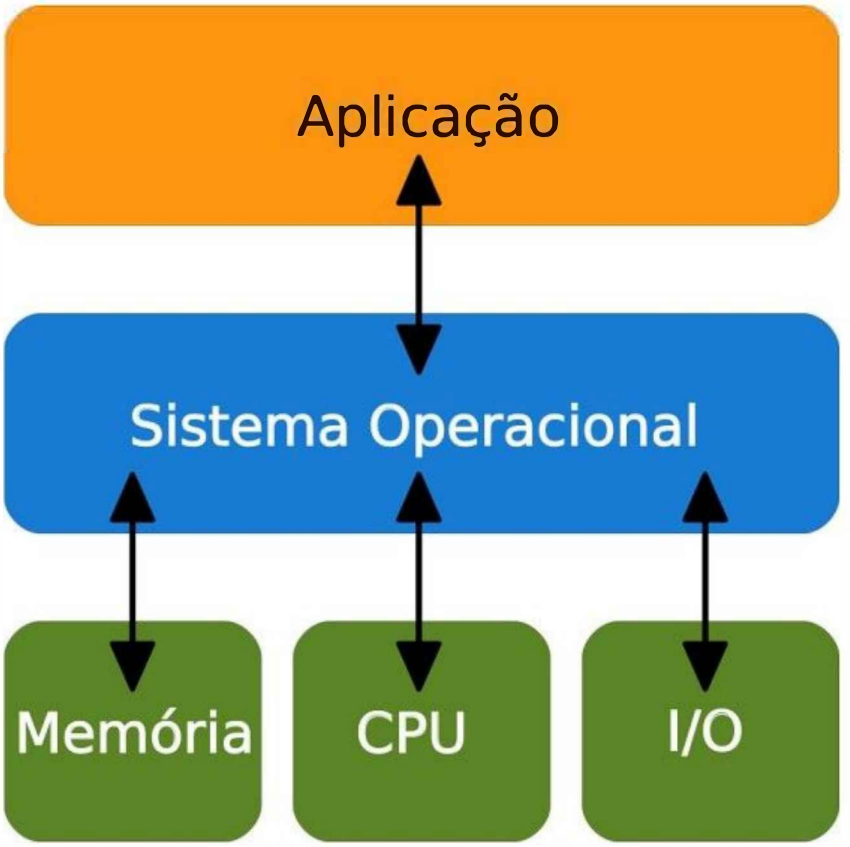


Figura 24.7: Interfaceamento realizado pelo sistema operacional

A capacidade de reutilizar os programas é benéfica por dois pontos principais: diminui o tempo de entrega do projeto e permite que o programador utilize melhor o tempo, eliminando os erros ao invés de recriar os códigos.

A figura 24.8 apresenta com mais detalhes os componentes de um sistema operacional. Nota-se que o núcleo de um SO é o kernel, e, do mesmo modo que o sistema operacional realiza a interface entre a aplicação e o hardware, o *kernel* faz a interface entre os códigos de acesso ao hardware, conhecidos como drivers, e as ferramentas disponibilizadas para que o programador crie as aplicações.

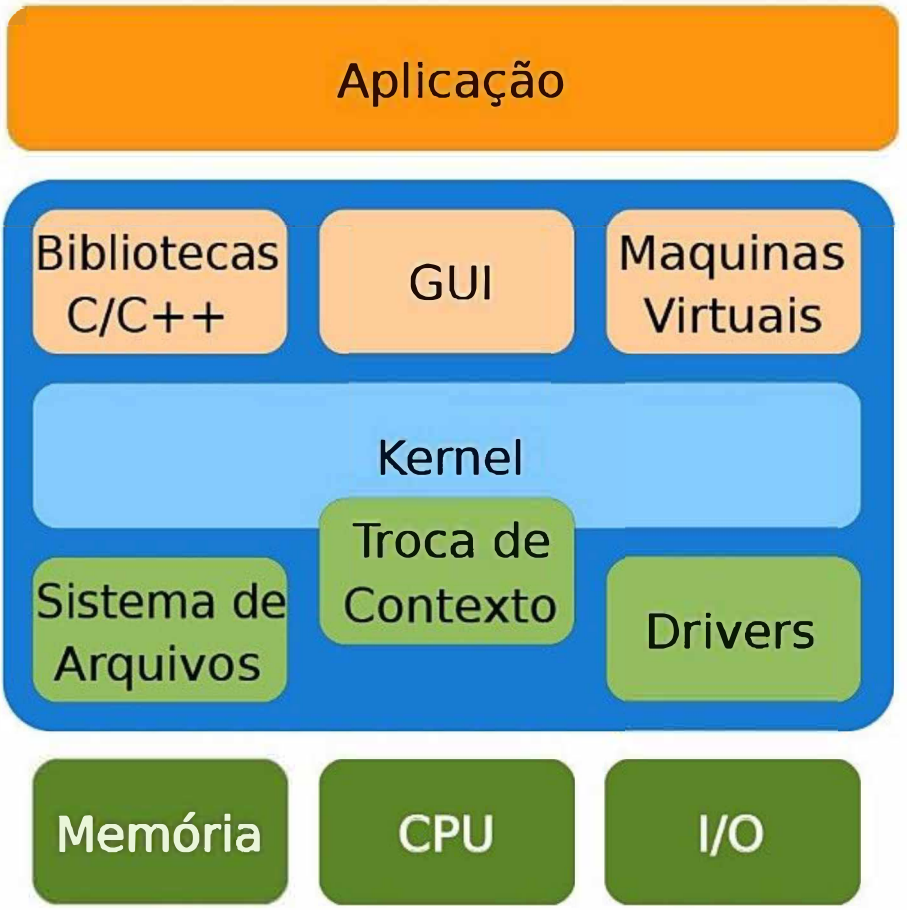


Figura 24.8: Relação entre troca de contexto e o *kernel*

As aplicações, na presença de um sistema operacional, são implementadas como processos, que passam a ser gerenciados pelo *kernel*. A sequência em que os processos são executados fica a cargo de algoritmos conhecidos como escalonadores, que, por sua vez, dependem de um procedimento de troca de contexto para efetuar a mudança de qual processo será executado. Estes três conceitos: processos, escalonador e troca de contexto, serão aprofundados nos próximos itens.

Processo

Na utilização de um sistema operacional, as tarefas a serem executadas pelo processador são organizadas em programas. O programa é uma sequência de comandos ordenados com uma finalidade específica. No momento em que este programa estiver em execução no processador ele passa a ser definido como processo.

Além do código a ser executado, os processos necessitam de posições de memória extras para armazenar seus dados e variáveis, sejam eles persistentes ou não. São necessárias também regiões de memória, geralmente implementadas em estrutura de pilha, para armazenamento de informações referentes à sequência de execução do programa.

Para realizar o correto gerenciamento dos processos é necessário que o kernel possua informações sobre os mesmos, agrupadas de maneira consistente. As informações mínimas necessárias são:

• O código a ser executado;
• As variáveis internas do processo;
• Ponteiros para as informações anteriores, permitindo sua manipulação.

Em geral, o código fica numa memória não volátil por questões de custo. Para microcontroladores, essas memórias são implementadas em tecnologia EEPROM, ou flash. Já as variáveis são armazenadas em memória volátil, pela maior velocidade de acesso e facilidade de escrita. As duas tecnologias mais utilizadas para este tipo de memória são a SRAM e a DRAM.

O armazenamento das informações de um processo, de modo automático e incremental, em estruturas do tipo pilha, permite que um mesmo programa possa ser executado mais de uma vez sem que nenhuma das instâncias de execução tenha suas variáveis modificadas indevidamente. Na figura 24.9 é apresentada uma situação onde dois processos compartilham o mesmo código.

Além da possibilidade de múltiplas execuções utilizando-se da estrutura de pilha, as trocas de contexto são implementadas de maneira muito simples, principalmente se cada processo possuir sua própria região de pilha, bem definida e separada das demais.

Escalonadores

Uma das funções principais do kernel de um sistema operacional é o gerenciamento dos processos em execução. Tal tarefa possui maior importância no contexto de sistemas embarcados, nos quais os processos costumam apresentar restrições rígidas quanto ao atraso de execução.

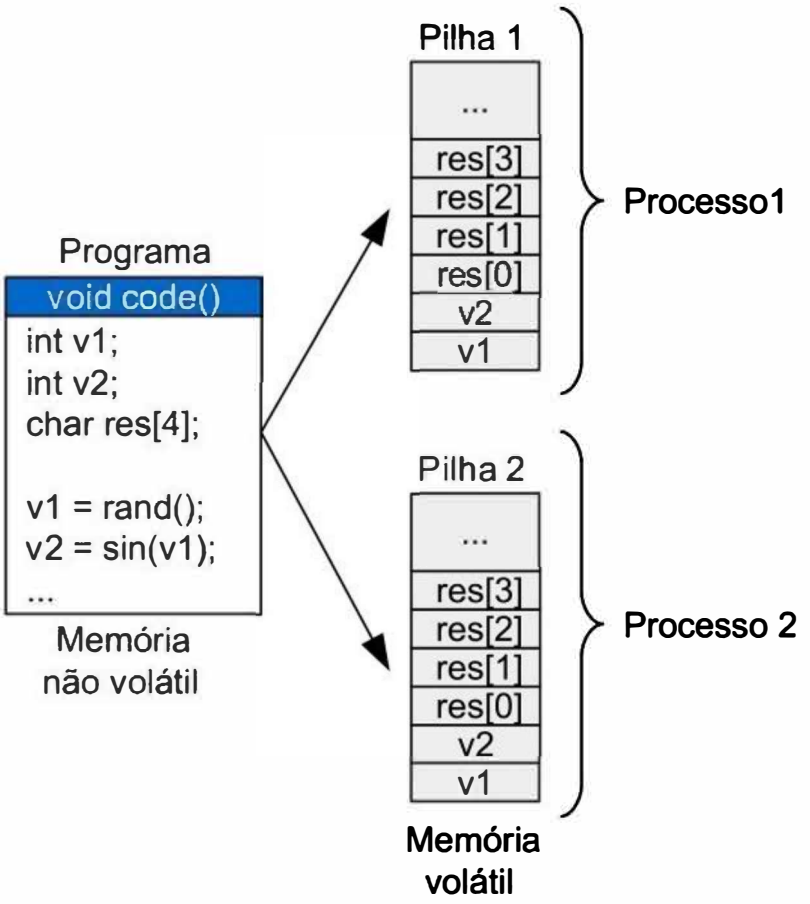


Figura 24.9: Geração de dois processos de um mesmo código de programa

Os algoritmos responsáveis por gerenciar e decidir qual dos processos será executado pelo processador são conhecidos como escalonadores. Existem diversas abordagens diferentes para realizar este gerenciamento. Em geral, os algoritmos visam equilibrar o atraso entre o início da execução e a quantidade de processos executados por unidade de tempo. Outros parâmetros importantes para a comparação dos escalonadores são o consumo extra de processamento (CPU overhead), a quantidade de processos executados por unidade de tempo (throughput), o tempo entre a submissão de um processo e o fim da sua execução (turnarround time) e o tempo entre a submissão do processo e a sua primeira resposta válida (response time).

Na tabela 24.1 são apresentados quatro algoritmos de escalonamento e as características destes.

Nota-se, pela tabela 24.1, que não existe alternativa ótima, sendo necessário escolher então a que mais se ajusta ao sistema que será desenvolvido.

Para um sistema de tempo real, o tempo de resposta é um dos quesitos mais importantes, dado que a perda de um prazo pode impactar negativamente no funcionamento deste. Algumas aplicações podem falhar se estes requisitos não forem atendidos. Apesar disto, nem todos os processos em um sistema embarcado precisam deste nível de determinismo. Deste modo, um escalonador que permita ao programador escolher entre pelo menos dois níveis de prioridade (normal e tempo real) é uma boa alternativa.

Tabela 24.1: Algoritmos para escalonamento, vantagens e desvantagens

Algoritmo de escalonamento	*CPU Overhead*	*Throughput*	*Turnaround time*	*Response time*
Escalonador baseado em prioridade	Baixo	Baixo	Alto	Alto
Escalonador round-robin (RR)	Baixo	Médio	Médio	Alto
Deadline mais crítico primeiro (EDF)	Médio	Baixo	Alto	Baixo
Escalonador de fila multinível	Alto	Alto	Médio	Médio

A maioria dos sistemas operacionais de tempo real apresenta como opção implementada o escalonador baseado em prioridades. Este fato se deve ao baixo consumo, mas principalmente à capacidade deste sistema de garantir que os processos mais críticos serão sempre executados. Na Tabela 24.2 é apresentada a tendência entre vários sistemas operacionais, comerciais e de código aberto.

Tabela 24.2: Sistemas operacionais de tempo real e escalonadores utilizados

Sistema operacional	Escalonador	Preempção
BRTOS	Prioridade	Sim
eCos	Prioridade/filas multi-nível	Sim
FreeRTOS	Prioridade	Sim
Micrium uC/OSII	Prioridade/round robin	Sim
Salvo	Prioridade/por interrupções	Não
SDPOS	Prioridade/round robin	Sim
VxWorks	Prioridade	Sim

Os escalonadores podem ainda ser divididos entre cooperativos ou preemptivos. Os cooperativos executam os processos de forma sequencial deixando que o processo tome o

tempo que lhe for necessário para que complete sua execução. Deste modo, se um processo atrasar em sua execução todo o sistema é impactado.

Já os preemptivos são aqueles que conseguem pausar um processo que está em execução, salvar o estado atual do processo e do sistema, carregar o estado do sistema para um segundo processo e iniciar a execução deste último. Isto permite que, mesmo que um dos processos atrase, seja possível continuar a execução dos demais de acordo com os requisitos destes. O procedimento de substituir um processo em execução por outro é denominado troca de contexto.

24.6 Exercícios

Ex. 24.1 — Qual a função do loop infinito num sistema embarcado?

Ex. 24.2 — Como construir uma estrutura de loop principal de modo que a velocidade de execução do loop possa ser controlada?

Ex. 24.3 — Qual a vantagem do uso de interrupções num projeto embarcado?

Ex. 24.4 — Qual(is) a(s) diferença(s) de um sistema operacional convencional e um sistema operacional de tempo real?

Ex. 24.5 — Como é possível executar várias tarefas num sistema que possui apenas um processador?

CAPÍTULO

25

Desenvolvimento de um kernel cooperativo

25.1Buffers circulares
25.2Ponteiros para void
25.3Ponteiros de função
25.4Execução das tarefas
25.5Adição e reexecução de processos
25.6Exercícios

"Bons programadores usam seus cérebros, mas bons guias nos salvam de ter que pensar em cada um dos casos."

Francis Glassborow

Sistemas embarcados são executados em computadores que controlam dispositivos que geralmente não são considerados computadores e que não aceitam softwares instalados por usuários. Exemplos típicos são fornos de micro-ondas, aparelhos de TV, carros, aparelhos de dvd, entre outros. A principal propriedade que distingue os sistemas embarcados dos portáteis é a certeza de que nenhum software não confiável jamais será executado nele.

A arquitetura de hardware de um sistema embarcado pode conter um ou mais processadores, memórias, interfaces para periféricos e blocos dedicados. Os componentes são interligados por uma estrutura de comunicação que pode variar de um barramento a uma rede complexa. Os processadores podem ser de diversos tipos, dependendo da aplicação. No caso de sistemas contendo componentes programáveis, o software de aplicação pode ser composto por múltiplos processos, distribuídos entre diferentes processadores, e podem se comunicar através de vários mecanismos.

Para o desenvolvimento de um kernel, o desenvolvedor deve levar em consideração alguns itens antes do início do processo de desenvolvimento:

Gerenciamento de dispositivos I/O:

Uma das principais funções do SO é gerenciar os dispositivos e I/O ligados ao computador. É tarefa do SO enviar sinais, informando as ações que o usuário espera que o dispositivo realize; e tratar as interrupções e os erros gerados pelos dispositivos. Existem duas versões sobre o hardware de I/O: a dos engenheiros, que os veem como chips, ligações elétricas; e a visão dos programadores, que os veem como uma interface de programação para se comunicar com o dispositivo.

Os dispositivos de hardware precisam ser controlados para proporcionar a entrada e a saída de dados para o processador. Esse controle do hardware é realizado por meio de hardware (denominado controlador de hardware, e que segue padrões determinados pelo barramento, como, por exemplo, IDE, SCSI, USB etc.) e software apropriados. Para utilizar um dispositivo de hardware, é necessário conectá-lo à interface física do controlador de hardware. Em geral, o SO pode ter softwares controladores de dispositivo (driver de dispositivos). Periférico é dado como qualquer dispositivo de hardware conectado a um computador de forma a permitir a sua interação com o mundo externo.

Algumas questões precisam ser levadas em consideração para o gerenciamento de I/O, por exemplo: como o kernel implementará a interface do dispositivo? No interior do kernel? Utilizando drivers de dispositivos? Utilizará uma controladora de driver à parte ou estará implícito nas atividades do kernel? O acesso direto entre aplicação e driver será possível? Em qual caso? Em casos de dispositivos hot-plug, como o kernel carregará dinamicamente o driver?

O principal objetivo do software gerenciador de E/S é padronizar ao máximo o acesso e o controle dos dispositivos, permitindo a inserção de novos dispositivos no sistema operacional, sem a necessidade de um outro software auxiliar. Isso se torna uma tarefa bastante complicada devido à grande variedade, complexidade e particularidades dos dispositivos periféricos encontrados. Para facilitar isso, o software de E/S é geralmente dividido em camadas. Cada camada tem uma função bem definida para executar e uma interface também bem definida para as camadas adjacentes, por meio de uma série de operações comuns a todos os dispositivos. Uma forma de implementação dessa estrutura é dividir o software em quatro camadas, onde tem-se, a camada superior, sendo a E/S vista pelo usuário; a segunda camada, o software que enxerga a E/S da mesma forma, independentemente do dispositivo; a terceira camada, que serve como interface padrão para drivers; e a última (mais inferior), composta pelos drivers propriamente ditos.

Gerência de processos:

Um processo é um programa em execução. É mais do que o código do programa, que às vezes é chamado de seção de texto. Um processo inclui a pilha de processo, que contém dados temporários, e uma seção de dados, que contém variáveis globais. O sistema operacional é responsável pelas seguintes atividades em relação à gerência de processos: criar e excluir processos de usuário e de sistema; suspender e retomar processos; fornecer mecanismos para a sincronização de processos; fornecer mecanismos para a comunicação de processos; fornecer mecanismos para o tratamento de deadlocks.

A comunicação entre os processos pode ocorrer de duas formas: comunicação direta e comunicação indireta. Na comunicação direta, cada processo que deseja se comunicar precisa identificar o destinatário e o remetente da comunicação. Nesse caso, as primitivas send e receive são definidas. Na comunicação indireta, as mensagens são enviadas e recebidas através de caixas de correio ou portas.

Independente de a comunicação ser direta ou indireta, as mensagens trocadas pelos processos durante a comunicação são armazenadas temporariamente em uma fila temporária. Essa fila pode ser implementada da seguinte maneira: capacidade zero (a fila tem tamanho 0 e o canal de comunicação não pode ter mensagens em espera); capacidade limitada (a fila tem tamanho finito x, ou seja, no máximo x mensagens podem ser armazenadas) e capacidade ilimitada (a fila tem tamanho infinito, ou seja, qualquer número de mensagens pode ficar armazenado na fila).

Algumas questões que precisam ser levadas em consideração a respeito da gerência de processos: A troca de contexto do kernel será cooperativa ou preemptiva? Como um processo pode se comunicar com outros? Uma fila de mensagens será implementada? Possuirá uma

memória compartilhada? Como controlar seu acesso? Semáforos estarão disponíveis? Existe necessidade de se implementar checagem de prioridade dos processos?

Segurança do sistema:

Existe algum meio de segurança de hardware a ser utilizado (watchdog, memória protegida)? Será utilizada proteção hierárquica? Caso seja, a CPU fornece MMU (Memory Management Unit / Unidade de Gerenciamento de Memória) ou o desempenho pode diminuir com a checagem de proteção do software? O sistema deve tentar fechar e reiniciar um processo sem resposta automaticamente?

Tais decisões devem ser tomadas cuidadosamente, pois algumas delas não podem ser mudadas sem uma reescrita completa do código-fonte, enquanto outras podem ser adiadas durante o desenvolvimento do projeto.

Neste livro, será utilizado um microkernel cooperativo, não preemptivo, sem memória compartilhada e com uma controladora de drivers para isolar os dispositivos do kernel. Os processos serão agendados com base nas suas necessidades em termos de tempo/frequência de execução.

Nas próximas seções serão abordados conceitos computacionais que facilitarão o projeto e a implementação de um kernel.

25.1 Buffers circulares

Os buffers são espaços de memória cujo objetivo é armazenar dados temporários enquanto eles não forem utilizados.

Um exemplo é o buffer de comunicação serial. Neste tipo de comunicação os dados são enviados bit a bit. No entanto, para que a mensagem possa ser entendida, é necessário interpretar um conjunto de bits de uma única vez, geralmente em pacotes de 8 bits. Como eles chegam individualmente, é necessário armazená-los até que o pacote esteja completo, como pode ser visto na figura 25.1:

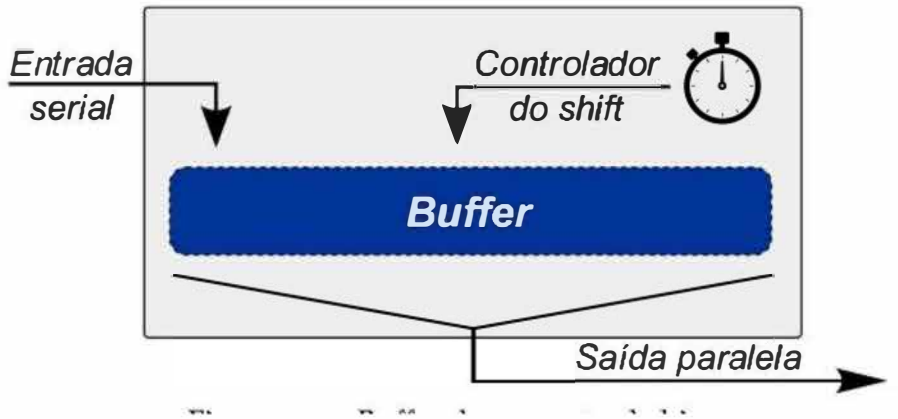


Figura 25.1: Buffer de recepção de bits

Dentro do controlador do shift existe um contador que indica quantos bits foram recebidos. No caso da maioria das comunicações UART, por exemplo, quando este contador chega a 8, ele indica que existe um novo dado disponível.

Já o buffer circular é um tipo especial de buffer. Ele possui um contador que, chegando ao seu limite, volta ao valor inicial.

O código 25.1 é um exemplo de como circular por um vetor tradicional um número infinito de vezes. Podemos notar que a variável **index**, com o uso do resto de divisão, irá de zero a **CIRCULAR_BUFFER_SIZE**, voltará a zero e continuará este processo enquanto o loop for verdadeiro.

Código 25.1: Circulando em um vetor infinito

```
 1 #define CIRCULAR_BUFFER_SIZE 12
 2 void main (void){
 3     int circular_buffer[CIRCULAR_BUFFER_SIZE];
 4     int index=0;
 5     for(;;){
 6         // utilizar o buffer para alguma finalidade
 7         circular_buffer[index] = index;
 8         // incrementa o índice
 9         index = (index+1)%CIRCULAR_BUFFER_SIZE;
10     }
11 }
```

O processo utilizado faz com que o buffer possa ser utilizado sem se preocupar com onde ele começa ou termina, caracterizando a ideia do buffer circular.

Para indicar quais posições estão disponíveis é possível utilizar dois índices, **start** e **end**, que indicarão o início e o fim dos dados. A figura 25.2 representa o vetor como um buffer circular, apresentado as informações de início, fim e índice do vetor.

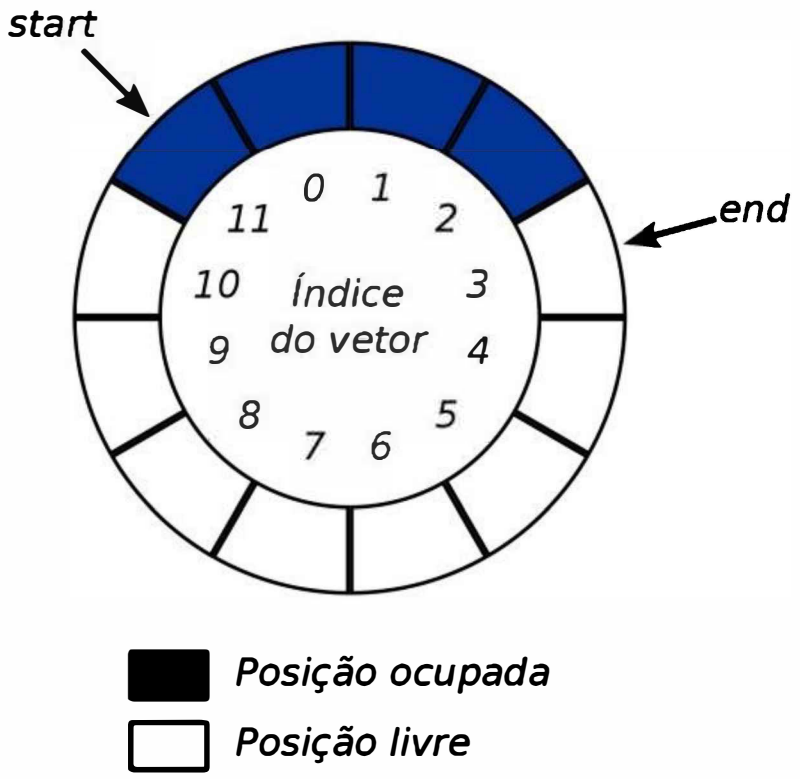


Figura 25.2: Geração de dois processos de um mesmo programa

O principal problema com o uso do vetor é definir quando ele está cheio ou vazio, pois em ambos os casos, os índices de início (**`start`**) e fim (**`end`**) apontariam para o mesmo lugar.

Existem várias alternativas para resolver este problema. Visando manter a simplicidade do sistema optou-se por sempre manter o último slot vazio. Neste caso, se o valor de **`start`** e **`end`** forem iguais, a lista está vazia.

A ideia do buffer é ser capaz de armazenar valores de modo temporário. Para isso, podemos criar funções que facilitem essa manipulação.

Para adicionar um elemento ao buffer, deve-se tomar cuidado em verificar se existe espaço disponível. O código 25.2 apresenta um exemplo de como implementar essa função.

Código 25.2: Buffer circular

```
 1 #define CB_SIZE 10
 2 int circular_buffer[CB_SIZE];
 3 int start=0;
 4 int end=0;
 5
 6 char AddBuff(int newData){
 7     // checa se há espaço para inserir um número
 8     if ( ((end+1)%CB_SIZE) != start){
 9         circular_buffer[end] = newData;
10         end = (end+1)%CB_SIZE;
11         return SUCCESS;
12     }
13     return FAIL;
14 }
```

25.2 Ponteiros para void

Para desenvolver o controlador de drivers de dispositivos, deve-se criar uma "central de distribuição de chamadas", que será responsável por receber um pedido da aplicação, via kernel, e redirecioná-la para o correto driver do dispositivo. O problema surge quando se planeja quantos parâmetros a função deve receber, como, por exemplo: um parâmetro representando qual driver é requisitado, outro representando qual função do driver será chamada e uma quantidade indeterminada de parâmetros necessários a serem passados para o driver. Como criar tal função?

Tal ação pode ser feita através de ponteiros para void. O ponteiro para void é um tipo de ponteiro de propósito geral que pode apontar para qualquer tipo de dado. É usado sempre que é necessário que uma função receba ou retorne um ponteiro genérico e opere independentemente do tipo de dado apontado. O conteúdo de uma variável apontada por um ponteiro void não pode ser acessado por meio desse ponteiro, sendo necessário criar outro ponteiro e fazer a conversão de tipo na atribuição.

Através do código 25.3 é possível notar como receber diferentes tipos utilizando a mesma função.

Código 25.3: Utilizando ponteiros para void

```
1 char * name = "Paulo";
2 char initial = 'A';
3 unsigned int children = 3;
4
5 void print(int option; void *parameter){
6     switch(option){
7         case 0:
8             lcdString((char*)parameter);
9             break;
10        case 1:
11            lcdChar(*((char*)parameter));
12            break;
13        case 2:
14            lcdNumber(*((unsigned int*)parameter));
15            break;
16    }
17 }
18 void main (void){
19    print(0, &name);
20    print(1, &weight);
21    print(2, &children);
22 }
```


25.3 Ponteiros de função

Um programa é um conjunto de instruções armazenado na memória, assim como seus dados. Por este motivo, é possível referenciar o endereço de uma função. Em C, o endereço de uma função é acessível ao programador através de uma variável do tipo ponteiro para função.

Ponteiros para funções podem ser passados como argumentos para outras funções, e a função apontada pode ser invocada a partir de seu ponteiro.

Em algumas situações, deseja-se que um programa possa escolher entre várias funções para executar uma determinada atividade. Por exemplo, um editor de imagens pode utilizar as funções Blur (borrar) ou Sharpen (aguçar) para realizar melhorias visuais em uma imagem. No código 25.4 são apresentados os protótipos dessas funções:

Código 25.4: Protótipo de funções para manipulação de imagens

```
1 //Função para borrar a imagem
2 image Blur(image nImg);
3 //Função para aguçar a imagem
4 image Sharpen(image nImg);
```

Pode-se construir uma engine simples de um editor de imagem, como no código 25.5:

Código 25.5: Engine simples de um editor de imagens

```
1  image imageEditorEngine(image nImg, int option){
2      image temp;
3      switch(option){
4          case 1:
5              temp = Sharpen(nImg);
6              break;
7
8          case 2:
9              temp = Blur(nImg);
10             break;
11     }
12     return temp;
13 }
```

Para adicionar outras funções, é necessário modificar o código da engine. Em geral, modificar o código significa mais possíveis erros e mais testes. Outra opção é tornar a engine um pouco mais genérica utilizando ponteiros de função. Um exemplo simples de sua utilização é visto no código 25.6, que mostra como é feita a declaração do ponteiro e sua utilização:

Código 25.6: Utilizando ponteiros de função

```
1 // definindo um ponteiro de função
2 typedef tipoDeRetorno (*nomeDoPonteiro)(tipoDoArgumento argumento);
3
4 // chamando uma função via ponteiro de função
5 variavelDeSelecaoDaFuncao = (*nomeDoPonteiro)(argumentoDeEntrada);
```

Esta é uma solução mais refinada do que a utilizada anteriormente. A nova engine pode ser visualizada no código 25.7:

Código 25.7: Engine genérica utilizando ponteiros de função

```
1 // declaração de ponteiro de função
2 typedef image (*ptrFunc)(image nImg);
3
4 // engine do editor de imagens
5 image imageEditorEngine(ptrFunc function, image nImg){
6     image temp;
7     temp = (*function)(nImg);
8     return temp;
9 }
```

A partir do código, pode-se notar que a função recebe um ponteiro de função como parâmetro. Deste modo, não é necessário se preocupar em adicionar novas funcionalidades à aplicação, pois o código principal permanecerá intacto. Um dos inconvenientes é que todas as funções agora necessitam da mesma "assinatura", ou seja, devem receber o mesmo tipo de parâmetro, na mesma ordem, e a variável de retorno deve ser do mesmo tipo.

Utilizando o conceito de ponteiros de função, é possível utilizar as funções Blur e Sharpen de uma maneira simplificada, como mostrado no código 25.8.

Código 25.8: Utilizando a engine genérica

```
1 //...
2 image nImage = getCameraImage();
3 nImage = imageEditorEngine(Blur, nImagem);
4 nImage = imageEditorEngine(Sharpen, nImagem);
5 //...
```

As funções são passadas como se fossem variáveis. Por ser essencialmente um ponteiro, deve-se dereferenciar a variável, como mostra o código 25.9, antes de se utilizar a função:

Código 25.9: Utilizando um ponteiro de função

```
1 temp = (*function)(nImg);
```

Pode-se também armazenar a função passada como parâmetro como uma variável convencional. Deste modo pode-se chamar a função posteriormente no programa (somente o ponteiro é armazenado; nenhum código é realmente copiado).

A sintaxe de declaração de um ponteiro de função é, de certa forma, complexa. Normalmente, se utilizam typedefs para tornar a implementação mais clara.

25.4 Execução das tarefas

Esta seção apresenta um primeiro exemplo, onde será criada a parte principal do kernel. Deve existir uma maneira de armazenar quais funções são necessárias para serem executadas e em qual ordem. Para concluir esta tarefa, serão utilizados ponteiros de funções, assim como mostrado no código 25.10:

Código 25.10: Declaração de ponteiros

```
1 // declaração do tipo ptrFunc: um ponteiro de função que não recebe nem retorna nada
2 typedef void(*ptrFunc)(void);
3
4 // vetor com os "processos"
5 static ptrFunc pool[4];
```

Os processos serão criados através de funções do tipo **ptrFunc**, que não recebem qualquer tipo de parâmetro e não retornam nada. No código 25.11 estão apresentadas três funções que apenas imprimem valores no LCD.

Código 25.11: Exemplo de processos

```
 1 void message(void){
 2     lcdCommand(0x80);
 3     lcdString("Teste kernel 1.0");
 4 }
 5 void keypad(void){
 6     kpDebounce();
 7     lcdCommand(0xC0);
 8     lcdNumber(kpRead());
 9 }
10 void serial(void) {
11     char rx;
12     rx = serialRead();
13     if(rx!=0){
14         lcdCommand(0xC6);
15         lcdChar(rx);
16     }
17 }
18 int globalCounter;
19 void counter(void) {
20     lcdCommand(0xC8);
21     lcdNumber(globalCounter);
22     globalCounter++;
23 }
```

Estes processos realizam diferentes atividades: escrever o próprio nome no LCD, imprimir a tecla pressionada ou o caractere recebido na serial e exibir o valor de um contador.

O kernel propriamente dito possui três funções: uma função de inicialização; uma função para adicionar novos processos, no pool de processos e uma terceira função para sua execução. É esperado que este kernel nunca termine sua execução e, para que isso seja possível, utiliza-se um loop infinito na função de execução. A primeira versão do kernel pode ser vista no código 25.12.

Código 25.12: Kernel - primeira versão

```
1 // variáveis internas do kernel
2 ptrFunc pool[4];
3 int end;
4
5 // protótipos das funções do kernel
6 void kernelInit(void);
7 void kernelAddProc(ptrFunc newFunc);
8 void kernelLoop(void);
9
10 // implementação das funções do kernel
11 void kernelInit(void){
12     end = 0;
13 }
14 void kernelAddProc(ptrFunc newFunc){
15     if (end <4){
16         pool[end] = newFunc;
17         end++;
18     }
19 }
20 void kernelLoop(void){
21     int i;
22     for(;;){
23         for(i=0; i<end;i++){
24             (*pool[i])();
25         }
26     }
27 }
```

Neste primeiro exemplo, mostrado no código 25.12, o kernel apenas executa as funções que lhe são dadas, na ordem em que são chamadas. Não existe outro tipo de controle, sendo o tamanho do pool de processos definido estaticamente.

Para utilizar o kernel deve-se seguir os três seguintes passos: inicializar o kernel; adicionar os processos desejados; e executá-lo. O código 25.13 mostra um exemplo de utilização do kernel:

Código 25.13: Utilizando a primeira versão do Kernel

```
 1 void main(void){
 2     initSystem();
 3     lcdInit();
 4     kpInit();
 5     serialInit();
 6     kernelInit();
 7     kernelAddProc(message);
 8     kernelAddProc(keypad);
 9     kernelAddProc(serial);
10     kernelAddProc(counter);
11     kernelLoop();
12 }
```

25.5 Adição e reexecução de processos

O kernel apresentado anteriormente executa os processos do pool de processos indiscriminadamente. Num sistema real, os processos podem necessitar de serem executados apenas uma vez. Além disso, o pool tem que ser capaz de receber novos processos mesmo com a placa em execução. Isto envolve a necessidade de adicionar e remover elementos do pool de processos.

Para implementar esses requisitos, o primeiro passo é transformar o vetor de processos num buffer circular. Deste modo fica mais simples adicionar/remover processos com o sistema em andamento. Todo novo processo é inserido no final do buffer. Os processos do início do buffer são executados e "removidos". Caso um processo queira ser reexecutado, basta reagendá-lo inserindo-o novamente, mas, desta vez, no fim do buffer. A definição do buffer é apresentada no código 25.14.

Código 25.14: Definição das variáveis do pool de processos

```
1 // informações do kernel
2 #define POOL_SIZE 4
3 static ptrFunc pool[SLOT_SIZE];
4
5 //primeiro elemento do buffer
6 int start;
7
8 //último elemento do buffer
9 int end;
```

Todos os processos agora retornarão um código, indicando se eles querem ser reexecutados ou não. Este código também pode ser utilizado para informar algum problema para o kernel. Os códigos de retorno e a nova declaração do **prtFunc** são dados a seguir:

```
1 // código de retorno
2 #define SUCCESS  0
3 #define FAIL     1
4 #define REPEAT   2
5
6 // declaração de ponteiro de função
7 typedef char(*ptrFunc)(void);
```

Não existe nenhuma mudança com relação à utilização do kernel. As mudanças estão todas relacionadas com os processos, que agora têm que retornar se querem ser reexecutados ou não. No código 25.15 é possível notar que apenas o processo **message** retorna **SUCCESS**. Todos os demais, como querem continuar executando indefinidamente, retornam **REPEAT**.

Código 25.15: Processos com código de retorno

```
 1 char message(void){
 2     lcdCommand(0x80);
 3     lcdString("Teste kernel 2.0");
 4     return SUCCESS;
 5 }
 6
 7 char keypad(void){
 8     kpDebounce();
 9     lcdCommand(0xC0);
10     lcdNumber(kpRead());
11     return REPEAT;
12 }
13
14 char serial(void) {
15     char rx;
16     rx = serialRead();
17
18     if(rx!=0){
19         lcdCommand(0xC6);
20         lcdChar(rx);
21     }
22     return REPEAT;
23 }
24
25 int globalCounter;
26 char counter(void) {
27     lcdCommand(0xC8);
28     lcdNumber(globalCounter);
29     globalCounter++;
30     return REPEAT;
31 }
```

A função de execução dos processos **kernelLoop()** do kernel é a que mais possui modificações. Agora é necessário verificar se a função executada deseja ser reagendada e agir de acordo, como pode ser visto no código 25.16:

Código 25.16: Função kernelLoop - segunda versão

```
1 void kernelLoop(void){
2     for(;;){
3         // Há algum processo a ser executado?
4         if (start != end){
5             // executa a primeira função e checa se há necessidade de reagendá-la
6             if ( (*(pool[start].Func))() == REPEAT){
7                 kernelAddProc(pool[start]);  //reagendando
8             }
9             // preparando para ir para o próximo processo
10            start = (start+1)%POOL_SIZE;
11        }
12    }
13 }
```

A função **AddProc()**, apresentada no código 25.17, deve verificar se existem ao menos dois slots disponíveis no buffer (lembre-se de que a última posição deve estar disponível o tempo todo) para que o processo possa ser inserido.

Código 25.17: Função AddProc - segunda versão

```
1 char kernelAddProc(ptrFunc newProc){
2     // checando a existência de espaço livre
3     if ( ((end+1)%SLOT_SIZE) != start){
4         pool[end] = newProc;
5         end = (end+1)%POOL_SIZE;
6         return SUCCESS;
7     }
8     return FAIL;
9 }
```

A rotina de inicialização, mostrada no código 25.18, apenas configura as variáveis **start** e **end** para a primeira posição.

Código 25.18: Função kernelInit

```
1 char kernelInit(void){
2     start = 0;
3     end = 0;
4     return SUCCESS;
5 }
```

25.6 Exercícios

Ex. 25.1 – Dado o vetor de ponteiros de função abaixo, faça um programa que execute ciclicamente todas as funções do vetor. A função deve receber como parâmetro um número indicando quantas vezes ela já foi executada. Considere que o vetor já está inicializado e preenchido com apenas 6 (seis) funções.

```
1 //definição do processo
2 typedef int (*ptrFunc)(char param);
3
4 //definição do pool de processos
5 #define POOLSIZE 10
6 ptrFunc pool[POOLSIZE];
```

Ex. 25.2 – Um processo é definido como uma função que recebe 1 parâmetro inteiro e retorna um char indicando se a função foi executada corretamente e se ela deseja ser reagendada. Crie um processo que é responsável por ligar um led do barramento de leds da porta A. O led é indicado pelo parâmetro inteiro recebido pela função. Depois de ligar o led o processo deve verificar se ele foi ligado corretamente. Se foi ligado deve retornar sucesso, se não, retorna falha.

```
1 //definição do processo
2 typedef char (*ptrFunc)(int n);
3
4 //definição dos códigos de retornro
5 #define SUCCESS   0
6 #define FAIL      1
7 #define REPEAT    2
```

Ex. 25.3 — Num dado sistema existem dois processos já implementados. O primeiro é responsável por verificar se existe algum dado na serial e armazená-lo num buffer. O segundo é responsável por verificar se o buffer possui alguma mensagem completa e executar o comando. Dado o kernel utilizado em aula, crie a função main que inicializa o sistema corretamente e adiciona os processos ao kernel.

```
1 //Funções disponíveis no kernel.h
2 char kernelInit(void);
3 char kernelAddProc(process* newProc);
4 void kernelLoop(void);
5 void KernelClock(void);
6
7 //Processos já implementados
8 char LeSerial(void);         //processo 1
9 char ProcessaBuffer(void);   //processo 2
```

Ex. 25.4 — Implemente um buffer circular de 10 posições com duas funções: uma para adicionar elementos e outra para remover elementos.

CAPÍTULO

26

Projeto de kernel com soft realtime

26.1O tempo real: soft e hard realtime
26.2Atendendo requisitos temporais
26.3Kernel cooperativo com soft realtime
26.4Exercícios

"Melhor três horas mais cedo do que um minuto mais tarde."

William Shakespeare

Boa parte dos sistemas embarcados precisa garantir que uma determinada função seja executada com uma certa frequência. Em alguns sistemas, não atingir esses requisitos temporais, ou deadlines, pode fazer com que o equipamento falhe. Estas falhas podem variar desde defeitos visuais de curta duração até a destruição do equipamento.

Em geral, utilizamos contadores dedicados, mais conhecidos como timers, ou relógios, para garantir os requisitos temporais.

No contexto de sistemas embarcados, a expressão tempo real descreve sistemas computacionais que estão sujeitos a restrições temporais em suas atividades. Estas restrições estão geralmente relacionadas ao tempo que o sistema deve responder a um determinado evento. Este tempo é comumente chamado de "deadline". Um sistema que não foi desenvolvido pensando nestes requisitos, geralmente não possui a capacidade de garantir uma resposta dentro de um prazo determinístico. Nestes sistemas, é possível, no máximo, apresentar um tempo esperado para as respostas.

26.1 O tempo real: soft e hard realtime

Para se desenvolver um sistema de tempo real é necessário utilizar algum conjunto de ferramentas que permita descrever a atividade do sistema e qual o prazo para cada uma das respostas. A ferramenta mais comum é fazer uso de um sistema operacional de tempo real.

Podemos classificar um sistema como de tempo real se o seu correto funcionamento não depende apenas de uma lógica correta, mas também do tempo consumido para executar tal lógica. Em geral, estes sistemas são classificados de acordo com as consequências do não atendimento do tempo (deadline):

- **`Hard`** - perder apenas um deadline faz com que o sistema falhe completamente.
- **`Firm`** - perdas não muito frequentes do deadline são toleradas, mas elas degradam a qualidade do serviço. Se o resultado da computação for obtido apenas depois do

deadline, ele não serve para nada.

- **Soft** - a utilidade do resultado da computação degrada com a perda do deadline, degradando a qualidade do sistema.

Sistemas hard realtime são comuns apenas em equipamentos específicos, principalmente na área médica, como marca-passos, na aviação, como sistemas de controle de voo ou área nuclear, como sistemas de monitoramento de variáveis de segurança.

Um bom exemplo de sistema firm realtime é a transmissão digital de som numa rede de celular. Se alguns poucos pacotes, espaçados no tempo, forem "perdidos", a mensagem como um todo não é impactada. Se as perdas forem frequentes, será difícil manter a comunicação. Se os pacotes "perdidos" chegarem atrasados, eles não serão mais necessários, já que, se eles forem utilizados, a comunicação começará a ter um atraso entre quando algo foi dito e quando foi ouvido.

Os sistemas soft realtime costumam estar em equipamentos mais simples. Podemos citar um ar-condicionado ou um aquecedor como exemplo. Esses equipamentos fazem a medição da temperatura ambiente e ligam a saída, com maior ou menor intensidade, dependendo da temperatura desejada. Em geral esses sistemas utilizam algum tipo de controle que precisa executar uma lógica em tempo determinístico.

Um atraso na execução desta lógica, para o caso de um simples controle de temperatura, fará com que o sistema não se comporte exatamente como projetado. Ele poderá consumir mais energia ou demorar mais para atingir a temperatura desejada. Mesmo sendo indesejada, esta situação não impede o funcionamento do sistema. Quanto maior o atraso da resposta, pior o sistema se comportará.

No entanto, estas definições não são precisas. Um mesmo sistema pode ser classificado como hard ou firm, dependendo das restrições impostas. Um controlador de temperatura, que acabamos de classificar como soft, poderia ser classificado como firmou, até mesmo, hard se for utilizado para controlar a temperatura de um processo químico onde a temperatura da substância se altera muito rapidamente.

Dadas essas questões, o objetivo de um sistema de tempo real é garantir que todos os tempos críticos sejam atendidos. Como o sistema é geralmente composto de várias atividades, o gestor das atividades, um kernel, por exemplo, pode ter como objetivo garantir que apenas algumas delas sejam atendidas plenamente, por serem mais críticas.

Para ser capaz de garantir hard realtime, o kernel tem que ser capaz de realizar a preempção das tarefas, evitando que problemas não antecipados possam atrapalhar os tempos. Com um kernel cooperativo, é possível construir um sistema de atendimento temporal que, usado corretamente, permite atingir níveis de soft realtime para várias aplicações.

A implementação mais simples para uma atividade que exija hard realtime é utilizar uma interrupção de timer. Deste modo, pode-se ter certeza de que a função sempre será iniciada no tempo correto. O único requisito é que o intervalo entre as interrupções seja suficiente para executar a função desejada.

26.2 Atendendo requisitos temporais

Há pelo menos três condições que necessitam ser satisfeitas para que possamos implementar um sistema de atendimento a requisições temporais no kernel:

1. Deve haver um tick (evento temporal) que ocorre com uma frequência precisa;
2. O kernel deve ser informado da frequência/período de execução dos processos;
3. A soma da duração dos processos deve se encaixar com o tempo ocioso do processador.

A primeira condição pode ser facilmente satisfeita se existir um temporizador ou relógio interno disponível que possa gerar uma interrupção. Tal fato é verdadeiro para a grande maioria dos microcontroladores. Nesse caso, não existe a necessidade de uma rotina de interrupção dedicada.

A segunda condição depende que o programador defina os tempos de cada processo e os disponibilize para o kernel. Isto pode ser feito adaptando-se a função de adição de processos, para que, além do ponteiro de função, ele passe a receber também o valor do tempo exigido.

A terceira condição depende inteiramente do sistema em si. Supondo um sistema cuja função **`ssdUpdate()`** precisa ser chamada em um intervalo de 5ms. Se o tempo de execução desta função for superior a 5ms, é impossível garantir o intervalo de execução. Outro quesito válido a ser considerado é a respeito do tipo de kernel: preemptivo ou cooperativo. Em um sistema cooperativo, o processo deve finalizar sua execução antes que outro possa ser processado pela CPU. Já em um caso preemptivo, o kernel pode parar a execução de um processo a qualquer momento para executar outro processo. Se um sistema não se adéqua ao tempo disponível, existem três opções: trocar o processador por um mais rápido, otimizar o tempo de execução dos processos ou redefinir as frequências necessárias para cada processo.

Quando trabalhamos com condições temporais utilizando um contador de tempo, um problema pode surgir, dependendo da implementação. Supondo um sistema com um contador de tempo de milissegundos com 16 bits não sinalizado, que pode conter valores de 0 a 65.535, que possua dois processos: P1 e P2. Se o contador atual (**`actual_time`**) estiver marcando 45.535, o primeiro processo for agendado para ocorrer após 10 segundos a partir do contador atual e o segundo processo para daqui a 50 segundos, teremos uma situação similar à da figura 26.1:

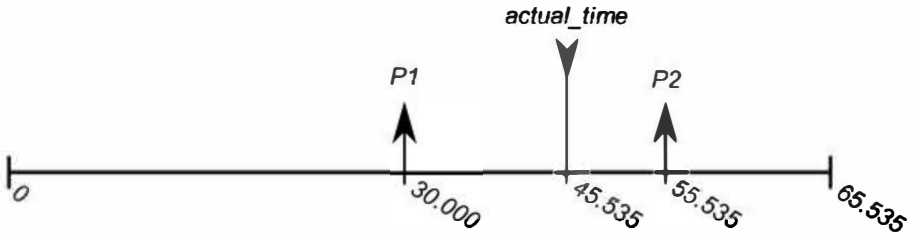


Figura 26.1: Visualização dos processos em fila em relação ao tempo

Pela figura 26.1, o processo P2 foi corretamente agendado para ser executado em 50 segundos, pois **`actual_time`** + 50.000 = 45.535 + 50.000 = 30.000; levando em conta o overflow do contador de tempo. O processo P1 será executado em 10 segundos, quando a variável **`actual_time`** atingir o valor 55.535.

O problema de usar um único contador finito para medir tempo aparece quando dois processos forem agendados para serem chamados em um pequeno espaço de tempo, ou até mesmo simultaneamente, como na figura 26.2. Nela podemos ver que ambos os processos, tanto P1 quanto P2, estão agendados para ocorrer exatamente em **`actual_time`** = 30.000, em dez segundos.

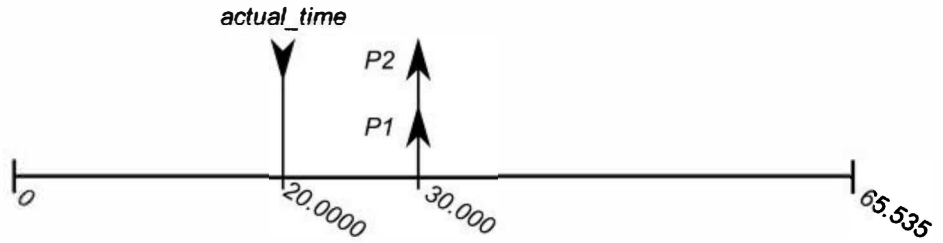


Figura 26.2: Processos adicionados para iniciarem no mesmo instante

Supondo que P1 seja chamado primeiro, independentemente do motivo, e que para executar toda a sua rotina interna, ele consuma 10 segundos. Após sua execução a linha do tempo pode ser visualizada como na figura 26.3.

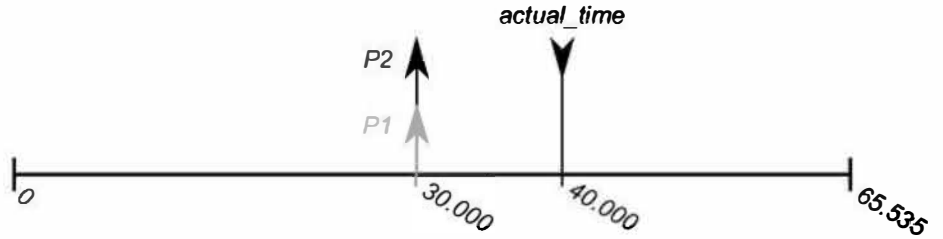


Figura 26.3: Definindo prioridade de processamento na fila

Observando apenas a linha da figura 26.3 do tempo (que é a única informação disponível para o kernel), podemos levantar a seguinte dúvida:

- O processo P2 já foi executado ou foi agendado para ocorrer daqui a 50 .535 ms, a partir de **`actual_time`**?

Para resolver esta situação, há duas opções:

1. Criar um flag para cada processo, indicando quando já foi passado pelo contador de tempo. Deste modo, é possível visualizar se um processo anterior ao contador está atrasado ou se foi agendado para ocorrer posteriormente.
2. Criar um contador que decrementa a cada tick do kernel. A execução do processo ocorrerá quando seu contador atingir o valor zero.

A segunda opção gera maior sobrecarga, pois é necessário decrementar todos os contadores dos processos. Por outro lado, se for permitido que o contador assuma valores negativos (de acordo com o que ocorreu com o decremento), pode-se examinar por quanto tempo o processo está aguardando. Com esta informação, pode-se tomar uma providência para se evitar que o processo continue atrasando, ou simplesmente informar ao usuário que o sistema apresenta algum problema.

Para que o kernel consiga atender os requisitos temporais é necessário que a informação do tempo (frequência/período de execução) esteja disponível. Uma solução é modificar o pool de processos para que, ao invés de conter apenas o ponteiro da função, contenha um ponteiro para uma estrutura com todos os dados do processo.

Além de o ponteiro de função, a estrutura também contém o período de execução. Além do período, cada processo terá seu próprio contador de tempo através da variável **`deadline`**. Ela servirá para indicar, para cada processo, quando seu tempo de espera terminou, o que diz ao kernel que o processo está pronto para ser executado novamente. Esta struct pode ser vista no código 26.1.

O contador **deadline**, será decrementado uma unidade a cada evento de tempo (**tick**) do kernel. Quando ele atingir o valor zero, o processo deve ser executado.

Como existem vários processos no pool, a função **kernelLoop()** deverá percorrer todos os slots ativos e encontrar o processo que, dentre todos os processos adicionados ao pool, deve ser executado agora.

É possível que nenhum dos processos esteja pronto para ser executado (nenhum processo possui **deadline<=0**). Caso isto aconteça, a **kernelLoop()** selecionará o processo com o menor tempo e ficará aguardando seu contador zerar.

Código 26.1: Struct do processo com as informações extras de tempo

```
1 // struct do processo
2 typedef struct {
3     //ponteiro para a função a ser executada
4     ptrFunc function;
5     //período de reexecução da função
6     int period;
7     //contador de tempo de cada processo
8     int deadline;
9 } process;
```

O código 26.2 apresenta a implementação do kernel com os requisitos temporais:

Código 26.2: Função kernelLoop() para requisitos temporais

```
1 //executa os processos do 'pool' de acordo com seus tempos de execução
2 void kernelLoop(void){
3     unsigned int count;
4     unsigned int next;
5     process* tempProc;
6     for(;;){
7         if (start != end){
8             //Procura a próxima função a ser executada com base no tempo
9             count = (start+1)%POOL_SIZE;
10            next = start;
11            while(count!=end){
12                if ((pool[count]->deadline) < (pool[next]->deadline)){
13                    next = count;
14                }
15                count = (count+1)%POOL_SIZE; //próximo processo
16            }
17            //troca processo de menor tempo como o primeiro
18            tempProc = pool[next];
19            pool[next] = pool[start];
20            pool[start] = tempProc;
21            while((pool[start]->deadline) > 0){
22                //coloca a cpu em modo de economia de energia
23            }
24            //retorna se precisa repetir novamente ou não
25            switch (pool[start]->function()) {
26                case REPEAT:
27                    kernelAddProc(pool[start]);
28                    break;
29                case FAIL:
30                    break;
31                default:
32                    break;
33            }
34            start = (start + 1) % POOL_SIZE; //próximo processo
35        }
36    }
37 }
```

O tempo gasto aguardando o processo, aparentemente sem utilidade, na verdade, é essencial para sincronizar todos os eventos. Além disto, esta abordagem permite que possamos colocar o sistema em modo de baixo consumo de energia durante o tempo em que se aguarda o próximo processo.

Para que o loop funcione é necessário que uma rotina externa realize os decrementos dos contadores de cada um dos processos ativos.

Para isso, podemos criar uma rotina **kernelTick()**, que será chamada a cada interrupção de tempo. Como a interrupção de tempo é bastante precisa, os ticks poderão ser utilizados como referência temporal para os processos. Esta rotina é mostrada no código 26.3, decrementando o campo **deadline** de todos os processos:

Código 26.3: Rotina de tratamento de interrupção

```
1 //atualiza os tempos de execução dos processos
2 void kernelTick(void){
3     int proc;
4     proc = start;
5     while(proc!=end){
6         if((pool[proc]->deadline)>(MIN_INT)){
7             pool[proc]->deadline--;
8         }
9         proc = (proc+1)%POOL_SIZE;
10    }
11 }
```

A função **kernelAddProc()**, mostrada no código 26.4, será responsável por, além de adicionar o processo ao kernel, inicializá-lo com um valor adequado para a variável **deadline**.

Código 26.4: Função AddProc - Exemplo 2

```
1 //adiciona os processos no pool
2 char kernelAddProc(process* func){
3     // adiciona processo somente se houver espaço livre
4     //o fim nunca pode coincidir com o inicio
5     if ( ((end+1)%POOL_SIZE) != start){
6         //adiciona o novo processo e agenda para executar imediatamente
7         func->deadline += func->period;
8         pool[end] = func;
9         end = (end+1)%POOL_SIZE;
10        return SUCCESS; //sucesso
11    }
12    return FAIL; //falha
13 }
```

Ao invés de resetar o contador **deadline** com o valor de **period**, optou-se por adicionar este valor ao **period**. Isto foi feito pensando em um possível atraso na execução do processo.

Em situações normais, quando o processo for executado, o contador **deadline** será exatamente zero. Caso o processo demore muito para executar, ou se outro processo evitou que ele executasse no tempo correto, ao fim de sua execução seu contador terá um valor negativo.

Ao adicionar o período a este valor negativo, garante-se que a reexecução do processo se dará com um intervalo igual ao planejado, baseado no início da última execução.

26.3 Kernel cooperativo com soft realtime

O desenvolvimento do kernel foi apresentado nos capítulos anteriores. Sua estrutura completa pode ser vista na figura 26.4:

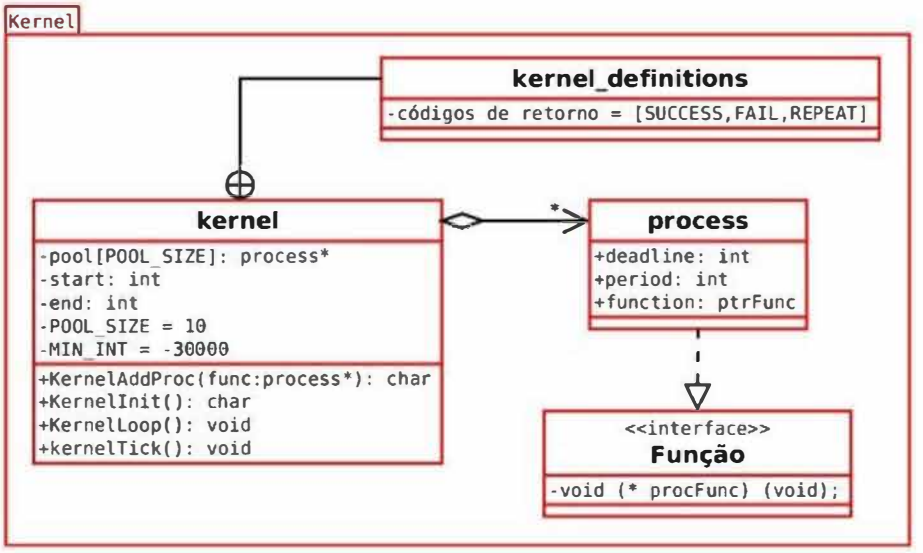


Figura 26.4: Diagrama da estrutura do kernel

A versão final do kernel cooperativo pode ser visualizada no código 26.5.

Código 26.5: Kernel completo - código

```
 1 #include "kernel.h"
 2
 3 #define POOL_SIZE 10
 4 #define MIN_INT -30000
 5
 6 static process* pool[POOL_SIZE];
 7 //primeiro elemento do buffer
 8 int start;
 9 //último elemento do buffer
10 int end;
11
```

```
12 //adiciona os processos no pool
13 char kernelAddProc(process* func){
14     // adiciona processo somente se houver espaço livre
15     //o fim nunca pode coincidir com o inicio
16     if ( ((end+1)%POOL_SIZE) != start){
17         //adiciona o novo processo e agenda para executar imediatamente
18         func->deadline += func->period;
19         pool[end] = func;
20         end = (end+1)%POOL_SIZE;
21         return SUCCESS; //sucesso
22     }
23     return FAIL; //falha
24 }
25 //inicializa o kernel em conjunto com a controladora de drivers
26 void kernelInit(void){
27     start = 0;
28     end = 0;
29 }
30 //executa os processos do 'pool' de acordo com seus tempos de execução
31 void kernelLoop(void){
32     unsigned int count;
33     unsigned int next;
34     process* tempProc;
35     for(;;){
36     if (start != end){
37         //Procura a próxima função a ser executada com base no tempo
38         count = (start+1)%POOL_SIZE;
39         next = start;
```

```
40          while(count!=end){
41          if ((pool[count]->deadline) < (pool[next]->deadline)){
42              next = count;
43          }
44          //para poder incrementar e ciclar o contador
45          count = (count+1)%POOL_SIZE;
46      }
47      //troca e coloca o processo com menor tempo como o próximo
48      tempProc = pool[next];
49      pool[next] = pool[start];
50      pool[start] = tempProc;
51      while((pool[start]->deadline) > 0){
52          //coloca a cpu em modo de economia de energia
53      }
54      //retorna se precisa repetir novamente ou não
55      switch (pool[start]->function()) {
56          case REPEAT:
57              kernelAddProc(pool[start]);
58              break;
59          case FAIL:
60              break;
61          default: ;
62      }
63      //próxima função
```

```
64          start = (start + 1) % POOL_SIZE;
65          }
66      }
67  }
68  //atualiza os tempos de execução dos processos
69  void kernelTick(void){
70      int proc;
71      proc = start;
72      while(proc!=end){
73          if((pool[proc]->deadline)>(MIN_INT)){
74              pool[proc]->deadline--;
75          }
76          proc = (proc+1)%POOL_SIZE;
77      }
78  }
```

O header do kernel é apresentado no código 26.6. A definição da estrutura, bem como os códigos de retorno, está definida neste cabeçalho para permitir que o programador consiga compatibilizar os processos que forem implementados.

Código 26.6: Kernel completo - header

```
#ifndef KERNEL_H_
#define KERNEL_H_

// códigos de retorno
#define SUCCESS  0
#define FAIL     1
#define REPEAT   2

    // declaração de ponteiro de função
    typedef char(*ptrFunc)(void);

    typedef struct {
        ptrFunc function;
        int period;
        int deadline;
    } process;

    // protótipos das funções do kernel
    void kernelInit(void);
    char kernelAddProc(process* func);
    void kernelLoop(void);// declaração de ponteiro de função
    void kernelTick(void);

#endif
```

Para usar o kernel, é preciso fazer, antes de tudo, o setup da interrupção de tempo que chamará a função **kernelTick()**. Na rotina **main()**, as flags de interrupção devem ser ligadas de acordo com a arquitetura utilizada.

Como os processos são temporizados, é necessário passar essa informação quando forem adicionados ao pool de processos. Deste modo, deve-se criar uma struct **process** para cada processo a ser adicionado. Por fim, é chamada a função **kernelLoop()**. O código 26.7 apresenta um exemplo para a placa Freedom.

Código 26.7: Utilizando o kernel com requisitos temporais

```
#include "lcd.h"
#include "keypad.h"
#include "serial.h"
#include "timer.h"
```

```
#include "kernel.h"

void systemInit(void) {
    //init clock das portas
    SIM_BASE_PTR ->SCGC5 |= (SIM_SCGC5_PORTA_MASK | SIM_SCGC5_PORTB_MASK);
    //configura para usar clock interno em 24MHz
    MCG_BASE_PTR ->C4 |= 0x80;
    //evitar a interrupção NMI na porta B pino 5
    PORTB_BASE_PTR ->PCR[5] = (PORT_PCR_MUX(1) | PORT_PCR_DSE_MASK);
}
//Criação das funções dos processos
char message(void){
    lcdCommand(0x80);
    lcdString("Teste kernel 3.0");
    return SUCCESS;
}
char keypad(void){
    kpDebounce();
    lcdCommand(0xC0);
    lcdNumber(kpRead());
    return REPEAT;
}
char serial(void) {
    char rx;
    rx = serialRead();
    if(rx!=0){
        lcdCommand(0xC6);
        lcdChar(rx);
    }
    return REPEAT;
}
int globalCounter;
char counter(void) {
    lcdCommand(0xC8);
    lcdNumber(globalCounter);
    globalCounter++;
    return REPEAT;
}
//Interrupção do timer chamando a função kernelTick()
void LPTimer IRQHandler (void){
    //limpar a flag da interrupção
```

```
46      LPTMR0_CSR |= 1 << 7;
47      kernelTick();
48  }
49  void main(void){
50  //criação dos processos
51      process pMessage = {message,  0,0};
52      process pKeypad =  {keypad,  10,0};
53      process pSerial =  {serial,  10,0};
54      process pCounter = {counter,100,0};
55  //inicialização dos periféricos
56      systemInit();
57      lcdInit();
58      kpInit();
59      serialInit();
60      timerInit();
61      kernelInit();
62  //adição dos processos no kernel
63      kernelAddProc(&pMessage);
64      kernelAddProc(&pKeypad);
65      kernelAddProc(&pSerial);
66      kernelAddProc(&pCounter);
67  //setup do tempo do tick
68      timerStart(10);
69  //Habilita interrupção do timer
70      LPTMR0_CSR |= 1 << 6;
71  //Permite que as interrupções do timer sejam atendidas
72      NVIC_ICPR |= 1 << (28 % 32);
73      NVIC_ISER |= 1 << (28 % 32);
74  //Execução do kernel
75      kernelLoop();
76  }
```

26.4 Exercícios

Ex. 26.1 — O que significa tempo real no contexto de sistemas embarcados? Como garantir

que um sistema possui a característica de tempo real?

Ex. 26.2 — Num sistema de tempo real, o que pode acontecer se os períodos de execução das funções não forem obedecidos?

Ex. 26.3 — Quais os problemas que podem aparecer quando utilizamos um contador finito para realizar a comparação de tempo para o agendamento de processos? Como solucionar esse problema?

Ex. 26.4 — Qual a finalidade do código abaixo? Sabendo que apenas alguns processos precisam de requisitos temporais, altere o código abaixo para que os processos que não precisam de contagem de tempo sejam desconsiderados. Os processos que requisitam tempo real possuem o campo **`ExigeRT`** igual a 1.

```
1 //definição da estrutura processo
2 typedef struct {
3     ptrFunc func;
4     int period;
5     int start;
6     char ExigeRT;
7 } process;
8
9 #define MIN_INT -30000
10 //KernelTick() é chamado na interrupção do timer 0
11 void KernelTick(void){
12     unsigned char i;
13     i = ini;
14     while(i!=fim){
15         if((pool[i].start)>(MIN_INT))
16         pool[i].start--;
17         i = (i+1)%SLOT_SIZE;
18     }
19 }
```

CAPÍTULO

27

Controladora de dispositivos

27.1Padrão de um driver
Driver do timer
27.2Mecanismo da controladora
Utilizando a controladora de drivers
Camada de abstração da interrupção
27.3Exercícios

> *"A primeira regra de qualquer tecnologia usada em um negócio é de que a automação aplicada a uma operação eficiente irá amplificar a eficiência. A segunda é que a automação aplicada a uma operação ineficiente, vai amplificar a ineficiência."*
>
> Bill Gates

Uma das funções principais de um sistema operacional é controlar todos os periféricos de um computador; tratar erros; interceptar interrupções; fornecer uma interface entre o dispositivo e o sistema; emitir comandos para os dispositivos.

A grande maioria dos periféricos possuem um controlador. Os controladores possuem registradores que são usados para se comunicar com o processador. Por meio da escrita nesses registradores, o sistema operacional pode comandar o dispositivo para aceitar, executar, desligar. A partir da escrita nesses registradores, o sistema operacional pode saber o estado de um dispositivo, como, por exemplo, se ele está apto a receber um novo comando ou não.

A função de um controlador de dispositivo (driver de dispositivo) é aceitar as requisições abstratas do software e cuidar para que a solicitação seja executada, permitindo que o software interaja com o periférico. Um driver não é um processo ou tarefa (programa) gerenciado pelo sistema, mas sim um conjunto de funções e tabelas contendo informações sobre cada periférico.

Os drivers são pequenos programas que fazem a comunicação entre o sistema operacional do computador e o hardware. Um driver de dispositivo simplifica a tarefa da aplicação, tendo como função atuar como um tradutor entre o dispositivo e as aplicações ou o sistema operacional. O código de alto nível das aplicações pode ser escrito independentemente do dispositivo que será utilizado. Qualquer versão de um dispositivo, como uma impressora, por exemplo, requer os seus próprios comandos. Entretanto, a maioria das aplicações tem acesso aos dispositivos usando comandos genéricos de alto nível, como o comando **`print()`**, que

imprime um texto. O driver converte esses comandos genéricos para um código de baixo nível interpretado pelo dispositivo. Em outras palavras, um driver opera ou controla um hardware.

Até o momento, trabalhamos com o conceito de bibliotecas para ter acesso aos periféricos. A diferença entre as bibliotecas criadas até o momento e os drivers é que estes últimos possuem uma padronização, de modo que o sistema operacional e as aplicações possam acioná-los.

Antes de descrevermos o funcionamento da controladora é importante definir como serão desenvolvidos os drivers.

27.1 Padrão de um driver

Todos os sistemas operacionais apresentam um certo padrão na construção de suas controladoras e de seus drivers. Somente possuindo uma interface padrão, o kernel pode se comunicar com qualquer tipo de driver, mesmo desconhecendo-o em tempo de compilação.

Neste capítulo será apresentado um modelo de implementação. Este modelo, apesar de funcional, tem como objetivo demonstrar o processo de criação de uma controladora. Existem vários detalhes que não serão levados em conta, seu propósito é apresentar os conceitos de gerenciamento de drivers de modo simples.

Os drivers, em geral, apresentam um conjunto de funções que permite que o aplicativo acesse o periférico. Por esse motivo o modo de acesso aos drivers será dado por ponteiros de função. Assim, a controladora pode acionar as funções do driver, independendo de seu código. Isto permite também que todas as funções dos drivers sejam declaradas apenas no arquivo de código e não em seu header. Isto força que o programador utilize as interfaces de acesso disponibilizadas pela controladora, o que, apesar de mais burocrático, evita possíveis erros de uso.

Com o intuito de simplificar o uso dos ponteiros de função será definido o tipo **ptrFuncDrv**, conforme o código 27.1:

Código 27.1: Assinatura padrão dos drivers

```
1 typedef char(*ptrFuncDrv)(void *parameters);
```

Segundo esse padrão, todas as funções dos drivers deverão retornar um valor indicando se a função foi executada corretamente ou não. Como parâmetro as funções dos drivers receberão apenas um ponteiro para void. Isto permite passar qualquer tipo de parâmetro para a função.

O driver será composto de dois arquivos, do mesmo modo que as bibliotecas de acesso aos periféricos, um header e um arquivo de código.

Para acessar o driver a controladora terá um ponteiro para uma estrutura **driver**. Esta estrutura será composta por três elementos: um identificador **drv_id**; o vetor **functions**, com referências para todas as funções disponibilizadas pelo driver, e o ponteiro de função **initFunc**, que servirá para inicializar o driver, conforme o código 27.2:

Código 27.2: Struct driver

```
1 typedef struct {
2     //identificador do driver
3     int drv_id;
4     //vetor com as funções disponíveis no driver
5     ptrFuncDrv *functions;
6     //ponteiro para a função de inicialização do driver
7     ptrFuncDrv initFunc;
8 } driver;
```

Cada driver terá em seu arquivo de código uma variável da estrutura tipo **driver**. Esta variável funcionará como um resumo do driver, reunindo todas as informações importantes do mesmo.

No arquivo de header, deverá ser disponibilizada apenas uma função, que retornará um ponteiro para a estrutura do driver. Deste modo, a controladora poderá pedir ao driver uma referência para sua estrutura interna. Nenhuma das outras funções do driver estarão disponibilizadas diretamente no header. Além da função, deve ser implementado um enumerador com a lista das funções disponíveis no driver.

As funções que retornam as estruturas dos drivers serão todas agrupadas na controladora de drivers. Para padronizar o código, todas serão definidas como ponteiros de função, de acordo com o código 27.3.

Código 27.3: Ponteiro de função para drivers

```
1 typedef driver* (*ptrGetDrv)(void);
```

Em termos de código, o driver deve implementar pelo menos duas funções: **init()** e **getDriver()**. A primeira serve para realizar a inicialização e configuração do periférico, do mesmo modo que nas bibliotecas já estudadas. A segunda tem como função garantir que a estrutura **driver** está preenchida e retornar um ponteiro para ela.

No arquivo de código, também devem ser criadas duas variáveis: uma estrutura do tipo **driver** e um vetor de ponteiros de função.

As demais funcionalidades do hardware são implementadas de acordo com a necessidade do periférico.

O código 27.4 apresenta o modelo de header para um driver:

Código 27.4: Exemplo do header de um driver genérico

```
 1 #ifndef GEN_DRIVER_H
 2 #define GEN_DRIVER_H
 3 //enumerador com a lista das funções disponíveis
 4 enum {
 5     GEN_FUNC1,
 6     GEN_FUNC2,
 7     GEN_END
 8 };
 9 //função para receber a referência do driver.
10 driver* getGenericDriver(void);
11 #endif
```

O código 27.5 apresenta a implementação mínima do arquivo de código de um driver. Note que o tamanho do vetor de funções **thisFunctions** depende da quantidade de funções descritas no enumerador do arquivo de header.

Código 27.5: Arquivo de código de um driver exemplo

```
 1 //estrutura do driver
 2 static driver     thisDriver;
 3 //funções disponpiveis no driver
 4 static ptrFuncDrv thisFunctions[GEN_END];
 5
 6 char initGeneric(void *parameters) {
 7     // Configura os registros de hardware necessários para a operação do driver
 8     // Inicializa variáveis internas
 9     // ...
10
11     //atualiza o novo ID do driver
12     thisDriver.drv_id = parameters;
13     return OK;
14 }
15
16 char function2(void *parameters) {
17     //função exemplo, não executa nada
18     return OK;
19 }
20
```

```
21 char function1(void *parameters) {
22     //função exemplo, não executa nada
23     return OK;
24 }
25
26 driver* getGenericDriver(void) {
27     //Realiza o link da estrutura do driver
28     thisDriver.functions = thisFunctions;
29
30     //Inicializa as referências das funcionalidades de acordo com a descrição do ↩
           header
31     thisFunctions[GEN_FUNC1] = function1;
32     thisFunctions[GEN_FUNC2] = function2;
33
34     //Configura a função de inicialização
35     thisDriver.initFunc = initGeneric;
36
37     //Retorna a referência do driver
38     return &thisDriver;
39 }
```

A figura 27.1 mostra a representação visual, através de um diagrama UML, da estrutura do driver de exemplo dos códigos 27.4 e 27.5:

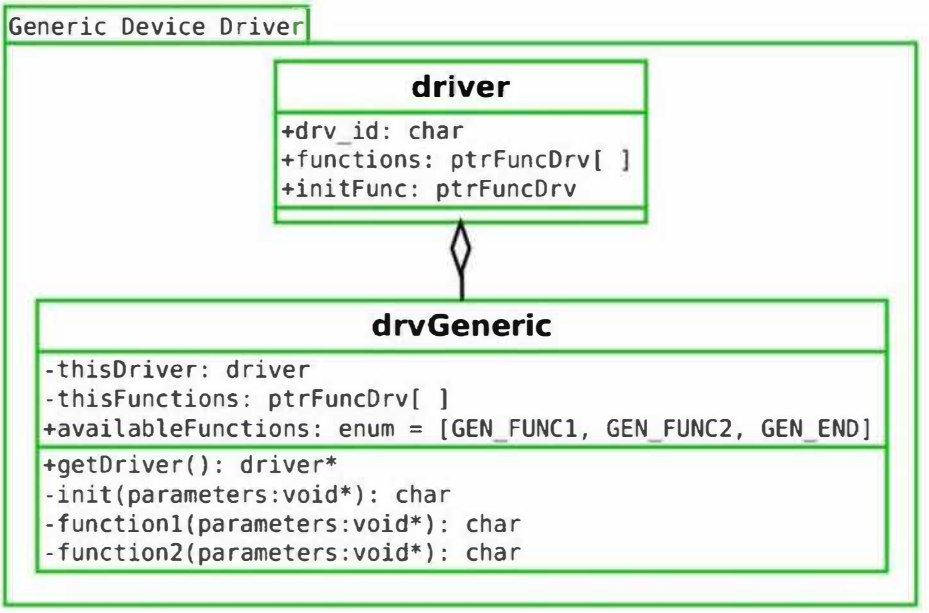


Figura 27.1: Diagrama da estrutura de um driver genérico

As funções e informações disponíveis no header são marcadas pelo sinal mais (**+**) antes do seu nome. O sinal menos (**-**), representa as funções disponíveis apenas no arquivo de código.

O ponto importante do padrão desenvolvido para o driver é o fato de cada driver possuir uma variável do tipo **`driver`**. Isto está representado pelo link entre as duas estruturas do diagrama.

A criação de um driver nada mais é do que inserir uma camada padronizada de chamadas de funções e estruturas para gerenciar as funcionalidades já desenvolvidas nas bibliotecas.

Driver do timer

O driver do timer apresentará as mesmas funcionalidade que a biblioteca. A figura 27.2 apresenta o modelo da estrutura do driver:

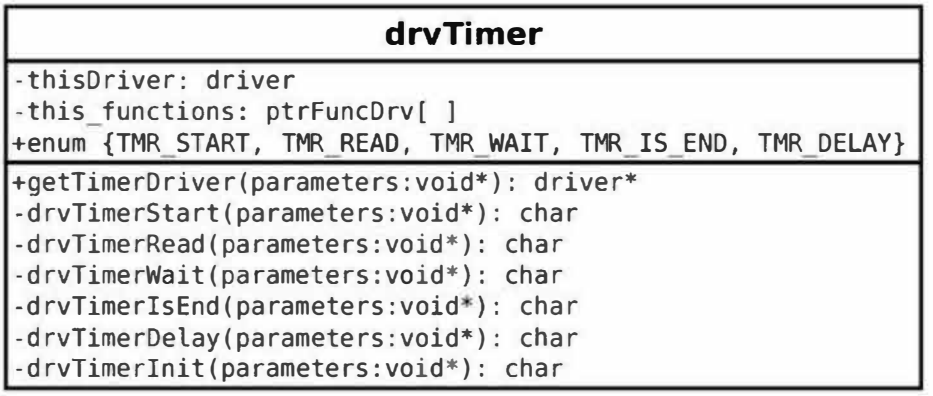


Figura 27.2: Esquema de implementação do driver de timer

As funções originais da biblioteca do timer não estão no padrão dos ponteiros utilizados nos drivers. Para resolver este problema podemos utilizar duas abordagens: modificar a biblioteca original do driver ou criar um conjunto de funções para fazer a adaptação.

As funções que originalmente recebiam algum parâmetro vão ter seu parâmetro recebido pelo ponteiro de void. Basta repassar o valor.

As funções que originalmente retornavam algum valor, irão retornar o valor via referência, usando o valor recebido por parâmetro como endereço da variável original. Para isso, deve ser realizado um typecast para um ponteiro do tipo da variável de retorno.

O código 27.6 apresenta as adaptações realizadas para a criação do timer. O cabeçalho é apresentado no código 27.7.

Código 27.6: Driver timer - código drvTimer.c

```
 1 #include "timer.h"
 2 static driver     thisDriver;
 3 static ptrFuncDrv thisFunctions[TIMER_END];
 4
 5 char drvTimerStart(void * parameters){
 6     timerStart((unsigned int) parameters);
 7     return SUCCESS;
 8 }
 9 char drvTimerRead(void * parameters){
10     int read = timerRead();
11     //faz um typecast para ponteiro de int e retorna o valor por referência
12     *((unsigned int*) parameters) = read;
13 }
14 char drvTimerWait(void * parameters){
15     timerWait();
16     return SUCCESS;
17 }
18 char drvTimerFinished(void * parameters){
19     int read = timerFinished();
20     //faz um typecast para ponteiro de int e retorna o valor por referência
21     *((unsigned int*) parameters) = read;
22     return SUCCESS;
23 }
24 char drvTimerDelay(void * parameters){
25     timerDelay((unsigned int) parameters);
26     return SUCCESS;
27 }
28 // inicialização do driver
29 char drvTimerInit(void *parameters) {
30     thisDriver.drv_id = parameters; //atualiza o novo ID do driver
31     timerInit();                    //chama a rotina de incialização
32     return OK;
33 }
34 driver* getTimerDriver(void) {
35     //Realiza o link da estrutura do driver
36     thisDriver.functions = thisFunctions;
37     this_functions[TMR_START] = drvTimerStart;
38     this_functions[TMR_READ] = drvTimerRead;
39     this_functions[TMR_WAIT] = drvTimerWait;
40     this_functions[TMR_FINISHED] = drvTimerFinished;
41     this_functions[TMR_DELAY] = drvTimerDelay;
42
43     //Configura a função de inicialização
44     thisDriver.initFunc = drvTimerInit;
45
46     //Retorna a referência do driver
47     return &thisDriver;
48 }
```

Código 27.7: Driver timer - header drvTimer.h

```
 1 #ifndef GEN_DRIVER_H
 2 #define GEN_DRIVER_H
 3     //enumerador com a lista das funções disponíveis
 4     enum {
 5         TMR_START,
 6         TMR_READ,
 7         TMR_WAIT,
 8         TMR_FINISHED,
 9         TMR_DELAY,
10         TMR_END
11     };
12     //função para receber a referência do driver.
13     driver* getTimerDriver(void);
14 #endif
```

27.2 Mecanismo da controladora

A controladora de drivers é a responsável por fazer o gerenciamento das chamadas das funções realizadas pelos processos. Através dela os drivers podem ser inicializados e chamados.

Para isso, ela precisa conhecer cada um dos drivers. Isto é feito através da estrutura **driver**. A controladora terá, em seu arquivo de código, um vetor com as referências para todos os drivers. Para preencher esse vetor, ela deve ter acesso à função de retorno da estrutura **driver** de cada um dos drivers. Para simplificar o funcionamento um segundo vetor será criado, com as funções de acesso aos drivers conhecidos. Junto deste vetor, será criado um enumerador descrevendo os drivers que a controladora conhece, do mesmo modo que os drivers descrevem as funcionalidades que eles possuem. A estrutura da controladora pode ser vista na figura 27.3:

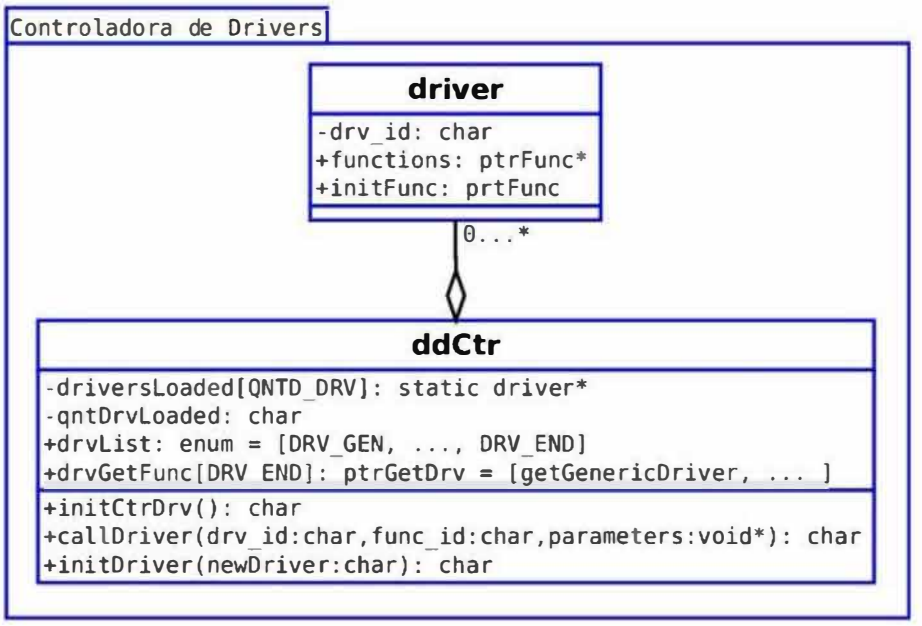


Figura 27.3: Diagrama da estrutura da controladora de drivers

Podemos notar que a controladora possui uma lista das referências de estruturas do tipo drivers. É esta lista que permite a controladora gerenciar todos os drivers em um único lugar.

A controladora possui, em seu arquivo de código, uma lista de todos os drivers carregados. Essa lista é implementada através do vetor **`driversLoaded`**. Este vetor é preenchido à medida que os drivers são inicializados.

O funcionamento da controladora pode ser descrito por suas três funções. A **`initCtrDrv()`** inicializa as variáveis internas da controladora. A **`initDriver()`** chama a função que inicializa o driver desejado e o insere na lista de drivers carregados. Por fim, a **`callDriver`** faz a chamada de uma função específica para o driver desejado, se ele já estiver inicializado. Estas funções são apresentadas no código 27.8:

Código 27.8: Arquivo de código da controladora de drivers

```
1 static driver* driversLoaded[QNTD_DRV];
2 static char qntDrvLoaded;
3
4 char initCtrDrv(void) {
5     //incializa sem drivers carregados
6     qntDrvLoaded = 0;
7     return SUCCESS;
8 }
9 char initDriver(char newDriver) {
10    char resp = FAIL;
11    //verifica se há espaço para inicializar o driver
12    if(qntDrvLoaded < QNTD_DRV) {
13        //pega a referência do driver e salva no vetor
14        driversLoaded[qntDrvLoaded] = drvGetFunc[newDriver]();
15        //tenta inicializar o driver passando o seu novo ID
16        resp = driversLoaded[qntDrvLoaded]->initFunc(&newDriver);
17        qntDrvLoaded++;
18    }
19    return resp;
20 }
21 char callDriver(char drv_id, char func_id, void *parameters) {
22    int i;
23    //procura se o driver está carregado
24    for (i = 0; i < qntDrvLoaded; i++) {
25        if (drv_id == driversLoaded[i]->drv_id) {
26            //se estiver chama a função correspondente repassando o parâmetro
27            return driversLoaded[i]->func_ptr[func_id](parameters);
28        }
29    }
30    return DRV_FUNC_NOT_FOUND;
31 }
```

Como é possível ver pelo código da controladora, a função **`callDriver()`** apenas repassa os parâmetros recebidos, testando apenas se o driver correspondente foi inicializado. Nesta função, pode-se desenvolver um sistema mais robusto e seguro realizando outros tipos de teste, como:

- Validação se a função escolhida existe
- Checagem de intervalo de valores válidos para os parâmetros
- Compartilhamento do driver por mais de uma tarefa
- Driver ocupado com outra operação
- Verificação se o processo está autorizado a usar o driver requisitado

Parte dessas verificações são executadas nos sistemas reais. Para a implementação de algumas delas, é necessário que o microcontrolador utilizado ofereça suporte em hardware. No entanto, elas não serão abordadas no escopo deste livro.

O header da controladora possui três regiões distintas: uma para definição dos tipos de dados utilizados nos drivers, uma para inserção dos drivers conhecidos e a última para apresentar os protótipos das funções da própria controladora. Estas informações são apresentadas no código 27.9, utilizando três drivers como exemplo:

Código 27.9: Header da controladora de drivers

```
#ifndef DD_CTRL_H
#define DD_CTRL_H

// 1) Definição de tipos e estruturas utilizados nos drivers

    //definição de função do driver
    typedef char(*ptrFuncDrv)(void *parameters);

    //definição do driver
    typedef struct {
        char drv_id;
        ptrFuncDrv *func_ptr;
        ptrFuncDrv drv_init;
    } driver;

    //definição de função para pegar a referencia da estrutura driver
    typedef driver* (*ptrGetDrv)(void);

// 2) Configuração dos drivers conhecidos

    //include dos cabeçalhos dos drivers
    #include "drvInterrupt.h"
    #include "drvTimer.h"
    #include "drvLcd.h"

    //enumerador dos drivers conhecidos
    enum {
        DRV_INTERRUPT,
        DRV_TIMER,
        DRV_LCD,
        DRV_END // DRV_END deve sempre ser o último, para facilitar o controle da
             quantidade de drivers
    };
```

```
33
34      //vetor com as Funções para retorno das estruturas dos drivers. Deve estar na ↩
            mesma ordem que o enumerador acima
35      static ptrGetDrv drvInitVect[DRV_END] = {
36          getInterruptDriver,
37          getTimerDriver,
38          getLCDDriver
39      };
40
41  // 3) Funções disponibilizadas pela controladora
42      char initController();
43      char initDriver(char newDriver);
44      char callDriver(char drv_id, char func_id, void *parameters);
45  #endif
```

Utilizando a controladora de drivers

Para utilizar corretamente a controladora, é necessário incluí-la no arquivo principal e fazer uso de tipo enumerado definido em seu header, bem como nos headers dos drivers.

O programa 27.10 demonstra como utilizar a controladora para acessar o driver de LCD. A função **LCD_STRING** no driver **DRV_LCD** envia uma string (codificada em ASCII) para o LCD interligado ao microcontrolador. Se houver necessidade de trocar o LCD para outro modelo, ou alterar os terminais que o controlam, o programador precisa apenas modificar o driver. A aplicação, neste caso, não sofrerá alterações.

Código 27.10: Utilizando a controladora

```
1 void main(void) {
2     // inicialização do sistema
3     initSystem();
4     initCtrDrv();
5     initDriver(DRV_LCD);
6     callDriver(DRV_LCD, LCD_STRING, "Embarcado");
7     for(;;);
8 }
```

Camada de abstração da interrupção

A controladora em conjunto com os drivers, fornecem uma solução interessante para auxiliar o programador a desenvolver aplicações em alto nível sem se preocupar com as interações com o hardware.

Algumas interações, no entanto, são um pouco mais complexas para serem codificadas em drivers. Uma delas é a interrupção.

A interrupção é um mecanismo de hardware que permite ao programador definir uma função que será executada quando um determinado evento acontecer. Apesar de ser uma ideia bastante simples, sua implementação prática é dificultada pela imensidão de compiladores e microcontroladores existes. Cada um realiza a implementação dessa ferramenta de um modo diferente.

Para simplificar esta estrutura, podemos criar uma camada de abstração de interrupção que funcionará basicamente como um driver de interrupção.

O programador cria a função responsável por responder ao evento que ele deseja e avisa o driver. O driver, por sua vez, irá armazenar o endereço dessa função dentro de si. Quando o evento acontecer ele chamará a função definida pelo programador. O programador, por sua vez, não precisará se preocupar com a implementação de baixo nível dessa estrutura. Numa mudança de arquitetura, apenas o driver da interrupção deverá ser alterado.

Esta abordagem simplifica o acesso às interrupções e é exatamente o que foi feito na plataforma Wiring através da função **`attachInterrupt()`**. O código desta função é apresentada no código 27.11:

Código 27.11: Código da função attachInterrupt

```
1 //função a ser executada quando não houver nenhuma função definida para atender ↩
      as interrupções
2 static void nothing(void) {
3
4 }
5
6 //mostrando apenas para 2 interrupções, o código verdadeiro chega até 8
7 static volatile voidFuncPtr intFunc[EXTERNAL_NUM_INTERRUPTS] = {
8     nothing,
9     nothing
10 };
11
12 void attachInterrupt(uint8_t interruptNum, void (*userFunc)(void), int mode) {
13     if(interruptNum < EXTERNAL_NUM_INTERRUPTS) {
14         //atualiza o vetor com o endereço da nova função
15         intFunc[interruptNum] = userFunc;
16
17         // habilita a interrupção, o parâmetro mode define o tipo de interrupção
18         switch (interruptNum) {
19             case 0:
20                 EICRB = (EICRB & ~((1 << ISC40) | (1 << ISC41))) | (mode << ISC40);
21                 EIMSK |= (1 << INT4);
22                 break;
23             case 1:
24                 EICRB = (EICRB & ~((1 << ISC50) | (1 << ISC51))) | (mode << ISC50);
25                 EIMSK |= (1 << INT5);
26                 break;
27             //os cases para as outras funções entram aqui
28         }
29     }
30 }
31
32 //Implementação das funções de interrupção de acordo com o padrão do ↩
      microcontrolador/compilador.
33 //Estas funções apenas chamam os endereços salvos no vetor.
34 ISR(INT0_vect) {
35     intFunc[EXTERNAL_INT_2]();
36 }
37
38 ISR(INT1_vect) {
39     intFunc[EXTERNAL_INT_3]();
40 }
```

Com esta estrutura, o programador precisa apenas de: 1) criar a função que deve ser executada na interrupção e 2) atrelá-la à interrupção com a **`attachInterrupt()`**. O código 2 7.12 mostra como esta camada simplifica o uso das interrupções:

Código 27.12: Código da função attachInterrupt

```
//função a ser executada na interrupção
#define key 13
int state = 0;

void setup() {
    //configura o terminal do teclado como entrada
    pinMode(key, INPUT);
    //atrela a função isr_key() para atender a interrupção do teclado
    attachInterrupt(digitalPinToInterrupt(pin), blink, CHANGE);
}

void loop() {
    //não precisa fazer mais nada
}

void isr_key() {
    //esta função é executada automaticamente quando o terminal 13 mudar de valor.
}
```

Para montar a mesma estrutura em linguagem C, basta criar um ponteiro de função que armazenará o endereço da função a ser executada. Esta variável estará armazenada no arquivo de código do driver de interrupção. O driver então implementa uma funcionalidade para receber o endereço da função e salvá-lo. Este processo pode ser visualizado no código 27.13:

Código 27.13: Abstração da interrupção

```
// Definindo o tipo intFunc para utilizá-lo como interrupção
typedef void (*intFunc)(void);

// Ponteiro de função para armazenar o endereço da função
static intFunc thisInterrupt;

// Esta função recebe o endereço da função e atualiza o endereço
char setInterruptFunc(void *parameters) {
    thisInterrupt = (intFunc) parameters;
    return OK;
}
```

O código 27.14 apresenta o header do driver de interrupção. Neste momento, o código apenas implementa os dados de interrupção do timer:

Código 27.14: Header do driver de interrupção

```
 1 #ifndef INT_DRIVER_H
 2 #define INT_DRIVER_H
 3
 4     //enumerador com a lista das funções disponíveis
 5     enum {
 6         INT_TIMER_SET,
 7         INT_END
 8     };
 9     //função para receber a referência do driver.
10     driver* getInterruptDriver(void);
11 #endif
```

O código 27.15 apresenta a implementação mínima do arquivo de código de um driver. Note que o tamanho do vetor de funções **thisFunctions** depende da quantidade de funções descritas no enumerador do arquivo de header.

Código 27.15: Arquivo de código de um driver–exemplo

```
1 //estrutura do driver
2 static driver     thisDriver;
3
4 //funções disponpiveis no driver
5 static ptrFuncDrv thisFunctions[GEN_END];
6
7 // Definindo o tipo intFunc para utilizá-lo como interrupção
8 typedef void (*intFunc)(void);
9
10 // Ponteiro de função para armazenar o endereço da função
11 static intFunc timerInterrupt;
12
13 //Esta é a função de interrupção do timer
14 void LPTimer_IRQHandler(void) {
15     //limpar a flag da interrupção
16     LPTMR0_CSR |= 1 << 7;
17     //Verifica se a função é válida
18     if (timerInterrupt != 0){
19         timerInterrupt();
20     }
21 }
22
```

```
23 char initInterrupt(void *parameters) {
24     //atualiza o novo ID do driver
25     thisDriver.drv_id = parameters
26 
27     //valor inicial, representa sem função configurada
28     timerInterrupt = 0;
29 
30     //as inicializações são necessárias apenas para cada interrupção
31     return OK;
32 }
33 
34 // Esta função recebe o endereço da função e atualiza o endereço
35 char setTimerInterrupt(void *parameters) {
36     timerInterrupt = (intFunc) parameters;
37 
38     //como agora temos uma função responsável por responder à interrupção, ↩
        podemos ligar os bits que a habilitam
39     LPTMR0_CSR |= 1 << 6;
40     NVIC_ICPR |= 1 << (28 % 32);
41     NVIC_ISER |= 1 << (28 % 32);
42 
43     //ainda é necessário ligar o timer, mas como é um outro driver isso fica a ↩
        cargo do programa principal
44     return OK;
45 }
46 
47 driver* getGenericDriver(void) {
48     //Realiza o link da estrutura das
49     thisDriver.functions = thisFunctions;
50 
51     //Inicializa as referências das funcionalidades de acordo com a descrição do ↩
        header
52     thisFunctions[INT_SET_TIMER] = setTimerInterrupt;
53 
54     //Configura a função de inicialização
55     thisDriver.initFunc = initGeneric;
56 
57     //Retorna a referência do driver
58     return &thisDriver;
59 }
```

Utilizando esta estrutura através da controladora de drivers, os detalhes da programação de baixo nível do compilador se tornam transparentes para a aplicação. O programa 27.16 faz uso da interrupção de timer para executar a função **`kernelClock()`** exigida pelo kernel.

Código 27.16: Utilização do driver de interrupção

```
1 //função a ser executada pela interrupção
2 void timerISR(void) {
3     kernelClock();
4 }
5 void main (void){
6     systemInit();
7     kernelInit();
8
9     initDriver(DRV_TIMER);
10    initDriver(DRV_INTERRUPT);
11
12    //configurando o timer para gerar uma timeout a cada 10ms
13    callDriver(DRV_TIMER, TMR_TIMEOUT_SET, 10);
14    //habilita o timer
15    callDriver(DRV_TIMER, TMR_INT_EN, 0);
16    //configura a função e habilita a interrupção
17    callDriver(DRV_INTERRUPT, INT_TIMER_SET, timerISR);
18
19    //A função kernelClock, neste ponto, já está configurada para rodar na ↩
          interrupção do timer. O kernelLoop pode inicar sem problemas.
20    kernelLoop();
21 }
```

27.3 Exercícios

Ex. 27.1 — Porque é preciso padronizar as chamas de função dos drivers?

Ex. 27.2 — Qual a função da camada de abstração de interrupções?

Ex. 27.3 — Monte o driver para um dispositivo de entrada digital (chaves). Este dispositivo possui duas chaves, a primeira no bit 2 da porta B e a segunda no bit 4 da porta A. Seguem abaixo as definições padrões de um driver conforme a controladora de drivers do livro:

```
1 //ptr. de func. para uma função do driver
2 typedef char(*ptrFuncDrv)(void *parameters);
3 //estrutura do driver
4 typedef struct {
5 char drv_id;
6 ptrFuncDrv *drv_func;
7 ptrFuncDrv drv_init;
8 } driver;
9 //função de retorno do driver
10 typedef driver* (*ptrGetDrv)(void);
```

Ex. 27.4 — Qual a função do código a seguir? Por que é utilizado um ponteiro para void como parâmetro da função dos drivers?

```
1 char callDriver(char drv_id, char func_id, void *param) {
2     char i;
3     for (i = 0; i < dLoaded; i++) {
4         //find the right driver
5         if (drv_id == drivers[i]->drv_id) {
6             //call driver's function passing the parameters
7             return drivers[i]->func[func_id].func_ptr(param);
8         }
9     }
10    return DRV_FUNC_NOT_FOUND;
11 }
```

Ex. 27.5 — Monte o driver para o conversor AD. A biblioteca AD possui apenas duas funções: void InicializaAD(void) e int LeValorAD(void):

```
1 //ptr. de func. para uma função do driver
2 typedef char(*ptrFuncDrv)(void *parameters);
3 //estrutura do driver
4 typedef struct {
5     char drv_id;
6     ptrFuncDrv *drv_func;
7     ptrFuncDrv drv_init;
8 } driver;
```

Parte IV

Anexos

Circuitos utilizados nas experiências
Projeto da placa de desenvolvimento
Exercícios resolvidos
Índice Remissivo

Circuitos utilizados nas experiências

O aprendizado em programação embarcada envolve o contato tanto com o software quanto com o hardware. As placas de controle utilizadas ao longo do livro (Arduino, Chipkit e Freedom) facilitam a utilização da parte eletrônica, entregando uma solução simples de gravação/regravação do firmware do microcontrolador. Elas também provêm um caminho de comunicação serial e acesso aos periféricos internos.

Para os periféricos externos é necessário montar os circuitos adjacentes ao microcontrolador. O esquemático destes circuitos foi apresentado ao longo dos capítulos do livro. Para simplificar as atividades práticas estes circuitos podem ser montados tanto em protoboards ou utilizar a placa de desenvolvimento projetada para o livro.

São ao todo seis circuitos externos: o led RBG, o display quádruplo de 7 segmentos, o teclado matricial 2x5, o display de LCD, os sensores analógicos com o buzzer e o RTC DS1307.

As conexões com o led RGB são bastante simples. Elas utilizam os mesmos terminais de ativação dos displays de 7 segmentos. Os resistores utilizados são de 1k, conforme figura 1:

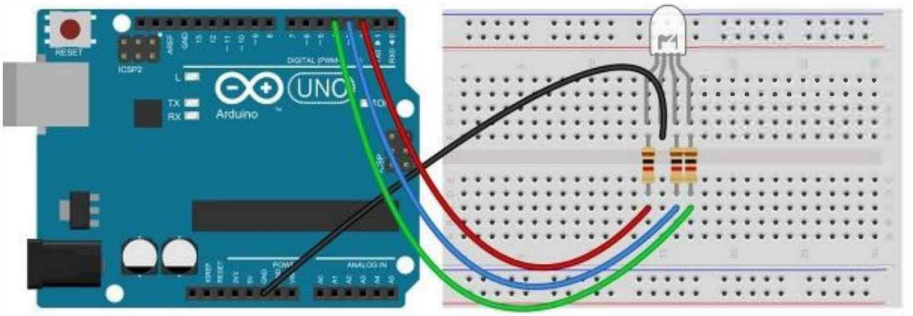


Figura 1: Ligação em protoboard do circuito do LED RGB
Fonte: Imagem produzida com Fritzing/Inkscape

O circuito de conexão dos displays de 7 segmentos exige o uso do 74HC595. Três linhas de comando são enviadas a ele, que, por sua vez, transforma os dados seriais das três linhas em 8 saídas digitais, que seguem para os displays. O acionamento de cada display é feito por quatro transistores controlados individualmente pelo microcontrolador. A conexão no protoboard segue conforme a figura 2:

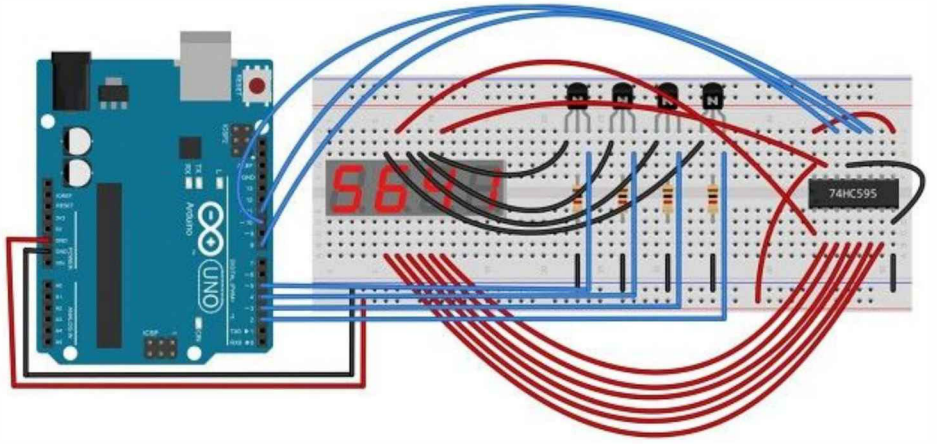


Figura 2: Ligação em protoboard do circuito de display de 7 segmentos
Fonte: Imagem produzida com Fritzing/Inkscape

O circuito de acionamento do LCD é bastante parecido com o do 7 segmentos. A diferença é que os sinais do 74HC595 seguem para o display de LCD. O LCD também exige um potênciometro para controle de contraste, que é conectado ao terminal de número 3. Todos os terminais não utilizados são conectados ao terra. A conexão no protoboard segue conforme a figura 3:

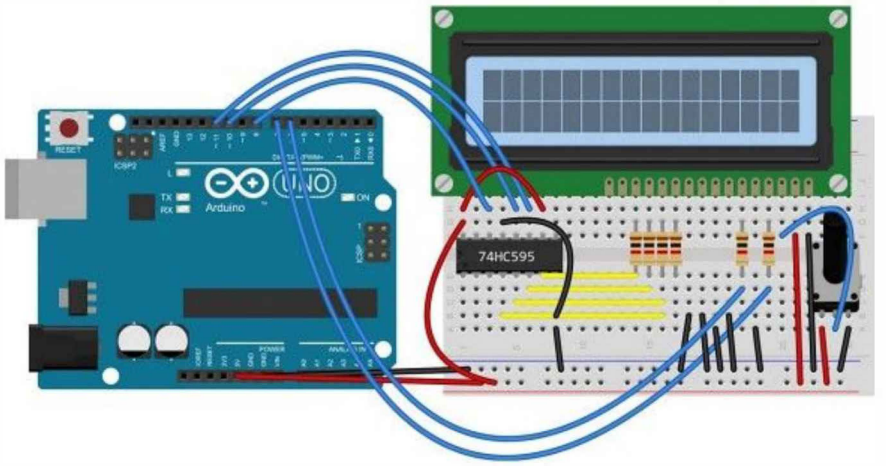


Figura 3: Ligação em protoboard do circuito do display de LCD
Fonte: Imagem produzida com Fritzing/Inkscape

A conexão do teclado exige alguns cuidados extras. As teclas possuem curto interno entre dois de seus quatro terminais, por isso as teclas que estão conectadas no centro do protoboard possuem seus contatos na vertical e as conectadas em cima ou embaixo têm seus contatos na horizontal. Os dados novamente saem do 74HC595, mas seguem para um conjunto de diodos, que servem tanto para proteção quanto para evitar acionamentos indevidos das teclas. Os sinais saem dos diodos e, passando pelas chaves, voltam a se reunir nos transistores, que farão a conversão de tensão para a leitura das teclas pelo microcontrolador. A conexão no protoboard segue conforme a figura 4:

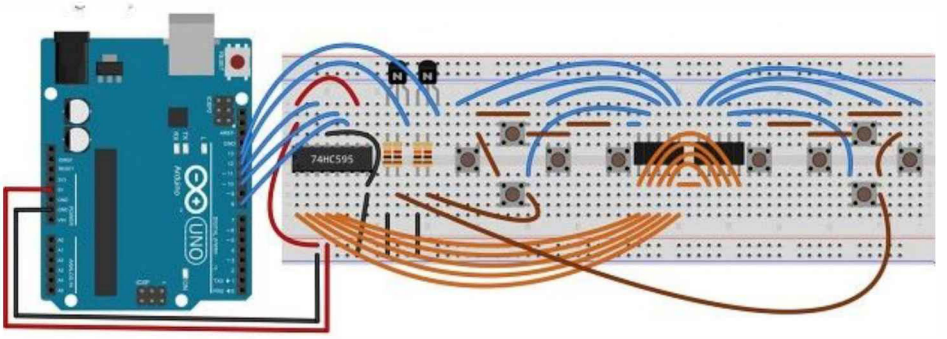


Figura 4: Ligação em protoboard do circuito de leitura do teclado matricial 5x2
Fonte: Imagem produzida com Fritzing/Inkscape

Os dispositivos analógicos são bastante simples de serem conectados. Em geral, é necessário levar apenas a alimentação (GND e VCC) a eles e conectar a saída no terminal de leitura analógica da placa. O buzzer é utilizado com um circuito transistorizado para seu acionamento. A conexão no protoboard segue conforme a figura 5:

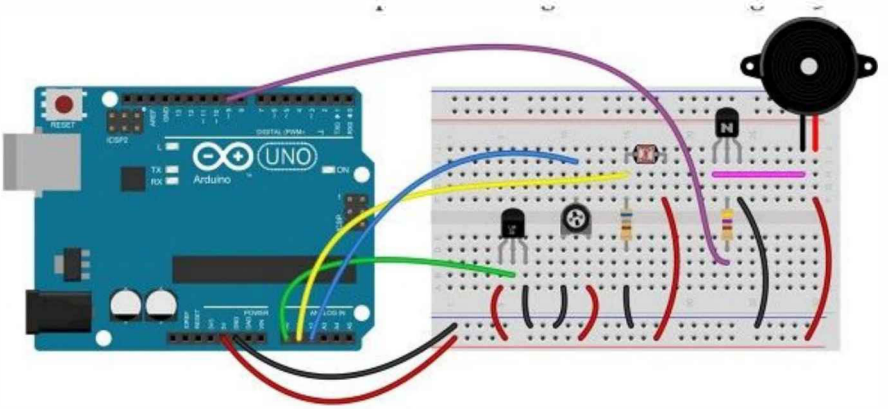


Figura 5: Ligação em protoboard dos circuitos sensores analógicos e do buzzer
Fonte: Imagem produzida com Fritzing/Inkscape

O último circuito externo utilizado é o relógio de tempo real DS1307. Ele se comunica via protocolo I²C com a placa de controle, por isso os dois terminais de dados possuem os resistores de pull up. Além da alimentação da placa, o circuito conta com a possibilidade de adição de uma bateria externa, evitando perder a contagem de tempo quando a placa for desligada. Por fim um cristal dedicado é adicionado para o correto funcionamento do relógio. A conexão no protoboard segue conforme a figura 6.

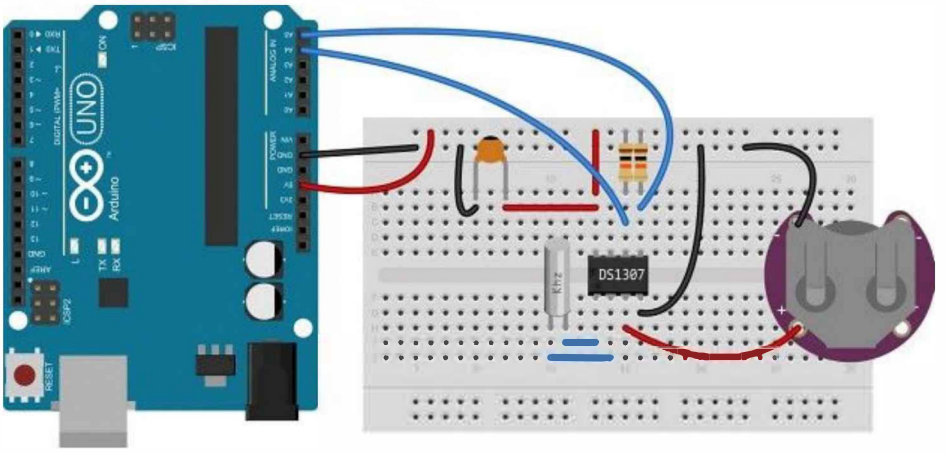


Figura 6: Ligação em protoboard do circuito do relógio DS1307
Fonte: Imagem produzida com Fritzing/Inkscape

Projeto da placa de desenvolvimento

A placa de desenvolvimento foi projetada para conter todos os dispositivos necessários para apresentar os conceitos abordados nesse livro. Por limitações de terminais disponíveis no barramento das placas compatíveis com o barramento do Arduino, foi utilizado um circuito de expansão de terminais que faz a conversão de serial para paralelo.

Outra preocupação foi deixar a placa base compatível com circuitos de 5 ou 3,3 volts. Esta providência foi tomada para poder abranger o maior número de placas e permitir o uso da placa base sem restrições.

No entanto, essa abordagem pode levantar alguns problemas: queimar a placa base ou a placa de processamento, se as tensões não foram compatíveis.

Para evitar esse problema, a placa base não deverá sofrer nenhuma avaria se for utilizado uma placa de processamento com tensões de 5 volts e a placa de desenvolvimento não poderá causar nenhum dano às placas de processamento de 3,3 volts. As soluções encontradas foram:

1. Alimentar todos os componentes externos da placa de desenvolvimento em 5 volts. A maioria dos componentes externos possui capacidade de operar em ambas as tensões fornecidas pela placa (5 v e 3 ,3 v). No entanto, alimentar os componentes em 3 ,3 v e enviar sinais de controle em 5 volts poderia causar danos. O contrário, alimentar em 5 v e enviar sinais de 3 ,3 v não causa problemas físicos (queima).
2. Utilizar circuitos digitais que, mesmo alimentados em 5 volts, aceitem a tensão de 3 ,3 volts como nível alto. Deste modo se a tensão do sinal for zero volts, ele será considerado de nível baixo. Se a tensão for tanto 3 ,3 quanto 5 volts será considerado de nível alto.
3. Os sinais digitais que sairão da placa de desenvolvimento serão todos em 3 ,3 volts, assim as placas de controle não serão afetadas. Do mesmo modo que os componentes, as placas de controle entendem o nível de 3 ,3 v como alto, mesmo as alimentadas em 5 volts. Os sinais digitais utilizarão um transistor para efetuar a mudança dos níveis de tensão para 3,3 volts.
4. Os sinais analógicos devem possuir valores que poderão variar entre zero e 3 ,3 v. Quando for utilizada uma placa de processamento de 5 volts, será perdido um pouco da faixa de conversão, não utilizando toda a escala do conversor. Isto significa uma perda de resolução do sinal, o que, para aplicações didáticas, não compromete o uso.

O esquemático apresentado na figura 7 foi desenvolvido levando todos os requisitos em conta.

Para simplificar o projeto, depuração, fabricação e montagem da placa, a numeração dos componentes segue um padrão. Todos eles são identificados por um conjunto de letras mais três dígitos numéricos. As letras indicam o tipo de componente. O primeiro dígito representa de qual circuito o componente faz parte. Os dois últimos dígitos são números sequenciais para diferenciar os componentes dentro do mesmo circuito. Os circuitos foram divididos em:

- Entradas digitais;
- Entradas analógicas e saída PWM;
- Display de 7 segmentos;
- Display de LCD;
- Led RGB;
- Conversor serial-paralelo (74 HC595);
- Circuito do RTC DS1307 ;

• Shield com footprint do Arduino.

Com relação às letras utilizadas na identificação dos componentes temos:

• R - Resistências;
• C - Capacitores;
• U - Circuitos integrados / chips;
• D - Diodos;
• X - Cristal de quartzo;
• SHIELD - Terminais de conexão;
• LDR - Sensor de luminosidade;
• RV - Potenciômetro;
• Q - Transistores;
• RBG - Led com três cores;
• BZ - Buzzer;
• BT - Bateria;
• SW - Chaves do tipo microswitch.

Com relação ao projeto do layout da placa, este foi planejado para ser possível de ser fabricado em face única. Isto permite a confecção caseira da placa de modo rápido e simples. Além disso, todos os traços possuem a espessura mínima de 15mils (0,381mm), com isolação de 30mils (0,762mm). Isto evita reduz a possibilidade de curtos ou problemas de corrosão na fabricação caseira da placa. Por esse motivo algumas providências tiveram que ser tomadas para garantir essas premissas:

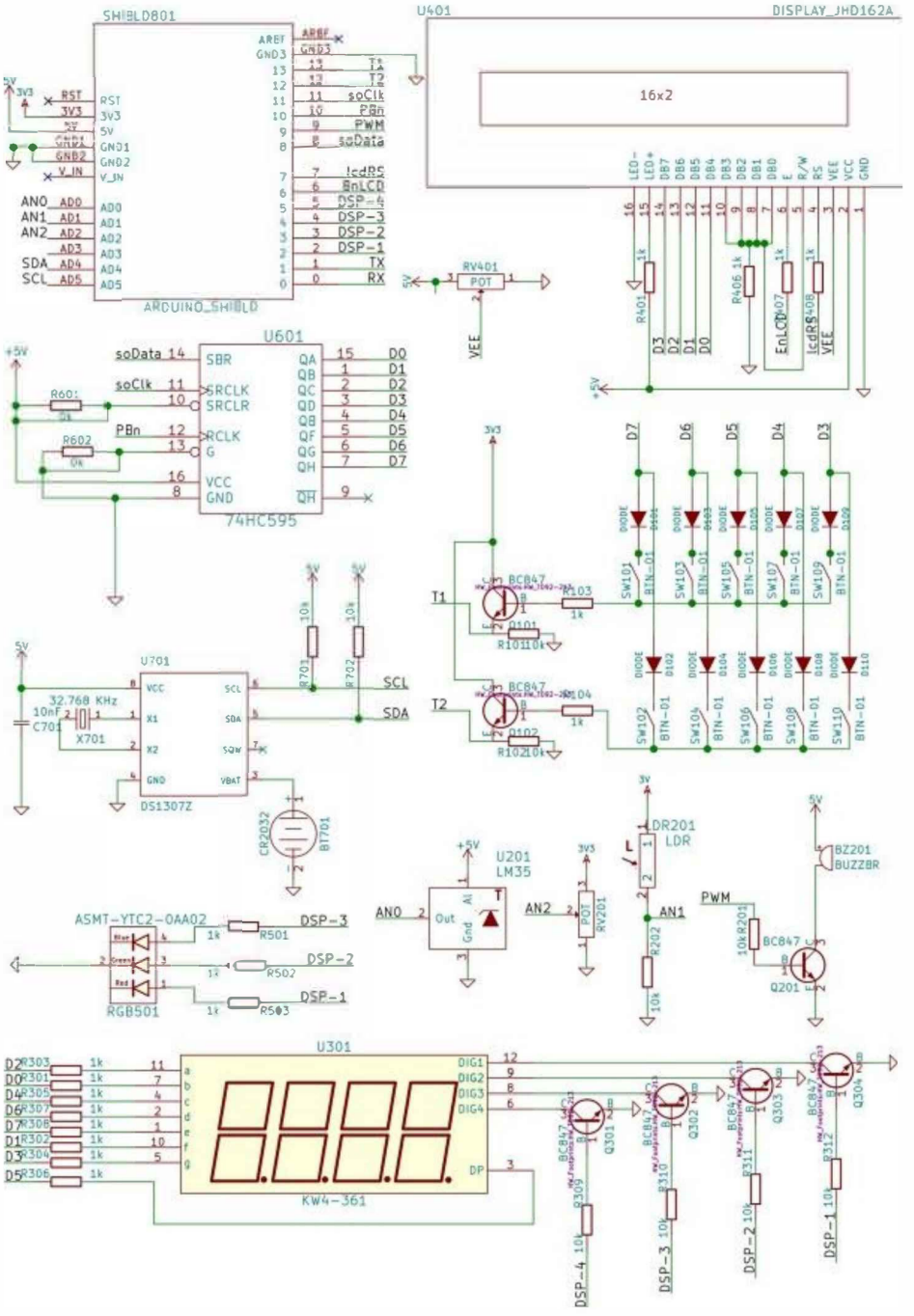


Figura 7: Esquemático completo da placa de desenvolvimento

- O LCD faz a comunicação através de um barramento de 4 bits. Isto reduz a quantidade de trilhas necessárias e facilita o roteamento.
- Os leds do display de 7 segmentos estão conectados de acordo com o posicionamento dos terminais físicos. Por isso a sequência dos segmentos na saída do 74 HC595 é dada por **`ed0cgafb`**, e não por **`0gfedcba`**.
- O footprint (desenhos do componente) utilizado para os diodos foi alongado de modo que houvesse mais espaço entre os seus terminais. Deste modo, criou-se um caminho

livre embaixo dos diodos que permitiu a passagem de sinais do lado esquerdo para o lado direito da placa.

- Dois resistores de zero ohms foram adicionados (R601 e R602). Eles funcionam como jumpers para conectar as trilhas em regiões onde não foi possível contornar os caminhos dos sinais.
- O espaço entre os terminais dos componentes não foi utilizado para a passagem de trilhas visando simplificar a fabricação da placa e respeitar os espaçamentos definidos.

O posicionamento dos componentes é apresentado na figura 8 (fora de escala). O layout final do traçado é apresentado, em escala, na figura 9

A lista de componentes utilizados, contendo sua identificação e sua descrição, é apresentada na tabela 1. Os resistores R601 e R602 possuem a resistência de zero ohms pois são apenas fios. Eles foram utilizados para permitir que a placa fosse produzida em face simples.

Ao todo, para montar a placa são necessários:

- 15 Resistores 1 k
- 10 Resistores 10 k
- 2 Resistores (jumpers) 0 ohm
- 1 capacitor 10 nF
- 1 cristal 32 ,768 kHz
- 1 74HC595
- 1 placa de controle Arduino/Chipkit/Freedom
- 28 terminais macho, divididos em 4 barras: 6, 6, 8 e 8 pinos cada.
- 1 Led RGB
- 7 transistores BC337
- 10 chaves microswitch, 2 ou 4 terminais
- 1 buzzer
- 10 diodos, 1 N4148 ou similar
- 1 display LCD 12 x6 JHD162 A ou compatível
- 1 display quádruplo 7 segmentos, KW4 -361 ou compatível
- 1 LDR 10 k
- 2 potênciômetros 10 k

O modelo 3D do circuito é apresentado na figura 10. O posicionamento dos botões foi modelado para simular um controle de vídeo-game. Os dispositivos de exibição (LCD e sete segmentos) são posicionados na parte superior da placa. O shield se localiza lateralmente para permitir conexão dos cabos de alimentação e comunicação. Os potenciômetros estão na borda para facilitar o acesso. Os demais componentes foram colocados de modo a minimizar problemas com roteamento.

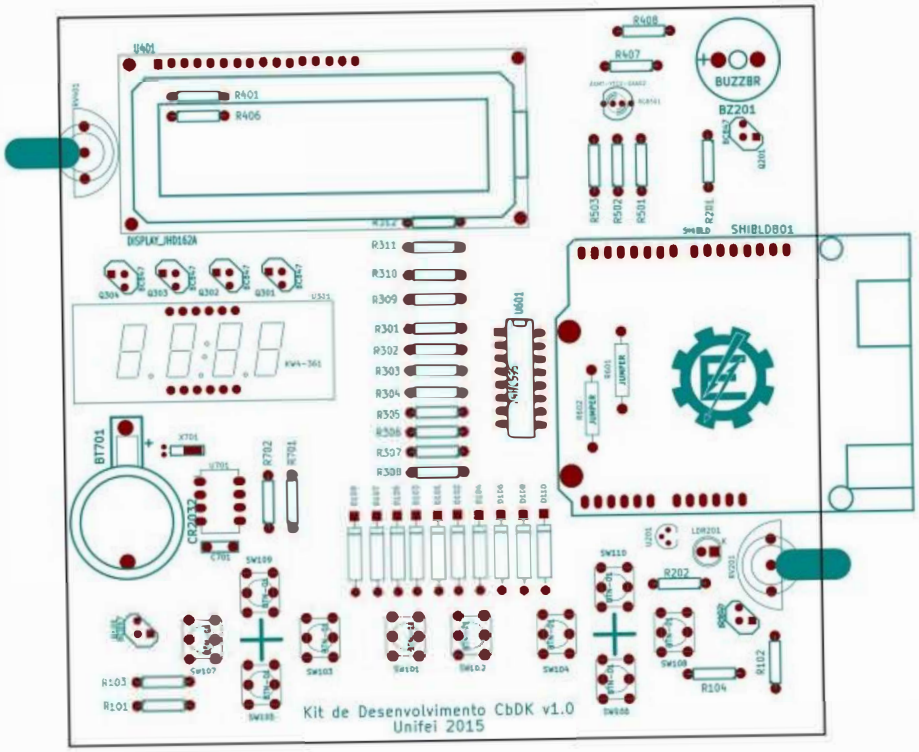


Figura 8: Posicionamento dos componentes

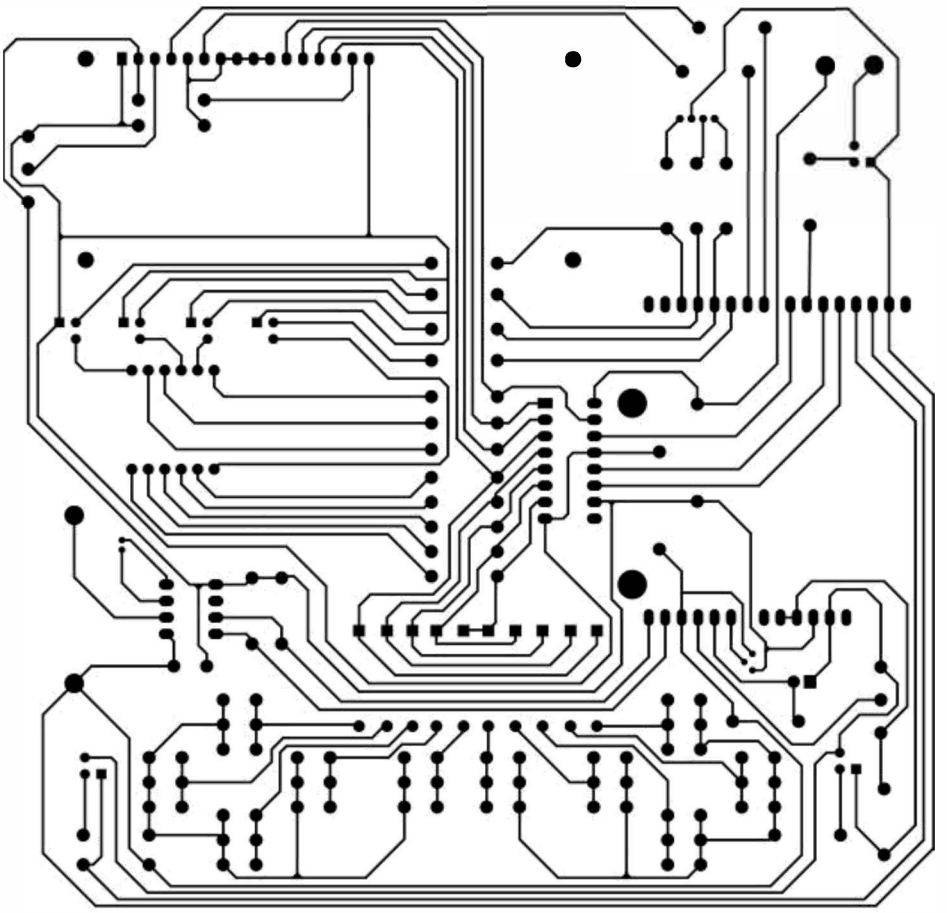


Figura 9: Layout da placa de desenvolvimento

Tabela 1: Componentes utilizados e descrição

ID	Item	ID	Item	ID	Item
BT701	CR2032	R101	10 k	R502	1 k
BZ201	BUZZER	R102	10 k	R503	1 k
C701	10 nF	R103	1 k	R601	0 k
D101	1 n4148	R104	1 k	R602	0 k
D102	1 n4148	R201	10 k	R701	10 k
D103	1 n4148	R202	10 k	R702	10 k
D104	1 n4148	R301	1 k	RGB501	Led RGB
D105	1 n4148	R302	1 k	SW101	Microswitch
D106	1 n4148	R303	1 k	SW102	Microswitch
D107	1 n4148	R304	1 k	SW103	Microswitch
D108	1 n4148	R305	1 k	SW104	Microswitch
D109	1 n4148	R306	1 k	SW105	Microswitch
D110	1 n4148	R307	1 k	SW106	Microswitch
LDR201	LDR	R308	1 k	SW107	Microswitch
Q101	BC847	R309	10 k	SW108	Microswitch
Q102	BC847	R310	10 k	SW109	Microswitch

Q201	BC847	R311	10 k	SW110	Microswitch
Q301	BC847	R312	10 k	U201	LM35
Q302	BC847	R401	1 k	U301	Display quádruplo 7 segmentos
Q303	BC847	R406	1 k	U401	Display LCD16 x2
Q304	BC847	R407	1 k	U601	74 HC595
RV201	Potenciômetro 10 k	R408	1 k	U701	DS1307 Z
RV401	Potenciômetro 10 k	R501	1 k	X701	Cristal 32 .768 kHz

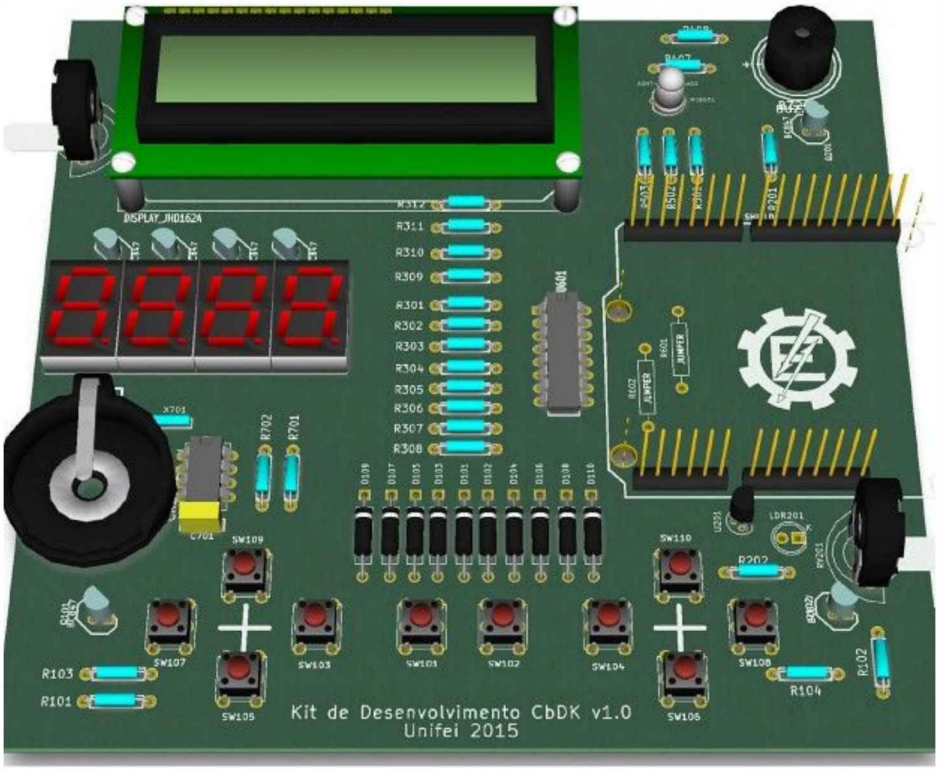


Figura 10: Renderização 3d da placa com os componentes adicionados.

Exercícios resolvidos

Resolução do ex. 6.2 — Nas operações Lógicas, os operandos são tomados como um todo, nas operações Bitwise, os operandos são processados bit a bit.

```
 1 ec0 = 0;
 2 ec1 = 1;
 3 ec2 = 3;    //ou 0b11
 4 ec3 = 1;
 5 ec4 = 0;
 6 ec5 = 1;
 7 ec6 = 255; //ou 0xFF
 8 ec7 = 1;
 9 ec8 = 0;
10 ec9 = 240; //ou 0xF0
11 eca = 0;
12 ecb = 237; //ou 0b11101101
13 ecc = 0;
14 ecd = 90;  //ou 0x5A
15 ece = 1;   //ou (0b11) & 0b01 = 0b01
16 ecf = 1;   //ou (0b00) | 0b01 = 0b01
```

Resolução do ex. 7.2 — Inicializam-se as variáveis com valores aleatórios apenas para testes.

```
1 void main (void){
2     int n1,n2,n3,n4,n5;
3     int media;
4
5     lcdInit();
6     n1 = 5;
7     n2 = 7.4;
8     n3 = 1.3;
9     n4 = 9.8;
10    n5 = 7.2;
11    media = (n1+n2+n3+n4+5)/5;
12    if(media>9.0){
13        lcdChar('A');
14    }else if(media>8.0){
15        lcdChar('B');
16    }else if(media>7.0){
17        lcdChar('C');
18    }else
19        lcdChar('D');
20    }
21 }
```

Resolução do ex. 7.8 — Deve-se criar uma variável do tipo ponteiro para apontar para o endereço pedido pelo exercício.

```
 1 void main (void){
 2     //como o valor é maior que 256 a variável deve ser int
 3     int * corrente;
 4     //aponta para o endereço da variável corrente
 5     corrente = 0x5f4;
 6     for(;;){
 7         if (*corrente > 512){
 8             BitClr(PORTA,2);
 9         }
10     }
11 }
```

Resolução do ex. 8.10 — Para calcular o valor do fatorial, basta multiplicar o número por todos os valores de **1** até **N**. Deve-se tomar o cuidado de não incluir o valor **0**.

```
 1 void main (void){
 2     int N;
 3     int fat;
 4     int i;
 5
 6     //inicialização do sistema
 7     systemInit();
 8     lcdInit();
 9
10     N = 10;
11     fat = 1;
12     //deve começar com 1 e não com zero para não zerar o resultado
13     for(i=1; i<=N; i++){
14         fat = fat * i;
15     }
16     lcdNumber(i);
17     for(;;);
18 }
```

Resolução do ex. 8.20 — O primeiro loop inicialize as matrizes **A** e **B** com valores baseados em **i** e **j**, já que o exercício não pediu nenhum valor específico. O segundo loop realiza o cálculo.

```
 1 void main(void){
 2     int i, j, matA[5][3], matB[5][3], matC[5][3];
 3     //inicializacao das matrizes A e B
 4     for(i = 0; i < 5; i++){
 5         for(j = 0; j < 3; j++){
 6             matA[i][j] = i+j;
 7             matB[i][j] = i*j;
 8         }//end for j
 9     }//end for i
10     //calculo da matriz matC
11     for(i = 0; i < 5; i++){
12         for(j = 0; j < 3; j++){
13             matC[i][j] = matA[i][j] + matB[i][j];
14         }//end for j
15     }//end for i
16 }//end main
```

Resolução do ex. 8.21 — Antes de armazenar o valor da matriz, é verificado se o item atual é um item da diagonal principal. Isso acontece quando **`i==j`**. Neste caso, salva-se o valor **`1`**.

```
 1 void main(void){
 2     int i, j, matI[1000][1000];
 3     //inicializacao da matriz identidade
 4     for(i = 0; i < 1000; i++){
 5         for(j = 0; j < 1000; j++){
 6             if(i == j){
 7                 matI[i][j] = 1;
 8             }else{
 9                 matI[i][j] = 0;
10             }//end else
11         }//end for j
12     }//end for i
13 }//end main
```

Resolução do ex. 8.23 — A variável a é um ponteiro que terá o valor de seu endereço apontado para onde os dados estão. Para salvar o valor cifrado, basta somar o valor **0x200** antes de derreferenciar o ponteiro: por exemplo, **0x200 -> 0x400**, **0x235 -> 0x435**.

```
1 void main (void){
2     unsigned char *a;
3     for(a=0x100; a<0x2ff; a++){
4         *(a+0x200)=*a ^ 0xFE;
5     }
6 }
```

Resolução do ex. 8.24 — Para simplificar o código, pode ser usado um loop para testar cada uma das posições através da função **bitTst**. Para cada tecla pressionada, envia-se o valor para o LCD.

```
 1 void main (void) {
 2     int i;
 3     for(;;){
 4         for(i=0;i<16;i++){
 5             if(bitTst(TECLADO,i){
 6                 lcdNumber(i);
 7             }
 8         }
 9     }
10 }
```

Resolução do ex. 8.25 — Existem diversos modos de realizar o controle. O mais econômico é criar uma linha reta entre o início e o fim. O mais simples é corrigir primeiro a longitude e depois a latitude. A resposta é baseada na segunda abordagem. Enquanto houver erro, o sistema corrige no sentido contrário: se a latitude atual é menor, move o quadcóptero para o norte, se for maior, para o sul.

```
#include "gps.h"
#include "quad.h"

//longitude -> esquerda/direita
//latitude-> norte/sul
Void main (void){
    //inicializa com os valores do alvo
    int longDest=10;
    int latDest=-5;

    InicializaQuad();
    InicializaGPS();

    //executa os comandos em loops para acertar primeiro a longitude e depois a latitude
    for(;;){
        // se o quad estiver para "baixo", deve ir pro norte chegar na mesma linha.
        while(longDest > LerLongitude()){
            VelNorteSul(+1);//anda até chegar na longitude correta
        }
        // se o quad estiver para "cima", tem que ir para o sul
        while(longDest < LerLongitude()){
            VelNorteSul(-1);//anda até chegar na longitude correta
        }
        //Se estiver a esquerda
        while (latDest > LerLatitude()){
            VelLesteOeste(+1);
        }
        //Se estiver a direita
        while (latDest < LerLatitude()){
            VelLesteOeste(-1);
        }
    }
}
```

Resolução do ex. 9.6 — A criação do header deve ser feita apresentando todos os protótipos das funções. As variáveis que devem armazenar valores dentro da biblioteca são definidas com o modificador **`static`**, dentro do arquivo de código e fora de qualquer função, para serem acessadas por todas as funções do código.

```
1 //alarmes.h
2 #ifndef ALARMES_H
3 #define ALARMES_H
4     void ValorTemp(int val);
5     void ValorPres(int val);
6     void InicializaAlarme(void);
7     void AtualizaAlarme(void);
8 #endif
9
10 //alarmes.c
11 #include "alarmes.h"
12 #include "basico.h"
13 static int alarmeTemp;
14 static int alarmePres;
15
16 void ValorTemp(int val){
17     alarmeTemp = val;
18 }
19 void ValorPres(int val){
20     alarmePress = val;
21 }
22 void InicializaAlarme(void){
23     InicializaADC();
24 }
25 void AtualizaAlarme(void){
26     if( LerValorAD(3) > alarmeTemp)
27     {
28         BitSet(PORTE,3);//liga alarme
29     }
30     if( LerValorAD(6) > alarmePress)
31     {
32         BitSet(PORTE,6);//liga alarme
33     }
34 }
```

Resolução do ex. 13.6 — As condições para ligar a prensa têm que acontecer simultaneamente. Se apenas uma delas estiver ligada, deve-se esperar que as duas sejam soltas para testar novamente.

```
#include "io.h"
void main (void){
    TRISE = 0xDF; //apenas bit 5 é saída, 7 4 3 e 2 são entradas
    for(;;){
        //apenas bt 2 ativado, espera os dois serem soltos
        if(bitTst(PORTE,2) && !(bitTst(PORTE,3)){
            while (bitTst(PORTE,2) || (bitTst(PORTE,3));
        }
        //apenas bt 3 ativado, espera os dois serem soltos
        if(!(bitTst(PORTE,2)) && bitTst(PORTE,3){
            while (bitTst(PORTE,2) || (bitTst(PORTE,3));
        }

        //itens de segurança ativados
        if(bitTst(PORTE,4) && bitTst(PORTE,7)){
            //botões pressionados ao mesmo tempo
            if(bitTst(PORTE,2) && bitTst(PORTE,3){
                //ativa prensa
                bitSet(PORTE,5);
                bitClr(PORTE,5);
            }
        }
    }
}
```

Resolução do ex. 14.4 — Considerando entrada **1** e saída **0**, Binário: **0b00010110**, Hexadecimal: **0x16**, Decimal: **22**.

Resolução do ex. 14.6 — Além de inicializar adequadamente o conversor AD, o programa deve realizar a leitura e converter corretamente a escala. Neste caso, fazemos uma conversão de 0-1023 para 0-8. Depois devemos ligar todos os leds e desligar apenas a quantidade correta, pois, quando a entrada é zero, todos os leds estão acesos (zero apagados) e, quando a entrada é 1023, todos os oito estão apagados.

```
1 #include "adc.h"
2
3 void main (void){
4     char i;
5     int dist;
6     adInit();
7     TRISB = 0x00;
8     for(;;){
9         dist = adRead();
10        //transforma a escala de 0 à 1023 para 0 à 8
11        dist = (dist * 8) / 1023;
12        PORTB = 0x00; //acende os 8 leds
13        for(i=0;i< dist; i++){
14            //Apaga uma quantidade de leds relativa a distância
15            BitClr (PORTB, i);
16        }
17    }
18 }
```

Resolução do ex. 15.5 — Como os leds estão ligados em sequência direta com os terminais da porta (gfedbca), basta seguir apenas o fluxo normal ligando os leds da porta C de 0*a*6, já que o ponto decimal não conta.

```
1 void main(void){
2     char led;
3     int  time;
4     TRISE = 0x2E;  //habilitando como saída apenas os terminais dos displays
5     PORTE |= 0x2E; //ligando apenas os terminais dos displays
6     TRISC = 0x00;  //habilitando como saída os terminais dos leds
7
8     for(;;){       //loop infinito
9         //contagem de tempo
10        for(time = 0; time<1000; time++);
11
12        //desligando todos os leds
13        PORTC = 0x00;
14        //liga apenas o led atual
15        BitSet(PORTC, led);
16        led++;
17        if(led >=7){
18            led = 0;
19        }
20    }
21 }
```

Resolução do ex. 16.4 — Como existem 3 linhas e três colunas, é necessário configurar os terminais adequadamente como saída ou entrada. O procedimento é o mesmo adotado na biblioteca **keypad**, com a diferença que o acionamento não é mais feito através do conversor serial-paralelo, mas diretamente nos terminais da porta.

```
1 //variável para armazenar o resultado
2 unsigned int Teclas;
3
4 void VarreduraChaves(void){
5     unsigned char i,j;
6     TRISB = 0x07;               //configura pinos (210) como entrada
7     TRISC = 0x00;               //configura pinos (210) como saída
8     for(i = 0; i < 3; i++){
9         PORTC = 0x00;           //desliga todas as colunas
10        BitSet(PORTC,i);        //liga a coluna correspondente
11        for(j = 0; j < 3; j++){ //teste para cada bit, atualiza a variável
12            if (BitTst(PORTB,j)){
13                BitSet(valorNovo,(i*3)+j);
14            }else{
15                BitClr(valorNovo,(i*3)+j);
16            }
17        }
18    }
19 }
```

Resolução do ex. 16.5 — Podemos supor que os terminais são configurados com entrada em outra rotina de inicialização. A função debounce realiza a comparação com o valor inteiro da porta D, já que todos os 8 bits representam chaves.

```
1
2 char debounce(void){
3     char tecla = 0; //sem ninguem pressionado
4     char novatecla;
5     char tempo = 10;
6     char i;
7
8     //enquanto tempo não esgotar espera estabilizar
9     while(tempo){
10        novatecla = PORTD;
11        //testa se está estável
12        if (novatecla != tecla){
13            tecla = novatecla;
14            tempo = 10;
15        } else {
16            tempo--;
17        }
18    }
19
20    //deste ponto em diante tecla já está estável
21    for(i = 0; i<8;i++){
22        //reliza o teste e retorna a primeira chave que encontrar pressionada
23        if(BitTst(tecla,i){
24            return i+1;
25        }
26    }
27    return 0; //sem tecla pressionada
28 }
```

Resolução do ex. 16.6 — Cada tecla pressionada incrementa o contador de valores. Por fim, um loop é utilizado para ligar a quantidade correta.

```
#include "so.h"
#include "io.h"
void main (void){
  int n_leds, i;
  unsigned char ledsAcesos;

  //inicializa entradas
  pinMode(D0,OUTPUT);
  pinMode(D1,OUTPUT);
  pinMode(D2,OUTPUT);

  //inicializa conversor para poder acionar os leds
  soInit();

  for(;;){
    n_leds = 0;
    //teste led 1
    if (digitalRead(D0)){
      n_leds+=1;
    }
    if (digitalRead(D1)){
      n_leds+=2;
    }
    if (digitalRead(D2)){
      n_leds+=3;
    }
    ledsAcesos = 0x00; //apaga todos os leds
    //acende apenas a quantidade necessária
    for(i=0; i < n_leds; i++){
      BitSet(ledsAcesos,i);
    }
    //envia o valor pelo conversor
    soWrite(ledsAcesos);
  }
}
```

Resolução do ex. 18.2 — Cada vez que a função **`serialRead()`** é chamada, o valor deve ser armazenado numa variável temporária para realizar as comparações. Dependendo do valor a decisão correta é tomada.

```
 1 #include "lcd.h"
 2 #include "ssd.h"
 3 #include "serial.h"
 4
 5 void main (void){
 6     char txt;
 7     int time;
 8     adInit();
 9     lcdInit();
10     ssdInit();
11     for(;;){
12         txt = serialRead();
13         if(txt>= '0' && txt <= '9'){
14             //converter de ASCII para número
15             ssdDigit(txt-48, 0);
16         }
17         if(txt>= 'A' && txt <= 'Z'){
18             lcdChar(txt, 0);
19         }
20         ssdUpdate();
21         for(time=0; time<1000; time++);
22     }
23 }
```

Resolução do ex. 18.4 — O primeiro passo é aguardar a chegada do número 42. Em qualquer outra situação o valor recebido é ignorado. A partir do momento que chegou o 42, qualquer valor recebido, com exceção de **0xff**, é adicionado a uma variável temporária. Quando chegar o número 24, finaliza, a soma e fica-se esperando o próximo valor diferente de $0xff$. Quando ele chegar, compara-se e verifica-se se a mensagem está correta.

```c
#include "serial.h"

//suposições iniciais:
//não é necessário armazenar os valores da mensagem
//não é necessário tomar nenhuma ação quando a mensagem for válida ou inválida
//24 e 42 nunca são enviados como dados ou resultado de soma
//0xFF indica que não foi recebido valor pela serial

void main (void){
    char msg_ok;
    int temp, soma;

    //inicializa os periféricos necessários
    serialInit();

    for(;;){
        msg_ok = 0; //começa supondo mensagem errada
        temp = serialRead();
        //se recebeu começo da mensagem correta pode começar a ler e armazenar a soma
        if (temp == 42){
            soma = 0;
            //enquanto temp for diferente de 24 (fim) continua a soma
            while(temp != 24){
                temp = serialRead();
                //testa se chegou alguma coisa
                if(temp != 0xff){
                    soma += temp;
                }
            }

            //Quando chegar aqui o ultimo valor lido foi o 24.
            //O próximo byte válido é o resultado da soma
            do{
                temp = serialRead();
            }while(temp == 0xff);;

            //se a soma for igual a mensagem é valida
            if (soma == temp){
                msg_ok = 1;
            } else {
                msg_ok = 0;
            }
        }
    }
}
```

Resolução do ex. 18.5 — A biblioteca console precisa receber uma string. Para isso, será criado um vetor de 2 posições onde a última sempre é um terminador **'\0'**. A primeira posição é preenchida com o valor recebido (se ele for diferente de 0xff).

```
1 void main(void) {
2     //criando uma string para enviar para o console
3     char rx[2];
4     systemInit();
5     consoleInit();
6     serialInit();
7     //colocando o terminador na string
8     rx[1] = '\0';
9     for (;;) {
10        //le o caracter da serial
11        rx[0] = serialRead();
12        //se for valido
13        if (rx[0] != 0xff) {
14            //testa se é um dos caracteres de controle de movimento w/s
15            if (rx[0] == 'w') {
16                consoleMoveLine(-1);
17            } else if (rx[0] == 's') {
18                consoleMoveLine(+1);
19            } else {
20                //se for valido mas não de monvimentação, imprime no console
21                consolePrint(rx);
22            }
23            consoleUpdate();
24        }
25    }
26 }
```

Resolução do ex. 18.6 — O primeiro passo é conseguir criar um sistema que contabilize quantas letras certas chegaram. Para isso, é utilizado um contador auxiliar. Para cada letra certa, o contador incrementa e uma nova condição **if** é habilitada. Se, em qualquer momento, chegar uma letra errada, o contador volta para zero.

```
#include "lcd.h"
#include "serial.h"

void main(void) {
    int qt_letras = 0; //conta quantas letras corretas já chegaram
    char letra = 0;
    serialInit();
    lcdInit();
    for(;;){ //loop infinito
        //aguarda chegar uma letra
        do{
            letra = serialRead();
        }while(letra == 0);
        //testa se chegou 'n'
        if (letra == 'I'){
            qt_letra=1; //chegou uma letra certa
        }
        //testa se a letra atual é o 'D' e se já chegou 1 certa antes
        if ((letra == 'D')&& (qt_letra == 1){
            qt_letra=2; //já chegaram duas letras certas
        } else {
            qt_letras = 0; //chegou uma errada, tem que reiniciar
        }
        //testa se a letra é 'E' e se já chegou 2 certas antes
        if ((letra == 'E')&& (qt_letra == 2){
            qt_letra=3; //já chegaram três letras certa
        } else {
            qt_letras = 0; //chegou uma errada tem que reiniciar
        }

        //se chegou a palavra "IDE" via serial envia a identificação via serial
        if(qt_letras == 3){
            lcdString("11952");
            qt_letras = 0;
        }
    }
}
```

Resolução do ex. 19.4 — Para armazenar os dados ao longo do tempo, foi criado um vetor de 5 posições. Eles iniciam com valor zero. O primeiro passo do loop é deslocar cada valor uma casa para trás. O último valor será perdido e a primeira posição ficará disponível para receber a nova medida. O valor filtrado é exibido no LCD.

```
#include "io.h"
#include "ad.h"
#include "lcd.h"

void main(void){
    int adValue[5]; //armazena as cinco amostras
    int i;
    systemInit();
    lcdInit();
    adInit();
    for(i=0;i<5;i++){
        adValue[i] = 0; //inicializa todas as medidas com zero
    }
    for(;;){
        for(i=5;i>=1;i--){
            adValue[i] = adValue[i-1]; //empurra todos os valores pra trás
        }
        //Leitura do potenciômetro
        adValue[0] = analogRead(AN0);

        //muda pra sempre imprimir na mesma posição
        lcdPosition(0,0);
        lcdNumber((adValue[0]+adValue[1]+adValue[2]+adValue[3]+adValue[4])/5);
    }
}
```

Resolução do ex. 20.4 — Para transmitir os dados adequadamente, o PWM tem que ser ligado durante 1 ou 2ms com a frequência desejada. Para cada bit será realizado um loop dentro do segundo **`for`**. Em todos os casos, a saída ficará desligada 1ms e ligada 1ms. Se o bit valer 1 ela fica ligada mais 1ms, totalizando os 2ms exigidos.

```
#include "io.h"
#include "pwm.h"

void main(void){
    unsigned char data; //valor a ser transmitido
    int i;
    systemInit();
    pwmInit();
    pwmFrequency(38000);
    for(;;){
        for(i=0;i<8;i++){
            //desliga o PWM
            pwmDuty(0);
            //gera o atraso de 1ms. Tem que ser ajustado de acordo com a plataforma
            for(i=0;i<1000;i++);

            //liga com duty cycle de 50%
            pwmDuty(50);
            //gera o atraso de 1ms. Tem que ser ajustado de acordo com a plataforma
            for(i=0;i<1000;i++);

            if(bitTst(data,i)){
                //se for um bit 1 gasta mais 1ms, totalizando 2
                for(i=0;i<1000;i++);
            }
        }
    }
}
```

Resolução do ex. 22.1 — As interrupções são chamadas através de eventos. Esses eventos são geralmente iniciados por dispositivos de hardware. Deste modo, para definir quando a interrupção é chamada, devemos entender qual é o evento que dispara essa chamada. Para uma interrupção de timer, por exemplo, a função será chamada de modo periódico. Para uma interrupção que monitore uma entrada externa, ela será chamada quando houver mudança no valor do terminal.

Resolução do ex. 22.3 — Para a arquitetura apresentada no exercicio, basta criar a função com o nome **`SerialISR()`** que automaticamente ela responderá às interrupções. Para receber o valor do dado, basta fazer a leitura no registro RX_DATA. O valor é armazenado numa variável temporária para a comparação.

```
1 //variavel global
2 static unsigned char valor_serial;
3 void SerialISR (void){
4     unsigned char temp;
5     temp = RX_DATA;
6     if (temp >= '0'  && temp <= '9'){
7         valor_serial = temp;
8     } else {
9         valor_serial = 0xFF;
10    }
11 }
```

Resolução do ex. 22.4 — Como a variável tick é global e efetua a contagem de tempo corretamente em millissegundos, basta realizar as conversões para segundos, minutos e horas e imprimir corretamente no lcd.

```
 1 #include "io.h"
 2
 3 unsigned long int tick;
 4 //início do programa
 5 void main(void){
 6     unsigned int seg;
 7     unsigned int min;
 8     unsigned int hora;
 9     systemInit();
10     lcdInit();
11     for(;;) {
12         //separando os segundos
13         seg = (tick/1000)%60;
14
15         //separando os minutos
16         min = (tick/1000/60)%60;
17
18         //separando as horas
19         hora = (tick/1000/60/60)%24;
20         //imprime as informações
21         lcdPosition(0,1);
22         lcdChar(hora/10+'0');
23         lcdChar(hora%10+'0');
24         lcdChar(':');
25         lcdChar(min/10+'0');
26         lcdChar(min%10+'0');
27         lcdChar(':');
28         lcdChar(seg/10+'0');
29         lcdChar(seg%10+'0');
30     }
31 }
```

Resolução do ex. 24.4 — Num sistema operacional de tempo real, existem mecanismos que permitem ao programador garantir que a taxa de execução dos processos, ou de pelo menos parte dos processos, será cumprida.

Resolução do ex. 25.1 —

```
1 int qtd[POOLSIZE];
2 void main (void){
3     //zera as quantidades iniciais
4     for(i=0;i<POOLSIZE;i++){
5         qtd[i] = 0;
6     }
7
8     for(;;){
9         for(i=0;i<6;i++){
10            //chama a função e passa a quantidade de
11            //vezes que ela já foi executada como parâmetro
12            pool[i](qtd[i]);
13            qtd[i]++;
14        }
15    }
16 }
```

Resolução do ex. 25.2 —

```
1 char procLed(int n){
2     BitSet(PORTA, n);
3     if (BitTst(PORTA,n){
4         return SUCESS;
5     }else{
6         return FAIL;
7     }
8 }
```

Resolução do ex. 25.3 —

```
1 void main(void){
2     KernelInit();
3     KernelAddProc(LeSerial);
4     KernelAddProc(ProcessaBuffer);
5     KernelLoop();
6 }
```

Resolução do ex. 26.4 — O código é responsável por decrementar um temporizador interno de reexecução de cada um dos processos.

```
1 void KernelTick(void){
2     unsigned char i;
3     i = ini;
4     while(i!=fim){
5         //Apenas quando exige RT é necessário fazer contagem
6         if((pool[i].start)>(MIN_INT) && (ExigeRT))
7         pool[i].start--;
8         i = (i+1)%SLOT_SIZE;
9     }
10 }
```

Resolução do ex. 27.3 — Como as funções básicas do driver devolvem informação, elas devem retornar o valor através do parâmetro, bastando realizar o teste nos bits adequados do teclado. Para a inicialização, deve-se configurar os terminais como entrada e ajustar as funções internas do driver. O restante do driver apenas segue o padrão definido.

```
1 //teclado.c
2
3 void LerTecla1(void *parameters){
4     *(* char)(parameters) = BitTst(PORTB,2);
5 }
6
7 void LerTecla2(void *parameters){
8     *(* char)(parameters) = BitTst(PORTA,4);
9 }
10
11 void InicializaTeclado(void *parameters){
12     BitSet(DDRA,4); //Set = Output
13     BitSet(DDRB,2);
14     thisDriver.drv_id = *(char*) parameters;
15     this_functions[TECLADO_TECLA1] = LerTecla1;
16     this_functions[TECLADO_TECLA2] = LerTecla2;
17 }
18
19 driver* getTecladoDriver(void){
20     //to ensure that at least the init function will be known
21     thisDriver.drv_init = InicializaTeclado;
22     thisDriver.func_ptr = (ptrFuncDrv*) & this_functions;
23     return &thisDriver;
24 }
25
26 //teclado.h
27
28 #ifndef TECLADO_H
29 #define TECLADO_H
30 #include "ddCtr_types.h"
31
32 enum{   TECLADO_TECLA1, TECLADO_TECLA2, TECLADO_END};
33
34 //apenas retorna o "objeto" do driver
35 driver* getTecladoDriver(void);
36 #endif
```

Resolução do ex. 27.5 — Como as funções básicas do driver estão implementadas na biblioteca **adc.h**, basta realizar a adequação de cada uma das funções da biblioteca para o padrão do driver. A função que devolve informação **LeValorAD()**, deve retornar o valor através do parâmetro. Na inicialização do driver, é necessário apenas realizar a chamada da função **InicializaAd()** da biblioteca e ajustar as funções internas do driver. O restante do driver apenas segue o padrão definido.

```
1  //drv_adc.h
2  #ifndef DRVADC_H
3  #define DRVADC_H
4
5  enum {ADC_LE_VALOR, ADC_END} func;
6  ptrfunc thisFunctions[ADC_END];
7
8  #endif
9
10 //drv_adc.c
11 char init(void* parameters){
12     drv_id = (char) parameters;
13     InicializaAd();
14     return SUCESS;
15 }
16
17 char LeValor(void* parameters){
18     int v;
19     v = LeValorAD();
20     *(int*)parameter = v;
21 }
22
23 driver* getDriver(void){
24     thisDriver.Functions = thisFunctions;
25     thisDriver.init = init;
26     thisFunction[ADC_LE_VALOR] = LeValor;
27     return thisDriver;
28 }
```

Índice Remissivo

Symbols
#define, 37, 100
#include, 37, 132
#pragma, 41, 350
74HC595, 212

A
Acesso aos terminais digitais, 196
Acionamentos, 202
Arduino, 7, 188
ARM, 9
ASCII, 24
ATmega, 7

B
Barramento, 169, 214
Bibliotecas, 128
Bibliotecas padrão do C, 128
Binário, 16, 49
bit, 96
Bit fields, 75
Botões, 234
Bouncing, 235
Buffer circular, 388
Buzzer, 331, 437

C
CAN, 287
case, 106
char, 48
Chipkit, 8, 188
Clock, 177, 212
Codificação BCD, 21
Codificação Gray, 22
Compilação, 31
Comunicação serial, 271
const, 58
Contagem de tempo, 339
Conversor AD, 307, 312, 315

D

Debounce, 236, 237
Decimal, 15, 49
delay, 115
Deslocamento de bits, 93
Detecção de eventos, 244
Display 7 segmentos, 219, 435
do, 112
double, 48
Driver, 136, 416
DS1307, 437

E
Encoder, 247
Entradas digitais, 233
enum, 77
extern, 60

F
Filtro passa-baixas, 324
float, 48
for, 113
Freedom, 9, 188
Função, 124

G
GPS, 301

H
Header, 132
Hexadecimal, 17, 49

I
I²C, 272
if, 104
Indentação, 34
int, 48
Interrupção, 348, 366, 425
ISR, 349

K
Kernel, 376, 406, 408

L
LCD, 251, 262, 436
LDR, 310, 311, 437
Led, 202
Led RGB, 203, 210, 435
Linguagem C, 29

LM35, 310, 311, 437
long, 56
Loop infinito, 44
loop(), 44

M
Máquina de estados, 370
main(), 43, 44
main, função, 144
Matriz, 71
Memória, 173
Microcontrolador, 166, 178
Multiplexação, 314
Multiplexação de saídas, 222
Multitask cooperativo, 369

N
NMEA, 301
Nome de função, 125

O
One-single-loop, 364
Operação E, 85, 90
Operação NÃO, 84, 89
Operação OU, 86, 91
Operação OU EXCLUSIVO, 83, 87, 92

P
Periféricos, 174
PIC32, 8
Piezoelétrico, 331
Ponte H, 207
Ponteiro de função, 391
Ponteiros, 60
Ponteiros para void, 390
Pooling, 347
Potenciômetro, 309, 311, 437
Processador, 168
Processo, 381
Protocolos de comunicação, 299
Pull-down, 234
Pull-up, 234
Pulso, 212
PWM, 321

R
Reed switch, 246
Referência Circular, 129

register, 59
Registros de configuração, 188
Relé, 205
Relé de estado sólido, 206
Relógio, 229, 341
Reprodução de melodias, 343
Reprodução de sons, 331
return, 125
Rotação de bits, 95
RS232, 290

S
Saída digital, 201
Sensores, 308
Serialização, 212
Servomotor, 330
setup(), 44
Shift circular, 95
short, 56
signed, 56
Sistema embarcado, 5
Sistema operacional, 379
Soft I²C, 275
Soft PWM, 326
SPI, 284
static, 59
String, 52, 70
struct, 74
switch, 106

T
Tarefa, 381
Teclado matricial, 241, 436
Tempo real, 402
Temporizadores, 335
Terminal entrada/saída digital, 187, 195
Transistor, 203
typedef, 77

U
UART, 291
unsigned, 56
USB, 295

V
Variável, 47, 51
Vetores, 67
volatile, 57

W
watchdog, 356
while, 112